

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI

Khoa Công nghệ Thông tin

Bộ môn Khoa học máy tính

TRẦN VĂN DŨNG

BÀI GIẢNG

NGUYÊN LÝ CÁC NGÔN NGỮ LẬP TRÌNH

SỐ TÍN CHỈ: 3

Hà nội - 2015

CHƯƠNG 1: TỔNG QUAN VỀ NGÔN NGỮ LẬP TRÌNH

1. Giới thiệu
2. Mục đích
3. Chương trình dịch và cú pháp
4. Lambda Calculus
5. Ngôn ngữ lập trình hàm và mệnh lệnh

1.1 Ngôn ngữ lập trình

Các ngôn ngữ lập trình là phương tiện thể hiện trong nghệ thuật lập trình máy tính. Một ngôn ngữ lập trình lý tưởng sẽ làm cho các lập trình viên dễ dàng viết chương trình chính xác và rõ ràng. Vì vậy các chương trình là môi trường trung gian để được hiểu, được sửa, và được bảo trì qua thời gian sống của nó, một ngôn ngữ lập trình tốt sẽ giúp những người khác đọc chương trình và hiểu nó làm việc như thế nào. Thiết kế và xây dựng phần mềm là các bài toán phức tạp. Nhiều hệ thống phần mềm bao gồm các phần tương tác. Các phần này, hoặc các thành phần phần mềm có thể tương tác theo các cách phức tạp. Để quản trị sự phức tạp, giao diện và truyền thông giữa các thành phần được thiết kế cẩn thận. Một ngôn ngữ tốt cho lập trình mức độ cao sẽ hỗ trợ các lập trình viên quản lý tương tác giữa các thành phần phần mềm một cách hiệu quả. Trong đánh giá các ngôn ngữ lập trình, chúng ta cần xem xét các bài toán thiết kế, cài đặt, kiểm thử và bảo trì phần mềm, xem xét ngôn ngữ có hỗ trợ tốt từng phần của vòng đời phần mềm không.

Có nhiều cân đối khó khăn trong thiết kế ngôn ngữ lập trình. Một số đặc trưng ngôn ngữ làm cho dễ dàng viết chương trình một cách nhanh chóng, nhưng có thể làm cho chúng ta khó thiết kế các công cụ và phương pháp kiểm thử. Một số cấu trúc ngôn ngữ làm cho nó dễ dịch để tối ưu chương trình, nhưng có thể làm cho chương trình rắc rối. Vì các môi trường và ứng dụng tính toán khác nhau, đòi hỏi các đặc tính chương trình khác nhau, những người thiết kế các ngôn ngữ lập trình khác nhau buộc phải lựa chọn các cân đối khác nhau. Trên thực tế, các ngôn ngữ lập trình thành công thực sự được thiết kế ban đầu cho mục đích sử dụng chuyên biệt. Điều này không nói lên rằng, mỗi một ngôn ngữ là tốt chỉ cho một mục đích. Tuy nhiên, tập trung vào một ứng dụng duy nhất sẽ giúp những người thiết kế ngôn ngữ đưa ra các quyết định đúng đắn và có mục tiêu. Một ứng dụng đơn cũng giúp một trong những phần khó nhất của thiết kế ngôn ngữ: bỏ các ý tưởng tốt ra ngoài.

Hàng trăm ngôn ngữ lập trình đã được thiết kế và cài đặt từ đầu những năm 1950. Hầu hết chúng đều chứa các khái niệm mới, các cải tiến hữu ích và sáng tạo đáng kể. Vì quá nhiều ngôn ngữ không thể bao quát hết, nên trong giáo trình này chúng ta tập trung vào 5 ngôn ngữ sau: C, C++, ML, Java và Prolog. Các ngôn ngữ này chứa hầu hết các đặc tính quan trọng của các ngôn ngữ lập trình bậc cao.

1.2 Mục đích

Ở đây chúng ta sẽ xét các khái niệm cơ bản mà xuất hiện trong các ngôn ngữ lập trình hiện đại, sự tương tác của chúng, và quan hệ giữa các ngôn ngữ lập trình và các phương pháp phát triển chương trình. Một chủ đề lặp lại là sự cân bằng giữa khả năng biểu diễn và sự đơn giản trong cài đặt của một ngôn ngữ lập trình. Đối với mỗi đặc tính của ngôn ngữ lập

trình, chúng ta xem xét, đánh giá các cách mà có thể được sử dụng trong lập trình và các kiểu kỹ thuật cài đặt mà được sử dụng để dịch và thực thi nó một cách có hiệu quả.

1.2.1 Các mục tiêu chung

Ở đây chúng ta có các mục tiêu chung sau:

- Để hiểu *không gian thiết kế* của các ngôn ngữ lập trình. Điều này bao gồm các khái niệm và các cấu trúc từ các ngôn ngữ lập trình trước kia, cũng như những ngôn ngữ mà còn có thể được sử dụng rộng rãi hơn trong tương lai. Chúng ta cũng thử tìm hiểu một số xung đột chính và các cân bằng giữa các tính chất ngôn ngữ, bao gồm cả các chi phí cài đặt.
- Để phát triển sự hiểu biết tốt hơn về các ngôn ngữ mà chúng ta sử dụng hiện nay bằng việc so sánh chúng với các ngôn ngữ khác.
- Để hiểu các kỹ thuật lập trình gắn kết với các tính chất ngôn ngữ đa dạng. Nghiên cứu các ngôn ngữ lập trình là trong đó bao gồm nghiên cứu các khung khái niệm để giải quyết bài toán, xây dựng và phát triển phần mềm.

Rất nhiều ý tưởng ở đây là kiến thức chung giữa các nhà lập trình chuyên nghiệp. Tài liệu và các cách suy nghĩ được thể hiện trong giáo trình này sẽ là có ích trong lập trình tương lai. Bạn sẽ có thể đánh giá các đặc trưng của ngôn ngữ, các chi phí của chúng và làm sao chúng gắn kết với nhau.

1.2.2 Các chủ đề chuyên biệt

Ở đây có một số chủ đề chuyên biệt mà được đề cập thường xuyên:

- *Tính toán được - Computability*: Một số bài toán không thể giải quyết được bằng máy tính. Tính không giải được của bài toán dừng (cho một chương trình và một đầu vào bất kỳ, hỏi chương trình có dừng không) suy ra trình biên dịch và thông dịch của ngôn ngữ lập trình không thể làm mọi thứ mà chúng ta muốn chúng thực hiện.
- *Phân tích tĩnh - Static analysis*: Có sự khác biệt giữa thời gian dịch và thời gian chạy. Tại thời điểm dịch, chương trình đã được biết, nhưng đầu vào thì chưa. Tại thời điểm chạy, chương trình và đầu vào cả hai đã sẵn sàng cho hệ thống thời gian chạy. Mặc dù người thiết kế chương trình hoặc người cài đặt muốn tìm ra lỗi trong thời gian dịch, nhưng nhiều lỗi không hiện ra cho đến khi chạy. Các phương pháp mà phát hiện lỗi chương trình trong thời gian dịch thường là bảo thủ, mà có nghĩa là khi họ nói chương trình không có bất cứ kiểu lỗi nào, thì khẳng định đó là đúng. Tuy nhiên, các phương pháp phát hiện lỗi chương trình thời gian dịch thường sẽ nói rằng một số chương trình có lỗi, ngay cả khi các lỗi đó có thể thực tế không xảy ra khi chương trình chạy.
- *Tính thể hiện đối nghịch với tính hiệu quả*: Có nhiều tình huống mà ở đó là thuận tiện có một cài đặt của ngôn ngữ lập trình làm mọi thứ tự động. Một ví dụ như trong môi trường thực thi của Java, chương trình quản lý bộ nhớ tự động cấp và thu dọn các đối tượng dữ liệu được dùng trong chương trình. Khi một điều gì đó được làm tự động, điều đó phải trả giá. Mặc dù một phương pháp tự động có thể tiết kiệm cho lập trình viên phải suy nghĩ về về gì đó, cài đặt ngôn ngữ có thể chạy chậm hơn. Trong một số trường hợp, phương pháp tự động có thể làm cho nó dễ viết chương trình hơn và làm cho lập trình ít có lỗi hơn. Trong một số trường hợp khác, kết quả làm chậm việc thực hiện chương trình có thể làm cho phương pháp tự động là không khả thi.

1.3 Chương trình dịch và cú pháp

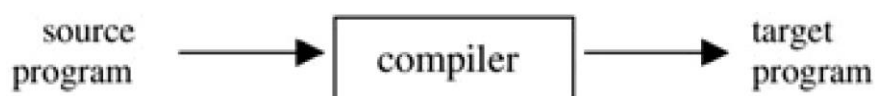
Trong chương này, kiến thức nền tảng về cài đặt ngôn ngữ lập trình sẽ được trình bày ngắn gọn qua cú pháp, phân tích và một số bước được sử dụng trong trình biên dịch truyền thống. Chúng ta cũng xem xét một bộ khung cơ sở mà được sử dụng trong phân tích và thiết kế ngôn ngữ lập trình: tính toán lambda. Tính toán lambda là một bộ khung tốt để định nghĩa các khái niệm cú pháp chung cho nhiều ngôn ngữ lập trình và để nghiên cứu việc tính toán tượng trưng. Còn ngữ nghĩa ký hiệu chỉ ra rằng, về nguyên tắc, các chương trình có thể được rút gọn về các hàm số. Một số khung lý thuyết khác cũng là hữu ích trong thiết kế và phân tích ngôn ngữ lập trình. Những lý thuyết đó trải rộng từ lý thuyết tính toán, mà làm sáng tỏ sức mạnh và giới hạn của chương trình, đến lý thuyết kiểu, mà bao gồm các khía cạnh của cú pháp và ngữ nghĩa của các ngôn ngữ lập trình. Sau nhiều năm nghiên cứu lý thuyết, lý thuyết ngôn ngữ lập trình hiện tại vẫn chưa trả lời được một số câu hỏi nền tảng quan trọng. Chẳng hạn, chúng ta không có một lý thuyết toán học tốt mà bao gồm cả các hàm bậc cao, biến đổi trạng thái và tính tương tranh. Tuy nhiên, các khung lý thuyết đó vẫn có ảnh hưởng đến thiết kế ngôn ngữ lập trình và có thể được sử dụng để xác định vấn đề như trong các ngôn ngữ lập trình.

Một chương trình là một mô tả của một tiến trình động. Văn bản của một chương trình được gọi là cú pháp của nó; các việc mà chương trình làm được gọi là ngữ nghĩa của nó. Hàm cài đặt của một ngôn ngữ lập trình chuyển đổi cú pháp của một chương trình sang các lệnh của máy mà có thể thực thi để buộc dãy các hành động xảy ra, được gọi là chương trình dịch.

1.3.1 Cấu trúc của một chương trình dịch đơn giản

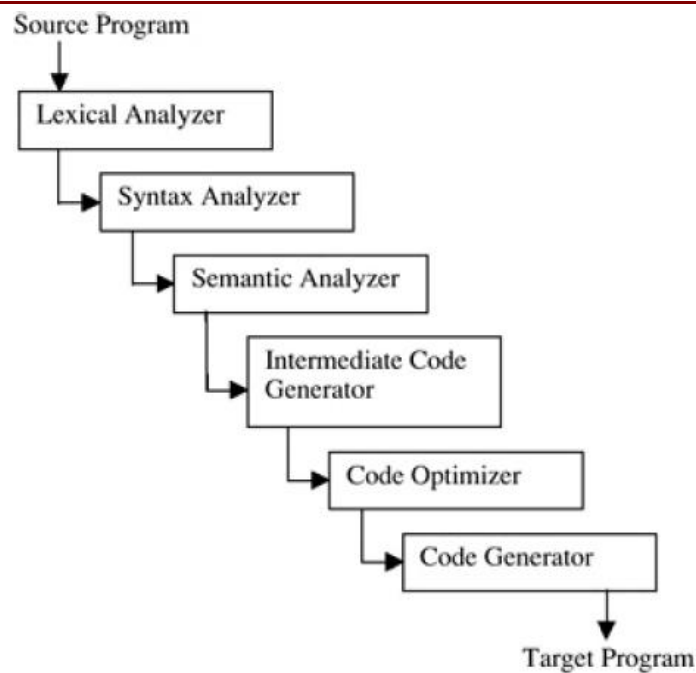
Các ngôn ngữ lập trình mà thuận tiện cho mọi người sử dụng, được xây dựng xung quanh các khái niệm và trừu tượng, mà có thể không tương ứng trực tiếp đến các đặc trưng của máy tính bên dưới. Vì vậy, một chương trình cần được chuyển đổi thành một tập các chỉ lệnh cơ bản của máy trước khi nó có thể thực thi. Điều này có thể được thực hiện bởi một *trình biên dịch*, mà chuyển đổi toàn bộ một chương trình sang code của máy trước khi chương trình chạy, hoặc một *trình thông dịch*, mà kết hợp giữa chuyển và thực thi chương trình. Chúng ta sẽ trình bày việc triển khai ngôn ngữ lập trình bằng chương trình dịch, mà làm cho nó dễ dàng được tách thành một số bài toán chính.

Chức năng chính của một trình biên dịch được thể hiện trong lược đồ đơn giản sau:



Cho trước một chương trình trong một ngôn ngữ nguồn nào đó, trình biên dịch sẽ tạo ra một chương trình trong ngôn ngữ đích, mà thông thường là một tập chỉ lệnh, hoặc ngôn ngữ máy của một máy nào đó.

Hầu hết các trình biên dịch được cấu tạo như một dãy các pha, với mỗi pha thực hiện một bước trong việc chuyển đổi từ chương trình nguồn sang chương trình đích. Trình biên dịch thông thường có thể bao gồm các pha trong lược đồ sau:



Mỗi pha đó được mô tả ngắn gọn. Mục đích của chúng ta là hiểu các phần của trình biên dịch để làm sao các đặc trưng khác nhau của ngôn ngữ lập trình có thể được thể hiện. Chúng ta sẽ không trao đổi làm sao xây dựng được trình biên dịch.

Phân tích từ vựng

Các ký hiệu đầu vào được quét và nhóm vào các đơn vị có ý nghĩa được gọi là các từ tố. Chẳng hạn, phân tích từ vựng của biểu thức $temp := x + 1$, mà sử dụng lệnh gán $:=$, có thể chia dãy các ký hiệu trên thành năm từ tố: định danh $temp$, ký hiệu gán $:=$, một biến x , một ký hiệu phép cộng $+$ và số 1 . Phân tích từ vựng có thể phân biệt số với định danh. Tuy nhiên, vì phân tích từ vựng được dựa trên việc quét trái qua phải (và trên xuống dưới), phân tích từ vựng không phân biệt các định danh: cái nào là tên biến và cái nào là tên hằng số. Vì các biến và các hằng số được khai báo khác nhau, nên các biến và các hằng số sẽ được phân biệt trong pha phân tích ngữ nghĩa.

Phân tích cú pháp

Trong pha này các từ tố được nhóm lại thành các đơn vị cú pháp như các biểu thức, các lệnh, và các khai báo mà cần tuân theo các qui tắc ngữ pháp trong ngôn ngữ lập trình. Hành động được thực hiện trong pha này được gọi là phân tích parsing. Mục đích của phân tích là tạo ra một cấu trúc dữ liệu được gọi là cây cú pháp, mà biểu diễn cấu trúc cú pháp của chương trình theo cách thuận tiện cho các pha tiếp sau của trình biên dịch. Nếu một chương trình không thỏa mãn các điều kiện để là một chương trình dạng tốt, thì pha phân tích sẽ đưa ra thông báo lỗi và dừng trình biên dịch.

Phân tích ngữ nghĩa

Trong pha này của trình biên dịch, các qui tắc và các thủ tục mà phụ thuộc vào ngữ cảnh bao quanh biểu thức được áp dụng. Chẳng hạn, trở lại ví dụ $temp := x + 1$, chúng ta nhận thấy cần phải tin tưởng là các kiểu đều phù hợp. Nếu lệnh gán xảy ra trong một ngôn ngữ, mà ở đó các số nguyên được tự động chuyển kiểu thành số thực khi cần thiết, thì có một số

cách mà các kiểu cần được gắn kết với các phần của biểu thức đó. Trong phân tích ngữ nghĩa chuẩn, kiểu của *temp* và *x* cần phải được xác định từ khai báo của chúng. Nếu cả hai đều là số nguyên, thì số 1 cần được đánh dấu như một số nguyên và $+$ là phép cộng số nguyên, và biểu thức được xem là đúng. Nếu một trong hai định danh, chẳng hạn *x* là số thực, thì số 1 cần được coi như số thực và $+$ là phép cộng số thực. Tùy thuộc vào *temp* là số nguyên hay số thực, mà nó cần thiết chèn vào một phép chuyển kiểu của biểu thức $x + 1$. Đầu ra của pha này là cây phân tích được gia tăng thông tin mà biểu diễn một cấu trúc cú pháp của chương trình và bao gồm một số thông tin bổ sung như kiểu của các định danh và vị trí trong chương trình, ở đó mỗi định danh được khai báo.

Mặc dù pha tiếp sau pha phân tích được gọi là *phân tích ngữ nghĩa*, việc sử dụng từ ngữ nghĩa *semantics* là khác với sử dụng từ chuẩn ý nghĩa *meaning*. Một số người viết trình biên dịch sử dụng từ ngữ nghĩa, vì pha này dựa trên thông tin ngữ cảnh, và kiểu văn phạm sử dụng để phân tích cú pháp không bao hàm được thông tin ngữ cảnh. Tuy nhiên, từ ngữ nghĩa thường được dùng để tham chiếu xem chương trình được thực hiện như thế nào, không phải nói về các tính chất cú pháp quan trọng mà nảy sinh ở pha thứ ba của trình biên dịch.

Sinh code trung gian

Mặc dù có thể sinh ra chương trình đích từ các kết quả của phân tích cú pháp và ngữ nghĩa, nhưng khó sinh code hiệu quả trong một pha. Do đó, nhiều trình biên dịch trước hết tạo ra một dạng code trung gian và sau đó tối ưu code này để tạo ra chương trình đích hiệu quả hơn.

Vì pha cuối cùng này của trình biên dịch có thể biến đổi một tập chỉ lệnh sang một tập khác, nên code trung gian không cần được viết với tập lệnh thực tế của một máy đích. Quan trọng là sử dụng biểu diễn trung gian mà dễ dàng tạo và dễ dàng chuyển sang ngôn ngữ đích. Biểu diễn trung gian có thể là một dạng code mức độ tổng quát thấp mà có các tính chất chung với một số máy tính. Khi một biểu diễn trung gian được sử dụng, có thể sử dụng cùng trình biên dịch đó để sinh ra chương trình đích cho một số máy tính khác nhau.

Tối ưu code

Có nhiều kỹ thuật mà có thể được sử dụng để nâng cao hiệu quả của chương trình. Các kỹ thuật này thường được áp dụng cho biểu diễn trung gian. Nếu một số kỹ thuật tối ưu được viết để chuyển đổi biểu diễn trung gian, thì các kỹ thuật này cũng được áp dụng tiếp cho đến khi điều kiện dừng đạt được.

Danh sách sau đây mô tả một số tối ưu chuẩn:

- Loại trừ biểu thức con chung: nếu một chương trình tính cùng một giá trị nhiều hơn một lần và trình biên dịch phát hiện điều đó, sau đó có thể biến đổi chương trình sao cho giá trị được tính toán chỉ một lần và lưu trữ cho lần sử dụng tiếp theo.
- Truyền bá sao chép: nếu một chương trình chứa một phép gán như $x := y$, thì có thể thay đổi lệnh tiếp theo đó tham chiếu đến *y* bằng tham chiếu đến *x* và loại bỏ phép gán đó.
- Loại bỏ code chết: nếu một dãy chỉ lệnh nào đó mà không bao giờ cần đến, thì nó có thể được loại bỏ khỏi chương trình.
- Tối ưu vòng lặp: có một số kỹ thuật mà có thể được áp dụng để loại bỏ các chỉ lệnh khỏi vòng lặp. Chẳng hạn, nếu một biểu thức xuất hiện bên trong vòng lặp, nhưng có giá trị như nhau qua mỗi vòng lặp, thì biểu thức đó có thể được loại đưa ra ngoài vòng lặp.

- Lời gọi hàm trên dòng: nếu một chương trình gọi một hàm t , có thể thay code cho t tại vị trí t được gọi. Điều này làm cho chương trình đích hiệu quả hơn, như các chỉ lệnh gắn kết với lời gọi một hàm có thể được loại bỏ, nhưng nó làm tăng kích thước chương trình. Hệ quả quan trọng nhất của lời gọi hàm trên dòng thông thường là chúng cho phép một số tối ưu khác được thực hiện bởi việc loại bỏ các bước nhảy trong code.

Ví dụ 1.1:

```
int sumcalc(int a, int b, int N)
{
    int i;
    int x, y;
    x = 0;
    y = 0;
    for (i = 0; i <= N, i++) {
        x = x + (4*a/b)*i + (i+1)*(i+1);
        x = x + b*y;
    }
    return x;
}
```

Truyền bá hằng số: phép gán $y = 0$ được truyền vào thay y bằng 0 trong biểu thức sử dụng y :

```
int sumcalc(int a, int b, int N)
{
    int i;
    int x, y;
    x = 0;
    y = 0;
    for (i = 0; i <= N, i++) {
        x = x + (4*a/b)*i + (i+1)*(i+1);
        x = x + b*0;
    }
    return x;
}
```

Rút gọn:

```
int sumcalc(int a, int b, int N)
{
    int i;
    int x;
    x = 0;
    for (i = 0; i <= N, i++) {
        x = x + (4*a/b)*i + (i+1)*(i+1);
    }
    return x;
}
```

Loại biểu thức con chung: Biểu thức $(i+1)$ được lặp trong tính toán x , nên được tính một lần qua biến t .

```

int sumcalc(int a, int b, int N)
{
    int i;
    int x, t;
    x = 0;
    for (i = 0; i <= N, i++) {
        t = i+1;
        x = x + (4*a/b)*i + t*t;
    }
    return x;
}

```

Loại bất biến vòng lặp: biểu thức $(4*a/b)$ không thay đổi trong vòng lặp, đặt $u = (4*a/b)$ và đưa u ra ngoài vòng lặp:

```

int sumcalc(int a, int b, int N)
{
    int i;
    int x, t, u;
    x = 0;
    u = 4*a/b;
    for (i = 0; i <= N, i++) {
        t = i+1;
        x = x + u*i + t*t;
    }
    return x;
}

```

Thay phép nhân bằng phép cộng: $v = u*i$

$i = 0, v = u*0 = 0$ thay bằng $v = 0;$
 $i = 1, v = u*1 = u$ thay bằng $v = v + u;$
 $i = 2, v = u*2 = 2u$ thay bằng $v = v + u;$

Tóm lại thay $v = u*i$ bằng $v = 0$ ở trước vòng lặp và $v = v + u$ trong vòng lặp

```

int sumcalc(int a, int b, int N)
{
    int i;
    int x, t, u, v;
    x = 0;
    u = 4*a/b;
    v = 0;
    for (i = 0; i <= N, i++) {
        t = i+1;
        x = x + v + t*t;
        v = v + u;
    }
    return x;
}

```

Thay phép $4*a$ bằng phép tịnh tiến biểu diễn bit của a sang trái 2 bit:

```
int sumcalc(int a, int b, int N)
{
    int i;
    int x, t, u, v;
    x = 0;
    u = (a << 2) / b;
    v = 0;
    for (i = 0; i <= N, i++) {
        t = i + 1;
        x = x + v + t * t;
        v = v + u;
    }
    return x;
}
```

Cuối cùng ta được một chương trình tối ưu, lợi ích đem lại là chương trình máy đích này sẽ chạy nhanh hơn đáng kể. Theo thử nghiệm là có thể nhanh hơn 2 đến 3 lần so với chưa tối ưu.

Phát sinh code

Pha cuối cùng của trình biên dịch là chuyển code trung gian sang code máy đích. Nó bao gồm chọn vị trí bộ nhớ, thanh ghi, hoặc cả hai, cho mỗi biến xuất hiện trong chương trình. Ở đây có nhiều thuật toán bố trí thanh ghi mà tái sử dụng thanh ghi một cách hiệu quả. Nó quan trọng vì nhiều máy có số thanh ghi cố định, và các thao tác trên đó hiệu quả hơn là truyền dữ liệu vào và ra bộ nhớ.

1.3.2 Văn phạm và cây cú pháp

Chúng ta sử dụng văn phạm để mô tả nhiều ngôn ngữ khác nhau. Chúng ta sẽ xem xét qua việc tạo cây cú pháp từ dãy các từ tố.

Văn phạm

Các văn phạm cung cấp một phương pháp thuận tiện định nghĩa tập vô hạn các biểu thức. Ngoài ra, cấu trúc tạo bởi văn phạm cho chúng ta cách xử lý có hệ thống các biểu thức. Văn phạm bao gồm một ký hiệu bắt đầu *Start*, một tập các ký hiệu chưa tận, một tập các ký hiệu tận cùng và một tập các luật sinh. Các chưa tận là các ký hiệu được sử dụng để viết ra văn phạm, và các tận cùng là các ký hiệu xuất hiện trong ngôn ngữ được sinh bởi văn phạm. Các luật sinh của văn phạm có dạng $s \rightarrow tu$, nghĩa là trong một xâu chứa ký hiệu s , ta có thể thay s bằng tu . Tuy nhiên, ở đây chúng ta sử dụng ký hiệu chặt chẽ hơn, được gọi là BNF (Backus Naur Form). Ý tưởng chính sẽ được mô tả qua ví dụ. Một ngôn ngữ các biểu thức số đơn giản được định nghĩa bởi văn phạm sau:

$$\begin{aligned} e &::= n \mid e+e \mid e-e \\ n &::= d \mid nd \\ d &::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

trong đó e là ký hiệu bắt đầu, các ký hiệu e , n và d là các ký hiệu chưa tận, và $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +$ và $-$ là các ký hiệu tận cùng. Ngôn ngữ sinh bởi văn phạm này bao gồm tất cả các dãy tận cùng mà chúng ta có thể tạo bắt đầu từ ký hiệu e và thay các chưa tận bằng luật sinh tương ứng. Chẳng hạn, thay e bằng luật sinh thứ ba dòng đầu tiên ta có ba ký hiệu đầu là $e-e$. Quá trình có thể được lặp với bất cứ luật sinh nào. Một số biểu thức trong ngôn ngữ được sinh bởi văn phạm là

$$0, 1+3+5, 2+4-6-8$$

Các dãy ký hiệu mà chứa chưa tận, như

$$e, e+e, e+6-e$$

không phải là các biểu thức trong ngôn ngữ cho bởi văn phạm trên. Mục đích của các chưa tận là giữ dạng mong muốn của biểu thức khi nó được tạo. Mọi chưa tận cần phải được thay bằng tận cùng để tạo biểu thức dạng đúng của ngôn ngữ.

Suy diễn

Một dãy các phép thay thế cho kết quả là một xâu các tận cùng được gọi là một phép suy diễn.

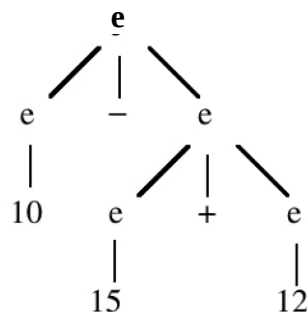
Ở đây có hai phép suy diễn trong văn phạm trên, cái thứ nhất được cho ở dạng đầy đủ và cái thứ hai bỏ qua một số bước mà đã rõ ràng và có thể lấp đầy bởi người đọc:

$$\begin{aligned} e &\rightarrow n \rightarrow nd \rightarrow dd \rightarrow 2d \rightarrow 25 \\ e &\rightarrow e - e \rightarrow e - e+e \rightarrow \dots \rightarrow n-n+n \rightarrow \dots 10-15+12 \end{aligned}$$

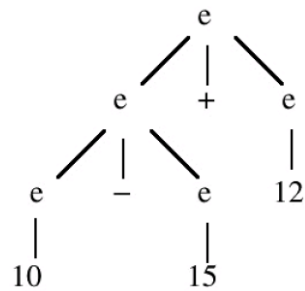
Cây cú pháp và tính nhập nhằng

Thông thường thuận tiện biểu diễn một phép suy diễn bằng một cây. Cây này, được gọi là cây cú pháp (*parse tree*) của phép suy diễn, hay là *cây suy diễn*, được xây dựng với ký hiệu ban đầu là gốc của cây đó. Nếu một bước trong phép suy diễn là thay thế s bằng $x_1, \dots, x_n$, thì các con của s trong cây đó sẽ là các đỉnh được gán nhãn $x_1, \dots, x_n$.

Cây cú pháp đối với phép suy diễn của $10-15+12$ trong văn phạm ở mục trên có dạng sau, khi luật sinh đầu tiên có dạng $e - e$:



ở đó chúng ta nhập các cây con cho số có hai chữ số vào một đỉnh duy nhất. Phép suy diễn này có thể có cây cú pháp dạng khác, nếu ta lựa chọn khi luật sinh đầu tiên có dạng $e + e$:



Vấn đề quan trọng ở đây về cây cú pháp là mỗi cây tương ứng với một cách đóng ngoặc duy nhất của biểu thức. Cụ thể, cây cú pháp thứ nhất tương ứng với $10 - (15 + 12)$, trong khi cây thứ hai tương ứng với $(10 - 15) + 12$. Ví dụ này chỉ rõ là giá trị của biểu thức có thể phụ thuộc vào cách phân tích hay đóng ngoặc lại.

Một văn phạm được gọi là *nhập nhằng*, nếu một biểu thức nào đó có nhiều hơn một cây cú pháp. Nếu mỗi biểu thức có nhiều nhất một cây cú pháp, thì văn phạm là *không nhập nhằng*.

Ví dụ 1.2.

Ở đây có sự nhập nhằng thú vị về câu lệnh *if - then - else*. Nó có thể được mô tả bởi văn phạm sau:

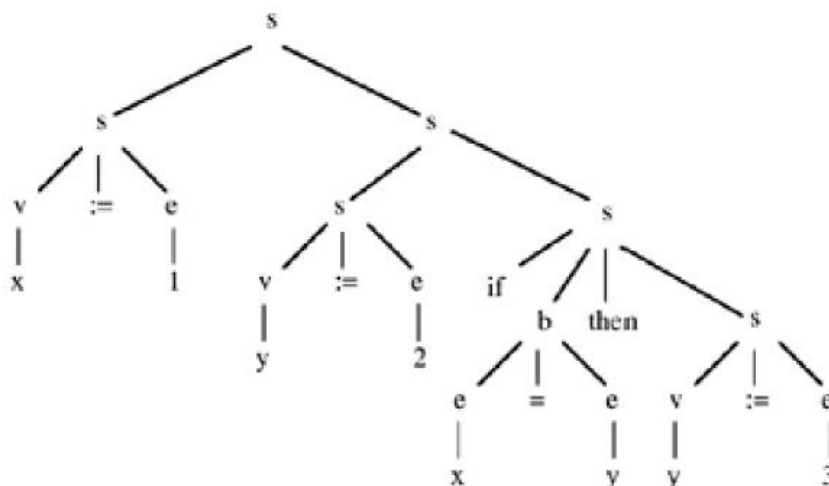
```

s ::= v := e | s; s | if b then s | if b then s else s
v ::= x | y | z
e ::= v | 0 | 1 | 2 | 3 | 4
b ::= e = e

```

trong đó s là ký hiệu ban đầu, s , v , e và b là các chưa tận, và các ký hiệu khác là tận cùng. Các chữ s , v , e , và b ký hiệu tương ứng cho các lệnh, biến, biểu thức, và kiểm tra bool. Chúng ta gọi các biểu thức sinh bởi văn phạm này là *các lệnh*. Đây là một ví dụ của một lệnh có dạng tốt và một trong các cây cú pháp của nó:

$x:=1; y:=2; \text{if } x=y \text{ then } y:=3$



Lệnh này còn có một cây cú pháp khác, mà chúng ta nhận được bằng cách đặt hai lệnh gán vào bên trái gốc, và lệnh *if - then* vào bên phải. Tuy nhiên, sự khác nhau giữa hai cây cú pháp này sẽ không tác động đến hành vi của code được sinh bởi trình biên dịch thông thường. Nguyên nhân là $s_1; s_2$ thông thường được dịch là code của s_1 được nối tiếp bởi code của s_2 . Như kết quả, cùng một code có thể sẽ được sinh, vì chúng ta coi $s_1; s_2; s_3$

như là $(s1; s2); s3$ hoặc $s1; (s2; s3)$. Nhiều tình huống phức tạp hơn nảy sinh khi *if – then* được kết hợp với *if – then – else* theo cách sau:

if b1 then if b2 then s1 else s2

Cái gì sẽ xảy ra, nếu $b1$ là true và $b2$ là false. Lệnh $s2$ có được thực hiện hay không? Như chúng ta thấy, điều này phụ thuộc vào việc lệnh này được phân tích như thế nào. Một văn phạm mà cho phép sự kết hợp các điều kiện này là *nhập nhằng*, với hai ý nghĩa có thể có đối với các lệnh của dạng đó.

1.3.3 Phân tích và độ ưu tiên

Phân tích là một quá trình xây dựng cây cú pháp cho dãy các ký hiệu. Giả sử chúng ta định nghĩa một ngôn ngữ L bằng việc viết ra một văn phạm G . Khi đó, cho trước một dãy ký hiệu s , chúng ta mong muốn là có thể xác định được cây cú pháp, nếu s nằm trong ngôn ngữ L đó. Nếu vậy, thì chúng ta có thể biên dịch hoặc thông dịch được s , vì chúng ta đã xây dựng được cây cú pháp cho s . Một thuật toán mà xác định s có ở trong ngôn ngữ L không, và xây dựng cây cú pháp cho nó, nếu nó thuộc L , được gọi là thuật toán phân tích cho G . Có một số phương pháp xây dựng thuật toán phân tích từ văn phạm. Nhiều thuật toán chỉ làm việc cho một số dạng cụ thể của văn phạm. Vì vậy phân tích là phần quan trọng trong việc biên dịch ngôn ngữ lập trình, phân tích thường được dạy trong môn chương trình dịch.

Hai vấn đề chúng ta sẽ xem xét ngắn gọn là chuyển đổi cú pháp của độ ưu tiên và kết hợp phải hoặc kết hợp trái. Điều này được minh họa ngắn gọn trong ví dụ sau.

$e ::= 0 \mid 1 \mid e+e \mid e-e \mid e*e$

Văn phạm này là nhập nhằng, vì nhiều biểu thức có nhiều hơn một cây cú pháp. Đối với các biểu thức như $1 - 1 + 1$, giá trị của biểu thức sẽ phụ thuộc vào cách mà nó được phân tích. Một giải pháp cho vấn đề này là yêu cầu đóng mở ngoặc đầy đủ. Nói cách khác chúng ta cần thay đổi văn phạm thành

$e ::= 0 \mid 1 \mid (e+e) \mid (e-e) \mid (e*e)$

sao cho không còn nhập nhằng nữa. Tuy nhiên, điều này có thể là bất tiện phải viết nhiều ngoặc. Ngoài ra, đối với nhiều biểu thức như $1 + 2 + 3 + 4$, giá trị của biểu thức không phụ thuộc vào cách nó được phân tích. Do đó, không cần thiết công kênh yêu cầu ngoặc cho mỗi thao tác.

Giải pháp chuẩn cho vấn đề này là làm theo các chuẩn phân tích mà đặc tả một cây cú pháp duy nhất cho mỗi biểu thức. Các chuẩn này được gọi là độ ưu tiên và tính kết hợp. Đối với văn phạm chuyên biệt này, chuẩn ưu tiên tự nhiên là phép nhân $*$ có độ ưu tiên cao hơn phép $+$ và phép $-$. Chúng ta tích hợp độ ưu tiên này phân tích bằng cách coi biểu thức không ngoặc

$e \text{ op1 } e \text{ op2 } e$

như là các ngoặc được chèn vào xung quanh phép toán có độ ưu tiên cao hơn. Với qui tắc này tác động, biểu thức

$5 * 4 - 3$

sẽ được phân tích như là nó được viết dạng

$(5 * 4) - 3$

Điều này trùng với cách mà chúng ta hay nghĩ thông thường về biểu thức $5*4-3$. Vì ở đây không có cách chuẩn khi hầu hết mọi người phân tích $1 + 1 - 1$, chúng ta có thể cho phép cộng và phép trừ có độ ưu tiên bằng nhau. Trong trường hợp này trình biên dịch có thể đưa

ra thông điệp lỗi yêu cầu lập trình viên đóng ngoặc $1 + 1 - 1$. Có cách khác là, biểu thức trông như vậy được coi là nhập nhầm bằng việc sử dụng thêm chuẩn bổ sung.

Tính kết hợp được xét đến khi hai phép toán cùng độ ưu tiên xuất hiện cạnh nhau.

Với tính kết hợp trái, một biểu thức $e \text{ op1 } e \text{ op2 } e$ được phân tích là $(e \text{ op1 } e) \text{ op2 } e$

Với tính kết hợp phải, một biểu thức $e \text{ op1 } e \text{ op2 } e$ được phân tích là $e \text{ op1 } (e \text{ op2 } e)$

Biểu thức	Độ ưu tiên	Kết hợp trái	Kết hợp phải
$5*4-3$	$(5*4)-3$	(không thay đổi)	(không thay đổi)
$1+1-1$	(không thay đổi)	$(1+1)-1$	$1+(1-1)$
$2+3-4*5+2$	$2+3-(4*5)+2$	$((2+3)-(4*5))+2$	$2+(3-((4*5)+2))$

1.4 Tính toán Lambda

Tính toán Lambda là một hệ thống toán học mà mô tả một số khái niệm quan trọng của ngôn ngữ lập trình trong một dạng thuần túy và đơn giản. Tính toán lambda truyền thống có ba phần chính: ký hiệu để định nghĩa hàm số, hệ thống chứng minh để chứng minh sự bằng nhau của các biểu thức và một tập các qui tắc tính toán được gọi là rút gọn. Từ đầu tiên trong tên của tính toán lambda được lấy từ việc sử dụng chữ Hy Lạp lambda (λ) trong biểu thức hàm. Từ thứ hai đến từ cách rút gọn có thể được sử dụng để *tính toán* kết quả của việc áp dụng hàm số cho một hay nhiều đối số. Việc *tính toán* này là một dạng đánh giá tương trưng các biểu thức. Tính toán lambda cung cấp một ký hiệu thuận tiện để mô tả các ngôn ngữ lập trình và có thể được coi như cơ sở của nhiều cấu trúc ngôn ngữ. Cụ thể, tính toán lambda cung cấp các dạng cơ bản tham số hóa bằng cách dùng biểu thức hàm và trói buộc bằng cách khai báo. Đây là các khái niệm cơ bản chung đối với hầu hết các ngôn ngữ lập trình hiện đại. Do đó cần phải làm quen với tính toán lambda để coi các biểu thức trong ngôn ngữ ưu thích của bạn như một dạng của biểu thức lambda. Để đơn giản chúng ta sẽ xét tính toán lambda không kiểu; còn có phiên bản kiểu của tính toán lambda. Trong tính toán lambda kiểu có kiểm tra ràng buộc kiểu mà loại trừ một số dạng của biểu thức. Tuy nhiên các khái niệm cơ bản và các qui tắc tính toán vẫn là như nhau.

1.4.1 Các hàm số và biểu thức hàm

Theo trực quan, một hàm số là một qui tắc định nghĩa một giá trị từ một đối số. Góc nhìn này của hàm số được sử dụng không hình thức trong hầu khắp toán học. Một số ví dụ của hàm số được học trong toán là

$$f(x) = x^2 + 3,$$

$$g(x + y) = \sqrt{x^2 + y^2}.$$

Trong dạng thuần túy và đơn giản của tính toán lambda, không có phép toán chuyên biệt miền nào như phép cộng hay lũy thừa, chỉ có định nghĩa hàm và áp dụng hàm. Nó cho phép chúng ta viết hàm số như

$$h(x) = f(g(x))$$

vì h được định nghĩa chỉ theo thuật ngữ áp dụng hàm và các hàm khác mà chúng ta giả thiết là đã được định nghĩa. Có thể bổ sung các phép toán như phép cộng và lũy thừa vào tính toán lambda thuần túy. Tuy nhiên những người thuần túy muốn xét tính toán lambda không có phép cộng và nhân, chúng ta sẽ sử dụng các phép toán này trong các ví dụ, điều này làm cho hàm số mà chúng ta định nghĩa quen thuộc hơn.

Cấu trúc chính của tính toán lambda là *trừu tượng* và *áp dụng lambda*. Chúng ta sử dụng trừu tượng lambda để viết các hàm số: nếu M là một biểu thức nào đó, thì

$$\lambda x.M$$

là một hàm số chúng ta nhận được bằng cách coi M như một hàm của đối số x . Chẳng hạn

$$\lambda x.x$$

là một trừu tượng lambda định nghĩa hàm đồng nhất, hàm số mà giá trị tại x là x . Cách quen thuộc hơn để định nghĩa hàm đồng nhất là viết dưới dạng

$$I(x) = x$$

Tuy nhiên, cách định nghĩa một hàm số như thế này buộc chúng ta phải đặt tên cho mỗi hàm chúng ta muốn. Tính toán lambda cho phép chúng ta viết hàm vô danh và sử dụng nó trong biểu thức lớn hơn.

Trong ký hiệu lambda, theo truyền thống viết áp dụng hàm chỉ bằng cách đặt biểu thức hàm trước một hoặc nhiều đối số; các dấu ngoặc là tùy chọn. Chẳng hạn, chúng ta có thể áp dụng hàm đồng nhất cho biểu thức M bằng cách viết

$$(\lambda x.x)M$$

Giá trị của việc áp dụng này là hàm đồng nhất áp dụng cho M bằng cách thay M cho x dưới biểu thức hàm và nó kết thúc bằng M . Như vậy chúng ta có

$$(\lambda x.x)M = M$$

Phần tính toán lambda là một tập hợp các qui tắc suy luận như trên. Ví dụ khác của một biểu thức lambda là

$$\lambda f.\lambda g.\lambda x.f(gx)$$

Cho trước các hàm f và g , hàm này tạo nên hàm hợp $\lambda x.f(gx)$ của f và g .

Chúng ta mở rộng tính toán lambda thuần túy bằng cách bổ sung thêm nhiều cấu trúc khác. Chúng ta gọi mở rộng của tính toán lambda thuần túy với các phép toán bổ sung là tính toán lambda áp dụng (*applied lambda calculus*). Ý tưởng cơ bản nằm dưới quan hệ giữa tính toán lambda và khoa học máy tính là biểu tượng sau

$$\begin{aligned}\text{Ngôn ngữ lập trình} &= \text{applied } \lambda\text{-calculus} \\ &= \lambda\text{-calculus thuần túy} + \text{các kiểu dữ liệu bổ sung}\end{aligned}$$

Điều trên đúng ngay cả cho các ngôn ngữ lập trình với tác động phụ, như cách mà một chương trình phụ thuộc vào trạng thái của máy, có thể được biểu diễn bằng các cấu trúc dữ liệu tường minh cho trạng thái của máy. Đây là một trong các ý tưởng cơ bản dưới ngữ nghĩa ký hiệu của Scott-Stracchey.

1.4.2. Các biểu thức lambda

Cú pháp của biểu thức

Cú pháp của các biểu thức lambda không kiểu có thể được định nghĩa với một văn phạm BNF. Chúng ta giả thiết có một tập vô hạn các biến V và sử dụng $x, y, z, \dots$ ký hiệu cho các biến bất kỳ. Văn phạm cho biểu thức lambda là

$$M ::= x \mid MM \mid \lambda x.M$$

trong đó x có thể là biến bất kỳ (một phần tử của V). Một biểu thức dạng M_1M_2 được gọi là một áp dụng và một biểu thức dạng $\lambda x.M$ được gọi là một trừu tượng lambda. Bằng trực quan, một biến x tham chiếu đến một hàm số nào đó, hàm cụ thể đó được xác định bởi ngữ cảnh; M_1M_2 là áp dụng hàm số M_1 cho đối số M_2 ; và $\lambda x.M$ là một hàm mà cho trước đối số x , trả về giá trị M . Trong một số tài liệu về tính toán lambda, thông thường biểu thức lambda được gọi là số hạng lambda (*lambda terms*).

Đây là một số ví dụ về số hạng lambda:

$\lambda x.x$ một trừu tượng lambda được gọi là hàm đồng nhất (*identity function*)
 $\lambda x.(f(gx))$ một trừu tượng lambda khác
 $(\lambda x.x)5$ một áp dụng (application)

Có một số qui định cú pháp mà nói chung thuận lợi cho các chuyên gia tính toán lambda, nhưng chúng dễ nhầm lẫn khi học. Chúng ta nói chung có thể tránh chúng bằng cách viết thêm ngoặc. Tuy nhiên, một qui tắc mà chúng ta sẽ sử dụng là trong một biểu thức chứa λ , phạm vi của λ mở rộng sang bên phải nhất có thể. Chẳng hạn, $\lambda x . xy$ cần phải được đọc là $\lambda x.(xy)$, chứ không phải $(\lambda x.x)y$.

Trói buộc biến

Một xuất hiện của một biến trong một biểu thức có thể là tự do hoặc bị trói buộc. Nếu biến là tự do trong một biểu thức, điều đó có nghĩa là biến đó không được khai báo trong biểu thức đó. Chẳng hạn, biến x là tự do trong biểu thức $x + 3$. Chúng ta không thể đánh giá biểu thức $x + 3$ vì nó đứng riêng không đặt trong một biểu thức lớn hơn mà sẽ gắn kết một giá trị nào đó với x . Nếu một biến là không tự do, thì nó cần phải là bị trói buộc (*bound*). Ký hiệu λ được gọi là thao tác trói buộc (*binding operator*), như nó trói buộc một biến trong một phạm vi chuyên biệt (phần của một biểu thức). Biến x là bị trói buộc trong $\lambda x.M$. Điều này có nghĩa x là một giữ chỗ, giống như x trong tích phân $\int f(x) dx$ hoặc công thức logic $\forall x.P(x)$, và ý nghĩa của $\lambda x.M$ không phụ thuộc vào x . Do đó, cả hai $\int f(x) dx$ và $\int f(y) dy$ biểu diễn cùng một tích phân như nhau, chúng ta có thể đổi tên biến x bị trói buộc λ sang biến y mà không thay đổi ý nghĩa của biểu thức đó. Cụ thể, $\lambda x.x$ định nghĩa cùng một hàm số như $\lambda y.y$.

Các biểu thức mà chỉ khác nhau ở tên của các biến bị trói buộc được gọi là α tương đương. Khi chúng ta muốn nhấn mạnh hai biểu thức đó là α tương đương, chẳng hạn, chúng ta viết

$$\lambda x.x = \alpha \lambda y.y$$

Trong $\lambda x.M$, biểu thức M được gọi là phạm vi của trói buộc λx . Biến x xuất hiện trong biểu thức M là bị trói buộc nếu nó xuất hiện trong phạm vi của một λx và ngược lại là tự do. Để chính xác hơn về điều này chúng ta có thể định nghĩa $FV(M)$ tập các biến tự do của một biểu thức M bằng phép qui nạp trên cấu trúc của biểu thức như sau:

$$\begin{aligned} FV(x) &= \{x\}, \\ FV(MN) &= FV(M) \cup FV(N), \text{ (ở đây } \cup \text{ là phép hợp hai tập hợp)} \\ FV(\lambda x.M) &= FV(M) - x, \end{aligned}$$

trong đó – là phép trừ tập hợp, tức là, $S - x = \{y \in S \mid y \neq x\}$. Chẳng hạn, $FV(\lambda x.x)$ là tập rỗng, vì không có biến tự do nào trong $\lambda x.x$, và

$$FV(\lambda f.\lambda x.(f(g(x)))) = \{g\}.$$

Đôi khi là cần thiết nói về sự xuất hiện khác nhau của một biến trong một biểu thức và phân biệt sự xuất hiện tự do và bị trói buộc. Trong biểu thức:

$$\lambda x.(\lambda y.xy)y,$$

sự xuất hiện đầu tiên của x được gọi là xuất hiện trói buộc (*binding occurrence*), và nó chỉ ra ở đâu x sẽ bị trói buộc. Còn xuất hiện thứ hai của x là xuất hiện bị trói buộc. Đọc từ trái sang phải xuất hiện đầu tiên của y là xuất hiện trói buộc, xuất hiện thứ hai là bị trói buộc và xuất hiện thứ ba của y là tự do, vì nó nằm ngoài phạm vi của λy ở trong ngoặc.

Có thể bị nhầm lẫn khi làm việc với các biểu thức mà sử dụng cùng một biến theo nhiều hơn một cách. Một chuyển đổi có ích là đổi tên biến bị trói buộc sao cho mọi biến trói buộc là khác nhau và khác với mọi biến tự do. Theo qui tắc này, ta sẽ viết

$$(\lambda y.(\lambda z.z)y)x \text{ thay cho } (\lambda x.(\lambda x.x)x)x.$$

Chuyển đổi biến sẽ là hữu ích khi chúng ta đến phần suy luận bằng nhau và rút gọn.

Trừu tượng Lambda trong C

Chẳng hạn, đoạn chương trình của C

```
int f(int x) {return x};
block_body;
```

với khai báo hàm ở đầu khối của là dễ chuyển đổi sang tính toán lambda. Việc chuyển đổi này là dễ hiểu hơn nếu ta trước hết bổ sung khai báo vào tính toán lambda. Chẳng hạn đơn giản khai báo như sau:

$$\text{let } x = M \text{ in } N,$$

mà khai báo là x có giá trị M trong thân N , có thể được coi như một hình thức cú pháp cho kết hợp cho trừu tượng và ứng dụng lambda:

$$\text{let } x = M \text{ in } N \text{ có nghĩa là } (\lambda x.N)M.$$

Đối với những ai không quen với cú pháp, điều này có nghĩa là câu $\text{let } x = M \text{ in } N$ là dễ viết hơn trong một số trường hợp, nhưng chúng ta có thể chỉ nghĩ là cú pháp $\text{let } x = M \text{ in } N$ được dùng cho $(\lambda x.N)M$.

Sử dụng khai báo let , chúng ta có thể viết chương trình C ở trên như

$$\text{let } f = (\lambda x.x) \text{ in } \text{block_body}$$

Lưu ý rằng biểu thức dạng C và biểu thức lambda có cùng các biến tự do và bị trói buộc và có cùng cấu trúc như nhau. Điểm khác biệt giữa C và tính toán lambda là C có lệnh gán và tác động phụ, trong khi tính toán lambda là thuần túy hàm số. Tuy nhiên bằng việc đưa vào *stores*, ánh xạ từ vị trí các biến vào các giá trị, chúng ta có thể chuyển đổi chương trình C vào các số hạng lambda được. Sự chuyển đổi đó giữ cấu trúc tổng thể của chương trình,

nhưng làm cho chương trình trông phức tạp hơn vì sự phụ thuộc vào trạng thái của máy là tường minh.

Sự tương đương và phép thế

Chúng ta đã bàn về sự tương đương α của các số hạng. Đây là một trong những tiên đề cơ bản của tính toán lambda, nghĩa là nó là một trong những tính chất của số hạng lambda mà định nghĩa hệ thống và nó được sử dụng trong chứng minh các tính chất khác của các số hạng lambda. Khẳng định cẩn thận hơn trước, hệ thống chứng minh bằng nhau của tính toán lambda có tiên đề bằng nhau:

$$\lambda x. M = \lambda y. [y/x]M, \quad (\alpha)$$

trong đó $[y/x]M$ là kết quả của việc thay y cho các xuất hiện tự do của x trong M và y không xuất hiện trong M trước đó. Có ba tiên đề khác và bốn luật suy diễn để chứng minh bằng nhau giữa các số hạng. Tuy nhiên chúng ta sẽ chỉ xét một tiên đề bằng nhau khác.

Tiên đề bằng nhau trung tâm của tính toán lambda được sử dụng để tính giá trị của áp dụng hàm số $(\lambda x.M)N$ bằng phép thế. Vì $\lambda x.M$ là hàm số mà chúng ta nhận được coi M như một hàm của x , chúng ta có thể viết giá trị của hàm này tại N bằng cách thay N cho x . Lại viết $[N/x]M$ sẽ cho kết quả thay thế N cho các xuất hiện tự do của x trong M , ta có

$$(\lambda x.M)N = [N/x]M, \quad (\beta)$$

mà được gọi là tiên đề tương đương β . Các chú ý quan trọng về phép thế sẽ đề cập trong mục sau. Giá trị của $\lambda f. f x$ áp dụng cho $\lambda y.y$ có thể được tính nếu đối số $\lambda y.y$ được thay cho biến bị trói buộc f :

$$(\lambda f.f x) (\lambda y.y) = (\lambda y.y)x$$

Tất nhiên, $(\lambda y.y)x$ có thể được rút gọn tiếp bằng thay thế nữa, và chúng ta có

$$(\lambda f.f x)(\lambda y.y) = (\lambda y.y)x = x$$

Đổi tên các biến bị trói buộc

Vì trói buộc λ trong M có thể xung đột với các biến tự do trong N , phép thế $[N/x]M$ là phức tạp hơn so với chúng ta nghĩ ban đầu. Tuy nhiên, chúng ta có thể tránh mọi phức tạp này bằng cách đổi biến sau: đổi tên biến bị trói buộc trong $(\lambda x.M)N$ sao cho mọi biến bị trói buộc là khác nhau và khác với mọi biến tự do. Chẳng hạn, chúng ta rút gọn số hạng

$$(\lambda f.\lambda x.f(fx))(\lambda y.y+x)$$

Nếu trước hết ta đổi tên các biến bị trói buộc và sau đó thực hiện phép rút gọn β , chúng ta nhận được

$$(\lambda f.\lambda z.f(fz))(\lambda y.y+x) = \lambda z.((\lambda y.y+x)((\lambda y.y+x)z)) = \lambda z.z+x+x$$

Nếu chúng ta mà quên đổi tên các biến bị trói buộc và hay thế một cách mù quáng, chúng ta có thể đơn giản hóa như sau:

$$(\lambda f.\lambda x.f(fx))(\lambda y.y+x) = \lambda x.((\lambda y.y+x)((\lambda y.y+x)x)) = \lambda x.x+x+x$$

Tuy nhiên, chúng ta cần nghi ngờ về cách rút gọn thứ hai, vì biến x là tự do trong $(\lambda y.y + x)$ nhưng nó trở thành bị trói buộc trong $\lambda x.x + x + x$. *Hãy nhớ rằng:* trong khi thể $[N/x]M$, chúng ta cần đổi biến bất kỳ biến bị trói buộc nào trong M mà cũng có thể là như nhau với biến tự do trong N . Để chính xác về đổi tên các biến bị trói buộc trong phép thế, chúng ta định nghĩa kết quả $[N/x]M$ của phép thế N cho x trong M bằng qui nạp trên cấu trúc của M :

$$\begin{aligned} [N/x]x &= N, \\ [N/x]y &= y, \text{ trong đó } y \text{ là biến bất kỳ khác với } x, \\ [N/x](M_1M_2) &= ([N/x]M_1)([N/x]M_2), \\ [N/x](\lambda x.M) &= \lambda x.M, \\ [N/x](\lambda y.M) &= \lambda y.([N/x]M), \text{ trong đó } y \text{ là không tự do trong } N. \end{aligned}$$

Vì chúng ta có thể đổi tên biến bị trói buộc y trong $\text{rename } \lambda y.M$, mệnh đề cuối $\lambda y.([N/x]M)$ là luôn có ý nghĩa. Với định nghĩa chính xác này, bây giờ chúng ta đã có định nghĩa chính xác của tương đương β .

1.4.3 Lập trình trong tính toán Lambda

Tính toán Lambda có thể được xem như một ngôn ngữ lập trình hàm đơn giản. Chúng ta sẽ thấy rằng, ngay cả khi ngôn ngữ là rất đơn giản, chúng ta có thể lập trình tương đối tự nhiên, nếu chúng ta cho phép một số rút gọn. Trước khi bàn về khai báo và đệ qui, chúng ta sẽ nhắc lại khái niệm hàm nhiều đối số.

Các hàm của một số đối số

Trừu tượng lambda cho phép chúng ta xét biểu thức M bất kỳ như một hàm số của biến x bất kỳ bằng cách viết $\lambda x.M$. Tuy nhiên, cái mà chúng ta muốn bây giờ là coi M như một hàm hai biến x và y ? Chúng ta có cần dạng trừu tượng thứ hai $\lambda 2x, y.M$ để coi M như một hàm của hai biến x, y không? Mặc dù chúng ta có thể bổ sung toán tử lambda cho dãy các tham số hình thức, chúng ta không cần như vậy, vì trừu tượng lambda thông thường đã đủ cho hầu hết các mục tiêu. Chúng ta có thể biểu diễn một hàm f của hai đối số bởi hàm

$$\lambda x.(\lambda y.M)$$

của một đối số, khi áp dụng, trả về một hàm thứ hai mà nhận đối số thứ hai và sau đó tính kết quả theo cách thông thường cho f . Chẳng hạn, hàm số

$$f(g, x) = g(x)$$

có hai đối số, nhưng có thể biểu diễn trong tính toán lambda bằng trừu tượng lambda thông thường. Chúng ta định nghĩa f_{curry}

$$f_{\text{curry}} = \lambda g. \lambda x. gx$$

Sự khác biệt giữa f và f_{curry} là f nhận một cặp (g, x) làm một đối số, trong khi đó f_{curry} nhận một đối số đơn g . Tuy nhiên, f_{curry} có thể được sử dụng thay cho f vì

$$f_{\text{curry}}g \ x = gx = f(g, x)$$

Như vậy, cuối cùng, hàm f_{curry} cùng làm cùng một việc như f . Ý tưởng đơn giản này được phát hiện bởi Schönfinkel, người đã nghiên cứu tính chất hàm số vào những năm 1920. Tuy nhiên kỹ thuật này cho hàm đa đối số trong tính toán lambda thường được gọi là *Currying* theo tên của nhà khoa học tiên phong về tính toán lambda Haskell Curry.

Các khai báo

Chúng ta đã có thể coi các khai báo let

$$\text{let } x = M \text{ in } N$$

như các số hạng lambda bằng cách viết gọn như sau

$$\text{let } x = M \text{ in } N = (\lambda x. N)M$$

Cấu trúc let này có thể được sử dụng định nghĩa hàm hợp, như biểu thức

$$\text{let } \text{compose} = \lambda f. \lambda g. \lambda x. f(gx)$$

trong áp dụng hàm $\text{compose } h \ h$. Sử dụng tương đương β , chúng ta có thể đơn giản biểu thức let này về biểu thức

$$\text{compose } h \ h = \lambda x. h(hx),$$

là hàm hợp của h với chính nó. Trong ngôn ngữ lập trình, cấu trúc let cung cấp các khai báo cục bộ.

Đệ qui và các điểm bất động

Một việc đáng ngạc nhiên của tính toán lambda thuần túy là nó có thể cho phép viết hàm đệ qui bằng việc sử dụng mẹo áp dụng chính nó. Điều này không có nhiều việc phải làm khi so sánh với các ngôn ngữ lập trình hiện đại, nhưng nó có thể làm cho bạn đọc thích thú với kỹ thuật thông minh. Nhiều ngôn ngữ lập trình cho phép định nghĩa hàm đệ qui. Tính chất đặc thù của một định nghĩa đệ qui là của một hàm f là thân hàm này chứa một hoặc nhiều hơn lời gọi đến chính hàm f . Để chọn ví dụ chuyên biệt này, giả sử chúng ta định nghĩa hàm giai thừa trong một ngôn ngữ lập trình bảo đó bằng việc khai báo như

$$\text{function } f(n) \text{ \{if } n=0 \text{ then } 1 \text{ else } n*f(n-1)\};$$

trong đó thân của hàm là biểu thức trong ngoặc kép. Định nghĩa này có giá trị tính toán trực tiếp: khi f được gọi với đối số a , tham số hàm số n được đặt bằng a và thân hàm sẽ được tính toán. Khi quá trình tính toán thân hàm đạt đến lời gọi đệ qui, thì quá trình này được lặp lại. Như các định nghĩa của một thủ tục tính toán, các định nghĩa đệ qui là có ý nghĩa rõ ràng và hữu ích. Một cách để hiểu tiếp cận đệ qui của tính toán lambda là gắn kết một phương trình với định nghĩa đệ qui. Chẳng hạn, chúng ta có thể gắn kết phương trình

$$f(n) = \text{if } n=0 \text{ then } 1 \text{ else } n*f(n-1)$$

với khai báo đệ qui trước. Phương trình này khẳng định tính chất của giai thừa.

Đặc biệt, nếu giá trị của biểu thức $f(n)$ là bằng giá trị của biểu thức nếu $n=0$ thì bằng 1 ngược lại là $n*f(n-1)$, thì khi đó f là hàm giai thừa. Cách tiếp cận tính toán lambda này có thể được xem như phương pháp tìm lời giải cho phương trình ở đó, một định danh (tên của hàm đệ qui) xuất hiện trong cả hai vế của phương trình.

Chúng ta có thể đơn giản phương trình trên bằng sử dụng trừu tượng lambda để khử n khỏi vế bên trái. Nó cho

$$f = \lambda n. \text{ if } n=0 \text{ then } 1 \text{ else } n*f(n-1)$$

là khai báo đệ qui trước đó. Phương trình này khẳng định tính chất của giai thừa.

Bây giờ chúng ta xem xét hàm G mà chúng ta nhận được bằng cách chuyển f sang về phải của phương trình:

$$G = \lambda f. \lambda n. \text{ if } n=0 \text{ then } 1 \text{ else } n*f(n-1)$$

Mặc dù có thể là không rõ ràng kiểu đại số gì được sử dụng ở đây để biến đổi phương trình, chúng ta có thể kiểm tra bằng việc sử dụng suy luận tính toán lambda và hiểu biết cơ bản về sự bằng nhau của phương trình, rằng hàm giai thừa f thỏa mãn phương trình

$$f = G(f)$$

Điều này chỉ ra rằng các khai báo đệ qui dẫn đến tìm các điểm bất động.

Điểm *bất động* của một hàm G là một giá trị f sao cho $f = G(f)$. Trong tính toán lambda, các điểm bất động có thể được định nghĩa với *phép toán điểm bất động*

$$Y = \lambda f. (\lambda x. f(xx))(\lambda x. f(xx)).$$

Điều ngạc nhiên về biểu thức lambda rắc rối này là đối với bất kỳ f nào, áp dụng Yf là điểm bất động của f . Chúng ta có thể tính bằng tương đương β

$$Yf = (\lambda x. f(xx))(\lambda x. f(xx)).$$

Sử dụng tương đương β một lần nữa cho số hạng ở về phải, ta có

$$Yf = (\lambda x. f(xx))(\lambda x. f(xx)) = f(\lambda x. f(xx))(\lambda x. f(xx)) = f(Yf).$$

Như vậy, Yf là điểm bất động của f .

Ví dụ 1.3

Chúng ta có thể định nghĩa giai thừa bởi $fact = YG$, trong đó các số hạng lambda Y và G như được cho ở trên, và tính $fact\ 2 = 2!$ bằng cách các qui tắc tính lambda. Ở đây có các bước tính thể hiện lời gọi thứ nhất đến giai thừa:

$$\begin{aligned} fact\ 2 &= (YG)\ 2 \\ &= G(YG)\ 2 \\ &= (\lambda f. \lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } n * f(n-1))(YG)\ 2 \\ &= (\lambda n. \text{ if } n = 0 \text{ then } 1 \text{ else } n * ((YG)(n-1)))\ 2 \\ &= \text{if } 2 = 0 \text{ then } 1 \text{ else } 2 * ((YG)(2-1)) \\ &= \text{if } 2 = 0 \text{ then } 1 \text{ else } 2 * ((YG)\ 1) \\ &= 2 * ((YG)\ 1). \end{aligned}$$

Sử dụng các bước tương tự, chúng ta có thể tính $(YG)\ 1 = 1! = 1$ để kết thúc tính toán này. Đáng lưu ý là Y không chỉ là một số hạng lambda mà tìm các điểm bất động của phương trình. Có các biểu thức khác mà cũng làm việc tốt. Tuy nhiên, thao tác điểm bất động cụ thể này đóng vai trò quan trọng trong lịch sử tính toán lambda.

1.4.4 Rút gọn, tương đẳng và dạng chuẩn

Các tính chất tính toán của lambda thường được mô tả với một dạng phương trình tượng trưng được gọi là rút gọn. Theo thuật ngữ đơn giản, rút gọn là suy luận bằng nhau, nhưng theo một hướng nào đó. Đặc biệt, mặc dù tương đương β được viết dưới dạng một phương trình, nói chung chúng ta sử dụng nó theo một hướng tính toán lời gọi hàm. Nếu chúng ta viết $\rightarrow$ thay cho $=$ để chỉ ra hướng mà theo đó chúng ta muốn sử dụng phương trình, thì chúng ta nhận được một bước tính cơ bản được gọi là một rút gọn β .

$$(\lambda x. M)N \rightarrow [N/x]M. \quad (\beta)$$

Chúng ta nói rằng, M rút gọn β đến N , và viết $M \rightarrow N$, nếu N là kết quả của việc áp dụng một bước rút gọn β ở đâu đó bên trong M . Đa số các ví dụ tính toán trong mục này đều sử dụng rút gọn β , tức là một tương đương β được sử dụng từ trái qua phải. Chẳng hạn,

$$(\lambda f. \lambda z. f(fz))(\lambda y. y + x) \rightarrow \lambda z. ((\lambda y. y + x)(\lambda y. y + x)z) \rightarrow \lambda z. z + x + x.$$

Dạng chuẩn

Theo trực quan, chúng ta nghĩ $M \rightarrow N$ có nghĩa là, trong một bước tính toán, biểu thức M có thể được tính về một biểu thức N . Nói chung, quá trình này có thể được lặp lại như được mô tả trong mục trước. Tuy nhiên, đối với nhiều biểu thức, quá trình chần chẫn đạt đến điểm dừng. Một điểm dừng hoặc biểu thức không thể được tiếp tục tính toán, được gọi là một *dạng chuẩn*. Đây là một ví dụ dãy rút gọn dẫn đến *dạng chuẩn*:

$$\begin{aligned} & (\lambda f. \lambda x. f(fx))(\lambda y. y + 1) 2 \\ & \rightarrow (\lambda x. (\lambda y. y + 1)((\lambda y. y + 1)x)) 2 \\ & \rightarrow (\lambda x. (\lambda y. y + 1)(x + 1)) 2 \\ & \rightarrow (\lambda x. (x + 1 + 1)) 2 \\ & \rightarrow (2 + 1 + 1). \end{aligned}$$

Biểu thức cuối cùng này là một dạng chuẩn, nếu như qui tắc tính toán của chúng ta là β , nhưng nó không là dạng chuẩn nếu chúng ta có qui tắc tính toán cho phép cộng. Giả sử qui tắc tính toán thông thường có phép cộng, chúng ta có thể tiếp tục.

$$\begin{aligned} & 2 + 1 + 1 \\ & \rightarrow 3 + 1 \\ & \rightarrow 4. \end{aligned}$$

Ví dụ này cho chúng ta một ý tưởng tốt về việc rút gọn trong tính toán lambda tương ứng với tính toán như thế nào. Từ những năm 1930, tính toán lambda đã có mô hình toán học đơn giản cho tính toán biểu thức

Tương đẳng

Trong ví dụ trên, chúng ta bắt đầu với

$$(\lambda f. \lambda x. f(fx))(\lambda y. y + 1) 2,$$

có một số bước, ở đó chúng ta đã chọn một số biểu thức con có thể để tính toán. Chẳng hạn, trong biểu thức thứ hai

$$(\lambda x.(\lambda y.y + 1)((\lambda y.y + 1)x))2,$$

chúng ta có thể tính toán một trong hai lời gọi hàm bắt đầu với λy . Điều này không phải mẹo tính toán lambda, vì chúng ta có hai lựa chọn để tính ra biểu thức số học thuần túy $2 + 1 + 1$.

Một tính chất của tính toán lambda được gọi là tương đẳng. Trong tính toán lambda, kết quả của tương đẳng là thứ tự tính toán không ảnh hưởng đến kết quả cuối cùng của biểu thức. Nói cách khác, nếu một biểu thức M có thể rút gọn về một dạng chuẩn, thì chỉ có một dạng chuẩn của M , không phụ thuộc vào thứ tự ở đó ta chọn để tính toán các biểu thức con.

1.4. Ngôn ngữ lập trình hàm và mệnh lệnh

1.4.1. Cấu trúc mệnh lệnh và khai báo

Trong nhiều ngôn ngữ lập trình, các cấu trúc cơ sở là các câu lệnh mệnh lệnh. Chẳng hạn, một lệnh gán như

```
x:=5
```

là một lệnh cho máy tính lưu giá trị 5 vào một vị trí nào đó. Các ngôn ngữ lập trình cũng chứa một số cấu trúc khai báo chẳng hạn như khai báo một hàm số:

```
function f(int x) {return x+1;}
```

mà khẳng định một sự kiện. Người ta đọc điều này như một câu khẳng định về một chủ đề có tên là f và một câu về f như “ f là một hàm mà trả về giá trị lớn hơn đối số của nó 1 đơn vị”.

Trong lập trình, sự khác biệt giữa các cấu trúc mệnh lệnh và khai báo nằm ở sự khác biệt giữa thay đổi giá trị tồn tại và khai báo một giá trị mới. Cái đầu tiên là mệnh lệnh, còn cái sau là khai báo. Chẳng hạn, xét đoạn chương trình sau:

```
{ int x=1; /* declares new x */
  x = x+1; /* assignment to existing x */
  { int y = x+1; /* declares new y */
    { int x = y+1; /* declares new x */
      }}}

```

Ở đây, chỉ có dòng thứ hai là câu lệnh mệnh lệnh. Đây là câu lệnh mệnh lệnh mà thay đổi trạng thái của máy bằng việc lưu giá trị 2 vào vị trí gắn với biến x . Các dòng khác chứa các khai báo của các biến mới.

Thực tế, chúng ta có thể đổi tên các biến bị trói buộc trong đoạn chương trình trên để nhận được

```
{ int x=1; /* khai báo biến mới x */
  x = x+1; /* phép gán cho biến đã có x */
  { int y = x+1; /* khai báo biến mới y */
    { int z = y+1; /* khai báo biến mới z */
      }}}

```

Sau khi viết lại dạng tương đương, chúng ta dễ dàng thấy khai báo biến mới z không thay đổi giá trị của bất cứ biến tồn tại trước đó nào.

1.4.2 Chương trình hàm và chương trình mệnh lệnh

Câu ngôn ngữ hàm số được sử dụng tham chiếu đến ngôn ngữ lập trình ở đó hầu hết tính toán được làm bởi tính giá trị biểu thức chứa hàm số. Như ngôn ngữ ML sẽ được xét trong chương sau. Ngôn ngữ chứa cả cấu trúc khai báo và mệnh lệnh. Tuy nhiên, có thể viết một chương trình trong ngôn ngữ ML mà không sử dụng cấu trúc mệnh lệnh nào. Một số người sử dụng thuật ngữ ngôn ngữ hàm số để tham chiếu đến ngôn ngữ mà không có biểu thức với tác động phụ hoặc một dạng nào khác của cấu trúc mệnh lệnh. Tuy nhiên, chúng ta sử dụng câu nhấn mạnh hơn “ngôn ngữ hàm số thuần túy” cho ngôn ngữ khai báo mà được thiết kế quanh các cấu trúc linh hoạt để định nghĩa và sử dụng hàm số.

Ngôn ngữ hàm số thuần túy vượt qua phép kiểm tra sau:

Kiểm tra ngôn ngữ khai báo: Trong phạm vi các khai báo cụ thể của $x_1, \dots, x_n$, mọi xuất hiện của một biểu thức e chỉ chứa các biến $x_1, \dots, x_n$ có cùng giá trị như nhau. Như hệ quả, ngôn ngữ hàm số có tính chất tối ưu có ích sau: nếu biểu thức e xuất hiện một số chỗ trong một phạm vi cụ thể, thì biểu thức này chỉ cần tính toán một lần.

Tranh cãi lịch sử

John Backus nhận giải thưởng Turing ACM năm 1977 về phát triển Fortran. Trong bài phát biểu trong buổi lễ nhận giải thưởng này, Backus tranh luận rằng lập trình hàm thuần túy là tốt hơn lập trình mệnh lệnh. Bài diễn văn và bài báo đi kèm được đăng trong ACM giúp làm sáng tỏ một số dự án nghiên cứu hướng tới sự phát triển ngôn ngữ lập trình hàm thuần túy. Tiền đề chính tranh luận của Backus là lập trình hàm thuần túy là dễ dàng suy luận hơn, vì chúng ta có thể hiểu biểu thức độc lập với ngữ cảnh mà nó xuất hiện.

Backus khẳng định rằng, khi chạy lâu dài, tính đúng đắn chương trình, tính dễ đọc và tính tin cậy là quan trọng hơn các yếu tố như tính hiệu quả. Điều này là có vẻ gây tranh cãi trong năm 1977, khi các chương trình thường là nhỏ hơn các chương trình hiện nay và máy tính khi đó chậm hơn. Trong những năm 1990, máy tính cuối cùng đạt đến giai đoạn, ở đó mọi tổ chức thương mại bắt đầu lựa chọn phương pháp phát triển phần mềm mà thời gian phát triển của lập trình viên là quan trọng hơn tính hiệu quả thời gian chạy. Vì niềm tin vào tầm quan trọng của tính đúng đắn, tính đọc được và tính tin cậy, Backus nghĩ rằng, các ngôn ngữ lập trình hàm sẽ được đánh giá cao hơn so với các ngôn ngữ có tác động phụ.

Để đẩy mạnh tranh luận này, Backus đề xuất một ngôn ngữ lập trình hàm thuần túy được gọi là FP (Functional programming). FP chứa một số hàm số cơ sở và một tập lớn các dạng kết hợp với chúng được xây dựng các hàm mới từ các hàm cũ. Một ví dụ từ bài báo của Backus là một chương trình đơn giản tính tích vô hướng của hai véc tơ. Trong C tích vô hướng của hai véc tơ được lưu trong các mảng a và b có thể được viết như sau:

```
int i, prod;
prod=0;
for (i=0; i < n; i++) prod = prod + a[i] * b[i];
```

Ngược lại, hàm tích vô hướng có thể được định nghĩa trong FP kết hợp giữa $+$ và $*$ với các phép toán vector. Cụ thể tích vô hướng có thể được biểu diễn như

$$\text{Inner product} = (\text{Insert } +) \circ (\text{ApplyToAll } x) \circ \text{Transpose}$$

trong đó $\circ$ là phép hợp hàm và Insert, ApplyToAll, và Transpose là các phép toán vector. Cụ thể phép toán Transpose của hai cặp danh sách tạo ra một danh sách của các cặp và ApplyToAll áp dụng phép toán đã cho cho mỗi phần tử trong một danh sách. Cho một phép

toán hai ngôi, và một danh sách, Insert có tác động chèn phép toán giữa từng cặp của các phần tử kề nhau trong danh sách để tính kết quả. Chẳng hạn

$$(\text{Insert } +) \langle 1, 2, 3, 4 \rangle = 1+2+3+4=10,$$

trong đó $\langle 1, 2, 3, 4 \rangle$ là ký hiệu FP cho danh sách với các phần tử 1, 2, 3, 4.

Mặc dù cú pháp của C có vẻ trông rõ ràng hơn với nhiều bạn đọc, Nó đáng giá để thử tượng tượng chúng sẽ được so sánh với nhau như thế nào. Một mặt biểu thức FP là tất cả các phần của nó đều là hàm số mà chúng ta có thể hiểu không cần nghĩ vectơ được biểu diễn trong bộ nhớ như thế nào. Ngược lại, chương trình C có thêm biến i và $prod$ mà không là một phần của hàm số mà chúng ta đang tính toán. Theo nghĩa này chương trình FP ở bậc cao hơn và trừu tượng hơn code của C.

Một điểm tổng quát hơn về chương trình FP là nếu một biểu thức hàm tương đương với một cái khác, thì chúng ta có thể thay thế nó với cái khác đó trong bất cứ chương trình nào. Điều đó dẫn đến một tập các luật đại số cho các chương trình FP. Bổ sung thêm vào các luật đại số, một ưu điểm khác của ngôn ngữ lập trình hàm là có thể được cài đặt song song. Một nguyên nhân là các yếu tố về tính đúng đắn, đọc được và tin cậy tác động đến thời gian phát triển, bắt lỗi và bảo trì code. Khi các máy tính trở nên nhanh hơn, nói chung là có thể chấp nhận chạy các chương trình kém hiệu quả hơn, cải tiến phần cứng bù trừ với phần mềm. Tuy nhiên, tăng tốc độ máy tính không làm cho con người hiệu quả hơn. Về khía cạnh này, Backus hoàn toàn đúng đắn liên quan đến các khía cạnh của ngôn ngữ lập trình trong tương lai.

Ngôn ngữ FP của Backus, một mặt chưa thành công. Sau nhiều năm nỗ lực phát triển cài đặt FP ở IBM, nhưng ngôn ngữ không được sử dụng rộng rãi. Một vấn đề là ngôn ngữ FP có cú pháp khó, Có lẽ quan trọng hơn là, có một số hạn chế về kiểu các cấu trúc dữ liệu và các cấu trúc điều khiển mà có thể được định nghĩa. Ngôn ngữ FP mà cho phép tên các biến và trói buộc là thành công hơn, như có các ngôn ngữ lập trình hỗ trợ tính thành phần và các thành phần thư viện tái sử dụng. Tuy nhiên Backus đưa ra vấn đề quan trọng hơn mà dẫn đến định hướng có ích và tranh luận.

Lập trình hàm và song song

Một khía cạnh hấp dẫn của lập trình hàm và chương trình được viết trong ngôn ngữ tập con lập trình hàm thuần túy của một ngôn ngữ lớn hơn là các chương trình được thực hiện song song. Điều này là hệ quả của phép kiểm tra ngôn ngữ khai báo. Chúng ta có thể thấy tính song song được nảy sinh trong ngôn ngữ hàm thuần túy bằng việc sử dụng một lời gọi hàm $f(e_1, \dots, e_n)$, ở đó các đối số của hàm số là các biểu thức cần được tính toán.

Lập trình hàm: Chúng ta có thể tính $f(e_1, \dots, e_n)$ bằng cách tính toán $e_1, \dots, e_n$ song song, vì giá trị các biểu thức này là độc lập.

Lập trình mệnh lệnh: Đối với biểu thức dạng $f(g(x), h(x))$, hàm g có thể thay đổi giá trị của x . Vì vậy các đối số của ngôn ngữ mệnh lệnh cần được tính toán theo một thứ tự tuần tự, cố định. Thứ tự này hạn chế sử dụng song song.

Backus sử dụng thuật ngữ nút cổ chai von Neumann để chỉ sự việc là trong việc thực hiện một chương trình mệnh lệnh, tính toán cần được tiến hành một bước tại một thời điểm. Vì mỗi bước trong chương trình có thể phụ thuộc vào bước trước đó, chúng ta cần truyền giá trị mỗi một tại một thời điểm từ bộ nhớ đến CPU và ngược lại. Kênh tuần tự này giữa CPU và bộ nhớ là cách Backus gọi là nút cổ chai von Neumann.

Mặc dù các chương trình hàm cung cấp cơ hội song song, và song song thường là cách hiệu quả hơn nhờ tăng tốc độ tính toán, tính hiệu quả sử dụng ưu điểm của tính toán song

song cổ hữu là khó. Một vấn đề mà tương đối dễ hiểu là lập trình hàm cung cấp rất nhiều khả năng song song. Mọi tính toán có thể đều có thể được thực hiện song song, nhiều bước tính toán hơn sẽ được thực hiện hơn cần thiết. Chẳng hạn, tính toán song song đầy đủ của một lệnh điều kiện

if e_1 then e_2 else e_3

sẽ bao gồm tính toán ba biểu thức. Trước hết, khi giá trị e_1 được tính, một trong hai tính toán khác sẽ trở nên không liên quan. Trong trường hợp này, tính toán không liên quan sẽ được dừng, nhưng giữa chừng, các tài nguyên dành cho tính toán sẽ không được sử dụng. Trong chương trình lớn, dễ dàng sinh ra rất nhiều các nhiệm vụ song song mà thời gian khởi tạo và chuyển đổi các tiến trình song song sẽ làm mất hiệu quả của tính toán. Nói chung ngôn ngữ lập trình song song cần cung cấp cách cho lập trình viên đặc tả ở đâu tính song song có thể là có ích. Cài đặt song song ngôn ngữ hàm thường có nhược điểm là các lập trình viên có ít kiểm soát với lượng song song được sử dụng trong tính toán.

Lập trình hàm thực tế

Bài giảng khi nhận giải thưởng Turing của Backus đã nảy sinh câu hỏi cơ bản: Các ngôn ngữ lập trình hàm thuần túy có ưu điểm thực tế đáng kể so với các ngôn ngữ mệnh lệnh không?

Mặc dù chúng ta đã xem xét nhiều ưu điểm của ngôn ngữ FP thuần túy, chúng ta vẫn chưa có câu trả lời xác định. Từ góc độ lý thuyết, ngôn ngữ FP cũng tốt như các ngôn ngữ lập trình mệnh lệnh. Điều này có thể được thể hiện khi chuyển đổi được thực hiện từ các chương trình C sang chương trình FP, các biểu thức tính toán lambda. Để trả lời câu hỏi trên thực tế, tuy nhiên chúng ta cần tiến hành các dự án lớn trên ngôn ngữ lập trình hàm và xem nó có thể tạo ra các phần mềm có ích không trong thời gian chấp nhận được. Một số công việc hướng tới trả lời câu hỏi này được thực hiện ở IBM trong dự án FP. Một số nỗ lực khác với các ngôn ngữ như Haskell và Lazy ML cũng đang được tiếp tục tiến hành trong các phòng thí nghiệm nghiên cứu ở các trường đại học. Mặc dù hầu hết lập trình được làm trong các ngôn ngữ mệnh lệnh, có thể trong tương lai gần các ngôn ngữ FP thuần túy sẽ trở nên phổ dụng hơn. Dù điều đó có xảy ra hay không, dự án FP cũng tạo ra nhiều ý tưởng thiết kế ngôn ngữ thú vị và các kỹ thuật cài đặt mà có ảnh hưởng lớn đến lập trình hàm thuần túy.

CHƯƠNG 2: NGÔN NGỮ LẬP TRÌNH ML

2.1 Pascal

Pascal được thiết kế trong những năm 1970 bởi Niklaus Wirth, người sử dụng các ý tưởng cấu trúc dữ liệu bởi C.A.R (Tony) Hoare. Trước hết Wirth thiết kế và cài đặt ngôn ngữ gọi là Algol W và sau đó cải tiến thiết kế Algol W thành Pascal. Wirth thiết kế Pascal từ một loạt các bài giảng trong cuốn sách “Algorithms + Data Structures = Programs” (Prentice-Hall, 1975). Ông ta thiết kế ngôn ngữ sao cho có thể có chương trình dịch đơn giản dịch qua một lần đọc.

Mặc dù việc sử dụng Pascal kết thúc vào những năm 1990, nhưng Pascal là một trong những ngôn ngữ lập trình được sử dụng rộng rãi nhất. trên gần 20 năm. Pascal đã rất thành công như một ngôn ngữ lập trình để giảng dạy, một phần vì nó được thiết kế cho mục đích này. Pascal cũng được sử dụng cho một số đáng kể các dự án lập trình bao gồm cả hệ điều hành và ứng dụng cho Apple Macintosh.

Đóng góp quan trọng của Hệ thống kiểu Pascal là một tập hợp phong phú các khái niệm cấu trúc dữ liệu. Chúng bao gồm cả bản ghi Record (giống như cấu trúc trong C), các biến thể của bản ghi (như dạng kiểu kết hợp union type), và các phạm vi con. Chẳng hạn, phạm vi $[1, \dots, 10]$ của các số nguyên giữa 1 và 10 trở thành kiểu lần đầu tiên trong Pascal. Hạn chế của Pascal là các tham số thủ tục không thể là các thủ tục với các tham số thủ tục. Trong cú pháp Pascal, thủ tục dạng sau

```
procedure DoSomething( j, k : integer );
procedure DoSomething( procedure P(i:integer); j,k, : integer);
```

là cho phép. Trên là thủ tục với tham số nguyên, thứ hai là thủ tục với tham số là thủ tục với tham số nguyên. Tuy nhiên, thủ tục dạng sau

```
procedure NotAllowed( procedure MyProc( procedure P(i:integer)));
```

là không được phép, vì tham số thủ tục không thể là các thủ tục với các tham số thủ tục. Có một chỗ ở đó Wirth tập trung vào giảng dạy và thiết kế ngôn ngữ một cách có hệ thống mà tạo ra vấn đề nhỏ là trong kiểu của tham số mảng. Trong Pascal kiểu của mảng có dạng

```
array [indexType] of entryType
```

trong đó indexType thường là kiểu phạm vi con. Chi tiết quan trọng ở đây là index type là một phần kiểu của mảng và hai kiểu mảng là bằng nhau chỉ khi chúng có cùng index type và entry type. Định nghĩa kiểu mảng này cho phép khai báo thủ tục như sau:

```
procedure p(a : array [1..10] of integer)
```

trong đó đối số của thủ tục là mảng của kiểu mảng nào đó, nhưng không cho phép

procedure p(n: integer, a : array [1..n] of integer)

mảng: array [1..n] of integer không được xem là kiểu hợp lệ trong Pascal. Thủ tục dạng

procedure p(a : array [1..10] of integer)

có thể được gọi chỉ với mảng có độ dài 10 như tham số thực tế. Đây là hạn chế không may mắn. Chẳng hạn, nếu ta muốn viết thủ tục sắp xếp, thì ta buộc phải viết thủ tục sắp xếp mảng có độ dài cố định. Nếu muốn sắp xếp mảng với một số độ dài khác nhau, thì trong Pascal sẽ phải copy thủ tục một số lần và thay đổi kích thước mảng cho mỗi bản copy.

Điều này là không cần thiết, vì bộ nhớ gắn kết với mảng đã được cấp trước khi truyền nó cho thủ tục. Do đó không có lý do gì khác ngoài việc kiểm tra kiểu đối với thủ tục cho phép tham số mảng chỉ có độ dài cố định. Sau đó các hạn chế này được gỡ bỏ cho các phiên bản sau của Pascal.

2.2. Phát triển của C

Mặc dù Pascal là một ngôn ngữ giảng dạy và hàn lâm rất thành công, nhưng C đã làm lu mờ Pascal như một sản phẩm ngôn ngữ lập trình. Một lý do mà không liên quan đến bản thân thiết kế ngôn ngữ, là sự phổ cập của hệ điều hành Unix, mà được viết trên C. Khi mà các chương trình được viết chạy trên Unix, mọi lời gọi hệ thống cơ bản sẽ ngay lập tức được hỗ trợ bởi C. Do đó, dễ dàng viết nhiều ứng dụng Unix trên C hơn là các ngôn ngữ lập trình khác. Một lý do khác cho sự phổ cập của C là nó có mô hình bộ nhớ đặc biệt mà rất gần với phần cứng bên dưới. Mặc dù C có nhiều khái niệm giống với Pascal, C ít cứng nhắc hơn trong việc tuân thủ các nguyên lý cơ bản và các hạn chế.

C được thiết kế nguyên bản và cài đặt từ 1969 đến 1973, như một phần của dự án Hệ điều hành Unix tại Bell Laboratories. C được thiết kế bởi Dennis Ritchie, một trong những người ban đầu thiết kế ra Unix, vì vậy ông ta và Ken Thompson có thể xây dựng Unix trên ngôn ngữ nào mà họ muốn. Thiết kế được bắt nguồn từ ngôn ngữ B (do đó sau này có tên là C, chữ cái tiếp theo trong bảng chữ cái), mà bản thân B dựa trên ngôn ngữ gọi là BCPL. Những thay đổi đáng kể của C được xảy ra từ 1977 đến 1979, như phần bắt buộc để đạt được tính chuyển mang của hệ thống Unix, và vào giữa những năm 1980 khi mà hội đồng chuẩn quốc gia Hoa kỳ chuẩn hóa ngôn ngữ. BCPL là ngôn ngữ lập trình hệ thống được thiết kế vào những năm 1960 và được sử dụng bởi các thành viên của dự án Hệ điều hành Multics trong Bell Laboratories. B là phương án cắt gọt bớt của BCPL, được thiết kế chạy trên máy tính nhỏ sử dụng bởi dự án Unix. Sự khác biệt chính giữa B và C ở chỗ B là ngôn ngữ không kiểu còn ngôn ngữ C có kiểu và có các qui tắc kiểm tra kiểu.

Một đặc tính để phân biệt C với các ngôn ngữ phổ cập khác là cách nó xử lý các vị trí nhớ, các mảng và con trỏ. Đây là một phần của C được kế thừa từ BCPL và B. Kiểu dữ liệu duy nhất trong B là 'word' hoặc 'cell', mẫu bit có độ dài cố định. Bộ nhớ trong BCPL và B được thể hiện cho lập trình viên dưới dạng mảng tuyến tính các từ 'word'. Con trỏ được xử lý như các chỉ số nguyên đến mảng này, và các mảng được khai báo bởi lập trình viên được xem như các từ kế nhau lấy từ mảng lớn của tất cả các từ bộ nhớ. Cách nhìn như vậy có một số hệ quả sau. Một là mảng và con trỏ khá tương đương nhau như mô tả dưới đây. Hai là, vì con trỏ là chỉ số dương trong mảng bộ nhớ, phép toán trên con trỏ cũng được

xem xét là có ý nghĩa: nếu p là địa chỉ của một vị trí nhớ, thì $p+1$ là địa chỉ của vị trí nhớ tiếp theo.

Con trỏ và mảng C

Trong C, các con trỏ và mảng được khai báo khác nhau, vì con trỏ và mảng là các kiểu khác nhau. Sau đây p là con trỏ số nguyên và A là mảng các số nguyên:

```
int *p;
```

```
int A[5];
```

Trong hầu hết các ngôn ngữ lập trình, có một thao tác nào đó dereferencing con trỏ. Dereferencing là thao tác trả về vị trí được trỏ bởi con trỏ. Trong hầu hết các ngôn ngữ có dùng mảng, có một thao tác đánh chỉ số mà có thể được dùng để tìm một vị trí trong mảng. Có sự giống nhau giữa hai thao tác này, nhưng các ngôn ngữ có kiểu đều coi dereferencing đến tên mảng và đánh chỉ số con trỏ đều không hợp lệ. Tuy nhiên, trong C các mảng được xử lý rất hiệu quả xem như con trỏ. Để trích dẫn xem C Reference Manual, Dennis Ritchie, 1975:

Mỗi khi định danh kiểu mảng xuất hiện trong biểu thức, nó chuyển về con trỏ đến phần tử đầu tiên của mảng. Theo định nghĩa, thao tác chỉ số $[]$ được dịch theo cách sau ' $E1[E2]$ ' được đồng nhất với ' $*((E1)+(E2))$ '. Vì theo qui tắc chuyển kiểu mà áp dụng cho $+$, nếu $E1$ là mảng và $E2$ là số nguyên, thì $E1[E2]$ tham chiếu đến phần tử thứ $E2$ của $E1$.

Phê phán

Một đặc tính quan trọng của C là khả năng tự khắc phục lỗi kiểu của chương trình dịch C. Điều đó được thực hiện một phần nào đó, vì C được tạo ra từ các ngôn ngữ không có kiểu. Như C được phát triển tiếp theo và được chuẩn hóa bởi hội đồng ANSI, sự tương thích ngược với code C đã tồn tại đã ngăn cản các hạn chế kiểu mạnh. Mặc dù một số lập trình viên C thích việc viết và dịch chương trình có một số lỗi, hầu hết lập trình viên C đều coi việc kiểm tra kiểu yếu của nhiều chương trình dịch C là một nhược điểm. Trên thực tế, một trong những ưu điểm của C++ đối với C là C++ cung cấp việc kiểm tra kiểu tốt hơn.

Như Dennis Ritchie đã nói về sự phát triển của ngôn ngữ C (tại hội nghị ACM thứ hai về lịch sử các ngôn ngữ lập trình vào năm 1993), 'C nhanh, không hoàn thiện và cũng là thành công lớn'. Mặc dù có những sự cố trong quá khứ, nhưng C hiển nhiên đáp ứng yêu cầu là ngôn ngữ cài đặt hệ thống đủ hiệu quả để thay thế ngôn ngữ Assembly, cũng khá khái quát và uyển chuyển để thể hiện các thuật toán và tương tác trong nhiều môi trường khác nhau.

2.3. Hệ thống LCF và ML

ML có thể gọi gần như là ngôn ngữ lập trình hàm với các đặc trưng mệnh lệnh hoặc có thể là ngôn ngữ mệnh lệnh hướng hàm số. ML có các đặc trưng hàm linh hoạt, tương tự như LISP, cho phép tạo hàm trên cùng hàng như một phần của biểu thức, hàm số được truyền như đối số cho hàm và trả về như kết quả hàm số. ML cũng được mở rộng song song, làm cho nó phù hợp với việc phát triển các hệ thống song song và có cả mở rộng hướng đối tượng. Nhưng mục đích của chúng ta dùng ML là để kiểm tra các khái niệm dùng chung

của các ngôn ngữ mệnh lệnh, các ngôn ngữ lập trình hàm, các mở rộng song song và hướng đối tượng của chúng.

Sau đây ta sẽ liệt kê các lý do chính mà ta xem xét ML:

- ML thể hiện mọi khái niệm quan trọng của họ các ngôn ngữ LISP/Algol/Pascal
- Các hệ thống kiểu là phần quan trọng của ngôn ngữ lập trình được thiết kế từ những năm 1960 đến nay và hệ thống kiểu của ML được xem như là hệ thống kiểu thuần túy nhất và có khả năng biểu diễn nhất cho đến nay.
- Vì hầu hết các bạn đọc đều đã quen với C và nhiều người chưa viết nhiều chương trình trên các ngôn ngữ rất khác nhau khác, nên sẽ là có ích nếu ta dùng ngôn ngữ khác C trong các ví dụ ở các chương tiếp theo.
- ML cho phép các hàm bậc cao và các cấu trúc khác mà sẽ được bàn đến trong các chương tiếp theo.

Một trong những đặc tính quan trọng của ML là hệ thống kiểu của nó được mở rộng từ hệ thống kiểu rất thành công của Pascal. Không giống như C mà có nhiều kẽ hở, hệ thống kiểu ML là được chứng minh theo nghĩa toán học chính xác.

Đặc biệt, nếu bộ kiểm tra kiểu của ML xác định biểu thức có một kiểu nào đó, thì mọi tính toán của biểu thức sẽ đảm bảo rằng sẽ tạo ra giá trị hợp lệ của kiểu đó. Chẳng hạn, biểu thức có kiểu ‘con trỏ đến xâu’, thì giá trị của biểu thức đó được đảm bảo là con trỏ đến bộ nhớ được cấp phát cho xâu. Nó không thể là con trỏ ‘đu đưa’ đến vị trí được cấp để lưu một giá trị nào khác ngoài xâu.

Trước ML, các ngôn ngữ, có hệ thống kiểu được chứng minh, thường được xem là có những hạn chế không mong muốn. Nhiều lập trình viên C cho là quan trọng việc có thể ‘bẻ’ hệ thống kiểu theo cách khác nhau, chẳng hạn ‘lẫn lộn giữa số nguyên và con trỏ’. Những người yêu thích LISP thì đánh giá cao việc họ tự do với kiểu tĩnh. Tuy nhiên, hệ thống kiểu ML là không ép buộc, nhiều khai báo kiểu được chương trình dịch tự động suy luận, và hệ thống kiểu ML linh hoạt vì nó cho phép một biểu thức có thể có nhiều kiểu khác nhau.

Hầu hết các ngôn ngữ lập trình thành công đều thiết kế cho một ứng dụng hoặc một tập các nhiệm vụ lập trình liên quan chặt với nhau. Ngôn ngữ lập trình ML được thiết kế bởi Robert Milner và các cộng sự như một phần của dự án LCF. Dự án LCF hướng tới phát triển Logic cho các hàm tính toán, lấy cảm hứng từ tập các nguyên lý logic được phác thảo bởi Dana Scott. Mục đích của R.Milner là xây dựng hệ thống mà trên thực tế có thể chứng minh các tính chất thú vị của chương trình hàm một cách tự động hoặc bán tự động. Dự án LCF bắt đầu ở Stanford vào những năm 1970 và tiếp tục ở Edinburgh suốt những năm 1980, có những bước tiến quan trọng đến mục tiêu và làm động lực cho nhiều nỗ lực chung trong quá trình thực hiện.

ML được thiết kế như siêu ngôn ngữ Meta Language cho hệ thống LCF. Mục tiêu chính là để viết chương trình mà có thể xây dựng chứng minh toán học được. Như mọi bạn đọc, mà đã phát triển chứng minh toán học, đều biết đây là nhiệm vụ rất khó khăn. Trong một số trường hợp phải thử nhiều phương pháp để tìm được chứng minh. Một khái niệm chính của hệ thống LCF là chiến thuật chứng minh (proof tactic). Chiến thuật chứng minh là một hàm cho bởi công thức, cho trước giả thiết, thử tìm chứng minh của công thức. Vì chiến thuật có thể tìm độc lập hoặc đạt đến tình huống mà rõ ràng là không thể có việc tìm chứng minh tiếp theo được nữa, ở đây có ba khả năng áp dụng chiến thuật cho công thức.

$$\text{tactic}(\text{formula}) = \begin{cases} \text{succeed and return proof} \\ \text{search forever} \\ \text{fail} \end{cases}.$$

Chúng ta có thể sử dụng khái niệm chiến thuật để hiểu một số tính chất cơ bản của ngôn ngữ lập trình ML. Vì mục tiêu chính của hệ thống LCF là tìm các chứng minh đúng, một cơ chế của ngôn ngữ lập trình đảm bảo tính đúng đắn tổng thể hay bộ phận, có thể nâng cấp hệ thống LCF. Ý tưởng chính trong hệ thống LCF là tìm cách sử dụng hệ thống kiểu để tách chứng minh thành công khỏi các chứng minh không thành công.

Cụ thể, có chứng minh kiểu, với mục đích là giá trị của chứng minh kiểu là chứng minh đúng, chứ không phải chứng minh sai.

Một khi chúng ta có kiểu chứng minh đúng, sẽ nảy sinh vấn đề sau. Nếu chiến thuật tìm chứng minh thất bại, thì hàm sẽ là gì? Chính xác hơn, vì chiến thuật là hàm từ công thức vào chứng minh, nên kiểu của chiến thuật là kiểu hàm:

`tactic : formula → proof`

Tuy nhiên, kiểu này có vẻ cho biết nếu chiến thuật áp dụng vào hàm sẽ cho ra chứng minh. Nhưng nếu, công thức không là lệnh đúng, thì sẽ không cho chứng minh. Giải pháp được đưa ra là phát triển hệ thống xử lý lỗi (exception), mà cho phép chiến thuật tạo ra trường hợp ngoại lệ, nếu tính toán xác định không tìm được chứng minh. Xuất phát từ đó, Milner phát triển cơ chế kiểu ngoại lệ an toàn đầu tiên, mà thành tựu đạt được giúp ông giành giải thưởng Turing 1991. Cho phép ngoại lệ, hàm f mà ánh xạ từ A vào B được viết:

`f.A → B`

trong ML, có nghĩa là, với mọi x trong A , nếu $f(x)$ kết thúc bình thường mà không có ngoại lệ, thì $f(x)$ trong B .

Vậy đúng đắn kiểu và ngoại lệ là hai khái niệm cơ bản trong ML, như kết quả tự nhiên của mục đích ứng dụng ML.

Một điểm nhấn khác của ML là sử dụng các hàm bậc cao. Điều này cũng tạo thêm khó khăn trong việc xác định chiến thuật tìm chứng minh: vì chiến thuật là hàm, phương pháp kết hợp chiến thuật để tìm chiến thuật chứng minh là hàm từ các hàm vào hàm. Chẳng hạn, đây là phác thảo hàm mà kết hợp hai chiến thuật vào chiến thuật if-then-else:

```
f(tactic1, tactic2) =
  λformula. try tactic1(formula)
    else tactic2(formula)
```

Cho trước hai chiến thuật `tactic1` và `tactic2`, hàm f trả về chiến thuật mà cho trước công thức, trước hết thử chứng minh theo chiến thuật `tactic1` và nếu `tactic1` thất bại thì sử dụng `tactic2`.

2.4. Ngôn ngữ lập trình ML

Vì ML được sử dụng trong một số chương tiếp theo trong cuốn sách này để minh họa các tính chất của ngôn ngữ lập trình, chúng ta sẽ tìm hiểu ML kỹ càng hơn một số ngôn ngữ

lập trình khác. Phiên bản ML mà chúng ta sẽ sử dụng gọi là ML chuẩn Standard ML 97 (SML97). Chương trình dịch SML97 có trên Internet miễn phí. Có một số sách về ngôn ngữ này, trong đó cuốn của Ulman: Elements of ML Programming (Prentice-Hall, 1994) là trích dẫn tốt.

2.4.1. Các phần tương tác và Hệ thống thời gian chạy

Hầu hết các chương trình dịch ML dựa trên cùng một kiểu vòng lặp đọc-tính-in (read-eval-print). Cách chuẩn tương tác với hệ thống ML là nhập biểu thức và khai báo chúng cùng một lúc. Mỗi lần nhập, code nguồn được kiểm chứng kiểu, dịch và thực hiện. Khi một định danh được cho một giá trị bởi khai báo, thì định danh đó có thể được sử dụng trong các biểu thức tiếp theo.

Biểu thức

Đối với biểu thức, tương tác của người sử dụng với chương trình dịch có dạng sau:

```
-- <expression>;
val it = <print_value> : <type>
```

trong đó, ‘-’ là dấu nhắc người sử dụng nhập vào người sử dụng và dòng phía dưới là đầu ra của chương trình dịch ML và hệ thống thời gian chạy. Các dòng trên chỉ ra rằng, nếu bạn gõ vào một biểu thức, chương trình dịch sẽ dịch biểu thức và tính toán nó. Đầu ra hơi khó hiểu một chút: một định danh riêng sẽ gắn kết với giá trị của biểu thức cuối cùng nhập vào, vì vậy `it = <print_value> : <type>` có nghĩa là giá trị của biểu thức là `<print_value>` và kiểu của biểu thức là `<type>`. Để dễ hiểu hơn, ta xét một số ví dụ sau. Ở đây có bốn dòng đầu vào và đầu ra kết quả của chương trình dịch:

```
-- (5+3) -2;
val it = 6 : int
-- it + 3;
val it = 9 : int
-- if true then 1 else 5;
val it = 1 : int
-- (5 = 4);
val it = false : bool
```

Biểu diễn bằng lời: giá trị của biểu thức thứ nhất là số nguyên 6. Biểu thức thứ hai cộng 3 vào giá trị `it` của biểu thức trước đó, cho giá trị nguyên 9. Biểu thức thứ ba là if-then-else, mà tính giá trị là số nguyên 1, và biểu thức thứ tư là biểu thức Bool với giá trị False.

Mỗi biểu thức được phân tích cú pháp, kiểm tra kiểu và thực hiện trước khi đầu vào tiếp theo được đọc. Nếu biểu thức không được phân tích cú pháp đúng hoặc không qua được kiểm tra kiểu của chương trình dịch, không code nào được sinh và không code nào được thực hiện. Chẳng hạn, biểu thức kiểu lỗi

```
if true then 3 else false;
```

Phân tích đúng, vì có dạng đúng đối với if-then-else. Tuy nhiên kiểm tra kiểu từ chối biểu thức này vì kiểm tra kiểu ML đòi hỏi các phần then và else của biểu thức if-then-else phải

có cùng kiểu, như mô tả trong mục sau. Đầu ra của chương trình dịch cho biểu thức này gồm cả thông báo lỗi

```
stdin:1:1-29:10 Error: types of rules don't agree [literal]
```

chỉ rõ kiểu không phù hợp. Bàn luận đầy đủ về kiểm tra kiểu của ML sẽ nói đến trong chương sau.

Khai báo

Đầu vào của người sử dụng cần là biểu thức hoặc khai báo. Dạng chuẩn của khai báo ML, theo đầu ra của ML là

```
-- val <identifier> = <expression>;
val <identifier> = <print_value> : <type>
```

Từ khóa *val* dành cho giá trị. Khi khai báo truyền cho chương trình dịch, giá trị liên kết với định danh được tính toán và gán với định danh đó. Vì giá trị của biểu thức được sử dụng trong khai báo có tên, nên đầu ra chương trình dịch sử dụng tên này thay cho *it* để cho giá trị của biểu thức. Có một số ví dụ sau :

```
-- val x=7+2;
val x = 9 : int

-- val y = x+3;
val y = 12 : int
-- val z = x*y - (x div y);
val z = 108 : int
```

Bằng lời, khai báo thứ nhất gán giá trị nguyên 9 cho định danh *x*. Khai báo thứ hai tham chiếu đến giá trị của *x* trong khai báo thứ nhất và gán giá trị nguyên 12 cho định danh *y*. Khai báo thứ ba tham chiếu đến hai khai báo trước đó và gán giá trị nguyên 108 cho định danh *z*.

Bạn có thể nhận thấy rằng phép chia nguyên trong ML được viết là *div* thay cho */*. Ở đây có một số ký hiệu cú pháp riêng biệt của ML giống như vậy, đặc biệt khi liên quan đến số học số nguyên và số thực. Như bạn sẽ thấy từ việc bàn luận về kết thừa kiểu trong ML ở chương sau, sẽ có ích cho chương trình dịch phân biệt thao tác của các kiểu khác nhau.

Trong ML, */* được dùng cho chia số thực, và ở đây không có việc chuyển kiểu từ nguyên sang số thực. Như vậy trong ví dụ trên nếu chúng ta viết *x/y* thay cho *x div y*, thì *x/y* không thể là dạng cú pháp tốt.

Các hàm có thể được khai báo với từ khóa *fun* thay cho *val*. Dạng tổng quát của đầu vào người sử dụng và đầu ra của chương trình dịch là

```
-- fun <identifier> <arguments> = <expression>;
val <identifier> = fn <arg_type> -> <result_type>
```

Nó khai báo hàm có tên là identifier. Kiểu đối được xác định bởi dạng `<arguments>` và kiểu kết quả được xác định bởi dạng của `<expression>`.

Sau đây là ví dụ :

```
— fun f(x) = x + 5;
val f = fn : int → int
```

Khai báo này gán giá trị hàm với định danh f. Giá trị của f là một hàm số từ số nguyên vào số nguyên. Cũng hàm đó có thể khai báo bởi khai báo val như sau :

```
— val f = fn x => x+5;
val f = fn : int → int
```

Trong khai báo trên, định danh f được cho bởi giá trị của biểu thức `fn x => x + 5`. Chúng ta sẽ bàn về khai báo hàm và biểu thức hàm ở mục sau.

Hệ thống kiểu in ra `fn` cho giá trị của hàm f, vì giá trị của một hàm là không in được. Một lý do, vì sao giá trị của hàm là không in được, là hầu hết các hàm đều có tập giá trị là vô hạn, như hàm số nguyên có vô hạn các kết quả có thể có, mỗi một cho một giá trị nguyên của đối số. Sau khi hàm được khai báo, chương trình dịch sẽ lưu đoạn code dịch cho hàm. Nó có thể in đoạn code được dịch, nhưng nó không phải ở trong ML. Nó là ngôn ngữ đích mức thấp nào đó, mà in nó sẽ không có ích cho người lập trình.

Định danh và biến. Một đặc điểm quan trọng của ML là giá trị của định danh không thể thay đổi bởi lệnh gán. Đặc biệt, nếu định danh x được khai báo bởi `val x = 3`, chẳng hạn, thì giá trị của x luôn bằng 3. Không thể thay đổi giá trị của x bằng lệnh gán. Nói cách khác, khai báo ML đưa ra các hằng chứ không phải biến. Một cách khai báo các biến gán được trong ML là định nghĩa ô tham chiếu (reference cell). Tham chiếu và gán sẽ được giải thích về sau trong chương.

Mặc dù hầu hết bạn đọc ban đầu sẽ nghĩ khác, nhưng cách xử lý của ML với định danh và biến là nhất quán hơn so với cách xử lý định danh và biến trong các ngôn ngữ như Pascal và C. Nếu một định danh nguyên được khai báo trong C hoặc Pascal, nó được coi là biến có thể gán được. Mặt khác, nếu hàm được khai báo và cho tên trong các ngôn ngữ này, thì tên của hàm là hằng số, không phải biến. Không thể gán cho tên hàm và không thể thay đổi nó sang hàm khác. Vì vậy, Pascal và C lựa chọn giữa biến và hằng là tùy thuộc giá trị gán cho định danh. Trong ML, khai báo biến sẽ làm việc như nhau đối với mọi kiểu giá trị.

2.4.2. Các kiểu cơ bản

Các phần biểu thức, khai báo và lệnh cơ bản của ML được tóm tắt tốt nhất bởi danh sách các kiểu cơ bản với các biểu thức gán với từng kiểu.

Unit.

Đơn vị kiểu chỉ có một phần tử, được viết như ngược rỗng:

```
() : unit
```

Giống *void* trong C, *unit* được sử dụng như kiểu kết quả cho hàm mà được thực hiện chỉ vì các tác động phụ. Kiểu *unit* cũng được sử dụng như kiểu của đối số cho hàm số mà không có đối số. Những lập trình viên C có thể sẽ bị lẫn bởi sự việc *unit* cho là có một phần tử, trong khi đó *void* có vẻ như không có phần tử nào. Từ góc độ toán học, ‘một phần tử’ là đúng đắn. Cụ thể, nếu hàm được giả thiết trả về phần tử kiểu rỗng, thì hàm đó không thể trả về, vì tập rỗng (hoặc kiểu rỗng) không có phần tử nào. Mặt khác, hàm mà trả về một phần tử của kiểu một phần tử, có thể trả về. Tuy nhiên chúng ta không cần theo dõi hàm như vậy trả về giá trị nào, vì chỉ có một thứ mà nó có thể trả về. Hệ thống kiểu ML dựa trên nhiều năm nghiên cứu kiểu; hầu hết các khái niệm kiểu trong ML đã được xem xét rất cẩn thận.

Bool

Có hai giá trị kiểu Bool: *true* và *false*:

```
true : bool
false : bool
```

Biểu thức chung nhất liên kết với kiểu Bool là biểu thức có điều kiện, với

```
if e1 then e2 else e3
```

có cùng kiểu với *e2* và *e3*, nếu chúng có cùng kiểu và *e1* có kiểu Bool. Ở đây không có *if-then* mà không có *else*, cũng như biểu thức điều kiện cần phải có giá trị nếu phép kiểm tra điều kiện là đúng hoặc sai. Chẳng hạn, biểu thức

```
--- val nonsense = if a then 3;
```

là không hợp lệ trong ML, vì không có giá trị cho *nonsense* nếu biểu thức *a* là *false*. Cụ thể, nếu đầu vào *if a then 3* không được phân tích cú pháp đúng, thì không có cây phân tích cú pháp cho xâu này trong cú pháp của ML.

Ở đây cũng có các phép toán Bool cho *and*, *or*, *not*, ... Tương tự AND, OR và NOT trong Pascal hoặc *&&*, *?*, *!* trong C, với sự khác biệt nhỏ. Phủ định được viết là *not*, hội là *andalso* và tuyển là *orelse*.

Chẳng hạn, sau đây là hàm mà xác định hai biến Bool của nó có cùng giá trị Bool không, tiếp theo là biểu thức mà gọi hàm đó

```
--- fun equiv(x,y) = (x andalso y) orelse ((not x) andalso (not y));
val equiv = fn : bool * bool -> bool
--- equiv(true,false);
val it = false : bool
```

Ta nói các biến Bool *x* và *y* có cùng giá trị Bool, tức là chúng cùng *true* hoặc cùng *false*. Biểu thức con thứ nhất *(x andalso y)* là *true* nếu cả hai *x* và *y* cùng *true* và biểu thức con thứ hai *((not x) andalso (not y))* là *true* nếu cả hai cùng *false*.

Nguyên nhân tên dài *andalso* và *orelse* là để nhấn mạnh thứ tự tính toán. Trong biểu thức *(a andalso b)* trong đó *a* và *b* là hai biểu thức; *a* được tính trước, nếu *a* là *true*, thì *b* được

tính tiếp. Ngược lại giá của biểu thức $(a \text{ andalso } b)$ là *false* mà không cần tính b . Tương tự, b trong $(a \text{ orelse } b)$ chỉ được tính chỉ trong trường hợp a là *false*.

Integer

Nhiều biểu thức nguyên được theo cách thông thường, với hằng số số và các phép toán số học chuẩn:

```
0, 1, 2, ..., -1, -2, ... : int
+, -, *, div : int * int → int
```

Phép toán *div* là phép toán nhị phân trung tố, được sử dụng như sau:

```
--- fun quotient(x,y) = x div y;
val quotient = fn : int * int → int
```

Bản thân định danh *div* không phải là biểu thức. Tương tự, $+$, $-$, $*$ cũng là các phép toán nhị phân trung tố.

String

Các xâu được viết như dãy các ký hiệu được bao quanh dấu ngoặc kép:

```
"William Jefferson Clinton" : string
"Boris Yeltsin" : string
```

Nối xâu được viết dạng $^$, như vậy ta có

```
--- "Chelsey" ^ "" ^ "Clinton";

val it = "Chelsey Clinton" : string
```

Real

Kiểu của ML cho số dấu phẩy động là số thực. Để dễ dàng hiểu khi chúng ta suy luận kiểu, ML đòi hỏi phải có dấu chấm thập phân trong các hằng số thực:

```
1.0, 2.0, 3.14159, 4.44444, ... : real
```

Các phép toán số học $+$, $-$ và $*$ có thể áp dụng cho số nguyên hoặc số thực. Ở đây chúng ta có một vài biểu thức ví dụ và đầu ra kết quả của chương trình dịch:

```
--- 3+4;
val it = 7 : int
--- 4.0 + 5.1;
val it = 9.1 : real
```

Lưu ý rằng khi dấu $+$ có hai đối nguyên, thì kết quả là nguyên và khi $+$ có hai đối thực, thì kết quả là số thực. Tuy nhiên, sẽ là lỗi kiểu, nếu kết hợp đối nguyên với đối thực. Đây là một phần đầu ra của chương trình dịch cho biểu thức mà cộng số nguyên với số thực:

```

--- 4+5.1;
stdIn:1.1-1.6 Error: operator and operand don't agree [literal]
operator domain: int * int
operand: int * real

```

Thông báo lỗi báo cho chúng ta biết rằng, vì đối đầu tiên là nguyên, nên ký hiệu $+$ được xem là phép cộng nguyên. Do đó phép cộng có miền xác định là $int*int$ là ký hiệu của kiểu cặp số nguyên. Tuy nhiên, ở đây phép toán áp dụng cho cặp kiểu $int*real$ là ký hiệu của ML cho cặp kiểu một là nguyên và một là số thực.

Chuyển kiểu từ nguyên sang thực được thực hiện bởi hàm chuyển kiểu tường minh *real*. Chẳng hạn, giá trị của biểu thức *real(3)* là 3.0. Chuyển kiểu từ *real* sang số nguyên có thể được thực hiện bởi hàm *floor* (làm tròn xuống), *ceil* (làm tròn lên), và *trunc*.

Mặc dù các biểu thức số học trong ML là ít rắc rối hơn so với trong một số ngữ khác; ngôn ngữ này được sử dụng tổng quát cho nhiều mục đích. Cụ thể, gõ tường minh hằng số số và chuyển kiểu tường minh phải trả giá cho việc suy luận kiểu tự động, là đặc trưng có ích của ML được mô tả trong chương sau.

Tuples

Bộ có thể là cặp đôi, bộ ba, bộ bốn, v.v... Trong ML, bộ có thể ở dạng các kiểu giá trị bất kỳ. Giá trị của bộ được viết trong các dấu ngoặc và các kiểu của bộ được viết với dấu $*$. Chẳng hạn, đây là đầu ra của chương trình dịch cho bộ đôi, bộ ba và bộ bốn:

```

--- (3,4);
val it = (3,4) : int * int
--- (4,5,true);
val it = (4,5,true) : int * int * bool
--- ("Bob", "Carol", "Ted", "Alice");
val it = ("Bob","Carol","Ted","Alice") : string * string * string * string

```

Đối với mọi kiểu t_1 và t_2 , kiểu t_1*t_2 là kiểu cặp đôi mà thành phần thứ nhất có kiểu t_1 và thành phần thứ hai có kiểu t_2 . Kiểu $t_1*t_2*t_3$ là kiểu bộ ba, kiểu $t_1*t_2*t_3*t_4$ là kiểu bộ bốn.

Các thành phần của bộ được truy cập bởi các hàm mà nêu tên vị trí của thành phần mong muốn. Chẳng hạn, $\#1$ chọn thành phần thứ nhất của bộ, $\#2$ chọn thành phần thứ hai của bộ. Sau đây là một số ví dụ:

```

--- #2(3,4);
val it = 4 : int
--- #3("John", "Paul", "George", "Ringo");
val it = "George" : string

```

Bản ghi

Giống như bản ghi Pascal và cấu trúc C, bản ghi ML tương tự như bộ, nhưng với các thành phần có tên. Các giá trị và các kiểu của bản ghi được viết dạng ngoặc móc như sau:

```
-- { First_name = "Donald", Last_name = "Knuth" };
val it = {First_name="Donald,Last_name="Knuth"}
: {First_name:string, Last_name:string}
```

Biểu thức ở đây có hai thành phần, một được gọi là `First_name` và thành phần kia được gọi là `Last_name`. Kiểu của bản ghi cho ta biết kiểu của từng thành phần. Các thành phần của bản ghi được truy cập bởi hàm `#` giống như bộ, nhưng được gọi tên dựa theo tên của thành phần thay vì vị trí. Sau đây là ví dụ:

```
--- #First_name({First_name="Donald, Last_name="Knuth"});
val it = "Donald" : string
```

Cách khác chọn các thành phần của bộ và bản ghi là sánh mẫu mà được mô tả trong mục sau.

Lists

Danh sách của ML có độ dài tùy ý, nhưng mọi phần tử của danh sách phải có cùng kiểu. Chúng ta có thể viết danh sách bằng cách liệt kê các phần tử của nó, tách nhau bởi dấu phẩy, bên trong ngoặc vuông. Sau đây là một số ví dụ danh sách các kiểu khác nhau:

```
--- [1,2,3,4];
val it = [1,2,3,4] : int list
--- [true, false];
val it = [true,false] : bool list
--- ["red", "yellow", "blue"];
val it = ["red","yellow","blue"] : string list
--- [ fn x => x+1, fn x => x+2];
val it = [fn,fn] : (int -> int) list
```

Đối với danh sách ngắn, chương trình dịch in ra các phần tử của danh sách khi chỉ ra rằng biểu thức danh sách đã được tính toán. Đối với danh sách dài, các phần tử cuối được thay thế bởi ba dấu chấm. Như ví dụ danh sách cuối ở trên chỉ ra có thể viết danh sách các hàm.

Nói chung, `t` là kiểu của danh sách, nếu mọi phần tử của nó có kiểu `t`.

Danh sách rỗng được viết là `nil` trong ML. Phép ghép nối danh sách *list cons* là phép trung tố được viết như một cặp dấu hai chấm:

```
--- 3 :: nil;
val it = [3] : int list
--- 4 :: 5 :: it;
val it = [4,5,3] : int list
```

Trong danh sách đầu, 3 được ghép nối với danh sách rỗng `nil`. Kết quả là danh sách gồm phần tử duy nhất 3, trong biểu thức thứ hai, 4 được ghép với 5 và được ghép với danh sách trước đó. Trong cả hai trường hợp, kết quả là danh sách *int*.

2.4.3. Các mẫu, các khai báo và các biểu thức hàm

Dạng tổng quát của khai báo giá trị là gán giá trị với mẫu. Một mẫu là một biểu thức chứa các biến (như: $x, y, z, \dots$) and các hằng (như *true*, *false*, 1, 2, 3, ...), được kết hợp bởi một dạng nào đó như biểu thức bộ, bản ghi và dạng một phép toán được gọi là *constructor*. Dạng tổng quát của khai báo giá trị là

```
val <pattern> = <exp> ;
```

Trong đó, dạng chung của mẫu pattern được tóm tắt trong văn phạm sau:

```
<pattern> ::= <id> | <tuple> | <cons> | <record> | <constr>
<tuple> ::= (<pattern>, ..., <pattern>)
<cons> ::= <pattern>::<pattern>
<record> ::= {<id>=<pattern>, ..., <id>=<pattern>}
<constr> ::= <id>(<pattern>, ..., <pattern>)
```

Biểu diễn bằng lời, mẫu có thể là định danh, mẫu bộ, mẫu ghép danh sách, mẫu bản ghi hoặc mẫu một cấu trúc *constructor* kiểu dữ liệu đã khai báo. Một mẫu bộ là một dãy các mẫu nằm trong các ngoặc, một mẫu ghép nối danh sách là hai mẫu tách nhau bởi cặp hai dấu hai chấm, mẫu bản ghi là biểu thức dạng bản ghi với mỗi trường có dạng mẫu, và mẫu *constructor* là một định danh (một *constructor* được khai báo) áp dụng vào bên phải nó một số các đối là mẫu. Dạng chuẩn BNF trên không định nghĩa tập các mẫu trên một cách chính xác, cũng như mọi điều kiện trên mẫu không phải phi ngữ cảnh và do đó không thể biểu diễn được bởi BNF. Chẳng hạn, điều kiện mà trên mẫu *constructor* định danh cần được khai báo *constructor* và *constructor* đó cần được áp dụng về phía phải nó một số đối số mẫu không phải là điều kiện phi ngữ cảnh. Một điều kiện bổ sung trên mẫu, mà sẽ được bàn đến cùng với khai báo hàm, là không có biến nào được xuất hiện hai lần trong mẫu.

Vì biến là mẫu, một khai báo giá trị có thể chỉ là gán giá trị đó với biến. Chẳng hạn, sau đây là khai báo mà gán bộ đến một định danh, tiếp theo là khai báo mà sử dụng mẫu bộ để gán các thành phần của bộ đó.

```
-- val t = (1,2,3);
val t = (1,2,3) : int * int * int
-- val (x,y,z) = t;
val x = 1 : int
val y = 2 : int
val z = 3 : int
```

Lưu ý rằng ở đây có hai dòng đầu vào và bốn dòng đầu ra của chương trình dịch trong ví dụ trên. Trong khai báo đầu, định danh t được gán với một bộ. Trong khai báo sau, một mẫu bộ (x,y,z) được cho giá trị là t . Khi mẫu (x,y,z) sánh với bộ ba t , định danh x nhận giá trị 1, định danh y nhận giá trị 2 và định danh z nhận giá trị 3.

Khai báo hàm

Dạng tổng quát của khai báo hàm sử dụng các mẫu. Định nghĩa mệnh đề đơn có dạng

```
fun f( <pattern> ) = <exp>
```

Và định nghĩa mệnh đề bội có dạng

```
fun f( <pattern1> ) = <exp1> | . . . | f( <patternn> ) = <expn>
```

Chẳng hạn, hàm cộng các đối số của nó có thể viết như sau

```
fun f(x,y) = x+y;
```

Về mặt kỹ thuật, tham số hình thức của hàm này là mẫu (x, y) mà cần phải sánh với tham số thực tế khi gọi hàm f . Tham số hình thức của f là một bộ, mà được tách bởi sánh mẫu thành thành phần thứ nhất và thành phần thứ hai. Bạn có thể nghĩ là bạn đang gọi hàm có hai đối số. Trên thực tế, bạn đang gọi hàm có một đối số. Đối số đó là một cặp giá trị. Phép sánh mẫu sẽ tách bộ, gắn x vào cái mà bạn nghĩ là tham số thứ nhất và y vào tham số thứ hai.

Ví dụ mà ở đó có nhiều hơn một mệnh đề được sử dụng là hàm sau, mà tính độ dài của danh sách:

```
--- fun length(nil) = 0
    | length( x :: xs ) = 1 + length(xs);
val length = fn : 'a list → int
```

Đoạn mã này được giải thích như sau. Hai dòng đầu là đầu vào (khai báo hàm độ dài) và dòng sau là đầu ra của chương trình dịch cho kiểu của hàm này. Đây là ví dụ tính độ dài danh sách và cho ra kết quả:

```
--- length ["a", "b", "c", "d"];
val it = 4 : int
```

Khi hàm độ dài được áp dụng vào đối số, các mệnh đề sẽ sánh theo thứ tự mà chúng được viết. Nếu đối số sánh với hằng *nil* (tức là đối số là danh sách rỗng) thì hàm trả về giá trị 0, như đã đặc tả trong mệnh đề đầu. Ngược lại tham số sánh với mẫu đã cho trong mệnh đề thứ hai $(x::xs)$; và khi đó đoạn code của nhánh thứ hai sẽ được thực hiện. Vì kiểm tra kiểu đảm bảo rằng độ dài sẽ được áp dụng chỉ cho danh sách, nên hai trường hợp này sẽ bao quát mọi giá trị có thể có mà được truyền cho hàm này. Kiểu của hàm độ dài là *'list -> int'* sẽ được giải thích trong chương sau.

Thêm vào đó, ML có cú pháp để khai báo hàm vô danh. Chúng ta đã nhìn thấy một số ví dụ đơn giản. Dạng tổng quát cho phép đối số được cho qua mẫu:

```
fn <pattern> => <exp> ,
```

Sau đây là ví dụ mà có đầu ra của chương trình dịch.

```
--- fn (x,y) => x+y;

val it = fn : int * int → int
```

Hàm biểu diễn ở đây nhận cặp số và cộng hai thành phần của nó. Kiểu của hàm này là $int * int \rightarrow int$, nghĩa là hàm ánh xạ cặp số nguyên vào một số nguyên.

Sau đây đưa ra thêm một số ví dụ nữa mô tả các dạng khác của mẫu, mỗi cái được gắn kết với đầu ra của chương trình dịch:

```
--- fun f(x, (y,z)) = y;
val f = fn : 'a * ('b * 'c) -> 'b
--- fun g(x::y::z) = x::z;
val g = fn : 'a list -> 'a list
--- fun h {a=x, b=y, c=z} = {d=y, e=z};
val h = fn : {a:'a, b:'b, c:'c} -> {d:'b, e:'c}
```

Đầu tiên là hàm trên bộ lồng nhau, thứ hai là hàm trên danh sách mà có ít nhất hai phần tử, thứ ba là hàm trên bản ghi. Khai báo thứ hai tạo nên cảnh báo của chương trình dịch, như hàm g không được định nghĩa cho danh sách mà có ít hơn hai phần tử.

Sánh mẫu được áp dụng theo thứ tự. Chẳng hạn, khi hàm số

```
fun f (x,0) = x
| f (0,y) = y
| f (x,y) = x+y;
```

được áp dụng vào đối số (a, b) , mệnh đề thứ nhất được sử dụng nếu $b = 0$, mệnh đề thứ hai được dùng nếu $b \neq 0$ và $a = 0$, và mệnh đề thứ ba được dùng khi $b \neq 0$ và $a \neq 0$. Hệ thống kiểu của ML sẽ giữ không cho f áp dụng cho đối số mà không phải là cặp (a, b) .

Điều kiện quan trọng trên mẫu là không có biến nào được xuất hiện hai lần trên mẫu. Chẳng hạn, khai báo hàm sau đây là không đúng về mặt cú pháp, vì biến x hai lần xuất hiện trong mẫu:

```
--- fun eq(x,x) = true
    | eq(x,y) = false;
stdIn:24.5-25.20 Error: duplicate variable in pattern(s):
```

Hàm này không cho phép vì xuất hiện lặp của biến thể hiện sự bằng nhau, và sự bằng nhau cần phải được viết tường minh vào thân của hàm.

2.4.4. Khai báo kiểu dữ liệu trong ML

Khai báo kiểu dữ liệu ML là dạng chuyên biệt của khai báo kiểu mà khai báo tên kiểu và thao tác để xây dựng và sử dụng các thành phần của kiểu. Khai báo kiểu dữ liệu ML có dạng cú pháp sau:

```
datatype <type_name> = <constructor_clause> | ... | <constructor_clause>
```

trong đó mệnh đề tạo *constructor clause* có dạng:

```
<constructor_clause> ::= <constructor> | <constructor> of <arg_types>
```

Ý tưởng là mỗi mệnh đề tạo cho biết một cách để xây dựng các phần tử của kiểu. Các phần tử của kiểu có thể được đúc kết thành các thành phần hợp thành của chúng bằng việc sánh mẫu. Điều này được minh họa bằng ba ví dụ sau mà chỉ ra cách chung sử dụng khai báo kiểu dữ liệu trong chương trình ML.

Ví dụ. Kiểu dữ liệu đếm được.

Các kiểu mà chỉ gồm hữu hạn các phần tử có thể khai báo như các kiểu của ML. Sau đây là kiểu gồm ba phần tử được đặt tên cho bộ ba, mà cụ thể:

```
--- datatype color = Red | Blue | Green;
    datatype color = Blue | Green | Red
```

Đầu ra của chương trình dịch, mà trông giống như đầu vào, chỉ rõ ba phần tử của kiểu color là Blue, Green và Red. Về mặt kỹ thuật, các giá trị Blue, Green, Red được gọi là các phần tử tạo constructor. Chúng được gọi như vậy, vì đó là cách xây dựng các giá trị với kiểu color.

Ví dụ. Kiểu dữ liệu hợp (Tagged Union).

Các phần tử kiểu của ML (*constructors*) có thể được khai báo sao cho chúng cần được áp dụng cho các đối số khi xây dựng các phần tử của kiểu đó. Các phần tử của kiểu không làm điều gì cho các đối số của chúng, ngoài việc cấp thẻ cho các đối số của chúng sao cho các giá trị được xây dựng bằng các cách khác nhau có thể được phân biệt bởi việc sánh mẫu.

Giả sử chúng ta lưu các bản ghi sinh viên, với tên của sinh viên đại học; tên và trường đã tốt nghiệp của sinh viên cao học; tên và khoa hướng dẫn của nghiên cứu sinh. Sau đó chúng ta muốn định nghĩa kiểu sinh viên mà cho phép ba dạng của các bộ như sau:

```
--- datatype student = BS of name | MS of name*school | PhD of name*faculty;
```

Trong khai báo dữ liệu trên, sinh viên đại học, cao học và nghiên cứu sinh là các phần tử tạo. Tuy nhiên, không giống như ví dụ color, mỗi phần tử tạo sinh viên cần áp dụng vào đối số của nó để tạo giá trị của kiểu sinh viên. Chúng ta cần áp dụng vào sinh viên đại học để xác định tên, sinh viên cao học cho cặp tên và trường, nghiên cứu sinh cho cặp tên và khoa để tạo ra kiểu sinh viên.

Kiểu sinh viên thực chất là hợp của ba kiểu.

```
student ≈ union {name, name*school, name*faculty }
```

ngoại trừ rằng trong “hợp” của ML (mà được định nghĩa bằng kiểu dữ liệu), mỗi giá trị của hợp được cấp thẻ bởi phần tử tạo mà cho biết từ kiểu thành phần nào giá trị đó được lấy. Điều này được minh họa trong hàm sau mà trả về tên của sinh viên:

```
--- fun name(BS(n)) = n
    | name(MS(n,s)) = n
    | name(PhD(n,f)) = n;
    val name = fn : student → name
```

Ba dòng đầu là khai báo của hàm tên `name`, và dòng cuối là đầu ra của chương trình dịch chỉ ra rằng `name` là hàm từ sinh viên vào tên. Hàm có ba mệnh đề, mỗi một cho một dạng của sinh viên.

Ví dụ. Kiểu dữ liệu đệ qui.

Khai báo kiểu dữ liệu có thể đệ qui ở đó tên của kiểu có thể xuất hiện trong một hoặc nhiều hơn các kiểu của đối số thành phần. Vì theo cách đó đệ qui kiểu được cài đặt. Kiểu dữ liệu ML cung cấp một cấu trúc ngôn ngữ bậc cao thuận tiện mà dấu dạng chung của thao tác con trỏ thông thường.

Tập các cây với nhãn nguyên tại các lá có thể được định nghĩa toán học như sau:

Một cây hoặc là

- lá liên kết với nhãn nguyên, hoặc là
- cây bao gồm cây con bên trái và cây con bên phải.

Định nghĩa trên được thể hiện trong khai báo kiểu của ML, với mỗi phần của định nghĩa tương ứng với một mệnh đề của khai báo kiểu dữ liệu:

```
datatype tree = LEAF of int | NODE of (tree * tree);
```

Các định danh LEAF và NODE là các cấu trúc tạo; và các phân tử của kiểu dữ liệu này là các giá trị mà có thể được tạo ra bằng việc áp dụng các cấu trúc tạo cho các đối số có kiểu hợp lệ. Diễn tả bằng lời, cây hoặc là kết quả của việc áp dụng cấu trúc LEAF cho một số nguyên (đánh dấu lá với nhãn nguyên) hoặc kết quả áp dụng cấu trúc NODE cho hai cây. Hai cây này, tất nhiên, cần phải được tạo tương tự với các cấu trúc LEAF hoặc NODE.

Hàm sau đây chỉ rõ các cấu trúc tạo được sử dụng như thế nào để định nghĩa hàm trên cây:

```
--- fun inTree(x, LEAF(y)) = x = y
    | inTree(x, NODE(y,z)) = inTree(x, y) orelse inTree(x, z);
val inTree = fn : int * tree -> bool
```

Hàm này tìm một giá trị nguyên cho trước x trên cây. Nếu cây có dạng $LEAF(y)$, thì x trên cây khi và chỉ khi $x = y$. Nếu cây có dạng $NODE(y, z)$, với các cây con là y và z , khi đó x trên cây khi và chỉ khi x trên cây trái y hoặc x trên cây phải z . Đầu ra kiểu của chương trình dịch chỉ rõ rằng `inTree` là hàm mà cho trước một số nguyên và một cây, hàm trả về giá trị Boole.

2.4.5. Các ô tham chiếu và gán ML

Không cấu trúc nào của ML mà được bàn trong các phần trước của chương này có tác động phụ. Mỗi biểu thức có giá trị, nhưng quá trình tính giá trị biểu thức không có tác động phụ làm thay đổi giá trị của bất cứ biểu thức nào. Hầu hết các chương trình ML lớn đều được viết theo phong cách mà tránh các tác động phụ mỗi khi có thể, hầu hết các chương trình ML lớn chỉ thỉnh thoảng sử dụng phép gán để thay đổi giá trị của biến.

Cách, mà các biến gán được thể hiện trong ML, là khác với các cách mà biến gán được xuất hiện trong các ngôn ngữ lập trình khác. Lý do chính là để giữ dạng đồng nhất của ML như ngôn ngữ lập trình và tách các tác động phụ ra khỏi các biểu thức thuần túy càng nhiều càng tốt.

Phép gán ML được hạn chế về các ô tham chiếu. Trong ML, một ô tham chiếu có kiểu khác với các giá trị không thay đổi được như số nguyên, xâu, danh sách, v.v... Vì các ô tham chiếu có kiểu tham chiếu chuyên biệt, nên các hạn chế lên phép gán ML là bắt buộc như một phần của Hệ thống kiểu. Đây là phần có tính thanh nhã của ML: hầu hết mọi hạn chế lên cấu trúc chương trình là một phần của Hệ thống kiểu, và Hệ thống kiểu có định nghĩa nhất quán và có tính hệ thống.

L-values và R-values

Trước khi xem xét phép gán trong ML, chúng ta nghĩ về sự khác nhau giữa vị trí ô nhớ và nội dung của nó. Sự khác nhau này là một phần của kiến trúc máy tính (vị trí ô nhớ chứa dữ liệu) và phù hợp với nhiều ngôn ngữ lập trình. Đoạn giả code sau đây sẽ mô tả ý tưởng đó:

```
x : int;
y : int;
x := y + 3;
```

Trong phép gán trên, giá trị được lưu trong biến y được cộng với 3 và kết quả lưu ở vị trí nhớ dành cho x. Mấu chốt ở đây là hai biến được sử dụng khác nhau. Lệnh chỉ sử dụng giá trị lưu ở y và không phụ thuộc vào vị trí của y. Ngược lại, lệnh trên chỉ sử dụng vị trí của x mà không phụ thuộc vào giá trị lưu trong x trước khi phép gán xảy ra.

Vị trí của một biến được gọi là *L-value*, và giá trị lưu ở vị trí đó được gọi là *R-value* của biến. Đây là thuật ngữ chuẩn mà bạn sẽ nhìn thấy trong nhiều cuốn sách về ngôn ngữ lập trình. Hai giá trị được gọi là L và R thay cho từ Left và Right, như thông thường ta sử dụng *L-values* ở vế trái của lệnh gán và *R-values* ở vế phải.

Ô tham chiếu ML

Trong ML, L-values và R-values có các kiểu khác nhau. Nói cách khác, vùng gán được của bộ nhớ có kiểu khác với giá trị, mà không thể thay đổi được. Trong ML, L-value hoặc vùng gán được của bộ nhớ, được gọi là *ô tham chiếu* (*reference cell*). Kiểu của ô tham chiếu chỉ ra rằng, đây là ô tham chiếu và chỉ rõ kiểu giá trị mà nó chứa. Chẳng hạn, ô tham chiếu mà chứa số nguyên thì có kiểu *int ref*, có nghĩa là ô tham chiếu nguyên.

Khi ô tham chiếu được tạo, nó cần được khởi tạo giá trị có kiểu đúng. Bởi vậy ML không có các biến không khởi tạo hoặc các con trỏ treo. Khi phép gán thay đổi giá trị trong ô nhớ, phép gán cần phải tương thích với kiểu của ô tham chiếu: ô tham chiếu nguyên sẽ luôn chứa số nguyên, ô tham chiếu danh sách sẽ luôn chứa danh sách, v.v...

Thao tác trên ô tham chiếu

ML có các thao tác để tạo ô tham chiếu, truy cập nội dung của nó và thay đổi nội dung của nó. Đó là *ref*, *!* và *:=*, mà có dạng sau:

`ref v` — creates a reference cell containing value `v`
`! r` — returns the value contained in reference cell `r`
`r := v` — places value `v` in reference cell `r`

Sau đây là các ví dụ:

```

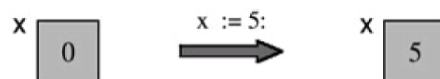
--- val x = ref 0;
val x = ref 0 : int ref
--- x := 3*(!x) + 5;

val it = () : unit
--- !x;
val it = 5 : int

```

Dòng nhập đầu tiên gán biến `x` với ô tham chiếu mới có nội dung 0. Đầu ra chương trình dịch chỉ rõ: giá trị của `x` là ô tham chiếu mà có kiểu `int ref`. Dòng đầu vào tiếp theo nhân 3 với nội dung của `x`, rồi cộng 5, và lưu giá trị kết quả nguyên vào ô `x`. Vì ML là ngôn ngữ hướng biểu thức, “lệnh” này là một biểu thức ML. Kiểu của biểu thức này là `unit`. Mà như mô tả trước đây, là kiểu được sử dụng cho biểu thức mà tính toán có tác động phụ. Dòng đầu vào cuối là biểu thức đọc nội dung của `x` mà là số nguyên 5.

Vì ML không có thao tác nào tính toán địa chỉ của giá trị, nên không có cách nào quan sát đây là phép gán bởi giá trị hay bởi con trỏ. Kết quả là rất thuận tiện coi ô tham chiếu như cái hộp giữ giá trị kích thước tùy ý và coi phép gán như thao tác đặt giá trị vào trong cái hộp. Ví dụ đoạn code trên tạo ô tham chiếu tên `x` và thay đổi nội dung của nó như hình vẽ sau, mũi tên kép chỉ rõ sự thay đổi như kết quả của phép gán:



Vì các ô tham chiếu có thể được tạo cho kiểu giá trị bất kỳ, ta có thể định nghĩa ô tham chiếu xâu và thay đổi nội dung của nó bằng lệnh gán:

```

--- val y = ref "Apple";
val y = ref "Apple" : string ref
--- y := "Fried green tomatoes";
val it = () : unit
--- !y;
val it = "Fried green tomatoes" : string

```

Như trong ví dụ trước, ô tham chiếu ở đây có thể biểu diễn như hộp chứa xâu:



Như bạn đã biết, các xâu khác nhau có thể đòi hỏi độ lớn bộ nhớ khác nhau. Do đó, có vẻ như là ô nhớ gắn với `y` có thể giữ xâu có độ dài bất kỳ.

Trên thực tế, khi khai báo $y = \text{ref } \text{"Apple"}$ được xử lý, bộ nhớ được cấp dành cho xâu "Apple" và ô tham chiếu y được khởi tạo để trỏ đến vị trí nhớ đó. Khi phép gán " $y := \text{Fried green tomatoes}$ " được xử lý, nội dung của ô y được thay đổi để trỏ đến "Fried green tomatoes". So sánh hai ví dụ số nguyên và xâu, ta thấy phép gán ML được cài đặt như phép gán giá trị thông thường cho một số kiểu nào đó của ô và phép gán con trỏ cho các kiểu khác. Tuy nhiên, vì ML không có cách nào khác tìm địa chỉ của biểu thức, sự khác biệt cài đặt này được hoàn toàn được che giấu không để người lập trình bận tâm. Nếu người viết chương trình dịch muốn cài đặt phép gán nguyên như phép gán con trỏ, mọi chương trình vẫn có kết quả như nhau.

Sau đây là ví dụ code đơn giản sau cùng chỉ ra rằng ô tham chiếu ML được sử dụng trong vòng lặp như thế nào. Vòng lặp này tính tổng các số từ 1 đến 10.

```
val i = ref 0;
val j = ref 0;
while !i < 10 do (i := !i + 1; j := !j + !i);
!j;
```

Trong hai dòng đầu, hai biến i và j gắn kết với các ô tham chiếu khởi tạo giá trị 0. Vòng lặp while tăng i cho đến khi nội dung của i là $!i$ không nhỏ hơn 10. Biểu thức cuối cho biết giá trị cuối cùng của j , vì chương trình dịch sẽ in ra giá trị của $!j$. Một số chi tiết quan trọng như kiểm tra $i < 10$ là không hợp lệ, vì nó so sánh ô tham chiếu với số nguyên. Tương tự $i = i + 1$ cũng không hợp lệ, vì ô tham chiếu không thể cộng thêm 1, chỉ có số nguyên và số thực mới có thể cộng được.

Như thể hiện trong ví dụ trên, hai biểu thức thực hiện có thể được kết hợp với nhau bởi dấu chấm phẩy ;. Các dấu ngoặc đơn được sử dụng để giữ độ ưu tiên của thân vòng lặp while.

Có thể tải để cài đặt ML for Windows từ trang Web

<http://www.smlnj.org/install/index.html>

Bài tập.

1. Xác định xem các hàm sau làm gì. Áp dụng các ví dụ cụ thể cho từng hàm và giải thích cách thực thi chương trình để nhận được các kết quả đó.

```
fun length nil = 0
| length (_,t) = 1 + length t;
```

```
fun append (nil, l) = l
| append (h::t, l) = h :: append (t, l);
```

```
fun insert(less,x,nil) = [x]
| insert(less,x,y::ys) = if less(x,y) then x::y::ys
else y::insert(less,x,ys);
```

```
fun sort(less,nil) = nil
| sort(less,x::xs) = insert(less,x,sort(less,xs));
```

2. Tìm ước chung lớn nhất

```
fun gcd (m:int, 0):int = m
| gcd (0, n:int):int = n
```

```

| gcd (m:int, n:int):int =
  if m>n then gcd (m mod n, n) else gcd (m, n mod m) ;

fun ggcd (0, n) = (n, 0, 1)
| ggcd (m, 0) = (m, 1, 0)
| ggcd (m, n) =
  if m>n then
    let
      val (d, a, b) = ggcd (m mod n, n)
    in
      (d, a, b - a*(m div n))
    end
  else
    let
      val (d, a, b) = ggcd (m, n mod m)
    in
      (d, a - b*(n div m), b)
    end
  end

```

3. Tính nhanh lũy thừa

```

fun square (x) = x * x;
fun mult (a, x) = a * x;

fun fast_exp (a, 0) = 1
| fast_exp (a, n) =
  if n mod 2 = 0 then
    square (fast_exp (a, n div 2))
  else
    mult (a, fast_exp (a, n-1)) ;

fun generalized_iterative_fast_exp (b, 0, a) = a
| generalized_iterative_fast_exp (b, n, a) =
  if n mod 2 = 0 then
    generalized_iterative_fast_exp (b*b, n div 2, a)
  else
    generalized_iterative_fast_exp (b, n-1, b*a) ;

fun generalized_iterative_fast_exp_mod (b, 0, a, m) = (a mod m)
| generalized_iterative_fast_exp_mod (b, n, a, m) =
  if n mod 2 = 0 then
    generalized_iterative_fast_exp_mod (((b*b) mod m), n div 2, a, m)

```

else

generalized_iterative_fast_exp_mod (b, n-1, ((b*a) mod m), m) ;

4. Chương trình sau cài đặt thuật toán Quicksort cho dãy số nguyên. Hãy áp dụng cho một ví dụ cụ thể và giải thích cách thực thi chương trình:

```
fun partition (pivot, nil) = (nil,nil)
| partition (pivot, first :: others) =
let
val (smalls, bigs) =
partition(pivot, others)
in
if first < pivot
then (first::smalls, bigs)
else (smalls, first::bigs)
end
```

```
fun qsort nil = nil
| qsort [singleton] = [singleton]
| qsort (first::rest) =
let
val (smalls, bigs) =
partition(first,rest)
in
qsort(smalls) @ [first] @ qsort(bigs)
end
```

```
fun qsort nil = nil
| qsort [singleton] = [singleton]
| qsort (first::rest) =
let
val (smalls, bigs) =
partition(first,rest)
in
append(append(qsort(smalls),[first]),qsort(bigs))
end
```

5. Chương trình sau cài đặt map-reduce. Áp dụng các hàm đó tính tổng và tích nghịch đảo các phần tử của trong dãy số [1, 2, ..., 10]

```
fun map(f, nil) = nil |
map(f,x::xs) = f(x)::(map(f,xs));
```

```
fun reduce (unit, opn, nil) = unit
| reduce (unit, opn, h::t) = opn (h, reduce (unit, opn, t))
```

```
fun add_up l = reduce (0, op +, l)
```

```
fun mul_up l = reduce (1, op *, l)
```

```
fun cub(x) = x*x*x;
```

```

val l = [1,2,3,4];
map(cub,l);
fun not(x) = if x = true then false else true;
val b = [true, true, false];
map(not,b);

```

6. Chương trình sau xây dựng cây tìm kiếm nhị phân. Viết ứng dụng Sổ tay tra cứu điện thoại của các bạn trong lớp

```

datatype 'a tree = Niltree |
Maketree of 'a * ('a tree) * ('a tree);

fun insert (new:int) Niltree =
Maketree(new,Niltree,Niltree)
| insert new (Maketree (root,l,r)) =
if new < root
then Maketree (root,(insert new l),r)
else Maketree (root,l,(insert new r))

fun buildtree [] = Niltree
| buildtree (fst :: rest) =
insert fst (buildtree rest)

fun find (elt:int) Niltree = false
| find elt (Maketree (root,left,right)) =
if elt = root
then true
else if elt < root
then find elt left
else find elt right

```

CHƯƠNG 3: HỆ THỐNG KIỂU VÀ SUY DIỄN KIỂU

Lập trình sử dụng phạm vi rất rộng các cấu trúc tính toán, như các cấu trúc dữ liệu, các hàm, các đối tượng, các kênh truyền và các luồng điều khiển. Vì ngôn ngữ lập trình được thiết kế giúp các lập trình viên tổ chức các cấu trúc tính toán và sử dụng chúng một cách đúng đắn. Nhiều ngôn ngữ lập trình tổ chức dữ liệu và cấu trúc tính toán thành các kiểu. Trong chương này, chúng ta sẽ xem xét các lý do sử dụng kiểu trong ngôn ngữ lập trình và các phương pháp kiểm tra kiểu, và một số vấn đề về kiểu như đa hình, ghi đè, và bằng nhau kiểu. Một mục lớn của chương dành cho suy diễn kiểu, quá trình xác định kiểu các biểu thức dựa trên các kiểu đã biết của các ký hiệu xuất hiện trong chúng. Suy diễn kiểu là tổng quát của kiểm tra kiểu, với rất nhiều điểm giống nhau, và các ví dụ tiêu biểu về các thuật toán kiểu này được sử dụng trong chương trình dịch và môi trường lập trình để xác định các tính chất của chương trình. Suy diễn kiểu cũng đưa ra khái niệm đa hình, mà cho phép một biểu thức có nhiều kiểu.

3.1 Kiểu trong lập trình

Về tổng quan kiểu là tập hợp các thực thể tính toán mà chia sẻ một tính chất chung nào đó. Một số ví dụ về kiểu là kiểu số nguyên, kiểu `int` -> `int` của các hàm từ số nguyên vào số nguyên và kiểu phạm vi của Pascal `[1, ..., 100]` của các số từ 1 đến 100. Trong ML tương tranh có kiểu `int channel` của các kênh truyền mang giá trị nguyên và trong Java có kiến trúc kiểu lỗi (exceptions).

Có ba kiểu chính sử dụng kiểu trong ngôn ngữ lập trình:

- Đặt tên và tổ chức các khái niệm
- Tin tưởng rằng dãy bit trong bộ nhớ máy tính được thông dịch đúng
- Cung cấp thông tin cho chương trình dịch về dữ liệu được thao tác bởi chương trình.

Các ý tưởng này sẽ được bàn đến trong các mục sau.

Mặc dù mô tả của một số ngôn ngữ lập trình có nói như “Lisp là ngôn ngữ không kiểu (untyped), trên thực tế không có ngôn ngữ lập trình nào không kiểu. Trong Lisp, chẳng hạn, danh sách lists và atoms là hai kiểu khác nhau: thao tác danh sách có thể áp dụng cho danh sách chứ không thể áp dụng cho atoms. Các ngôn ngữ lập trình xử lý rất khác nhau trong việc các kiểu được sử dụng như thế nào trong cú pháp và ngữ nghĩa (cài đặt) của ngôn ngữ đó.

3.1.1 Tổ chức chương trình và viết tài liệu.

Chương trình được thiết kế tốt sử dụng các khái niệm liên quan đến bài toán được giải. Chẳng hạn, chương trình banking sẽ được tổ chức xoay quanh các khái niệm của ngân hàng như: tài khoản, khách hàng, số dư, rút tiền và chuyển tiền. Trong các ngôn ngữ lập trình

hiện đại, khách hàng và tài khoản, chẳng hạn, được thể hiện bằng các kiểu khác nhau. Kiểm tra kiểu sau đó có thể kiểm tra để tin tưởng là tài khoản và khách hàng được xử lý tách biệt nhau, các thao tác tài khoản chỉ có thể áp dụng cho tài khoản chứ không được sử dụng cho khách hàng. Sử dụng kiểu để tổ chức chương trình làm cho nó dễ đọc, dễ hiểu và dễ bảo trì chương trình. Do đó các kiểu cũng đóng vai trò quan trọng trong việc viết tài liệu thiết kế và nêu mục đích chương trình.

Một ưu điểm của thông tin kiểu so với việc ghi chú (comments) được viết bởi lập trình viên, là các kiểu có thể được kiểm tra bằng chương trình dịch. Kiểm tra kiểu đảm bảo rằng các kiểu được viết trong chương trình là đúng đắn. Ngược lại, rất nhiều chương trình chứa các ghi chú không đúng, hoặc vì người viết giải thích không cẩn thận hoặc vì chương trình về sau đã thay đổi mà ghi chú thì lại chưa thay.

3.1.2 Lỗi kiểu

Lỗi kiểu xảy ra khi thực thể tính toán như hàm hoặc giá trị dữ liệu được sử dụng trong đó, mà không tương thích với khái niệm nó biểu diễn. Chẳng hạn, nếu giá trị nguyên được sử dụng như hàm, thì đây là lỗi kiểu. Lỗi kiểu chung là áp dụng phép toán cho toán hạng sai kiểu. Chẳng hạn, sẽ là lỗi kiểu nếu sử dụng phép cộng số nguyên để cộng vào số nguyên. Mặc dù hầu hết các lập trình viên hiểu tổng quan về kiểu lỗi, ở đây vẫn có một số tinh tế rất đáng phân tích thêm.

Lỗi phần cứng. Loại lỗi kiểu dễ hiểu nhất là các chỉ lệnh máy mà gây lỗi phần cứng. Chẳng hạn, việc thực hiện lời gọi hàm:

`x()`

là lỗi kiểu, nếu `x` không phải là hàm. Nếu `x` là biến nguyên với giá trị 256 chẳng hạn, thì việc thực hiện `x()` sẽ làm cho máy nhảy đến vị trí 256 và bắt đầu thực hiện chỉ lệnh lưu ở vị trí này trong bộ nhớ. Nếu vị trí 256 chứa dữ liệu không phải là lệnh máy đúng, thì nó sẽ gây ra ngắt phần cứng. Ví dụ khác về lỗi phần cứng xảy ra khi thực hiện phép toán:

`float_add(3, 4.5)`

ở đó đơn vị dấu phẩy động phần cứng được gọi cho biến nguyên 3. Vì mẫu bit sử dụng cho 3 không biểu diễn số dấu phẩy động theo dạng mà phần cứng dành cho dấu phẩy động mong đợi, nên chỉ lệnh này sẽ gây ra ngắt phần cứng.

Ngữ nghĩa không mong muốn. Một số lỗi kiểu không gây ra lỗi hay ngắt phần cứng vì đoạn mã dịch không chứa thông tin giống như đoạn code chương trình nguồn. Chẳng hạn, phép toán

`int_add(3, 4.5)`

là lỗi kiểu, vì ở đây `int_add` là phép cộng nguyên và được áp dụng cho số phẩy động. Hầu hết phần cứng sẽ thực hiện phép toán này. Vì các bit được dùng để biểu diễn số phẩy động 4.5 lại thể hiện số nguyên mà không liên quan về mặt toán học đến 4.5, phép toán này sẽ

không có ý nghĩa nữa. Cụ thể hơn, `int_add` muốn thực hiện phép cộng, nhưng kết quả `int_add(3, 4.5)` không là tổng của hai số đó nữa.

Lỗi gắn với `int_add(3, 4.5)` sẽ trở nên rõ ràng hơn, nếu chúng ta nghĩ về cách chương trình sẽ thực hiện phép cộng số nguyên với đối số dấu phẩy động như thế nào. Cụ thể, giả sử chương trình định nghĩa hàm `f` mà cộng 3 cho đối số của nó

```
fun f(x) = 3+x;
```

Mà ở đâu đó trong phạm vi định nghĩa này chúng ta cũng khai báo giá trị dấu phẩy động `z`:

```
float z = 4.5;
```

Nếu chương trình dịch hay thông dịch của ngôn ngữ lập trình cho phép gọi `f(z)` và ngôn ngữ lập trình không tự động chuyển số phẩy động về số nguyên như trong tình huống này, thì lời gọi hàm `f(z)` sẽ gây ra lỗi kiểu thời gian thực vì `int_add(3, 4.5)` sẽ được thực hiện. Đây là lỗi kiểu vì phép cộng nguyên được áp dụng cho đối số không nguyên.

Nguyên nhân nhiều người mập mờ về lỗi kiểu là do lỗi kiểu nói chung phụ thuộc vào các khái niệm được định nghĩa trong chương trình hoặc ngôn ngữ lập trình, không phụ thuộc vào cách chương trình được thực hiện như thế nào trên phần cứng bên dưới. Cụ thể, ở đây có thể coi là lỗi kiểu do áp dụng phép toán nguyên cho đối số dấu phẩy động, cũng như là lỗi kiểu do áp dụng phép toán dấu phẩy động cho đối số nguyên. Không quan trọng cái nào gây ra lỗi ngắt phần cứng trên máy tính cụ thể.

Bên trong máy tính, mọi giá trị đều được lưu dạng dãy bit. Vì các số nguyên và số phẩy động được lưu dạng bốn byte trên nhiều máy, một số số nguyên và số phẩy động chồng chéo nhau; một mẫu bit đơn có thể biểu diễn số nguyên khi sử dụng một cách nào đó và biểu diễn số phẩy động theo cách khác. Do đó, lỗi kiểu xảy ra khi một mẫu được lưu biểu diễn giá trị của kiểu này lại được dùng như biểu diễn của giá trị kiểu khác.

3.1.3 Các kiểu và tối ưu

Thông tin kiểu trong chương trình có thể được sử dụng cho nhiều loại tối ưu khác nhau. Một ví dụ là tìm thành phần của bản ghi records (như chúng được gọi trong Pascal hoặc ML) hoặc cấu trúc structs (như trong C). Bài toán tìm thành phần cũng được đề cập đến trong các ngôn ngữ hướng đối tượng. Bản ghi bao gồm tập các thực thể có các kiểu khác nhau. Chẳng hạn, bản ghi sinh viên có thể gồm tên sinh viên kiểu `string` và mã sinh viên kiểu số nguyên. Trong chương trình, còn có bản ghi dành cho sinh viên chính qui còn có thêm trường lưu năm nhập trường. Cả hai kiểu được viết dạng biểu thức ML như sau:

```
Student = {name : string, number : int}
```

```
Undergrad = {name : string, number : int, year : int}
```

Trong chương trình mà thao tác bản ghi, có thể có biểu thức dạng `r.name`, nghĩa là trường tên của bản ghi `r`. Chương trình dịch sẽ sinh ra mã máy mà cho trước vị trí của bản ghi `r` trong bộ nhớ thời gian chạy, tìm vị trí của trường `name` của bản ghi này trong thời gian chạy. Nếu chương trình dịch có thể tính kiểu của bản ghi trong thời gian dịch, thì thông tin kiểu này được sử dụng để sinh ra code hiệu quả. Cụ thể hơn, nếu kiểu của `r` là `Student`, thì tính vị trí của `r.name` tương đối so với vị trí của `r` trong thời gian dịch. Chẳng hạn, nếu kiểu

của r là *Student*, thì chương trình dịch có thể tạo bảng nhỏ lưu thông tin mà tên xảy ra trước mã sinh viên trong mỗi bản ghi sinh viên. Sử dụng bảng này, chương trình dịch sẽ xác định tên ở vị trí thứ nhất được cấp cho bản ghi đó. Trong trường hợp này, biểu thức $r.name$ được dịch thành mã mà đọc giá trị tại vị trí $r+1$ (nếu r được dành cho cái gì đó khác trường thứ nhất). Tuy nhiên đối với các bản ghi khác nhau, trường tên có thể xuất hiện thứ hai hoặc thứ ba. Do đó nếu kiểu của r không biết trong thời gian dịch, thì chương trình dịch cần sinh đoạn mã tính vị trí name từ vị trí của r trong thời gian chạy. Điều này làm chương trình chạy chậm hơn. Tóm lại: một số phép toán có thể tính hiệu quả hơn nếu kiểu của toán tử được biết trong thời gian dịch.

Trong một số ngôn ngữ lập trình hướng đối tượng, kiểu của một đối tượng có thể được dùng để tìm vị trí tương đối của các thành phần của đối tượng. Trong ngôn ngữ khác, tuy nhiên, hệ thống kiểu không cho biết thông tin này và cần phải sử dụng việc tìm kiếm thời gian chạy.

3.2 An toàn kiểu và kiểm tra kiểu

3.2.1 An toàn kiểu

Một ngôn ngữ lập trình là an toàn kiểu nếu không có chương trình nào được cho phép vi phạm sự khác biệt kiểu của nó. Đôi khi điều này là không rõ ràng hiểu khác biệt kiểu là gì trong một ngôn ngữ lập trình cụ thể nào đó. Tuy nhiên, có một số sự khác biệt kiểu mà có ý nghĩa và quan trọng trong tất cả các ngôn ngữ. Chẳng hạn, hàm có kiểu khác với số nguyên. Như vậy, bất cứ ngôn ngữ nào mà cho phép số nguyên được sử dụng như hàm là không an toàn kiểu. Một hành động khác mà chúng ta luôn xem xét là lỗi kiểu là truy cập bộ nhớ mà không cấp cho chương trình.

Bảng sau đặc trưng an toàn kiểu của một số ngôn ngữ lập trình. Chúng ta sẽ thảo luận về mỗi dạng lỗi kiểu mà liệt kê trong bảng.

Safety	Example languages	Explanation
Not safe	C and C++	Type casts, pointer arithmetic
Almost safe	Pascal	Explicit deallocation; dangling pointers
Safe	Lisp, ML, Smalltalk, Java	Complete type checking

Chuyển kiểu – Type casts. Chuyển kiểu cho phép giá trị của một kiểu được sử dụng như kiểu khác. Trong C cụ thể, số nguyên có thể được chuyển thành hàm số, cho phép nhảy đến vị trí mà không chứa dạng đúng của chỉ lệnh mà là hàm của C.

Số học con trỏ. Số học con trỏ C không là an toàn kiểu. Biểu thức $*(p+1)$ có kiểu A nếu p được định nghĩa có kiểu A*. Vì giá trị được lưu ở vị trí $p+1$ có thể có kiểu bất kỳ, nên phép gán như $x = *(p+1)$ có thể lưu giá trị của một kiểu vào biến của kiểu khác và do đó nó gây ra lỗi kiểu.

Giải phóng tường minh và con trỏ treo. Trong Pascal, C và một số ngôn ngữ khác, vị trí đặt được qua con trỏ có thể được giải phóng bởi lập trình viên. Nó tạo nên con trỏ treo, tức là con trỏ mà trỏ đến vị trí không được cấp cho chương trình. Chẳng hạn, nếu p là con trỏ đến số nguyên, sau đó ta giải phóng bộ nhớ được tham chiếu bởi p , chương trình có thể cấp bộ nhớ mới để lưu kiểu giá trị khác. Bộ nhớ mới này có thể đặt được thông qua con trỏ cũ p , như thuật toán cấp bộ nhớ có thể tái sử dụng không gian được giải phóng. Con trỏ cũ p cho phép ta xử lý bộ nhớ mới này như giá trị nguyên, vì p vẫn có kiểu `int`. Điều này vi phạm an toàn kiểu. Pascal được xem là hầu như an toàn vì đây là vi phạm kiểu an toàn duy nhất của nó.

3.2.2 Kiểm tra thời gian dịch và thời gian chạy

Trong nhiều ngôn ngữ lập trình, kiểm tra kiểu được sử dụng để ngăn ngừa một số hoặc tất cả các lỗi kiểu. Một số ngôn ngữ sử dụng ràng buộc kiểu trong định nghĩa chương trình hợp lệ. Cài đặt các ngôn ngữ này kiểm tra kiểu trong thời gian dịch, trước khi chương trình bắt đầu. Trong các ngôn ngữ này, các chương trình vi phạm ràng buộc kiểu sẽ không được dịch và không thể chạy. Trong các ngôn ngữ khác, kiểm tra lỗi kiểu được thực hiện khi chương trình đang chạy.

Kiểm tra thời gian chạy.

Trong các ngôn ngữ lập trình có kiểm tra kiểu trong thời gian chạy, chương trình dịch sinh ra mã sao cho khi mã được thực hiện, sẽ kiểm tra để tin tưởng rằng các toán hạng có kiểu đúng đắn. Ưu điểm của kiểm tra kiểu thời gian chạy là nó bắt được lỗi kiểu. Nhược điểm là thời gian chạy dành cho các kiểm tra này.

Kiểm tra thời gian dịch. Nhiều ngôn ngữ lập trình hiện đại được thiết kế sao cho có khả năng kiểm tra các biểu thức có tiềm năng lỗi kiểu. Trong các ngôn ngữ này, nói chung là từ chối các chương trình không qua được kiểm tra kiểu thời gian dịch. Ưu điểm của kiểm tra kiểu thời gian dịch là nó bắt lỗi sớm hơn so với kiểm tra kiểu thời gian chạy: người phát triển chương trình được cảnh báo về lỗi trước khi chương trình được đưa cho người khác hoặc chuyển giao như sản phẩm. Vì kiểm tra kiểu trong thời gian chạy có thể loại bỏ sự cần thiết để kiểm tra một số lỗi trong thời gian chạy, kiểm tra thời gian dịch làm cho nó tạo mã hiệu quả hơn. Chẳng hạn, mã dịch ML là nhanh hơn Lisp từ hai đến bốn lần. Lý do chính tăng tốc độ này là kiểm tra kiểu tĩnh của chương trình ML giảm đáng kể sự cần thiết phải kiểm tra thời gian chạy.

Tính thận trọng của Kiểm tra kiểu thời gian dịch. Tính chất của kiểm tra kiểu thời gian dịch là chương trình dịch cần thận trọng. Điều đó có nghĩa là kiểm tra kiểu thời gian dịch sẽ tìm mọi câu lệnh và biểu thức mà sinh ra lỗi thời gian chạy, nhưng nó cũng đánh dấu các lệnh và biểu thức có lỗi ngay cả nếu chúng không gây ra lỗi thời gian chạy. Cụ thể hơn, đa số bộ kiểm tra là đúng và bảo thủ. Bộ kiểm tra được gọi là đúng, nếu không có chương trình

nào có lỗi được xem là đúng đắn. Bộ kiểm tra được gọi là bảo thủ, nếu có một số chương trình không có lỗi vẫn được coi là có lỗi.

Có lý do cho việc đa số bộ kiểm tra là bảo thủ: Đối với mọi ngôn ngữ lập trình Turing - đầy đủ, tập các chương trình có thể gây ra lỗi thời gian chạy là không giải được. Điều đó suy ra từ tính không giải được của bài toán dừng. Để xem tại sao, xét đoạn chương trình sau:

```
if (complicated-expression-that-could-run-forever)
  then (expression-with-type-error)
  else (expression-with-type-error)
```

Không giải được bài toán, biểu thức này có sinh ra lỗi thời gian chạy hay không, vì chỉ có một cách cho biểu thức có lỗi kiểu được tính toán là ‘biểu thức phức tạp mà có thể chạy mãi mãi’ phải dừng. Do đó, việc giải quyết ‘biểu thức có gây ra lỗi thời gian chạy không’ bao gồm cả việc giải quyết ‘biểu thức phức tạp có thể chạy mãi mãi’ có dừng hay không.

Vì tập các chương trình có lỗi thời gian chạy là không giải được, nên không có bộ kiểm tra kiểu thời gian dịch có thể tìm được lỗi kiểu một cách chính xác. Vì mục đích của kiểm tra kiểu là ngăn chặn lỗi, nên các bộ kiểm tra kiểu của ngôn ngữ có kiểu an toàn là bảo thủ. Sẽ có ích nếu kiểm tra kiểu tìm ra lỗi kiểu, và như hệ quả của việc không giải được ‘bài toán dừng’ một số chương trình mà có thể được thực hiện không có lỗi thời gian chạy sẽ không vượt qua được kiểm tra kiểu thời gian dịch.

Sự cân bằng chính giữa kiểm tra kiểu thời gian dịch và thời gian chạy được tổng kết trong bảng sau:

Form of Type Checking	Advantages	Disadvantages
Run-time	Prevents type errors	Slows program execution
Compile-time	Prevents type errors Eliminates run-time tests Finds type errors <i>before</i> execution and run-time tests	May restrict programming because tests are <i>conservative</i> .

Kết hợp kiểm tra kiểu thời gian dịch và thời gian chạy.

Đa số các ngôn ngữ lập trình trên thực tế sử dụng việc kết hợp giữa kiểm tra kiểu thời gian dịch và thời gian chạy. Trong Java, chẳng hạn, kiểm tra kiểu tĩnh được sử dụng để phân biệt mảng với số nguyên, nhưng lỗi biên của mảng (mà là một dạng lỗi kiểu) có thể được kiểm tra trong thời gian chạy.

3.3 Suy luận kiểu

Suy luận kiểu là quá trình xác định kiểu của biểu thức dựa trên các kiểu đã biết của các ký hiệu xuất hiện trong các biểu thức đó. Sự khác nhau giữa suy luận kiểu và kiểm tra kiểu thời gian dịch là ở mức độ. Thuật toán kiểm tra kiểu duyệt qua chương trình, kiểm tra các kiểu khai báo bởi lập trình viên có phù hợp với các yêu cầu của ngôn ngữ không. Trong suy luận kiểu, ý tưởng đưa ra là một số thông tin không cần đặc tả, và một số dạng suy diễn logic sẽ được dùng để xác định kiểu của các biến dựa theo cách chúng được sử dụng. Suy diễn kiểu được phát triển bởi Robin Milner cho ngôn ngữ lập trình ML. Các ý tưởng tương tự được phát triển độc lập bởi Curry và Hindley trong khi nghiên cứu tính toán lambda.

Mặc dù các suy diễn kiểu thực tế đã được phát triển cho ML, nhưng suy diễn kiểu có thể được áp dụng cho rất nhiều ngôn ngữ lập trình. Chẳng hạn, về mặt nguyên tắc suy diễn kiểu có thể áp dụng cho C hoặc cho các ngôn ngữ khác. Chúng ta tìm hiểu suy diễn kiểu tương đối chi tiết, vì nó thể hiện các vấn đề trọng tâm của kiểm tra kiểu và vì suy diễn kiểu cũng thể hiện một số vấn đề quan trọng của thuật toán tìm các kiểu lỗi của chương trình.

Thêm vào đó, để cung cấp dạng kiểm tra kiểu thời gian dịch linh hoạt, suy diễn kiểu ML cũng hỗ trợ đa hình. Như chúng ta đã thấy, khi xem xét kỹ thuật toán suy diễn kiểu, thuật toán này sẽ nhận biết mọi kiểu của các biến và xác định chúng cần phải bằng một kiểu chuyên biệt nào đó như *int*, *bool* hoặc *string*. Trong trường hợp khác, kiểu của một hàm nào đó có thể chứa các biến kiểu mà không ràng buộc theo cách mà hàm được định nghĩa. Trong các trường hợp này, hàm đó có thể được áp dụng cho các đối số mà kiểu của chúng sánh với dạng được cho bởi biểu thức kiểu chứa các biến kiểu.

Mặc dù suy diễn kiểu và đa hình là các khái niệm độc lập, chúng ta sẽ bàn về đa hình trong bối cảnh suy diễn kiểu, vì đa hình được nảy sinh một cách tự nhiên theo cách các biến kiểu được sử dụng trong suy diễn kiểu.

3.3.1 Các ví dụ đầu tiên về suy luận kiểu

Sau đây là hai ví dụ suy luận kiểu trong ML cho bạn cảm giác suy luận kiểu ML làm việc như thế nào. Các bước của thuật toán suy luận kiểu được giải thích ngắn gọn trong các ví dụ này, đơn giản mới cho một số ý tưởng chính. Chúng ta sẽ xem xét kỹ quá trình suy diễn kiểu này trong phần sau.

Ví dụ 3.1:

```
- fun f1(x) = x+2;  
val f1 = fn : int → int
```

Hàm *f1* cộng 2 cho đối số của nó. Trong ML, 2 là hằng nguyên; số thực 2 cần phải viết dạng 2.0. Phép toán + được tải đề; nó có thể là phép cộng nguyên hoặc phép cộng thực. Trong hàm này, tuy nhiên, phép cộng cần phải là cộng nguyên, vì 2 là số nguyên. Do đó,

đối số của hàm x cần phải là số nguyên. Tập hợp tất cả các quan sát này, chúng ta nhận được $f1$ cần phải có kiểu $int \rightarrow int$.

Ví dụ 3.2:

```
- fun f2(g,h) = g(h(0));
val f2 = fn : ('a → 'b) * (int → 'a) → 'b
```

Kiểu $('a \rightarrow 'b) * (int \rightarrow 'a) \rightarrow 'b$ được suy bởi chương trình dịch và được phân tích như $(('a \rightarrow 'b) * (int \rightarrow 'a)) \rightarrow 'b$.

Thuật toán suy luận kiểu chỉ ra rằng, vì h được áp dụng cho biến nguyên, h cần phải là hàm biến nguyên vào một kiểu gì đó. Thuật toán thể hiện kiểu gì đó bằng cách đưa ra biến kiểu mà được viết dạng $'a$. Sau đó thuật toán suy luận kiểu kết luận rằng g cần phải là hàm mà lấy giá trị của h trả về (một cái gì đó kiểu $'a$) và trả về một cái gì đó khác đi. Vì g không bị ràng buộc trả về cùng kiểu với giá trị như h , thuật toán biểu diễn cái gì đó thứ hai này bằng biến kiểu giá trị mới $'b$. Đặt các kiểu của h và g cùng nhau, chúng ta có thể thấy đối số thứ nhất của $f2$ có kiểu $('a \rightarrow 'b)$ và đối số hai có kiểu $(int \rightarrow 'a)$. Hàm $f2$ lấy cặp hai hàm này như đối số và trả về giá trị như hàm g . Do đó kiểu của hàm $f2$ sẽ là $(('a \rightarrow 'b) * (int \rightarrow 'a)) \rightarrow 'b$ như đầu ra của chương trình dịch đã nêu.

3.3.2 Thuật toán suy luận kiểu

Thuật toán suy luận kiểu ML sử dụng ba bước sau:

1. Gán kiểu cho biểu thức và mỗi biểu thức con. Sử dụng biến kiểu đối với mọi biểu thức phức hợp hoặc biến. Với các thao tác hoặc các hằng đã biết, như $+$ hoặc 3 , sử dụng kiểu mà được biết cho ký hiệu đó.
2. Sinh ra tập các ràng buộc trên kiểu, sử dụng cây phân tích cú pháp của biểu thức đó. Các ràng buộc này phản ánh một tính chất là nếu hàm được áp dụng cho một đối số nào đó, thì kiểu của đối số đó cần phải bằng kiểu miền xác định của hàm số.
3. Giải các ràng buộc này theo cách đồng nhất, mà là thuật toán dựa trên phép thế để giải hệ các phương trình. (Sẽ có nhiều thông tin hơn về đồng nhất unification trong chương về lập trình logic).

Thuật toán suy luận kiểu được giải thích qua một loạt các ví dụ sau. Các ví dụ này thể hiện các khía cạnh sau:

- giải thích thuật toán
- định nghĩa hàm đa hình
- áp dụng hàm đa hình
- hàm đệ qui
- hàm với nhiều mệnh đề
- suy luận kiểu chỉ ra lỗi của chương trình

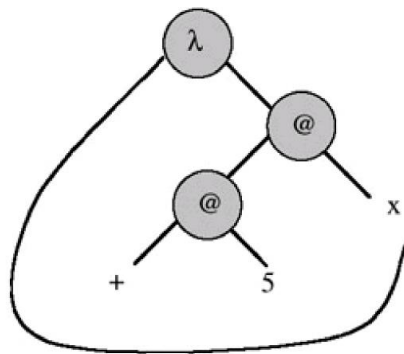
Gộp lại, cả 6 ví dụ này có thể cho bạn hiểu rõ về thuật toán suy luận kiểu, trừ việc tương tác giữa suy luận kiểu và ghi đè. Tương tác giữa ghi đè và suy luận kiểu nằm ngoài phạm vi của sách này.

3.3.3 Ví dụ giải thích thuật toán suy luận kiểu

Chúng ta sẽ nhìn thấy thuật toán suy luận kiểu làm việc như thế nào qua việc xét ví dụ sau đây:

```
- fun g(x) = 5 + x;  
val g = fn : int → int
```

Cách dễ nhất để nhìn thấy thuật toán làm việc thế nào là vẽ ra cây cú pháp của biểu thức. Chúng ta sử dụng dạng viết tắt của cây cú pháp mà chỉ liệt kê ra các ký hiệu mà xuất hiện trong biểu thức, cùng với ký hiệu @ dành cho việc áp dụng hàm cho đối số. Đối với biểu thức vừa nêu ta sử dụng đồ thị sau. Đây là dạng của cây cú pháp với cạnh đặc biệt chỉ ra việc gắn kết lambda với mỗi biến trói buộc của nó. Ở đây việc gắn kết từ x đến λ chỉ ra rằng x được trói buộc lambda ngay từ đầu biểu thức.



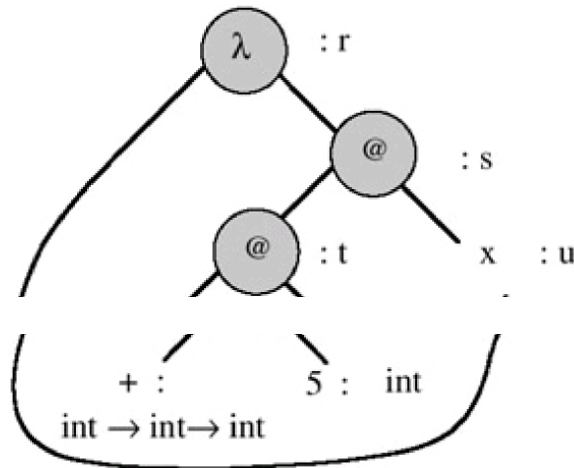
Chúng ta sử dụng đồ thị này để duyệt qua các bước suy luận kiểu của chúng ta:

1. Ta gán kiểu cho biểu thức mà mỗi biểu thức con: Chúng ta biểu diễn bước này bằng cách vẽ lại đồ thị, viết kiểu ngay sau mỗi đỉnh. Để đơn giản ta dùng các chữ đơn: $r, s, t, u, v, \dots$ cho các biến kiểu thay vì cú pháp 'a', 'b, v.v ... trong ML.

Nhắc lại, mỗi đỉnh trên cây cú pháp biểu diễn một biểu thức con. Thông tin trên cây cú pháp về các biểu thức con và kiểu của chúng được thể hiện trên bảng sau

$\lambda x. ((+ 5) x)$	r
$((+ 5) x)$	s
$(+ 5)$	t
$+$	$\text{int} \rightarrow (\text{int} \rightarrow \text{int})$
5	int
x	u

Ở đây chúng ta coi phép cộng như là hàm có kiểu $\text{int} \rightarrow (\text{int} \rightarrow \text{int})$. Ký hiệu tiền tố này đối với phép cộng không phải là cú pháp của ML, nhưng nó được sử dụng ở đây để vẽ đơn giản hơn. Như chúng ta đã biết, $+$ có thể là cộng số nguyên hoặc cộng số thực. Trong bối cảnh ở đây chúng ta có thể thấy đây là phép cộng số nguyên, vì một toán hạng của nó là số nguyên 5. Thuật toán suy luận kiểu đòi hỏi một số bước để có được kết luận đó, nhưng chúng ta sẽ không bàn đến cơ chế phân giải việc ghi đề ở đây.



2. Chúng ta sinh ra tập các ràng buộc trên kiểu, sử dụng cây cú pháp của biểu thức.

Các ràng buộc là các phương trình giữa các biểu thức kiểu mà cần được giải. Cách mà chúng ta sinh ra chúng phụ thuộc vào dạng của mỗi biểu thức con. Đối với mỗi việc áp dụng hàm, các ràng buộc được sinh dựa theo qui tắc sau:

Áp dụng hàm: nếu kiểu của f là a , kiểu của e là b , và kiểu của fe là c , thì chúng ta cần phải có $a = b \rightarrow c$

Qui tắc kiểu trên đối với việc áp dụng hàm cần được sử dụng hai lần trong biểu thức của ta:

Biểu thức con (+5); Ràng buộc $\text{int} \rightarrow (\text{int} \rightarrow \text{int}) = \text{int} \rightarrow t$

Biểu thức (+5) x; Ràng buộc $t = u \rightarrow s$.

Trong biểu thức con $(+5)$, kiểu của hàm cộng là $\text{int} \rightarrow (\text{int} \rightarrow \text{int})$; kiểu của đối số 5 là int và kiểu của kết quả áp dụng là t . Suy ra chúng ta phải có $\text{int} \rightarrow (\text{int} \rightarrow \text{int}) = \text{int} \rightarrow t$. Lập luận tương tự đối với biểu thức $(+5) x$; trong biểu thức $(+5) x$, kiểu của hàm $(+5)$ là t , kiểu của đối số x là u và kiểu của kết quả áp dụng là s . Suy ra ta có $t = u \rightarrow s$.

Khái quát Lambda (Biểu thức hàm): Nếu kiểu của x là a và kiểu của e là b , thì kiểu của $\lambda x.e$ cần phải bằng $a \rightarrow b$.

Đối với biểu thức trên, ta có một khái quát lambda. Nó cho ràng buộc sau:

Biểu thức con $\lambda x. ((+5)x)$; Ràng buộc $r = u \rightarrow s$

Nói bằng lời, kiểu của toàn biểu thức là r , kiểu của x là u và kiểu của biểu thức $((+5)x)$ là s . Điều trên cho ta phương trình: $r = u \rightarrow s$.

3. Chúng ta giải các ràng buộc này bằng phương pháp Hợp nhất

Hợp nhất là thuật toán chuẩn để giải hệ phương trình bằng cách thay thế. Các tính chất chung của thuật toán này không được bàn ở đây. Thay vào đó, các bước được chỉ ra trên ví dụ khá chi tiết để tự bạn có thể suy ra kiểu của các biểu thức đơn giản.

Đối với biểu thức trên, ta có các ràng buộc sau:

$$int \rightarrow (int \rightarrow int) = int \rightarrow t,$$

$$t = u \rightarrow s$$

$$r = u \rightarrow s$$

Nếu có cách gắn kết biểu thức kiểu với biến kiểu mà làm cho mọi phương trình đều đúng, thì biểu thức đó có kiểu đúng đắn. Trong trường hợp này, kiểu của biểu thức sẽ là biểu thức kiểu bằng với biến kiểu r . Nếu không có cách nào gắn kết biểu thức kiểu với biến kiểu mà làm cho mọi phương trình đều đúng, thì không có kiểu cho biểu thức đó. Trong trường hợp đó thuật toán suy luận kiểu sẽ thất bại, và ra thông báo lỗi là biểu thức đó không có kiểu đúng.

Chúng ta cần giải các phương trình này đồng thời. Thứ tự không quá quan trọng, mặc dù sẽ thuận tiện hơn nếu đặt phương trình chứa kiểu của toàn bộ biểu thức xuống cuối cùng, như là đầu ra của thuật toán suy luận kiểu.

Phương trình đầu tiên đúng nếu $t = int \rightarrow int$. Vì chúng ta cần $t = int \rightarrow int$, ta thế $int \rightarrow int$ cho t trong các phương trình sau. Nó cho ta hai phương trình cần giải sau:

$$int \rightarrow int = u \rightarrow s,$$

$$r = u \rightarrow s$$

Chỉ còn một cách duy nhất để có $int \rightarrow int = u \rightarrow s$ là nếu $u = s = int$. Xử lý như trước, ta thế int cho cả u và s trong phương trình còn lại. Do đó ta có

$$r = int \rightarrow int$$

mà cho ta biết rằng kiểu r của toàn bộ biểu thức là $int \rightarrow int$. Vì mọi ràng buộc đều đã được giải quyết, chúng ta biết rằng biểu thức có thể gán kiểu và chúng ta đã tính toán kiểu cho biểu thức đó.

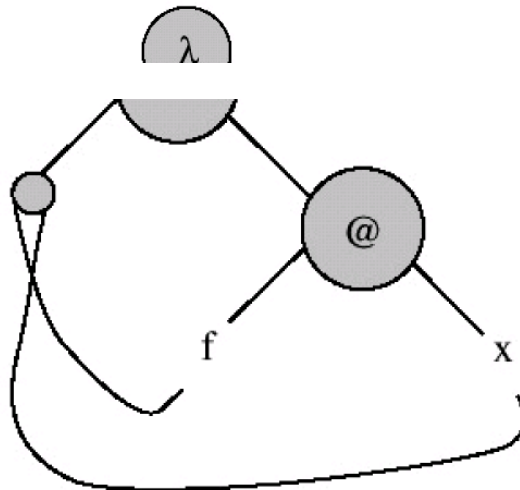
3.3.4 Ví dụ: định nghĩa hàm đa hình

```
- fun apply(f,x) = f(x);
val apply = fn : ('a → 'b) * 'a → 'b
```

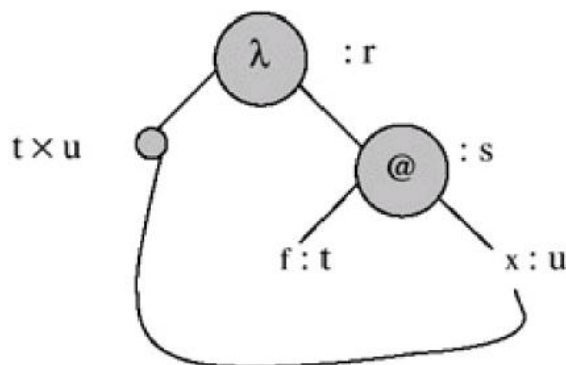
Đây là ví dụ về hàm kiểu của nó bao gồm các biến kiểu. Thuật toán suy luận kiểu bắt đầu

từ việc gán kiểu cho mỗi biểu thức. Để làm cho dễ hiểu nhất về thuật toán, chúng ta viết hàm như biểu thức lambda $\lambda (f,x).fx$ ánh xạ cặp (f,x) vào kết quả áp dụng hàm f cho x .

Đây là cây cú pháp của $\lambda (f,x).fx$, trong đó đỉnh cặp được sử dụng ở bên trái để chỉ ra rằng đối (f,x) của hàm đó là một cặp gắn kết tới f và x .



Bước thứ nhất của Thuật toán là gán kiểu cho mỗi đỉnh trên đồ thị như trên đồ thị sau:



Thông tin trên cây được thể hiện lại trên bảng sau, chỉ ra các biểu thức con và kiểu của nó gắn với mỗi đỉnh:

Biểu thức	Kiểu
$\lambda (f,x).fx$	r
(f,x)	$t \times u$
Fx	s
F	t
x	u

Bước thứ hai của Thuật toán là sinh tập các ràng buộc. Đối với ví dụ trên, có một ràng buộc cho việc áp dụng hàm và một ràng buộc cho khái quát lambda. Việc áp dụng hàm cho ta

$$t = u \rightarrow s$$

Nói cách khác, kiểu của việc áp dụng $f x$ có kiểu s , như vậy kiểu của hàm f bằng *kiểu của đối số* $\rightarrow s$. Vì kiểu của đối số là u , nên cho ta ràng buộc $t = u \rightarrow s$.

Ràng buộc thứ hai lấy từ khái quát lambda có dạng như sau:

$$r = t * u \rightarrow s$$

Thể hiện qua lời nói, thì kiểu của $\lambda (f,x).fx$ cần phải bằng *kiểu của đối số* $\rightarrow s$, với s là kiểu của cây con thể hiện kết quả hàm. Vì đối là cặp (f,x) , nên kiểu của đối là $t * u$.

Các ràng buộc có thể được giải theo thứ tự. Trước hết ta có $t = u \rightarrow s$, sau đó thế $u \rightarrow s$ cho t vào ràng buộc còn lại, ta nhận được

$$r = (u \rightarrow s) * u \rightarrow s$$

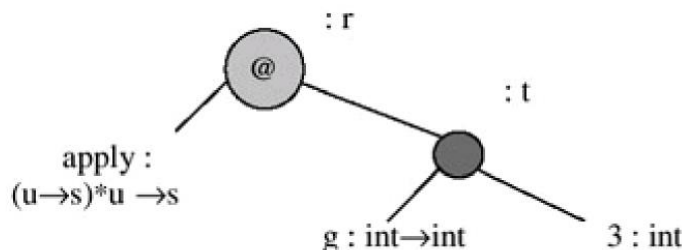
Đây là kiểu của hàm số đã cho. Nếu ta viết lại $(u \rightarrow s) * u \rightarrow s$ với ' a ' và ' b ' thay cho u và s , thì ta nhận được kiểu mà chương trình dịch đưa ra. Vì ở đây có các biến kiểu trong kiểu của biểu thức, nên hàm có thể được sử dụng $(u \rightarrow s) * u \rightarrow s$ cho nhiều kiểu của đối số. Nó được minh họa trong ví dụ sau, mà sử dụng kiểu chúng ta vừa tính toán cho việc áp dụng.

3.3.5 Ví dụ áp dụng hàm đa hình

Trong ví dụ cuối cùng, chúng ta tính kiểu của việc áp dụng `apply`. Kiểu của việc áp dụng `apply` là $(a \rightarrow b) * a \rightarrow b$, mà chứa các biến kiểu. Các biến kiểu trong kiểu này có nghĩa là `apply` là hàm đa hình, hàm có thể được áp dụng cho các kiểu khác nhau của đối số. Trong trường hợp của `apply`, kiểu $(a \rightarrow b) * a \rightarrow b$ có nghĩa là `apply` có thể áp dụng cho một cặp đối số kiểu $(a \rightarrow b) * a$ đối với các kiểu ' a ' và ' b ' bất kỳ. Chẳng hạn, sử dụng hàm $g(x) = 5 + x$ trong ví dụ trước có kiểu $\text{int} \rightarrow \text{int}$. Suy ra cặp $(g,3)$ có kiểu $(\text{int} \rightarrow \text{int}) * \text{int}$, mà sánh với kiểu $(a \rightarrow b) * a$ đối với hàm `apply`. Trong ví dụ sau ta tính kiểu của việc áp dụng:

```
apply(g,3);
```

Theo các bước của Thuật toán suy luận kiểu, ta gán kiểu cho mỗi đỉnh trên cây cú pháp của biểu thức:



Trên hình vẽ này, vòng tròn nhỏ không gán nhãn là đỉnh của cặp. Đỉnh này và hai đỉnh bên dưới biểu diễn cặp $(g,3)$. Trong ví dụ trước chúng ta gán kiểu tích với đỉnh cặp. Ở đây, để chỉ ra rằng nó tương đương với việc sử dụng biến và hằng kiểu, chúng ta gán kết biến kiểu t với đỉnh của cây cú pháp và sinh ra ràng buộc $t = (\text{int} \rightarrow \text{int}) * \text{int}$.

Có hai ràng buộc, một cho định cặp và một cho định áp dụng. Đối với định cặp ta có

$$t = (int \rightarrow int) * int$$

Đối với định áp dụng ta có

$$(u \rightarrow s) * u \rightarrow s = t \rightarrow r$$

Diễn tả bằng lời, kiểu $(u \rightarrow s) * u \rightarrow s$ của hàm cần phải bằng kiểu của đối số $\rightarrow r$, trong đó r là kiểu của kết quả áp dụng.

Bây giờ chúng ta cần giải các ràng buộc. Ràng buộc thứ nhất cho biểu thức kiểu đối với t , mà ta có thể thế đối với t trong ràng buộc thứ hai.

$$(u \rightarrow s) * u \rightarrow s = (int \rightarrow int) * int \rightarrow r$$

Ràng buộc này có biểu thức ở hai phía của dấu bằng. Để giải ràng buộc này, các phần tương ứng của mỗi biểu thức cần phải bằng nhau. Nói cách khác, chúng ta giải ràng buộc này một cách chính xác bằng việc giải bốn ràng buộc sau:

$$u = int, s = int, u = int, s = r.$$

Vì từ đó $u = s = int$, nên ta có $r = int$. Do mọi ràng buộc đều được giải quyết nên biểu thức `apply(g,3)` được xác định kiểu trong hệ thống kiểu của ML. Kiểu của `apply(g,3)` là lời giải cho biến kiểu r , tức là int .

Chúng ta có thể áp dụng cho hai kiểu khác của đối số. Nếu `not: bool  $\rightarrow$  bool`, khi đó

`apply(not, false)`

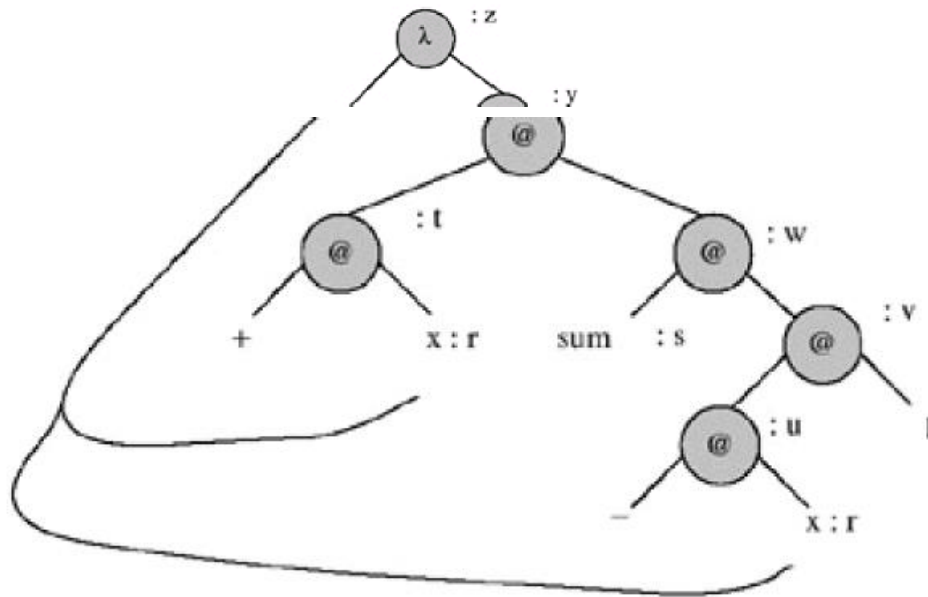
Là biểu thức kiểu đúng đắn với kiểu `bool` bằng quá trình suy luận kiểu giống như đã làm đối với `apply(g,3)`. Điều này mô tả tính đa hình của `apply`. Vì kiểu $(a \rightarrow b) * a \rightarrow b$ của `apply` chứa các biến kiểu, hàm này có thể áp dụng cho kiểu bất kỳ của đối số mà có thể nhận được, nếu các biến kiểu trong $(a \rightarrow b) * a \rightarrow b$ được thay thế bằng tên kiểu hoặc biểu thức kiểu.

3.3.6 Ví dụ về hàm đệ qui

Khi hàm được định nghĩa đệ qui. Chúng ta cần xác định kiểu thân của hàm mà không cần biết kiểu của lời gọi hàm đệ qui. Để thấy rõ điều đó thực hiện như thế nào, ta xét ví dụ hàm đệ qui đơn giản sau, mà tính tổng các số nguyên đến một số cho trước. Hàm này không dừng nhưng nó vẫn kiểm tra kiểu.

```
- fun sum(x) = x+sum(x-1);
val sum = fn : int -> int
```

Đây là đồ thị cú pháp của hàm, với biến kiểu gắn kết với mỗi đỉnh trừ đối với $+$, $-$ và 1 , ở đây chúng ta bỏ qua các phép ghi đề và coi đây là các phép toán nguyên và hằng nguyên. Vì chúng ta đang xác định kiểu của tổng, ta gắn biến kiểu với tổng và tiếp tục xử lý.



Bắt đầu từ việc áp dụng phép +, - và xử lý từ dưới thấp bên phải, các ràng buộc gắn kết với hàm và trừu tượng lambda trong biểu thức này là

$$\text{int} \rightarrow (\text{int} \rightarrow \text{int}) = r \rightarrow t,$$

$$\text{int} \rightarrow (\text{int} \rightarrow \text{int}) = r \rightarrow u,$$

$$u = \text{int} \rightarrow v,$$

$$s = v \rightarrow w,$$

$$t = w \rightarrow y,$$

$$z = r \rightarrow y.$$

Ta bổ sung thêm một ràng buộc vào danh sách trên, vì kiểu của tổng phải bằng kiểu của cả biểu thức

$$s = z$$

Ràng buộc $s = z$ là ràng buộc bổ sung gắn kết với nhận định rằng đây là khai báo đệ qui của hàm. Giải chúng theo thứ tự ta có

$$r = \text{int}, t = \text{int} \rightarrow \text{int},$$

$$u = \text{int} \rightarrow \text{int},$$

$$v = \text{int},$$

$$s = \text{int} \rightarrow w,$$

$$t = w \rightarrow y,$$

$$z = r \rightarrow y,$$

$$w = \text{int}, y = \text{int},$$

$$z = \text{int} \rightarrow \text{int},$$

$$s = \text{int} \rightarrow \text{int}.$$

Vì các ràng buộc này là giải được, nên hàm đã cho gán kiểu được. Trong quá trình giải các ràng buộc này, chúng ta đã tính được kiểu của tổng là $\text{int} \rightarrow \text{int}$.

3.3.7 Ví dụ về hàm với nhiều mệnh đề

Suy luận kiểu cho hàm có một số mệnh đề có thể được thực hiện bằng việc kiểm tra kiểu cho từng mệnh đề riêng biệt. Khi đó, vì tất cả các mệnh đề cùng xác định một hàm, chúng ta đưa ra ràng buộc là các kiểu của mọi mệnh đề cần phải bằng nhau. Ví dụ xét hàm bổ sung trên danh sách được định nghĩa như sau:

```
- fun append(nil, l) = l
  | append(x::xs, l) = x::append(xs, l);
val append = fn : 'a list * 'a list -> 'a list.
```

Như kiểu: $'a \text{ list} * 'a \text{ list} \rightarrow 'a \text{ list}$ chỉ ra, hàm `append` có thể được áp dụng cho một cặp danh sách mà cả hai danh sách đều có cùng kiểu các phần tử của danh sách. Vì vậy hàm `append` là hàm đa hình trên danh sách.

Chúng ta bắt đầu suy luận kiểu cho `append` bằng thuật toán ba bước cho mệnh đề thứ nhất của định nghĩa, sau đó lặp lại các bước này cho mệnh đề thứ hai. Nó cho ta hai kiểu:

```
append : 'a list * 'b -> 'b
append : 'a list * 'b -> 'a list
```

Bằng trực quan, mệnh đề thứ nhất có kiểu $'a \text{ list} * 'b \rightarrow 'b$, vì đối thứ nhất cần sánh với `nil`, nhưng đối thứ hai có thể là bất cứ cái gì. Mệnh đề thứ hai có kiểu $'a \text{ list} * 'b \rightarrow 'a \text{ list}$, vì kết quả trả về là danh sách chứa phần tử thứ nhất của danh sách truyền vào như đối thứ nhất.

Nếu ta buộc có ràng buộc:

$$'a \text{ list} * 'b \rightarrow 'b = 'a \text{ list} * 'b \rightarrow 'a \text{ list}$$

thì $'b = 'a \text{ list}$, điều đó cho ta kiểu cuối cùng của append là

$$\text{append} : 'a \text{ list} * 'a \text{ list} \rightarrow 'a \text{ list}$$

3.3.8 Ví dụ suy luận kiểu chỉ ra lỗi của chương trình

Sau đây là ví dụ mà cho biết suy luận kiểu chỉ ra lỗi của chương trình như thế nào, ngay cả khi chương trình có thể được gõ đúng chính tả. Đây là khai báo mẫu (không đúng) về hàm đảo ngược trên danh sách.

```
- fun reverse (nil) = nil
  | reverse (x::lst) = reverse (lst);
val reverse = fn : 'a list → 'b list
```

Như chương trình dịch nêu ra, hàm này có kiểu; ở đây không có lỗi kiểu trong khai báo này. Tuy nhiên, xem kỹ kiểu của hàm ngược. Kiểu $'a \text{ list} \rightarrow 'b \text{ list}$ có nghĩa là ta có thể áp dụng reverse cho kiểu bất kỳ của danh sách và nhận được kiểu bất kỳ của danh sách như kết quả. Tuy nhiên, kiểu của danh sách đảo ngược mà lại không phải kiểu của danh sách ban đầu!

Vì không có ý nghĩa đối với reverse trả về danh sách mà có kiểu khác với đối số, do đó cần phải có gì đó không đúng đối với đoạn code này. Vấn đề nằm ở chỗ, trong mệnh đề thứ hai phần tử đầu tiên của danh sách không được sử dụng như một phần của đầu ra. Do đó reverse luôn trả về danh sách rỗng.

Như ví dụ này chỉ ra, thuật toán suy luận kiểu đôi khi có thể trả về kiểu mà tổng quát hơn cả cái mà ta mong đợi. Nó không chỉ ra lỗi kiểu. Trong ví dụ này, hàm reverse lỗi có thể được sử dụng ở bất cứ đâu mà hàm reverse đúng có thể được sử dụng. Tuy nhiên, kiểu của hàm reverse cũng có ích vì nó báo cho người lập trình biết rằng ở đâu đó có lỗi trong chương trình.

3.4 Đa hình và ghi đề

Đa hình, mà về mặt từ ngữ có nghĩa là “có nhiều dạng khác nhau”, hướng tới cấu trúc mà có thể có các kiểu khác nhau, nếu cần. Chẳng hạn, hàm tính độ dài của danh sách kiểu bất kỳ là đa hình vì nó có kiểu là $'a \text{ list} \rightarrow \text{int}$ cho mỗi kiểu $'a$.

Có ba kiểu đa hình trong các ngôn ngữ lập trình đương thời:

- Đa hình tham số mà ở đó hàm số có thể áp dụng cho các đối số bất kỳ mà kiểu của nó sánh với biểu thức kiểu gồm các biến kiểu
- Đa hình có chủ định, hay nói cách khác là ghi đề ở đó hai hay nhiều cài đặt có các kiểu khác nhau lại có cùng một tên

- Đa hình kiểu con, ở đó quan hệ kiểu con giữa các kiểu cho phép một biểu thức có nhiều kiểu có thể

Đa hình tham số và ghi đè được xét trong mục này, Đa hình kiểu con được xét trong các chương sau cùng với lập trình hướng đối tượng.

3.4.1 Đa hình tham số

Đặc trưng chính của đa hình tham số là tập các kiểu gắn kết với hàm hoặc một giá trị khác được cho trước bởi biểu thức kiểu mà chứa biến kiểu. Chẳng hạn, hàm ML mà sắp xếp các danh sách có thể có kiểu ML

```
sort('a' * 'a → bool) * 'a list → 'a list
```

Diễn tả bằng lời, sắp xếp có thể áp dụng cho một cặp bất kỳ bao gồm một hàm và một danh sách, hàm có kiểu `'a * 'a → bool`, trong đó kiểu `'a` cần phải là kiểu của các phần tử của danh sách. Đối số hàm là thao tác nhỏ hơn được sử dụng để xác định thứ tự trên danh sách được sắp xếp.

Trong đa hình tham số, hàm có thể có vô hạn kiểu, sẽ có vô hạn cách thay các biến kiểu bằng các biến thực tế. Chẳng hạn, hàm sắp xếp có thể được dùng để sắp xếp danh sách số nguyên, danh sách các danh sách số nguyên hoặc danh sách các danh sách các danh sách số nguyên, ...

Đa hình tham số có thể là ẩn hoặc tường minh. Trong đa hình tham số tường minh, văn bản chương trình chứa các biến kiểu mà xác định cách hàm hoặc các giá trị khác có thể được coi như đa hình. Thêm vào đó, đa hình tường minh thường bao gồm việc khởi tạo tường minh hoặc áp dụng kiểu để chỉ ra biến kiểu được thay thế như thế nào vào các kiểu nào đó trong khi dùng giá trị đa hình. Các templates C++ là các ví dụ phổ cập về đa hình tường minh. Đa hình ML được gọi là đa hình ẩn, vì chương trình mà khai báo và sử dụng hàm đa hình không cần chứa kiểu – thuật toán kiểm tra kiểu tính toán khi hàm là đa hình và tính toán khởi tạo biến kiểu nếu cần.

C++ Function Templates

Đối với nhiều độc giả, cơ chế tham số kiểu quen nhất là cơ chế C++ Templates. Mặc dù một số lập trình viên C++ gắn kết Templates với các lớp và lập trình hướng đối tượng, templates hàm cũng có ích đối với các chương trình không khai báo bất cứ lớp nào.

Xét ví dụ sau, giả sử bạn viết một hàm đơn giản đổi hai giá trị của hai biến nguyên.

```
void swap(int& x, int& y){
    int tmp = x; x = y; y = tmp;
}
```

Mặc dù đoạn code này dùng để đổi giá trị của hai biến nguyên, dãy lệnh đó cũng dùng được cho các kiểu khác của các biến. Nếu ta muốn đổi giá trị của các biến kiểu khác, thì ta có thể định nghĩa template hàm, mà sử dụng biến kiểu T ở vị trí tên kiểu int.

```
template <typename T>
void swap(T& x, T& y){
    T tmp = x; x = y; y = tmp;
}
```

Đối với ai mà chưa quen với templates, ý tưởng chính là nghĩ đến tên kiểu T như một tham số cho hàm từ kiểu đến các hàm. Khi áp dụng hoặc khởi tạo một kiểu cụ thể nào đó, kết quả của phiên bản swap đối với biến int đã được thay với kiểu khác. Nói cách khác, swap là hàm tổng quát mà làm việc tốt đối với nhiều kiểu của các đối số. Templates cho phép ta coi swap là hàm với đối số kiểu.

Trong C++, templates hàm được khởi tạo tự động khi cần, với kiểu của đối số hàm số được dùng để xác định khởi tạo nào cần. Điều này được mô tả trong các dòng code minh họa sau:

```
int i,j; ... swap(i,j); // Use swap with T replaced with int
float a,b; ... swap(a,b); // Use swap with T replaced with float
String s,t; ... swap(s,t); // Use swap with T replaced with String
```

So sánh với đa hình ML

Trong đa hình ML, thuật toán suy luận kiểu đưa ra kết luận kiểu của hàm và kiểu áp dụng hàm. Khi hàm là đa hình, các hành động của thuật toán suy luận kiểu có thể được hiểu như việc chèn tự động khai báo template (“template declarations”) và khởi tạo template (“template installation”) vào chương trình. Chúng ta có thể thấy nó làm việc như thế nào bằng việc xem xét hàm sắp xếp ML sort mà tương tự với hàm sắp xếp C++ sort khai báo trước đây:

```
fun insert(less, x, nil) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                           else y::insert(less,x,ys);
fun sort(less, nil) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

Để sort là đa hình, thao tác nhỏ hơn cần phải được truyền như đối số của hàm cho sort. Các kiểu của insert và sort được suy ra bằng thuật toán suy luận kiểu như sau.

```
val insert = fn : ('a * 'a -> bool) * 'a * 'a list -> 'a list
val sort = fn : ('a * 'a -> bool) * 'a list -> 'a list
```

Trong các kiểu trên, biến kiểu ‘a có thể được khởi tạo là kiểu bất kỳ, nếu cần thiết. Kết quả, các hàm này được coi như chúng là các “templates”. Bằng việc sử dụng kết hợp

template C++, các hàm ML, cú pháp kiểu, các hàm được định nghĩa trước đây có thể được viết như sau:

```
template <type 'a>
fun insert(less : 'a * 'a -> bool, x : 'a, nil : 'a list) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                           else y::insert(less,x,ys);
template <type 'a>
fun sort(less : 'a * 'a -> bool, nil : 'a list) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

Các khai báo trên là các phương án khai báo kiểu tường minh của các hàm ML đa hình ẩn. Nói cách khác, thuật toán suy luận kiểu ML có thể được hiểu chương trình tiền xử lý mà chuyển biểu thức ML không có thông tin kiểu về biểu thức trong ngôn ngữ trung gian nào đó có kiểu tường minh với các templates. Từ góc độ này, sự khác biệt giữa đa hình tường minh và ẩn là chương trình tiền xử lý của ngôn ngữ lập trình (như chương trình dịch ML) dùng cú pháp ẩn đơn giản hơn và tự động chèn các thông tin kiểu tường minh, chuyển từ dạng ẩn sang tường minh, trước khi chương trình được dịch và thực hiện.

Kết thúc ví dụ này, giả sử ta khai báo hàm nhỏ hơn cho số nguyên:

```
- fun less(x,y) = x < y;
val less = fn : int * int -> bool
```

Trong áp dụng sau của hàm sắp xếp đa hình, sort template được tự động khởi tạo thành kiểu int, như vậy sort có thể được dùng để sắp xếp danh sách nguyên.

```
- sort (less, [1,4,5,3,2]);
val it = [1,2,3,4,5] : int list
```

3.4.2 Cài đặt đa hình tham số

Templates C++ và các hàm đa hình tham số ML được cài đặt khác nhau. Nguyên nhân của sự khác nhau đó không liên quan gì đến sự khác nhau giữa cú pháp đa hình tường minh và cú pháp đa hình ẩn. Sự cần thiết dùng các kỹ thuật cài đặt khác nhau xuất phát từ sự khác nhau giữa biểu diễn dữ liệu trong C và trong ML.

Cài đặt C++

Templates C++ được khởi tạo tại thời điểm link chương trình. Cụ thể hơn, giả sử template hàm swap được lưu ở một file và đã được dịch và một chương trình gọi swap được lưu ở một file khác và được dịch độc lập. Các files được gọi là đối tượng tái phân bổ được tạo bởi việc dịch của chương trình gọi sẽ bao gồm thông tin chỉ ra rằng đoạn code được dịch gọi hàm swap của một kiểu nào đó. Chương trình linker được thiết kế để kết hợp hai chương trình thành phần bằng việc gắn kết lời gọi swap trong lời gọi chương trình với định nghĩa

của swap trong đơn vị chương trình dịch riêng biệt. Nó thực hiện bằng cách khởi tạo đoạn code đã được dịch của swap ở dạng mà tạo ra code thích hợp với lời gọi tới swap.

Nếu chương trình gọi swap với một số kiểu khác nhau, thì một số bản sao khởi tạo khác nhau của swap sẽ được tạo ra. Một lý do mà các bản sao khác nhau cần cho mỗi kiểu của lời gọi là hàm swap khai báo biến cục bộ tmp của kiểu T. Không gian cho tmp cần được cấp trong bản ghi kích hoạt cho swap. Do đó đoạn code được dịch cho swap cần được chỉnh sửa theo kích thước của biến kiểu T. Chẳng hạn, nếu T là cấu trúc hoặc đối tượng, thì kích thước là khá lớn. Mặt khác, nếu T là int, thì kích thước sẽ nhỏ. Trong mọi trường hợp, đoạn code đã dịch cho swap cần biết kích thước của dữ liệu để đánh địa chỉ vào bản ghi kích hoạt được chính xác.

Tiến trình linking cho C++ là tương đối phức tạp. Chúng ta không tìm hiểu chi tiết ở đây. Tuy nhiên, cần phải nhấn mạnh rằng, nếu < là phép toán ghi đè, thì phiên bản đúng của < cần phải được định danh khi đoạn code đã dịch cho sort được link với chương trình gọi. Chẳng hạn, xét hàm tổng quát sort sau:

```
template <typename T>
void sort( int count, T * A[count] ) {
    for (int i=0; i < count-1; i++)
        for (int j=i+1; j < count-1; j++)
            if (A[j] < A[i]) swap(A[i],A[j]);
}
```

Nếu A là mảng kiểu T, thì sort(n, A) sẽ làm việc chỉ khi phép toán < được định nghĩa cho kiểu T. Yêu cầu này của sort không được khai báo ở đâu trong đoạn code C++. Tuy nhiên khi template hàm được khởi tạo, kiểu thực tế T cần phải có phép toán < được định nghĩa hoặc lỗi trong thời gian link sẽ thông báo và không đoạn code đối tượng thực thi nào được tạo ra.

Cài đặt ML

Trong ML có một dãy các chỉ lệnh được dịch cho mỗi hàm đa hình. Ở đây không cần tạo ra các bản sao khác nhau của đoạn code này cho các kiểu khác nhau của đối số, vì các kiểu dữ liệu liên quan được biểu diễn theo các cách tương tự. Cụ thể hơn, các con trỏ được sử dụng trong việc truyền tham số và trong việc biểu diễn các cấu trúc dữ liệu như danh sách, nên khi hàm là đa hình, nó có thể truy cập đến mọi dữ liệu cần thiết theo cùng một cách, không quan tâm đến kiểu của chúng. Tính chất này của ML được gọi là biểu diễn dữ liệu đồng nhất (uniform data representation).

Ví dụ đơn giản về biểu diễn dữ liệu đồng nhất là phiên bản ML của hàm đa hình swap:

```
- fun swap(x,y) = let val tmp = x in x := !y; y := !tmp end;
val swap = fn : 'a ref * 'a ref -> unit
```

Như kiểu đã chỉ rõ, hàm swap này có thể áp dụng cho hai tham chiếu bất kỳ cùng kiểu. Mặc dù các tham chiếu ML nói chung làm việc giống như các biến có thể gán được trong ngôn ngữ khác, giá trị tham chiếu này là con trỏ đến ô chứa giá trị. Do đó, khi một cặp

tham chiếu được truyền cho hàm swap, hàm swap sẽ nhận cặp con trỏ này. Hai con trỏ có cùng kích thước (thông thường 32 bit), không quan tâm đến kiểu của giá trị chứa tại các vị trí mà nó trỏ đến. Trên thực tế, chúng ta có thể hoàn tất toàn bộ tính toán bằng chỉ một phép gán con trỏ. Vậy, không có đoạn code đã dịch nào cho swap phụ thuộc vào kích cỡ dữ liệu được tham chiếu bởi các đối số x và y.

Biểu diễn dữ liệu đồng nhất có ưu điểm và có nhược điểm. Vì không cần nhân bản code cho các kiểu đối số khác nhau, biểu diễn dữ liệu đồng nhất dùng kích thước code nhỏ và tránh việc dịch theo phong cách như link C++. Mặt khác, đoạn code kết quả có thể kém hiệu quả hơn, vì biểu diễn dữ liệu đồng nhất thường sử dụng các con trỏ đến dữ liệu thay vì lưu trữ dữ liệu trực tiếp trong cấu trúc.

Để cho các hàm list đa hình làm việc đúng, mọi danh sách cần phải được biểu diễn chính xác theo cùng một cách. Vì yêu cầu đồng nhất này, các giá trị nhỏ, mà có thể đưa trực tiếp vào phần car của ô list cons, không thể đặt ở đó vì giá trị lớn không thể lưu ở đó. Vì vậy chúng ta cần lưu con trỏ cho giá trị nhỏ trong danh sách, như chúng ta lưu con trỏ cho giá trị lớn. Lập trình viên ML và người viết chương trình dịch gọi tiến trình xem mọi dữ liệu đều giống nhau bằng các con trỏ.

So sánh

Hai điểm so sánh là tính hiệu quả và đưa ra thông báo lỗi. Nói về tính hiệu quả, cài đặt C++ đòi hỏi nhiều công sức hơn trong thời gian link và tạo ra code lớn hơn, như khởi tạo các templates nhiều lần tạo ra một số bản sao của đoạn code đó. Cài đặt ML sẽ chạy chậm hơn, trừ khi sử dụng cách tối ưu đặc biệt; biểu diễn dữ liệu đồng nhất bao gồm việc sử dụng tích cực các con trỏ và các con trỏ này cần được lưu và truy cập hiệu quả.

Như nguyên lý lập trình tổng quát, sẽ thuận tiện nếu có thông báo lỗi trong thời gian dịch hơn là thời gian link. Một lý do là các modules chương trình rời nhau được dịch độc lập, nhưng sẽ link cùng nhau chỉ khi toàn bộ chương trình được chuyển sang ngôn ngữ Assembly. Do đó, việc dịch là quá trình cục bộ, mà có thể được thực hiện bởi người thiết kế hoặc cài đặt của thành phần đơn lẻ đó. Ngược lại, lỗi trong thời gian link thể hiện tính chất tổng thể của hệ thống mà chưa được biết khi toàn bộ hệ thống được dịch sang Assembly. Vì lý do này, lỗi thời gian link C++ gắn kết với thao tác trên templates có thể gây nên khó chịu và là nguyên nhân gây thất bại.

Tốt hơn là báo lỗi Templates C++ có thể đạt được nếu cú pháp template bao gồm cả mô tả các thao tác cần thiết trên tham số kiểu. Tuy nhiên, điều này tương đối phức tạp trong C++ do tính ghi đè và một số tính chất khác của ngôn ngữ này. Ngược lại, ML có tính ghi đè đơn giản hơn và chứa nhiều thông tin hơn trong cấu trúc được tham số hóa, cho phép mọi lỗi kiểu được thông báo khi đơn vị chương trình được dịch.

3.4.3 Ghi đè

Đa hình tham số có thể tương phản với ghi đè. Một ký hiệu được ghi đè nếu nó có hai (hoặc nhiều hơn) ý nghĩa, khác nhau bởi kiểu, và được hóa giải trong thời gian dịch. Trong bài báo lịch sử có tầm ảnh hưởng, Christopher Strachey nói đến đa hình kiểu ML như mô hình tham số và ghi đè như đa hình được sắp đặt trước (ad hoc).

Ví dụ 3.4. Trong ML chuẩn, như trong nhiều ngôn ngữ khác, phép toán $+$ có hai cài đặt khác nhau liên kết với nó, một của kiểu $\text{int} * \text{int} \rightarrow \text{int}$ và một của kiểu $\text{real} * \text{real} \rightarrow \text{real}$. Lý do cả hai thao tác này đều được đặt tên là $+$ vì chúng đều là phép cộng số. Tuy nhiên, ở mức cài đặt, hai thao tác này thực ra là rất khác nhau. Vì số nguyên được biểu diễn một cách (như các số nhị phân) và các số thực biểu diễn cách khác (như lũy thừa và mantissa trong ký hiệu khoa học), cách mà phép cộng kết hợp các bit của các đối số để tạo ra bits của kết quả là rất khác với cách mà được thực hiện trong phép cộng số thực.

Sự khác biệt quan trọng giữa đa hình tham số và ghi đè là các hàm tham số sử dụng một thuật toán để thao tác trên các đối số của nhiều kiểu khác nhau, trong khi đó các hàm ghi đè có thể dùng các thuật toán khác nhau cho mỗi kiểu đối số.

Một đặc trưng của ghi đè là ghi đè được hóa giải trong thời gian dịch. Nếu hàm được ghi đè, thì chương trình dịch sẽ chọn giữa các thuật toán có thể trong thời gian dịch. Chọn một thuật toán trong số các thuật toán có thể gắn kết với hàm ghi đè này được gọi là hóa giải việc ghi đè. Chẳng hạn, xét hai biểu thức sau:

```
3 + 2; /* add two integers */
3.0 + 2.0; /* add two real (floating point) numbers */
```

Sau đây sẽ giải thích chương trình dịch sẽ sinh đoạn code để tính từng biểu thức như thế nào:

- $3+2$: Pha phân tích của chương trình dịch sẽ xây dựng cây cú pháp của biểu thức này và pha kiểm tra kiểu sẽ tính toán kiểu cho mỗi ký hiệu. Vì pha kiểm tra kiểu sẽ xác định rằng $+$ cần phải có kiểu $\text{int} * \text{int} \rightarrow \text{int}$, pha sinh code của chương trình dịch sẽ tạo ra chỉ lệnh mã máy để thực hiện phép cộng nguyên.
- $3.0 + 2.0$: Pha phân tích của chương trình dịch sẽ xây dựng cây cú pháp của biểu thức này và pha kiểm tra kiểu sẽ tính toán kiểu cho mỗi ký hiệu. Vì pha kiểm tra kiểu sẽ xác định rằng $+$ cần phải có kiểu $\text{real} * \text{real} \rightarrow \text{real}$, pha sinh code của chương trình dịch sẽ tạo ra chỉ lệnh mã máy để thực hiện phép cộng số thực.

Chuyển đổi tự động là cơ chế riêng biệt mà có thể kết hợp với ghi đè. Tuy nhiên, có thể đối với ngôn ngữ có ghi đè và không có chuyển đổi tự động. Chẳng hạn, ML không có chuyển đổi tự động.

3.5 Đa hình và ghi đè

Nhiều cách khai báo kiểu và nhiều cách đồng nhất kiểu đã có trong các ngôn ngữ lập trình trước đây. Nguyên nhân tại sao khai báo kiểu và đồng nhất kiểu lại liên quan đến nhau là, khi tên kiểu được khai báo, điều quan trọng là quyết định xem đây có phải là kiểu mới không, nó có khác mọi kiểu khác không hoặc tên mới nhưng về ý nghĩa nó đồng nhất với một kiểu nào đó mà đã được sử dụng ở đâu đó trong chương trình.

Một số ngôn ngữ lập trình đã dùng một số dạng đồng nhất kiểu tương đối phức tạp dẫn đến một số dạng khai báo kiểu không rõ ràng. Thay vì tranh luận một số dạng trước đây, ta xét một số khả năng sau.

3.5.1 Khai báo kiểu trong suốt

Có hai cách khai báo kiểu:

- Trong suốt (transparent), nghĩa là tên khác được gán cho một kiểu mà có thể được biểu diễn không cần tên này.
- Không trong suốt (opaque), nghĩa là kiểu mới được đưa vào chương trình mà không bằng kiểu nào khác.

Trong ML dạng khai báo kiểu trong suốt:

```
type <identifier> = <type expression>
```

ở đó identifier trở thành một từ đồng nghĩa với biểu thức kiểu. Chẳng hạn,

```
type Celsius = real;
type Fahrenheit = real;
```

khai báo hai tên kiểu, Celsius và Fahrenheit, mà ý nghĩa của nó là kiểu số thực, giống như cách hai biểu thức giá trị

```
val x = 3;
val y = 3;
```

khai báo hai biến mà giá trị của nó là 3. (Nên nhớ rằng, các biến ML là các biến không gán được: biến x sẽ có giá trị 3 bất kể nó được dùng ở đâu). Nếu ta khai báo hàm ML chuyển đổi từ Fahrenheit sang Celsius, hàm này sẽ có kiểu real -> real:

```
- fun toCelsius(x) = ((x-32.0)* 0.555556);
val toCelsius = fn : real → real
```

Điều này không ngạc nhiên, vì ở đây không có dấu hiệu là đối số của hàm và giá trị trả về có kiểu khác với real. Nếu chúng ta muốn chỉ rõ các kiểu của đối số và kết quả, ta cần bổ sung chúng vào khai báo hàm số. Nó sẽ tạo ra hàm của kiểu Fahrenheit -> Celsius:

```
- fun toCelsius(x : Fahrenheit) = ((x-32.0)* 0.555556) : Celsius;
val toCelsius = fn : Fahrenheit → Celsius
```

Đây là phương án của hàm toCelsius mà có nhiều thông tin hơn, vì các kiểu được nêu rõ như mong muốn đối với hàm. Tuy nhiên, vì Fahrenheit và Celsius là các đồng nghĩa của real, nên hàm này có thể áp dụng cho các đối số real:

```
- toCelsius(74.5);
val it = 23.61113 : Celsius
```

Bộ kiểm tra kiểu của ML cho kiểu kết quả là Celsius, nhưng vì Celsius = real, nên kết quả có thể được sử dụng dạng biểu thức số thực.

Các kiểu trừu tượng ML là ví dụ khai báo kiểu không trong suốt. Kiểu dữ liệu ML là dạng khác của khai báo không trong suốt, tính không trong suốt của khai báo kiểu dữ liệu ML được bàn đến trong mục sau nữa.

Hai cái tên có tính lịch sử cho hai dạng bằng nhau của kiểu là *bằng nhau kiểu tên* và *bằng nhau kiểu cấu trúc*. Về trực quan, bằng nhau kiểu tên có nghĩa là hai tên kiểu được xem xét là bằng nhau khi kiểm tra kiểu nếu chúng có cùng tên. Bằng nhau cấu trúc có nghĩa là hai tên kiểu là như nhau nếu như các kiểu mà chúng gán tên là như nhau (tức là có cùng cấu trúc). Tuy các khái niệm này có vẻ đơn giản và vô thưởng, vô phạt, nhưng ở đây có nhiều thứ không rõ ràng liên quan đến sự bằng nhau kiểu tên và kiểu cấu trúc trong các ngôn ngữ lập trình.

3.5.2 Khai báo và cấu trúc (Structs) C

Cấu trúc khai báo kiểu cơ bản trong C là typedef. Sau đây là một số ví dụ đơn giản:

```
typedef char byte;
typedef byte ten_bytes[10];
```

Lệnh đầu khai báo kiểu byte bằng với kiểu char và lệnh thứ hai kiểu mảng ten_bytes bằng với mảng của 10 bytes. Nói một cách tổng quát, typedef C làm việc giống như khai báo kiểu trong suốt của ML được bàn đến trong mục trước. Tuy nhiên, khi sử dụng cấu trúc structs, bộ kiểm tra kiểu C coi như các tên kiểu khai báo riêng là không bằng nhau, ngay cả khi chúng được khai báo gán tên cho cùng kiểu cấu trúc struct type. Sau đây là ví dụ một chương trình ngắn chỉ ra điều đó được vận hành như thế nào:

```
typedef struct {int m;} A;
typedef struct {int m;} B;
Ax;
By;
x=y; /* incompatible types in assignment */
```

Ở đây, mặc dù hai cấu trúc được sử dụng trong hai khai báo là như nhau, bộ kiểm tra kiểu C không coi A và B là hai kiểu bằng nhau. Tuy nhiên, nếu ta thay hai khai báo trên bằng typedef int A; typedef int B; mà sử dụng int thay cho struct, phép gán x = y được cho là đúng kiểu.

3.5.3 Khai báo và cấu trúc (Structs) ML

Khai báo kiểu dữ liệu ML, đã bàn trong chương trước, là một dạng khai báo kiểu mà đồng thời định nghĩa tên kiểu mới và các thao tác để tạo và sử dụng các phần tử của kiểu đó. Vì mọi ví dụ trong chương đó đều là đơn hình, ta sẽ xem nhanh khai báo đa hình trước khi bàn về bằng nhau kiểu.

Đây là ví dụ kiểu cây dữ liệu đa hình. Bạn có thể so sánh nó với ví dụ đơn hình trong chương trước:

```
datatype 'a tree = LEAF of 'a | NODE of ('a tree * 'a tree);
```

Khai báo này định nghĩa kiểu đa hình của tree 'a, với khởi tạo cây int, cây string, v.v..., cùng với các cấu trúc đa hình LEAF và NODE:

```
- LEAF;  
val it=fn: 'a-> 'a tree  
- NODE;  
val it=fn: 'a tree * 'a tree -> 'a tree
```

Hàm sau kiểm tra xem phần tử có xuất hiện trên cây không. Hàm này sử dụng exception, được xét trong chương ..., khi phần tử không được tìm thấy. ML yêu cầu exception được khai báo trước khi nó được sử dụng.

```
- exception NotFound;  
exception NotFound  
- fun inTree(x, EMPTY) = raise Not Found  
  | inTree(x, LEAF(y)) = x = y  
  | inTree(x, NODE(y,z)) = inTree(x, y) orelse inTree(x, z);  
  
val inTree = fn : 'a * 'a tree -> bool
```

Mỗi khai báo kiểu dữ liệu mới của ML được coi như định nghĩa kiểu mới khác với mọi kiểu khác. Ngay cả khi nếu hai kiểu có cùng cấu trúc, chúng cũng không được coi là tương đương. Thiết kế ML làm cho nó khó khai báo hai kiểu dữ liệu tương đương, như mỗi hàm tạo chỉ có một kiểu. Chẳng hạn, hai khai báo

```
datatype A = C of int;  
datatype B = C of int;
```

Khai báo hai kiểu A, B khác nhau. Vì khai báo thứ hai đứng sau cái thứ nhất và ML coi mỗi khai báo là bắt đầu một phạm vi mới, hàm tạo C có kiểu int -> B sau khi cả hai khai báo được xử lý. Tuy nhiên chúng ta có thể thấy A và B được coi là khác nhau bằng cách viết hàm mà tìm cách xét giá trị của một kiểu như của kiểu khác.

```
fun f(x:A) = x:B;
```

Mà dẫn đến thông báo lỗi Error: expression doesn't match constrain [tycoon mismatch].

3.6 Tóm lược chương

Trong chương này, chúng ta đã xét các nguyên nhân sử dụng kiểu trong ngôn ngữ lập trình, các phương pháp kiểm tra kiểu, và một số vấn đề về kiểu như đa hình, ghi đè và đồng nhất kiểu.

Nguyên nhân sử dụng kiểu:

Có ba sử dụng kiểu chính trong ngôn ngữ lập trình:

- Đặt tên và tổ chức các khái niệm: Các hàm và các cấu trúc dữ liệu cần phải là các kiểu đã cho mà phản ánh cách các cấu trúc tính toán này được sử dụng trong chương trình.
- Làm cho tin tưởng rằng các dãy bit trong bộ nhớ máy tính được biên dịch một cách đúng đắn: kiểm tra kiểu giữ cho các phép toán không bị áp dụng vào các toán hạng theo cách không đúng. Chẳng hạn, nó ngăn chặn phép toán dấu phẩy động không cho áp dụng vào dãy bit mà thể hiện xấu.
- Cung cấp thông tin cho chương trình dịch về dữ liệu mà được thao tác bởi chương trình: trong ngôn ngữ mà ở đó chương trình dịch có thể xác định kiểu của một cấu trúc dữ liệu, chẳng hạn, thông tin kiểu có thể được sử dụng để xác định vị trí tương đối của một thành phần trong cấu trúc. Thông tin kiểu thời gian dịch có thể sinh ra code hiệu quả đánh chỉ số cho cấu trúc dữ liệu trong thời gian chạy.

Suy luận kiểu:

Suy luận kiểu là quá trình xác định kiểu của các biểu thức dựa trên các kiểu đã biết của một số ký hiệu xuất hiện trong chúng. Chẳng hạn, chúng ta đã thấy suy luận như thế nào để hàm khai báo bởi:

```
fun g(x) = 5+x;
```

Có kiểu `int -> int`. Sự khác biệt giữa suy luận kiểu và kiểm tra kiểu thời gian dịch là ở mức độ. Thuật toán kiểm tra kiểu duyệt qua chương trình để kiểm tra các kiểu khai báo bởi lập trình viên có đáp ứng các yêu cầu của ngôn ngữ lập trình không. Trong suy luận kiểu, ý tưởng là một số thông tin không được đặc tả và một số dạng suy luận logic được yêu cầu để xác định để xác định kiểu của các định danh từ cách mà chúng được sử dụng.

Một số bước sau được sử dụng cho suy luận kiểu:

1. Gán kiểu cho biểu thức mà mỗi biểu thức con bằng việc sử dụng kiểu đã biết của ký hiệu một biến kiểu.
2. Sinh tập các ràng buộc trên kiểu bằng việc sử dụng cây cú pháp của biểu thức.
3. Giải các ràng buộc đó bằng việc sử dụng Hợp nhất, mà là thuật toán dựa trên phép thế để giải hệ phương trình.

Trong một loạt các ví dụ, chúng ta đã thấy việc áp dụng thuật toán đó cho nhiều dạng biểu thức. Suy luận kiểu có nhiều đặc tính chung với các thuật toán mà được sử dụng trong chương trình dịch và môi trường lập trình để xác định các tính chất của chương trình. Chẳng hạn, một số thuật toán phân tích bí danh hữu dụng, mà tìm cách xác định xem liệu hai con trỏ có thể trỏ đến cùng một vị trí hay không, có cùng một lược đồ chung như thuật toán suy luận kiểu.

Đa hình và ghi đè

Có ba dạng đa hình: đa hình tham số, đa hình ad hoc (một khái niệm khác của ghi đè), và đa hình kiểu con. Hai kiểu đầu được xét trong chương này, để lại đa hình kiểu con trong các chương sau khi xét đến các ngôn ngữ hướng đối tượng. Đa hình tham số có thể ẩn, như trong ML, hoặc tường minh như với các C++ templates. Có hai cách cài đặt đa hình tham số, một cách trong đó việc biểu diễn dữ liệu như nhau được dùng cho nhiều kiểu dữ liệu và cách khác ở đó việc khởi tạo đoạn chương trình tham số tường minh được dùng để sánh với mỗi biểu diễn dữ liệu khác nhau.

Sự khác biệt giữa đa hình tham số và ghi đè là đa hình tham số cho phép một thuật toán cho nhiều kiểu dữ liệu, trong khi đó ghi đè dùng nhiều thuật toán khác nhau. Chẳng hạn, hàm + được ghi đè trong nhiều ngôn ngữ lập trình. Trong biểu thức cộng hai số nguyên, thuật toán phép cộng số nguyên được sử dụng. Trong phép cộng hai số phẩy động, thuật toán hoàn toàn khác được sử dụng để tính tổng.

Khai báo kiểu và Đồng nhất kiểu.

Chúng ta đã bàn đến các khai báo kiểu trong suốt và không trong suốt. Trong khai báo không trong suốt, tên kiểu cho một kiểu nào đó sẽ khác biệt với mọi kiểu đã có. Trong khai báo trong suốt, tên khai báo là đồng nghĩa cho kiểu khác. Cả hai dạng đều được sử dụng trong nhiều ngôn ngữ lập trình

Bài tập

3.1 Các kiểu ML

Giải thích kiểu ML cho mỗi khai báo sau đây:

- a. `fun a(x,y) = x+2*y;`
- b. `fun b(x,y) = x+y/2.0;`
- c. `fun c(f) = fn y => f(y);`
- d. `fun d(f,x) = f(f(x));`
- e. `fun e(x,y,b) = if b(y) then x else y;`

Bạn có thể nhập các biểu thức này vào bộ biên dịch ML để xác định kiểu, viết giải thích ngắn chỉ ra bạn hiểu tại sao hàm lại có kiểu đó.

3.2 Sắp xếp đa hình

Hàm này, sẽ sắp xếp chèn trên danh sách, có các đối số là hàm so sánh less và một danh sách l gồm các phần tử cần sắp xếp. Đoạn code được dịch và chạy đúng:

```

fun sort(less, nil) = nil |
    sort(less, a: :l) =
        let
            fun insert(a, nil) = a: :nil |
                insert(a, b: :l) = if less(a,b) then a: :(b: :l)
                                   else b:: insert(a, l)
        in
            insert(a, sort(less, l))
        end;

```

Kiểu của hàm đó là gì? Giải thích ngắn gọn, bao gồm cả kiểu của hàm phụ insert. Bạn không cần chạy thuật toán trên đoạn code này, chỉ cần giải thích vì sao người lập trình ML thông thường mong muốn đoạn code kiểu đó.

3.3 Kiểu thu dọn rác

Ngôn ngữ D cho ép chuyển kiểu mà ở đó một biểu thức có một kiểu có thể được xem như có kiểu khác. Chẳng hạn, nếu x là biến kiểu nguyên, thì $(\text{string})x$ là biểu thức kiểu string. Không có việc chuyển kiểu nào cả. Giải thích nó ảnh hưởng như thế nào đến việc thu dọn rác của D.

Đề đơn giản, giả sử D là ngôn ngữ tương tác thông thường với các số nguyên, số thực, cặp và con trỏ. Bạn không cần xét các đặc trưng khác của ngôn ngữ.

3.4 Điểm bất động đa hình

Điểm bất động của hàm f là giá trị sao cho $x = f(x)$. Ở đây có mối liên hệ giữa đệ qui và điểm bất động mà mô tả bởi định nghĩa ML của hàm giai thừa factorial: $\text{int} \rightarrow \text{int}$:

```

fun Y f x = f(Yf)x;

fun F f x = if x = 0 then 1 else x*f(x-1);

val factorial = Y F;

```

Hàm đầu Y là phép toán điểm bất động. Hàm thứ hai F là hàm trên các hàm mà điểm bất động của nó của nó là factorial. Cả hai đều là các hàm: sử dụng cú pháp ML $\text{fn } x \rightarrow \dots$ for $\lambda x \dots$, chúng ta sẽ viết hàm F như sau:

```

fun F(f) = fn x =>
    if x=0 then 1 else x*f(x-1)

```

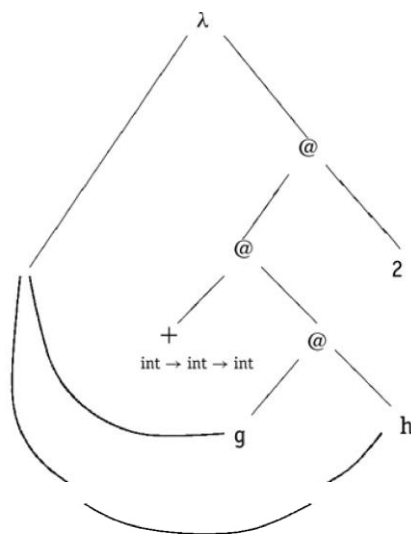
F là hàm mà, khi áp dụng vào thuật toán f , trả về hàm mà khi áp dụng vào x , có giá trị được cho bởi biểu thức nếu $x = 0$ thì bằng 1 nếu không thì bằng $x*f(x-1)$.

- Chương trình dịch ML sẽ kết luận kiểu gì cho F?
- Chương trình dịch ML sẽ kết luận kiểu gì cho Y?

3.5 Đồ thị cú pháp

Sử dụng đồ thị cú pháp sau tính kiểu ML cho hàm số:

```
fun f(g,h)=g(h) + 2;
```

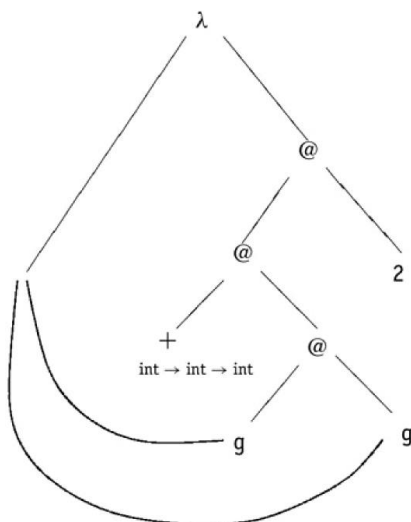


3.6 Đồ thị cây cú pháp

Sử dụng cây cú pháp sau để thực hiện các bước của thuật toán suy luận kiểu ML dựa trên khai báo hàm:

```
fun f(g)=g(g) + 2;
```

Đầu ra của kiểm tra kiểu là gì?



3.7 Suy luận kiểu và lỗi

Kiểu của hàm ML sau là gì?

```
fun append(nil, l) = l
```

```
| append(x :: l, m) = append(l, m);
```

Viết một hoặc hai câu để giải thích ngắn gọn và không hình thức tại sao append có kiểu như bạn đưa ra. Hàm này có mục đích là bổ sung một danh sách vào một danh

sách khác. Tuy nhiên nó có lỗi. Biết kiểu của hàm này giúp người lập trình tìm ra lỗi như thế nào?

3.8 Suy luận kiểu và bắt lỗi

Hàm rút gọn thực hiện phép toán hai ngôi, dưới dạng một hàm f và một danh sách, và tạo ra kết quả là sự kết hợp mọi phần tử trong danh sách bằng việc sử dụng một phép toán hai ngôi. Chẳng hạn,

`reduce plus [1,2,3] = 1 + 2 + 3 = 6`

nếu `plus` được định nghĩa bởi

`fun plus (x, y : int) = x+y`

Bạn của bạn sẽ thử học ML và viết hàm rút gọn `reduce`. Đây là định nghĩa không đúng của anh ta:

`fun reduce(f, x) = x`

`| reduce(f, (x : y)) = f(x, reduce(f, y));`

Anh ta nói với bạn rằng không biết cái gì sẽ trả về đối với danh sách rỗng, nhưng nó cần phải làm việc với danh sách khác rỗng. Nếu danh sách có một phần tử, thì mệnh đề thứ nhất sẽ trả về chính nó. Nếu danh sách có nhiều hơn một phần tử, thì mệnh đề thứ hai sẽ sử dụng hàm f . Điều này trông như giải thích hợp lý, nhưng kiểm tra kiểu sẽ cho bạn thông báo sau:

`val reduce = fn : (((* 'a list) -> 'a list) * 'a list) -> 'a list`

Bạn sử dụng kiểu này để giải thích code của bạn đó sai như thế nào.

3.9 Đa hình trong C

Trong hàm min của C sau, kiểu `void` được sử dụng trong kiểu của hai đối số. Tuy nhiên, hàm này có nghĩa và có thể áp dụng cho một danh sách các đối số mà ở đó `void` được thay bằng một kiểu khác. Nói cách khác, mặc dù kiểu trong C của hàm này không phải đa hình, hàm có thể được gán kiểu đa hình, nếu C có hệ thống kiểu đa hình. Sử dụng ký hiệu ML cho kiểu, viết kiểu cho hàm min này mà bao quát được cách mà min có thể được áp dụng có ý nghĩa cho các đối số của nhiều kiểu khác nhau. Giải thích, tại sao bạn tin tưởng hàm có kiểu mà bạn đã viết

```

int min (
    void *a[ ],    /* a is an array of pointers to data of unknown type */
    int n,         /* n is the length of the array */
    int (*less)(void*, void*) /* parameter less is a pointer to function */
) /* that is used to compare array elements */
{
    int i;
    int m;
    m=0;
    for (i = 1; i < n; i++)
        if (less(a[i], a[m])) m = i;
    return(m);
}

```

3.10 Hành vi lúc viết và lúc chạy

Các hàm ML sau có hành vi tính toán như nhau,

```
fun f(x) = not f(x);
```

```
fun g(y) = g(y) * 2;
```

vì ngoại trừ sự khác biệt khi viết, chúng ta có thể thay một hàm bằng hàm khác trong bất kỳ chương trình nào mà không thay đổi hành vi quan sát được của chương trình. Cụ thể hơn, giả sử ta tắt bộ kiểm tra kiểu ML và dịch chương trình dạng

$P[\text{fun } f(x) = \text{not } f(x)].$

Bất kể chương trình này làm cái gì, chương trình

$P[\text{fun } f(y) = f(y)*2]$ ta nhận được bằng cách thay định nghĩa một hàm bằng hàm khác, sẽ thực hiện chính xác như nhau. Cụ thể, nếu hàm thứ nhất không dẫn đến lỗi thời gian chạy, như cộng số nguyên vào nhau, hàm thứ hai cũng không thể.

- Kiểu ML của f là gì?
- Kiểu ML của g là gì?
- Đưa ra giải thích tại sao hai hàm này có cùng hành vi thời gian chạy.
- Vì hai hàm tương đương, có thể tốt hơn là cho chúng cùng một kiểu. Bạn nghĩ tại sao những người thiết kế thuật toán kiểu ML không làm nhiều hơn để thực hiện điều đó. Bạn có nghĩ là họ đã có lỗi không?

3.11 Kiểu động trong ML

Nhiều lập trình viên tin rằng ngôn ngữ lập trình gán kiểu trong thời gian chạy như Lisp hoặc Scheme có khả năng thể hiện nhiều hơn ngôn ngữ gán kiểu trong thời gian dịch như ML, vì ở đây không có hệ thống kiểu “để nhận được theo cách của bạn”. Mặc dù có nhiều tình huống, mà ở đó tính linh hoạt của Lisp hoặc Scheme là ưu thế rất lớn, chúng ta cũng có thể xét theo khía cạnh ngược lại. Đặc biệt, ML là có khả

năng thể hiện lớn hơn Lisp và Scheme, vì chúng ta có thể kiểu dữ liệu ML cho các biểu thức Lisp hoặc Scheme.

Sau đây là khai báo kiểu cho Lisp thuần túy ban đầu:

```
datatype LISP = Nil
    | Symbol of string
    | Number of int
    | Cons of LISP * LISP
    | Function of (LISP -> LISP)
```

Mặc dù ta có thể sử dụng ký hiệu nil thay cho kiểu nguyên thủy Nil, nhưng sẽ thuận tiện hơn nếu xử lý Nil riêng biệt.

Viết khai báo ML cho hàm Bạn không cần chạy thuật toán trên đoạn code này, chỉ cần giải thích vì sao người lập trình ML thông thường mong muốn đoạn code kiểu đó.

3.12 Xử lý lỗi

```
exception GCD_ERROR
```

```
fun checked_gcd (m, n) =
  let
    val (d, a, b) = ggcd (m, n)
  in
    if m mod d = 0 andalso n mod d = 0 andalso d = a*m+b*n then
      d
    else
      raise GCD_ERROR
  end
```

```
fun rev_gcd (m, n) =
  let
    val (d, a, b) = ggcd (m, n)
  in
    if d = a*m+b*n then
      a
    else
      raise GCD_ERROR
  end
```

```
exception Factorial
local
  fun fact 0 = 1
    | fact n = n * fact (n-1)
in
  fun checked_factorial n =
    if n >= 0 then
      fact n
    else
```

```

        raise Factorial
    end

```

3.13 Tính toán tối ưu nhờ xử lý lỗi

```

fun prod(LF x) = x | prod(ND(x,y)) = prod(x)*prod(y);
fun prod(tree) =
    let exception Zero
        fun p(LF x) = if x = 0 then (raise Zero) else x
            | p(ND(x,y)) = p(x)*p(y)
        in
            p(tree) handle Zero => 0
        end;
val t:int tree = ND(ND(LF 0,LF 2),LF 3);

```

3.14 Kiểu tham chiếu

```

fun f (x : int ref) =
    ( x := !x+1; !x );
val y = ref 0;
f(y) + !y;
fun f (z : int) =
    let val x = ref z in
        x := !x+1; !x
    end;
val y = ref 0;
f(!y) + !y;

```

BÀI 4: PHẠM VI HÀM VÀ QUẢN LÝ BỘ NHỚ

Trong chương này quản lý bộ nhớ đối với ngôn ngữ cấu trúc khối sẽ được mô tả bằng cấu trúc dữ liệu thời gian thực mà được sử dụng trong cài đặt tham chiếu đơn giản. Các đặc trưng của ngôn ngữ lập trình mà tạo nên liên kết giữa các tên trong chương trình với các vị trí bộ nhớ quan tâm là phạm vi, mà cho phép hai tên cú pháp khác nhau tham chiếu đến hai vị trí khác nhau, và các lời gọi hàm, mà mỗi lời gọi yêu cầu một vùng nhớ mới ở đó lưu trữ các tham số của hàm và các biến cục bộ. Một số chủ đề quan trọng trong chương này là truyền tham số, truy cập biến tổng thể và tối ưu bộ nhớ gắn kết với kiểu gọi hàm chuyên biệt gọi là tail call. Chúng ta sẽ thấy quản lý bộ nhớ trở nên phức tạp trong các ngôn ngữ có các khai báo hàm lồng nhau mà cho phép hàm được truyền như các tham số hoặc trả về như kết quả của lời gọi hàm.

4.1 Ngôn ngữ cấu trúc khối

Hầu hết các ngôn ngữ lập trình hiện đại cung cấp một dạng khối nào đó. Khối là một vùng của đoạn chương trình, được xác định bởi đánh dấu bắt đầu và kết thúc, mà chứa các khai báo cục bộ trong vùng đó. Sau đây là một số dòng mã C minh họa cho ý tưởng đó.

```

outer {
  int x = 2;
  {
    int y = 3;
    x = y+2;
  }
}
  
```

Trong đoạn code này có hai khối. Mỗi khối bắt đầu bằng mở ngoặc trái { và kết thúc bằng đóng ngoặc phải }. Khối ngoài được bắt đầu bằng mở ngoặc móc thứ nhất và đóng ngoặc móc phải sau cùng. Khối trong được lồng trong khối ngoài. Nó bắt đầu bằng mở ngoặc móc thứ hai và đóng ngoặc móc phải thứ nhất. Biến x được khai báo trong block ngoài và biến y trong block trong. Một biến được khai báo bên trong một block được coi là cục bộ đối với block đó. Một biến được khai báo trong một block bao bọc một block đang xét được coi là tổng thể của block đó. Trong ví dụ này, x là cục bộ đối với block ngoài, y là cục bộ đối với block trong và x là tổng thể đối với block trong.

C, Pascal, ML là các ngôn ngữ cấu trúc khối. Các khối được xác định bởi {...} trong C, *begin ... end* trong Pascal và *let...in...end* trong ML. Thân của các thủ tục hoặc hàm cũng là các khối trong các ngôn ngữ lập trình đó.

Cơ chế quản lý bộ nhớ liên kết với cấu trúc khối cho phép các hàm được gọi đệ qui.

Các phiên bản Fortran được sử dụng rộng rãi trong những năm 1960 và 1970 không là cấu trúc khối. Trong Fortran kinh điển, mỗi biến bao gồm các tham số của mỗi thủ tục (được gọi là subroutine trong Fortran) được gán với một vị trí bộ nhớ cố định. Điều đó làm cho không thể gọi thủ tục đệ qui, hoặc trực tiếp hoặc gián tiếp. Nếu trong Fortran, procedure P gọi Q và Q gọi R và sau đó R thủ gọi P, thì lần gọi P thứ hai là không cho phép. Giả sử P

được gọi lần hai, thì trong chuỗi gọi đó, lần gọi hai sẽ ghi đè lên các tham số và địa chỉ trả về của lần gọi thứ nhất. Điều này làm cho nó không thể gọi để trả về kết quả một cách đúng đắn.

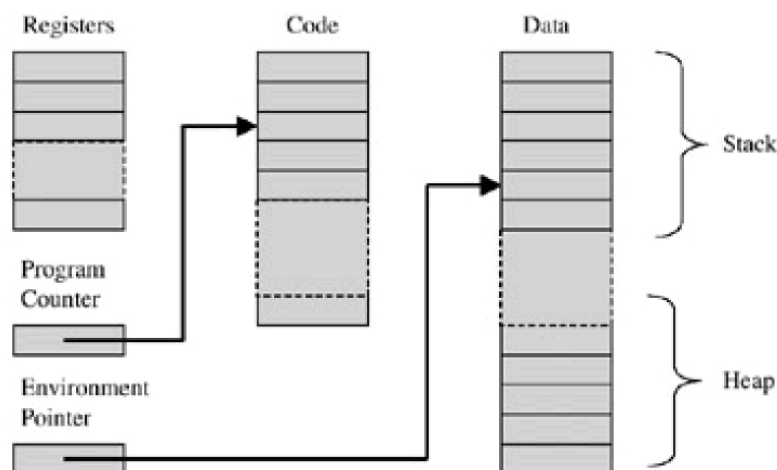
Các ngôn ngữ cấu trúc khối được đặc trưng bởi các tính chất sau:

- Các biến mới có thể được khai báo ở các vị trí khác nhau trong chương trình
- Mỗi khai báo được nhìn thấy bên trong một vùng nào đó của chương trình được gọi là block. Block có thể lồng nhau, nhưng không được chồng cắt nhau từng phần. Nói cách khác, hai block chứa một biểu thức hoặc các lệnh chung nào đó, thì một block sẽ chứa trọn vẹn trong block kia.
- Khi chương trình bắt đầu thực hiện các chỉ lệnh chứa trong một block trong thời gian thực, bộ nhớ được cấp cho các biến được khai báo trong block.
- Khi chương trình thoát ra khỏi block, một số hoặc toàn bộ bộ nhớ cấp cho các biến khai báo trong block được thu hồi.
- Định danh mà không khai báo trong block hiện thời được coi là tổng thể đối với block và được tham chiếu đến thực thể có tên đó được khai báo trong block bao bọc gần nhất.

Có vẻ ngạc nhiên là hầu hết các phức tạp liên quan đến việc truy cập các biến tổng thể. Tuy nhiên, điều này trên thực tế là kết quả của việc quản lý bộ nhớ dựa trên ngăn xếp. Ngăn xếp được sử dụng làm cho dễ dàng cấp và truy cập các biến cục bộ. Trong khi các biến cục bộ được đặt gần, thì các biến tổng thể được cất trong ngăn xếp dưới một số bản ghi kích hoạt.

Mô hình máy tính đơn giản

Chúng ta sử dụng mô hình máy tính đơn giản trên Hình sau để xem xét việc quản trị bộ nhớ trong các ngôn ngữ cấu trúc khối.



Hình vẽ 4.1 : Ngăn xếp chương trình

Mô hình máy trên Hình 4.1 tách bộ nhớ code khỏi bộ nhớ dữ liệu. Bộ đếm chương trình lưu trữ địa chỉ của chỉ lệnh hiện tại của chương trình và được tăng một cách bình thường sau mỗi chỉ lệnh. Khi chương trình vào một block mới, một bản ghi kích hoạt activation record chứa bộ nhớ cho các biến cục bộ khai báo trong block được bổ sung thêm vào ngăn xếp thời gian chạy (run-time stack – được vẽ trên đỉnh bộ nhớ dữ liệu), và con trỏ môi trường được thiết lập đến bản ghi mới này. Khi chương trình thoát khỏi khối, bản ghi kích hoạt đó được xóa khỏi ngăn xếp và con trỏ môi trường sẽ được đặt lại vị trí cũ. Chương trình có thể lưu lại dữ liệu, mà có thể tồn tại lâu hơn sau khi thực hiện khối hiện thời, trên đồng heap. Trên thực tế, bản ghi kích hoạt được cấp mới nhất là cái đầu tiên sẽ bị thu hồi, thường được gọi là stack discipline. Mặc dù hầu hết các ngôn ngữ cấu trúc khối được cài đặt bởi ngăn xếp, các hàm bậc cao có thể làm cho stack discipline bị lỗi.

Tuy Hình 4.1 có một số thanh ghi, nói chung được dùng để lưu trữ địa chỉ và dữ liệu trong thời gian ngắn, chúng ta sẽ không bàn đến thanh ghi hoặc chỉ lệnh mà có thể lưu trữ trên đoạn code của bộ nhớ.

Cài đặt tham chiếu. Cài đặt tham chiếu là cài đặt ngôn ngữ mà được thiết kế để xác định hành vi của ngôn ngữ. Nó không cần phải là cài đặt hiệu quả. Mục đích của chương này là cho bạn thông tin về việc các khối được cài đặt như thế nào trong hầu hết các ngôn ngữ lập trình để bạn có thể hiểu khi nào bộ nhớ cần được cấp phát, kiểu dữ liệu nào được lưu trữ trên ngăn xếp thời gian chạy, làm thế nào để chương trình đang thực hiện truy cập đến các bộ nhớ dữ liệu khi cần thiết. Chúng ta sẽ làm điều đó bằng cách mô tả cài đặt tham chiếu. Vì mục đích của chúng ta là hiểu chương trình chứ không phải xây dựng chương trình dịch, chương trình tham chiếu này sẽ đơn giản và trực tiếp. Các phương pháp hiệu quả hơn để làm nhiều việc mà được mô tả trong chương này, tùy thuộc vào các ngôn ngữ cụ thể, có thể tìm trong các cuốn sách về chương trình dịch.

Ghi chú về C.

Ngôn ngữ lập trình C được thiết kế làm cho C dễ dịch và dễ thực hiện, tránh một số kỹ thuật phạm vi tổng quát và quản trị bộ nhớ mô tả trong chương này. Hiểu các trường hợp tổng quát xét ở đây sẽ cho các lập trình viên C hiểu một số cách mà ở đó C đơn giản hơn các ngôn ngữ khác. Thêm vào đó, các lập trình viên C, người mà muốn hiệu quả trong việc truyền hàm và môi trường của nó cho hàm khác, có thể sử dụng ý tưởng được mô tả trong chương này cho chương trình của họ.

Một số cài đặt thương mại của C và C++ thực tế hỗ trợ các tham số hàm và giá trị trả về với việc giữ phạm vi tĩnh bởi việc sử dụng các bao đóng (Sẽ bàn đến trong mục 4.4). Thêm vào đó thư viện C++ Standard Template Library cung cấp dạng bao đóng hàm như nhiều lập trình viên dùng chúng một cách hữu ích cho các đối số hàm và giá trị trả về.

4.2 Khối In-line

Khối in-line là khối mà không là thân của một hàm hoặc một thủ tục. Trước hết chúng ta sẽ tìm hiểu khối in-line như trường hợp đơn giản hơn so với khối gắn kết với lời gọi hàm.

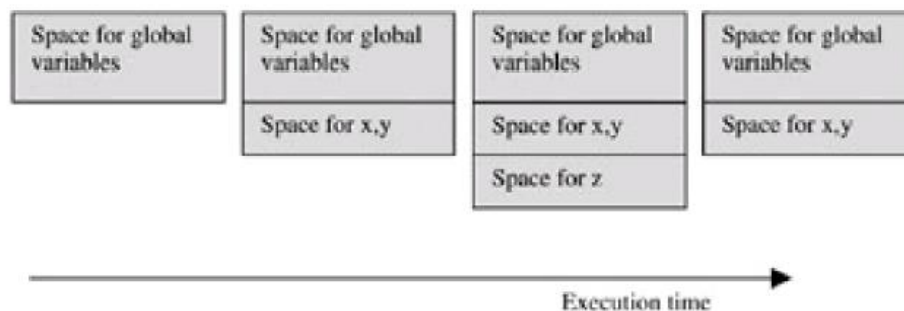
4.2.1 Bản ghi kích hoạt và các biến cục bộ

Khi chương trình đang chạy đến khối in-line, không gian cần được cấp cho các biến khai báo trong khối. Chúng ta làm điều đó bằng cách cấp bộ nhớ được gọi là bản ghi kích hoạt (activation record) trên ngăn xếp thời gian chạy. Bản ghi kích hoạt đôi khi được gọi là khung ngăn xếp (stack frame).

Để thấy rõ nó làm việc như thế nào, xét ví dụ đoạn code sau. Nếu code này là một phần của chương trình lớn hơn, ngăn xếp này có thể chứa không gian cho các biến khác, trước khi khối này được thực hiện. Khi bắt đầu vào khối ngoài, bản ghi kích hoạt chứa không gian cho x và y được đẩy vào ngăn xếp. Khi các lệnh đặt giá trị cho x và y sẽ được thực hiện, làm cho các giá trị của x và y được lưu trên thanh ghi kích hoạt. Khi bắt đầu vào khối trong, bản ghi kích hoạt khác chứa không gian cho z được bổ sung vào ngăn xếp. Sau khi giá trị của z gán, bản ghi kích hoạt chứa giá trị này sẽ được lấy ra khỏi ngăn xếp. Cuối cùng, khi thoát khỏi khối ngoài, bản ghi kích hoạt chứa không gian cho x và y được lấy ra khỏi ngăn xếp.

```
{ int x=0;
  int y=x+1;
  { int z=(x+y)*(x-y);
    };
};
```

Chúng ta có thể thấy rõ điều đó qua việc sử dụng một dãy các hình vẽ về ngăn xếp. Như trên Hình 4.1, ngăn xếp được chỉ ra là tăng giảm trong bộ nhớ trên Hình 4.2 sau



Hình 4.2 : Ngăn xếp tăng, giảm trong quá trình thực hiện chương trình.

Tối ưu đơn giản bao gồm kết hợp các khối lồng nhau nhỏ vào một khối duy nhất. Đối với chương trình trong ví dụ trước, có thể tiết kiệm thời gian trong việc đẩy vào và lấy ra khối trong cho z , bằng cách cho z được lưu trong cùng bản ghi kích hoạt như đối với x và y . Tuy nhiên, vì chúng ta muốn thể hiện trường hợp tổng quát bằng việc sử dụng các ví dụ nhỏ, chúng ta không sử dụng tối ưu này trong khi bàn tiếp về việc cấp bộ nhớ ngăn xếp. Trong mọi ví dụ chương trình đang xét, chúng ta giả thiết rằng bản ghi kích hoạt mới được cấp trên ngăn xếp thời gian chạy mỗi khi chương trình bắt đầu khối.

Số vị trí mà cần được cấp trong thời gian chạy phụ thuộc vào số biến khai báo trong khối và kiểu của chúng. Vì số lượng bộ nhớ này được biết trước trong thời gian dịch, chương trình dịch cần xác định định dạng của mỗi bản ghi kích hoạt và lưu thông tin này như một phần của mã được dịch.

Các kết quả trung gian

Nói chung, bản ghi kích hoạt có thể chứa không gian cho các kết quả trung gian. Đây là những giá trị mà không phải là các tên cho trước trong code, nhưng có thể cần lưu tạm thời. Chẳng hạn, bản ghi cho khối sau:

```
{ int z = (x+y)*(x-y);
}
```

có thể có dạng

Space for z
Space for x+y
Space for x-y

vì các giá trị của các biểu thức con $x+y$ và $x-y$ có thể được tính toán và lưu trữ ở đâu đó trước khi chúng được nhân.

Trên các máy tính hiện đại, ở đó có đủ các thanh ghi để nhiều kết quả trung gian có thể lưu trữ trên thanh ghi mà không đặt trên ngăn xếp. Tuy nhiên, vì việc cấp thanh ghi là kỹ thuật cài đặt mà không ảnh hưởng đến thiết kế ngôn ngữ lập trình, chúng ta không bàn luận về thanh ghi và việc cấp thanh ghi.

Phạm vi và thời gian sống

Quan trọng là cần phân biệt phạm vi khai báo và thời gian sống của vị trí cấp.

- Phạm vi: vùng văn bản mà ở đó khai báo được nhìn thấy.
- Thời gian sống là quãng thời gian trong khi chạy chương trình, ở đó vị trí bộ nhớ được cấp như kết quả của một khai báo nào đó.

Chúng ta có thể so sánh thời gian sống và phạm vi qua ví dụ sau, các gạch dọc được sử dụng để chỉ ra điểm vào và điểm ra của từng khối.

```

{ int x = ... ;
  { int y = ... ;
    { int x = ... ;
      ....
    };
  };
};

```

Trong ví dụ này, khai báo bên trong của x giấu khai báo bên ngoài của nó. Khối bên trong được gọi là *lỗi hồng trong phạm vi* của khai báo bên ngoài của x , vì x bên ngoài không thể truy cập được ở trong khối bên trong. Ví dụ này chỉ ra rằng thời gian sống không trùng với phạm vi, vì thời gian sống của c bên ngoài bao gồm cả thời gian khi khối bên trong được thực hiện, nhưng phạm vi của x bên ngoài không bao gồm phạm vi của cái bên trong.

Các khối và các bản ghi kích hoạt đối với ML

Qua việc tranh luận về các khối và các bản ghi kích hoạt, chúng ta tuân theo qui ước là khi chương trình bước vào khối mới, bản ghi kích hoạt mới được cấp trên ngăn xếp thời gian chạy. Trong mã ML mà có một dãy các khai báo, chúng ta sẽ xử lý từng khai báo như khối riêng biệt. Chẳng hạn, trong đoạn code sau

```

fun f(x) = x+1;
fun g(y) = f(y) + 2;
g(3);

```

Chúng ta coi khai báo của f là một khối và khai báo của g là khối khác bên trong khối bên ngoài. Nếu đoạn code này không nằm trong một cấu trúc nào khác, thì các khối này cả hai sẽ kết thúc tại điểm kết thúc chương trình.

Khi một biểu thức ML chứa các khai báo như một phần của cấu trúc *let-in-end*, chúng ta coi các khai báo này như một phần của cùng một khối. Chẳng hạn, xét biểu thức ví dụ sau

```

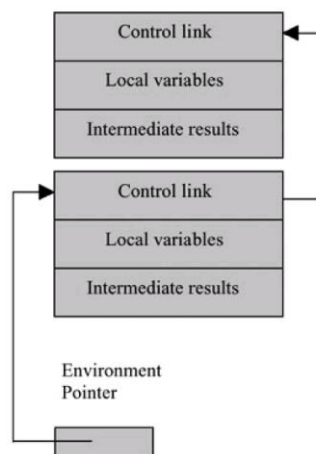
let fun g(y) = y+3
    fun h(z) = g(g(z))
in
    h(3)
end;

```

Biểu thức này chứa khối, bắt đầu với *let* và kết thúc ở *end*. Khối này chứa hai khai báo, các hàm g và h , và một biểu thức $h(x)$, gọi một trong các hàm đó. Cấu trúc *let ... in ... end* gần tương đương với $\{...\}$ trong C. Khác biệt cú pháp chính là các khai báo mà xuất hiện giữa các từ khóa *let* và *in*, và các biểu thức sử dụng các khai báo đó xuất hiện giữa các từ khóa *in* và *end*. Vì các khai báo của các hàm g và h xuất hiện trong cùng một khối, các tên g và h sẽ được cho các giá trị trong cùng một bản ghi kích hoạt.

4.2.2 Các biến tổng thể và các liên kết điều khiển

Vì các bản ghi khác nhau có kích thước khác nhau, các thao tác đẩy vào các bản ghi kích hoạt từ ngăn xếp thời gian chạy, lưu con trỏ trong bản ghi kích hoạt đến đỉnh của bản ghi trước đó. Con trỏ đến đỉnh của bản ghi trước đó này được gọi là liên kết điều khiển control link, như đây là liên kết mà theo đó, khi điều khiển trả về đến các chỉ lệnh của khối trước đó. Điều đó cho ta một cấu trúc được nêu trong Hình 4.3. Một số tác giả gọi liên kết điều khiển này là liên kết động, vì liên kết điều khiển này đánh dấu một dãy động các lời gọi hàm được tạo ra khi thực hiện chương trình.



Hình 4.3 Bản ghi kích hoạt với liên kết điều khiển

Khi bản ghi kích hoạt mới được bổ sung vào ngăn xếp, liên kết điều khiển của bản ghi kích hoạt mới được gán cho giá trị trước đó của biến môi trường, và biến môi trường được cập nhật để trỏ đến bản ghi kích hoạt mới. Khi bản ghi kích hoạt được lấy ra khỏi ngăn xếp, biến môi trường được khởi tạo lại dựa theo bởi liên kết điều khiển từ bản ghi kích hoạt.

Đoạn mã đẩy vào và lấy ra các bản ghi kích hoạt từ ngăn xếp được sinh ra bởi chương trình dịch trong thời gian dịch và trở thành một phần của mã dịch đối với chương trình. Vì kích thước của bản ghi kích hoạt có thể xác định từ văn bản của khối, chương trình dịch có thể sinh mã tính toán kích thước của bản ghi kích hoạt cho cả khối.

Khi biến tổng thể xuất hiện trong biểu thức, chương trình dịch cần sinh mã tìm vị trí của biến trong thời gian chạy. Tuy nhiên, chương trình dịch cần tính số khối giữa khối hiện tại và khối khối mà biến đó được khai báo. Điều này dễ dàng được xác định từ văn bản của chương trình. Thêm vào đó, vị trí tương đối của mỗi biến bên trong khối đó cũng được biết trong thời gian dịch. Vì vậy chương trình dịch có thể sinh mã tìm kiếm mà suy ra số liên kết được xác định trước.

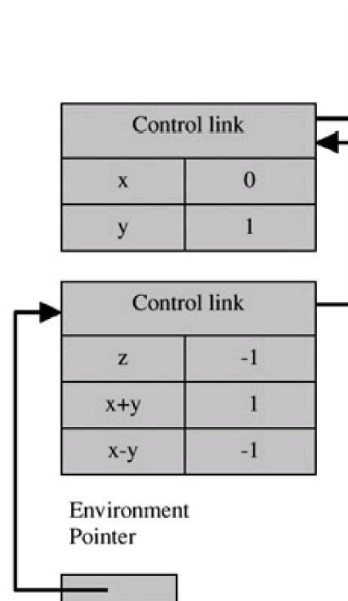
Ví dụ 4.1.

```

{ int x=0;
  int y=x+1;
  { int z=(x+y)*(x-y);
    };
  };

```

Khi các biểu thức $x+y$ và $x-y$ được tính toán trong khi thực hiện đoạn mã này, ngăn xếp thời gian thực sẽ có các bản ghi kích hoạt cho các khối bên trong và bên ngoài như trên Hình vẽ sau:



Trên máy tính dựa thanh ghi, mã máy được sinh ra từ chương trình dịch sẽ tìm các biến x và y , tải mỗi biến vào thanh ghi rồi cộng hai giá trị lại. Đoạn mã tải x sử dụng con trỏ môi trường tìm đỉnh của bản ghi kích hoạt hiện tại, sau đó tính thêm vị trí của x bằng cách cộng 1 vào vị trí lưu liên kết điều khiển của bản ghi kích hoạt hiện tại. Chương trình dịch sinh ra đoạn mã này bằng cách phân tích văn bản chương trình trong thời gian dịch. Biến x được khai báo cách khối hiện tại một khối, và x là biến đầu tiên khai báo trong khối. Vì vậy, liên kết điều khiển từ bản ghi kích hoạt hiện tại sẽ dẫn đến bản ghi kích hoạt chứa x , và vị trí của x là vị trí bên dưới vị trí ở đầu của khối đó. Các bước tương tự được sử dụng để tìm y ở vị trí thứ hai bên dưới đỉnh của thanh ghi kích hoạt đó. Mặc dù chi tiết có thể khác nhau từ chương trình dịch này đến chương trình dịch khác, nhưng điểm chính là chương trình dịch có thể xác định số liên kết điều khiển cần lần theo và vị trí tương đối của biến trong khối chứa nó từ mã nguồn. Đặc biệt không cần thiết phải lưu tên biến trong các bản ghi kích hoạt.

4.3 Các hàm và các thủ tục

Hầu hết các ngôn ngữ cấu trúc khối có các thủ tục hoặc các hàm mà chứa các tham số, các biến cục bộ, và thân gồm biểu thức bất kỳ hoặc dãy các câu lệnh. Chẳng hạn, sau đây là một số dạng giống C và Algol:

```

procedure P(<parameters>)    <type> f(<parameters>)
begin                        {
    <local variables>;        <local variables>;
    <procedure body>;        <function body>;
end;                        };

```

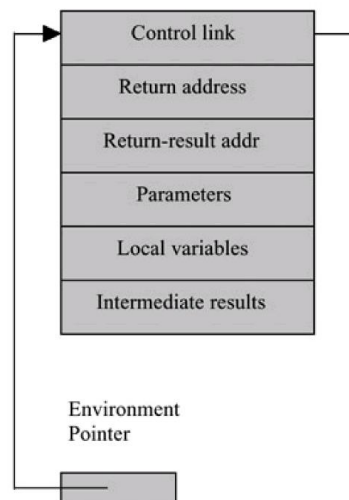
Sự khác biệt giữa thủ tục và hàm là hàm có trả giá trị về, còn thủ tục thì không. Trong hầu hết các ngôn ngữ, các hàm và thủ tục có thể có một số tác động phụ. Tuy nhiên, thủ tục chỉ có tác động phụ là lời gọi thủ tục là lệnh chứ không phải là biểu thức. Vì các hàm và các thủ tục có nhiều tính chất chung, chúng ta sử dụng thuật ngữ hàm như có thể thay đổi qua lại được trong phần cuối của chương. Ví dụ, bình luận có thể bàn về các tính chất của hàm, và sau đó ví dụ code có thể thể hiện các tính chất với một thủ tục nào đó. Cũng cần phải nhắc bạn rằng việc bàn luận này áp dụng cho cả các hàm và thủ tục trong nhiều ngôn ngữ lập trình, cho dù ngôn ngữ đó có xử lý thủ tục khác với hàm không.

4.3.1 Bản ghi kích hoạt cho các hàm

Bản ghi kích hoạt của khối hàm hoặc thủ tục cần phải có không gian cho các tham số và giá trị trả về. Vì một thủ tục có thể được gọi từ các lời gọi khác nhau, nó cũng cần phải lưu địa chỉ trả về, mà là vị trí của chỉ lệnh tiếp theo cần thực hiện sau khi thủ tục kết thúc. Đối với hàm, bản ghi kích hoạt cần phải chứa vị trí mà chương trình gọi chờ đợi có giá trị trả về của hàm.

Bản ghi kích hoạt liên kết với một hàm (xem Hình 7.4) cần chứa không gian cho các thông tin sau:

- Control link – Liên kết điều khiển: trở đến bản ghi kích hoạt trước trên ngăn xếp
- Access link – Liên kết truy cập, mà ta sẽ bàn sau trong mục 7.3.3.
- Return address – Địa chỉ trả về, cho địa chỉ của dòng lệnh đầu tiên cần thực hiện khi mà hàm kết thúc
- Return-result address – Địa chỉ kết quả trả về, vị trí mà ở đó lưu giá trị trả về của hàm
- Actual parameters - Các tham số thực tế của hàm
- Local variables – biến cục bộ khai báo trong hàm
- Temporary storage – bộ nhớ tạm thời cho các kết quả trung gian được tính toán với việc thực hiện hàm



Hình 4.4: Bản ghi kích hoạt gắn kết với lời gọi hàm

Thông tin này có thể được lưu theo thứ tự khác nhau và theo các cách khác nhau trong việc cài đặt các ngôn ngữ lập trình khác nhau. Cũng như đã lưu ý trước đây, nhiều chương trình dịch thực hiện một số tối ưu mà đặt một số giá trị này trong các thanh ghi. Để đảm bảo tính đúng đắn, chúng ta giả thiết rằng, không có thanh ghi nào được sử dụng và sáu thành phần của thanh ghi kích hoạt được lưu theo thứ tự đã nêu trước đó.

Mặc dù tên của các biến được loại bỏ trong quá trình dịch, chúng ta thường vẽ các bản ghi kích hoạt với tên của biến trong thanh ghi. Điều này được làm chỉ để ta hiểu được các hình vẽ.

Ví dụ 4.2

Chúng ta có thể thấy các bản ghi kích hoạt được bổ sung hoặc loại bỏ khỏi ngăn xếp thời gian chạy bằng cách theo dõi việc thực hiện hàm giai thừa quen thuộc.

```
fun fact(n) = if n <= 1 then 1 else n * fact(n-1);
```

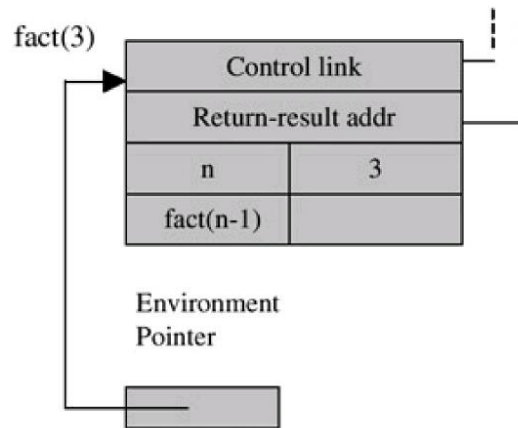
Giả thiết rằng một khối nào đó chứa biểu thức $\text{fact}(3)+1$, dẫn đến lời gọi $\text{fact}(3)$. Chúng ta giả sử rằng bản ghi kích hoạt của khối gọi chứa vị trí mà sẽ được sử dụng để lưu giá trị $\text{fact}(3)$ trước khi tính $\text{fact}(3) + 1$.

Tuy nhiên, chương trình dịch cần tính số khối giữa khối biến tổng thể xuất hiện trong biểu thức,

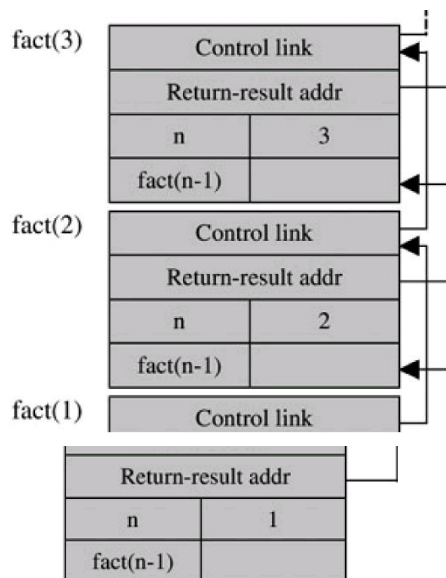
Bản ghi kích hoạt tiếp theo mà đặt trên đỉnh ngăn xếp sẽ là bản ghi kích hoạt cho lời gọi $\text{fact}(3)$. Trong bản ghi này sẽ chỉ rõ danh sách:

- Control link – liên kết điều khiển trở đến bản ghi kích hoạt của khối chứa lời gọi $\text{fact}(3)$,

- Return-result link – liên kết kết quả trả về trở đến vị trí trong bản ghi kích hoạt của khối gọi mà được cấp cho kết quả trung gian của biểu thức gọi $\text{fact}(3)+1$,
- Actual parameter value – giá trị tham số thực tế 3 được đặt tại vị trí cấp cho tham số hình thức n ,
- Vị trí được cấp cho giá trị trung gian $\text{fact}(n-1)$ mà luôn cần trong trường hợp $n > 0$.

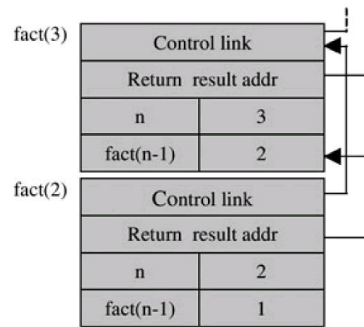


Sau khi bản ghi kích hoạt được đặt trên ngăn xếp, đoạn mã tính giai thừa được thực hiện. Vì $n > 0$, nên có lời gọi đệ qui đến $\text{fact}(2)$. Điều đó dẫn đến lời gọi đệ qui $\text{fact}(1)$, kết quả là có một dãy các bản ghi như chỉ ra trên hình vẽ sau đây.



Lưu ý rằng trong mỗi bản ghi kích hoạt ở dưới, địa chỉ kết quả trả về trở đến không gian được cấp trong bản ghi kích hoạt ở trên nó. Đến lượt $\text{fact}(1)$, chẳng hạn, kết quả trả về của lời gọi này cần được lưu trong bản ghi cho $\text{fact}(2)$. Tại thời điểm đó chỉ lệnh cuối cùng tính $\text{fact}(2)$ được thực hiện, nhân biến cục bộ n với kết quả trung gian $\text{fact}(1)$.

Chỉ lệnh cuối cùng của ví dụ này đưa ra tình huống, khi kết quả trả về của $\text{fact}(2)$ được đặt trong bản ghi kích hoạt của $\text{fact}(3)$, nhưng bản ghi kích hoạt của $\text{fact}(2)$ chưa được lấy ra khỏi ngăn xếp.



4.3.2 Truyền tham số

Tên tham số được sử dụng trong khai báo hàm được gọi là tham số hình thức. Khi hàm được gọi, biểu thức gọi tham số thực tế được sử dụng để tính giá trị tham số cho lời gọi đó. Sự khác biệt giữa tham số hình thức và thực tế được mô tả trong đoạn code sau.

```
proc p (int x, int y) {
    if (x > y) then ... else ... ;
    ...
    x := y*2 + 3;
    ...
}
p (z, 4*z+1);
```

Các định danh `x` và `y` là các tham số hình thức của thủ tục `p`. Các tham số thực tế trong lời gọi đến `p` là `z` và `4*z+1`.

Cách mà các tham số thực tế được tính toán và truyền cho hàm phụ thuộc vào ngôn ngữ lập trình và cơ chế truyền tham số mà nó sử dụng. Sự khác biệt chính giữa các cơ chế truyền tham số là

- Thời điểm mà tham số thực tế được tính toán
- Vị trí được sử dụng để lưu giá trị tham số

Hầu hết các ngôn ngữ lập trình tính toán các tham số thực tế trước khi thực hiện thân hàm, nhưng cũng có một số ngoại lệ. (Một nguyên nhân là tối ưu ngôn ngữ hoặc chương trình có thể muốn trì hoãn việc tính toán tham số hình thức, vì việc tính toán này có thể lâu và có thể không được sử dụng trong một số lời gọi). Trong số các cơ chế mà tính toán các tham số thực tế trước khi thực hiện thân hàm, thông thường là

- Truyền tham chiếu: truyền L-giá trị (địa chỉ) của tham số thực tế
- Truyền tham trị: truyền R-giá trị (nội dung của địa chỉ) của tham số thực tế

Nhắc lại là chúng ta đã bàn luận về L-giá trị và R-giá trị trong mục 3.4.5 phần liên quan đến ô tham chiếu ML (vị trí gán được) và phép gán. Chúng ta sẽ bàn truyền tham chiếu và truyền tham trị làm việc như thế nào chi tiết hơn dưới đây. Một cơ chế khác là truyền kết quả giá trị (pass-by-value-result) được xét đến trong phần bài tập.

Sự khác nhau giữa truyền tham trị và truyền tham chiếu là rất quan trọng đối với người lập trình bởi một số lý do sau:

- Tác động phụ: việc gán bên trong thân hàm có thể có các tác động phụ khác nhau khi truyền tham trị hay truyền tham chiếu
- Bí danh: bí danh xảy ra khi hai tên tham chiếu đến cùng một đối tượng hoặc cùng một vị trí. Bí danh có thể xảy ra khi hai tham số được truyền tham chiếu hoặc một tham số được truyền tham chiếu có cùng vị trí như biến tổng thể của thủ tục.
- Tính hiệu quả: Truyền tham trị có thể không hiệu quả đối với các cấu trúc lớn, nếu giá trị của cấu trúc lớn cần sao lưu. Truyền tham chiếu có thể kém hiệu quả hơn truyền tham trị đối với các cấu trúc nhỏ mà có thể đẩy trực tiếp vào ngăn xếp, vì khi tham số truyền tham chiếu, chúng ta cần lần theo con trỏ để lấy giá trị của chúng.

Ở đây có hai cách giải thích ngữ nghĩa của lời gọi theo tham chiếu và theo tham trị. Một là vẽ ra bức tranh bộ nhớ máy tính và ngăn xếp chương trình thời gian chạy, chỉ ra ngăn xếp có chứa bản sao của tham số thực tế hay tham chiếu đến nó. Cách giải thích khác thể hiện bằng cách dịch mã sang ngôn ngữ mà phân biệt giữa L-giá trị và R-giá trị. Chúng ta sẽ dùng cách thứ hai ở đây, vì vậy phần còn lại của chương sẽ cho bạn cơ hội làm việc với một số hình ảnh về ngăn xếp thời gian chạy.

Ngữ nghĩa Truyền tham trị.

Trong truyền tham trị, tham số thực tế được tính toán. Sau đó giá trị của tham số thực tế được lưu ở vị trí mới cấp cho tham số hàm. Chẳng hạn, xét định nghĩa và lời gọi hàm sau:

```
function f (x) = { x := x+1; return x };
... f(y) ...;
```

Nếu tham số được truyền tham trị và y là biến số nguyên, thì đoạn mã trên có cùng ý nghĩa như đoạn mã ML sau:

```
fun f (z : int) = let x = ref z in x := !x+1; !x end;
... f(!y) ...;
```

Như bạn có thể thấy từ khai báo kiểu, giá trị được truyền cho hàm f là biến nguyên. Số nguyên là R-giá trị của biến thực tế y, như chỉ ra trong biểu thức !y của lời gọi. Trong thân của hàm f, vị trí số nguyên mới được cấp và khởi tạo bằng R-giá trị của y.

Nếu giá trị của y bằng 0 trước khi gọi, thì giá trị của f(!y) bằng 1, vì hàm f tăng tham số lên và trả về giá trị của nó. Tuy nhiên, giá trị của y vẫn là 0 sau khi gọi, vì phép gán bên trong thân hàm f thay đổi nội dung chỉ của vị trí tạm thời.

Ngữ nghĩa Truyền tham chiếu

Trong truyền tham chiếu, tham số thực tế cần có L-giá trị. L-giá trị của tham số thực tế sau đó được gắn với tham số hình thức. Xét định nghĩa hàm và lời gọi giống như trong giải thích Truyền tham trị.

```
function f (x) = { x := x+1; return x };
... f(y) ...;
```

Nếu tham số được truyền tham chiếu và y là biến nguyên, thì đoạn mã này có cùng ý nghĩa như đoạn mã ML sau:

```
fun f (x : int ref) = ( x := !x+1; !x );
... f(y)
```

Như bạn có thể thấy từ khai báo kiểu, giá trị được truyền cho hàm f là tham chiếu nguyên (L-giá trị).

Nếu giá trị của y bằng 0 trước khi gọi, thì giá trị của f(!y) bằng 1, vì hàm f tăng tham số lên và trả về giá trị của nó. Tuy nhiên, không giống như trường hợp truyền tham trị, giá trị của y vẫn là 1 sau khi gọi, vì phép gán bên trong thân hàm f thay đổi giá trị của tham số thực tế.

Ví dụ 4.3

Sau đây là ví dụ, viết trên ngôn ngữ tựa Algol, mà kết hợp truyền tham chiếu và truyền tham trị.

```
fun f(pass-by-ref x : int, pass-by-value y : int)
begin
  x:= 2;
  y:= 1;
  if x = 1 then return 1 else return 2;
end;
var z : int;
z := 0;
print f(z,z);
```

Dịch đoạn mã trên sang ML ta có:

```
fun f(x : int ref, y : int) =
  let val yy = ref y in
    x := 2;
    yy := 1;
    if (!x = 1) then 1 else 2
  end;
val z = ref 0;
f(z, !z);
```

Đoạn mã này, xử lý các L-giá trị và R-giá trị một cách tường minh, chỉ ra rằng, đối với truyền tham chiếu ta truyền L-giá trị, tham chiếu nguyên z. Đối với truyền tham trị, ta truyền R-giá trị, nội dung !z của z. Truyền tham trị được gán cho vị trí tạm thời mới.

Với y truyền tham trị như viết bên trên, z được gán giá trị 2. Nếu y thay vào đó truyền tham chiếu, thì x và y là bí danh và z được gán giá trị 1.

Ví dụ 4.4

Sau đây là hàm mà kiểm tra xem hai tham số của nó có phải là bí danh không?

```
function (y,z){
  y:= 0;
  z :=0;
  y:= 1;
  if z = 1 then y :=0; return 1 else y :=0; return 0
}
```

Nếu y và z là các bí danh, thì đặt y bằng 1 sẽ kéo theo đặt z bằng 1 và hàm sẽ trả về 1. Ngược lại hàm sẽ trả về 0. Do đó, lời gọi f(x,x) có hành vi khác nếu các tham số được truyền tham trị so với các tham số được truyền tham chiếu.

4.3.3 Các biến tổng thể

Nếu định danh x xuất hiện trong thân của hàm, nhưng x không được khai báo bên trong hàm, thì giá trị của x phụ thuộc vào một khai báo nào đó ở bên ngoài của hàm. Trong tình huống đó, vị trí của x nằm bên ngoài bản ghi kích hoạt của hàm. Vì x cần được khai báo ở khối nào đó khác, truy cập đến biến tổng thể x bao gồm cả việc tìm bản ghi kích hoạt nằm đâu đó trên ngăn xếp.

Ở đây có hai qui tắc tìm khai báo của định danh tổng thể:

- Phạm vi tĩnh: định danh tổng thể tham chiếu đến tên đó mà được khai báo trong phạm vi đóng gần nhất của văn bản chương trình.
- Phạm vi động: định danh tổng thể tham chiếu đến định danh đó được gán kết với bản ghi kích hoạt mới nhất

Các định nghĩa này khá rắc rối để hiểu được, vì vậy cần đọc các ví dụ sau cẩn thận. Một khác biệt quan trọng giữa phạm vi tĩnh và động là tìm khai báo trong phạm vi tĩnh sử dụng quan hệ tĩnh (không thay đổi) giữa các khối trong văn bản chương trình. Ngược lại, phạm vi động sử dụng dãy thực tế các lời gọi mà được thực thi trong quá trình thực hiện động (thay đổi) của chương trình.

Mặc dù hầu hết các ngôn ngữ lập trình đa năng sử dụng phạm vi tĩnh cho khai báo các biến và các hàm, phạm vi động là khái niệm quan trọng mà được sử dụng trong các ngôn ngữ chuyên dụng và cho các cấu trúc chuyên dụng như biểu thức.

Dynamically Scoped**Statically Scoped**

Older Lisps

Newer Lisps, Scheme

TeX/LaTeX document languages

Algol and Pascal

Exceptions in many languages

C

Macros

ML

Other current languages

Ví dụ 4.5

Sự khác biệt giữa phạm vi tĩnh và động được thể hiện qua đoạn mã sau, mà chứa hai khai báo của x .

```
int x=1;
function g(z) = x+z;
function f(y) = {
    int x = y+1;
    return g(y*x)
};
f(3);
```

Lời gọi $f(3)$ dẫn đến lời gọi $g(12)$ bên trong hàm f . Điều này buộc biểu thức $x+z$ trong thân của g cần được tính toán. Sau khi gọi đến g , ngăn xếp thời gian chạy sẽ chứa các bản ghi kích hoạt cho các khai báo bên ngoài của x , khởi tạo của f , và khởi tạo của g , như chỉ ra trong thể hiện sau.

outer block	<table><tr><td>x</td><td>1</td></tr></table>	x	1		
x	1				
f(3)	<table><tr><td>y</td><td>3</td></tr><tr><td>x</td><td>4</td></tr></table>	y	3	x	4
y	3				
x	4				
g(12)	<table><tr><td>z</td><td>12</td></tr></table>	z	12		
z	12				

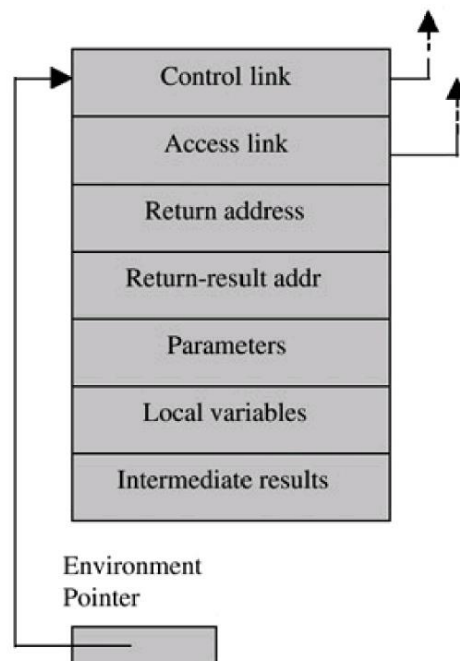
Tại điểm này, hai số nguyên tên x được lưu trên ngăn xếp, một của khối ngoài và một của khai báo x ở bên trong f . Trong phạm vi động, định danh x trong biểu thức $x+z$ sẽ được biên dịch như một biến từ trong bản ghi kích hoạt tạo mới nhất, ở đó $x = 4$. Trong phạm vi tĩnh, định danh $x+4$ sẽ tham chiếu đến khai báo x từ khối chương trình gần nhất, tìm trở lên từ chỗ $x+z$ xuất hiện trong văn bản chương trình. Trong phạm vi tĩnh, khai báo liên quan đó của x là khai báo của khối ngoài, với $x = 1$.

Liên kết truy cập được sử dụng để bảo trì phạm vi tĩnh

Liên kết truy cập của bản ghi kích hoạt trở đến bản ghi kích hoạt của khối bao nó gần nhất trong chương trình. Các khối trực tiếp (in-line) không cần đến liên kết truy cập, vì khối bao gần nhất chính là khối vừa mới vào – đối với khối trực tiếp, liên kết điều khiển trở đến khối khối bao gần nhất. Tuy nhiên đối với hàm số, khối bao gần nhất được xác định bởi hàm được khai báo ở đâu. Vì điểm khai báo thường khác với điểm ở đó hàm được gọi, nên liên kết truy cập trở đến bản ghi khác với liên kết điều khiển. Một số tác giả gọi liên kết truy

cập là liên kết tĩnh (static link), vì liên kết truy cập thể hiện cấu trúc lồng tĩnh của các khối trong chương trình nguồn.

Định dạng tổng quát của bản ghi kích hoạt với liên kết truy cập được chỉ ra trên Hình 4.5.

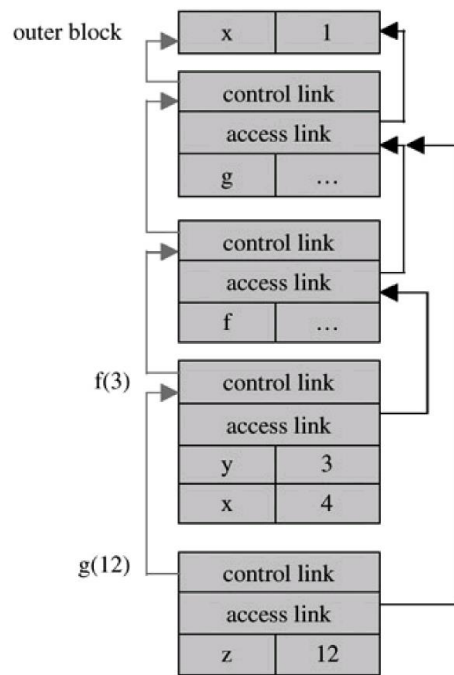


Hình 4.5. Bản ghi kích hoạt với liên kết truy cập cho hàm số có phạm vi tĩnh

Ví dụ 4.6

Chúng ta xem xét bản ghi kích hoạt và các liên kết truy cập cho đoạn mã trong Ví dụ 4.5, xử lý mỗi khai báo ML như một khối riêng.

Hình 4.6 chỉ ra ngăn xếp thời gian chạy sau khi gọi g bên trong thân hàm f. Như thường lệ, các liên kết điều khiển mỗi cái trỏ đến bản ghi kích hoạt trước đó trên ngăn xếp.



Hình 4.6 Ngăn xếp thời gian thực sau khi gọi g bên trong f

Các liên kết điều khiển ở bên trái để giành không gian cho các liên kết truy cập ở bên phải. Liên kết truy cập đối với mỗi khối trỏ đến bản ghi kích hoạt của khối bao gần nhất trong văn bản chương trình. Ở đây có một số điểm quan trọng trên hình vẽ minh họa này, mà theo qui ước chúng ta sẽ bắt đầu mỗi khối mới cho mỗi khai báo ML.

- Khai báo của g xảy ra bên trong phạm vi khai báo của x. Do đó liên kết truy cập cho khai báo g trỏ đến bản ghi kích hoạt cho khai báo của x.
- Khai báo của f tương tự ở bên trong khai báo của g. Do đó, liên kết truy cập cho khai báo của f trỏ đến bản ghi kích hoạt cho khai báo của g.
- LỜI gọi f(3) buộc cho bản ghi kích hoạt được cấp cho phạm vi liên kết với thân của f. Thân của f xảy ra bên trong phạm vi khai báo của f. Do đó liên kết truy cập cho f(3) trỏ đến bản ghi kích hoạt cho khai báo của f.
- LỜI gọi g(12) tương tự buộc cho bản ghi kích hoạt được cấp cho phạm vi liên kết với thân của g. Thân của g xảy ra bên trong phạm vi khai báo của g. Do đó liên kết truy cập cho g(12) trỏ đến bản ghi kích hoạt cho khai báo của g.
- Chúng ta tính toán giá trị của biểu thức $x+z$ bằng cách cộng giá trị của tham số z, lưu cục bộ trong bản ghi kích hoạt của g, với giá trị của biến tổng thể x. Chúng ta tìm vị trí của biến tổng thể x bằng cách lần theo liên kết truy cập của bản ghi cho g đến bản ghi kích hoạt liên kết với khai báo của g. Do đó ta theo liên kết kích hoạt trong bản ghi này để tìm bản ghi kích hoạt chứa biến x.

Như đối với các khối trực tiếp, chương trình dịch có thể xác định cần lần theo bao nhiêu liên kết truy cập và tìm ở đâu một biến bên trong một bản ghi kích hoạt tại thời điểm dịch. Các tính chất này có thể dễ dàng xác định từ cấu trúc của mã nguồn.

Tóm lại, liên kết điều khiển là liên kết đến bản ghi kích hoạt của khối trước (gọi). Liên kết truy cập là liên kết đến bản ghi kích hoạt của khối bao gần nhất trong văn bản chương trình. Liên kết điều khiển phụ thuộc vào hành vi động của chương trình, trong khi đó liên kết truy cập phụ thuộc chỉ vào dạng tĩnh của văn bản chương trình. Liên kết truy cập được dùng để tìm vị trí các biến tổng thể trong ngôn ngữ phạm vi tĩnh với các khối lồng nhau trong thời gian chạy.

Các liên kết truy cập chỉ cần trong các ngôn ngữ lập trình mà ở đó các hàm có thể được khai báo bên trong hàm hoặc các khối lồng nhau khác. Trong C, ở đó các hàm đều được khai báo ở phạm vi tổng thể ngoài nhất, các liên kết truy cập là không cần thiết.

4.3.4 Đệ qui đuôi

Trong mục này, chúng ta xét tối ưu chương trình dịch có ích gọi là khử đệ qui. Đối với các hàm đệ qui đuôi, mà sẽ mô tả sau đây, có thể tái sử dụng bản ghi kích hoạt cho lời gọi đệ qui của hàm. Điều này làm giảm kích thước không gian được dùng bởi hàm đệ qui.

Khái niệm ngôn ngữ lập trình chính mà chúng ta cần, là khái niệm lời gọi đuôi. Giả sử hàm f gọi hàm g . Hàm f và g có thể là các hàm khác nhau hoặc f và g có thể là cùng một hàm. Lời gọi hàm f trong thân hàm g được gọi là gọi đuôi (tail call) nếu g trả về kết quả của lời gọi f mà không cần bất cứ tính toán nào nữa. Chẳng hạn, trong hàm

```
fun g(x) = if x=0 then f(x) else f(x)*2
```

Lời gọi thứ nhất của f trong g là lời gọi đuôi, vì nó trả về giá trị của g chính bằng giá trị của lời gọi f . Lời gọi thứ hai đến f trong thân của g không phải là lời gọi đuôi, vì g thực hiện tính toán gồm giá trị trả về của f trước khi g trả về kết quả.

Hàm f được gọi là đệ qui đuôi, nếu mọi lời gọi đệ qui trong thân hàm f đều là lời gọi đuôi đến f .

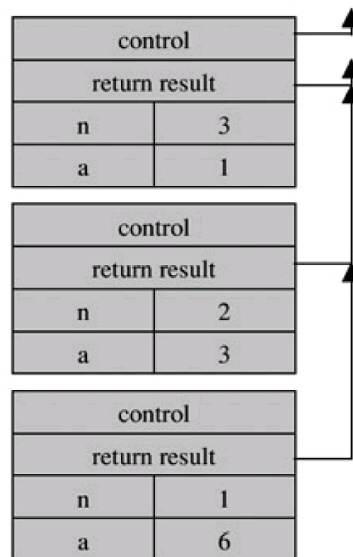
Ví dụ 4.7

Sau đây là hàm đệ qui đuôi tính toán giai thừa:

```
fun tfact(n,a) = if n <= 1 then a else tfact(n-1,n*a);
```

Cụ thể hơn, đối với mọi số nguyên n , $tfact(n,1)$ trả về $n!$. Chúng ta có thể thấy $tfact$ là hàm đệ qui đuôi, vì lời gọi duy nhất trong thân hàm $tfact$ đó là lời gọi đuôi.

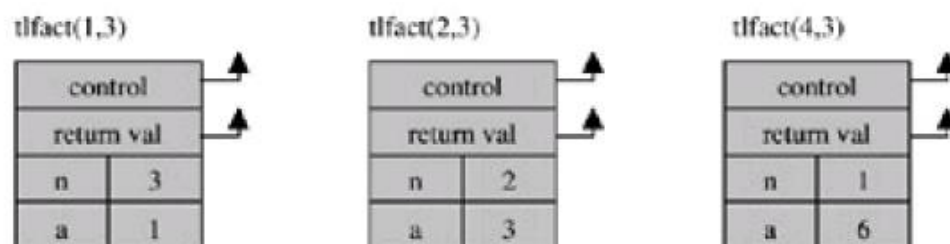
Ưu điểm của đệ qui đuôi là chúng ta có thể sử dụng cùng bản ghi kích hoạt cho mọi lời gọi đệ qui. Xét lời gọi $tfact(3,1)$. Hình 4.7 chỉ ra các phần liên quan đến tính toán nói bên trên:



Hình 4.7. Ba lời gọi đệ qui đuôi đến tlfact chưa tối ưu.

Sau lời gọi thứ ba kết thúc, nó truyền kết quả trả lại lời gọi thứ hai, mà sau đó truyền kết quả trả lại lời gọi thứ nhất. Chúng ta đơn giản hóa quá trình bằng cách cho lời gọi thứ ba trả kết quả về bản ghi kích hoạt mà tạo nên lời gọi từ đầu, tlfact(3,1). Khi thực hiện phương án này, lời gọi thứ ba kết thúc, ta có thể gỡ bản ghi kích hoạt cho lời gọi thứ hai, vì ta không cần nó nữa. Trên thực tế vì bản ghi kích hoạt cho lời gọi thứ nhất không cần nữa khi lời gọi thứ hai bắt đầu, ta có thể gỡ bỏ bản ghi kích hoạt thứ nhất ra khỏi ngăn xếp trước khi cấp bản ghi thứ hai. Sẽ tốt hơn, thay vì thu hồi bản ghi thứ nhất và sau đó cấp bản ghi thứ hai đồng nhất với bản ghi thứ nhất, ta sử dụng một bản ghi kích hoạt cho cả ba lời gọi.

Hình 4.8 chỉ ra cùng một bản ghi kích hoạt có thể được sử dụng ba lần cho ba lời gọi liên tiếp đến tlfact đệ qui đuôi như thế nào. Hình vẽ này chỉ ra nội dung bản ghi kích hoạt cho từng lời gọi. Khi lời gọi thứ nhất, tlfact(3,1) bắt đầu, bản ghi kích hoạt với tham số (1,3) được tạo ra. Khi lời gọi thứ hai tlfact(2,3) bắt đầu, giá trị tham số thay đổi sang (2,3). Khi đệ qui đuôi tái sử dụng bản ghi kích hoạt duy nhất cho mọi lời gọi hàm, sử dụng phép gán thay đổi giá trị các tham số của hàm cho mỗi lời gọi.



Hình 4.8. Ba lời gọi đến tlfact đệ qui đuôi có tối ưu

4.3.5 Đệ qui đuôi

Khi hàm đệ qui đuôi được dịch, nó có thể sinh ra mã mà tính toán giá trị hàm bằng việc sử dụng vòng lặp. Khi tối ưu này được sử dụng, hàm đệ qui đuôi có thể được tính toán chỉ bằng một bản ghi kích hoạt cho lời gọi đầu tiên và ghi kết quả của mọi lời gọi đệ qui, không cần mỗi bản ghi kích hoạt bổ sung cho mỗi lời gọi đệ qui. Chẳng hạn, hàm giai thừa đệ qui đuôi trước đó có thể được dịch sang mã cùng mục đích nhưng của hàm lặp như sau:

```
fun itfact(n,a) = ( while !n>0 do(a:=!n * !a;n:=!n-1);!a);
```

Bản ghi kích hoạt cho itfact trông giống như bản ghi kích hoạt cho tfact. Nếu ta xem các giá trị của n và a trên mỗi lần lặp, ta sẽ thấy chúng thay đổi chính xác như đối với các lời gọi đệ qui đuôi tới tfact. Hai hàm này tính ra cùng một kết quả bằng việc thực hiện cùng dãy chỉ lệnh như nhau. Như vậy khử đệ qui đuôi dịch hàm đệ qui đuôi thành các vòng lặp.

4.4 Các hàm bậc cao

Một ngôn ngữ có các hàm bậc cao nếu các hàm đó có thể

- Được khai báo trong phạm vi bất kỳ
- Được truyền như các tham số đến các hàm khác
- Được trả về như kết quả của hàm

Trong ngôn ngữ với các hàm bậc cao và phạm vi tĩnh, một giá trị hàm nói chung được thể hiện bởi một bao đóng, mà là một cặp gồm một con trỏ đến mã của hàm và một con trỏ đến bản ghi kích hoạt.

Đây là ví dụ hàm ML mà yêu cầu một đối số hàm:

```
fun map (f, nil) = nil
  | map(f, x::xs) = f(x) :: map(f, xs)
```

Hàm map nhận hàm f và danh sách m như các đối số, áp dụng f lần lượt cho mỗi phần tử của m. Kết quả của map(f, m) là danh sách các kết quả f(x) cho các phần tử x trong danh sách m. Hàm này được sử dụng trong nhiều tình huống lập trình ở đó danh sách được sử dụng. Chẳng hạn, ta có danh sách các thời gian hết hạn của các sự kiện và ta muốn tăng mỗi thời gian hết hạn, chúng ta có thể áp dụng hàm tăng cho map.

Chúng ta sẽ thấy tại sao bao đóng là cần thiết bằng cách xét tương tác giữa phạm vi tĩnh và các đối số của hàm và giá trị trả về. C và C++ không hỗ trợ bao đóng vì nó kèm theo trả giá cài đặt. Tuy nhiên, việc cài đặt các đối tượng trong C++ và các ngôn ngữ khác liên quan đến cài đặt các giá trị hàm sẽ được bàn trong chương này. Lý do là bao đóng và đối tượng cả hai đều kết hợp dữ liệu với mã của hàm.

Mặc dù một số lập trình C có thể không có kinh nghiệm với việc truyền hàm như tham số đến hàm khác, có nhiều tình huống mà ở đó việc này là một phương pháp lập trình có ích. Công nhận sử dụng một hàm như đối số hàm số đến từ khái niệm tổ chức phần mềm có ảnh hưởng. Trong lập trình hệ thống, thuật ngữ upcall muốn chỉ đến lời gọi hàm trên ngăn xếp. Trong bài báo quan trọng có tên là “The Structuring of Systems Using upcalls” (ACM

Symp, Operating Systems principles, 1985) David Clark mô tả phương pháp sắp xếp các hàm của hệ thống thành các lớp (layers). Phương pháp này làm cho dễ code, modun hóa và suy luận về hệ thống. Như trong ngăn xếp giao thức mạng, các lớp cao hơn là clients của các dịch vụ được cung cấp bởi các lớp dưới. Trong hệ thống file được phân lớp, lớp phân cấp file được xây dựng trên vnode, mà nó lại được xây dựng trên inode và các lớp khối của đĩa. Trong phương pháp Clark, mà được đưa vào sử dụng rộng rãi, các mức cao hơn truyền hàm điều khiển đến các mức thấp hơn. Các hàm điều khiển này được gọi, khi lớp dưới cần cần nhắc lớp trên về một việc gì đó. Các lời gọi này đến lớp trên được gọi là upcalls. Phương pháp thiết kế có ảnh hưởng lớn này chỉ ra giá trị của việc hỗ trợ ngôn ngữ cho việc truyền hàm như tham số.

4.4.1 Truyền hàm đến hàm

Chúng ta sẽ thấy khi hàm f được truyền cho hàm g , chúng ta có thể cần truyền bao đóng của f , mà chứa con trỏ đến bản ghi của nó. Khi f được gọi trong thân của g , con trỏ môi trường của bao đóng được sử dụng để đặt liên kết truy cập trong bản ghi kích hoạt cho lời gọi đến f cho đúng. Sự cần thiết của bao đóng trong tình huống này đôi khi được gọi là downward funarg problem, vì nó cho kết quả từ việc truyền hàm như đối số xuống cho các phạm vi lặp.

Ví dụ 4.8

Chương trình ví dụ, với hai khai báo của biến x và hàm f được truyền cho hàm khác g , được sử dụng để thể hiện các vấn đề chính sau:

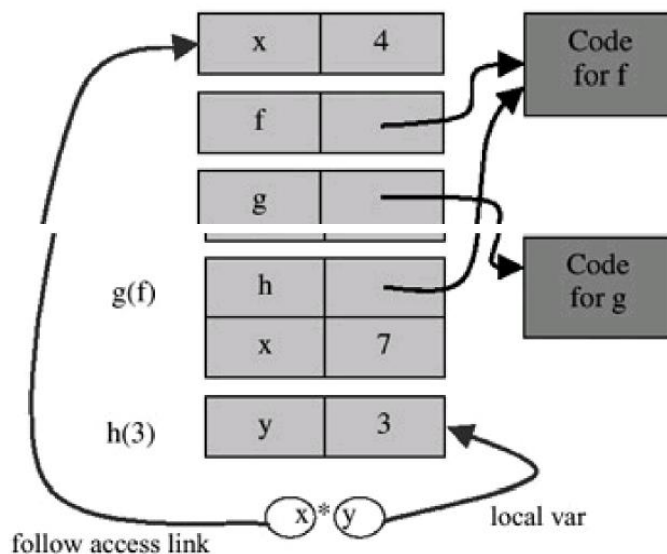
```
val x = 4;
fun f(y) = x*y;
fun g(h) = let val x=7 in h(3) + x;
g(f);
```

Trong chương trình này, thân hàm f chứa biến cục bộ x mà được khai báo bên ngoài hàm f . Khi f được gọi bên trong g , giá trị của x cần phải được lấy từ bản ghi kích hoạt gắn kết với khối bên ngoài. Tuy nhiên thân của f cần được tính với biến cục bộ x khai báo bên trong hàm g , mà có thể vi phạm phạm vi tĩnh. Đây là chương trình cùng mục đích đó viết trên cú pháp giống C (ngoại trừ cho biểu thức kiểu $\text{int} \rightarrow \text{int}$) dành cho những ai cảm thấy dễ đọc hơn:

```
{ int x=4;
  {int f(int y) {return x*y;}
    {int g(int → int h){
      int x=7;
      return h(3) + x;
    }
    g(f);
  }}}}
```

Phiên bản tựa C của đoạn mã trên phản ánh quyết định được dùng để đơn giản hóa trong suốt cuốn sách này là xử lý mỗi khai báo ở mức cao của ML như bắt đầu một khối riêng.

Chúng ta có thể thấy vấn đề tìm kiếm biến bằng cách xem trên ngăn xếp thời gian thực sau lời gọi đến f từ việc khởi tạo g .



Thẻ hiện đơn giản này chỉ đưa ra dữ liệu chứa trong mỗi bản ghi kích hoạt. Trong hình vẽ trên, biểu thức $x*y$ từ thân của hàm f được chỉ ra ở đáy, bản ghi kích hoạt gắn kết với khởi tạo f (qua tham số hình thức h của g). Như trên hình vẽ, biến y là cục bộ đối với hàm đó và do đó cần phải tìm trong bản ghi kích hoạt hiện thời. Tuy nhiên biến x là tổng thể, được đặt trên vài bản ghi kích hoạt so với bản ghi hiện thời. Vì chúng ta tìm biến tổng thể theo liên kết truy cập, liên kết truy cập của bản ghi kích hoạt dưới đáy cần phải cho phép ta tìm đến bản ghi kích hoạt ở trên đỉnh của hình vẽ.

Khi các hàm được truyền cho hàm, chúng ta cần đặt liên kết truy cập cho bản ghi của mỗi hàm sao cho chúng ta có thể tìm các biến tổng thể của hàm này một cách đúng đắn. Chúng ta không thể giải quyết vấn đề này một cách dễ dàng mà không mở rộng cấu trúc dữ liệu thời gian chạy theo một cách nào đó.

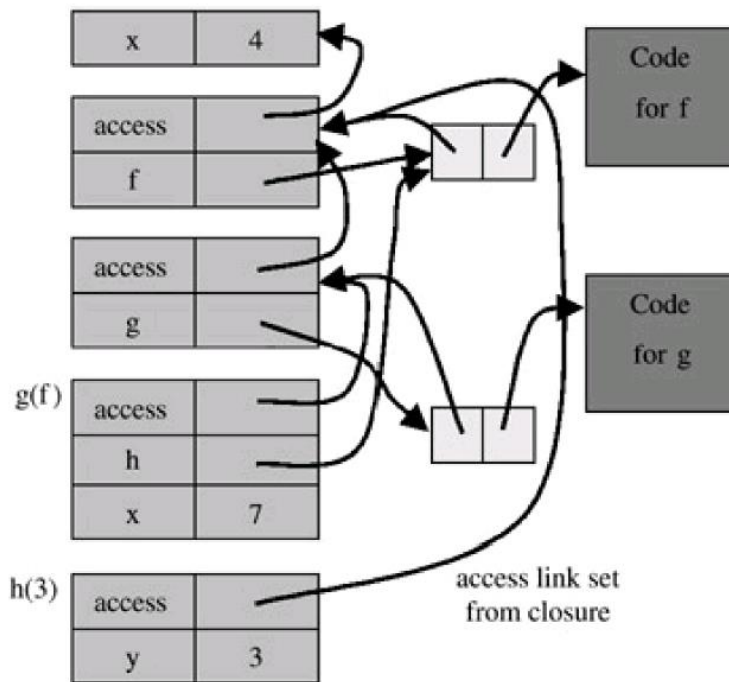
Sử dụng bao đóng

Giải pháp chuẩn để bảo trì phạm vi tĩnh khi các hàm được truyền cho hàm hoặc trả về như kết quả là sử dụng cấu trúc dữ liệu gọi là bao đóng. Bao đóng là cặp gồm con trỏ đến mã hàm và con trỏ đến bản ghi kích hoạt. Vì mỗi bản ghi kích hoạt chứa liên kết truy cập trỏ đến bản ghi cho bao đóng phạm vi gần nhất, con trỏ đến phạm vi mà ở đó hàm được khai báo cũng như cung cấp các liên kết đến các bản ghi cho các khối bao.

Khi một hàm được truyền cho hàm khác, giá trị thực tế mà được truyền là con trỏ đến bao đóng. Các bước sau được sử dụng để gọi hàm, cho trước bao đóng.

- Cấp bản ghi kích hoạt cho hàm được gọi như thông thường.
- Đặt liên kết truy cập trên bản ghi kích hoạt bằng việc con trỏ bản ghi kích hoạt từ bao đóng đó.

Chúng ta có thể thấy nó giải quyết vấn đề truy cập biến như thế nào cho hàm được truyền đến hàm bằng hình vẽ các bản ghi kích hoạt trên ngăn xếp thời gian chạy khi chương trình trên Hình 4.8 thực hiện. Các điều đó được vẽ trên Hình 4.9.



Hình 4.9. Đặt liên kết truy cập từ bao đóng

Chúng ta có thể hiểu Hình vẽ 4.9 bằng duyệt qua các bước thời gian chạy mà dẫn đến cấu hình cấu hình như trên Hình vẽ.

1. Khai báo của `x`. Một bản ghi kích hoạt cho khối mà ở đó `x` khai báo được đẩy vào trên ngăn xếp. Bản ghi kích hoạt này chứa giá trị của `x` và liên kết điều khiển (mà chưa được nêu).
2. Khai báo của `f`. Một bản ghi kích hoạt cho khối mà ở đó `f` khai báo được đẩy vào trên ngăn xếp. Bản ghi kích hoạt này chứa con trỏ đến biểu diễn thời gian chạy của `f`, mà là bao đóng với hai con trỏ. Con trỏ thứ nhất trong bao đóng trỏ đến bản ghi kích hoạt cho phạm vi tĩnh của `f`, mà là bản ghi cho khai báo của `f`. Con trỏ thứ hai của bao đóng trỏ đến mã của `f`, mà được tạo ra khi dịch và đặt ở vị trí nào đó mà chương trình dịch biết, khi mã dịch của chương trình này được sinh ra.
3. Khai báo của `g`. Như với khai báo của `f`, một bản ghi kích hoạt cho khối mà ở đó `g` khai báo được đẩy vào trên ngăn xếp. Bản ghi kích hoạt này chứa con trỏ đến biểu diễn thời gian chạy của `g`, mà là bao đóng.
4. Lỗi gọi tới `g(f)`: Lỗi gọi này tạo ra bản ghi kích hoạt cho hàm `g` được cấp trên ngăn xếp. kích thước và thể hiện của bản ghi này được xác định bởi mã của `g`. Liên kết truy cập sẽ được đặt vào bản ghi kích hoạt này cho phạm vi mà ở đó `g` được khai báo; liên kết truy cập này trỏ đến cùng bản ghi kích hoạt như bản ghi kích hoạt trong bao đóng cho `g`. Bản ghi kích hoạt này chứa không gian tham số `h` và biến

cục bộ x . Vì tham số thực tế là bao đóng cho f , giá trị tham số cho h là con trỏ tới bao đóng cho f . Biến cục bộ x có giá trị 7, được cho trong mã nguồn.

5. Lời gọi tới $h(3)$: Cơ chế để thực thi lời gọi này là điểm chính trong ví dụ này. Vì h là tham số hình thức đến g , mã của g được dịch mà không cần biết hàm h được khai báo ở đâu. Như kết quả, chương trình dịch không thể chèn bất cứ chỉ lệnh nào nói về việc đặt liên kết truy cập cho bản ghi kích hoạt này đối với lời gọi $h(3)$. Tuy nhiên, việc sử dụng các bao đóng cung cấp giá trị cho liên kết truy cập này – liên kết truy cập đến bản ghi này được đặt bởi con trỏ đến bản ghi từ bao đóng của h . Vì tham số thực tế là f , liên kết truy cập trỏ đến bản ghi kích hoạt cho phạm vi mà ở đó f được khai báo. Khi mã của f thực hiện, liên kết truy cập này được sử dụng để tìm x . Đặc biệt, mã sẽ lần theo liên kết truy cập lên trên tới bản ghi kích hoạt thứ hai của mình họa trên, theo một liên kết truy cập nữa vì chương trình đã biết khi sinh ra mã cho f là khai báo của x nằm một phạm vi bên trên khai báo của f , và tìm giá trị 4 cho x tổng thể trong thân hàm f .

Như mô tả trong bước 5, bao đóng của f cho phép mã thực hiện trong thời gian chạy để tìm bản ghi kích hoạt chứa khai báo tổng thể của x .

Khi ta truyền các hàm như các tham số, các liên kết truy cập bên trong ngăn xếp tạo thành cây. Cấu trúc này không tuyến tính, như bản ghi kích hoạt tương ứng với lời gọi hàm $h(3)$ cần bỏ qua bản ghi kích hoạt ở giữa của $g(f)$ để tìm x cần thiết. Tuy nhiên, mọi liên kết truy cập trỏ lên trên. Do đó vẫn còn khả năng cấp và thu hồi bản ghi kích hoạt bằng việc sử dụng nguyên lý của ngăn xếp (cấp sau cùng, thu hồi trước).

4.4.2 Trả hàm về từ phạm vi lồng nhau

Một vấn đề liên quan nhưng phức tạp hơn là đôi khi được gọi là upward funnarg problem, mặc dù có thể gọi chính xác hơn là vấn đề kết quả - hàm phía trên, vì nó xảy ra khi trả về giá trị hàm từ phạm vi lồng nhau, nói chung như giá trị trả về của hàm số.

Ví dụ đơn giản của hàm mà trả về hàm là mã ML sau cho việc hợp hàm:

```
fun compose(f,g) = (fn x => g(f x));
```

Cho hai đối số hàm số f và g , phép hợp hàm trả về hàm hợp của f và g . Thân của phép hợp là mã yêu cầu tham số hàm x và sau đó tính toán $g(f(x))$. Mã này có ích chỉ khi nó gắn kết với cơ chế nào đó để tìm giá trị của f và g . Do đó, bao đóng được sử dụng để thể hiện phép hợp hàm (f,g) . Con trỏ mã của bao đóng này trỏ tới mã dịch “để nhận tham số x của hàm và sau đó tính toán $g(f(x))$ ” và con trỏ bản ghi kích hoạt của bao đóng này trỏ đến bản ghi kích hoạt của phép hợp lời gọi (f,g) vì vậy bản ghi kích hoạt này cho mã để tìm các hàm thực tế f và g cần thiết để tính hợp hàm của chúng.

Chúng ta sẽ sử dụng ví dụ hàm gây ngạc nhiên một chút để thấy các bao đóng giải quyết vấn đề tìm kiếm biến như thế nào khi các hàm được trả về từ phạm vi lồng nhau. Ví dụ sau có thể cho cảm giác về sự tương tự giữa bao đóng và đối tượng.

Ví dụ 4.9

Trong mã ví dụ này, `counter` là hàm mà có giá trị nguyên được lưu trữ. Khi được gọi, `counter` tăng giá trị bên trong của nó và trả về giá trị mới. Giá trị mới này trở thành giá trị riêng được lưu trữ cho lời gọi tiếp theo. Hàm ML `make_counter` sau, nhận đối số nguyên và trả về `counter` khởi tạo giá trị nguyên này:

```
fun make_counter (init : int) =
  let val count = ref init (* private variable count *)
      fun counter(inc:int) = (count := !count + inc; !count)
  in
    counter (* return function counter from make counter *)
  end;
val c = make_counter(1); (* c is a new counter *)
c(2) + c(2);           (* call counter c twice *)
```

Hàm `make_counter` cấp bộ nhớ cho biến cục bộ `count`, khởi tạo bằng giá trị của tham số nguyên `init`. Hàm `make_counter` sau đó trả về một hàm, mà khi được gọi, sẽ tăng thêm giá trị của `count` bằng tham số `inc`, và sau đó trả về giá trị mới của `count`.

Các kiểu và giá trị gắn kết với các khai báo này, được in ra bởi chương trình dịch là

```
val make_counter = fn : int → (int → int)
val c = fn : int → int
8 : int
```

Sau đây là cũng là chương trình đó được viết trên cú pháp tựa C cho những ai thích dùng nó hơn:

```
{int → int mk_counter (int init)
  {int count = init;

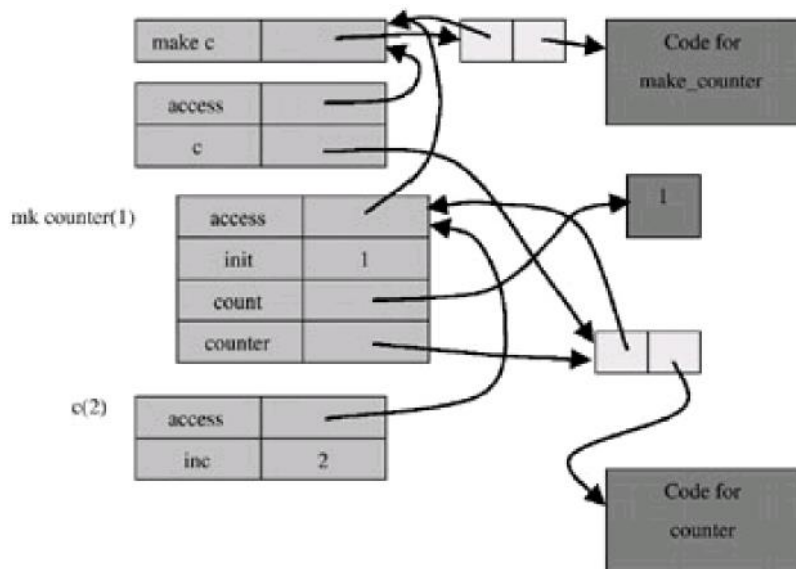
  int counter(int inc){ return count += inc;}
  return counter
}
int → int c = mk_counter(1);
print c(2) + c(2);xs
}
```

Nếu ta lần theo việc cấp bộ nhớ gắn kết với quá trình dịch và thực thi, ta có thể thấy là nguyên lý ngăn xếp không còn đúng nữa. Cụ thể, cần thiết lưu bản ghi kích hoạt mà có thể được lấy ra từ ngăn xếp, nếu ta tuân theo nguyên lý chuẩn cấp sau-giải phóng trước.

Hình 4-10 chỉ ra rằng, các bản ghi được cấp và giải phóng khi thực thi mã từ ví dụ 4-9. Sau đây là dãy các bước để tạo ra các bản ghi chỉ ra trên Hình 4-10.

1. Khai báo của `make_counter`. Một bản ghi kích hoạt cho khối bao gồm việc khai báo hàm `make_counter` được cấp trên ngăn xếp thời gian chạy. Tên hàm dùng cho `counter` được viết ngắn ở đây là `make_c` để tên trùng với trên hình vẽ. Giá trị của `make_c` là bao đóng. Con trỏ bản ghi kích hoạt của bao đóng này trở

- này trỏ đến bản ghi kích hoạt của `make_counter`. Con trỏ mã của bao đóng trỏ đến trỏ đến mã của `make_counter` được tạo trong thời gian dịch.
2. Khai báo của `c`: Bản ghi kích hoạt của khối gồm khai báo hàm `c` được cấp trên ngăn xếp thời gian chạy. Con trỏ truy cập của bản ghi kích hoạt này trỏ bản ghi kích hoạt đầu tiên, như khai báo của `make_counter` là khối trước đó. Giá trị của `c` là hàm biểu diễn bởi bao đóng. Tuy nhiên, biểu thức định nghĩa hàm `c` không phải là khai báo hàm. Đây là biểu thức mà gửi mà lời gọi yêu cầu đến hàm `make_counter`. Do đó, sau khi bản ghi kích hoạt này được thiết lập, chương trình sẽ thực hiện và cần phải gọi đến `make_counter`.
 3. Lời gọi đến `make_counter`: Một bản ghi kích hoạt được cấp để thực thi hàm `make_counter`. Bản ghi kích hoạt này chứa không gian cho tham số mà sẽ là kết quả trả về của lời gọi. Con trỏ mã của bao đóng cho `counter` sẽ trỏ đến mã mà được sinh ra và lưu trong thời gian dịch. Con trỏ bản ghi kích hoạt trỏ đến bản ghi kích hoạt thứ ba, vì biến cục bộ của hàm thường trú ở đây. Chương trình vẫn cần bản ghi kích hoạt ba sau lời gọi đến `make_counter` được trả về, vì `counter` của hàm trả về từ lời gọi đến `make_counter` tham chiếu đến các biến `init` và `count` mà được lưu trong (hoặc đạt được thông qua) bản ghi kích hoạt cho `make_counter`. Nếu bản ghi kích hoạt ba (sử dụng để gọi đến `make_counter` được lấy ra từ ngăn xếp), hàm `counter` sẽ làm việc không đúng đắn.
 4. Lời gọi đầu tới `c(2)`: Khi biểu thức đầu `c(2)` được tính toán, bản ghi kích hoạt được tạo ra cho lời gọi hàm. Bản ghi kích hoạt này có con trỏ truy cập được thiết lập từ bao đóng cho `c`. Vì bao đóng trỏ đến bản ghi kích hoạt ba, nên con trỏ truy cập cho bản ghi kích hoạt 4 cũng vậy. Điều này quan trọng vì mã cho `counter` tham chiếu đến biến `count` và `count` được cấp theo con trỏ trong bản ghi kích hoạt ba.



Hình 4.10: Các bản ghi kích hoạt cho bao đóng hàm trả về từ hàm.

Ở đây có hai điểm chính cần nhớ trong ví dụ này:

- Bao đóng được dùng để thiết lập con trỏ truy cập khi hàm trả về từ phạm vi lồng nhau được gọi

- Khi hàm được trả về từ phạm vi lồng nhau, các bản ghi kích hoạt không cần tuân theo nguyên lý ngăn xếp. Bản ghi kích hoạt được cấp với lời gọi hàm không thể được giải phóng khi hàm trả về, nếu giá trị trả về của hàm là một hàm khác mà có thể đòi hỏi bản ghi kích hoạt này.

Giải pháp cho vấn đề quản trị bộ nhớ.

Bạn có thể nhận thấy rằng, sau khi mã trong Ví dụ 4.9 được thực hiện, theo các bước mới được mô tả, chúng ta còn một số bản ghi kích hoạt còn lại trên ngăn xếp mà có thể được sử dụng trong phần còn lại của chương trình. Cụ thể nếu hàm *c* được gọi nữa trong một biểu thức khác, thì chúng ta cần các bản ghi kích hoạt này mà duy trì phạm vi tĩnh của nó. Tuy nhiên, nếu hàm *c* không được sử dụng nữa, thì chúng ta không cần các bản ghi kích hoạt này. Ta có thể hỏi, chương trình dịch hoặc hệ thống thời gian chạy xác định như thế nào về việc khi nào một bản ghi kích hoạt có thể được giải phóng.

Có một số phương pháp giải quyết vấn đề này. Tuy nhiên, việc bàn luận đầy đủ là phức tạp và không cần thiết để hiểu các cân nhắc thiết kế ngôn ngữ cơ bản mà là đối tượng của chương này. Một giải pháp mà là tương đối trực tiếp và không phải là không hiệu quả là đơn giản sử dụng thuật toán thu gom rác để tìm các bản ghi mà không cần nữa. Trong cách tiếp cận này, thu gom rác sẽ lần theo các con trỏ đến các bản ghi kích hoạt và thu gom các bản ghi không đạt được khi mà không có các con trỏ bao đóng nào trỏ đến chúng.

4.5 Tóm tắt chương

Một khối là một vùng văn bản chương trình, được xác định bằng các dấu bắt đầu và kết thúc, mà có thể chứa khai báo cục bộ trong vùng này. Các khối có thể xuất hiện bên trong khối khác hoặc như thân của một hàm hay một thủ tục. Các ngôn ngữ cấu trúc khối được cài đặt bởi các bản ghi kích hoạt mà chứa không gian cho các biến cục bộ hoặc thông tin liên quan đến khối khác. Vì chỉ có một cách để cho một khối chồng chéo lên khối khác là một khối chứa trọn vẹn khối kia. Các bản ghi kích hoạt nói chung được quản lý trên ngăn xếp thời gian chạy với chính sách bản ghi kích hoạt mới được cấp sẽ được giải phóng trước tiên (last allocated – first deallocated).

Các tham số được truyền cho hàm và thủ tục được lưu trong các bản ghi kích hoạt, cũng như các biến cục bộ. Bản ghi kích hoạt có thể chứa giá trị tham số thực tế (truyền theo giá trị) hoặc địa chỉ (truyền theo tham chiếu). Lời gọi đuôi có thể có thể được tối ưu để tránh trả về các thủ tục gọi. Trong trường hợp hàm đệ qui đuôi mà không truyền đối số hàm, điều đó có nghĩa là cùng một bản ghi kích hoạt có thể được dùng cho mọi lời gọi đệ qui, loại bỏ việc cần thiết phải cấp, khởi tạo, và sau đó giải phóng bản ghi kích hoạt cho mỗi lời gọi. Kết quả là hàm đệ qui đuôi có thể được thực hiện cũng hiệu quả như vòng lặp.

Truy cập đúng đến các biến tổng thể phạm vi tĩnh bao gồm ba khái niệm cài đặt sau:

- Các bản ghi kích hoạt của các hàm và các thủ tục chứa các liên kết truy cập (phạm vi tĩnh) mà trỏ đến các bản ghi kích hoạt liên kết với khối bao trung gian.
- Các hàm được truyền như các tham số hoặc trả về như kết quả cần được biểu diễn dạng bao đóng (closures) gồm mã hàm cùng với con trỏ đến môi trường từ vựng đúng.

- Khi hàm trả về hàm mà dựa trên các biến được khai báo trong phạm vi lồng nhau, phạm vi tĩnh đòi hỏi suy diễn từ nguyên lý ngăn xếp: Bản ghi kích hoạt có thể cần được duy trì cho đến khi giá trị hàm (bao đóng) không cần sử dụng nữa trong chương trình.

Mỗi khái niệm đặt được bàn trong chương này, có thể được gắn chặt với một tính chất của ngôn ngữ cụ thể, được tổng kết trong bảng sau:

Language Feature	Implementation construct
Block	Activation record with local memory
Nested blocks	Control link
Functions and procedures	Return address, return-result address
Functions with static scope	Access link
Function as arguments and results	Closures
Functions returned from nested scope	Failure of stack deallocation order for activation records

Bài tập

1. Chương trình sau tính hàm giai thừa theo và hàm đảo ngược thứ tự danh sách theo tiếp cận đệ qui đuôi. Áp dụng với một ví dụ cụ thể và hãy giải thích cách thực thi chương trình để nhận được kết quả

```

fun fact_helper (0,r:int) = r
  | fact_helper (n:int,r:int) = fact_helper (n-1,n*r)

fun factorial n:int = fact_helper (n, 1)

local
  fun rev_helper (nil, a) = a
    | rev_helper (h::t, a) = rev_helper (t, h::a)
in
  fun rev l = rev_helper (l, nil)
end

```

2. Sắp xếp quicksort tham số. Hãy viết chương trình Insertsort tham số

```

fun partition (pivot, nil) lessThan = (nil,nil)
  | partition (pivot, first :: others) lessThan =
let
  val (smalls, bigs) =
partition(pivot, others) lessThan

```

```

in
if (lessThan first pivot)
then (first::smalls, bigs)
else (smalls, first::bigs)
end;

fun qsort nil lessThan = nil
| qsort [singleton] lessThan = [singleton]
| qsort (first::rest) lessThan =
let
val (smalls, bigs) =
partition(first,rest) lessThan
in
qsort smalls lessThan @ [first]
@ qsort bigs lessThan
end;

```

3. Chương trình sau thay đổi giá trị của hai biến. Thử áp dụng đổi giá trị của hai biến tham chiếu sau: $u = \text{ref } [1, 2, 3, 4]$, $v = \text{ref } [5, 6, 7]$.

```

fun swap(x,y) =
let val z = !x in x := !y; y := z end;

```

4. Trả về hàm số là đặc trưng hàm bậc cao. Giải thích kết quả áp dụng hàm sau:

```

fun mk_counter(init:int) =
let val count = ref init
fun counter(inc:int) =
(count := !count + inc; !count)
in
counter
end;

```

Áp dụng hàm:

```

val c = mk_counter(2);
c(3) + c(4);

```

5. Áp dụng hàm và giải thích kết quả nhận được:

```

val x=1;
fun g(z) = x+z;
fun f(y) =
let val x = y+1
in g(y*x)
end;
f(3);

```

```
*****
```

```
val x = 4;
fun f(y) = x*y;
fun g(h) =
  let val x = 7
  in h(3) + x
end;
g(f);
```

6. Xét các hàm tham số sau. Áp dụng để tính tổng và tích của một dãy số thực.

```
fun map(f,nil) = nil |
  map(f,x::xs) = f(x)::(map(f,xs));

fun reduce (unit, opn, nil) = unit
  | reduce (unit, opn, h::t) = opn (h, reduce (unit, opn, t))

fun add_up l = reduce (0, op +, l)
fun mul_up l = reduce (1, op *, l)
```

7. Xử lý hàm đa đối số

```
fun map' (f, nil) = nil
  | map' (f, h::t) = (f h) :: map' (f, t)

val constantly = fn k => (fn a => k)
fun constantly k a = k

fun map f nil = nil
  | map f (h::t) = (f h) :: (map f t)

fun curry f x y = f (x, y)

fun map f l = curry map' f l

fun add_up nil = 0
  | add_up (h::t) = h + add_up t

fun mul_up nil = 1
  | mul_up (h::t) = h * mul_up t

fun reduce (unit, opn, nil) = unit
  | reduce (unit, opn, h::t) = opn (h, reduce (unit, opn, t))

fun add_up l = reduce (0, op +, l)
fun mul_up l = reduce (1, op *, l)
fun mystery l = reduce (nil, op ::, l)
fun better_reduce (unit, opn, l) =
  let
    fun red nil = unit
      | red (h::t) = opn (h, red t)
  in
```

```

    red l
  end
  fun staged_reduce (unit, opn) =
    let
      fun red nil = unit
        | red (h::t) = opn (h, red t)
    in
      red
    end
  end

```

8. Hàm nhớ

```

fun curried_reduce (unit, opn) nil = unit
  | curried_reduce (unit, opn) (h::t) = opn (h, curried_reduce (unit, opn) t)

fun append (nil, l) = l
  | append (h::t, l) = h :: append(t,l)

fun curried_append nil l = l
  | curried_append (h::t) l = h :: append t l

fun staged_append nil = fn l => l
  | staged_append (h::t) =
    let
      val tail_appender = staged_append t
    in
      fn l => h :: tail_appender l
    end
  end

```

9. Xử lý ngoại lệ

```

* Exceptions *)

exception Factorial

fun checked_factorial n =
  if n < 0 then
    raise Factorial
  else if n=0 then
    1
  else
    n * checked_factorial (n-1)

exception Factorial

local
  fun fact 0 = 1
    | fact n = n * fact (n-1)
in
  fun checked_factorial n =
    if n >= 0 then
      fact n
    else
      raise Factorial
end

```

end

10. Xử lý lỗi

```
exception X;
(let fun f(y) = raise X
  and g(h) = h(1)
  handle X => 2
```

```
in
  g(f) handle X => 4
end) handle X => 6;
```

```
val x=6;
(let fun f(y) = x
  and g(h) = let val x=2 in
    h(1) end
  in
    let val x=4 in g(f) end
  end);
```

11. Hàng đợi

```
signature QUEUE =
sig
  type 'a queue
  exception Empty
  val empty : 'a queue
  val insert : 'a * 'a queue -> 'a queue
  val remove : 'a queue -> 'a * 'a queue
end
```

```
structure Queue :> QUEUE =
struct
  type 'a queue = 'a list * 'a list
  val empty = (nil, nil)
  fun insert (x, (bs, fs)) = (x::bs, fs)
  exception Empty
  fun remove (nil, nil) = raise Empty
  | remove (bs, f::fs) = (f, (bs, fs))
  | remove (bs, nil) = remove (nil, rev bs)
end
```

```
signature PQ =
sig
  type elt
  val lt : elt * elt -> bool
  type queue
  exception Empty
  val empty : queue
  val insert : elt * queue -> queue
  val remove : queue -> elt * queue
end
```

BÀI 5: TRỪ TƯỢNG DỮ LIỆU VÀ TÍNH MODULARITY

Các lập trình viên máy tính phải mất thời gian khá lâu mới công nhận giá trị của việc xây dựng của các hệ thống phần mềm mà bao gồm một số các modules. Trong thiết kế hiệu quả, mỗi module cần được thiết kế và kiểm chứng độc lập. Hai mục tiêu quan trọng trong tính module là cho phép một module được viết với rất ít kiến thức về code trong một module khác và cho phép một module được thiết kế và cài đặt lại mà không sửa các phần khác của hệ thống. Các ngôn ngữ lập trình và môi trường phát triển phần mềm hiện đại hỗ trợ tính module theo nhiều cách khác nhau.

Trong chương này, chúng ta xét một số cách mà chương trình được chia ra các phần có ý nghĩa và cách mà các ngôn ngữ lập trình có thể được thiết kế để hỗ trợ việc chia đó. Trong chương này chúng ta chỉ xét đến các cơ chế module mà không nói đến các đối tượng. Chủ đề chính là lập trình cấu trúc hỗ trợ trừu tượng hóa và modules. Hai ví dụ được sử dụng để mô tả hệ thống module và lập trình khái quát là hệ thống module ML chuẩn và thư viện mẫu chuẩn C++ (Standard Template Library – STL).

5.1 Lập trình cấu trúc

Trong bài báo có ảnh hưởng lớn vào năm 1969 mang tên Structured Programming, E.W.Dijkstra chỉ rõ rằng cần phát triển chương trình bằng cách nêu ra các bài toán chính mà cần phải thực hiện và sau đó làm mịn dần các bài toán này thành các bài toán nhỏ hơn, cho đến khi đạt đến mức độ, ở đó mỗi nhiệm vụ còn lại có thể dễ dàng biểu diễn bằng các thao tác cơ bản. Điều đó tạo ra các vấn đề con mà đủ nhỏ đến mức có thể hiểu được và đủ riêng biệt để có thể giải quyết độc lập.

Trong Ví dụ 5.1, các cấu trúc dữ liệu được truyền giữa các phần riêng biệt của chương trình là đơn giản và trực tiếp. Điều này làm cho có thể xác định sớm các cấu trúc dữ liệu chính trong quá trình đó. Vì các cấu trúc dữ liệu này là bất biến qua hầu hết quá trình thiết kế. Ví dụ của Dijkstra tập trung vào việc làm mịn các thủ tục thành các thủ tục nhỏ hơn. Trong các hệ thống phức tạp hơn, cần thiết làm mịn các cấu trúc dữ liệu cũng như các thủ tục. Điều này được thể hiện trong Ví dụ 5.2.

Nhà bác học kiên trì và nồng hậu, E.W. Dijkstra có nhiều đóng góp quan trọng cho lĩnh vực Khoa học máy tính. Ông biết đến qua semaphore, mà được sử dụng cho điều khiển song song, các thuật toán như phương pháp tìm đường đi ngắn nhất trong đồ thị, ngôn ngữ ‘guarded command’ và các phương pháp suy luận chương trình.

Qua một vài năm, Dijkstra đã viết một loạt các bài báo viết tay cẩn thận, được mọi người biết đến như EWD. Đến đầu năm 2002, nhà bác học đã viết hơn 1309 EWDs, đã được scanned và được đưa lên trang web của ông. Như ông viết:

‘Lĩnh vực quan tâm của tôi tập trung vào các công cụ toán học để làm tăng khả năng suy luận, đặc biệt là sử dụng các phương pháp hình thức’.

Sự quan tâm về công cụ toán học được minh chứng trong EWDs, mỗi cái phát triển một giải pháp đẹp đẽ cho một bài toán hấp dẫn trong một vài trang.

Như nhiều trường phái Châu Âu cổ, và không giống như nhiều người Mỹ, Dijkstra coi trọng việc viết tay. Một phần như vui đùa và một phần như để tỏ lòng kính trọng Dijkstra, nhà nghiên cứu ngôn ngữ lập trình Luca Cardelli đã sao chép cẩn thận các bản viết tay từ tập EWDs và tạo thành font EWD. Nếu bạn có thể tìm thấy font này trên web, bạn có thể viết các ghi chú ngắn theo cách viết tay nổi tiếng của Dijkstra.

Ví dụ 5.1

Dijkstra xét bài toán tính và in 1000 số nguyên tố đầu tiên. Phiên bản đầu tiên của chương trình có chứa một chút cú pháp cho ta suy nghĩ về việc viết chương trình. Ngược lại, nó trông giống như mô tả tiếng Anh về bài toán mà chúng ta muốn giải.

Chương trình 1:

```
begin
  print first thousand prime numbers
end
```

Bài toán này bây giờ được làm mịn thành các bài toán nhỏ. Để chia bài toán này thành hai, một cấu trúc dữ liệu nào đó cần phải được chọn để truyền kết quả của bài toán thứ nhất cho bài toán thứ hai. Trong ví dụ Dijkstra, cấu trúc dữ liệu này là bảng, mà sẽ điền 1000 số nguyên tố đầu tiên.

Chương trình 2 :

```
begin variable table p
  fill table p with first thousand primes
  print table p
end
```

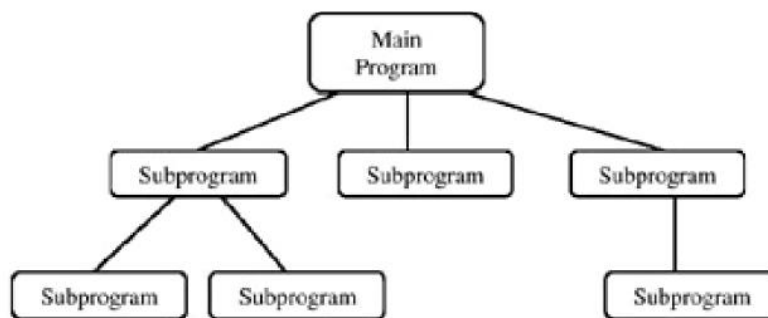
Trong phần làm mịn tiếp theo, mỗi bài toán sẽ được xét tiếp. Một ý tưởng quan trọng trong lập trình cấu trúc là mỗi bài toán được xét độc lập. Trong ví dụ trên, bài toán điền vào bảng các số nguyên tố là độc lập với bài toán in bảng. Do đó mỗi bài toán có thể giao cho lập trình viên khác nhau, cho phép các bài toán được giải vào cùng một thời điểm bởi những người khác nhau. Ngay cả nếu chương trình được viết bởi một người duy nhất, ở đây có lợi ích quan trọng là tách bài toán phức tạp thành các bài toán nhỏ độc lập. Đặc biệt lắm, một người mới có thể nghĩ về nhiều chi tiết cùng một lúc. Chia bài toán thành các bài toán

con làm cho có thể chỉ nghĩ về một bài toán tại mỗi thời điểm, giảm số chi tiết mà cần phải xét tại mỗi thời điểm.

Chương trình 3.

```
begin integer array p[1:1000]
make for k from 1 through 1000
  p[k] equal to the kth prime number
print p[k] for k from 1 through 1000
end
```

Tại điểm này, cấu trúc chương trình cơ bản đã được xác định và lập trình có thể tập trung về thuật toán tính số nguyên tố tiếp theo. Mặc dù ví dụ này rất đơn giản, nó cho một ý tưởng cơ bản nào đó về lập trình bằng việc làm mịn từng bước. Làm mịn từng bước nói chung đưa về chương trình với cấu trúc giống khái niệm cây.



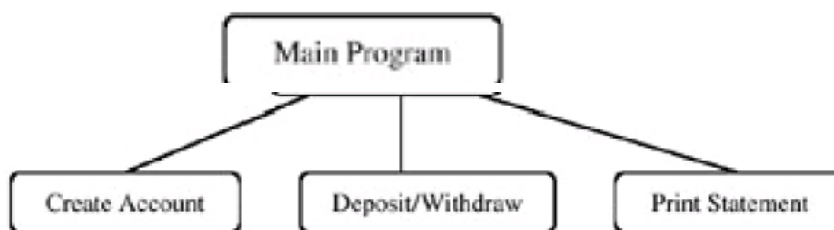
Một khía cạnh khó của việc phát triển chương trình trên-xuống là làm cho bài toán trở nên đơn giản hơn trên mỗi bước làm mịn. Nếu không, có thể làm mịn bài toán và tạo ra một danh sách các bài toán lập trình mà mỗi một trong chúng khó hơn bài toán gốc. Điều đó có nghĩa là người thiết kế mà sử dụng việc làm mịn từng bước cần phải có ý tưởng tốt trước các bài toán sẽ được thực hiện như thế nào.

5.1.1 Làm mịn dữ liệu

Bổ sung thêm cho việc làm mịn các bài toán thành các bài toán đơn giản hơn, sự tiến triển trong thiết kế hệ thống có thể dẫn đến các thay đổi trong các cấu trúc dữ liệu được sử dụng để kết hợp các hành động của các modules độc lập.

Ví dụ 5.2

Xét bài toán thiết kế một chương trình ngân hàng đơn giản. Mục đích của chương trình này là xử lý việc gửi tiền tài khoản, rút tiền và in sao kê hàng tháng. Trong lần duyệt đầu, ta có thể hình thành thiết kế hệ thống trông như sau:



Trong thiết kế này, chương trình chính nhận được một danh sách các giao dịch đầu vào và các lời gọi các chương trình con thích hợp. Nếu chúng ta giả thiết rằng các sao kê chứa số và số dư tài khoản, thì chúng ta có thể biểu diễn tài khoản ngân hàng duy nhất bằng số nguyên và lưu mọi tài khoản ngân hàng vào một mảng số nguyên duy nhất.

Nếu sau đó, chúng ta làm mịn bài toán ‘In Sao kê’ để bao gồm nhiệm vụ nhỏ ‘In danh sách các giao dịch’, sau đó chúng ta sẽ bảo trì bản ghi các giao dịch ngân hàng. Để làm mịn như vậy, chúng ta sẽ thay mảng số nguyên bằng một cấu trúc dữ liệu khác mà ghi lại dãy các giao dịch mà đã xảy ra từ sao kê cuối cùng. Nó có thể đòi hỏi thay đổi trong hành vi của tất cả các chương trình con, vì tất cả chúng thực hiện các thao tác trên tài khoản ngân hàng.

5.1.2 Tính modularity

Chia để trị là một trong những kỹ thuật cơ bản của Khoa học máy tính. Vì các hệ thống phần mềm có thể rất phức tạp, quan trọng là chia chương trình thành các phần riêng biệt mà có thể xử lý độc lập.

Phát triển chương trình trên-xuống, khi triển khai, là một phương pháp tạo ra chương trình gồm các phần riêng biệt. Trong một số trường hợp, cũng có ích khi triển khai dưới-lên, thiết kế các phần cơ bản mà sẽ cần trong hệ thống phần mềm lớn và sau đó kết hợp chúng lại thành các hệ thống con lớn hơn. Từ năm 1970, một số phương pháp phát triển chương trình khác nhau được đề xuất.

Một phương pháp phát triển có ích, đôi khi được gọi là prototyping, bao gồm các phần cài đặt chương trình theo cách đơn giản để hiểu xem thiết kế có làm việc thực tế không. Sau khi thiết kế được kiểm chứng theo một cách nào đó, có thể cải tiến các phần của chương trình một cách độc lập bằng cách cài đặt chúng. Quá trình này được thực hiện tiến triển từng bước bằng chuỗi các prototypes tỉ mỉ hơn để phát triển hệ thống đáp ứng yêu cầu. Ở đây cũng có phương pháp thiết kế hướng đối tượng mà sẽ bàn đến trong chương sau.

Một cách quan trọng để ngôn ngữ lập trình hỗ trợ các phương pháp lập trình modular là giúp lập trình viên theo dõi được sự phụ thuộc giữa các phần khác nhau của hệ thống. Để dễ tranh luận, chúng ta gọi phần có ý nghĩa của chương trình là phần chương trình được tách độc lập với các phần khác.

Hai khái niệm quan trọng trong phát triển phần mềm modular là giao diện và đặc tả:

- **Giao diện:** mô tả các phần của một thành phần mà được nhìn thấy đối với các thành phần khác của một chương trình.

- Đặc tả: mô tả hành vi của một thành phần, như quan sát được qua giao diện của nó.

Khi chương trình được thiết kế modularity, có thể thay đổi cấu trúc bên trong của thành phần bất kỳ, trong khi hành vi được nhìn thấy qua giao diện được giữ không thay đổi.

Ví dụ đơn giản của thành phần chương trình là hàm. Giao diện của hàm gồm tên hàm, số và kiểu các tham số, và kiểu của kết quả trả về. Giao diện của hàm còn được gọi là function header.

Đặc tả hàm thường mô tả quan hệ giữa các đối số của hàm số và giá trị trả về tương ứng. Nếu hàm số làm việc đúng đắn chỉ trên một số đối số, thì hạn chế này cần phải là một phần của đặc tả hàm. Chẳng hạn, giao diện của hàm khai căn bậc hai có thể như sau:

function sqrt (float x) returns float

Còn đặc tả đối với hàm này có thể được viết dạng:

If $x > 0$, then $\text{sqrt}(x) * \text{sqrt}(x) \approx x$.

Trong đó dấu gần bằng được sử dụng theo nghĩa tính toán với các số dấu phẩy động được thực hiện với độ chính xác nào đó.

Trong một số dạng lập trình modular, người thiết kế hệ thống viết đặc tả cho mỗi thành phần. Khi mỗi thành phần được cài đặt, nó cần được thiết kế để làm việc đúng đắn khi tương tác với mọi thành phần thỏa đặc tả của chúng. Nói cách khác, tính đúng đắn của mỗi thành phần không phụ thuộc vào chi tiết cài đặt của thành phần bất kỳ nào khác. Một lý do để cố gắng đạt được mức độ độc lập này là, nó cho phép các thành phần được cài đặt lại một cách độc lập. Đặc biệt, trong hệ thống mà mỗi thành phần chỉ dựa trên đặc tả đã phát biểu của thành phần khác, chúng ta có thể thay thế thành phần bất kỳ bằng cái khác mà thỏa mãn cùng đặc tả đó. Điều đó cho phép chúng ta tối ưu các thành phần một cách độc lập hoặc bổ sung các chức năng mà không vi phạm đặc tả ban đầu.

Có nhiều ngôn ngữ lập trình và phương pháp khác nhau để viết đặc tả, trải rộng từ tiếng Anh cho đến các ký hiệu đồ họa mà có một ít cấu trúc ngôn ngữ hình thức, và có thể thao tác với các công cụ đặc tả. Vấn đề cơ bản gắn với đặc tả chương trình là không có phương pháp có tính thuật toán để kiểm tra module có thỏa mãn đặc tả của nó không. Điều này là hệ quả của hạn chế toán học nền tảng, tương tự như tính không giải được của bài toán dừng. Vì vậy lập trình với đặc tả đòi hỏi nỗ lực to lớn và tính kỷ luật.

Để mô tả việc sử dụng các cấu trúc dữ liệu và đặc tả, chúng ta xét thuật toán sắp xếp mà sử dụng cấu trúc dữ liệu tổng quan và cũng phục vụ các mục đích khác.

Ví dụ 5.3. Thuật toán sắp xếp modular

Hàng đợi ưu tiên nguyên là cấu trúc dữ liệu với ba thao tác:

```

empty : pqueue
insert : int * pqueue → pqueue
deletemax : pqueue → int * pqueue

```

Diễn tả bằng lời, đây là cách thể hiện hàng đợi ưu tiên rỗng, và thao tác insert mà bổ sung một số nguyên vào hàng đợi ưu tiên, và thao tác deletemax mà loại bỏ một phần tử từ hàng đợi ưu tiên. Ba thao tác này tạo thành giao diện đến hàng đợi ưu tiên. Để làm chi tiết hơn, chúng ta có các đặc tả sau:

- Mỗi hàng đợi có một multiset các phần tử. Có một thứ tự $\leq$ trên các phần tử mà được sử dụng để đặt trên hàng đợi ưu tiên. (Đối với hàng đợi ưu tiên nguyên, chúng ta sử dụng thứ tự $\leq$ thông thường trên số nguyên).
- Hàng đợi ưu tiên rỗng không có phần tử.
- Insert (elt, pq) trả về hàng đợi ưu tiên có các phần tử là elt và các phần tử của pq.
- deletemax(pq) trả về phần tử của pq mà lớn hơn mọi phần tử khác của pq, đồng thời với cấu trúc dữ liệu thể hiện hàng đợi nhận được khi phần tử đó bị loại bỏ.

Các đặc tả này không buộc các hạn chế nào trên việc cài đặt hàng đợi ưu tiên khác với các tính chất mà quan sát được qua giao diện các hàng đợi ưu tiên.

Biết trước là chúng ta sẽ sử dụng các hàng đợi ưu tiên trong thuật toán sắp xếp, chúng ta có thể bắt đầu quá trình thiết kế trên xuống của chúng ta bằng việc trình bày bài toán dưới dạng của chương trình:

Chương trình 1:

```

function sort
begin
  sort an array of integers
end

```

Bước tiếp theo là làm mịn câu lệnh sort trên mảng số nguyên thành các bài toán con. Một cách để làm điều đó mà sử dụng các hàng đợi ưu tiên là truyền các phần tử của mảng vào hàng đợi ưu tiên và sau đó loại bỏ chúng mỗi lần một phần tử. Thêm vào đó, chúng ta sẽ quyết định tại điểm đó là hàm có nhận mảng và độ dài nguyên của nó như các đối số độc lập.

Chương trình 2:

```

function sort(n:int, A : array [1..n] of int)
begin
  place each element of array A in a priority queue
  remove elements in decreasing order and place in array A
end

```

Cuối cùng, chúng ta có thể chuyển các câu lệnh mô tả bằng tiếng Anh sang dạng mã chương trình nào đó. (Ở đây, chương trình được viết trên ký hiệu tựa Algol hay Pascal).

Chương trình 3:

```
function sort(n:int, A : array [1..n] of int)
begin priority queue s;
  s := empty;
  for i := 1 to n do s := insert(A[i], s);
  for i := n downto 1 do (A[i],s) := deletemax(s);
end
```

Một ưu điểm của thuật toán sắp xếp này là ở đây có sự tách biệt rõ ràng giữa cấu trúc điều khiển của thuật toán và cấu trúc dữ liệu cho hàng đợi ưu tiên. Chúng ta có thể cài đặt hàng đợi ưu tiên không hiệu quả bắt đầu với việc sử dụng thuật toán để lập trình và sau đó tối ưu cài đặt khi thấy cần thiết.

Như đã viết, trông có vẻ khó sắp xếp mảng tại chỗ bằng thuật toán này. Tuy nhiên, có thể đến gần hơn với ý tưởng thuật toán heapsort.

5.2 Ngôn ngữ hỗ trợ trừu tượng

Các lập trình viên và các thiết kế phần mềm thường nói về ‘tìm trừu tượng đúng đắn’ cho bài toán. Điều đó có nghĩa là họ tìm các khái niệm tổng quan, như các cấu trúc dữ liệu hoặc metaphores xử lý mà làm cho bài toán chi tiết, phức tạp trông có trật tự và có hệ thống hơn. Một cách mà ngôn ngữ lập trình có thể giúp các lập trình viên tìm được trừu tượng đúng đắn là cung cấp nhiều cách tổ chức dữ liệu và tính toán. Một cách khác, mà ngôn ngữ lập trình có thể giúp ích tìm trừu tượng đúng là làm cho có thể xây dựng các thành phần chương trình mà bao quát được các mẫu có nghĩa trong tính toán.

5.2.1 Trừu tượng

Trong các ngôn ngữ lập trình, trừu tượng là một cơ chế mà nhấn mạnh tính chất tổng quan của một đoạn mã nào đó và giấu các chi tiết. Các cơ chế trừu tượng nói chung bao gồm tách chương trình thành các phần nhỏ mà chứa các chi tiết nào đó và các phần mà ở đó các chi tiết này được giấu. Các thuật ngữ chung liên kết với trừu tượng là

- Client: phần chương trình mà sử dụng thành phần chương trình
- Cài đặt: phần của chương trình mà xác định thành phần chương trình.

Sự tương tác giữa client của sự trừu tượng và cài đặt của sự trừu tượng thường được hạn chế đến một giao diện chuyên biệt

Trừu tượng thủ tục

Một trong những cơ chế trừu tượng lâu đời nhất trong ngôn ngữ lập trình là thủ tục và hàm. Client của hàm là chương trình có lời gọi hàm. Cài đặt của hàm là thân hàm, mà gồm các chỉ lệnh mà được thực thi mỗi khi hàm được gọi.

Chẳng hạn, nếu ta có vài dòng code mà lưu căn bậc hai của biến x trong biến y , thì ta cần phải đóng gói đoạn code này thành hàm. Nó kèm theo một số thứ:

1. Hàm có giao diện được định nghĩa rõ ràng tường minh trong đoạn code đó. Giao diện gồm tên hàm, mà được sử dụng để gọi hàm, và các tham số đầu vào (và kiểu của chúng, nếu đây là ngôn ngữ lập trình có kiểu) và kiểu của đầu ra.
2. Nếu đoạn code để tính toán giá trị hàm sử dụng các biến khác, thì chúng cần phải là biến cục bộ đối với hàm. Nếu các biến này khai báo bên trong thân hàm, thì chúng không được nhìn thấy đối với các phần khác của chương trình mà sử dụng hàm. Nói cách khác, không phép gán hoặc việc sử dụng nào khác của biến cục bộ có ảnh hưởng phụ đến các phần khác của chương trình. Điều này cung cấp dạng thông tin ẩn, thông tin về cách hàm tính kết quả được chứa trong khai báo hàm, nhưng giấu với chương trình mà sử dụng hàm.
3. Hàm có thể được gọi trên nhiều đối số khác nhau. Nếu đoạn code tính toán được viết trên một dòng, thì tính toán này được thực hiện trên các biến đặc biệt. Bằng cách đóng gói đoạn code trong khai báo của hàm, chúng ta nhận được thực thể trừu tượng, mà có ý nghĩa tách rời khỏi việc sử dụng các biến đặc biệt này. Nói cách khác, code đóng trong hàm làm cho code tổng quát và tái sử dụng được.

Đây là mô tả lý tưởng về ưu điểm của code đóng gói bên trong hàm. Trong hầu hết các ngôn ngữ lập trình, hàm có thể đọc hoặc gán cho biến tổng thể. Các biến tổng thể này không được liệt kê trong giao diện của hàm. Do đó, hành vi của hàm không phải luôn luôn được xác định chỉ bởi giao diện của nó. Chính vì lý do này, một số nhà thiết kế chương trình đề xuất phản đối việc sử dụng các biến tổng thể trong các hàm.

Trừu tượng dữ liệu

Trừu tượng dữ liệu hướng tới việc giấu thông tin về cách dữ liệu được thể hiện. Cơ chế ngôn ngữ chung đối với trừu tượng dữ liệu là khai báo kiểu dữ liệu trừu tượng (bàn đến trong 5.2.2) và modules (bàn đến trong 5.3).

Chúng ta đã thấy trong mục 5.1.2 thuật toán sắp xếp sử dụng cấu trúc dữ liệu gọi hàng đợi ưu tiên như thế nào. Nếu chương trình sử dụng hàng đợi ưu tiên, thì người viết chương trình cần biết có các thao tác nào trên các hàng đợi và giao diện của chúng. Do đó, tập các thao tác và giao diện của chúng được gọi là giao diện của trừu tượng dữ liệu. Về nguyên tắc, chương trình sử dụng hàng đợi ưu tiên không phụ thuộc vào hàng đợi ưu tiên được biểu diễn trên cây tìm kiếm nhị phân, cây heap hay mảng sắp xếp. Các chi tiết cài đặt này được giấu tốt nhất bởi cơ chế đóng gói dữ liệu.

Như đối với trừu tượng thủ tục, ở đây có ba mục đích của trừu tượng dữ liệu:

1. Xác định giao diện của cấu trúc dữ liệu. Giao diện của trừu tượng dữ liệu gồm các thao tác trên cấu trúc dữ liệu, các đối số của nó và các kết quả trả về.

2. Cung cấp các thông tin được giấu bằng việc tách các quyết định cài đặt khỏi phần của chương trình sử dụng các cấu trúc này.
3. Cho phép cấu trúc dữ liệu được sử dụng theo nhiều cách khác nhau bởi nhiều chương trình khác nhau. Mục tiêu này được hỗ trợ tốt nhất bởi trừu tượng mẫu chung được bàn đến trong 5.4.

5.2.2 Các kiểu dữ liệu trừu tượng

Mối quan tâm về trừu tượng dữ liệu đến từ trước những năm 1970. Nó dẫn đến việc phát triển cấu trúc ngôn ngữ lập trình gọi là khai báo kiểu dữ liệu trừu tượng.

Đây là định nghĩa ngắn gọn chung về kiểu dữ liệu trừu tượng:

Kiểu dữ liệu trừu tượng bao gồm kiểu cùng với tập các thao tác chuyên biệt

Ngôn ngữ tốt cho việc lập trình với kiểu dữ liệu trừu tượng không những cho phép lập trình viên nhóm kiểu và các thao tác, mà còn sử dụng kiểm tra kiểu để hạn chế truy cập đến biểu diễn của cấu trúc dữ liệu. Nói cách khác, không chỉ kiểu dữ liệu trừu tượng giao diện đặc thù mà có thể được sử dụng như một phần của chương trình, nhưng truy cập được hạn chế sao cho chỉ sử dụng kiểu dữ liệu trừu tượng thông qua giao diện của nó.

Nếu ngăn xếp được cài đặt với mảng, thì chương trình mà sử dụng kiểu dữ liệu ngăn xếp trừu tượng chỉ có thể sử dụng các thao tác ngăn xếp (đẩy, gỡ, chẳng hạn), chứ không phải các thao tác của mảng như gọi chỉ số của mảng tại điểm bất kỳ. Nó giấu các thông tin về cài đặt cấu trúc dữ liệu và cho phép người cài đặt cấu trúc dữ liệu được thay đổi mà không ảnh hưởng đến các phần của chương trình mà sử dụng cấu trúc dữ liệu đó.

Chúng ta có thể đánh giá một số khía cạnh của các kiểu dữ liệu trừu tượng bằng việc hiểu ý tưởng lịch sử tại thời điểm phát triển của chúng. Vào đầu những năm 1970, khi đó có phong trào nghiên cứu các ngôn ngữ có thể mở rộng được. Mục đích của phong trào này là tạo ra ngôn ngữ lập trình ở đó lập trình viên có thể định nghĩa các cấu trúc với độ linh hoạt như những người thiết kế ngôn ngữ. Ví dụ, một người nào đó trong nhóm các lập trình viên muốn viết chương trình bằng việc sử dụng dạng mới của vòng lặp, họ có thể sử dụng ‘khai báo vòng lặp’ để định nghĩa nó và sử dụng nó trong chương trình của họ. Ý tưởng này sau đó được nhận thấy là không thành công, vì chương trình với mọi kiểu qui ước được định nghĩa bởi người lập trình có thể rất khó đọc và sửa. Tuy nhiên, ý tưởng mà người lập trình có thể định nghĩa kiểu mà có vị thế như mọi kiểu khác được cung cấp bởi ngôn ngữ lập trình tỏ ra hữu ích và đã được trải nghiệm.

Sự không rõ ràng tiềm ẩn về kiểu dữ liệu trừu tượng chính là ý nghĩa mà ở đó chúng được trừu tượng. Sự khác biệt là kiểu dữ liệu, mà chi tiết biểu diễn và thao tác của nó được giấu đối với client, là trừu tượng. Ngược lại, kiểu dữ liệu mà chi tiết biểu diễn được client nhìn thấy được gọi là kiểu trong suốt. ML abstype, được bàn đến trong mục sau xác định các

kiểu dữ liệu trừu tượng, trong khi đó kiểu dữ liệu ML đã xét ở mục 6.5.3 là dạng khai báo kiểu trong suốt.

5.2.3 ML abstype

Chúng ta sử dụng cấu trúc kiểu dữ liệu trừu tượng lịch sử của ML, gọi là *abstype*, để bàn luận các ý tưởng chính gắn với cơ chế kiểu trừu tượng trong ngôn ngữ lập trình.

Như đã bàn trong mục trước, cơ chế kiểu dữ liệu trừu tượng gắn kết kiểu với cấu trúc dữ liệu theo cách một tập các hàm được truy cập trực tiếp đến cấu trúc dữ liệu, nhưng code ở các phần khác của chương trình không được truy cập. Chúng ta sẽ thấy điều này làm việc trong ML như thế nào bằng việc xét ví dụ đơn giản, các số phức.

Chúng ta có thể biểu diễn số phức như một cặp các số thực. Số thực thứ nhất là phần thực của số phức và số thực thứ hai là phần ảo của số phức. Nếu chúng ta muốn tính toán với các số phức, thì chúng ta cần có cách tạo số phức từ hai số thực và các cách lấy phần thực và phần phức từ số phức. Tính toán các số phức bao gồm phép cộng, phép nhân và các phép toán chuẩn khác. Ở đây, đơn giản ta xét phép cộng. Các phép toán khác có thể đưa vào cấu trúc toán một cách tương tự.

Khai báo ML cho kiểu dữ liệu trừu tượng của số phức có thể viết như sau:

```
abstype cmplx = C of real * real with
  fun cmplx(x,y: real) = C(x,y)
  fun x_coord(C(x,y)) = x
  fun y_coord(C(x,y)) = y
  fun add(C(x1, y1), C(x2, y2)) = C(x1+x2, y1+y2)
end
```

Khai báo này ràng buộc năm định danh để sử dụng bên ngoài khai báo: kiểu *cmplx* và các hàm *cmplx*, *x_coord*, *y_coord* và *add*. Khai báo này trói tên *C* vào cấu trúc mà có thể sử dụng bên trong thân của hàm, như một phần của khai báo. Đặc biệt *C* có thể xuất hiện bên trong code đối với *cmplx*, *x_coord*, *y_coord* và *add*, nhưng không ở phần khác của chương trình.

Tên kiểu *cmplx* là kiểu số phức. Khi chương trình sử dụng các số phức, mỗi số phức được biểu diễn bên trong như một cặp số thực. Tuy nhiên, vì tên kiểu *cmplx* là khác so với kiểu ML *real*real* cho cặp số thực, hàm mà áp dụng trên cặp số thực không thể áp dụng cho giá trị kiểu *cmplx*. Kiểm tra kiểu ML không cho phép điều đó. Hạn chế này là một trong những tính chất căn bản của bất cứ cơ chế kiểu trừu tượng tốt nào. Chương trình sẽ được hạn chế sao cho chỉ có các thao tác khai báo của kiểu trừu tượng mới được áp dụng.

Tuy nhiên, bên trong khai báo kiểu dữ liệu, các hàm mà là các phần của kiểu dữ liệu trừu tượng cần phải có khả năng xử lý các số phức như cặp các số thực. Mặt khác, cũng không thể cài đặt nhiều thao tác. Trong ML, có cấu trúc được sử dụng để phân biệt việc dùng kiểu số phức ‘trừu tượng’ với kiểu ‘cụ thể’. Đặc biệt, nếu *z* là số phức, thì việc sánh *z* với mẫu *C(x,y)* sẽ trói *x* là phần thực của *z* và *y* là phần ảo của *z*. Dạng này của sánh mẫu được

sử dụng để cài đặt chẳng hạn phép cộng, ở đó phép cộng kết hợp hai phần thực của hai đối số và hai phần ảo của hai đối số. Khi đó cặp biểu diễn số phức tổng được xác định như số phức nhờ việc áp dụng cấu trúc *C*.

Khi ML được biểu diễn với khai báo này của số phức, nó trả về thông tin kiểu sau:

```
type cmplx
val cmplx = fn : real * real → cmplx
val x_coord = fn : cmplx → real
    val y_coord = fn : cmplx → real
    val add = fn : cmplx * cmplx → cmplx
```

Dòng đầu chỉ ra rằng khai báo này đưa ra kiểu mới, tên là *cmplx*. Bốn dòng tiếp theo liệt kê các thao tác được phép trên các biểu thức kiểu *cmplx*. Các kiểu của các thao tác này bao gồm kiểu *cmplx*, không phải kiểu *real*real* mà được sử dụng để biểu diễn số phức.

Nói chung, khai báo ML abstype có dạng sau:

```
abstype t = <constructor> of <type>
with
    val <pattern> = <body>
    ...
    fun f(<pattern>) = <body>
    ...
end
```

Cú pháp

```
t = <constructor> of <type>
```

là ký hiệu giống nhau được sử dụng để định nghĩa kiểu dữ liệu. Định danh *t* là tên của kiểu mới, <constructor> là tên hàm tạo của kiểu mới *t* và <type> cho kiểu được dùng để biểu diễn các phần tử của kiểu trừu tượng. Sự khác nhau giữa abstype và kiểu dữ liệu nằm ở phần sau của cú pháp trước đó. Giá trị và các khai báo hàm xảy ra giữa các từ khóa with và end chỉ là các thao tác mà có thể được viết với hàm tạo. Các phần khác của chương trình có thể tham chiếu đến tên kiểu *t* và các giá trị được khai báo giữa with và end. Tuy nhiên, các phần khác của chương trình mà nằm ngoài phạm vi khai báo của hàm tạo và do đó không thể chuyển giữa kiểu trừu tượng và biểu diễn của nó. Các thao tác khai báo trong khai báo abstype được gọi là giao diện của kiểu trừu tượng và kiểu dữ liệu ẩn và thân các hàm liên kết được gọi là cài đặt.

Như nhiều bạn đọc đã biết, có hai cách biểu diễn số phức. Kiểu trừu tượng trên sử dụng các tọa độ vuông góc – mỗi số phức được biểu diễn bởi cặp gồm tọa độ thực và ảo của nó. Biểu diễn chuẩn cách khác được gọi là tọa độ cực. Trong biểu diễn cực, mỗi số phức được biểu diễn bằng khoảng cách đến gốc tọa độ và góc cực chỉ hướng so với trục thực. Vì phần cài đặt kiểu dữ liệu trừu tượng *cmplx* được che giấu, nên chương trình mà sử

dụng tọa độ vuông góc có thể được thay bằng chương trình sử dụng tọa độ cực mà không thay đổi hành vi của bất cứ chương trình nào sử dụng kiểu trừu tượng này.

Biểu diễn cực của số phức được sử dụng trong khai báo kiểu dữ liệu trừu tượng:

```
abstype cmplx = C of real * real with
  fun cmplx(x,y: real) = C(sqrt(sq(x)+sq(y)), arctan(y/x))
  fun x_coord(C(r, theta)) = r * cos(theta)
  fun y_coord(C(r, theta)) = r * sin(theta)
  fun add(C(x1,y1), C(x2,y2)) = C(..., ...)
end
```

Trong đó cài đặt *add* sẽ được bổ sung cho phù hợp.

Ví dụ 5.4. Kiểu trừu tượng tập hợp.

Chúng ta có thể tạo kiểu dữ liệu trừu tượng *abstype* đa hình, như khai báo *abstype* như sau:

```
abstype 'a set = SET of 'a list with
  val empty = SET(nil)
  fun insert(x, SET(elts)) = ...
  fun union(SET(elts1), SET(elts2)) = .
  fun isMember(x, SET(elts)) = ...
end
```

Giả thiết rằng các chỗ để trống ... được lấp đầy bằng các đoạn mã thích hợp để cài đặt *insert*, *union* và *isMember*, ML sẽ trả về kết quả của khai báo trên như sau:

```
type 'a set
val empty = - : 'a set
val insert = fn : 'a * ('a set) -> ('a set)
val union = fn : ('a set) * ('a set) -> ('a set)
val isMember = fn : 'a * ('a set) -> bool
```

Nhận thấy rằng giá trị cho tập rỗng được viết là - thay vì *nil*. Việc che giấu này ngăn cản người sử dụng kiểu dữ liệu trừu tượng *abstype a'set* biết là nó được cài đặt như một danh sách.

5.2.4 Độc lập biểu diễn

Chúng ta có thể hiểu tầm quan trọng của khai báo kiểu trừu tượng bằng việc xét một số tính chất của một số kiểu có sẵn như *int*:

- Ta có thể khai báo biến *x: int* của kiểu này
- Có tập các thao tác sẵn có trên kiểu này +, -, *, ...
- Chỉ có những thao tác sẵn có này có thể áp dụng cho giá trị của kiểu *int*, không là kiểu đúng khi áp dụng sâu hoặc các kiểu thao tác khác cho số nguyên.

Vì *int* có thể được truy cập bởi các thao tác có sẵn, chúng thỏa mãn với tính chất độc lập biểu diễn, có nghĩa là các biểu diễn số nguyên của các máy tính khác nhau không ảnh hưởng đến hành vi của chương trình. Một máy tính có thể biểu diễn số nguyên dạng bù 1, và cái khác dùng dạng bù 2, và cùng một chương trình chạy trên hai máy tính sẽ tạo các đầu ra như nhau (giả thiết mọi cái khác đều bằng nhau).

5.2.5 Qui nạp kiểu dữ liệu

Qui nạp kiểu dữ liệu là nguyên lý có ích để suy luận về kiểu dữ liệu trừu tượng. Chúng ta không quan tâm ở đây về khía cạnh hình thức của nguyên lý này, mà chỉ nói cách suy nghĩ về lập trình và tương đương kiểu dữ liệu. Tương đương kiểu dữ liệu là quan hệ quan trọng giữa các kiểu dữ liệu trừu tượng. Chúng ta có thể thay thế một kiểu dữ liệu bằng kiểu dữ liệu tương đương bất kỳ mà không thay đổi hành vi của bất cứ chương trình *client* nào. Nguyên lý này được sử dụng không hình thức trong phát triển và bảo trì chương trình. Cụ thể, nói chung trước hết xây dựng hệ thống phần mềm với việc cài đặt prototype tiềm năng không hiệu quả của một kiểu dữ liệu và sau đó thay nó bằng cài đặt hiệu quả hơn nếu thời gian cho phép.

Phân loại các thao tác

Đối với nhiều kiểu dữ liệu, có thể tách các thao tác trên kiểu thành ba nhóm:

1. Constructors (các hàm tạo): thao tác tạo các phần tử của kiểu
2. Operator: các thao tác mà ánh xạ các phần tử của kiểu mà chỉ được định nghĩa bởi constructors vào các phần tử khác của kiểu mà cũng được định nghĩa bởi constructors
3. Observers: các thao tác mà trả về kết quả của một kiểu khác nào đó.

Ý tưởng chính là mọi phần tử của kiểu dữ liệu có thể được định nghĩa bởi constructors; các thao tác operators có ích để tính toán với các phần tử của kiểu, nhưng không định nghĩa các giá trị mới. Observers là các hàm cho phép ta phân biệt một phần tử của kiểu này với kiểu khác. Chúng cho ta nhận biết về sự bằng nhau của quan sát mà thông thường khác với sự bằng nhau về biểu diễn.

Ví dụ 5.5

Đối với kiểu dữ liệu tập số nguyên

```
empty : set
insert : int * set → set
union : set * set → set
isMember: int * set → bool
```

Thao tác có thể chia ra như sau:

1. *Constructors: empty and insert*
2. *Operator: union*
3. *Observer: isMember*

Chúng ta có thể hiểu về cách chia các thao tác như vậy bằng cách nghĩ về các tập hợp có thể được sử dụng như thế nào trong chương trình. Vì ở đây không có thao tác print trên tập hợp, chương trình không thể tạo ra tập hợp trực tiếp như đầu ra. Thay vào đó, nếu mọi đầu ra in được của chương trình phụ thuộc vào giá trị của một biểu thức tập hợp nào đó, nó có thể chỉ vì có phép kiểm tra thành viên nào đó trên tập hợp. Do đó, nếu có hai tập hợp, $s1$ và $s2$ có tính chất

$$\text{Đối với mọi số nguyên } n: \text{isMember}(n, s1) = \text{isMember}(n, s2)$$

thì không chương trình nào có thể phân biệt tập này với tập khác dựa trên cách quan sát bất kỳ. Điều đó trên thực tế cho chúng ta quan hệ tương đương hữu ích trên tập hợp: Hai tập hợp $s1$ và $s2$ tương đương, nếu $\text{isMember}(n, s1) = \text{isMember}(n, s2)$ đối với mỗi số nguyên n . Đối với tập hợp, nguyên lý tương đương này trên thực tế là tiên đề khai triển của lý thuyết tập hợp: Hai tập hợp bằng nhau khi chúng có đúng các phần tử như nhau.

Cho chi tiết các phần tử của tập hợp, dễ dàng thấy mọi tập hợp có thể được định nghĩa bằng việc bổ sung một số phần tử vào tập rỗng. Cụ thể hơn, đối với mỗi tập hợp s , ở đây có một dãy c các phần tử, $n1, n1, \dots, nk$ với

$$s \approx \text{insert}(n1, \text{insert}(n2, \dots \text{insert}(nk, \text{empty}) \dots)).$$

Điều này cho thấy insert và empty là các hàm tạo constructors đối với kiểu dữ liệu tập hợp. Mỗi tập hợp có thể được định nghĩa với chỉ hai thao tác này.

Để một cách hình thức chỉ ra rằng phương thức đã cho là operator, chúng ta cần chỉ ra rằng, đối với một lần sử dụng cho trước một operator, có một dãy những lời gọi constructor mà tạo ra cùng kết quả. Như chúng ta mong đợi, union là thao tác có ích trên tập hợp, nhưng nếu $s1$ và $s2$ được định nghĩa với các thao tác của kiểu dữ liệu này, thì $\text{union}(s1, s2)$ có thể được định nghĩa chỉ với insert và empty. Với lý do này union được phân loại như operator chứ không phải constructor.

Trên thực tế, không phải khi nào cũng dễ dàng tách các thao tác của một kiểu dữ liệu vào ba nhóm đó. Một số hàm trông có vẻ phù hợp vào hai nhóm. Tuy nhiên, nguyên lý qui nạp kiểu dữ liệu vẫn cho chỉ dẫn có ích để suy luận về các kiểu dữ liệu trừu tượng bất kỳ.

Qui nạp trên Constructors

Vì mọi phần tử của kiểu trừu tượng cho trước được cho bởi một dãy các thao tác constructor, chúng ta có thể chứng minh các tính chất của mọi phần tử của kiểu trừu tượng bằng qui nạp trên số lần sử dụng constructor cần thiết cần thiết để sinh ra phần tử đã cho.

Một khi ta đã chỉ ra rằng một hàm nào đó đã là operator, nói chung ta có thể loại nó ra không cần xét tiếp.

Ví dụ 5.6

Như minh chứng cho qui nạp kiểu dữ liệu, chúng ta sẽ lướt qua dàn ý chứng minh rằng hai cài đặt khác nhau của tập số nguyên là tương đương. Khái niệm tương đương có nghĩa là, nếu ta thay một cài đặt này bằng một cài đặt khác, thì không chương trình client nào có thể phát hiện được sự thay đổi đó.

Giả sử ta bắt đầu xét định nghĩa tương đương, tính chất mà ta muốn chứng minh là:

Hai cài đặt set và set' là tương đương, nếu mọi giá trị của mọi tham số, mọi áp dụng của observers đến các biểu thức tập hợp là bằng nhau.

Chúng ta tham chiếu đến các phép toán trên set bằng *empty*, *insert*, *union*, *isMember* và các thao tác trên set' bằng *empty'*, *insert'*, *union'*, *isMember'*. Một số ví dụ các áp dụng tương ứng của observers là:

$isMember(6, insert(n1, \dots insert(nk, empty) \dots))$ và
 $isMember'(6, insert'(n1, \dots insert'(nk, empty') \dots))$

Các biểu thức này tương ứng theo nghĩa mọi đối số không phải tập hợp là như nhau, nhưng chúng ta phải thay các thao tác của một cài đặt này bằng của cài đặt khác.

Cảm giác là định nghĩa tương đương kiểu dữ liệu này cũng tương tự như quan hệ tương đương trên biểu thức tập hợp mà chúng ta đã bàn trước đây. Chỉ có một cách mà chương trình *client* có thể sử dụng một trong chúng để tạo đầu ra in được (quan sát được) là sử dụng các hàm tạo tập hợp (*set constructors*) và các phép toán (*operators*) để xây dựng nên các tập hợp có tiềm năng phức tạp và sau đó quan sát (*observer*) các tập hợp kết quả bằng việc sử dụng các thao tác *observer*.

Vì chúng ta đã thiết lập rằng *union* là operator, không phải là constructor, và chỉ có hàm quan sát là *isMember*, việc chứng minh sự tương đương của hai cài đặt khác nhau của tập hợp đưa về việc chỉ ra rằng

với mọi z $isMember(z, aSet) = isMember'(z, aSet')$

trong đó $aSet$ và $aSet'$ là các biểu thức tương ứng mà ở đó chỉ có hàm tạo *empty* và *insert* (hoặc *empty'* và *insert'*) là được sử dụng.

Bây giờ chúng ta có thể qui bài toán thiết lập sự tương đương kiểu dữ liệu về bài toán chỉ ra rằng

$isMember(n, insert(n1, \dots insert(nk, empty) \dots))$

$$= \text{isMember}'(n, \text{insert}'(n1, \dots \text{insert}'(nk, \text{empty}') \dots))$$

đối với mọi dãy số tự nhiên $n, n1, \dots, nk$. Chúng ta có thể làm điều này qui nạp theo k , và số các thao tác insert được đòi hỏi để xây dựng các tập hợp đó.

Chúng minh qui nạp được tiến hành như sau:

Trường hợp cơ sở: Thao tác chèn Zero. Trong trường hợp này chúng ta cần phải chỉ ra rằng

$$\text{Với mọi } n: \text{isMember}(n, \text{empty}) = \text{isMember}'(n, \text{empty}')$$

Chúng ta cần làm điều này bằng cách xem cài đặt thực tế của kiểu dữ liệu đó. Tuy nhiên, trong cài đặt đúng của tập hợp, tập rỗng không có phần tử nào. Do đó, nếu cả hai cài đặt là đúng, thì $\text{isMember}(n, \text{empty}) = \text{isMember}'(n, \text{empty}') = \text{false}$.

Bước qui nạp. Giả thiết rằng sự tương đương là đúng khi k phép toán chèn được thực hiện và xét trường hợp $k+1$ phép toán chèn. Điều đó dẫn đến việc chỉ ra rằng

$$\text{isMember}(n, \text{insert}(m, s)) = \text{isMember}'(n, \text{insert}'(m, s'))$$

dưới giả thiết đối với mọi n ta có $\text{isMember}(n, s) = \text{isMember}'(n, s')$. Và ta lại phải xem cài đặt thực tế. Tuy nhiên, nếu cả hai cài đặt là đúng, thì ta cần phải có $\text{isMember}(n, s) = \text{isMember}'(n, s')$.

Một khía cạnh thú vị của cách lập luận này là chúng ta đã chứng minh một điều gì đó về mọi chương trình có thể mà sử dụng một kiểu dữ liệu bằng cách chỉ sử dụng qui nạp thông thường trên các constructors. Nguyên nhân để điều này có thể là giả thiết rằng trong ngôn ngữ với các kiểu dữ liệu trừu tượng, chỉ có thao tác của một kiểu dữ liệu có thể được áp dụng cho các giá trị của kiểu dữ liệu đó. Không thể sử dụng dạng chứng minh này, nếu qui tắc kiểm tra kiểu không đảm bảo rằng chỉ có các thao tác tập hợp có thể áp dụng cho tập hợp. Trên thực tế, tuy nhiên, ý tưởng thể hiện ở đây có thể có ích cho việc lập trình trong các ngôn ngữ lập trình như C mà không ép buộc trừu tượng dữ liệu, như các chương trình thực tế được xây dựng không thao tác trên các cấu trúc dữ liệu trừ các thao tác được thiết kế cho mục đích này.

“Chứng minh” được mô tả trên đây thực tế là dàn ý chứng minh mà giả thiết một số tính chất của mỗi cài đặt trên tập hợp. Để hiểu tại sao qui nạp kiểu dữ liệu lại tiến hành trên thực tế được bạn có thể thực hiện các bước chứng minh tương đương với hai cài đặt cụ thể trong đầu. Chẳng hạn, bạn có thể sử dụng qui nạp kiểu dữ liệu để chứng minh sự tương đương của cài đặt danh sách móc nối đơn và danh sách móc nối kép cho tập hợp.

5.3 Module

Các cơ chế kiểu dữ liệu trừu tượng trước kia chỉ khai báo một kiểu dữ liệu. Nếu bạn chỉ muốn kiểu dữ liệu trừu tượng của ngăn xếp, hàng đợi, cây hoặc các cấu trúc dữ liệu chung khác, thì dạng này là đủ: trong mỗi ví dụ này, chỉ có một kiểu cấu trúc dữ liệu được định nghĩa và nó cần phải là kiểu trừu tượng. Tuy nhiên có những tình huống mà ở đó sẽ có ích nếu định nghĩa một số kiểu dữ liệu trừu tượng. Tổng quát hơn, một tập hợp các kiểu, các hàm, các ngoại lệ và một số thực thể do người sử dụng định nghĩa có thể liên quan nhau và có các cài đặt phụ thuộc vào nhau.

Module là cấu trúc ngôn ngữ lập trình mà cho phép một số các khai báo nhóm với nhau. Dạng ban đầu của Module như trong ngôn ngữ Modula, cung cấp việc giấu thông tin tối thiểu. Tuy nhiên cơ chế module tốt sẽ cho phép lập trình viên kiểm soát tính nhìn thấy của một số cái khai báo trong module. Thêm vào đó, module tham số hóa, như mô tả trong mục sau, là cho nó có thể khái quát tập hợp các khai báo và khởi tạo chúng theo các cách khác nhau với các mục đích khác nhau.

5.3.1 ML Modules

Hệ thống module ML chuẩn được thiết kế vào giữa 1980 như một phần thiết kế lại và nỗ lực chuẩn hóa cho ngôn ngữ lập trình ML. Kiến trúc sư chính của hệ thống module ML là David MacQueen, người dựa trên các khái niệm từ lý thuyết kiểu và kinh nghiệm của ông với các ngôn ngữ lập trình trước đó.

Ba phần chính của hệ thống module ML chuẩn là: structures, signatures và functors. Cấu trúc ML là module mà là tập hợp của kiểu, giá trị và khai báo cấu trúc. Signature là giao diện module. Trong ML chuẩn, signature giống như dạng của kiểu đối với structure theo nghĩa một module có thể có nhiều hơn một signature và một signature có thể có nhiều hơn một module liên kết. Nếu structure thỏa mãn mô tả trong signature, structure sánh được với signature.

Functors là hàm từ structures vào structures. Functors được dùng để định nghĩa các module chung. Vì ML không hỗ trợ functors bậc cao (functors mà nhận các functors như các đối số hoặc sinh ra các functors như kết quả), nên không cần cho signature của functor.

Structure được định nghĩa với biểu thức structure, mà gồm dãy các khai báo giữa các từ khóa struct và end. Structures không là “lớp đầu tiên” mà ở đó chúng có thể chỉ bị trói buộc với định danh của cấu trúc hoặc được truyền như đối số cho functors. Khai báo sau định nghĩa structure với một kiểu và một thành phần giá trị.

```
structure S =
  struct
    type t = int
    val x:t=3
  end
```

Trong ví dụ này, biểu thức cấu trúc tiếp theo sau dấu bằng có thành phần kiểu t bằng int và thành phần giá trị x bằng 3. Trong ML chuẩn, cấu trúc này là nhãn thời gian, khi khai báo

được tạo ra, đánh dấu nó với tên duy nhất, phân biệt nó với bất kỳ cấu trúc khác với cùng các thành phần kiểu và giá trị. Biểu thức cấu trúc do đó được gọi là “generative” vì mỗi lần tạo ra được coi như sinh ra một cái mới. Lý do làm cho biểu thức cấu trúc generative là ngôn ngữ module đó cung cấp dạng kiểm soát phiên bản dựa trên việc chỉ rõ là hai cấu trúc hoặc kiểu có thể khác biệt cần phải bằng nhau.

Các thành phần của một structure được truy cập qua tên hạn chế, được viết dạng sử dụng để truy cập record trong nhiều ngôn ngữ. Chẳng hạn, cho trước khai báo cấu trúc trên đối với S , tên $S.x$ tham chiếu đến thành phần x của S và do đó có giá trị 3. Tương tự, $S.t$ tham chiếu đến thành phần t của S và tương đương với kiểu int qua kiểm tra kiểu. Nói theo cách khác, khai báo kiểu trên cấu trúc là trong suốt theo mặc định.

Signatures ML là giao diện cấu trúc và có thể khai báo như sau:

```
signature SIG =
  sig
    type t
    val x : t
  end
```

Signature trên mô tả cấu trúc mà có thành phần kiểu t và thành phần giá trị x , kiểu của nó là kiểu gắn với t trong cấu trúc đó. Vì cấu trúc S trên thỏa mãn các điều kiện này, nó được nói là sánh với với signature SIG. Cấu trúc S còn sánh với signature SIG' sau:

```
signature SIG' =
  sig
    type t

    val x : int
  end
```

Signature này được sánh bởi cấu trúc bất kỳ cung cấp kiểu t và giá trị x của kiểu int như cấu trúc S . Tuy nhiên có các cấu trúc mà sánh SIG, nhưng không sánh SIG', đó là cấu trúc bất kỳ mà cung cấp kiểu khác với int và giá trị của kiểu đó. Làm tăng thêm tính nhập nhằng của dạng đó, ở đây có lý do khác thực tế hơn là tại sao một cấu trúc cho trước có thể sánh với nhiều signature khác nhau. Các signatures có thể được sử dụng để cung cấp các views khác nhau của một cấu trúc. Ý tưởng chính là signature đó có thể mô tả ít hơn các thành phần mà trên thực tế được cung cấp. Chẳng hạn chúng ta có thể đưa ra signature

```
signature SIG'' =
  sig
    val x : int
  end
```

và kéo theo định nghĩa view T của cấu trúc S bằng việc khai báo

```
structure t : SIG'' = S
```

Rõ ràng là S sánh với signature SIG' vì nó cung cấp thành phần x kiểu int . Signature SIG' trong khai báo của T tạo ra thành phần t của S được giấu, và do đó chỉ có $T.x$ là được sử dụng.

Ví dụ 5.9 Các signatures và structures hình học trong ML

Ví dụ này cung cấp signatures và structures cho chương trình hình học đơn giản. Một functor liên kết và một cấu trúc tham số được sử dụng, trình bày trong ví dụ 5.11. Ba signatures sau points, circles và rectangles, với mỗi signature chứa tên kiểu và tên các thao tác liên kết. Hai signatures sử dụng SML gồm cả lệnh chứa signature trước. Hiệu quả của việc include này giống như sao chép thân của signature có tên đó và đặt nó vào trong biểu thức signature chứa câu lệnh include.

```
signature Point =
sig
  type point
  val mk_point : real * real → point
  val x_coord : point -> real

  val y_coord : point -> real
  val move_p : point * real * real → point
end;
signature Circle =
sig
  include Point
  type circle
  val mk_circle : point * real → circle
  val center : circle → point
  val radius : circle → real
  val move_c : circle * real * real → circle
end;
signature Rect =
sig
  include Point
  type rect
  (* make rectangle from lower right, upper left corners *)
  val mk_rect : point * point → rect
  val lleft : rect → point
  val uright : rect → point
  val move_r : rect * real * real → rect
end;
```

Đây là code cho các cấu trúc Point, Circle và Rect:

```

structure pt : Point =
  struct
    type point = real*real
    fun mk_point(x,y) = (x,y)
    fun x_coord(x,y) = x
    fun y_coord(x,y) = y
    fun move_p((x,y):point,dx,dy) = (x+dx, y+dy)
  end;
structure cr : Circle =
  struct
    open pt
    type circle = point*real
    fun mk_circle(x,y) = (x,y)
    fun center(x,y) = x
    fun radius(x,y) = y
    fun move_c(((x,y),r):circle,dx,dy) = ((x+dx, y+dy),r)
  end;
structure rc : Rect =
  struct
    open pt

    type rect = point * point
    fun mk_rect(x,y) = (x,y)
    fun lleft(x,y) = x
    fun uright (x,y) = y
    fun move_r(((x1,y1),(x2,y2)):rect,dx,dy) =
      ((x1+dx,y1+dy),(x2+dx,y2+dy))
  end

```

Đây là code cho các cấu trúc Point, Circle và Rect:

```

structure pt : Point =
  struct
    type point = real*real
    fun mk_point(x,y) = (x,y)
    fun x_coord(x,y) = x
    fun y_coord(x,y) = y
    fun move_p((x,y):point,dx,dy) = (x+dx, y+dy)
  end;
structure cr : Circle =
  struct
    open pt
    type circle = point*real
    fun mk_circle(x,y) = (x,y)
    fun center(x,y) = x
    fun radius(x,y) = y
    fun move_c(((x,y),r):circle,dx,dy) = ((x+dx, y+dy),r)
  end;
structure rc : Rect =
  struct
    open pt

    type rect = point * point
    fun mk_rect(x,y) = (x,y)
    fun lleft(x,y) = x
    fun uright (x,y) = y
    fun move_r(((x1,y1),(x2,y2)):rect,dx,dy) =
      ((x1+dx,y1+dy),(x2+dx,y2+dy))

  end;

```

5.4 Trừu tượng tổng quát

Kiểu dữ liệu trừu tượng như ngăn xếp hoặc hàng đợi là có ích để lưu trữ nhiều kiểu dữ liệu. Tuy nhiên, trong các ngôn ngữ lập trình có kiểu, code cho ngăn xếp số nguyên khác với code cho ngăn xếp xâu. Hai phiên bản khác nhau của ngăn xếp được viết với hai khai báo kiểu khác nhau và có thể được dịch sang code mà chiếm kích cỡ không gian khác nhau cho các biến cục bộ. Tuy nhiên, sẽ tốn thời gian để viết các phiên bản khác nhau cho ngăn xếp có các kiểu phần tử khác nhau và là vô nghĩa vì mã cho hai trường hợp hầu như là giống nhau. Vì vậy, hầu hết các ngôn ngữ kiểu mà nhấn mạnh tính trừu tượng và đóng gói có dạng nào đó của tham số kiểu.

5.4.1 C++ Function Templates

Đối với nhiều bạn đọc, cơ chế tham số kiểu quen thuộc nhất là cơ chế *template* của C++. Tuy một số lập trình viên C++ gắn kết *templates* với các lớp và lập trình hướng đối tượng, các *templates* hàm còn có ích cho các chương trình mà không khai báo bất cứ lớp nào.

Chúng ta xem xét *templates* hàm trước khi tìm hiểu cơ chế tham số hóa module của các ngôn ngữ khác.

Hàm đa hình đơn giản.

Giả sử bạn viết hàm đơn giản đảo giá trị của hai biến nguyên:

```
void swap(int& x, int& y){
    int tmp=x;x=y;y= tmp;
}
```

Mặc dù đoạn code này dùng để đảo giá trị hai biến nguyên, nó không được viết một cách tổng quát nhất có thể. Nếu bạn muốn đảo giá trị của hai biến kiểu bất kỳ, thì bạn cần định nghĩa template hàm mà sử dụng biến kiểu T thay thế cho tên kiểu int:

```
template<typename T>
void swap(T& x, T& y){
    T tmp=x;x=y;y= tmp;
}
```

Ý tưởng chính là nghĩ về tên kiểu T như một tham số của hàm từ các kiểu truyền cho hàm. Khi áp dụng hoặc khởi tạo cho một kiểu cụ thể nào đó, kết quả là phiên bản của swap mà có kiểu int được thay thế bởi kiểu khác. Nói cách khác, swap là hàm tổng quát mà có thể làm việc hoàn toàn đúng cho nhiều kiểu đối số. Template cho phép ta coi swap như hàm với đối số kiểu.

Trong C++, templates hàm được khởi tạo tự động khi cần thiết, sử dụng kiểu đối số hàm để xác định khởi tạo nào là cần thiết. Nó được mô tả trong các dòng code sau:

```
int i,j; ... swap(i,j); // Use swap with T replaced by int
float a,b; ... swap(a,b); // Use swap with T replaced by float
String s,t; ... swap(s,t); // Use swap with T replaced by String
```

Bạn có thể đã nhận thấy rằng từ khóa C++ gắn kết với biến kiểu là lớp trong C++, một số kiểu là lớp và một số như *int*, *float*, không phải là lớp. Như mô tả dưới đây, từ khóa lớp bỏ qua, vì *template* có thể được sử dụng với kiểu không phải lớp như *int* hoặc *float*.

Templates C++ được khởi tạo khi link chương trình. Cụ thể hơn, giả sử *template* hàm *swap* được lưu ở một file và được dịch, và một chương trình gọi *swap* được lưu trong file khác và được dịch riêng rẽ. Các tệp được gọi là đối tượng tái định vị được tạo ra khi chương trình gọi được dịch sẽ bao gồm cả thông tin chỉ ra rằng code được dịch gọi hàm swap của một kiểu nào đó. Chương trình linker được thiết kế để kết nối hai phần chương trình bằng việc gắn kết lời gọi hàm *swap* trong chương trình gọi đến định nghĩa của swap trong phần dịch riêng. Nó thực hiện như vậy bằng việc khởi tạo code được dịch cho swap ở dạng mà tạo ra code phù hợp với lời gọi đến swap. Nếu chương trình gọi swap với một số kiểu khác nhau, thì một số bản sao khởi tạo khác nhau sẽ được tạo ra. Mỗi bản sao khác nhau cần cho mỗi kiểu của lời gọi vì *swap* khai báo biến cục bộ *tmp* kiểu T. Không gian cho *tmp* cần được cấp trên bản ghi kích hoạt của *swap*. Do đó code dịch đối với swap cần được chỉnh sửa cho phù hợp với kích thước của biến kiểu T. Chẳng hạn, nếu T là cấu trúc hoặc đối tượng, thì kích thước là tương đối lớn. Mặt khác, nếu *t* là *int*, thì kích thước

sẽ nhỏ. Trong mọi trường hợp, code dịch cần biết kích thước của dữ liệu để việc tham chiếu đến bản ghi kích hoạt được thực hiện đúng.

Các thao tác trên tham số kiểu

Ví dụ swap là đơn giản hơn đa số các hàm theo nhiều khía cạnh. Quan trọng nhất là than của swap không đòi hỏi thao tác nào trên tham số kiểu T, ngoại trừ khai báo biến và phép gán. Ví dụ thể hiện rõ hơn của *template* hàm là hàm sort tổng quát sau:

```
template <typename T>
void sort( int count, T * A[count] ) {
    for (int i=0; i<count-1; i++)

        for (int j=i+1; j<count-1; j++)
            if (A[j] < A[i]) swap(A[i],A[j]);
}
```

Nếu A là mảng kiểu T, thì sort(n, A) sẽ hoạt động nếu phép toán < được định nghĩa trên kiểu T. Tính chất này của sort không được khai báo ở đâu trong mã C++. Tuy nhiên, khi *template* hàm này được khởi tạo trong thời gian link, kiểu thực tế của T cần phải có phép toán < được định nghĩa hoặc lỗi thời gian link sẽ được thông báo và không có mã đối tượng thực thi nào được tạo ra.

Quá trình link đối với C++ là tương đối phức tạp. Chúng ta không nghiên cứu chi tiết về việc này. Tuy nhiên, cần lưu ý rằng nếu < là phép toán overloaded, thì phiên bản đúng của < cần được xác định khi code đã dịch cho sort được link với chương trình gọi.

Như nguyên lý lập trình tổng quan, sẽ thuận tiện hơn là các lỗi chương trình được báo cáo trong thời gian dịch hơn là trong thời gian link. Một lý do là các modules chương trình riêng biệt được dịch độc lập, nhưng link cùng nhau chỉ khi toàn bộ hệ thống được thiết lập. Do đó, dịch là quá trình “cục bộ” mà cần được tiến hành bởi người thiết kế hoặc người lập trình một thành phần riêng rẽ. Ngược lại, các lỗi thời gian link thể hiện tính chất hệ thống tổng thể mà có thể chưa được biết trước khi toàn bộ hệ thống được thiết lập. Vì lý do này, các lỗi thời gian link của C++ gắn kết với các thao tác trên *templates* cần được xem xét và là nguồn gốc của thất bại.

Báo cáo lỗi cho *templates* C++ có thể đạt kết quả tốt hơn, nếu cú pháp *template* bao gồm cả mô tả các phép toán cần thiết trên tham số kiểu. Tuy nhiên, điều này tương đối phức tạp trong C++, vì tính ghi đè và các tính chất khác của ngôn ngữ này. Ngược lại, ML có tính ghi đè đơn giản hơn và chứa nhiều thông tin hơn trong cấu trúc tham số, như được mô tả trong mục sau.

So sánh với đa hình ML

Sau đây là code ML cho hàm đa hình sort. Thuật toán sử dụng trong đoạn code này là insertion sort, mà sử dụng hàm insert phụ trợ xuất hiện từ đầu:

```

fun insert(less, x, nil) = [x]
  | insert(less, x, y::ys) = if x < y then x::y::ys
    else y::insert(less, x, ys)
fun sort(less, nil) = nil
  | sort(less, x::xs) = insert(less, x, sort(less, xs))

```

Trong hàm sort đa hình, thao tác less-than được truyền như đối số của hàm sort. Không khởi tạo nào là cần thiết, vì mọi danh sách biểu diễn theo cùng một cách.

Biểu diễn dữ liệu thống nhất có ưu và nhược điểm của nó. Vì không cần lặp lại code cho các kiểu đối số khác nhau, biểu diễn dữ liệu đồng nhất kéo theo kích cỡ code nhỏ hơn và tránh phức tạp gắn kết với kiểu link C++. Mặt khác, code kết quả kém hiệu quả hơn, vì biểu diễn dữ liệu đồng nhất thường bao hàm việc sử dụng con trỏ thay vì lưu trữ cấu trúc dữ liệu trực tiếp.

5.4.2 Functor ML chuẩn

Trong hệ thống module ML, các cấu trúc có thể được tham số hóa theo nhiều cách rất linh hoạt. Chúng ta xem phần này của hệ thống kiểu ML như ví dụ tốt về các cách mà các modules cần được tham số hóa một cách có ích trong các ngôn ngữ lập trình. Tham số hóa Module khác với đa hình ML và về bản chất độc lập với nó. Cụ thể hơn, thiết kế module ML có thể thích nghi với các ngôn ngữ mà không có đa hình hoặc suy luận kiểu.

Trong ML chuẩn, cấu trúc được tham số hóa được gọi là functor. Các functor được viết trong dạng giống như hàm số với danh sách tham số chứa tên và các đặc trưng (signatures) của cấu trúc. Sau đây là ví dụ mà sử dụng signature SIG được định nghĩa trong mục trước.

```

functor F ( S : SIG ) : SIG =
  struct
    type t = S.t * S.t
    val x:t= (S.x,S.x)
  end

```

Đoạn code trên khai báo functor F mà nhận cấu trúc sánh với signature SIG như đối số và tạo ra cấu trúc sánh với cùng một cấu trúc. Khi áp dụng cho cấu trúc phù hợp S, functor F tạo ra như kết quả của nó một cấu trúc, mà thành phần kiểu t của nó được gắn với tích S.t với chính nó và thành phần giá trị x của nó, là cặp các thành phần được tính toán cho giá trị S.x.

Khi biến cấu trúc tự do trong signature được sử dụng, các dạng phụ thuộc vào kết quả functor trên đối số functor có thể được thể hiện rõ ràng. Chẳng hạn, khai báo sau nêu rõ kiểu của y trong signature kết quả của G trong thuật ngữ thành phần kiểu t của đối số S:

```

functor G ( S : SIG ) : sig val y : S.t * S.t end =
  struct
    val y = (S.x,S.x)
  end

```

Phần còn lại của mục này gồm các ví dụ chỉ ra rằng tham số hóa module ML có thể được sử dụng để giải quyết một số vấn đề lập trình. Ví dụ 5.10 thảo luận về một dạng cấu trúc container và ví dụ 5.11 hoàn thiện ví dụ geometry trong 5.9.

Ví dụ 5.10 Module Hàng đợi Ưu tiên được tham số hóa

Cấu trúc container như tập hợp, hàng đợi, từ điển,... được sử dụng trong nhiều chương trình. Ví dụ này chỉ ra cách định nghĩa cấu trúc container là hàng đợi, khi một hoặc nhiều hơn các thao tác container đòi hỏi một thao tác trên kiểu các đối tượng được lưu trong container. Vì hàng đợi ưu tiên yêu cầu đảm bảo thứ tự trên các phần tử lưu trữ trong hàng đợi, định nghĩa của chúng ta về hàng đợi được tham số hóa bởi một cấu trúc gồm kiểu và quan hệ thứ tự.

Chúng ta bắt đầu định nghĩa signature của kiểu với quan hệ thứ tự:

```

signature Ordered =
  sig
    type t;
    val lesseq : t * t → bool;
  end;

```

Bất cứ cấu trúc nào mà sánh với *signature Ordered* cần phải có kiểu *t* và quan hệ hai ngôi *lesseq*. Chúng ta giả thiết trong định nghĩa hàng đợi ưu tiên rằng quan hệ này là thứ tự tổng thể trên kiểu *t*. Trong định nghĩa functor này, chúng ta khai báo các thao tác hàng đợi ưu tiên cài đặt kiểu và tập các thao tác trên hàng đợi ưu tiên. Vì các thao tác insert hoặc deletemax cần kiểm tra một phần tử có nhỏ hơn các phần tử khác không, tối thiểu một trong các thao tác đó sẽ sử dụng thao tác *elem.lesseq* được cung cấp bởi cấu trúc tham số *elem*:

```

functor PQ(elem:Ordered) =
  struct
    type pq = ... elem.t ... ; (* representation uses element type *)
    val empty : pq = ... ;
    fun insert: elem.t * pq → pq = ... ; (* operations may use elem.lesseq *)
    fun deletemax: pq → elem.t * pq = ... ;
  end;

```

Ví dụ 5.11 Hoàn thiện ví dụ Geometry

Ví dụ này chỉ ra rằng làm thế nào các cấu trúc Point, Circle và Rect trong Ví dụ 5.9 có thể kết hợp để tạo nên cấu trúc Hình học với thao tác trên cả ba kiểu đối tượng hình học. Để chỉ ra làm thế nào các thao tác khác có thể bổ sung khi các module được kết hợp, signature Geom chứa tất cả các kiểu và thao tác hình học, thêm vào đó hàm bounding box là bbox với mục đích ánh xạ đường tròn vào hình chữ nhật nhỏ nhất chứa nó.

```
signature Geom =
sig
  include Circle
  (* add signature of Rect, except Point since Point is already defined in Circle*)
  type rect
  val mk_rect : point * point → rect
  val lleft : rect → point
  val uright : rect → point
  val move_r : rect * real → rect
  (* include function computing bounding box of circle *)
  val bbox : circle → rect
end;
```

Cấu trúc hình học được xây dựng với functor mà nhận các cấu trúc Circle và Rect như các đối số. Signatures Circle và Rect cả hai đặt tên cho một kiểu là Point. Hai kiểu point này cần phải được cài đặt theo cùng một cách. Nếu không, các thao tác mà bao gồm cả circles và rectangles sẽ không làm việc đúng. Ràng buộc cả hai cấu trúc (Circle và Rect) cần phải được chia sẻ cấu trúc con chung (point) được khẳng định một cách tường minh trong ràng buộc chung trong danh sách tham số của functor này.

```

functor geom(
  structure c:Circle
  structure r:Rect
  sharing type r.point = c.point (*constraint requires same type in both
structures *)
) : Geom =
  struct
    type point = c.point
    val mk_point = c.mk_point
    val x_coord = c.x_coord
    val y_coord = c.y_coord
    val move_p = c.move_p
    type circle = c.circle
    val mk_circle = c.mk_circle
    val center = c.center
    val radius = c.radius
    val move_c = c.move_c
    type rect = r.rect
    val mk_rect = r.mk_rect
    val lleft = r.lleft
    val uright = r.uright
    val move_r = r.move_r
    (* Bounding box of circle *)
    fun bbox(c) =
      let val x = x_coord(center(c))
          and y = y_coord(center(c))
          and r = radius(c)
      in
        mk_rect(mk_point(x-r,y-r),mk_point(x+r,y+r))
      end
    end
  end;

```

5.4.3 Thư viện Template chuẩn C++ (C++ Standard Template Library)

STL C++ là thư viện chương trình lớn được phát triển bởi Alex Stepanov và các cộng sự và được chấp nhận như một phần của chuẩn C++. STL cung cấp một họ các lớp containers, như lists, vectors, sets và maps, và các thuật toán cũng như các thành phần khác liên quan. Vì STL là ví dụ tốt của một thư viện thực tế, lớn với các cấu trúc tổng quát, chúng ta có thể tóm tắt một vài khái niệm chính. Mục đích của phần này không phải dạy đủ dùng để có thể sử dụng STL, mà là giải thích các khái niệm để bạn có thể đánh giá một số ý tưởng thiết kế và sức mạnh của hệ thống.

Một đặc trưng ấn tượng của STL là tính hiệu quả thời gian chạy của mã được tạo ra. STL không chỉ làm cho dễ dàng viết một chương trình bằng việc cung cấp các cấu trúc hữu dụng, mà STL còn làm cho có thể viết code dựa trên thư viện mà chạy nhanh hoặc nhanh hơn so với code mà bạn có thể viết, nếu bạn bỏ ra nhiều công sức hơn và không sử dụng

STL. Một lý do để STL hiệu quả là templates C++ được triển khai trong thời gian dịch/link, với các code riêng biệt được sinh (và tối ưu) cho mỗi khởi tạo. Lý do khác là việc sử dụng tải chồng, mà được phân giải trong thời gian dịch, cũng cho phép compiler tối ưu code mà được chọn chính xác cho kiểu dữ liệu được sử dụng trong chương trình. Mặt khác, vì việc lập code là kết quả của việc triển khai template, mã dịch của chương trình sử dụng STL có thể rất lớn.

Trong mục này, thuật ngữ object được sử dụng muốn nói về giá trị lưu trong bộ nhớ. Điều đó phù hợp với việc sử dụng thuật ngữ trong tài liệu C, C++ và STL. STL dựa trên templates C++, các thuật toán tải chồng và kiểu mà tìm ra các tham số kiểu cho templates. STL không sử dụng hàm ảo và theo người phát triển nó, không là ví dụ về lập trình hướng đối tượng.

Có sáu loại thực thể trong STL:

- Containers, mỗi cái là một họ các đối tượng kiểu.
- Iterators, mà cho truy cập đến các đối tượng trong container. Về trực quan, iterator là khái quát của con trỏ hoặc cho địa chỉ đến một vị trí nào đó trong container.
- Thuật toán.
- Adapters, mà cho chuyển đổi giữa một dạng của thực thể và dạng khác. Ví dụ là iterator ngược, mà duyệt ngược hướng của một iterator.
- Đối tượng hàm (Function objects) mà là dạng closure (mã hàm và môi trường gắn kết). Chúng được sử dụng nhiều dưới dạng cho phép các đối số hàm cho template ở dạng in-line. Cụ thể hơn, đối tượng hàm được truyền như hai đối số riêng biệt, đối số template mang code và đối số run-time mang trạng thái của hàm. Hệ thống kiểu được sử dụng để tin tưởng rằng code và trạng thái phù hợp với nhau.
- Allocators, mà đóng gói vùng bộ nhớ. Các allocators khác nhau có thể cung cấp bộ nhớ có khả năng dọn rác, bộ nhớ tham chiếu đếm, bộ nhớ bền,...

Ví dụ sau sẽ cho bạn cảm giác STL làm việc như thế nào:

Ví dụ 5.12. Danh sách STL

Danh sách là một kiểu container đơn giản được định nghĩa trong STL. Đây là code mẫu khai báo một danh sách xâu được gọi là wordlist:

```
#include <string>
#include <list>
int main (void) {
    list<string> wordlist;
}
```

Đoạn code `list<string>` khởi tạo template class `list<string>` và sau đó tạo đối tượng của kiểu này. STL cho bạn các thao tác đặt các phần tử vào trước hoặc sau danh sách. Đây là ví dụ bổ sung từ vào word list của chúng ta:

```
wordlist.push_back("peppercorn");
wordlist.push_back("funnybone");
wordlist.push_front("nosegay");
wordlist.push_front("supercilious");
```

Chúng ta có danh sách list với bốn xâu. Như các tên đã chỉ rõ, hàm thành viên danh sách *push_back()* đặt đối tượng vào cuối danh sách và hàm *push_front()* đặt phần tử vào đầu danh sách. Chúng ta cần phải in các từ trong danh sách bằng việc sử dụng bộ duyệt iterator. Đây là khai báo iterator cho danh sách xâu:

```
list<string>::iterator wordlistIterator;
```

Như đã nêu ở phần đầu, iterator STL giống như con trỏ - iterator trỏ tới phần tử trong container, tham chiếu đến iterator khác null sẽ tạo ra một đối tượng trong container. Iterator STL được khai báo có kiểu chuyên biệt; Điều này quan trọng vì việc di chuyển con trỏ duyệt qua container làm việc khác nhau đối với các kiểu container khác nhau. Đây là ví dụ mẫu in các từ trong wordlist:

```
for (wordlistIterator=wordlist.begin();
     wordlistIterator!=wordlist.end();
     ++wordlistIterator) {
    // dereference the iterator to get the list element
    cout << *wordlistIterator << endl;
}
```

Chúng ta xác định biên của this cho vòng lặp bằng việc gọi *wordlist.begin()*, mà trả về iterator trỏ đến đầu của container, và *wordlist.end()*, mà trả về iterator trỏ đến vị trí sau cuối của container. Biến vòng lặp được tăng bởi phép toán ++, mà được định nghĩa trên mỗi iterator và dịch chuyển iterator từ một phần tử của danh sách đến phần tử tiếp theo. Trong thân của vòng lặp, chúng ta tham chiếu iterator để nhận được phần tử của danh sách ở vị trí hiện tại.

Như ví dụ 5.12 chỉ ra, iterator là cần thiết trong lập trình với containers STL. Hầu hết các thuật toán STL sử dụng iterator của một kiểu nào đó, và có một số kiểu iterator khác nhau. Dạng đơn giản nhất, còn được gọi là iterator hiển nhiên, có thể được dùng tham chiếu đến một đối tượng nào đó, nhưng các thao tác như tăng hoặc so sánh không đảm bảo là được hỗ trợ, iterator đầu vào có thể được dùng để tham chiếu đến một đối tượng nào đó và có thể tăng để nhận được iterator tiếp theo trong container. Iterator đầu ra là dạng hạn chế đúng của iterator mà có thể được sử dụng để sắp xếp (nhưng không nhất thiết tăng dần) một dãy các giá trị. Một số dạng khác của iterator là forward iterator, bidirectional iterator và random-access iterator. Các thuật toán STL chuyên biệt có thể đòi hỏi các kiểu iterator chuyên biệt, phụ thuộc vào việc thuật toán đọc từ container ra hoặc lưu giá trị vào container như thế nào và việc truy cập là tuần tự (giống như đọc hoặc viết lên băng) hay truy cập ngẫu nhiên.

Khái niệm cơ bản khác trong STL là phạm vi (range). Phạm vi là một phần của container được định nghĩa bởi hai iterator. Các phần tử của phạm vi là mọi đối tượng có thể đạt

được bằng cách tăng iterator thứ nhất cho đến khi đạt được giá trị của iterator thứ hai. Nếu bạn nghĩ mảng như một ví dụ của container, thì phạm vi là cặp số nguyên, mà chỉ số thứ nhất nhỏ hơn chỉ số thứ hai và cả hai nhỏ hơn độ dài của mảng, nó biểu diễn vị trí của mảng nằm giữa chỉ số thứ nhất và chỉ số thứ hai.

Ví dụ sau bàn về hàm trộn merge của STL, mà sử dụng các phạm vi ranges như đầu vào và đầu ra. Nói chung, thao tác trộn kết hợp hai cấu trúc tuần tự đã được sắp xếp và tạo ra cái thứ ba. Chẳng hạn, trong merge sort pha trộn kết hợp hai danh sách đã sắp xếp thành một danh sách sắp xếp dài hơn mà chứa mọi phần tử của hai danh sách đầu vào.

Ví dụ 5.13. Hàm Merge tổng quát

Hàm merge tổng quát STL nhận hai phạm vi như đầu vào và trả về một phạm vi khác. Về khái niệm, kiểu của merge là

`merge : range(s) x range(t) x comparison(u) → range(u)`

trong đó *range(s)* có nghĩa là phạm vi của container mà chứa các đối tượng kiểu *s*. Vì C++ có kiểu con, các kiểu *s*, *t* và *u* không cần giống nhau. Tuy nhiên, kiểu *s* và *t* cần là kiểu con của *u*, có nghĩa là mỗi đối tượng của kiểu *s* hoặc *t* cần phải được xem như được chuyển sang đối tượng kiểu *u*. Ký hiệu không hình thức *comparison(u)* có nghĩa là hàm merge đòi hỏi phép so sánh *comparison* (nhỏ hơn hoặc bằng) trên kiểu *u*, sao cho merge có thể kết hợp hai cấu trúc theo thứ tự.

Trên thực tế, mỗi phạm vi đầu vào trong STL được truyền như hai iterator và mỗi phạm vi đầu ra được truyền như một iterator. Lý do chúng ta cần hai iterator cho mỗi phạm vi đầu vào là vì chúng ta cần phải biết có bao nhiêu phần tử trong phạm vi đó. Vì đầu ra của merge được xác định bởi đầu vào, nên phạm vi đầu ra được biểu diễn bởi iterator đầu ra duy nhất, nó đánh dấu vị trí bắt đầu viết container trộn. Do đó merge có các tham số sau:

`merge(InputIterator1 first1, InputIterator1 last1,
InputIterator2 first2, InputIterator1 last2,
OutputIterator result, Compare comp)`

Nói bằng lời, hai đối số đầu cung cấp phạm vi đầu vào thứ nhất, hai đối số sau cho ta phạm vi đầu vào thứ hai, đối số thứ năm cho phạm vi đầu ra và đối số thứ sáu cung cấp hàm so sánh *comparison*.

Merge là hàm đa hình, trộn hai phạm vi mà chứa các kiểu bất kỳ của các đối tượng, sao cho các kiểu đó tương thích với nhau. Tính đa hình của merge được cài đặt bởi template với 4 tham số. Template yêu cầu hai kiểu iterator, một cho mỗi một trong hai cấu trúc tuần tự và sẽ được trộn, cùng với kiểu của cấu trúc đầu ra và kiểu so sánh. Nó cho ta tiêu đề đầy đủ của merge như sau:

```
template<typename InputIterator1,
        typename InputIterator2,
        typename OutputIterator,
        typename Compare> inline
OutputIterator merge( InputIterator1 first1,
                     InputIterator1 last1,
                     InputIterator2 first2,
                     InputIterator2 last2,
                     OutputIterator result,
                     Compare comp)
```

Có thể gọi merge mà không cung cấp đối tượng hàm so sánh. Trong trường hợp này, phép toán < sẽ được sử dụng mặc định. Sau đây là chương trình ngắn mà sử dụng merge, lấy từ *Standard Template Library Programmer's Guide*:

```
int main()
{
    int A1[] = { 1, 3, 5, 7 };
    int A2[] = { 2, 4, 6, 8 };
    const int N1 = sizeof(A1) / sizeof(int);
    const int N2 = sizeof(A2) / sizeof(int);

    merge(A1, A1 + N1, A2, A2 + N2,
          ostream_iterator<int>(cout, ""));
    // The output is "1 2 3 4 5 6 7 8"
}
```

Như bạn thấy từ đoạn code ví dụ này, mặc dù các khái niệm và thiết kế của STL là tinh tế và phức tạp, việc sử dụng STL cần phải rất dễ và trực tiếp khi hiểu rõ các khái niệm.

5.5 Tóm tắt chương

Trong chương này, chúng ta đã tìm hiểu một số cách chương trình có thể chia thành các phần có ý nghĩa và cách mà các ngôn ngữ lập trình hỗ trợ các cách chia này. Chủ đề chính là lập trình có cấu trúc, hỗ trợ cho trừu tượng và modules. Hai ví dụ được sử dụng để nghiên cứu hệ thống modules và lập trình tổng quan: hệ thống kiểu ML chuẩn và Thư viện template chuẩn STL C++.

Lập trình cấu trúc

Về mặt lịch sử, mối quan tâm đến tổ chức chương trình phát triển cùng với phong trào lập trình cấu trúc.

Bài tập

BÀI 6: CÁC KHÁI NIỆM TRONG NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Trên 40 năm, lập trình hướng đối tượng đã trở thành xu thế thiết kế và cài đặt phần mềm hứa hẹn. Các vấn đề trình bày trong chương này là thiết kế hướng đối tượng và bốn khái niệm cơ bản trong ngôn ngữ lập trình hướng đối tượng và cách mà các khái niệm ngôn ngữ này hỗ trợ thiết kế và cài đặt hướng đối tượng.

Đối tượng bao gồm tập các thao tác trên dữ liệu ẩn giấu nào đó. Một đặc trưng quan trọng của các đối tượng là chúng cung cấp một cách thống nhất đóng gói hầu hết mọi kết hợp giữa dữ liệu và chức năng. Đối tượng có thể nhỏ như một số nguyên hoặc có thể lớn như hệ thống file hoặc cơ sở dữ liệu. Không phụ thuộc vào kích thước, mọi tương tác với đối tượng xảy ra qua môi trường các thao tác đơn giản mà được gọi là thông điệp hay lời gọi hàm thành viên.

Trong cuốn sách này, chúng ta quan tâm đến việc phân biệt có ý nghĩa giữa các tính chất ngôn ngữ khác nhau và hiểu được cách các tính chất này hỗ trợ các kiểu lập trình khác nhau. Do đó, thuật ngữ ngôn ngữ lập trình hướng đối tượng được sử dụng để nói về các ngôn ngữ lập trình mà có các đối tượng và bốn tính chất được nhấn mạnh trong chương này: tìm kiếm động, trừu tượng, kiểu con và kế thừa.

6.1 Thiết kế hướng đối tượng

Thiết kế hướng đối tượng bao gồm xác định các khái niệm quan trọng và sử dụng các đối tượng để cấu trúc cách mà các khái niệm đó được thể hiện trong hệ thống phần mềm. Các bước sau trong thiết kế hướng đối tượng được đề xuất bởi Grady Booch (Object-Oriented Design with Applications, Benjamin/Cummings, 1991) :

- Xác định các đối tượng tại mức trừu tượng đã cho
- Xác định ngữ nghĩa (hành vi mong muốn) của các đối tượng
- Xác định các quan hệ giữa các đối tượng
- Cài đặt các đối tượng

Thiết kế hướng đối tượng là quá trình lập dựa trên việc gắn kết các đối tượng với các thành phần trong hệ thống. Quá trình này lập vì thông thường chúng ta cài đặt đối tượng bằng việc sử dụng một số đối tượng con, cũng giống như chúng ta cài đặt một thủ tục bằng việc gọi một số các thủ tục nhỏ hơn. Do đó, sau khi một số đối tượng quan trọng trong hệ thống được xác định và cài đặt ở một mức trừu tượng, bước lập tiếp theo sẽ xác định tiếp các đối tượng và cài đặt chúng. Quan hệ giữa các đối tượng ở đây có thể là quan hệ giữa các giao diện của chúng hoặc quan hệ giữa các cài đặt của chúng. Ngôn ngữ lập trình hướng đối tượng hiện đại cung cấp cơ chế để sử dụng quan hệ giữa các giao diện và quan hệ giữa các cài đặt trong quá trình thiết kế và cài đặt đó.

Các cấu trúc dữ liệu được sử dụng trong các ví dụ trước đây của lập trình top-down là rất đơn giản và tồn tại bất biến trong quá trình làm mịn của chương trình. Khi việc làm mịn bao gồm thay thế một thủ tục bằng một thủ tục chi tiết hơn, thì các ngôn ngữ lập trình cấu trúc như Algol, Pascal và C là đáp ứng được. Tuy nhiên, đối với các bài toán

phức tạp hơn, thì cả thủ tục và cả cấu trúc dữ liệu của chương trình đều cần được làm mịn cùng nhau. Vì các đối tượng là sự kết hợp của hàm và dữ liệu, các ngôn ngữ lập trình hướng đối tượng hỗ trợ làm mịn kết hợp của hàm và dữ liệu hiệu quả hơn so với các ngôn ngữ hướng thủ tục.

6.2 Bốn khái niệm cơ bản trong ngôn ngữ hướng đối tượng

Mọi ngôn ngữ lập trình hướng đối tượng đều có dạng nào đó của đối tượng. Như đã nói trong mục trước, đối tượng bao gồm các hàm và dữ liệu, chỉ được truy cập qua giao diện chuyên biệt. Trong các ngôn ngữ lập trình hướng đối tượng phổ cập, bao gồm Smalltalk, Modula-3, C++, và Java, việc cài đặt đối tượng được xác định bởi lớp class của nó. Trong các ngôn ngữ này, chúng ta tạo các đối tượng bằng việc tạo *instance* các lớp của chúng.

Các phần hàm của đối tượng được gọi là các phương thức *methods* hoặc các hàm thành viên *member functions*; và các phần dữ liệu của đối tượng được gọi là các biến thành viên *instance variables*, các trường *fields* hoặc các thành viên dữ liệu *data members*.

Ngôn ngữ lập trình với các đối tượng và các lớp thông thường cung cấp tìm kiếm động, trừu tượng, kiểu con và kế thừa. Đây là bốn khái niệm quan trọng của ngôn ngữ đối với lập trình hướng đối tượng. Chúng được tóm tắt như sau :

- Tìm kiếm động (Dynamic lookup) có nghĩa là khi thông điệp được gửi tới đối tượng, mã hàm (hoặc phương thức) sẽ thực thi được xác định theo cách mà đối tượng được cài đặt, chứ không phải tính chất tĩnh nào đó của con trỏ hoặc biến được sử dụng để định danh đối tượng. Nói cách khác, đối tượng chọn cách phản hồi đến thông điệp và các đối tượng khác nhau có thể phản hồi khác nhau đến cùng một thông điệp.
- Trừu tượng (Abstraction) có nghĩa là chi tiết cài đặt được ẩn giấu bên trong đơn vị chương trình với giao diện riêng biệt. Đối với các đối tượng, giao diện thông thường bao gồm tập *public functions* (hoặc *public methods*) mà thao tác dữ liệu ẩn giấu đó.
- Kiểu con (Subtyping) có nghĩa là nếu một đối tượng a nào đó có mọi chức năng của đối tượng khác b, thì ta có thể sử dụng a trong mọi tình huống chờ đợi b.
- Kế thừa (Inheritance) là khả năng tái sử dụng định nghĩa một kiểu đối tượng để định nghĩa một kiểu khác của đối tượng.

Các thuật ngữ này được định nghĩa và các tính chất này được trình bày chi tiết hơn trong các mục sau :

Sau đây là một số dạng của các ngôn ngữ lập trình hướng đối tượng mà không được trình bày trực tiếp trong tài liệu này. Một dạng là ngôn ngữ dựa trên việc ủy quyền. Hai ngôn ngữ dựa trên ủy quyền là Dylan, ban đầu được thiết kế cho các trợ lý của chương trình Apple Newton, và Self ngôn ngữ đa dụng phát triển từ việc nghiên cứu cài đặt của các ngôn ngữ hướng đối tượng. Trong các ngôn ngữ dựa trên ủy quyền, các đối tượng được định nghĩa trực tiếp từ các đối tượng khác khi các phương thức mới được bổ sung bằng phương tiện *method addition* và các phương thức cũ được thay thế bởi phương tiện *method override*. Mặc dù các ngôn ngữ dựa ủy quyền không có class, chúng vẫn có bốn tính chất đặc trưng về bản chất đòi hỏi đối với các ngữ hướng đối tượng.

6.2.1 Tìm kiếm động

Trong bất cứ ngôn ngữ hướng đối tượng nào, có một số cách triệu gọi các thao tác gắn kết với một đối tượng. Cú pháp tổng quát để triệu gọi thao tác trên đối tượng, có thể với các đối số bổ sung là

`object -> operation (arguments)`

Trong Smalltalk, nó được gọi là ‘gửi thông điệp đến đối tượng’, trong khi đó trong C++ nó được gọi là ‘gọi hàm thành viên của đối tượng’. Để tránh việc chuyển tới, chuyển lui giữa các lựa chọn khác nhau của thuật ngữ, chúng ta sẽ sử dụng thuật ngữ của Smalltalk trong phần còn lại của mục này. Trong thuật ngữ Smalltalk, thông điệp bao gồm tên thao tác và tập các đối số bổ sung. Khi thông điệp được gửi tới đối tượng, đối tượng đó phản hồi tới thông điệp bằng cách thực thi hàm còn được gọi là phương thức.

Dynamic lookup có nghĩa là phương thức được lựa chọn động trong thời gian thực tùy thuộc vào cài đặt của đối tượng mà nhận được thông điệp. Tính chất quan trọng của tìm kiếm động là các đối tượng khác nhau có thể cài đặt cùng một thao tác theo các cách khác nhau. Chẳng hạn, lệnh

`x -> add(y)`

gửi thông điệp `add(y)` đến đối tượng `x`. Nếu đối tượng `x` là số nguyên, thì phương thức (code cài đặt thao tác này) có thể cộng số nguyên `y` vào `x`. Nếu `x` là tập hợp, thì phương thức có thể chèn `y` vào `x`. Các thao tác này có các tác động khác nhau và được cài đặt khác nhau. Tuy nhiên, dòng mã lệnh `x -> add(y)` trong vòng lặp có thể tạo ra việc cộng số nguyên lần đầu thực hiện và chèn tập hợp lần hai thực hiện, nếu giá trị của biến `x` thay đổi từ số nguyên thành tập hợp giữa một lần đi qua vòng lặp.

Tìm kiếm động đôi khi bị nhầm lẫn với tải động (overloading), mà là cơ chế dựa trên các kiểu và phép toán tĩnh. Tuy nhiên, hai khái niệm đó hoàn toàn khác nhau, như chúng ta sẽ thấy.

Tìm kiếm động là tính chất ngôn ngữ rất có ích và một phần quan trọng của lập trình hướng đối tượng. Xét ví dụ sau, một chương trình đồ họa đơn giản, mà thao tác các hình vẽ chứa các hình như hình chữ nhật, đường tròn và tam giác. Mỗi đối tượng hình chữ nhật có thể chứa phương thức vẽ draw với mã để vẽ hình chữ nhật, mỗi đối tượng đường tròn có thể chứa phương thức vẽ draw với mã để vẽ đường tròn, ... Khi chương trình muốn hiện một hình vẽ cho trước, việc gửi thông điệp draw đến mỗi hình trên hình vẽ có thể làm việc đó. Phần của chương trình mà gửi thông điệp draw không biết trước được dạng hình nào sẽ nhận được thông điệp này. Thay vào đó, mỗi hình nhận được thông điệp draw sẽ biết cách vẽ hình đó. Điều đó hoàn toàn có nghĩa vì người cài đặt hình cụ thể biết rõ nhất cách vẽ dạng này của hình.

Chúng ta cần phải hiểu một số khía cạnh của tìm kiếm động và phạm vi bằng việc sử dụng so sánh ngắn gọn với các kiểu dữ liệu trừu tượng. Sử dụng cơ chế kiểu dữ liệu trừu tượng, ta có thể định nghĩa ma trận như sau :

```
abstype matrix = ...
with
  create(...) = ...
  update(m, i, j, x) = ... set m(i, j) = x ...
  add(m1, m2) = ...
  ...
end;
```

Đặc trưng của cài đặt ma trận này là hàm add nhận hai ma trận như các đối số, với lời gọi có dạng

```
add(x, y)
```

Khai báo kiểu ma trận và các thao tác gắn kết có phạm vi riêng. Trong phạm vi này, add tham chiếu cụ thể đến hàm khai báo cho ma trận. Do đó, trong biểu thức add(x,y), cả hai biến x và y cần phải là ma trận. Nếu add được định nghĩa cho số phức ở phạm vi bên ngoài, thì hoặc khai báo trong giấu khai báo ngoài hoặc ngôn ngữ cần phải cung cấp cơ chế tải chồng tĩnh nào đó.

Với các đối tượng trong ngôn ngữ dựa vào lớp, chúng ta có thể khai báo ma trận như sau

```
class matrix
  ... (representation)
  update(i, j, x) = ... set (i, j) of *this* matrix ...
  add(m) = ... add m to *this* matrix ...
end
```

Phương thức *add* của ma trận yêu cầu có một ma trận như đối số. Phương thức này có thể được triệu gọi bằng biểu thức như sau

$x \rightarrow \text{add}(y)$

Trong biểu thức trên, thao tác *add* dường như chỉ có một đối số là ma trận mà được cộng vào ma trận *x* mà nhận được thông điệp *add(y)*.

Có một số cách mà tìm kiếm động có thể được cài đặt. Trong một cài đặt, mỗi đối tượng chứa con trỏ đến bảng tìm kiếm phương thức mà gắn kết thân phương thức với mỗi thông điệp được định nghĩa cho đối tượng đó. Khi thông điệp được gửi đến đối tượng trong thời gian chạy, phương thức tương ứng sẽ được chọn về từ bảng phương thức của đối tượng. Vì các đối tượng khác nhau có thể có các bảng tìm kiếm khác nhau, việc gửi cùng một thông điệp đến các đối tượng khác nhau có thể có kết quả là việc thực thi các đoạn code khác nhau.

Cũng có thể nghĩ rằng tìm kiếm động như một dạng thời gian chạy của tải chồng. Cụ thể hơn, chúng ta có thể nghĩ tên của mỗi phương thức như tên của hàm tải chồng. Khi thông điệp *m* được gửi đến đối tượng có tên là biến *x*, thì *x* sẽ được xử lý như đối số đầu tiên của hàm được tải chồng có tên là *m*. Tuy nhiên, không giống như tải chồng thông thường, code sẽ thực thi cần được chọn phù hợp với giá trị thời gian chạy của *x*. Ngược lại, tải chồng truyền thống sử dụng kiểu tĩnh của biến *x* để quyết định code nào được sử dụng.

Tìm kiếm động là một phần quan trọng của Smalltalk, C++ và Java. Trong Smalltalk và Java, phương thức lookup được thực hiện động theo mặc định. Trong C++, chỉ có các hàm thành viên ảo là được chọn động.

Có một họ ngôn ngữ lập trình hướng đối tượng mà dựa trên việc tải chồng thời gian chạy của tìm kiếm động. Thiết kế nổi bật nhất của dạng này là hệ thống đối tượng Lisp phổ biến (CLOS – common Lisp object system). Trong CLOS, biểu thức tương ứng với

$x \rightarrow f(y,z)$

được xử lý như lời gọi *f(x,y,z)* đến hàm được tải chồng với ba đối số. Mặc dù tìm kiếm động thông thường có thể chọn thân hàm *f* dựa trên cài đặt của mình *x*, tìm kiếm phương thức CLOS sử dụng cả ba đối số. Tính chất này đôi khi gọi là gửi lặp để phân biệt nó với các ngôn ngữ gửi đơn truyền thống hơn, ở đó chỉ có một trong các đối số của hàm (đối tượng nhận được thông điệp) xác định thân hàm mà được gọi trong thời gian chạy.

Gửi lặp (multiple dispatch) là có ích trong cài đặt các thao tác như bằng nhau, ở đó phép so sánh tương ứng được sử dụng phụ thuộc vào kiểu động của cả đối tượng nhận và đối tượng đối số. Mặc dù gửi lặp theo một số cách nào đó là tổng quan hơn gửi đơn như trong Smalltalk, C++ và Java, ở đây có sự mất mát nào đó của tính đóng gói. Đặc biệt,

để xác định hàm trên các kiểu khác nhau của đối số, mà hàm cần được truy cập đến dữ liệu bên trong của mỗi đối số hàm.

Vì các ngôn ngữ gửi đơn là dòng chính của hướng đối tượng, nên chúng ta tập trung vào các ngôn ngữ gửi đơn trong tài liệu này.

6.2.2 Trừu tượng

Như đã bàn trong chương trước, trừu tượng bao gồm hạn chế truy cập đến thành phần chương trình tùy theo giao diện đặc tả của nó. Trong các ngôn ngữ lập trình hiện đại, truy cập đến đối tượng được hạn chế đến tập các thao tác public mà được chọn bởi người thiết kế và cài đặt đối tượng. Chẳng hạn, trong chương trình mà thao tác các hình đồ họa, mỗi hình có thể được biểu diễn bởi một đối tượng. Chúng ta có thể cài đặt đối tượng biểu diễn đường tròn bằng việc lưu trữ tâm và bán kính của đường tròn. Người thiết kế đối tượng đường tròn có thể chọn hàm thay đổi tâm của đường tròn như một phần của giao diện hoặc không đặt hàm đó vào giao diện. Nếu ở đó không có hàm public để thay đổi tâm của đường tròn, thì không có client code nào có thể thay đổi tâm của đường tròn, vì client code chỉ có thể thao tác đối tượng thông qua các giao diện của chúng.

Trừu tượng dựa trên các đối tượng cũng tương tự theo nhiều khía cạnh như trừu tượng dựa trên kiểu dữ liệu trừu tượng. Các đối tượng và các kiểu dữ liệu trừu tượng cả hai đều kết hợp các hàm với dữ liệu, và trừu tượng trong cả hai trường hợp bao gồm cả việc phân biệt giữa giao diện công khai và cài đặt riêng. Tuy nhiên, các đặc trưng khác của ngôn ngữ hướng đối tượng làm cho trừu tượng trong ngôn ngữ hướng đối tượng linh hoạt hơn trừu tượng ở đó kiểu dữ liệu trừu tượng được sử dụng. Một cách để hiểu sự linh hoạt của ngôn ngữ hướng đối tượng là xem cách mà quan hệ giữa các trừu tượng tương tự được sử dụng để chứng tỏ ưu thế.

Xét hai kiểu dữ liệu trừu tượng được viết theo cú pháp ML. Cái thứ nhất là kiểu dữ liệu trừu tượng của hàng đợi, cái thứ hai là kiểu dữ liệu trừu tượng của hàng đợi ưu tiên. Để đơn giản, cả hai hàng đợi và hàng đợi ưu tiên đều định nghĩa cho dữ liệu số nguyên:

```
exception Empty;
abstype queue = Q of int list
with
  fun mk_Queue() = Q(nil)
  and is_empty(Q(l)) = l=nil
  and add(x,Q(l)) = Q(l @ [x])
  and first (Q(nil)) = raise Empty | first (Q(x::l)) = x
  and rest (Q(nil)) = raise Empty | rest (Q(x::l)) = Q(l)
  and length (Q(nil)) = 0 | length (Q(x::l)) = 1 + length (Q(l))
end;
```

Trong kiểu dữ liệu trừu tượng này, hàng đợi được biểu diễn qua danh sách. Thao tác add sử dụng phép toán bổ sung @ của ML để chèn phần tử mới vào cuối danh sách. Phép toán first và rest đọc và loại bỏ phần tử khỏi đầu danh sách. Vì client code không thể thao tác trực tiếp biểu diễn của hàng đợi, cài đặt luôn duy trì bất biến: các phần tử List xuất hiện theo thứ tự vào trước/ra trước, cho dù hàng đợi được sử dụng như thế nào trong chương trình của client.

Hàng đợi ưu tiên tương tự như hàng đợi, ngoại trừ các phần tử được loại bỏ theo thứ tự ưu tiên nào đó. Cụ thể hơn, sự ưu tiên nào đó được cho trước cho các phần tử, phép toán first và remove đọc và loại bỏ các phần tử của hàng đợi mà có độ ưu tiên cao nhất.

```

abstype pqueue = Q of int list
with
  fun mk_PQueue() = Q(nil)
  and is_empty(Q(l)) = l=nil
  and add(x,Q(l)) =
    let fun insert(x,nil) = [x:int]
        | insert(x,y::l) = if x<y then x::y::l else y::insert(x,l)
    in Q(insert(x,l)) end
  and first (Q(nil)) = raise Empty | first (Q(x::l)) = x
  and rest (Q(nil)) = raise Empty | rest (Q(x::l)) = Q(l)
  and length (Q(nil)) = 0 | length (Q(x::l)) = 1 + length (Q(l))
end;

```

Giao diện của kiểu dữ liệu trừu tượng là danh sách các hàm công khai và kiểu của chúng. Hàng đợi và hàng đợi ưu tiên có cùng giao diện. Cả hai có cùng số các thao tác như nhau, các thao tác có cùng tên và mỗi thao tác có cùng kiểu trong cả hai trường hợp ngoại trừ sự khác biệt giữa tên kiểu pqueue và queue. Điểm nhấn của ví dụ này là, mặc dù các giao diện của hàng đợi và hàng đợi ưu tiên là như nhau, sự tương ứng này không được sử dụng trong các ngôn ngữ truyền thống với kiểu dữ liệu trừu tượng. Trái lại nếu chúng ta cài đặt hàng đợi và hàng đợi ưu tiên trong một ngôn ngữ hướng đối tượng, thì chúng ta cần sử dụng tính ưu việt về sự tương tự giữa giao diện của hai cấu trúc dữ liệu này.

Nhược điểm của kiểu dữ liệu trừu tượng này được sử dụng trong ML và các ngôn ngữ tương tự lộ rõ khi chúng ta xét chương trình mà sử dụng cả hai hàng đợi và hàng đợi ưu tiên. Chẳng hạn, giả sử chúng ta xây dựng hệ thống với một số hàng chờ, như bệnh viện. Trong bộ phận khám bệnh tại bệnh viện, khách hàng được phục vụ theo nguyên tắc đến trước phục vụ trước. Tuy nhiên, trong phòng cấp cứu, bệnh nhân được chữa theo thứ tự tính nghiêm trọng của vết thương hoặc sự đau đớn. Trong chương trình bệnh viện, chúng ta có thể xử lý hàng đợi ưu tiên và hàng đợi thông thường cùng nhau. Chẳng hạn, chúng ta có thể đếm số người chờ trong mỗi hàng đợi vào bệnh viện. Để viết đoạn code này, chúng ta muốn có danh sách cả hai hàng đợi và hàng ưu tiên trong bệnh viện và tính tổng độ dài các hàng đợi trong danh sách. Tuy nhiên, nếu thao tác tính độ dài là khác nhau đối với hàng đợi và hàng đợi ưu tiên, chúng ta phải quyết định gọi q-length hoặc

pq-length, mặc dù thao tác đúng được xác định duy nhất bởi dữ liệu. Nhược điểm này của kiểu dữ liệu trừu tượng thông thường được loại bỏ trong ngôn ngữ lập trình hướng đối tượng bằng cách kết hợp kiểu con và tìm kiếm động.

Cài đặt hàng đợi ưu tiên chỉ cho ta một nhược điểm khác của kiểu dữ liệu trừu tượng truyền thống. Mặc dù hàm add của hàng đợi ưu tiên là khác với hàm add của hàng đợi, năm hàm khác có cài đặt giống nhau. Trong ngôn ngữ hướng đối tượng, chúng ta có thể sử dụng kế thừa để định nghĩa hàng đợi ưu tiên từ hàng đợi (hoặc ngược lại), chỉ cho thêm hàm add mới và tái sử dụng các hàm còn lại.

6.2.3 Kiểu con

Kiểu con là quan hệ trên các kiểu mà cho phép các giá trị của một kiểu được sử dụng vào chỗ các giá trị của kiểu khác. Mặc dù đơn giản nhất là mô tả kiểu con trong ngữ cảnh ngôn ngữ lập trình kiểu tĩnh, và không có quan hệ kiểu con ẩn trong ngôn ngữ lập trình không kiểu. Chúng ta sẽ bàn tới kiểu con với giả thiết chúng ta đang ở trong ngôn ngữ kiểu.

Trong hầu hết ngôn ngữ kiểu, áp dụng hàm f cho đối số x đòi hỏi quan hệ nào đó giữa kiểu của f và kiểu của x . Trong trường hợp chung f cần phải là hàm từ kiểu A vào kiểu B đối với A, B nào đó, và x cần là biến hoặc biểu thức kiểu A . Chúng ta cần nghĩ là so sánh kiểu này như so sánh bằng nhau. Kiểm tra kiểu sẽ tìm kiểu $A \rightarrow B$ cho hàm f và kiểu C cho x , và kiểm tra $A=C$ không.

Trong ngôn ngữ có kiểu con (subtyping), ở đây có quan hệ kiểu con trên các kiểu. Nguyên tắc cơ bản gắn kết với kiểu con là khả năng thay thế : nếu A là kiểu con của B , thì mọi biểu thức của kiểu A có thể được sử dụng không có lỗi kiểu trong mọi tình huống mà đòi hỏi biểu thức kiểu B . Chúng ta viết $A < B$ để chỉ ra rằng A là kiểu con của B .

Với subtyping, quan hệ kiểu con được sử dụng thay thế bằng nhau trong kiểm tra kiểu. Cụ thể, để xác định kiểu của việc áp dụng f cho đối số x , bộ kiểm tra kiểu tìm kiểu $A \rightarrow B$ cho hàm f và kiểu C cho x , và kiểm tra xem C có là kiểu con của A không.

Ưu điểm cơ bản của subtyping là nó cho phép các thao tác thống nhất trên các kiểu dữ liệu khác nhau. Chẳng hạn, subtyping làm cho có thể có các cấu trúc dữ liệu không đồng nhất và chứa các đối tượng thuộc các kiểu con khác nhau của cùng một kiểu chung. Xét ví dụ về hàng đợi chứa các tài khoản ngân hàng khác nhau cần phải cân đối. Các tài khoản này có thể là tài khoản tiết kiệm, séc, đầu tư, ... Tuy nhiên, mỗi kiểu tài khoản là kiểu con của `bank_account`, và hàng đợi các phần tử của kiểu `bank_account` có thể chứa mọi kiểu của tài khoản.

Subtyping trong ngôn ngữ hướng đối tượng cũng cho phép chức năng được bổ sung mà không cần điều chỉnh các phần chung của hệ thống. Nếu các đối tượng của kiểu B không có hành vi mong muốn nào đó, thì ta có thể thay các đối tượng kiểu B bằng các đối tượng của kiểu A khác mà có hành vi mong muốn đó. Trong nhiều trường hợp, kiểu A sẽ là kiểu con của B. Bằng việc thiết kế ngôn ngữ, sao cho sự thay thế là được phép, ta có thể bổ sung chức năng theo cách này mà không cần thay đổi gì trong chương trình gốc.

Việc sử dụng subtyping cho phép xây dựng một loạt các mẫu của hệ thống lập lịch sân bay. Trong mẫu trước đây, có thể định nghĩa class airplane với các phương thức position, orientation, và acceleration mà có thể cho phép đối tượng thao tác điều hành tác động hướng tiếp đất của máy bay. Và có lẽ các kiểu máy bay khác nhau cần được mô hình hóa. Nếu ta muốn bổ sung các lớp cho Boeing 757 và Beechcraft, cần phải có kiểu con của máy bay, chứa các phương thức và các trường bổ sung phản ánh các đặc trưng riêng biệt của các máy bay này, và các thuật toán điều hành chung mà áp dụng cho mọi máy bay có thể được sử dụng cho Boeing 757 và Beechcraft không cần chỉnh sửa gì.

6.2.4 Kế thừa

Kế thừa là tính chất của ngôn ngữ mà cho phép các đối tượng mới được định nghĩa từ những cái đã có. Chúng ta bàn luận về dạng kế thừa mà xuất hiện trong hầu hết các ngôn ngữ hướng đối tượng dựa trên lớp bằng việc sử dụng khái niệm trung lập. Class A định nghĩa các đối tượng với dữ liệu riêng v và các phương thức công khai f và g. Chúng ta định nghĩa class B bằng cách kế thừa các khai báo của A, định nghĩa lại g và bổ sung biến riêng w:

```
class A =
  private
    val v = ...
  public
    fun f(x) = ... g(...) ...
    fun g(y) = ... original definition ...
end;
class B = extend A with
  private
    val w = ...
  public
    fun g(y) = ... new definition ...
end;
```

```

private
  val v = ...
public
  fun f(x) = ... g(...) ...
  fun g(y) = ... original definition ...
end;
class B = extend A with
  private
    val w = ...
  public
    fun g(y) = ... new definition ...
end;

```

Về mặt nguyên tắc, kế thừa được cài đặt bằng cách nhân bản code. Với mỗi đối tượng hoặc lớp các đối tượng được định nghĩa bằng kế thừa, ở đây có định nghĩa tương ứng mà không kế thừa: nó nhận được bằng cách mở rộng định nghĩa trên mà code kế thừa được nhân bản. Quan trọng của kế thừa là nó tiết kiệm nỗ lực nhân bản code khi một lớp được cài đặt bằng kế thừa từ lớp khác, thay đổi tác động này sang tác động khác. Nó có sự nêm chặt quan trọng về mặt bảo trì và chỉnh sửa code.

Cài đặt trực tiếp của kế thừa mà ngăn việc nhân bản là xây dựng cấu trúc dữ liệu liên kết của các bảng tìm kiếm phương thức. Cụ thể hơn, với mỗi lớp, bảng tìm kiếm cần phải chứa danh sách các thao tác gắn kết với lớp. Khi một lớp kế thừa từ lớp khác, thì lớp thứ hai chứa con trỏ đến bảng tìm kiếm của lớp thứ nhất. Nếu phương thức được kế thừa, nó có thể được tìm kiếm trong bảng tìm kiếm phương thức của lớp mà ở đó nó được định nghĩa chính gốc.

Tối ưu quan trọng có thể được thực hiện trong ngôn ngữ kiểu tĩnh như C++, ở đó tập các thông điệp có thể đến mỗi đối tượng được xác định trong thời gian dịch. Nếu các bảng tìm kiếm có thể được xây dựng sao cho mọi lớp con của lớp đã cho lưu các con trỏ lặp tại các vị trí tương đối như nhau, thì offset của phương thức trong bảng tìm kiếm có thể được tính trong thời gian dịch. Điều này làm giảm giá tìm kiếm phương thức đến việc định hướng đơn giản mà không cần tìm kiếm, theo lời gọi hàm thông thường. Chúng ta sẽ xem xét cài đặt kế thừa chi tiết trong chương sau.

Kế thừa và trừu tượng

Trong modules và kiểu dữ liệu thông thường, ở đây có hai cách nhìn về trừu tượng: cách nhìn client và cách nhìn cài đặt. Với kế thừa, có ba cách nhìn về lớp: cách nhìn cài đặt, cách nhìn client và cách nhìn kế thừa. Cách nhìn kế thừa là cách nhìn về các lớp mà kế thừa từ một lớp. Vì các định nghĩa đối tượng có hai clients bên ngoài, có hai giao diện ra bên ngoài: danh sách giao diện công khai mà client tổng quát có nhìn thấy, trong khi đó danh sách giao diện protected mà chỉ người bên trong mới có thể nhìn thấy (Thuật ngữ lấy từ C++). Trong đa số các ngôn ngữ, giao diện công khai là tập con của giao diện protected. Trong Smalltalk, các giao diện này được sinh tự động: giao diện công

khai bao gồm mọi phương thức của đối tượng, trong khi đó giao diện `protected` là mọi phương thức và mọi biến thành viên. Trong C++ người lập trình khai báo tường minh thành phần nào của đối tượng là công khai và thành phần nào là `private` và `visible` định nghĩa chính lớp đó. Chúng ta sẽ trao đổi kỹ hơn trong chương sau.

6.2.5 Closures như các đối tượng

Đặc trưng thứ nhất của đối tượng, tìm kiếm động, cũng được cung cấp bởi các bản ghi (thuật ngữ trong Pascal và ML) hoặc `structs` (thuật ngữ trong C). Trong ngôn ngữ với closures, chúng ta có thể mô phỏng các đối tượng bằng cách sử dụng các bản ghi mà có thành phần hàm. Nếu các đối tượng có dữ liệu `private` (hoặc các hàm số), thì chúng ta có thể giấu chúng bằng cách sử dụng phạm vi tĩnh. Nó dẫn ta đến việc tìm kiếm closures bản ghi như mô hình đầu tiên của đối tượng. Sẽ là bài học tốt xem khái niệm này có ích khi nào và khi nào không.

```
exception Empty;
fun newStack(x) =
  let val store = ref [x]
  in
    {push = fn(y) => store := y :: !store,
     pop = fn() => case !store of
       nil => raise Empty
     | (y::ys) => (store := ys; y)
    }
  end;
val myStack = newStack(0);
#push(myStack)(1);
#pop(myStack)();
```

Ký hiệu `#field_name(record_value)` là ký hiệu của ML cho việc chọn trường. Trong cú pháp tựa Pascal, biểu thức này có thể được viết như `record_value.field_name`.

Hàm `newStack` trả về bản ghi với hai thành phần, thành phần thứ nhất gọi là `push`, thành phần thứ hai gọi là `pop`. Vì các trường của bản ghi này chứa các hàm, chúng được biểu diễn trong thời gian chạy như closures. Các con trỏ môi trường cho các closures này trỏ đến bản ghi kích hoạt cho hàm `newStack` mà lưu dữ liệu cục bộ `store`. Giá trị ban đầu của `store` là danh sách chỉ chứa phần tử ban đầu được truyền như đối số của `newStack`. Nếu bạn lấy ra các bản ghi kích hoạt và closures, bạn sẽ nhận được sơ đồ mà rất giống với những cái mà chúng ta vẽ để biểu diễn các đối tượng. Tuy nhiên, các ngôn ngữ hướng đối tượng tối ưu biểu diễn theo một hoặc một số cách.

Vì closures và đối tượng về bản chất có các chức năng giống nhau, lý do vì sao ta nói về lập trình hướng đối tượng thay vì lập trình hướng closures. Nói cách khác các ngôn ngữ lập trình hướng đối tượng làm gì mà ML không làm được. Câu trả lời là `subtyping` và kế thừa. Nếu chúng ta chuyển chương trình hướng đối tượng về ngôn ngữ không hướng đối tượng, bạn sẽ đánh giá được sự hỗ trợ của ngôn ngữ đối với `subtyping` và kế thừa.

6.2.6 Kế thừa không phải subtyping

Có thể sự nhầm lẫn chung nhất xung quanh các ngôn ngữ lập trình hướng đối tượng là sự khác nhau giữa kiểu con và kế thừa. Sự khác nhau đơn giản nhất giữa kiểu con và kế thừa là : kiểu con là quan hệ trên các giao diện, kế thừa là quan hệ trên các cài đặt.

Một lý do mà kiểu con và kế thừa thường bị nhầm lẫn cơ chế lớp nào đó kết hợp cả hai. Ví dụ điển hình trong C++, ở đó A được công nhận bởi chương trình dịch như kiểu con của B, chỉ nếu B là lớp cơ sở công khai của A. Kết hợp kiểu con và kế thừa là quyết định thiết kế lựa chọn, tuy nhiên C++ có thể được thiết kế khác đi không cần liên kết kiểu con với lớp cơ sở công khai theo cách này.

Chúng ta có thể thấy, về nguyên tắc, kiểu con và kế thừa không phải khi nào cũng đi cạnh nhau bằng cách xét ví dụ đề xuất bởi nhà nghiên cứu hướng đối tượng, Alan Snyder. Giả sử chúng ta quan tâm đến việc viết chương trình mà đòi hỏi dequeues, stacks, và queues. Ở đây có ba kiểu cấu trúc tương tự, với các đặc trưng cơ bản sau:

- Queues: cấu trúc dữ liệu với các thao tác insert và delete, sao cho phần tử đầu được chèn là phần tử đầu được loại bỏ (first in, first out)
- Stacks: cấu trúc dữ liệu với các thao tác insert và delete, sao cho phần tử đầu được chèn là phần tử cuối được loại bỏ (last in, first out)
- Dequeues : cấu trúc dữ liệu với hai thao tác insert và delete. Dequeue hoặc hàng đợi kết thúc hai đầu, bản chất là danh sách mà cho phép chèn và xóa từ mỗi đầu. Nếu một phần tử được chèn vào một đầu, thì nó sẽ là phần tử đầu tiên trả về từ dãy các phép removes từ đầu đó và là cuối cùng trả về từ dãy các phép removes từ đầu ngược lại.

Phần quan trọng của quan hệ giữa stacks, queues và dequeues là dequeue có thể coi như stack và queue hai đầu. Cụ thể, giả thiết dequeue có các thao tác insert như insert_front và insert_rear và các thao tác xóa như delete_front và delete_rear. Nếu ta chỉ muốn sử dụng insert_front và delete_rear, chúng ta có hàng đợi. Tuy nhiên, nếu ta sử dụng insert_front và delete_front, ta có ngăn xếp.

Một cách cài đặt ba lớp này là trước hết cài đặt dequeue và sau đó cài đặt stack và queue bằng việc hạn chế một cách thích hợp (và có thể đổi tên) các thao tác của dequeue. Chẳng hạn, chúng ta có thể nhận được stack từ dequeue bằng cách hạn chế truy cập đến các thao tác như add và remove các phần tử từ đầu cuối của dequeue. Tương tự, chúng ta có thể nhận được queue từ dequeue bằng cách hạn chế truy cập đến các thao tác như add các phần tử từ đầu cuối và remove các phần tử từ đầu kia. Phương pháp này định nghĩa stack và queue bằng việc kế thừa từ dequeue có thể thực hiện trong C++ qua việc

sử dụng kế thừa private. (Điều này không là phong cách cài đặt mong muốn, ví dụ này chỉ sử dụng đơn giản để minh họa cho sự khác biệt giữa subtyping và kế thừa).

Mặc dù stack và queue có thể được cài đặt từ dequeue, chúng không phải là subtypes của dequeue. Xét hàm f mà nhận dequeue d như đối số và sau đó add phần tử từ cả hai đầu của d . Nếu stack hoặc queue là subtypes của dequeue, thì hàm f cần làm việc tốt như nhau nếu cho trước stack s hoặc queue q . Tuy nhiên, thêm các phần tử vào cả hai đầu không là hợp lệ đối với cả stack và queue, và do đó cả stack và queue đều không là subtypes của dequeue. Thực tế, điều ngược lại là đúng. Dequeue là kiểu con của cả stack và queue, vì mọi thao tác của stack hoặc queue đều là thao tác hợp lệ của dequeue. Vì vậy, kế thừa và subtyping là quan hệ khác nhau về mặt nguyên tắc: hoàn toàn có lý nếu định nghĩa stack và queue là kế thừa từ dequeue, nhưng dequeue là subtypes của stack và queue, chứ không phải ngược lại.

6.3 Cấu trúc chương trình

Có một số sự khác nhau về hệ thống giữa chương trình cấu trúc định hướng hàm (hay gọi là hướng thủ tục) và chương trình hướng đối tượng. Một trong những sự khác biệt chính là ở trong tổ chức của các hàm và dữ liệu. Trong lập trình hướng hàm, dữ liệu và các hàm được khai báo riêng biệt. Nếu hàm được áp dụng cho nhiều kiểu dữ liệu, thì nói chung là sử dụng một dạng phân chia trường hợp nào đó và lệnh switch trong thân của hàm. Trong chương trình hướng đối tượng, các hàm được gắn kết với dữ liệu mà chúng được thiết kế để thao tác. Sử dụng tìm kiếm động, việc cài đặt ngôn ngữ lập trình sẽ lựa chọn hàm đúng cho mỗi loại dữ liệu. Sự khác nhau cơ bản này giữa chương trình hướng hàm và hướng đối tượng được thể hiện qua phép so sánh trong Ví dụ 6.1 và 6.2. Các ví dụ dài hơn mô tả điều này, được viết trong C và C++ mô tả trong phụ đề B.1.

Trong cả hai Ví dụ 6.1 và 6.2 chúng ta xem xét bối cảnh bệnh viện. Dữ liệu trong các ví dụ này thể hiện doctors, nurses và orderly (người phục vụ). Các hàm sẽ được áp dụng cho các kiểu dữ liệu này bao gồm hàm thể hiện thông tin về employee của bệnh viện và hàm để thiết lập hoặc xác định việc trả thù lao cho employee.

Ví dụ 6.1. Tổ chức hướng hàm truyền thống

Trong chương trình hướng hàm truyền thống, các thao tác nhóm lại thành hàm. Nếu chúng ta muốn một hàm đơn thể hiện thông tin về mọi kiểu nhân viên của bệnh viện, thì ta có thể sử dụng việc kiểm tra thời gian chạy để xác định áp dụng như thế nào mỗi thao tác đối với dữ liệu đã cho. Về tổng thể, codes để thể hiện và hàm trả thù lao có thể trông như sau:

```

display(x) =
  case type(x) of
    Doctor : [ "display Doctor" ]
    Nurse : [ "display Nurse" ]
    Orderly : [ "display Orderly" ]
  end;
end;
pay(x) =
  case type(x) of
    Doctor : [ "pay Doctor a lot" ]
    Nurse : [ "pay Nurse less" ]
    Orderly : [ "pay Orderly less than that" ]
  end;
end;

```

Ví dụ 6.2. Tổ chức hướng đối tượng.

Trong chương trình hướng đối tượng, các hàm được nhóm lại với dữ liệu mà chúng được thiết kế để thao tác. Đối với ví dụ bệnh viện, các lớp doctor, nurse và orderly sẽ chứa code cho hai hàm. Về tổng thể, nó tạo ra tổ chức chương trình như sau:

```

class Doctor =
  display = "Display Doctor";
  pay = "pay Doctor a lot ";
end;
class Nurse =
  display = "display Nurse";
  pay = "pay Nurse less";
end;
class Orderly =
  display = "display Orderly";
  pay = "pay Orderly less than that";
end;

```

So sánh các ví dụ 6.1 và 6.2

Dữ liệu và các thao tác sử dụng trong các ví dụ 6.1 và 6.2 có thể được sắp xếp vào ma trận sau. Trong tổ chức hướng hàm truyền thống, code được sắp xếp thành hàng vào các hàm mà làm việc với mọi kiểu dữ liệu. Trong tổ chức hướng đối tượng, code được sắp xếp thành cột, nhóm mỗi trường hợp với kiểu dữ liệu mà nó được thiết kế cho kiểu đó.

Operation	Doctor	Nurse	Orderly
Display	Display Doctor	Display Nurse	Display Orderly
Pay	Pay Doctor	Pay Nurse	Pay Orderly

Trong tổ chức hướng hàm, dễ dàng thêm thao tác mới, như PayBonus hoặc Promote, nhưng khó bổ sung kiểu dữ liệu mới như Administrator hoặc Intern. Trong tổ chức hướng đối tượng, dễ dàng bổ sung kiểu dữ liệu mới như Administrator hoặc Intern, nhưng rắc rối hơn để thêm thao tác mới như PayBonus hoặc Promote, vì nó bao gồm thay đổi mỗi một lớp.

6.4 Mẫu thiết kế (Design patterns)

Phương pháp mẫu thiết kế là tiếp cận phổ biến cho thiết kế phần mềm mà đã phát triển cùng với sự lớn mạnh của lập trình hướng đối tượng. Trong thuật ngữ cơ bản, design pattern là giải pháp tổng quát mà đến từ việc trích dẫn lặp của các vấn đề tương tự. Thiết kế mẫu không phải là giải pháp phát triển từ các nguyên lý đầu tiên hoặc code mẫu mà có thể đơn giản được khởi tạo cho nhiều mục đích. Thay vào đó, mẫu thiết kế là chỉ dẫn hoặc cách tiếp cận để giải quyết kiểu bài toán mà đã xảy ra trong nhiều dạng cụ thể. Giải pháp dựa trên thiết kế mẫu cần là tương tự, áp dụng mẫu thiết kế cho tình huống cụ thể cần đòi hỏi sự suy xét nào đó.

Khái niệm mẫu thiết kế có thể được sử dụng trong bất cứ nguyên lý thiết kế nào, như thiết kế cơ khí hoặc kiến trúc. Công trình của kiến trúc sư Christopher Alexander thường được tham khảo như nguồn cảm hứng cho thiết kế phần mềm. Ở đây có các ví dụ kiến trúc, lấy từ một trong số cuốn sách của Alexander (*A Pattern Language : Towns, Buildings, Construction, Oxford Univ. Press, 1977*). Nó bao gồm cả mô tả ngữ cảnh bài toán và giải pháp phát triển như kết quả của kinh nghiệm:

Vòng tròn ngồi

Một nhóm các ghế, sofa, một đồng các đệm- có hầu hết mọi thứ trong cuộc sống của con người – và bây giờ cần sắp xếp chúng để con người có thể chuyển động và sống trong chúng là một việc kinh doanh rất tinh tế. Hầu hết việc sắp xếp chỗ ngồi là khó, mọi người tránh nó, không có gì xảy ra ở đó được. Số khác tỏ ra không biết làm sao thu xếp cuộc sống quanh chúng để tập trung và giải phóng năng lượng. Sự khác biệt giữa hai cái đó là gì ?

Do đó, đặt mỗi không gian ngồi tại vị trí mà nó được bảo vệ, không cắt ngang đường đi hoặc chuyển động, gần như là vòng tròn, thực hiện sao cho bản thân phòng giúp cho

tạo nên vòng tròn với các đường đi và hoạt động xung quanh nó, sao cho mọi người có thể tự nhiên hút về phía các ghế khi họ rơi vào tâm trạng muốn ngồi. Đặt các ghế và đệm gần nhau thành vòng tròn và có rất nhiều chỗ.

Khi các chương trình nhận thấy rằng họ đã giải quyết đi và giải quyết lại cùng một kiểu bài toán theo các cách khác nhau đôi chút, nhưng sử dụng cùng ý tưởng thiết kế, họ có thể tìm cách đưa một mẫu thiết kế chung cho các giải pháp của họ. Sự phổ biến của tiến trình này dẫn đến việc định danh một số lớn các mẫu thiết kế phần mềm. Theo Jim Coplien, mẫu tốt có các đặc tính sau:

- Nó giải quyết bài toán: Mẫu bao gồm cả giải pháp, không chỉ là chiến lược và nguyên lý trừu tượng
- Nó là khái niệm được chứng minh. Mẫu bao gồm giải pháp với bản ghi dấu vết, không phải là lý thuyết hoặc nghiên cứu
- Giải pháp là không hiển nhiên. Nhiều kỹ thuật giải quyết vấn đề (như các xu thế hoặc phương pháp thiết kế phần mềm) tìm cách đưa ra giải pháp từ các nguyên lý đầu tiên. Các mẫu tốt nhất sinh ra gián tiếp lời giải cho bài toán – cách tiếp cận cần thiết cho các vấn đề thiết kế khó nhất.
- Nó mô tả quan hệ: các mẫu không chỉ mô tả modules mà còn mô tả sâu hơn cấu trúc và cơ chế hệ thống
- Các mẫu có thành phần con người quan trọng (cực tiểu sự can thiệp của con người). Mọi phần mềm phục vụ sự thuận tiện của con người hoặc chất lượng cuộc sống, các mẫu tốt nhất hiện hữu một cách tường minh cho tính thẩm mỹ và tiện ích.

Trước khi đọc các nguyên lý chung về mẫu thiết kế, cách tốt nhất học về mẫu thiết kế là tìm hiểu một số ví dụ và sử dụng mẫu trong lập trình của bạn. Cuốn sách được sử dụng rộng rãi là *Design Patterns : Element of Reusable Object-Oriented Software* của E.Gamma, R. Helm, R. Johnson và J. Vlissides (Addison-Wesley, 1994).

Ví dụ 6.3. Singleton Design Patterns

Mẫu thiết kế đơn là mẫu thiết kế tạo, nghĩa là đây là mẫu được dùng để tạo ra đối tượng theo một cách nào đó. Sau đây là tổng quan ngắn gọn về mẫu thiết kế đơn, mà sử dụng kiểu đối tượng tiêu đề được sử dụng chung trong các cuốn sách và các thể hiện khác của các mẫu thiết kế.

Mục tiêu

Mẫu thiết kế đơn có ích trong các tình huống, ở đó cần khởi tạo các đối tượng của một lớp. Mẫu này cho phép kiểm soát trực tiếp lớp về việc có bao nhiêu đối tượng cần được

tạo. Nó tốt hơn là buộc lập trình viên có trách nhiệm chỉ để tạo một instance, cũng như việc hạn chế này được xây dựng trong chương trình.

Cài đặt.

Chỉ cần viết một lớp để cài đặt mẫu đơn. Lớp sử dụng đóng gói để giữ cho hàm tạo của lớp ẩn đối với mã client. Lớp có phương thức public mà gọi hàm tạo chỉ nếu đối tượng của lớp đó chưa được tạo. Nếu đối tượng đã được tạo, thì hàm public sẽ trả con trỏ về đối tượng đó và không tạo đối tượng mới.

Code ví dụ

Đây là ví dụ về mẫu đơn tổng quát được viết như thế nào trong C++. Bạn đọc mà không quen với C++ có thể đọc giải thích này và sẽ quay trở lại sau khi đọc chương sau. Giao diện của lớp Singleton cung cấp phương thức public mà cho phép client code yêu cầu khởi tạo lớp đó:

```
class Singleton {
public:
    static Singleton* instance(); // function that returns an instance
protected:
    Singleton(); // constructor is not made public
private:
    static Singleton* _instance; // private pointer to single object
};
```

Sau đây là cài đặt. Ban đầu, con trỏ riêng `pointer_instance` được đặt bằng 0. Trong cài đặt phương thức public `instance()` một đối tượng mới được tạo và được gán to `_instance` chỉ nếu lời gọi trước chưa tạo đối tượng của lớp này :

```
Singleton* Singleton::_singleton = 0
Singleton* Singleton::instance() {
    if (_instance == 0){_instance = new Singleton; }
    return _instance;
}
```

Ví dụ 6.4. Facade

Facade là mẫu đối tượng cấu trúc (structural object pattern), có nghĩa đây là mẫu về việc kết hợp các đối tượng thành cấu trúc lớn hơn chứa nhiều đối tượng.

Mục tiêu

Mẫu facade cung cấp đối tượng đơn để truy cập đến tập các đối tượng mà được kết hợp để tạo thành một cấu trúc. Muốn vậy, facade cung cấp giao diện mức độ cao gom các đối tượng tạo thành họ để sử dụng hơn.

Cài đặt.

Đây là lớp facade, được định nghĩa cho tập các lớp mà được sử dụng để tạo nên cấu trúc ‘bên dưới’ facade. Trong việc sử dụng thông thường, đối tượng facade có đoạn code thực tế khá bé, truyền hầu hết các lời gọi đến các đối tượng trong cấu trúc đó ‘bên dưới’ facade.

Ví dụ mẫu Facade

Facade là mẫu rất phổ biến khi nhiệm vụ được hoàn thành bằng việc kết hợp từ các kết quả của một số nhiệm vụ con. Chẳng hạn, chương trình dịch có thể được xây dựng bằng việc cài đặt phân tích từ vựng, phân tích cú pháp, phân tích ngữ nghĩa và một số pha khác nữa. Nếu mỗi pha được cài đặt như đối tượng với các phương thức mà thực hiện các chức năng chính của nó, thì bản thân chương trình dịch sẽ là đối tượng facade mà nhận chương trình như đầu vào và sử dụng các đối tượng riêng lẻ cài đặt từng pha để dịch chương trình. Người sử dụng chương trình dịch có thể thấy giao diện được thể hiện bằng đối tượng của chương trình dịch. Đây là giao diện hữu ích hơn các giao diện chi tiết cho mỗi đối tượng thành phần mà ẩn giấu dưới facade.

6.5 Tóm tắt chương

Chương này trình bày tổng quan về thiết kế hướng đối tượng và tóm tắt bốn khái niệm cơ bản gắn kết với các ngôn ngữ hướng đối tượng: tìm kiếm động, trừu tượng, kiểu con và kế thừa

- *Dynamic lookup* nghĩa là thông điệp được gửi tới đối tượng, mã hàm (hoặc phương thức) mà được thực thi sẽ được xác định theo cách mà đối tượng được cài đặt. Các đối tượng khác nhau có thể phản hồi cùng một thông điệp theo các cách khác nhau.
- *Abstraction* có nghĩa là chi tiết cài đặt được che giấu bên trong đơn vị chương trình với giao diện chuyên biệt. Giao diện này của đối tượng thường là tập hợp các hàm công khai (hoặc các phương thức công khai) mà thao tác trên dữ liệu được che giấu.
- *Subtyping* có nghĩa là nếu một đối tượng a có mọi chức năng của một đối tượng khác b , thì chúng ta có thể sử dụng a trong bất cứ bối cảnh nào giành cho b .
- *Inheritance* là khả năng tái sử dụng định nghĩa một kiểu đối tượng để định nghĩa một kiểu đối tượng khác.

Trong các ngôn ngữ truyền thống mà cài đặt closures và cho phép các bản ghi chứa hàm, các bản ghi cung cấp dạng tìm kiếm động và trừu tượng. Subtyping và inheritance ở dạng cần thiết để hỗ trợ lập trình hướng đối tượng, nói chung không có trong các ngôn ngữ truyền thống.

Nhiều người nhầm lẫn subtyping với inheritance. Như thuật ngữ sử dụng trong sách này, subtyping là quan hệ trên kiểu mà cho phép các giá trị của kiểu này được sử dụng tại vị trí giành cho kiểu khác. Còn kế thừa cho phép các đối tượng mới được định nghĩa từ các đối tượng đã tồn tại. Trong các ngôn ngữ dựa lớp, kế thừa cho phép cài đặt một lớp được tái sử dụng như một phần cài đặt của lớp khác. Cách đơn giản nhất để phân biệt subtyping và inheritance là: subtyping là quan hệ trên giao diện còn inheritance là quan hệ trên cài đặt.

Bài tập

BÀI 7: CÁC ĐỐI TƯỢNG VÀ TÍNH HIỆU QUẢ THỜI GIAN CHẠY CỦA C++

TỔNG QUAN

C++ là mở rộng hướng đối tượng của ngôn ngữ C. Ban đầu nó được gọi là C với các lớp, và có tên là C++ vào khoảng 1984. Nhà nghiên cứu của Phòng thí nghiệm Bell quan tâm về mô phỏng tên là Bjarne Stroustrup bắt đầu dự án C++ vào đầu những năm 1980. Mục đích của ông là bổ sung các đối tượng và lớp cho C, sử dụng kinh nghiệm của ông với Simula như cơ sở cho thiết kế. Thiết kế và c kinh nghiệm của ông với Simula như cơ sở cho thiết kế. Thiết kế và cài đặt C++ ban đầu là nỗ lực của một người, không có mục đích tạo ra sản phẩm thương mại. Tuy nhiên, vì mối quan tâm đến đối tượng và cấu trúc chương trình tăng trong những năm 1980, nên C++ trở nên phổ cập và được sử dụng rộng rãi. Trong những năm 1990, C++ trở thành ngôn ngữ lập trình hướng đối tượng được sử dụng rộng rãi nhất, với chương trình dịch và môi trường phát triển tốt cho các máy trạm Macintosh, PC và dựa trên Unix.

Ngôn ngữ lập trình C được sử dụng để viết hệ điều hành Unix tại Bell Laboratories. Cài đặt gốc của C++ là bước tiền xử lý mà chuyển đổi C++ thành C.

7.1 Các mục đích và ràng buộc thiết kế

Mục đích chính của C++ là cung cấp các đặc tính hướng đối tượng trong ngôn ngữ dựa trên C mà không làm mất tính hiệu quả của C. Trong quá trình bổ sung đối tượng vào C, một số cải tiến khác đã được thực hiện. Mục tiêu thiết kế chính của C++ có thể tóm tắt như sau:

- Trừu tượng dữ liệu và các đặc tính hướng đối tượng
- Kiểm tra kiểu tĩnh tốt hơn
- Tính tương thích ngược lại với C. Nói cách khác hầu hết mã C cần được dịch như C++ hợp lệ, không đòi hỏi sự thay đổi đáng kể về mã.
- Tính hiệu quả của mã dịch, tuân thủ theo nguyên tắc ‘nếu bạn không sử dụng đặc tính nào đó, bạn không phải trả giá cho nó’

Nguyên lý này được khẳng định trong mục đích cuối là rất quan trọng và có thể đòi hỏi phải suy nghĩ để đánh giá. Nguyên lý này giả thiết chương trình C cần dịch hiệu quả dưới chương trình dịch C++ giống như dưới chương trình dịch C. Như vậy có thể vi phạm nguyên lý này khi cài đặt các số nguyên của C như các đối tượng và sử dụng phương thức tìm kiếm động để tìm hàm số nguyên trong thời gian chạy, như nó có thể giảm đáng kể việc thực thi tính toán số nguyên C. Nguyên lý này không có nghĩa là các lệnh C++ mà cũng xuất hiện trong C cần phải được cài đặt chính xác theo cùng một cách trong cả hai ngôn ngữ, nhưng bất cứ thay đổi nào được chấp nhận trong C++, không được làm chậm việc thực thi mã dịch trừ khi một số đặc tính chậm hơn của C++ cũng được sử dụng trong chương trình.

7.1.1 Tương thích với C

Quyết định giữ tính tương thích với C có tác động bao trùm lên thiết kế C++. Những người quen với C biết rằng C có mô hình máy chuyên biệt, bộc lộ nhiều cấu trúc của kiến trúc máy tính bên dưới. Cụ thể các thao tác C mà trả về địa chỉ của biến hoặc đặt mẫu bit bất kỳ vào vị trí tùy ý là cho chương trình C có thể dựa trên biểu diễn chính xác của dữ liệu. Do đó C++ cần tôn trọng biểu diễn dữ liệu giống như C.

Hầu hết các ngôn ngữ hướng đối tượng khác, bao gồm cả những ngôn ngữ thiết kế trước và sau C++, sử dụng thu dọn rác để giảm nhẹ cho lập trình viên khỏi nhiệm vụ định danh các đối tượng không truy cập và giải phóng các bộ nhớ gắn kết. Tuy nhiên, không có lý do cố hữu nào, tại sao các đối tượng cần được thu dọn rác. Kết nối mạnh nhất là với điểm nhấn gia tăng về tính trừu tượng và tính đúng đắn kiểu. Nó cần phải phù hợp với các mục tiêu khác của C++ để xuất dạng gom rác nào đó mà khả thi ở dạng không ảnh hưởng thời gian chạy của chương trình mà không sử dụng các đối tượng gom rác. Tuy nhiên, vì có các đặc tính của C mà làm cho gom rác rất khó, khó đặt các đối tượng C++ vào nơi gom rác. Không chỉ là bộ đếm gom rác cho triết lý C để lập trình viên kiểm soát bộ nhớ, mà còn là sự tương tự giữa con trỏ và số nguyên làm cho không thể xây dựng hiệu quả bộ gom rác mà làm việc trên chương trình C++ sử dụng số học con trỏ hoặc chuyển số nguyên thành con trỏ.

Một quyết định quan trọng khác là xử lý các đối tượng C++ như khái quát các cấu trúc struct của C. Nó buộc phải cho phép các đối tượng được khai báo và thao tác cùng một cách như structs. Nói riêng, các đối tượng được đặt trong các bản ghi kích hoạt của hàm hoặc các khối cục bộ, cũng như trên heap, và cần được thao tác trực tiếp (tức là không qua con trỏ). Đây là một điểm mà C++ đi trệch hướng so với Simula. Cụ thể, C++ cho phép dạng gán các đối tượng mà sao một đối tượng sang không gian trước đó được chiếm bởi đối tượng khác, trong khi đó hầu hết các ngôn ngữ đối tượng khác chỉ cho phép gán con trỏ cho đối tượng. Một số khía cạnh khác của các đối tượng C++ trên ngăn xếp được khám phá trong các bài tập về nhà.

7.1.2 Thành công của C++

C++ là ngôn ngữ được thiết kế rất cẩn thận mà được kế tiếp một cách đáng khâm phục, tuy rằng buộc thiết kế rất khó. Do theo số người sử dụng, C++ không nghi ngờ gì nữa là ngôn ngữ thành công nhất của thập niên từ khi nó phát triển đến giữa những năm 1980 cho đến khi Java ra đời giữa những năm 1990. Tuy nhiên, mục đích thiết kế và sự tương thích ngược với C không cho phép có nhiều không gian cho những xem xét bổ sung về mặt thẩm mỹ. Một số khía cạnh của C++ trở nên phức tạp và khó hiểu đối với nhiều lập trình viên. Mặt khác nhiều lập trình viên C sử dụng chương trình dịch C++ và đánh giá lợi ích của kiểm tra kiểu tốt hơn. Có lẽ tóm tắt công bằng cho thành công của C++ là nó được sử dụng rộng rãi, với hầu hết người sử dụng được chọn để lập trình trong một tập các ngôn ngữ mà họ hiểu và cảm thấy thuận tiện cho nhiệm vụ lập trình của họ. Nói cách khác, C++ là công cụ lập trình hữu ích mà cho phép người thiết kế tạo ra các chương trình hướng đối tượng tốt, nhưng nó không buộc phong cách lập trình tốt theo cách mà các thiết kế ngôn ngữ khác thường làm. Điều này hướng tới khẳng định rằng không có con số thống kê nào chống lại C++. Trên thực tế, nhiều thành công của nó có thể quy về cách mà C++ được thiết kế để cho các lập trình viên các lựa chọn và hạn chế lập trình viên theo một phong cách lập trình riêng nào.

Có nhiều hướng dẫn phong cách đã được in mà biện hộ việc sử dụng một số đặc tính của C++ và cảnh báo việc sử dụng những cái khác. Những ai mà quan tâm đến việc lập trình nghiêm túc C++ hoặc quan tâm tìm hiểu bao nhiêu lập trình viên dùng ngôn ngữ này, có thể muốn tham quan thư viện hoặc kho sách của chúng và đọc một số hướng dẫn hiện hành.

7.2 Tổng quan về C++

Trước khi xem xét các đặc tính của C++ trong mục sau, ta sẽ lướt qua một số bổ sung cho C mà không liên quan đến đối tượng.

7.2.1 Bổ sung cho C không liên quan đến đối tượng

Có một số khác biệt giữa C++ và C mà không liên quan đến đối tượng. Mặc dù chúng ta quan tâm chủ yếu đến hệ thống đối tượng của C++, nhưng vẫn nên xem xét một thay đổi quan trọng. Một số bổ sung thú vị nhất là

- Kiểu bool
- Kiểu tham chiếu và truyền qua tham chiếu (pass-by-reference)
- Tải chồng được định nghĩa bởi người sử dụng
- Templates hàm
- Ngoại lệ

Cũng có một số thay đổi trong lời gọi quản trị bộ nhớ (*new* và *delete* thay cho *malloc* và *free*), các thay đổi về stream và file đầu vào và đầu ra, bổ sung giá trị tham số mặc định trong khai báo hàm, và một số thay đổi nhỏ như bổ sung chú giải vào cuối dòng và loại bỏ sự cần thiết của từ khóa *typedef* với các khai báo *struct/unions/enum*.

Ba bổ sung đầu, *bool*, *pass-by-reference* và *overloading* được bàn trong phần cuối của mục này. *Templates* hàm đã xét trong chương trước.

Kiểu *bool*

Trong C, giá trị để kiểm tra logic là số nguyên. Chẳng hạn, C Reference Manual định nghĩa phép so sánh và phép toán logic *&&* như sau:

- Phép toán *<* (nhỏ hơn) trả về 1 nếu toán hạng thứ nhất nhỏ hơn toán hạng thứ hai và 0 trong trường hợp ngược lại
- Phép toán *&&* trả về 1 nếu cả hai toán hạng khác không và 0 - ngược lại.

Điều này cho phép viết lệnh của C với biểu thức như sau :

```
if ((5 < 3) + 1 && 2 == 3) { ... }
```

mà kết hợp các phép so sánh với số học.

Để phân biệt cú pháp giữa *Booleans* và *integers*, C++ có kiểu riêng: *bool* với giá trị *true* và *false*. Dựa vào chuyển đổi, điều này không hoàn toàn tách *integers* và *Booleans*. Cụ thể, giá trị nguyên có thể được gán cho biến kiểu *bool*, với chuyển đổi ẩn từ số khác 0 vào *true* và số 0 vào *false*. Tuy nhiên, kiểu *bool* làm cho dễ đọc nhiều chương trình hơn bằng việc chỉ ra rằng biến hoặc giá trị trả về của hàm sẽ được sử dụng như giá trị *Bool* thay vì như số nguyên.

Các thay đổi C++ có thể được bắt chước trong C bằng khai báo

```
typedef int bool;  
#define false 0  
#define true 1
```

mà định nghĩa kiểu *bool* và các giá trị *true*, *false*. Một sự khác biệt giữa *built-in Booleans* của C++ và *Booleans* trong C là khi *booleans* C++ được in ra, chúng in *true* và *false* chứ không phải 1 và 0.

Vì mọi chuyển đổi là ẩn, nên kiểu bool riêng biệt không giúp xử lý lỗi đơn giản thường gặp của lập trình viên trong C. Ít nhất là đối với người mới bắt đầu, lệnh điều kiện như sau:

```
if (a=b) c;
```

mà là kết quả của lỗi in ấn ; lập trình viên muốn viết

```
if (a==b) c;
```

Lý do tại sao kiểm tra kiểu câu lệnh đầu tiên và dịch trong C bỏ qua là vì phép gán số nguyên $a=b$ có kiểu nguyên và có giá trị b. Vì lệnh điều kiện C đòi hỏi số nguyên (không phải Boolean), nên không có cảnh báo gắn kết với câu lệnh này. Trong đa số các ngôn ngữ với bool khác số nguyên, có thể là lỗi nếu viết `if (a=b) c`. Tuy nhiên, vì Booleans C++ được tự động chuyển đổi thành integers, nên lệnh `if (a=b) c` vẫn là hợp lệ trong C++.

Vì việc đưa bool vào C++ có tác động rất nhỏ, bạn có thể ngạc nhiên tại sao phải làm như vậy. Trước khi kiểu Bool được bổ sung, C/C++ thường chứa các định nghĩa bool, true và false bằng macro, như mô tả trên. Tuy nhiên, bool cần được định nghĩa cách khác, với kiểu kết quả có ngữ nghĩa khác đôi chút. Chẳng hạn, bool có thể được định nghĩa như int, unsigned int, hoặc short int. Nó gây ra vấn đề khi kết hợp các thư viện mà sử dụng các định nghĩa khác nhau. Vì vậy là có ích nếu chuẩn code bằng cách bổ sung kiểu bool xây sẵn trong nó.

Reference type và Pass-By-Reference

Trong C, mọi tham số được truyền bằng giá trị. Nếu bạn muốn sửa đổi giá trị và truyền như tham số, bạn cần truyền con trỏ cho giá trị. Chẳng hạn, đây là code C cho hàm tăng số nguyên:

```
void increment(int *p) { /* parameter is pointer to int */
    (*p)++; }           /* modify value pointed to by p */
main()
{intk=0;
  increment(&k);        /* pass address of variable k */
  k;                    /* value of k is now 1 */
  ...
}
```

Trong C++, có thể dùng pass-by-reference như sau:

```

void increment(int & n) { /* parameter is reference to int */
    n++; }                /* modify reference param */
main(){
    intk=0;
    increment(k);         /* no need to explicitly compute address */
    k;                    /* value of k is now 1 */
    ...
}

```

Tác động tương tự như đối số con trỏ C, nhưng hàm gọi không cung cấp địa chỉ của đối số và hàm được gọi không có con trỏ tham chiếu. Sự trùng phạt nhẹ trong sự thuận tiện với đối số con trỏ C là lập trình viên cần phải nhớ hàm đã cho dùng pass-by-value hay pointer trong danh sách tham số. Nếu đối số con trỏ thay đổi thành pass-by-reference C++, thì không tính toán địa chỉ nào cần thiết như một phần của lời gọi và lập trình viên sử dụng hàm từ thư viện có thể tránh hoàn toàn vấn đề này.

Lợi ích của reference tường minh đôi khi gọi là pass-by-constant-reference. Nếu đối số hàm không bị thay đổi bởi hàm, thì có thể đặc tả đối số đó cần phải là hằng số, như trong void f (const int & x). Trong thân hàm với tham số x được truyền cách đó, sẽ là không hợp lệ khi gán cho x.

Tải chồng do người sử dụng định nghĩa

Như đã bàn trong chương trước, tải chồng cho phép một tên được sử dụng cho nhiều hơn một giá trị. Trong C++ có thể khai báo một số hàm với cùng một tên, trong khi các hàm có số và kiểu tham số khác nhau. Chẳng hạn, ở đây chương trình C++ với ba kiểu hàm in, mỗi cái được gọi là show:

```

#include <stdio.h>
void show(int val) {
    printf("Integer: %d\n", val);
}
void show(double val) {
    printf("Double: %lf\n", val);
}
void show(char *val) {
    printf("String: %s\n", val);
}
int main( ) {

    show(12);
    show(3.1415);
    show("Hello World\n!");
}

```

Vì ba hàm có các kiểu đối số khác nhau, chương trình dịch có thể xác định hàm nào được gọi bằng kiểu của tham số thực tế sử dụng trong lời gọi hàm.

C++ không cho phép hàm tải chồng mà có cùng số và cùng kiểu đối số nhưng khác nhau chỉ ở giá trị trả về của chúng vì các hàm C và C++ có thể được gọi như các lệnh. Khi hàm được gọi như lệnh, không để ý đến giá trị trả về của hàm, chương trình dịch không có cách nào xác định hàm nào được gọi.

Một nguồn gốc gây nhầm lẫn tiềm năng trong C++ được phát sinh từ kết hợp từ tải chồng và chuyển đổi tự động. Cụ thể, nếu các tham số thực tế trong lời gọi hàm không sánh một cách chính xác với phương án nào của hàm, chương trình dịch sẽ thử tạo ra một cách sánh nào đó bằng cách thăng cấp và/hoặc chuyển kiểu. Trong code ví dụ sau:

```
void f(int)
void f(int *)
...
f('a')
```

lời gọi `f('a')` sẽ có kết quả trong `f(int)` thay vì `f(int*)` vì `char` có thể được thăng cấp trở thành `int`. Khi hàm được tải chồng và có một số cách chuyển ẩn, rất khó cho lập trình viên có thể hiểu chuyển kiểu và lời gọi nào được sử dụng.

7.2.2 Các đặc tính hướng đối tượng

Phần quan trọng nhất của C++ là tập các khái niệm hướng đối tượng bổ sung vào C ; có các khái niệm chính sau:

- Classes, mà khai báo kiểu gắn kết với các đối tượng tạo nên từ lớp đó, các thành viên dữ liệu của mỗi đối tượng và các hàm thành viên của lớp.
- Objects, mà bao gồm dữ liệu riêng tư và các hàm công khai để truy cập dữ liệu ẩn đó, giống như trong ngôn ngữ hướng đối tượng khác.
- Dynamic lookup, đối với các hàm thành viên mà khai báo ảo. Hàm ảo trong lớp suy diễn (lớp con) có thể được cài đặt khác so với hàm ảo có cùng tên trong lớp cơ sở (superclass).
- Encapsulation, dựa trên sự chỉ định rõ của lập trình viên `public`, `private`, và `protected` mà xác định dữ liệu và các hàm khai báo trong lớp có được nhìn thấy bên ngoài định nghĩa lớp không.
- Inheritance, sử dụng subclassing: một lớp có thể được định nghĩa bằng việc kế thừa dữ liệu và các hàm khai báo trong lớp khác. C++ cho phép kế thừa đơn, ở đó một lớp có lớp cơ sở (superclass) duy nhất hoặc kế thừa lập ở đó lớp có nhiều hơn một lớp cơ sở.
- Subtyping, dựa trên subclassing: dùng cho một lớp định nghĩa kiểu con của một kiểu được định nghĩa bởi lớp khác, kế thừa cần được sử dụng. Tuy nhiên, người lập trình viên có thể quyết định kế thừa có cho kết quả kiểu con hay không.

Đây chỉ là tóm tắt ngắn; còn có nhiều tính chất khác của C++. Các mô tả tiếp theo về class, inheritance và object sẽ nêu trong mục sau.

Thuật ngữ C++. Mặc dù thuật ngữ C++ khác với thuật ngữ Java, ở đây có sự tương ứng khá gần nhau. Thuật ngữ class và object được sử dụng tương tự. Thuật ngữ subclass không được thường dùng với C++. Thay vào đó, superclass được gọi là base class và subclass gọi là derived class. Thuật ngữ inheritance có nghĩa như nhau trong cả hai ngôn ngữ.

7.2.3 Các quyết định tốt và các chỗ có vấn đề

C++ là kết quả của nỗ lực lớn gồm cả phê phán và đề xuất từ nhiều lập trình viên có kinh nghiệm. Trong nhiều khía cạnh, ngôn ngữ được thiết kế tốt nhất có thể, với mục tiêu bổ sung đối tượng và kiểm tra kiểu thời gian dịch tốt hơn cho C, không làm mất đi tính hiệu quả hoặc tương thích ngược. Một số phần thiết kế thành công riêng là

- Encapsulation: chú ý cẩn thận đến tính nhìn thấy và ẩn giấu, bao gồm các mức độ nhìn thấy public, protected, private và các hàm và lớp friend.
- Sự tách biệt giữa subtyping và inheritance: các lớp có thể có các lớp cơ sở public hoặc private, cho lập trình viên kiểm soát tường minh nào đó về phân cấp kiểu con kết quả.
- Templates
- Kiểm tra kiểu tĩnh tốt hơn C

Cũng có một số các quyết định thành công nhỏ hơn trong C++. Một ví dụ là cách mà phép toán hóa giải phạm vi (được viết như ::) được sử dụng trong kết nối với kế thừa đơn và lập để hóa giải sự nhập nhằng mà là vấn đề trong các ngôn ngữ khác.

Một số chỗ có vấn đề

Ở đây có một số khía cạnh của C++ mà lập trình viên đôi khi thấy khó khăn. Một số chỗ chính có vấn đề là

- Ép kiểu và chuyển kiểu, mà có thể phức tạp và khó lường trước trong một số tình huống
- Các đối tượng gắn kết với ngăn xếp và các khía cạnh khác của việc quản lý bộ nhớ đối tượng
- Tải chồng, cơ chế chọn mã phức tạp trong C++ mà có thể tương tác khó đoán với tìm kiếm động (tìm kiếm hàm ảo)
- Kế thừa bội, mà là phức tạp hơn trong C++ so với trong ngôn ngữ khác vì cách các đối tượng và các bảng hàm ảo được cấu hình và truy cập.

Các chỗ có vấn đề còn tồn tại không phải vì quên sót mà vì các mục tiêu của C++, theo các kết luận logic của chúng, dẫn đến thiết kế với các tính chất đó. Nói cách khác, các vấn đề này không phải là kết quả của sự không cẩn thận hoặc thiếu chú ý, mà là hệ quả của các quyết định được đưa ra với các suy nghĩ khách quan khác. Công bằng mà nói hầu hết chúng đều có nguồn gốc trong C, chứ không phải phần mở rộng C++ cho C. Các đặc tính này có thể là nguyên nhân làm cho các lập trình viên ưu thích các ngôn ngữ khác khi so sánh với C và tính hiệu quả tuyệt đối của mã dịch không là bản chất cho ứng dụng.

Một số lập trình viên có thể nói rằng không có gom rác, hoặc không có giao diện chuẩn để viết chương trình song song là vấn đề trong C++. Tuy nhiên có các công cụ mà đơn giản nằm ngoài phạm vi của thiết kế ngôn ngữ.

Ép kiểu và chuyển kiểu. Ép kiểu hướng chương trình dịch xử lý biểu thức của một kiểu như nó là biểu thức của kiểu khác. Chẳng hạn, *(float) i* chỉ dẫn cho chương trình dịch xử lý biến *i* như là float, cho dù kiểu của nó là khác và *(int*)x* buộc *x* được xử lý như con trỏ đến số nguyên. Trong một số tình huống, chương trình dịch C và C++ thực hiện chuyển kiểu ẩn. Chẳng hạn, khi biến được gán giá trị của biểu thức, biểu thức có thể được chuyển đến kiểu gắn kết với biến, nếu cần thiết. Trong hầu hết các ngôn ngữ hướng đối tượng, chuyển kiểu tự động cho đối tượng từ kiểu này sang kiểu khác không làm thay đổi biểu diễn của đối tượng. Chẳng hạn, trong Java đối tượng Point có thể được xử lý như Colored Point mà không thay đổi biểu diễn của đối tượng Point, vì Colored Point cũng được biểu diễn theo cách mà tương thích với biểu diễn của đối tượng Point. Tuy nhiên, nếu kế thừa lặp được sử dụng trong C++, chuyển một đối tượng từ subtype sang supertype có thể đòi hỏi sự thay đổi nào đó trong giá trị của con trỏ đến đối tượng. Điều này xảy ra có thể dẫn tới lỗi rất khó tìm và buộc lập trình viên phải hiểu biểu diễn bên trong của đối tượng. Khái quát hơn, Stroustrup tự nhủ rằng ‘Về cú pháp và ngữ nghĩa, ép kiểu là một đặc tính kỳ dị nhất của C và C++’ trong cuốn The Design and Evolution of C++ (Addison-Wesley, 1994).

Các đối tượng trên ngăn xếp. Simula, Smalltalk, Java cho phép các đối tượng được tạo ra chỉ trên heap, không phải trên ngăn xếp thời gian chạy. Trong ngôn ngữ khác này, các đối tượng được truy cập thông qua con trỏ, chứ không phải qua các biến ngăn xếp thông thường mà chứa không gian cho đối tượng có kích thước nào đó. Trong khi đó các đối tượng trên ngăn xếp của C++ được cấp và giải phóng một cách hiệu quả như một phần của việc nhập và xuất của các khối cục bộ, ở đây cũng có một số nhược điểm. Điểm thô hiển nhiên nhất là cách mà phép gán thực hiện kết hợp với subtyping, cắt (truncating) đối tượng đến kích thước vừa khi phép gán được thực hiện. Điều này có thể thay đổi hành vi của đối tượng, như việc loại bỏ một số thành viên dữ liệu buộc chương trình dịch thay đổi cách các hàm ảo được lựa chọn. Mặc dù định nghĩa ngôn ngữ C++ giải thích cái gì xảy ra và tại sao, hành vi của phép gán đối tượng có thể làm mất phương hướng lập trình viên.

Overloading. Tải chồng bản thân nó không phải đặc tính của ngôn ngữ lập trình tồi, nhưng tải chồng do người sử dụng định nghĩa của C++ có thể phức tạp. Thêm vào đó, sự tương tác với tìm kiếm động (các hàm ảo) có thể không đoán trước được. Vì tải chồng là cơ chế lựa chọn code thời gian dịch và tìm kiếm động là cơ chế lựa chọn code thời gian chạy, hai cơ chế có hành vi rất khác nhau. Nó gây ra nhầm lẫn cho nhiều lập trình viên.

Kế thừa bội. Đây là một số phức tạp cố hữu gắn kết với kế thừa bội mà khó tránh khỏi. Thiết kế ngôn ngữ C++ dàn xếp các vấn đề này với đối tượng phức tạp và định dạng bảng tìm kiếm phương thức. Không may, các chi tiết cài đặt kế thừa bội có vẻ được đưa vào câu trả dựa trên lập trình thông thường, đôi khi buộc lập trình viên người quan tâm đến sử dụng bội phải tìm hiểu một số chi tiết cài đặt. Ngay cả lập trình viên không sử dụng thừa kế bội có thể bị ảnh hưởng nếu lớp derived class được định nghĩa sử dụng kế thừa bội.

7.3 Các lớp, kế thừa và các hàm ảo

Các đặc tính hướng đối tượng chính của C++ qua ví dụ lớp Point và Colored Point.

7.3.1 Các lớp C++ và các đối tượng

Chúng ta đối tượng điểm trong C++ bằng việc khai báo lớp Pt, và khai báo riêng các hàm thành viên của lớp Pt. Khai báo lớp không có cài đặt định nghĩa cả giao diện của đối tượng điểm và dữ liệu dùng để cài đặt điểm, nhưng không cần chứa code cho các hàm thành viên. Khai báo xuất hiện trong lớp Pt được chia thành ba phần, public members, protected members và privated members của lớp.

Đơn giản, Pt là các điểm một chiều, mỗi điểm chỉ có một tọa độ x:

```
/* ----- Point Class Interface -----*/
class Pt {
public:
    Pt(int xv);
    Pt(Pt* pv);
    int getX();
    virtual void move(int dx);
protected:
    void setX(int xv);
private:
    int x;
};

/* ----- Declarations of Constructors -----*/
Pt::Pt(int xv) { x = xv; }
Pt::Pt(Pt* pv) { x = pv->x; }

/* ----- Declarations of Member Functions -----*/
int Pt::getX() { return x; }
void Pt::setX(int xv) { x = xv; }
```

Constructors. Hàm tạo được dùng để khởi tạo dữ liệu thành viên của đối tượng khi chương trình chứa lệnh hoặc biểu thức chỉ ra rằng đối tượng mới được tạo. Khi điểm mới được tạo, bộ nhớ được cấp, hoặc trên heap hoặc trong bản ghi kích hoạt của ngăn xếp, tùy thuộc vào lệnh tạo đối tượng. Do đó hàm tạo được gọi là khởi tạo vị trí cấp cho đối tượng. Hàm tạo được khai báo với cùng cú pháp như của hàm thành viên, loại trừ tên hàm trùng với tên của lớp. Trong lớp *Point*, hai hàm tạo được khai báo. Kết quả là hàm tải chồng *Pt*, với một hàm tạo được gọi nếu đối số là integer và hàm khác được gọi nếu đối số là *Pt*.

Visibility. Như đã nói, các khai báo xuất hiện trong lớp *Pt* được chia thành ba phần: thành viên *public*, thành viên *protected* và thành viên *private* của lớp. Các chỉ định rõ này, mà tác động đến tính nhìn thấy của khai báo, có thể được tóm tắt như sau:

- *public*: các thành viên mà nhìn thấy trong mọi phạm vi ở đó đối tượng của lớp được tạo và được truy cập.
- *protected*: các thành viên mà được nhìn thấy trong lớp và trong mọi lớp suy diễn
- *private*: các thành viên chỉ được nhìn thấy trong lớp ở đó các thành viên này được tạo.

Lớp *Pt* được viết theo một qui ước nhìn thấy chuẩn mà được sử dụng bởi nhiều lập trình viên C++. Dữ liệu thành viên là *private*, sao cho nếu lập trình viên muốn thay đổi cách biểu diễn các đối tượng của lớp, thì điều này có thể được thực hiện mà không làm ảnh hưởng đến các lớp khác (bao gồm cả *derived classes*) phụ thuộc vào lớp này. Các thành viên thay đổi dữ liệu *private* này được tạo ra là *protected*, sao cho các lớp suy diễn có thể thay đổi giá trị của dữ liệu thành viên, nhưng code bên ngoài thì không cho phép làm điều đó. Cuối cùng, các hàm thành viên, mà đọc giá trị của dữ liệu thành viên và cung cấp các thao tác có ích trên đối tượng, được khai báo *public*, sao cho bất cứ code nào với truy cập đến đối tượng của lớp này có thể thao tác các đối tượng này theo nhiều cách có ích.

Một đặc tính của C++ không được thể hiện trong ví dụ này là chỉ định *friend*, mà được sử dụng cho phép tính nhìn thấy có hiệu lực cho các phần *private* của lớp. Một lớp có thể khai báo các hàm *friend* và các lớp *friend*. Nếu lớp *Pt* chứa khai báo *friend class A*, thì code được viết như một phần của lớp *A* (như các hàm thành viên của lớp *A*) có thể được truy cập đến các phần *private* của lớp *Pt*. Cơ chế *friend* được sử dụng khi một cặp lớp có quan hệ chặt chẽ, như ma trận và vector.

Các hàm ảo. Các hàm thành viên có thể được chỉ định *virtual* hoặc còn lại là *non-virtual*, mà là mặc định cho các hàm thành viên mà không có từ khóa *virtual* đứng trước. Nếu phương thức là *virtual*, nó có thể được định nghĩa lại trong lớp suy diễn. Vì các đối tượng khác nhau có các hàm thành viên khác nhau có cùng tên, việc chọn các hàm thành viên ảo được tiến hành bởi tìm kiếm động: có cơ chế chọn code thời gian

chạy mà được sử dụng để tìm và triệu hồi hàm đúng. Các bước thêm này gọi các hàm ảo kém hiệu quả hơn hàm không ảo.

Các phương thức không ảo không thể định nghĩa lại trong các lớp suy diễn. Như kết quả, lời gọi đến các phương thức không ảo, được dịch và được thực hiện theo cùng một cách như lời gọi các hàm thông thường không gắn kết với đối tượng hoặc lớp.

Điểm chung hay nhầm lẫn là, về mặt cú pháp, khai báo lại của một hàm không ảo có thể xuất hiện trong lớp suy diễn. Tuy nhiên, nó tạo ra hàm tải chồng, với code được chọn trong thời gian dịch. Chúng ta sẽ bàn thêm về việc này trong mục sau nữa.

7.3.2 Các lớp suy diễn C++ (kế thừa)

Lớp ColorPt sau định nghĩa sự mở rộng của các điểm một chiều trong lớp Pt. Như tên nó đề xuất, các đối tượng ColorPt có màu bổ sung. Để đơn giản, chúng ta giả thiết các màu được biểu diễn bằng các số nguyên. Thêm vào đó, để thể hiện định nghĩa hàm ảo, dịch chuyển điểm màu sẽ làm cho màu tối đi một chút:

```
/* ----- Color Point Class Interface ----- */
class ColorPt: public Pt {
public:
    ColorPt(int xv,int cv);
    ColorPt(Pt* pv,int cv);
    ColorPt(ColorPt* cp);
    int getColor();
    virtual void darken(int tint);
    virtual void move(int dx);
protected:
    void setColor(int cv);
private:
    int color;
};

/* ----- Declarations of Constructors ----- */
ColorPt::ColorPt(int xv,int cv)
    : Pt(xv) /* call parent's constructor */
    { color = cv; } /* then initialize color */
ColorPt::ColorPt(Pt* pv,int cv): Pt(pv) { color = cv; }
/* ----- Declarations of Member Functions ----- */
int ColorPt::getColor() { return color; }
void ColorPt::darken(int tint) { color += tint; }
void ColorPt::move(int dx) {Pt::Move(dx); this->darken(1); }
void ColorPt::setColor(int cv) { color=cv; }
```

Inheritance. Dòng đầu khai báo là lớp ColorPt có lớp Pt là lớp cơ sở công khai; điều này là ý nghĩa của mệnh đề: public Pt đứng sau tên của lớp ColorPt. Nếu từ khóa *public* được bỏ qua, thì lớp cơ sở được gọi là private base class.

Khi một lớp có lớp cơ sở tiếp, thì lớp đó kế thừa tất cả các thành viên của lớp cơ sở đó. Điều đó có nghĩa là các đối tượng ColorPt có mọi thành viên public, protected và private của lớp Pt. Cụ thể, mặc dù lớp ColorPt khai báo chỉ một dữ liệu thành viên là color, các đối tượng ColorPt cũng có dữ liệu thành viên x, kế thừa từ Pt.

Sự khác biệt giữa các lớp cơ sở public và các lớp cơ sở private là khi lớp cơ sở là public, thì lớp khai báo (suy diễn) được khai báo là kiểu con của lớp cơ sở đó. Nếu không, C++ sẽ không xử lý lớp đó như kiểu con, ngay cả nó có đủ mọi thành viên của lớp cơ sở này. Nó sẽ được bàn cụ thể trong mục 7.4.

Constructors. Lớp ColorPt có ba hàm tạo. Như trong lớp Pt, kết quả là hàm tải chồng ColorPt với sự lựa chọn giữa ba hàm được thực hiện trong thời gian dịch tùy thuộc vào kiểu của các đối số đối với hàm tạo.

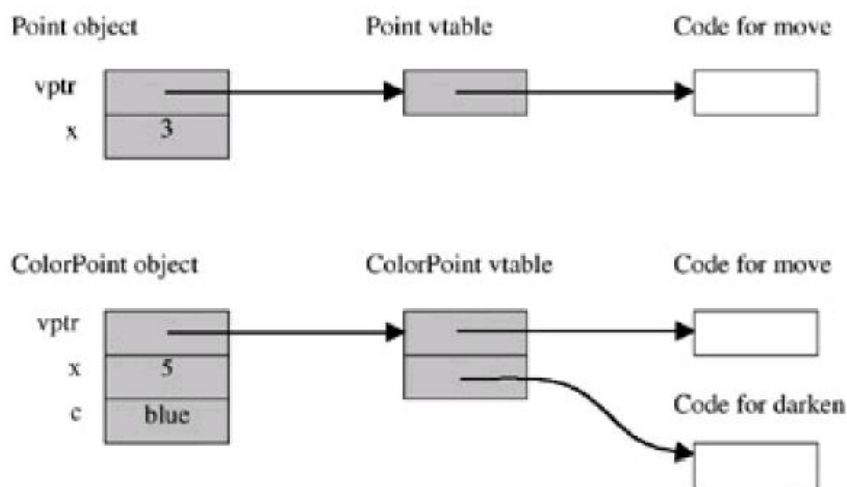
Thân của hai hàm tạo như đã nêu trước đây mô tả hàm tạo của lớp suy diễn có thể gọi hàm tạo của lớp cơ sở như thế nào. Như với Points, khi colored point mới được tạo, không gian được cấp, hoặc trên heap hoặc trên trong bản ghi kích hoạt trên ngăn xếp, phụ thuộc lệnh tạo đối tượng, và hàm tạo được gọi để khởi tạo vị trí cấp cho đối tượng đó. Vì lớp suy diễn có mọi thành viên dữ liệu của lớp cơ sở, hầu hết các hàm tạo của lớp suy diễn sẽ gọi hàm tạo của lớp cơ sở để khởi tạo các thành viên dữ liệu kế thừa. Nếu lớp cơ sở có các thành viên dữ liệu private, mà trong trường hợp này là của Pt, thì chỉ có một cách để lớp suy diễn ColorPt khởi tạo các thành viên private là gọi hàm tạo của lớp cơ sở.

Visibility. Khi một lớp kế thừa từ một lớp khác, các thành viên về bản chất có cùng tầm nhìn trong lớp suy diễn như trong lớp cơ sở. Cụ thể hơn, các thành viên public trong lớp cơ sở trở thành thành viên public trong lớp suy diễn, các thành viên protected trong lớp cơ sở truy cập được trong lớp suy diễn và các lớp suy diễn của nó, mà là cùng tầm nhìn như nếu thành viên này được khai báo là protected trong lớp suy diễn. Các thành viên private kế thừa tồn tại trong lớp suy diễn, nhưng không được gọi tên trực tiếp trong code được viết như một phần của lớp suy diễn. Chẳng hạn, mỗi đối tượng ColorPt có thành viên số nguyên x, nhưng chỉ có một cách gán hoặc đọc giá trị của thành viên này là gọi các hàm public và protected từ lớp Pt.

Virtual functions. Như đã nói từ trước, các hàm ảo trong lớp cơ sở có thể được định nghĩa lại trong lớp suy diễn. Hàm thành viên move được khai báo ảo virtual trong lớp Pt và được định nghĩa lại trong lớp ColorPt. Trong ví dụ này, hàm move cho các điểm chỉ thay đổi tọa độ x của điểm, trong khi đó hàm move trên các điểm màu thay đổi tọa độ x và màu. Nếu cài đặt của ColorPt::move được bỏ qua, thì hàm move từ Pt sẽ trở thành kế thừa cho ColorPt. Cài đặt các hàm ảo được bàn trong mục sau.

7.3.3 Các hàm ảo

Tìm kiếm động được sử dụng cho các hàm ảo. Hàm ảo f được định nghĩa trên đối tượng o được gọi bởi cú pháp $o.f(\dots)$ hoặc $p \rightarrow f(\dots)$ nếu p là con trỏ đến đối tượng o . Khi hàm ảo được gọi, mã của hàm này được xác định vị trí bởi dãy các bước thời gian chạy. Các bước này tương tự như thuật toán tìm kiếm của Smalltalk, nhưng đơn giản hơn, vì một số tối ưu đã được thực hiện bởi hệ thống kiểu tĩnh của C++. Mỗi đối tượng có một con trỏ đến cấu trúc dữ liệu gắn kết với lớp của nó, gọi là bảng hàm ảo hoặc viết tắt là vtable. Quan hệ giữa đối tượng, vtable lớp của nó và code của các hàm ảo được nêu trong Hình 7.1 với các điểm và các điểm màu trong C++.



Hình 7.1 Biểu diễn các điểm và các điểm màu trong C++

Hàm ảo trên lớp cơ sở. Giả sử rằng p là con trỏ đến đối tượng Pt trên Hình 7.1. Khi biểu thức dạng $p \rightarrow \text{move}(\dots)$ được tính toán, thì code của `move` được tìm và thực thi. Quá trình được bắt đầu bằng việc theo con trỏ `vtable` trong p để đến bảng dành cho lớp Pt . `Vtable` đối với Pt là một mảng của các con trỏ tới các hàm. Vì `move` là hàm ảo đầu tiên (và chỉ có nó) của lớp Pt , chương trình dịch có thể xác định được là vị trí đầu tiên trong mảng này là con trỏ của `move`. Do đó, mã thời gian chạy tìm cho `move` được lần theo con trỏ đầu tiên trong bảng `vtable` của Pt và gọi hàm đạt được nhờ con trỏ này. Không giống như Smalltalk, ở đây không cần tìm `vtable` trong thời gian chạy để xác định con trỏ nào là dành cho `move`. Hệ thống kiểu C++ cho phép chương trình dịch xác định kiểu của con trỏ đối tượng trong thời gian dịch và điều này cho phép chương trình dịch tìm vị trí tương đối của con trỏ hàm ảo trong `vtable` trong thời gian dịch, loại bỏ sự cần thiết phải tìm kiếm thời gian chạy trong `vtable`.

Hàm ảo trong lớp suy diễn. Giả sử cp là con trỏ đến đối tượng điểm màu trong Hình 7.1. Khi biểu thức dạng $cp \rightarrow \text{move}(\dots)$ được tính toán, thì thuật toán tìm code cho `move` cũng chính xác như đối với $p \rightarrow \text{move}(\dots)$: bảng `Vtable` đối với `ColorPt` là một mảng chứa hai con trỏ, một cho `move` và cái kia cho `darken`. Chương trình dịch có thể xác định trong thời gian dịch là cp là con trỏ tới đối tượng `ColorPt` và `move` này là hàm

ảo đầu tiên của lớp, vì vậy thuật toán thời gian chạy lần theo con trỏ đầu tiên trong vtable mà không cần tìm kiếm thời gian chạy như kiểu Smalltalk trong vtable.

Tương ứng giữa vtables của lớp cơ sở và lớp suy diễn. Như hệ quả của subtyping, chương trình có thể gán điểm màu cho con trỏ đến điểm và gọi move qua con trỏ lớp cơ sở như trong

```
p = cp;  
p-> move(...);
```

theo đó sẽ xác định tiếp các đối tượng và cài đặt chúng. Quan hệ giữa các đối tượng ở đây có thể là quan hệ giữa các giao diện của chúng hoặc quan hệ giữa các cài đặt của chúng. Ngôn ngữ lập trình hướng đối tượng hiện đại cung cấp cơ chế để sử dụng quan hệ giữa các giao diện và quan hệ giữa các cài đặt trong quá trình thiết kế và cài đặt đó. Các cấu trúc dữ liệu được sử dụng trong các ví dụ trước đây của lập trình top-down là rất đơn giản và tồn tại bất biến trong quá trình làm mịn của chương trình. Khi việc làm mịn bao gồm thay thế một thủ tục bằng một thủ tục chi tiết hơn, thì các ngôn ngữ lập trình cấu trúc như Algol, Pascal và C là đáp ứng được. Tuy nhiên, đối với các bài toán phức tạp hơn, thì cả thủ tục và cả cấu trúc dữ liệu của chương trình đều cần được làm mịn cùng nhau. Vì các đối tượng là sự kết hợp của hàm và dữ liệu, các ngôn ngữ lập trình hướng đối tượng hỗ trợ làm mịn kết hợp của hàm và dữ liệu hiệu quả hơn so với các ngôn ngữ hướng thủ tục.

BÀI 8: TÍNH CHUYÊN MẠNG VÀ AN TOÀN CỦA JAVA

Ngôn ngữ lập trình Java được thiết kế bởi James Gosling và những người khác tại Sun Microsystems. Ngôn ngữ, được phát sinh từ dự án bắt đầu từ 1990, được gọi là Oak và được dành cho việc sử dụng trong các thiết bị được gọi là set-top-box. Set-top-box được chủ định là thiết bị tính toán nhỏ, gắn với mạng dạng nào đó và đặt bên trên tivi. Có nhiều chức năng khác nhau mà set-top-box có thể cung cấp. Bạn có thể tưởng tượng ra là trình duyệt Web được hiển thị trên tivi và thay vì bàn phím bạn có thể sử dụng các nút ấn trên phím điều khiển từ xa. Bạn có thể chọn chương trình tivi hoặc phim hoặc download mô phỏng máy tính nhỏ mà có thể thực hiện trên thiết bị tính toán và hiển thị trên màn hình của bạn. Quảng cáo oto có thể cho phép bạn tải về một chuyến du lịch tương tác ảo của oto cho mỗi người xem cảm giác mô phỏng riêng của thể lái. Dù kịch bản có thể xuất hiện như thế nào, môi trường tính toán có thể bao gồm đồ họa, thực thi chương trình đơn giản và trao đổi thông tin giữa site từ xa và chương trình thực thi tại chỗ.

Trong quá trình phát triển Oak các kỹ sư và quản trị tại Sun Microsystems nhận thấy rằng cần một ngôn ngữ lập trình cho trình duyệt trên Internet, mà có thể được sử dụng để viết các ứng dụng nhỏ có thể truyền trên mạng và thực thi dưới sự kiểm soát của trình duyệt web chuẩn nào đó trên nền tảng chuẩn bất kỳ.

Gosling đi đến kết luận: các ngôn ngữ hiện tại không thể hỗ trợ dự án. C++ trở nên gần chuẩn đối với các lập trình viên xây dựng các ứng dụng mà ở đó tốc độ đóng vai trò quan trọng. Nhưng C++ không đủ tin cậy. Nó nhanh, nhưng giao diện của nó không tương thích, chương trình hay bị ngắt. Tuy nhiên, trên thiết bị dân dụng, tính tin cậy quan trọng hơn nhiều tốc độ. Gosling nhận định, cần có ngôn ngữ lập trình mới.

Với nhiều lý do khác nhau, kể cả các nỗ lực quảng cáo của Sun Microsystem, Java trở nên cực kỳ thành công sau một thời gian ngắn nó ra đời như một ngôn ngữ truyền thông Internet vào giữa 1995.

Các phần chính của hệ thống Java bao gồm:

- Ngôn ngữ lập trình Java
- Chương trình dịch Java và các hệ thống thời gian chạy của Java (máy ảo Java)
- Thư viện mở rộng, bao gồm công cụ gia diện đồ họa và các ứng dụng khác của Java như Java applets.

8.1 Tổng quan ngôn ngữ Java

8.1.1 Các mục tiêu của Java

Ngôn ngữ lập trình Java và môi trường thực thi được thiết kế với các mục tiêu sau :

- Portability – Tính chuyển mang: nó cần dễ dàng truyền chương trình qua mạng và nhận được chúng chạy đúng đắn trên môi trường nhận, không phụ thuộc vào phần cứng, hệ điều hành và trình duyệt Web được sử dụng.
- Reliability – Tính tin cậy: vì chương trình sẽ được chạy từ xa bởi người sử dụng, người đó không viết code, các thông báo lỗi và sự cố chương trình cần đảm bảo loại bỏ tối đa càng tốt.
- Safety – Tính an toàn: môi trường tính toán nhận được chương trình cần được bảo vệ khỏi các lỗi của người lập trình và các chương trình có hại.
- Dynamic link – Liên kết động: chương trình được phân tán thành các phần, với các phần riêng biệt được tải vào môi trường thời gian chạy của Java tùy theo yêu cầu.
- Multithreaded Execution: để các chương trình song song chạy trên các phần cứng và hệ điều hành khác nhau, ngôn ngữ lập trình cần gồm phần hỗ trợ tương minh và giao diện chuẩn cho lập trình song song.
- Simplicity and Familiarity – Tính đơn giản và quen thuộc: ngôn ngữ có cảm giác đối với người thiết kế website của bạn cũng bình thường như với người lập trình C hoặc với người khá quen C/C++.
- Efficiency – Tính hiệu quả: điều này quan trọng nhưng là thứ yếu so với các yêu cầu khác.

8.1.2 Các quyết định thiết kế

Một số mục tiêu thiết kế và các quyết định thiết kế được liệt kê trong Bảng 8.1. Ở đó dấu + chỉ ra là quyết định có hỗ trợ mục tiêu này và +/- chỉ ra rằng ở đây có ưu điểm và nhược điểm của quyết định. Một số ô vuông để trống, chỉ ra là quyết định thiết kế có ít hoặc không có tác động đến mục tiêu đó. Chúng ta có thể thấy tính quan trọng tương đối của tính hiệu quả trong quá trình thiết kế Java bằng cách xem cột bên phải nhất. Nó không có nghĩa là tính hiệu quả không được ưu tiên mà chỉ là tương đối so với các mục tiêu khác, tính hiệu quả không phải là mục tiêu quan trọng.

Hỗ trợ song song. Java có mô hình song song dựa trên các luồng, mà là các tiến trình song song độc lập. Đây là phần quan trọng của ngôn ngữ, cả hai vì thiết kế có ý nghĩa

lớn và vì sự quan trọng có các yếu tố song song chuẩn như một thành phần của ngôn ngữ Java. Rõ ràng, nếu chương trình Java dựa trên hệ điều hành dành cho các cơ chế song song, thì chương trình sẽ không thể được chuyển mang trên các nền tảng hệ điều hành khác nhau.

Tính đơn giản. Mặc dù Java được phát triển theo từng năm, các đặc trưng như tính phân xạ và lớp trong không cho cảm giác đơn giản, ngôn ngữ vẫn đủ nhỏ và đơn giản trong thiết kế so với hầu hết các ngôn ngữ lập trình đa dụng chất lượng cao khác. Một cách đơn giản để nhìn thấy sự đơn giản tương đối của Java là liệt kê các đặc trưng của C++ mà không xuất hiện trong Java. Nó bao gồm các đặc trưng sau:

- Structures and unions: các cấu trúc mà được gom lại bởi các đối tượng mà một số sử dụng tập hợp có thể được thay thế bởi các lớp được chia sẻ bởi lớp cha chung.
- Các hàm được thay thế bởi các phương thức tĩnh
- Kế thừa bội là phức tạp và hầu hết các trường hợp có thể loại bỏ nếu khái niệm giao diện đơn giản hơn được sử dụng
- Lệnh Goto là không cần thiết
- Tải đè toán tử là phức tạp và cho rằng là không cần thiết; các hàm Java có thể tải đè.
- Chuyển kiểu tự động là phức tạp và cho rằng là không cần thiết
- Con trỏ là mặc định cho đối tượng và không cần thiết cho các kiểu khác. Kéo theo kiểu con trỏ riêng biệt là không cần thiết.

Một số đặc tính này xuất hiện trong C++ như là các tính chất cơ bản, vì mục tiêu thiết kế của C++ là tương thích ngược với C. Các đặc tính khác được bỏ qua bởi Java sau khi bàn luận, vì đã được quyết định là sự phức tạp nếu đảm bảo nó sẽ đáng kể hơn nhiều so với tính năng mà nó đem lại. Việc bỏ qua quan trọng nhất là kế thừa bội, chuyển kiểu tự động, tải đè toán tử và phép toán con trỏ trong một số dạng có trong C và C++.

8.2 Các lớp Java và kế thừa

8.2.1 Các lớp và các đối tượng

Java được viết trong cú pháp giống C++ sao cho lập trình dễ dàng hơn đối với các lập trình viên C và C++. Điều này làm cho lớp các điểm một chiều được sử dụng trong chương trước trông giống khi được chuyển sang Java. Đây là phiên bản viết rút gọn của lớp đó bỏ qua chi tiết các phương thức:

```

class Point {
    public int getX() { ... }
    protected void setX (int x){ ... }
    private int x;
    Point(int xval) {x = xval;}
};

```

Giống như các ngôn ngữ dựa trên lớp khác, các lớp Java khai báo dữ liệu và các chức năng gắn kết với mọi đối tượng được tạo bởi lớp này. Khi các đối tượng Java được tạo, không gian được cấp để lưu giữ các trường dữ liệu của đối tượng và hàm tạo của lớp được gọi để khởi tạo các trường dữ liệu đó. Như trong C++, hàm tạo có cùng tên như lớp. Cũng như C++, lớp Point có các thành phần public, private và protected. Mặc dù public, private, protected là các từ khóa của Java, các đặc tả về tính nhìn thấy này không có nghĩa chính xác như nhau trong hai ngôn ngữ, như sẽ giải thích trong mục 8.2.2.

Thuật ngữ của Java khác một chút so với C++. Sau đây là tóm tắt ngắn gọn các thuật ngữ quan trọng khi nói về Java:

- Class và object có nghĩa như nhau như trong mọi ngôn ngữ hướng đối tượng dựa trên lớp khác, field là thành phần dữ liệu (data member).
- Method là hàm thành phần (member function); this: giống như this của C++, định danh this trong thân của của phương thức Java tham chiếu đến đối tượng mà ở đó phương thức này được triệu gọi.
- Native method: phương thức được viết trong ngôn ngữ khác chẳng hạn như C.
- Package: tập các lớp trong không gian tên chung.
- Chúng ta sẽ xét một vài đặc trưng của lớp và đối tượng trong Java, bao gồm cả các trường và phương thức tĩnh, tải đè, phương thức finalize, phương thức main, phương thức toString được sử dụng để biểu diễn in đối tượng và khả năng định nghĩa các phương thức native. Chúng ta sẽ bàn về biểu diễn các đối tượng trong thời gian chạy của Java và cài đặt phương thức lookup trong 8.3 trong kết nối với các khía cạnh khác của kiến trúc thời gian chạy.

Initialization. Java đảm bảo rằng hàm tạo sẽ được gọi bất cứ lúc nào đối tượng được tạo. Vì có một số vấn đề thú vị liên quan đến kế thừa, nên điều này sẽ được xét trong mục 8.2.3.

Các phương thức và các trường tĩnh. Nếu một trường được khai báo là static, thì đó là một trường của toàn bộ lớp, thay vì của mỗi một đối tượng. Nếu phương thức được khai báo là static, phương thức đó có thể được gọi mà không sử dụng đối tượng của lớp. Cụ thể, phương thức static có thể được gọi trước khi có bất cứ đối tượng nào của lớp được tạo. Các phương thức static chỉ có thể truy cập đến các trường tĩnh và các phương thức tĩnh khác của lớp. Chúng không thể tham chiếu đến this vì chúng không phải là một phần của một đối tượng cụ thể nào của lớp. Bên ngoài lớp, một thành phần tĩnh thường được truy cập với tên của lớp, như `class_name_static_method(ags)`, thay vì thông qua tham chiếu đối tượng.

Các trường tĩnh có thể được khởi tạo với các biểu thức initialization hoặc static initialization block. Cả hai được thể hiện trong đoạn code sau:

```
class ... {  
    /* --- static variable with initial value --- */  
    static int x = initial_value;  
    /* --- static initialization block --- */  
    static { /* code to be executed once, when class is loaded */  
    }  
}
```

Như được chỉ ra trong phần ghi chú của chương trình, khối khởi tạo tĩnh của Java được thực hiện một lần, khi lớp được tải. Tải lớp sẽ được bàn đến trong 8.3 trong liên kết với máy ảo Java JVM. Ở đây có các qui tắc riêng điều khiển thứ tự khởi tạo tĩnh, khi lớp chứa cả hai biểu thức khởi tạo và khối khởi tạo tĩnh. Cũng có những hạn chế về dạng của các khối khởi tạo tĩnh. Chẳng hạn, khối tĩnh không được xử lý ngoại lệ, vì nó không chắc chắn là phần điều khiển tương ứng được khởi tạo trong thời gian tải lớp.

Overloading – tải đè. Tải đè Java được dựa trên dấu hiệu của phương thức, mà bao gồm tên phương thức, số tham số và kiểu của từng tham số. Nếu hai phương thức của lớp của một lớp (khi cả hai được khai báo trong cùng một lớp hoặc cả hai được kế thừa hoặc một khai báo, một kế thừa), có cùng tên nhưng có dấu hiệu khác nhau, thì tên phương thức được tải đè. Như trong các ngôn ngữ khác, tải đè được hóa giải trong thời gian dịch.

Thu gom rác và finalize. Vì Java là ngôn ngữ tự gom rác, nó không cần thiết phải giải phóng tường minh các đối tượng. Thêm vào đó, các lập trình viên không cần quan tâm về tham chiếu treo được tạo khi giải phóng các đối tượng. Tuy nhiên, việc thu gom rác thu hồi không gian được sử dụng bởi đối tượng. Nếu đối tượng giữ truy cập đến một nguồn tài nguyên nào đó, thì nó sẽ khóa tài nguyên chia sẻ đó, do đó nó cần giải phóng khi đối tượng không còn được truy cập nữa. Với lý do này, các đối tượng Java có thể có phương thức finalize, mà được gọi dưới hai điều kiện: bởi garbage collector ngay trước khi không gian được thu hồi và bởi máy ảo khi máy ảo tồn tại. Qui ước có ích này trên phương thức finalize được gọi là super.finalize, như mô tả sau đây, sao cho mọi code kết thúc mà gắn kết với superclass cũng được thực hiện:

```
class...{  
    ...  
    protected void finalize () {  
        super.finalize();  
        close(file);  
    }  
};
```

Ở đây có tương tác thú vị giữa các phương thức finalize và cơ chế ngoại lệ của Java. Mọi ngoại lệ không nắm bắt được xảy ra khi phương thức finalize thực hiện đều được bỏ qua.

Vấn đề lập trình gắn kết với phương thức finalize là lập trình viên không có kiểm soát tường minh về việc khi nào phương thức finalize được gọi. Quyết định này dành cho hệ thống thời gian chạy. Nó có thể tạo ra vấn đề nếu đối tượng giữa khóa trên các đối tượng chia sẻ, chẳng hạn như khóa có thể không được giải phóng cho đến khi thu gom rác xác định chương trình cần thêm không gian. Một giải pháp là đặt các thao tác như giải phóng mọi khóa cũng như các tài nguyên khác trong phương thức mà được gọi tường minh trong chương trình. Điều này làm việc tốt, cho đến khi mọi người sử dụng của lớp biết tên của phương thức đó và nhớ gọi phương thức khi đối tượng không còn cần nữa.

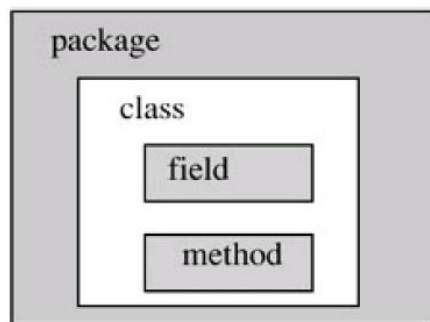
Một số khía cạnh khác thú vị của các đối tượng và các lớp Java là phương thức main được sử dụng để bắt đầu chương trình, phương thức ToString được sử dụng để tạo bản in biểu diễn của đối tượng và khả năng định nghĩa các phương thức native:

- **main:** Ứng dụng Java được triệu gọi với tên này của lớp mà khởi động ứng dụng. Lớp này cần có phương thức main, mà cần phải là public, static, cần trả về void và cần nhận đối số duy nhất kiểu string[]. Phương thức main được gọi với các đối số của chương trình dạng mảng xâu. Lớp bất kỳ với phương thức main có thể được triệu gọi trực tiếp nếu đây là chương trình ứng dụng stand-alone mà có ích khi dùng để kiểm chứng.
- **toString:** lớp bất kỳ có thể định nghĩa phương thức toString, mà được gọi khi việc chuyển sang dạng xâu là cần thiết, như in đối tượng.
- **native methods:** phương thức native là phương thức được viết trong ngôn ngữ khác, chẳng hạn như C. Tính chuyển mang và an toàn được qui về với các phương thức native: native code không thể được chuyển trên mạng theo yêu cầu, và điều khiển tương tác với JVM là không có hiệu quả vì phương thức không được dịch bởi máy ảo. Lý do sử dụng phương thức native là: tính hiệu quả của mã đối tượng native và việc truy cập đến các công cụ hoặc chương trình mà đã được viết trong các ngôn ngữ khác.

8.2.2 Các gói và tính nhìn thấy

Java có bốn sự khác nhau về tính nhìn thấy đối với các trường và các phương thức, ba tương ứng với các mức nhìn thấy của C++ và cái thứ tư xuất phát từ gói.

Các gói Java là cơ chế đóng gói tương tự như không gian tên của C++ mà cho phép các khai báo liên quan được nhóm lại cùng nhau, với một số khai báo ẩn được giấu trong các gói khác. Trong chương trình Java, mỗi trường hoặc phương thức thuộc về một lớp cụ thể và mỗi lớp là một phần của một gói như nêu trong hình 8.1. Lớp cần phải thuộc về gói mặc định không có tên hoặc một gói khác nào đó nếu được chỉ ra trong file chứa lớp.



Hình 8.1: Gói Java và tính nhìn thấy của lớp.

Các định nghĩa trong Java như sau:

- **public**: được truy cập mọi nơi mà lớp được nhìn thấy
- **protected**: được truy cập đến các phương thức của lớp đó và mọi lớp con cũng như đến các lớp khác trong cùng gói.
- **private**: được truy cập chỉ trong bản thân lớp đó
- **package**: được truy cập chỉ đến code trong cùng gói, không nhìn thấy lớp con trong các gói khác. Các thành viên được khai báo không được sửa quyền truy cập có tính nhìn thấy gói.

Nói cách khác, một phương thức có thể tham chiếu đến các thành viên **private** của lớp mà nó thuộc về, các thành viên **non private** của mọi lớp trong cùng gói, các thành viên **protected** của các lớp cha (bao gồm các lớp cha trong gói khác) và các thành viên **public** của mọi lớp trong gói bất kỳ nhìn thấy.

Các tên khai báo trong gói khác có thể được truy cập với **import**, mà đưa vào các khai báo từ gói khác hoặc với tên được qui chuẩn ở dạng sau, mà chỉ ra gói chứa tên tương minh:

```

java.lang . String . substring()
package class method

```

8.2.3 Kế thừa

Trong thuật ngữ của Java, lớp con được kế thừa từ lớp cha. Cơ chế kế thừa của Java là rất giống với C++ và các ngôn ngữ lập trình hướng đối tượng dựa trên lớp khác. Cú pháp liên kết với kế thừa là tương tự như C++ với từ khóa **extends**, như chỉ ra trong ví dụ lớp **ColorPoint** **extends** lớp **Point** trong 8.2.1:

```

class ColorPoint extends Point {
    // Additional fields and methods
    private Color c;
    protected void set C (Color d) {c=d;}
    public Color getC() {return c;}
    // Define constructor
    ColorPoint(int xval, Color cval) {
        super(xval); // call Point constructor
        c = cval; } // initialize ColorPoint field
};

```

Ghi đè và giấu trường. Như trong các ngôn ngữ lập trình khác, một lớp con kế thừa mọi trường và phương thức từ lớp cha, trừ khi một trường hoặc một phương thức có cùng tên được mô tả trong lớp con. Khi tên một phương thức trong lớp con trùng với tên phương thức trong lớp cha, định nghĩa lớp con ghi đè phương thức lớp cha với cùng dấu hiệu. Một phương thức ghi đè không được xung đột với định nghĩa mà nó ghi đè bằng cách có kiểu trả về khác. Một phương thức bị ghi đè của lớp cha cần phải được truy cập với từ khóa super. Đối với các trường, một khai báo trường trong lớp con che giấu trường trong lớp cha có cùng tên. Trường được che giấu có thể được truy cập bằng việc sử dụng tên đầy đủ (nếu nó là tĩnh) hoặc bằng việc sử dụng biểu thức truy cập trường mà chứa việc chuyển đến kiểu của superclass hoặc từ khóa super.

Constructor. Java đảm bảo rằng hàm tạo được gọi khi đối tượng được tạo. Trong khi dịch hàm tạo của lớp con, chương trình dịch kiểm tra để tin tưởng rằng, hàm tạo của lớp cha được triệu gọi. Điều này được làm theo cách chuyên biệt mà người lập trình nói chung muốn suy xét. Cụ thể, nếu lệnh đầu của hàm tạo lớp con không là lời gọi đến lớp cha, thì lời gọi super() sẽ được tự động chèn bởi chương trình dịch. Điều đó không phải lúc nào cũng làm việc tốt, vì nếu lớp cha không có hàm tạo mà không có tham số, thì lời gọi super() không sánh với hàm tạo khai báo, và chương trình dịch báo lỗi. Ngoại lệ cho việc kiểm tra này xảy ra khi một hàm tạo gọi hàm kia. Trong trường hợp này, hàm tạo đầu không cần gọi hàm tạo của lớp cha, nhưng hàm tạo thứ hai thì cần. Chẳng hạn, nếu khai báo hàm tạo ColorPoint(){ColorPoint(0,blue)} được bổ sung cho lớp ColorPoint trước, thì hàm tạo này được dịch mà không chèn thêm lời gọi đến hàm tạo của lớp cha Point.

Một khác biệt nhỏ của Java là qui tắc kế thừa cho finalize khác với qui tắc cho hàm tạo. Cho dù lời gọi đến lớp cha là được yêu cầu đối với hàm tạo, chương trình dịch không bắt buộc gọi đến phương thức finalize của lớp cha trong phương thức finalize của lớp con.

Phương thức và lớp Final. Java chức cơ chế thú vị để hạn chế các lớp con của lớp: một phương thức hoặc toàn bộ một lớp có thể được khai báo là final. Nếu phương thức được khai báo là final, thì phương thức sẽ không bị ghi đè trong bất cứ lớp con nào. Nếu lớp được khai báo là final thì lớp sẽ không thể có lớp con nào. Lý do của đặc trưng này là lập trình viên muốn định nghĩa hành vi của mọi đối tượng của một kiểu nào đó. Vì các

lớp con tạo ra các kiểu con như sẽ bàn trong 8.3, điều này yêu cầu hạn chế trên các lớp con. Một ví dụ đặc biệt này là singleton pattern chỉ ra rằng làm sao thiết kế được lớp mà chỉ có một đối tượng được tạo. Mẫu này che giấu hàm tạo của lớp và chỉ cho public đối với một hàm mà chỉ được gọi hàm tạo một lần trong khi thực hiện chương trình. Mẫu này giải quyết vấn đề hạn chế số đối tượng đối tượng của lớp, nếu không có lớp con nào ghi đè phương thức public bằng phương thức mà có thể tạo ra nhiều hơn một đối tượng. Nếu lập trình viên thực sự muốn ép buộc singleton pattern, cần có một cách giữ cho các lập trình viên khác không định nghĩa được lớp con của singleton class.

Lớp `java.lang.System` là một ví dụ khác về lớp final. Lớp này là final sao cho các lập trình viên không thể ghi đè các phương thức hệ thống.

Theo nghĩa không chặt java final là ngược lại với virtual C++: phương thức Java có thể ghi đè trừ khi nó được khai báo final, trong khi các hàm thành viên của C++ chỉ có thể được ghi đè nếu chúng là virtual. Sự tương đồng là không thật chính xác, như các hàm thành viên của C++ không thể là virtual trong một lớp mà lại là nonvirtual trong lớp cơ sở hoặc lớp suy diễn, vì nó vi phạm yêu cầu là vtables của lớp cơ sở và lớp suy diễn cần phải có cùng một cách trình bày.

Class Object. Về nguyên tắc, mỗi lớp được khai báo trong chương trình Java đều mở rộng lớp khác, như một lớp không có lớp superclass tường minh được dịch như subclass của lớp Object. Lớp Object là một lớp mà không có lớp cha. Lớp Object có các phương thức sau mà có thể ghi đè trong các lớp suy diễn:

- `getClass`, mà trả về đối tượng `Class` là biểu diễn lớp của đối tượng. Nó được sử dụng để khám phá tên đầy đủ của một lớp, các thành viên của nó các lớp cha trung gian và các giao diện mà nó cài đặt.
- `ToString`, mà trả về biểu diễn `String` của đối tượng.
- `Equals`, mà định nghĩa khái niệm bằng nhau của đối tượng mà dựa trên giá trị, không phải tham chiếu và so sánh.
- `hashCode`, mà trả về giá trị nguyên có thể được sử dụng lưu đối tượng trên bảng hash table
- `clone`, mà sử dụng để nhân bản đối tượng
- Phương thức `wait`, `notify` và `notifyall` sử dụng trong lập trình song song
- `Finalize`, mà được chạy ngay trước khi đối tượng được hủy.

Vì mọi lớp đều kế thừa lớp Object nên mỗi đối tượng đều có các phương thức này.

8.2.4 Các lớp trừu tượng và giao diện

Ngôn ngữ Java có cơ chế abstract class mà tương tự như C++. Như đã nói trong chương trước, lớp trừu tượng là lớp mà không cài đặt mọi phương thức của nó và do đó không thể có giao diện nào. Java sử dụng từ khóa `abstract` thay vì cú pháp `"=0"`, như trong đoạn code sau:

```

abstract class Shape {
    ...
    abstract point center();
    abstract void rotate(degrees d);
    ...
}

```

Java cũng có dạng lớp hoàn toàn abstract mà được gọi là interface. Giao diện được định nghĩa tương tự như lớp, ngoại trừ mọi thành phần của interface cần phải là hằng số hoặc là phương thức trừu tượng. Giao diện không có cài đặt trực tiếp, nhưng các lớp có thể được khai báo là cài đặt giao diện. Thêm vào đó giao diện có thể được khai báo là kế thừa (extends) giao diện khác cung cấp dạng kế thừa giao diện.

Một lý do các lập trình viên Java sử dụng giao diện interfaces thay cho lớp trừu tượng thuần túy (pure abstract) khi khái niệm này được định nghĩa nhưng không được cài đặt là Java cho phép một lớp cài đặt nhiều giao diện, trong khi một lớp chỉ có một lớp cha. Các giao diện và lớp sau mô tả khả năng này. Giao diện Shape xác định một số tính chất của các đối tượng hình học đơn giản, như mỗi đối tượng có điểm tâm center và phương thức quay rotate. Giao diện Drawable tương tự xác định các tính chất của đối tượng mà có thể hiện lên màn hình. Nếu các đường tròn là dạng (shape) hình học mà có thể hiện trên màn hình, thì lớp Circle có thể được khai báo cài đặt cả hai Shape và Drawable, như được nêu sau đây.

Giao diện thường được sử dụng như kiểu đối số của phương thức. Chẳng hạn, nếu hệ thống windows có phương thức để vẽ một số mục trên màn hình, thì kiểu đối số của phương thức này có thể là Drawable cho phép mỗi đối tượng mà cài đặt giao diện Drawable được hiện trên màn hình:

```

interface Shape {
    public Point center();
    public void rotate(float degrees);
}
interface Drawable {
    public void setColor(Color c);
    public void draw();
}
class Circle implements Shape, Drawable {
    // does not inherit any implementation
    // but must define Shape, Drawable methods
}

```

Không giống như kế thừa bội của C++ (nêu trong 7.5), ở đây không có vấn đề xung đột tên đối với giao diện Java. Cụ thể giả sử hai giao diện Shape và Drawable cùng có

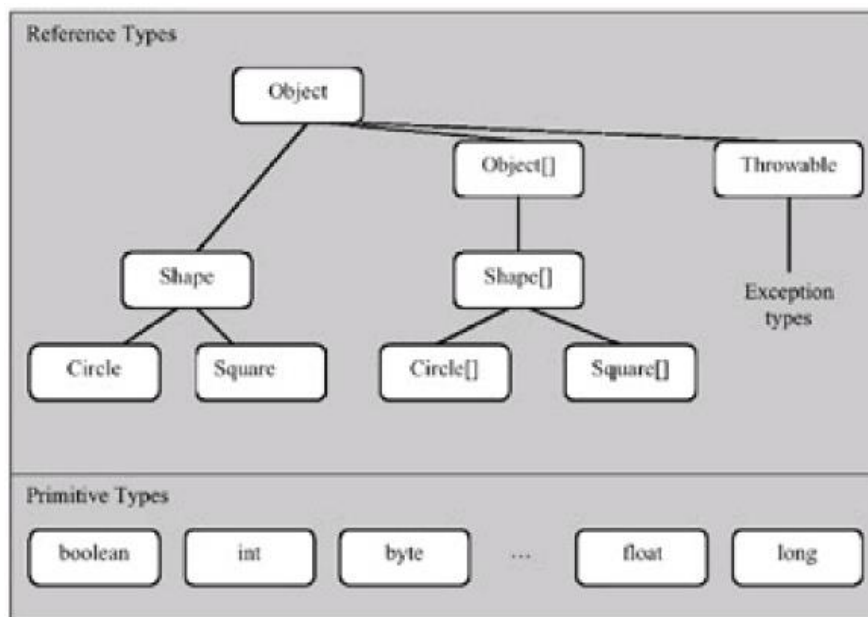
phương thức Size. Nếu cả hai phương thức Size có cùng số đối số và cùng kiểu đối số, thì lớp Circle cần cài đặt một phương thức Size với số đối số và kiểu đối số được cho trong hai giao diện đó. Mặt khác nếu hai phương thức Size có số đối số khác nhau hoặc các đối số có kiểu khác nhau, thì chúng được xem là hai phương thức khác nhau và Circle cần định nghĩa cài đặt cho mỗi giao diện đó. Vì tìm kiếm phương thức của Java sử dụng tên phương thức, số và kiểu đối số để chọn code của phương thức, hai phương thức cùng tên sẽ được xử lý khác nhau trong thời gian tìm kiếm phương thức.

8.3 Các kiểu Java và kiểu con

8.3.1 Phân loại kiểu

Các kiểu Java được chia ra thành hai chủng loại: các kiểu nguyên thủy và các kiểu tham chiếu. Tám kiểu nguyên thủy là kiểu bool và bảy kiểu số. Bảy kiểu số là các dạng interger byte, short, int, long, char, float và double. Ba kiểu tham chiếu là class types, interface types và array type. Có một kiểu đặc biệt là null type. Các giá trị của các kiểu tham chiếu là các tham chiếu đến các đối tượng (bao gồm cả mảng). Mọi đối tượng, bao gồm cả mảng, hỗ trợ các phương thức của lớp Object.

Quan hệ subtyping giữa các họ chính của kiểu được mô tả trên Hình 8.2, mà bao gồm cả giao diện Shape và các lớp Circle và Square chỉ ra rằng các lớp và giao diện do người sử dụng định nghĩa cũng được sắp xếp trên hình này. Các kiểu định nghĩa trước như String, ClassLoader, và Thread chiếm các vị trí tương tự như Exception trên hình này. Object[] là kiểu mảng Object cũng tương tự như kiểu mảng Shape[] và Square[].



Phân loại kiểu trong Java.

Mặc dù là thuật ngữ chuẩn của Java gọi Object và các kiểu tham chiếu Subtyp, nó có thể làm cho hơi rắc rối. C++ phân biệt Object với Object*, ở đây không có kiểu con trỏ

tường minh trong Java. Thay vào đó, vì sự khác biệt giữa con trỏ đến đối tượng và đối tượng là ẩn, nên việc định vị con trỏ kết hợp với các phép toán như triệu gọi phương thức và truy cập các trường. Nếu T là kiểu tham chiếu, thì biến x kiểu T là tham chiếu đến đối tượng T , trong C++ biến x cần có kiểu T^* . Vì ở đây không cần có cách tường minh định vị x để nhận giá trị kiểu T , Java không có kiểu riêng cho đối tượng không được tham chiếu bởi con trỏ.

Vì mỗi lớp là kiểu con của Object, biến kiểu Object có thể tham chiếu đến các đối tượng hoặc mảng kiểu bất kỳ.

8.3.2 Kiểu con cho lớp và giao diện

Kiểu con cho các lớp được xác định bởi phân cấp lớp và cơ chế giao diện. Cụ thể nếu lớp A mở rộng lớp B , thì kiểu các đối tượng A là kiểu con của kiểu các đối tượng B . Ở đây không có cách nào khác cho một lớp để định nghĩa kiểu con của lớp khác, và không có các lớp cơ sở private nào (như trong C++) cho phép kế thừa mà không phải kiểu con. Một lớp có thể được khai báo để cài đặt một hay nhiều giao diện, theo nghĩa một instance của một lớp cài đặt mọi phương thức trừu tượng được nêu trong giao diện. Kiểu con giao diện lập cho phép các đối tượng hỗ trợ hành vi chung mà không chia sẻ bất cứ cài đặt chung nào.

Chuyển kiểu thời gian chạy. Java không cho phép chuyển kiểu không kiểm tra. Tuy nhiên các đối tượng kiểu cha có thể được chuyển kiểu về kiểu con bằng cơ chế bao gồm việc kiểm tra kiểu thời gian chạy. Nếu bạn muốn tạo lớp list trong Java, bạn có thể giữ các đối tượng kiểu Object. Các đối tượng kiểu bất kỳ có thể đưa vào list, nhưng lấy chúng ra và sử dụng là không hiển nhiên, chúng cần phải chuyển ngược lại kiểu gốc của chúng (hoặc về kiểu cha nào đó). Trong Java, chuyển kiểu được kiểm tra trong thời gian chạy, tạo ra ngoại lệ, nếu đối tượng không có kiểu đích.

Cài đặt. Java sử dụng các chỉ lệnh bytecode để tìm kiếm các thành viên cho giao diện và tìm kiếm thành viên cho lớp và các lớp con. Trong chương trình dịch thực thi tốc độ cao, tìm kiếm cho lớp có thể được cài đặt trong C++, với offset được biết trong thời gian dịch. Tuy nhiên tìm kiếm cho giao diện thì không thể, vì lớp có thể cài đặt nhiều giao diện và giao diện có thể liệt kê các thành viên theo các trật tự khác nhau. Điều này sẽ được bàn đến trong 8.4.4.

8.3.3 Các mảng, Covariance và Contravariance

Đối với kiểu T bất kỳ, Java có kiểu mảng $T[]$ của các mảng mà các phần tử có kiểu T . Mặc dù kiểu mảng được nhóm thành các lớp và các giao diện, nhưng không thể kế thừa từ một kiểu mảng. Trong thuật ngữ Java, các kiểu mảng là final.

Các kiểu mảng là kiểu con của Object và do đó các mảng hỗ trợ mọi phương thức gắn kết với lớp Object. Giống như các kiểu tham chiếu khác, biến mảng là con trỏ đến mảng

và có thể null. Nói chung là sẽ tạo mảng khi một tham chiếu mảng được khai báo, như trong

```
Circle[] x = new Circle[array_size]
```

Tuy nhiên, cũng có thể tạo các đối tượng mảng vô danh. Giống như cách mà chúng ta có thể tạo các đối tượng khác. Chẳng hạn,

```
new int[] {1,2,3, ... 10}
```

là biểu thức và tạo mảng nguyên độ dài 10, với các giá trị 1, 2, 3, ..., 10. Vì biến của kiểu T[] có thể được gán cho mảng có độ dài tùy ý, độ dài của mảng không phải là một phần kiểu tĩnh.

Ở đây có một số rắc rối xung quanh cách mà các kiểu mảng của Java được đặt trong cây phân cấp kiểu. Một quyết định quan trọng là nếu $A \leq B$, thì kiểm tra kiểu của Java cũng sẽ sử dụng kiểu con $A[] \leq B[]$. Điều này dẫn đến vấn đề thường được tham chiếu đến như array covariance problem. Xét các khai báo lớp và mảng sau:

```
class A { ... }
class B extends A { ... }

B[] bArray = new B[10]
A[] aArray = bArray // considered OK since B[] <= A[]
aArray[0] = new A() // allowed but run-time type error; raises
ArrayStoreException
```

Trong đoạn code này, chúng ta có $B \leq A$, vì lớp B là extends lớp A. Tham chiếu mảng bArray trỏ đến mảng của các đối tượng kiểu B. Ban đầu tất cả đều null, và tham chiếu aArray trỏ đến cùng mảng đó. Khai báo $A[]$ aArray = bArray được cho phép bởi kiểm tra kiểu Java (mặc dù về mặt ngữ nghĩa nó không được phép), vì quyết định thiết kế của Java là nếu $B \leq A$, thì $B[] \leq A[]$. Vấn đề mà cho phép tham chiếu mảng $A[]$ aArray trỏ đến mảng của các đối tượng B được thể hiện qua lệnh cuối cùng. Lệnh gán $aArray[0] = new A()$ trông có vẻ là hợp lý: aArray là mảng với kiểu tĩnh A[], giả thiết là chấp nhận gán một đối tượng kiểu A cho vị trí bất kỳ trong mảng. Tuy nhiên vì aArray tham chiếu đến mảng của các đối tượng B, phép gán này vi phạm kiểu của bArray. Vì vậy phép gán này có thể sinh ra vấn đề kiểu, phần cài đặt Java không cho phép phép gán này được thực hiện. Kiểm tra kiểu thời gian chạy sẽ xác định là giá trị mà được gán cho mảng của các đối tượng B không là đối tượng B và sẽ thông báo xử lý lỗi ArrayStoreException.

Mặc dù những người thiết kế Java xem array covariance là ưu điểm với một số mục đích cụ thể (viết một số chương trình copy nhị phân), array covariance trong Java dẫn đến một số rắc rối và kiểm tra thời gian chạy. Nó không tỏ ra là thành công của quyết định thiết kế ngôn ngữ.

8.3.4 Phân lớp ngoại lệ của Java

Các chương trình Java có thể khai báo, phát sinh và kiểm soát lỗi. Cơ chế ngoại lệ của Java có các tính chất tổng thể chung. Ngoại lệ Java có thể là kết quả của lệnh throw trong chương trình người sử dụng hoặc kết quả của điều kiện lỗi nào đó được phát hiện bởi máy ảo chẳng hạn như vượt ra ngoài phạm vi của mảng. Trong thuật ngữ Java, ngoại lệ được nói là thrown từ điểm mà xảy ra và nó được nắm bắt (caught) tại điểm truyền điều khiển cho nó. Như trong ngôn ngữ khác, việc quăng ra (throwing) ngoại lệ buộc việc cài đặt Java dừng mỗi biểu thức, lệnh, phương thức, hàm tạo, khởi động, hoặc biểu thức tạo trường mà đã bắt đầu thực hiện nhưng chưa kết thúc. Tiến trình này tiếp tục khi handler được tìm thấy mà sánh với lớp ngoại lệ đã được quăng ra.

Một khía cạnh thú vị của cơ chế ngoại lệ Java là cách mà nó tích hợp thành lớp và phân loại kiểu. Mỗi ngoại lệ Java được thể hiện bằng một trường hợp (instance) của lớp Throwable hoặc một trong các lớp con của nó. Ưu điểm của việc biểu diễn ngoại lệ như những đối tượng là đối tượng ngoại lệ có thể được sử dụng để mang thông tin từ điểm mà lỗi xảy ra đến handler mà nắm bắt được nó. Cấu trúc kiểu con cũng có thể được sử dụng kiểm soát lỗi: handler sánh với lớp của ngoại lệ nếu nó tương minh nêu tên của lớp ngoại lệ mà nó quăng ra hoặc nêu tên lớp cha nào đó của lớp ngoại lệ.

Các ngoại lệ Java được nắm bắt bên trong cấu trúc được gọi là *try-finally-block*. Sau đây là một ví dụ chỉ ra khối với hai handlers, mỗi cái định danh bởi từ khóa catch. Về thực quan, *try-finally-block* tìm cách thực hiện dãy các lệnh. Nếu dãy các lệnh kết thúc bình thường, đó là kết quả cuối cùng của khối. Tuy nhiên, nếu phát sinh ngoại lệ, nó có thể được nắm bắt bên trong khối. Nếu ngoại lệ phát sinh và được nắm bắt, thì dãy các lệnh sau từ khóa *finally* sẽ được thực hiện sau khi kiểm soát ngoại lệ kết thúc:

Dưới đây là cấu trúc của mô hình xử lý ngoại lệ:

```
try
{
    // đoạn mã có khả năng gây ra ngoại lệ
}
catch(Exception e1)
{
    // Nếu các lệnh trong khối 'try' tạo ra ngoại lệ có loại e1, thì thực hiện //xử lý ngoại
    lệ nếu không chuyển xuống khối 'catch' tiếp theo
}
catch(Exception e2)
{
    // Nếu các lệnh trong khối 'try' tạo ra ngoại lệ có loại e2, thì thực hiện //xử lý ngoại
    lệ nếu không chuyển xuống khối 'catch' tiếp theo
}
```

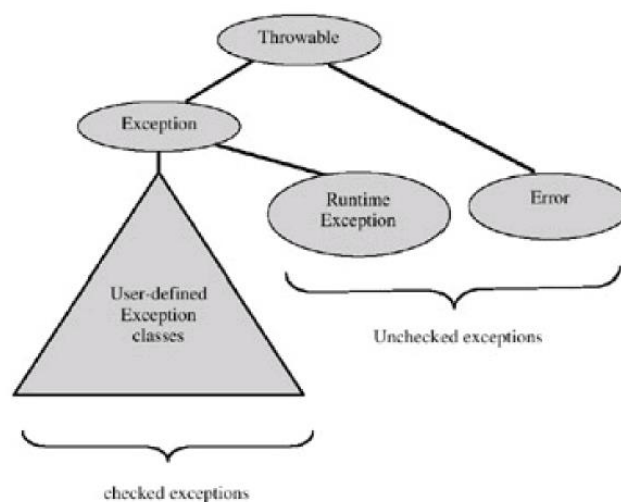
```

}
catch(Exception eN)
{
    // Nếu các lệnh trong khối 'try' tạo ra ngoại lệ có loại eN, thì thực hiện //xử lý ngoại
    // lệ nếu không chuyển xuống khối 'catch' tiếp theo
}
finally
{
    // khối lệnh nay luôn được thực hiện cho dù ngoại lệ có xảy ra hay không.
}

```

Mặc dù nó có thể không rõ ràng tại sao, ở đây có một số phức tạp trong JVM gắn kết với các khối *try-finally-block*. Cụ thể, phần quan trọng nhất của bộ kiểm tra bytecode phức tạp Java là kết quả của cách mà các mệnh đề *finally* được cài đặt như một dạng chương trình con cục bộ (được gọi là *jsr*) trong trình thông dịch bytecode Java.

Các lớp của ngoại lệ được nêu trong Hình 8.3. Mỗi một ngoại lệ theo định nghĩa là một đối tượng của một lớp nào đó trong *Throwable*. Lớp *Throwable* là lớp con trực tiếp của *Object*. Chương trình có thể sử dụng các lớp ngoại lệ đã có trong *Throwable* hoặc một trong những lớp con của nó. Để có được tính ưu việt của bộ kiểm tra thời gian dịch của nền tảng Java đối với việc xử lý lỗi, thông thường cần định nghĩa lớp ngoại lệ mới nhất như là các lớp ngoại lệ cần kiểm tra. Đó là các lớp con của *Exception* mà không là lớp con của *RuntimeException*.



Các lớp ngoại lệ của Java

Các câu *checked exceptions* và *unchecked exceptions* tham chiếu đến việc kiểm tra thời gian chạy của một tập các ngoại lệ mà có thể được quăng ra trong chương trình Java. Chương trình dịch Java kiểm tra trong thời gian dịch xem chương trình có chứa phần

xử lý (handlers) cho mỗi ngoại lệ được kiểm tra không. Chúng ta sẽ thực hiện điều này bằng cách phân tích các ngoại lệ được kiểm tra mà có thể được quăng ra trong khi thực hiện phương thức hay hàm tạo. Các kiểm tra này dựa trên tập các ngoại lệ được khai báo mà một phương thức của Java có thể quăng ra, được gọi là mệnh đề quăng của phương thức đó. Đối với mỗi ngoại lệ đã kiểm tra mà là kết quả có thể của lời gọi phương thức, mệnh đề quăng cho phương thức cần phải nhắc đến lớp của ngoại lệ này hoặc một trong những lớp con của lớp ngoại lệ đó. Kiểm tra thời gian dịch đó xem có xử lý ngoại lệ không được thiết kế để giảm số ngoại lệ mà không được xử lý đúng đắn.

Vì các ngoại lệ Error và RuntimeException nói chung được quăng ra bởi hệ thống thời gian chạy Java và không phải bởi mã người sử dụng, do đó không cần thiết đối với người lập trình phải khai báo các ngoại lệ này trong mệnh đề quăng của phương thức. Cụ thể, các lớp ngoại lệ thời gian chạy (RuntimeException và các lớp con của nó) là được miễn kiểm tra thời gian dịch, vì trong các xem xét của những người thiết kế ngôn ngữ lập trình Java, khai báo các ngoại lệ như vậy sẽ hỗ trợ không đáng kể trong việc thiết lập tính đúng đắn của chương trình. Nhiều thao tác và cấu trúc của ngôn ngữ lập trình Java có thể có kết quả là ngoại lệ thời gian chạy. Thông tin sẵn sàng cho chương trình dịch và mức độ phân tích mà chương trình dịch thực hiện là không đủ để thiết lập rằng các ngoại lệ thời gian chạy như vậy không thể xảy ra, ngay cả khi điều đó là rõ ràng đối với lập trình viên.

Các chương trình thông thường không thường khắc phục các ngoại lệ của lớp Error hoặc các lớp con của nó. Lớp Error là lớp con riêng của Throwable, khác với Exception trong cấu trúc phân lớp, để cho phép chương trình sử dụng thành ngữ

```
} catch (Exception e) {
```

để nắm bắt mọi ngoại lệ mà từ đó việc khôi phục có thể thực hiện được, mà không nắm bắt lỗi error, từ đó thông thường không khắc phục được.

8.3.5 Đa hình kiểu con và lập trình

Trong chương trước chúng ta đã bàn về một số cách tiếp cận lập trình generic dựa trên templates và các cơ chế liên quan như generic module và package. Ý tưởng chính là chúng ta có thể định nghĩa cấu trúc dữ liệu generic, như list hoặc cây nhị phân, sao cho bất cứ kiểu dữ liệu nào cũng có thể chèn vào hoặc được truy vấn từ cấu trúc dữ liệu đó, với điều kiện rằng một số phép toán yêu cầu, như kiểm tra bằng nhau hoặc thứ tự tuyến tính được định nghĩa và cài đặt trên dữ liệu đó. Các thuật toán generic như sắp xếp, ứng dụng cho list hoặc array dữ liệu bất kỳ, có thể được định nghĩa tương tự như việc sử dụng templates hoặc các cơ chế liên quan. Templates và generic modules là rất có ích trong lập trình generic vì chúng cung cấp dạng đa hình: cùng một code có thể được khởi tạo và thực hiện trên nhiều kiểu dữ liệu khác nhau.

Trong lập trình hướng đối tượng, thuật ngữ đa hình (polymorphism) thường được sử dụng để nói đến một kiểu của đa hình mà chúng ta gọi là đa hình kiểu con (subtype polymorphism). Trong đa hình kiểu con, chỉ một đoạn code, thông thường là tập các hàm hoặc các phương thức, có thể được áp dụng cho một hoặc nhiều kiểu tham số. Tuy nhiên, cơ chế ngôn ngữ mà cho phép làm điều này là không thông qua tham số biến ẩn (như trong đa hình suy luận biến của ML) hoặc tham số biến tường minh (như trong templates của C++), nhưng qua kiểu con (subtyping). Cụ thể, trong ngôn ngữ với subtyping, nếu phương thức m chấp nhận đối số bất kỳ kiểu A , thì m có thể được áp dụng cho đối số bất kỳ của kiểu con bất kỳ của A . Điều này làm việc hầu như tốt như đa hình tham số (đa hình dựa trên tham số kiểu), ngoại trừ thông thường bộ kiểm tra kiểu thời gian dịch có ít thông tin hơn về các kiểu của đối số. Điều này dẫn đến hoặc kiểm tra kiểu thời gian chạy hoặc chấp nhận vì an toàn kiểu. Kịch bản chung được mô tả trong ví dụ Java sau:

Ví dụ 8.1. Java Subtype Polymorphism

Giả sử ta xét bài toán định nghĩa lớp generic của các ngăn xếp. Chúng ta muốn xây dựng ngăn xếp với kiểu bất kỳ của đối tượng. Chúng ta sẽ cài đặt ngăn xếp như danh sách kết nối của các nút ngăn xếp, mỗi nút chứa một phần tử ngăn xếp.

Vì Object là kiểu cha của mọi kiểu tham chiếu (bao gồm mọi tham chiếu đến đối tượng và mảng), chúng ta cần cho các ô ngăn xếp chứa tham chiếu của kiểu Object. Chúng ta bắt đầu từ định nghĩa lớp phụ trợ, lớp các nút ngăn xếp:

```
class Node {  
    Object element; // Any object can be placed in a stack  
    Node next;  
    Node (Node n, int e) {  
        next=n;  
        element=e;  
    }  
}
```

Lớp này dự định sẽ thuộc cùng package với lớp Stack. Vì ở đây không có các từ nói đến quyền truy cập (public, private hoặc protected) các trường và các phương thức được truy cập chỉ đến các phương thức trong cùng gói.

Đây là dàn ý của lớp Stack, ở đó các đối tượng Node được sử dụng để lưu các phần tử của danh sách liên kết các phần tử của ngăn xếp. Các phương thức ở đây là rỗng mà nói lên rằng, nếu ngăn xếp rỗng, *push* sẽ bổ sung phần tử vào đỉnh ngăn xếp, và *pop* sẽ lấy bỏ phần tử ở đầu ngăn xếp và trả về đối tượng được lưu ở node đó. Lớp được khai báo public như vậy nó có thể được nhìn thấy ở bên ngoài gói mà ở đó nó được định nghĩa :

```

public class Stack {
    private Node top=null;    // top of the stack, starts as empty
    public Stack(){
    }
    public boolean empty(){    // determine whether stack is empty
        return top == null;
    }
    public void push(Object val) { // push element on top of stack
        top = new Node(top,val);
    }
    public Object pop() {        // Remove first node and return value.
        if (top==null) return -1;
        Node temp = top;
        top = top.next;
        return temp.element;
    }
}

```

Hầu hết code ở đây là hoàn toàn trực tiếp. Hạn chế quan trọng đối với lập trình generic là kiểu của các phần tử Node và kiểu trả về của pop. Vì kiểu của đối số của push là Object, đối tượng bất kỳ có thể có thể đẩy vào ngăn xếp. Tuy nhiên kiểu trả về của pop cũng là Object, nghĩ là khi một đối tượng được lấy bỏ ngăn xếp, kiểu tĩnh của biểu thức lấy bỏ đối tượng là Object. Do đó, nếu chương trình push các xâu vào ngăn xếp và sau đó lấy chúng ra khỏi ngăn xếp, kiểm tra kiểu Java sẽ không thể xác định các đối tượng lấy ra từ ngăn xếp là các xâu.

Đây là ví dụ về cách ngăn xếp có thể được sử dụng trong chương trình Java:

```

String s = "Hello";
Stack st = new Stack();
...
st.push(s);
...
s = (String) st.pop;

```

Trong đoạn code trên, đối tượng String được đẩy vào ngăn xếp. Khi đối tượng String được lấy ra khỏi ngăn xếp, kiểu tĩnh của st.pop() là Object, không phải String. Do đó để gán st.pop() cho biến string, cần phải chuyển kiểu Object sang String. Chuyển kiểu được kiểm tra trong thời gian chạy. Nếu phần tử ở đầu ngăn xếp không phải string, thì khi phép gán này thực hiện, kiểm tra lỗi thời gian chạy sẽ quăng ra ngoại lệ và ngăn không cho phép gán được thực hiện.

Để so sánh, C++ stack class templates có thể được định nghĩa dạng sau:

```
template <typename t> class Stack {
    private: Node<t> top;
    public: boolean empty() {...};
           void push (t* x) {...}
           t* pop ( ) {...}
};
```

Nếu Java có cơ chế template tương tự như trên, thì ta có thể định nghĩa lớp ngăn xếp generic như sau:

```
class Stack<A> {
    public Boolean empty(){...}
    public void push(A a){...}
    public A pop(){...}
}
```

Và sử dụng ngăn xếp generic như sau:

```
String s = "Hello";
Stack<String> st = new Stack<String>();
st.push(s);
...
s = st.pop();
```

Ở đây có một vài điểm quan trọng trong việc so sánh giữa hai phong cách mô tả ở đây. Trong code dựa trên template ngăn xếp st có kiểu Stack<string thay vì Stack. Như kết quả, st.pop() có kiểu tĩnh là String. Hai ưu điểm là rõ ràng và hiệu quả. Ý định của lập trình viên là rõ ràng, như ngăn xếp st được khai báo tường minh là ngăn xếp của các xâu. Ở đây có một số dự án nghiên cứu hướng tới việc bổ sung template cho Java. Mặc dù mục tiêu cơ bản của cơ chế như vậy trông có vẻ rõ ràng, nhưng ở đây có một số chi tiết cần bàn đến. Một vấn đề là cài đặt. Một cách đơn giản là dịch từ Java có templates sang Java không có templates. Điều này cho phép lập trình viên viết code với templates và cung cấp cài đặt tương đối đơn giản. Tuy nhiên kết quả của việc chuyển đổi là mã mà chứa nhiều chuyển kiểu kiểm tra thời gian chạy, vì vậy nó không hiệu quả như các kỹ thuật cài đặt khác. Một biến thể khác là dịch lớp generic như Stack<t> vào dạng class file mà có kiểu tham số và tải lớp Stack<String> bằng việc khởi tạo class file này tại thời gian tải. Nó tạo ra code hiệu quả hơn, nhưng có thể phải tải nhiều lớp hơn. Vì tải lớp là chậm, chương trình tổng thể có thể chạy chậm hơn trong một số trường hợp. Ở đây cũng có vấn đề khởi động class templates cho các kiểu nguyên thủy như bool, int, float, ... Phương thức cài đặt đầu tiên không làm việc cho các kiểu nguyên thủy, vì ở đây phải lưu khác nhau do các kiểu đối tượng. Phương thức thứ hai, khởi động class files trong thời gian tải, có thể làm việc được, cho dù kiểm soát các kiểu nguyên thủy bổ sung phức tạp thêm vào cơ chế này. Trong trường hợp khác, ở đây bổ sung một số điều chỉnh tinh tế trong thiết kế cơ chế kiểm tra kiểu đối với các lớp generic mà chúng ta chưa bàn tới.

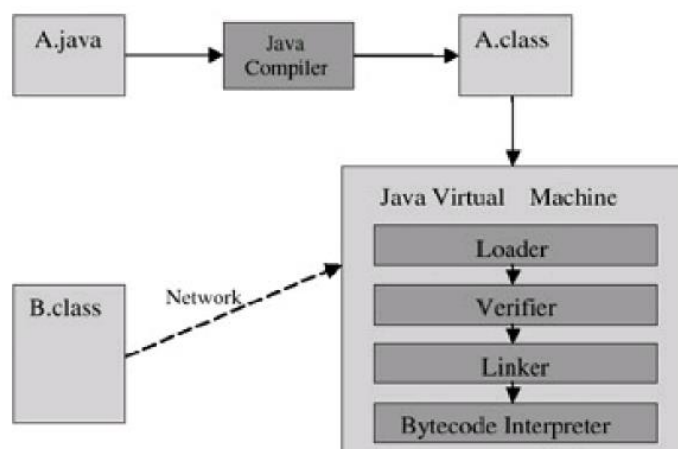
8.4 Kiến trúc hệ thống Java

8.4.1 Máy ảo Java

Có một số câu đặt của Java. Mục này bàn đến kiến trúc cài đặt được sử dụng trong chương trình dịch Java của Sun và máy ảo (JVM). Sự khác biệt lớn nhất giữa kiến trúc này và những cái khác ở chỗ một số cài đặt tối ưu sinh ra native code cho các phần cứng ở dưới thay vì code thông dịch. Tuy nhiên vì các chương trình dịch Just in time (JIT) sẽ sinh ra native code là rất phức tạp so với kiến trúc JVM, chúng ta sử dụng kiến trúc JVM như cơ sở để bàn và phân tích các vấn đề cài đặt Java.

Chương trình dịch Java tạo ra dạng ngôn ngữ máy gọi là bytecode. Cái gốc của tên bytecode, mà là thuật ngữ chuẩn, có trước Java, là vì các bộ bytecode là các chỉ lệnh cho máy ảo mà có thể có độ dài 1 byte. Nhiều chương trình dịch cho các ngôn ngữ bậc cao thực tế tạo ra bytecode thay vì mã máy tự nhiên. Chẳng hạn, chương trình dịch Pascal có nhiều ảnh hưởng UCSD tạo ra dạng bytecode gọi là P-code, mà sau đó được thông dịch bằng máy ảo. Cài đặt ML cũng sinh ra bytecode máy ảo.

Hình 8.4 chỉ ra các phần chính của JVM và bao gồm một số bước trong khi dịch và thực hiện mã nguồn Java. Các phần chính của JVM là tải lớp (class loader), kiểm chứng bytecode (bytecode verifier), liên kết (linker) và thông dịch bytecode (bytecode interpreter).



Phía trên Hình 8.4 chỉ ra file mã nguồn Java A.java được dịch tạo ra file class A.class. File class này ở dạng chuyên biệt chứa các chỉ lệnh bytecode và một số cấu trúc hỗ trợ, bao gồm cả bảng ký hiệu gọi là constant pool. Class loader đọc các file class bố trí các chỉ lệnh bytecode dạng phù hợp để kiểm chứng. Ngôn ngữ Java bytecode chứa thông tin kiểu cho phép verifier kiểm tra bytecode là kiểu đúng trước khi nó được thực hiện bởi máy ảo. Liên kết linker giải quyết các tham chiếu trong file và thông dịch bytecode thực hiện các chỉ lệnh bytecodes.

Hộp có nhãn B.class trong Hình 8.4 chỉ ra rằng các lớp Java được tải từng bước theo yêu cầu, khi thực hiện chương trình và có thể tải qua mạng cũng như từ hệ thống file cục bộ. Cụ thể hơn, nếu lớp A có phương thức main, thì A có thể được tải như chương

trình chính và thực hiện. Nếu một phương thức của lớp A tham chiếu đến lớp B, thì lớp B sẽ được tải, khi phương thức này được gọi. Class loader sẽ tìm cách tìm file lớp phù hợp và tải nó vào máy ảo để việc thực hiện chương trình có thể tiếp tục.

8.4.2 Class Loader

Hệ thống thời gian chạy Java sẽ tải lớp, nếu chúng cần, tìm kiếm bytecode đã được dịch cho lớp bất kỳ, mà được tham chiếu nhưng chưa tải. Ở đây có cơ chế tải mặc định, mà chúng ta có thể thay thế bằng việc định nghĩa một đối tượng ClassLoader. Một lý do để định nghĩa ClassLoader thay thế này là để nhận được bytecodes từ một nguồn từ xa và tải chúng trong thời gian chạy.

Chúng ta định nghĩa một class loader bằng việc mở rộng lớp trừu tượng ClassLoader và cài đặt phương thức loadClass của nó. Đây là dạng có thể có của định nghĩa class loader:

```
class _Load My_Classes extends ClassLoader {
    private Hashtable loaded_classes = new Hashtable();
    public Class loadClass(string name, boolean resolve)
        throws ClassNotFoundException {
        /* Code to try to load class "name" and add to "loaded_classes".
         * If boolean arg "resolve" is true, then invoke "resolveClass"
         * method to ensure that all classes referred to by class are
         * loaded. Raise exception if class cannot be found and loaded. */
    }
    ...
}
```

Khi loadClass tìm thấy bytecodes của lớp mà cần tải, nó có thể triệu gọi phương thức defineClass được kế thừa, mà lấy mảng byte, vị trí start và số bytes. Phương thức này khởi tạo dãy bytes như là cài đặt của lớp mong muốn.

Kiểu trả về của phương thức loadClass là Class. Các class là các đối tượng, và chúng ta có thể tìm class loader cho một lớp cho trước bằng cách gọi phương thức getClassLoader của lớp đó. Nếu lớp không có class loader, thì phương thức trả về null.

8.4.3 Java Linker, Verifier và nguyên lý kiểu

Linker lấy dạng nhị phân của lớp và giao diện như đầu vào và bổ sung nó vào trạng thái thời gian chạy của JVM.

Mỗi lớp được kiểm tra trước khi nó được liên kết và làm sẵn sàng cho việc thực hiện. Kiểm chứng Java bytecode là tiến trình chuyên biệt mà kiểm tra để tin tưởng rằng mỗi lớp có các tính chất sau:

- Mỗi chỉ lệnh cần có mã thao tác đúng

- Mỗi chỉ lệnh nhánh cần rẽ nhánh đến chỗ bắt đầu chỉ lệnh khác, chứ không vào giữa chỉ lệnh nào đó
- Mỗi phương thức có cấu trúc dấu hiệu đúng
- Mỗi chỉ lệnh tuân thủ nguyên tắc kiểu Java

Nếu lỗi xảy ra trong khi kiểm tra, thì khởi tạo instance của lớp `VerifyError` sẽ được quảng ra tại điểm mà chương trình Java nắm bắt lớp cần kiểm tra.

Quá trình liên kết bao gồm cả việc tạo nên các trường tĩnh của lớp hoặc giao diện và khởi gán chúng giá trị mặc định (thông thường là 0). Các tên cũng được hóa giải, nó bao gồm việc kiểm tra tham chiếu ký hiệu và thay chúng bằng tham chiếu trực tiếp mà có thể xử lý hiệu quả hơn khi tham chiếu được sử dụng thường xuyên.

Các lớp mà không được tải tường minh. Nếu không có khởi tạo nào của lớp đạt được, thì lớp có thể được loại bỏ như một phần của việc gom rác.

8.4.4 Thông dịch bytecode và tìm kiếm phương thức

Thông dịch bytecode thực hiện Java bytecode và thực hiện kiểm tra thời gian chạy như kiểm tra biên của mảng để tin tưởng rằng mỗi truy cập mảng đều nằm trong biên khai báo của mảng. Kiến trúc thời gian chạy cơ bản của trình biên dịch Java là tương tự với kiến trúc máy tính đơn giản nêu trong chương 4 với program counter, instruction area, stack và heap.

Chương trình Java chạy gồm một hoặc nhiều luồng tuần tự. Khi một luồng mới được tạo, nó được cấp một program counter và một stack. Program counter chỉ rõ chỉ lệnh tiếp theo cần thực hiện. Java stack bao gồm các bản ghi kích hoạt, mỗi một lưu trữ các biến cục bộ, các tham số, giá trị trả về và các giá trị trung gian đối với việc triệu gọi phương thức Java. Máy ảo Java JVM không có registers để giữ giá trị trung gian mà lưu trên Java stack. Cách tiếp cận này được quyết định bởi những người thiết kế Java để giữ tập các chỉ lệnh Java chặt và làm cài đặt dễ dàng trên nhiều kiến trúc máy tính nền. Khi một đối tượng hoặc mảng được tạo, nó được lưu trên heap. Mọi luồng chạy trên một máy ảo Java cùng chia sẻ heap.

Bản ghi kích hoạt có ba phần: vùng biến cục bộ, ngăn xếp toán hạng và vùng dữ liệu. Kích thước của vùng biến cục bộ và ngăn xếp toán hạng, mà được đo bằng words, phụ thuộc vào sự cần thiết của phương thức và được xác định trong thời gian dịch. Vùng biến cục bộ của bản ghi kích hoạt Java được tổ chức như mảng các từ, với các kiểu dữ liệu khác nhau chiếm số các từ sẽ khác nhau. Ngăn xếp toán hạng được dùng cho các tính toán trung gian và truyền tham số cho các phương thức khác. Thay vì truy cập dữ liệu trong bộ nhớ bằng địa chỉ ô nhớ, hầu hết các chỉ lệnh bytecode Java thao tác đẩy, kéo, thay thế giá trị từ đỉnh của ngăn xếp này trong ngăn xếp. Chỉ lệnh cộng với hai toán hạng, chẳng hạn, kéo hai toán hạng ra khỏi ngăn xếp toán hạng và đẩy tổng của nó

vào ngăn xếp toán hạng. Kiểu kiến trúc này dẫn đến mã chỉ lệnh ngắn hơn, vì chỉ lệnh với một số toán hạng cần chỉ để xác định toán hạng, chứ không cần để cho địa chỉ hoặc số thanh ghi cho toán hạng, vì toán hạng luôn là một số dữ liệu trên đỉnh ngăn xếp toán hạng. Vùng dữ liệu của bản ghi kích hoạt lưu dữ liệu sử dụng để hỗ trợ hóa giải constant pool (mà chúng ta sẽ bàn trong phần sau), trả về phương thức bình thường và rẽ nhánh ngoại lệ.

Các chỉ lệnh bytecode Java đã dịch được lưu trữ dạng file mà bao gồm cấu trúc dữ liệu gọi là constant pool. Đó là bảng tên ký hiệu như tên lớp, tên trường và tên phương thức. Khi chỉ lệnh bytecode tham chiếu đến trường, chẳng hạn, tham chiếu thực tế sẽ là một số biểu diễn chỉ số đến constant pool. Để nhận được một trường, chẳng hạn, chỉ lệnh có thể chứa số 27, chỉ ra tên ký hiệu thứ 27 trong constant pool. Cách tiếp cận này lưu tên ký hiệu từ mã nguồn trong file bytecode, nhưng dành không gian để lưu tên mỗi ký hiệu một lần.

Khi thực hiện chương trình, trình thông dịch bytecode sẽ làm các kiểm tra thời gian chạy sau để ngăn lỗi kiểu và giữ tính toàn vẹn của hệ thống thời gian chạy:

- Mọi chuyển kiểu đều được kiểm tra để đảm bảo chúng an toàn kiểu
- Mọi tham chiếu mảng đều được kiểm tra để đảm bảo chỉ số mảng nằm trong các biên của mảng
- Các tham chiếu được kiểm tra để tin tưởng rằng chúng không null trước khi tìm địa chỉ

Thêm vào đó, việc không có số học con trỏ và việc sử dụng gom rác tự động hỗ trợ cho việc thực thi an toàn của chương trình Java.

Khi các chỉ lệnh mà truy cập đến trường hoặc phương thức được thực hiện, thường phải tìm kiếm vị trí thích hợp trong object template hoặc bảng tìm kiếm phương thức (method lookup table), bằng việc sử dụng tên ký hiệu của trường hoặc phương thức trong constant pool. Điều này không hiệu quả nếu truy cập qua constant pool thường xuyên. Tìm kiếm qua constant pool cũng là nút cổ chai thực thi trong chương trình song song, vì điều này yêu cầu khóa constant pool và việc khóa constant pool sẽ hạn chế thực thi của những luồng khác.

Việc cài đặt JVM tối ưu tìm kiếm các trường và các phương thức của đối tượng bằng việc điều chỉnh bytecode khi thực thi chương trình. Cụ thể, các chỉ lệnh bytecode mà tham chiếu đến constant pool được thành các quick bytecode tương đương, mà tham chiếu đến địa chỉ tuyệt đối của trường hoặc phương thức. Chẳng hạn, chỉ lệnh bytecode

```
getfield #18 <Field Obj var>
```

đẩy giá trị của trường đối tượng vào ngăn xếp toán hạng. (Đối tượng mà trường của nó đang truy vấn là đối tượng ở trên đỉnh của ngăn xếp toán hạng trước khi thao tác này

được thực hiện). Khi chỉ lệnh `getfield` được thực hiện, sẽ tiến hành tìm kiếm tên ký hiệu của trường mà sánh với xâu ở vị trí #18 trong constant pool. Nếu trường được tìm thấy, và nó sẽ định vị 6 byte dưới vị trí đầu tiên của đối tượng, và chỉ lệnh trên sẽ được thay thế bằng phiên bản nhanh như sau

`getfield_quick 6`

mà sử dụng offset được tính toán của trường từ đỉnh của đối tượng. Nếu chương trình được truyền qua chỉ lệnh này nữa, như chỉ lệnh xuất hiện trong vòng lặp, chỉ lệnh nhanh sẽ được thực thi. Bạn không cần hiểu mọi phần của chỉ lệnh `getfield`, vấn đề quan trọng ở đây là cách mà thay đổi chỉ lệnh bytecode lần đầu thực hiện để làm cho chương trình chạy nhanh hơn nếu chỉ lệnh được sử dụng lại.

Tìm phương thức ảo của lớp

Có bốn loại bytecode khác nhau để triệu gọi phương thức:

- `invokevirtual`: được sử dụng khi lớp cha của đối tượng được biết trong thời gian dịch
- `invokeinterface`: được sử dụng chỉ khi giao diện của đối tượng được biết trong thời gian dịch
- `invokestatic`: được sử dụng để triệu gọi phương thức tĩnh
- `invokespecial`: được sử dụng trong một số trường hợp đặc biệt mà không bàn ở đây.

Trong thuật ngữ đơn giản nhất, `invokevirtual` được sử dụng trong các tình huống mà hàm ảo giống C++ gọi, vì Java là ngôn ngữ lập trình kiểu tĩnh mà mọi lớp con của lớp có thể được cài đặt với cùng thứ tự của bảng phương thức (vtable), `invokeinterface` tương tự như tìm kiếm phương thức Smalltalk, là giao diện được cài đặt bởi lớp của đối tượng không xác định vị trí tương đối của phương thức trong bảng phương thức. chúng ta sẽ xem `invokevirtual` trước khi xét các tính chất của `invokeinterface`.

Giống như `getfield`, `invokevirtual` bao gồm cả việc tìm kiếm tên ký hiệu khi chỉ lệnh thực hiện lần đầu. Cụ thể, giả sử ta có mã nguồn Java khai báo tham chiếu đối tượng x và triệu gọi phương thức của x:

```
Object x;  
...  
x.equals("test");
```

Để đảm bảo tính đúng đắn, và vì điều này ảnh hưởng đến cách mã được dịch, ta giả thiết rằng x là biến cục bộ đầu tiên trong khối. Lờ gọi `x.equals("test")` được dịch vào dãy bytecode như sau, trong đó các câu bên phải ; là các chú thích.

```

aload_1 ; push local variable 1 (which is 'x') onto the operand stack
ldc "test" ; push the string "test" onto the operand stack
invokevirtual java/lang/Object/equals(Ljava/lang/Object;)Z

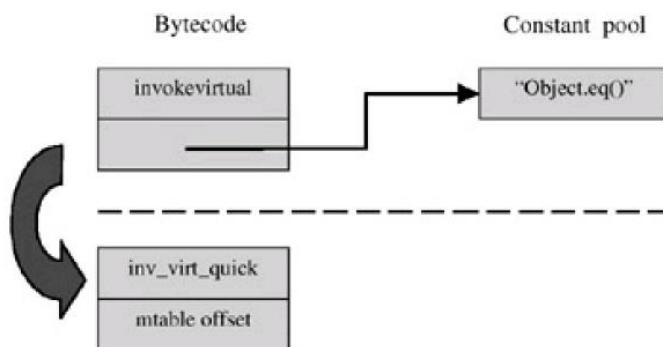
```

Xâu `java/lang/Object/equals(Ljava/lang/Object;)Z` được gọi là đặc tả phương thức. Xâu này không lưu theo nghĩa đen trong bytecode, mà lưu trong constant pool như mô tả trước đây.

Khi `invokevirtual` thực hiện, máy ảo tìm đặc tả phương thức và các thông tin liên quan trong file class đó và xác định phương thức yêu cầu bao nhiêu đối số. Đối với phương thức `equals` trong ví dụ của chúng ta ở đây chỉ có một đối số. Đối số và tham chiếu đối tượng, trong trường hợp này số 1 chỉ rõ biến cục bộ đầu tiên trong phạm vi hiện tại, được kéo ra khỏi ngăn xếp toán hạng. Sử dụng thông tin lưu trong tham chiếu đối tượng (biến cục bộ `x`) máy ảo java truy vấn lớp java cho đối tượng, tìm danh sách phương thức được định nghĩa bởi lớp đó và sau đó là lớp cha của nó, tìm kiếm phương thức mà sánh với đặc tả của phương thức. Khi đã định vị được phương thức đúng, phương thức được gọi theo cùng cách với lời gọi hàm được thực hiện trong hầu hết các ngôn ngữ lập trình cấu trúc khối.

Bytecode Rewriting for `invokevirtual`. Mặc dù quá trình tìm kiếm phương thức sẽ tìm được và gọi đúng phương thức, nó không có được tính ưu việt của thông tin kiểu tĩnh được tính bởi chương trình dịch Java trong thời gian dịch. Cụ thể, chương trình dịch Java có thể sắp xếp bảng phương thức của mỗi lớp con theo cùng một cách như bảng của lớp cha như chương trình dịch C++ đã làm. Bằng cách như vậy, chương trình dịch đảm bảo rằng mỗi phương thức được định vị ở cùng vị trí tương đối trong mọi bảng phương thức gắn kết với các lớp con của một lớp bất kỳ. Do đó mỗi khi, tìm kiếm phương thức được sử dụng để tìm phương thức trong bảng phương thức, offset của phương thức này sẽ là như nhau đối với mọi lần thực hiện trong tương lai của chỉ lệnh đó.

Hình 8.5 chỉ rõ bytecode có thể được điều chỉnh như thế nào những lần thực hiện sau đó của chỉ lệnh `invokevirtual`, phương thức đúng đắn được tìm thấy ngay trong bảng phương thức của đối tượng (được gọi là `mtable`)



Hình 8.5. Tối ưu `invokevirtual` bằng cách viết lại bytecode.

Phần của Hình 8.5 trên đường đứt nét chỉ ra lệnh `invokevirtual` trong dòng chỉ lệnh được nối tiếp bởi con trỏ đến đặc tả phương thức trong constant pool. Sau khi phương thức được tìm thấy, dòng chỉ lệnh có thể được chỉnh sửa như nêu ra dưới đường đứt nét. Trong dòng chỉ lệnh hiệu chỉnh, `invokevirtual` được thay bằng phiên bản nhanh `invokevirtual_quick` và con trỏ đặc tả phương thức được thay bằng offset của phương thức trong bảng phương thức. Khi `invokevirtual_quick` được thực hiện, máy ảo sẽ sử dụng tham chiếu đối tượng trong ngăn xếp toán hạng để định vị lớp và bảng phương thức và sau đó dùng mtable offset trong dòng chỉ lệnh để tìm phương thức thích hợp trong mà không cần tìm kiếm gì nữa.

Quan trọng để hiện thực việc nếu triệu gọi phương thức là trong vòng lặp, chẳng hạn, thì các đối tượng khác nhau có thể ở trên ngăn xếp toán hạng cho việc thực hiện khác nhau của lời gọi phương thức. Như kết quả, lớp của đối tượng có thể thay đổi từ lần gọi này đến lần tiếp theo. Tuy nhiên, offset của phương thức trong bảng phương thức sẽ là như nhau trong mỗi lời gọi, vì mỗi lớp sẽ là lớp con của kiểu tĩnh của tham chiếu đối tượng trong mã nguồn Java.

Tìm phương thức ảo bằng giao diện

Tình huống cũng tương tự như tìm phương thức khi giao diện đã được biết trong thời gian dịch. Điểm khác là tối ưu dùng `invokevirtual_quick` sẽ không làm việc đúng nữa. Do đó kỹ thuật khác được sử dụng.

Giả sử chương trình đã dịch của Java chứa khai báo phương thức

```
void add2(Incrementable x) { x.inc(); x.inc(); }
```

Trong đó `incrementable` là giao diện đảm bảo rằng là đối số `x` có phương thức `inc` khi `add2` được gọi trong thời gian chạy. Ở đây có thể có một số lớp cài đặt giao diện `incrementable`. Chẳng hạn có các lớp `IntCounter` và `FloatCounter` dạng sau:

```
interface Incrementable {
    public void inc();
}
class IntCounter implements Incrementable {
    public void add(int);
    public void inc();
    public int value();
}
class FloatCounter implements Incrementable {
    public void inc();

    public void add(float);
    public float value();
}
```

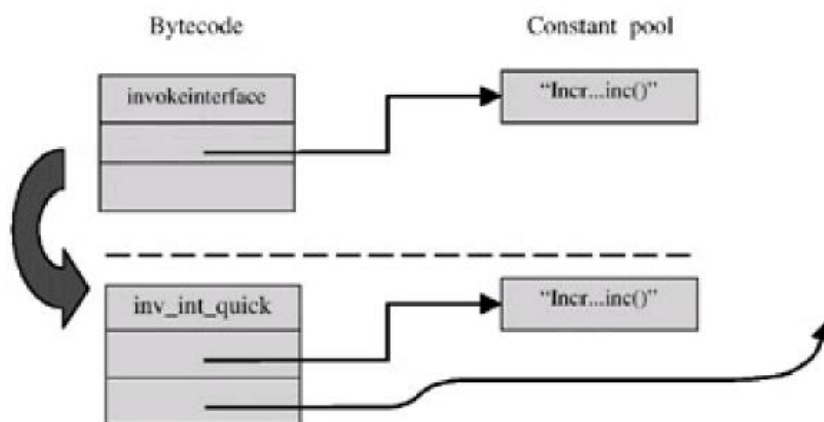
Vì các lớp cài đặt incrementable có thể được khai báo trong các gói khác nhau, không cái nào tham chiếu đến cái khác, không có lý do gì để tin rằng phương thức inc được định vị tại các offset giống nhau trong hai bảng phương thức. (Trên thực tế không khó xây dựng tập các giao diện và lớp mà đối với chúng không có cách nào để làm cho mọi lớp mà cài đặt mỗi giao diện được tương đồng với nhau theo cách như vậy).

Khi phương thức add2 được dịch, chương trình dịch Java sinh ra code ở dạng sau (với đặc tả phương thức từ constant pool được viết ở đây cho rõ ràng).

```
aload_1 ; push local variable 1 (i.e. x) onto the stack
invokeinterface package/inc()Z 1
```

Khi invokeinterface được thực hiện, lớp của đối tượng được tìm thấy và bảng phương thức cho lớp này được tìm kiếm để xác định phương thức với đặc tả đã cho. Khi phương thức được định vị, phương thức sẽ được gọi như đối với invokevirtual.

Vì offset của phương thức này trong bảng phương thức có thể khác trong lần thực thi tiếp theo của chỉ lệnh, nên sẽ là không đúng nếu viết lại invokeinterface thành invokevirtual_quick. Tuy nhiên, ở đây có khả năng là lần thực thi tiếp theo của chỉ lệnh sẽ dùng cho một đối tượng của cùng lớp đó, nên có cảm tưởng là lãng phí nếu bỏ qua offset và thông tin khác. Hình 8.6 chỉ ra việc điều chỉnh bytecode được sử dụng cho invokeinterface.



Hình 8.6: Tối ưu invokeinterface bằng cách viết lại bytecode

Phần trên đường gạch chấm của Hình 8.6 chỉ ra dãy chỉ lệnh ban đầu, với chỉ lệnh `invokeinterface` được tiếp nối bởi con trỏ đến đặc tả phương thức trong constant pool và vị trí trống chưa sử dụng. Sau khi thực hiện `invokeinterface`, chỉ lệnh này sẽ được thay thế bởi `invokeinterface_quick` và con trỏ đến lớp và phương thức được lưu ở bên dưới đặc tả phương thức. Khi `invokeinterface_quick` được thực hiện trên lần duyệt khác của chỉ lệnh này, lớp của toán hạng đối tượng sẽ được so sánh với lớp đang cached. Nếu đối tượng cũng từ lớp đó, thì cũng phương thức đó được sử dụng. Nếu đối tượng từ lớp khác, thì vì đặc tả phương thức vẫn còn trong dãy chỉ lệnh này, tiến trình tìm kiếm sẽ nối tiếp như là chỉ lệnh được `invokeinterface` lại.

Tóm tắt tính hiệu quả. Trong Java, lời gọi phương thức là phải tìm kiếm, lần đầu chỉ lệnh thực hiện chậm hơn nhiều so với các lần sau đó. Trong tìm kiếm lần đầu mà sử dụng `invokevirtual` có hai hướng giải quyết (một tìm kiếm bảng constant của lớp mà phương thức ở đó và cách hai tìm kiếm bảng phương thức của lớp của đối tượng), hai chỉ số mảng (trong bảng constant và danh sách phương thức), và giá viết lại bytecode đó. Trong các lần gọi tiếp theo, chỉ có một cách và một chỉ số. Đối với `invokeinterface`, có sự tìm kiếm theo bảng phương thức khi lần đầu chỉ lệnh thực hiện. Trong các lần gọi tiếp theo, có một chỉ số mảng hoặc tìm kiếm khác trên bảng phương thức nếu lớp bị thay đổi.

BÀI 9: LẬP TRÌNH SONG SONG VÀ PHÂN TÁN

Tổng quan

Một chương trình song song định nghĩa hai hay nhiều hơn các dãy hành động mà có thể được thực hiện một cách đồng thời. Các chương trình song song có thể được thực hiện theo hai cách tổng quát sau:

- *Đa lập trình (Multiprogramming)*. Một bộ xử lý duy nhất có thể chạy một số tiến trình đồng thời bằng việc đan xen các bước của một tiến trình với các bước của tiến trình khác. Mỗi tiến trình riêng biệt sẽ xử lý tuần tự, nhưng các hành động của một tiến trình có thể xảy ra giữa hai buowsc kế nhau của tiến trình khác.
- *Đa xử lý (Multiproceeing)*. Hai hoặc nhiều hơn bộ xử lý có thể chia sẻ bộ nhớ hoặc được kết nối với mạng, cho phép các tiến trình trên một bộ xử lý tương tác với các tiến trình khác chạy đồng thời trên bộ xử lý khác.

Tính song song là quan trọng vì một số lý do. Song song cho phép các nhiệm vụ khác nhau được xử lý với các tốc độ khác nhau. Chẳng hạn, đa lập trình cho phép một chương trình làm một việc có ích, trong khi đó một chương trình khác đang chờ để nhận đầu vào. Nó làm cho việc sử dụng hiệu quả hơn đối với một bộ xử lý. Song song cũng cung cấp các khái niệm lập trình mà là quan trọng trong giao diện người sử dụng, như hệ thống window mà hiển thị nhiều cửa sổ đồng thời và đối với các hệ thống mạng cần thiết phải gửi và nhận dữ liệu từ nhiều máy tính tại các thời điểm khác nhau. Đa xử lý làm cho sức mạnh xử lý lớn hơn sẵn sàng giải quyết một bà toán tính toán và đưa ra các vấn đề bổ sung như độ không tin cậy trong truyền thông mạng và khả năng một khi bộ xử lý đang thực thi thì cái khác đang bị sự cố. Tương tác giữa các đoạn chương trình tuần tự, cho dù chúng ở trên cùng một bộ xử lý hay trên các bộ xử lý khác nhau, nảy sinh các thách thức lập trình đáng kể.

Các ngôn ngữ lập trình song song cung cấp các trừu tượng kiểm soát và truyền thông để viết chương trình song song. Trong chương này, chúng ta xét một số vấn đề tổng quan trong lập trình song song và đặc biệt lập trình song song trong Java. Một số cấu trúc chúng ta xem xét là phù hợp hơn với đa lập trình, một số khá là đa xử lý, và một số là hữu ích cho cả hai.

Đã có nhiều hệ thống song song được viết trong các ngôn ngữ mà không hỗ trợ song song. Chẳng hạn, các tiến trình song song được sử dụng để cài đặt định tuyến mạng có thể được viết trong ngôn ngữ như C, với các lời gọi hệ thống được sử dụng tại nơi của các cấu trúc song song. Các chương trình tuần tự được thực thi song song có thể truyền thông bằng việc đọc và viết vào một cấu trúc chia sẻ hoặc bằng việc sử dụng các trừu tượng được cung cấp bởi hệ điều hành. Các chương trình có thể gửi và nhận dữ liệu từ Unix pipe, chẳng hạn, bởi các lời gọi hàm mà được kiểm soát bởi hệ điều hành. Các chương trình C có thể cũng tạo các tiến trình và dùng chúng bằng các lời gọi hệ thống.

Ở đây có một số lý do tại sao ngôn ngữ lập trình song song có thể là công cụ giải quyết bài toán tốt hơn cho lập trình song song so với một ngôn ngữ lập trình tuần tự. Một là ngôn ngữ lập trình song song có thể cung cấp trừu tượng và cấu trúc điều khiển mà được thiết kế chuyên dụng cho lập trình song song. Cái nữa là ngôn ngữ lập trình có thể cung cấp các tiến trình nhẹ. Các hệ điều hành nói chung cho mỗi tiến trình một không gian địa chỉ riêng, với ngăn xếp thời gian chạy, bộ đếm chương trình và đồng riêng. Tuy điều này bảo vệ tiến trình, nhưng có chi phí đáng kể gắn kết với việc thiết lập một tiến trình và chuyển luồng điều khiển từ một tiến trình sang cái khác. Cài đặt ngôn ngữ lập trình có thể cung cấp các tiến trình nhẹ mà chạy trên cùng một không gian địa chỉ của hệ điều hành, nhưng được bảo vệ bởi các tính chất của ngôn ngữ lập trình. Với các tiến trình nhẹ, ở đây có ít chi phí hơn cho lập trình song song, cho phép lập trình viên sử dụng song song thoải mái hơn. Cuối cùng, có một vấn đề chuyển mang qua các hệ điều hành. Mặc dù là có thể lập trình song song với C và lời gọi hệ thống Unix, chẳng hạn, các chương trình được viết với một phiên bản của Unix có thể không chạy đúng đắn trên phiên bản khác của Unix và có thể không dễ dàng chuyển mang đến hệ điều hành khác.

9.1 Các khái niệm cơ bản trong lập trình song song

Trước khi xem xét thiết kế ngôn ngữ cụ thể, chúng ta bàn luận về một số vấn đề cơ bản trong lập trình song song và một số cơ chế hệ điều hành truyền thống được sử dụng cho phép các tiến trình song song tương tác hiệu quả.

Một dãy các hành động có thể được gọi là một tiến trình, luồng hay một nhiệm vụ. Từ tiến trình thường được sử dụng tham chiếu đến một tiến trình của hệ điều hành, mà nói chung chạy trong không gian địa chỉ riêng của nó. Các luồng, trong ngôn ngữ chuyên biệt như CML hay Java. Có thể chạy dưới điều khiển của hệ thống thời gian thực của ngôn ngữ, chia sẻ cùng một không gian địa chỉ của hệ điều hành. Một số người gọi luồng là tiến trình nhẹ, có nghĩa là tiến trình mà không chạy trong không gian địa chỉ riêng. Các luồng trong chương trình Java là các tiến trình nhẹ.

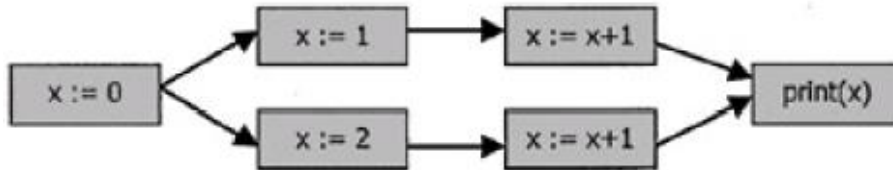
9.1.1. Thứ tự thực thi và tính không tất định

Các chương trình song song có thể có nhiều thứ tự thực hiện khác nhau. Một cấu trúc lập trình song song đơn giản có tính lịch sử là dạng *cobegin/coend* được sử dụng trong Concurrent Pascal. Ngôn ngữ này được phát triển bởi Brinch Hansen ở Caltech trong những năm 1970. Đây là ví dụ chương trình Concurrent Pascal rằng nó có một số thứ tự thực thi :

```
x:=0;
cobegin
begin x := 1; x := x+1 end;
begin x := 2; x := x+1 end;
coend;
print(x);
```

Chương trình thực hiện tuần tự như trong Pascal, ngoại trừ bên trong lệnh *cobegin/coend*, mọi khối độc lập được thực thi song song. Các khối song song có thể tương tác bằng cách đọc và gán cho các biến tổng thể. Lệnh *cobegin/coend* kết thúc, khi mọi khối song song bên trong kết thúc. Sau khi lệnh *cobegin/coend* kết thúc, chương trình tiếp tục tuần tự với các lệnh sau *coend*.

Sau đây là lược đồ chỉ ra chương trình song song của chúng ta có thể được thực thi như thế nào trên một bộ xử lý đơn. Mỗi lệnh gán được thực hiện không phân tách, theo nghĩa mặc dù phép gán được phân tách thành một số bước ngôn ngữ máy cơ sở, nhưng một khi bộ xử lý bắt đầu thực hiện lệnh gán, thì nó tiếp tục và kết thúc lệnh gán trước khi cho phép tiến trình khác được thực thi. Nói cách khác, chỉ một lệnh gán chạy tại một thời điểm. Tuy nhiên, thứ tự giữa các lệnh gán có thể là khác nhau dẫn đến thực thi khác nhau của chương trình. Mỗi mũi tên trên Hình mô tả ràng buộc thứ tự thực thi: lệnh ở đuôi bên trái của mũi tên cần kết thúc trước khi lệnh bên phải mũi tên bắt đầu. Tuy nhiên, không có thứ tự cần thiết giữa các lệnh mà không kết nối bởi dây các mũi tên. Cụ thể, lệnh gán $x := 1$ có thể được thực thi trước hoặc sau lệnh gán $x := 2$. Ở đây có một số thứ tự thực thi có thể có của chương trình này:



$x:=0; x:=1; x:=2; x:=x+1; x:=x+1; \text{print}(x)$ // Output 4
 $x:=0; x:=2; x:=1; x:=x+1; x:=x+1; \text{print}(x)$ // Output 3
 $x:=0; x:=1; x:=x+1; x:=2; x:=x+1; \text{print}(x)$ // Output 3
 $x:=0; x:=2; x:=x+1; x:=1; x:=x+1; \text{print}(x)$ // Output 2

Chương trình này mô tả một trong những vấn đề khó chính trong thiết kế và kiểm thử chương trình song song: tính không tất định. Một chương trình được gọi là tất định, nếu đối với mỗi dãy đầu vào chương trình, ở đây có một dãy hành động chương trình và cho ra kết quả. Ngược lại chương trình gọi là không tất định, nếu có nhiều hơn một dãy các hành động tương ứng với một dãy đầu vào duy nhất. Chương trình Pascal song song ví dụ của chúng ta là không tất định, vì thứ tự thực hiện các lệnh bên trong cấu trúc *cobegin/coend* là không xác định. Ở đây có một số thứ tự thực thi có thể và nếu chương trình chạy một số lần trên cùng một máy, nó có thể tạo ra các kết quả khác nhau cho các lần chạy khác nhau.

Tính không tất định làm cho nó khó thiết kế và bắt lỗi chương trình. Trong các hệ thống phần mềm phức tạp, có thể có nhiều thứ tự thực thi có thể. Khó cho người thiết kế chương trình nghĩ cẩn thận về hàng triệu thứ tự thực thi có thể và xác định lỗi mà có thể xảy ra trong một số nhỏ trong chúng. Và là không thực tế kiểm thử hành vi chương trình trong mọi thứ tự thực thi có thể.

9.1.2 Trao đổi, phối hợp và tính không phân tách.

Mỗi ngôn ngữ lập trình đối với song song tường minh cung cấp cơ chế nào đó để khởi đầu và kết thúc các tiến trình tuần tự độc lập. Chẳng hạn,

- *trao đổi* giữa các tiến trình, đạt được bằng các cơ chế như lưu bộ đệm hoặc kênh trao đổi đồng bộ, loan báo hoặc biến, đối tượng chia sẻ.
- *phối hợp* giữa các tiến trình, mà có thể tường minh hoặc ẩn buộc một tiến trình phải chờ tiến trình khác trước khi tiếp tục.
- Tính không phân tách: mà tác động cả đến tương tác giữa các tiến trình và kiểm soát các điều kiện lỗi.

Khi chúng ta so sánh các ngôn ngữ lập trình song song, chúng ta thường xem cách các tiến trình được định nghĩa, cách chúng trao đổi và phối hợp các hành động, và cách các lập trình viên có thể định nghĩa các hành động không phân tách trong ngôn ngữ đó.

Dạng cơ bản nhất của trao đổi bên trong tiến trình, như mô tả trong ví dụ Pascal song song là thông qua biến chia sẻ. Các tiến trình có thể trao đổi thông qua cấu trúc dữ liệu chia sẻ hoặc file. Một dạng khác của trao đổi tiến trình bên trong là *truyền thông điệp*. Ở đây có một khác biệt chính giữa các dạng của cơ chế truyền thông điệp:

- *Bộ đệm*. Nếu trao đổi được lưu ở bộ đệm, thì mỗi mục dữ liệu được gửi sẽ sẵn sàng cho đến khi nó được nhận, Trong truyền thông không bộ đệm, một mục dữ liệu được gửi trước khi người nhận sẵn sàng có thể bị mất.
- *Đồng bộ*. Trong trao đổi đồng bộ, người gửi không thể truyền mục dữ liệu trừ khi người nhận đã sẵn sàng nhận nó. Với trao đổi không đồng bộ, tiến trình gửi có thể truyền một mục dữ liệu và tiếp tục thực hiện ngay cả khi người nhận không sẵn sàng nhận dữ liệu.
- *Thứ tự thông điệp*. Một cơ chế trao đổi có thể bảo toàn thứ tự truyền hoặc có thể không. Nếu một cơ chế bảo toàn thứ tự truyền, thì dãy thông điệp sẽ được nhận theo thứ tự mà chúng được gửi.

Các cơ chế điều phối cho phép một tiến trình chờ tiến trình khác hoặc cảnh báo một tiến trình đang chờ là nó có thể tiếp tục. Lệnh Pascal song song *cobegin/coend* cung cấp một dạng điều phối tiến trình sơ đẳng là mọi tiến trình bắt đầu tại cùng một *cobegin* sẽ phải kết thúc trước khi lệnh *coend* được thực thi. Khóa và cờ sẽ được bàn trong các mục sau cung cấp dạng điều khiển song song tốt hơn.

Một hành động là không phân tách, nếu mỗi thực hiện sẽ hoặc là kết thúc thành công hoặc kết thúc ở trạng thái mà không khác so với trạng thái mà lúc hành động bắt đầu. Một hành động không là không phân tách được có thể có các trạng thái trung gian mà quan sát được bởi các tiến trình khác. Một hành động không là không phân tách được cũng có thể dừng ở lỗi trước khi hành động kết thúc, để lại vết nào đó của tiến trình bộ phận. Nói một cách tổng quát, bất cứ ngôn ngữ lập trình song song nào cần cung cấp một số hành động không phân tách, vì nếu không đảm bảo điều đó, rất khó đoán hành vi của bất cứ chương trình nào.

9.1.3 Loại trừ đa bên và khóa

Có một số tình huống lập trình mà ở đó hai hay nhiều hơn các tiến trình chia sẻ một số cấu trúc dữ liệu, và một sự phối hợp nào đó là cần thiết để duy trì tính đúng đắn của cấu trúc dữ liệu đó. Ở đây có một ví dụ đơn giản mà mô tả một số vấn đề.

Ví dụ 9.1

Giả sử chúng ta có một hệ thống mà cho phép mọi người đăng ký vào một danh sách. Hệ thống này có một số danh sách người dùng, và một số danh sách này có thể được truy cập bởi nhiều hơn một tiến trình. Đây là thủ tục để bổ sung một người vào một danh sách:

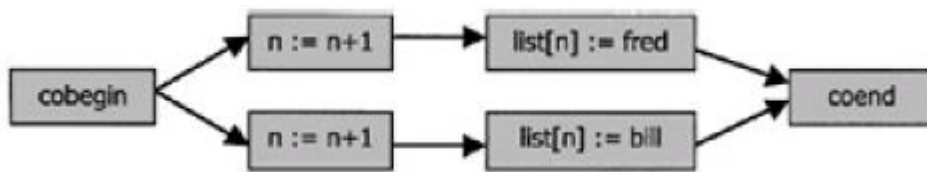
```
procedure sign-up(person)
begin
  n:=n + 1;
  list[n] := person;
end;
```

Thủ tục này dùng một mảng để lưu các tên trong danh sách và một số nguyên chỉ vị trí của người cuối cùng đã được bổ sung vào danh sách. Ở đây có thể có các thủ tục khác đọc các tên từ danh sách hoặc loại bỏ họ.

Thủ tục đăng ký làm việc đúng chỉ khi một tiến trình bổ sung vào danh sách ở một thời điểm, nhưng không phải hai tiến trình cùng bổ sung vào danh sách đồng thời. Chẳng hạn, xét một đoạn chương trình sau:

```
cobegin
sign-up(fred);
sign-up(bill);
end;
```

Như đã bàn ở mục trước, ở đây có một số thứ tự thực thi có thể đối với các phép gán số và danh sách, thứ tự giữa các phép gán được chỉ ra trong hình sau:



Một thứ tự thực thi có thể tăng n hai lần, sau đó viết tên fred và bill vào cùng một chỗ. Điều này là không đúng: thay vì bổ sung cả hai tên vào danh sách, chúng ta kết thúc với một vị trí trống, và chỉ một trong hai tên có trong danh sách. Trạng thái kết quả từ dãy các phép gán

$n := n+1; n := n+1; list[n] := fred; list[n] := bill;$

được gọi là *trạng thái không tương thích*, vì nó không đúng đắn với hành vi mong muốn của thao tác trên các danh sách. Một bất biến đối với danh sách và biến nguyên n là với mỗi số nguyên i cho đến n ở đây có dữ liệu lưu ở $list[i]$. Bất biến này là được bảo toàn bởi thủ tục *sign-up*, nếu nó được gọi tuần tự. Tuy nhiên, nếu nó được gọi song song, bất biến của cấu trúc dữ liệu có thể bị phá hủy.

Loại trừ đa bên

Ví dụ 9.1 chỉ ra rằng, đối với một số thao tác, điều quan trọng là hạn chế song song sao cho chỉ có một tiến trình tiến hành tại một thời điểm. Điều này dẫn đến khái niệm *vùng tới hạn*. Một vùng tới hạn là một đoạn chương trình mà truy cập các tài nguyên chung. Vì một tiến trình trong vùng tới hạn có thể phá vỡ tiến trình khác. Điều quan trọng là chỉ cho phép một tiến trình vào một vùng tới hạn tại một thời điểm. Cơ chế bảo vệ các hành động xung đột trong các tiến trình độc lập được gọi là cơ chế loại trừ đa bên.

Các cơ chế loại trừ đa bên có thể được thiết kế để thỏa mãn một số hoặc tất cả các tiêu chuẩn sau:

- *Loại trừ đa bên*: chỉ có một tiến trình tại một thời điểm có thể được ở trong vùng tới hạn.
- *Bước tiến*: nếu không có tiến trình nào trong vùng tới hạn và có một tiến trình nào đó muốn vào vùng tới hạn, thì nó có thể là một tiến trình đang chờ để vào vùng tới hạn.
- *Chờ đợi có giới hạn*: nếu có một tiến trình đang chờ, nó cần bị giới hạn một thời gian để các tiến trình khác mà đang được phép vào vùng tới hạn trước khi tiến trình này được phép vào. Nói cách khác, không tiến trình đang chờ nào phải chờ vô hạn. Cũng là mong muốn để tin tưởng rằng, nếu một tiến trình dừng trong vùng giới hạn, các tiến trình khác có thể chắc chắn sẽ vào được vùng giới hạn.

Khóa và hàng chờ nhận việc

Cách tiếp cận tổng quan cho loại trừ đa bên được chỉ ra trong đoạn giả code sau, mà sử dụng thủ tục đăng ký như xét ở mục trước. Ý tưởng chính là mỗi tiến trình thực hiện một kiểu hành động chờ *wait* trước khi gọi *sign-up* và thực hiện một kiểu hành động *signal* sau đó. Hành động chờ *wait* cần được kiểm tra để xem có tiến trình nào khác gọi *sign-up* hoặc đang thực hiện một thủ tục khác mà can thiệp với *sign-up*. Nếu vậy, hành động này làm cho tiến trình đó phải chờ, chỉ cho phép gọi *sign-up* xảy ra khi điều này là chấp nhận được. Hành động *signal* cho phép tiến trình chờ được tiếp tục tiến.

```
<initialize concurrency control>
cobegin
  begin
    <wait> // wait for access
    sign-up(fred); // critical section
    <signal> // allow waiting process to proceed
  end;
  begin
    <wait> // wait for access
    sign-up(bill); // critical section
    <signal> // allow waiting process to proceed
  end;
end;
```

Mặc dù các ý tưởng dưới các hành động *wait* và *signal* có thể trông có vẻ trực tiếp, cài đặt các hành động này là rắc rối. Trên thực tế, thời kỳ đầu lập trình song song, một số cài đặt không đúng được đề xuất trước khi giải pháp tốt cuối cùng được khám phá. Chúng ta có thể hiểu sự tinh vi bằng cách kiểm tra cài đặt trông hợp lý, nhưng không đúng.

Ví dụ 9.2

Giả sử chúng ta cài đặt *wait* và *signal* bằng việc sử dụng một biến nguyên thông thường như một “khóa” (*lock*). Mỗi tiến trình sẽ kiểm tra biến đó trước khi vào vùng tới hạn của nó. Nếu tiến trình khác đã có trong vùng tới hạn, thì khóa này sẽ chỉ ra rằng tiến trình đó phải chờ. Nếu giá trị của khóa chỉ ra rằng tiến trình có thể vào, thì tiến trình sẽ khởi tạo khóa để ngăn các tiến trình khác. Cách tiếp cận này được chỉ ra trong đoạn code sau:

```
lock := 0;
cobegin
  begin
    while lock=1 do end; // loop doing nothing until lock is 0
    lock:=1; // set lock to enter critical section
    sign-up(fred); // critical section
    lock:=0; // release lock
  end;
  begin
    while lock=1 do end; // loop doing nothing until lock is 0
    lock:=1; // set lock to enter critical section
    sign-up(bill); // critical section
    lock := 0; // release lock
  end;
end;
```

Mỗi tiến trình kiểm tra khóa trước khi vào vùng tới hạn (tức là trước khi gọi *sign-up*). Nếu *lock* là 1, chỉ ra rằng một tiến trình khác đang sử dụng *sign-up*, thì tiến trình sẽ lặp cho đến khi *lock* là 0. Khi *lock* là 0, tiến trình sẽ vào vùng tới hạn, đặt *lock* := 1 khi vào và *lock* := 0 khi ra. Sử dụng vòng lặp để chờ một điều kiện nào đó được gọi là chờ nhận việc (*busy waiting*).

Vấn đề với sử dụng một biến chia sẻ để loại trừ đa bên ở chỗ thao tác đọc một giá trị của một biến là khác với thao tác đặt giá trị cho biến đó. Với một phương thức chỉ ra trong code, điều này có thể là một tiến trình kiểm tra một biến và nhìn thấy khóa đang mở, nhưng trước khi tiến trình khóa ngăn tiến trình khác, tiến trình khác cũng nhìn thấy khóa mở và đã vào vùng tới hạn. Điều này cho phép hai tiến trình cùng gọi *sign-up* vào cùng một thời điểm.

Một ý tưởng cơ bản điều khiển song song là *kiểm tra và đặt không tách rời*. Một *kiểm tra và đặt không tách rời*, đôi khi được gọi là TSL (*Test and Set Lock*), sao chép giá trị của biến khóa và đặt nó một giá trị khác không (khóa lại), mọi cái đó trong một bước không tách rời được. Trong khi giá trị của biến khóa đang được kiểm tra, không tiến trình khác nào có thể vào vùng kiểm tra của nó. Hầu hết các bộ xử lý hiện đại cung cấp một dạng *kiểm tra và đặt* trong tập chỉ lệnh của chúng.

Tắc nghẽn (Deadlock)

Tắc nghẽn xảy ra khi một tiến trình không bao giờ có thể tiếp tục được vì trạng thái của một tiến trình khác. Tắc nghẽn có thể dễ dàng xảy ra nếu có hai tiến trình và hai khóa. Giả sử tiến trình 1 trước hết lấy khóa 1 và sau đó chờ khóa 2. Nếu tiến trình 2 trước hết lấy khóa 2 và sau đó chờ khóa 1, thì cả hai tiến trình đều đang chờ khóa được giữ bởi tiến trình kia. Trong tình huống này, không tiến trình nào có thể tiếp tục và tắc nghẽn đã xảy ra.

Một kỹ thuật để ngăn tắc nghẽn được gọi là *khóa hai pha*. Trong khóa hai pha, một tiến trình được xem như một dãy các nhiệm vụ độc lập. Với mỗi nhiệm vụ, một tiến trình trước hết cần chiếm mọi khóa mà là cần thiết để thực hiện nhiệm vụ đó. Trước khi tiếp tục từ một nhiệm vụ sang nhiệm vụ tiếp theo, một tiến trình cần phải giải phóng mọi khóa. Như vậy có hai pha trong việc sử dụng khóa, pha chiếm khóa ở đó mọi khóa được yêu cầu và pha giải phóng ở đó mọi khóa được nhả ra. Để tránh tắc nghẽn trong pha chiếm khóa, mọi tiến trình đồng ý một thứ tự các khóa và phải chiếm khóa theo thứ tự đó.

9.1.4 Cờ - Semaphore

Semaphore, đầu tiên được đề xuất bởi Edsger W.Dijkstra trong năm 1968, cung cấp một cách cho hệ điều hành đảm bảo loại trừ đa bên mà không có đợi chờ việc. Có một số cài đặt chi tiết khác nhau của semaphore. Tất cả đều sử dụng một nộ đếm hoặc một cờ Bool và yêu cầu hệ điều hành thực hiện toàn bộ thủ tục wait hoặc signal một cách không tách rời (nguyên vẹn). Tính nguyên vẹn ngăn các lệnh rời của một thủ tục wait khỏi việc bị đan xen với các lệnh rời của một thủ tục wait khác trên cùng một semaphore.

Một semaphore chuẩn được thể hiện bởi một biến nguyên, một số nguyên tối đa và một hàng đợi các tiến trình đang chờ. Khởi đầu, biến nguyên được đặt bằng số nguyên tối đa. Số tối đa chỉ ra số tiến trình có thể vào vùng tới hạn cùng một thời điểm; trong nhiều tình huống số tối đa bằng một. Nếu một chương trình chứa một số tài nguyên chia sẻ, thì chương trình có thể sử dụng một semaphore riêng cho mỗi tài nguyên.

Khi một tiến trình chờ một semaphore, thao tác chờ wait kiểm tra giá trị nguyên của semaphore. Nếu giá trị là lớn hơn 0, điều đó chỉ ra rằng tiến trình có thể tiếp tục. Giá trị này được giảm trước trước khi xử lý để hạn chế số tiến trình mà cho phép tiếp tục. Nếu giá trị cực đại bằng 1, thì semaphore sẽ giảm về 0 và không tiến trình nào được tiếp tục.

Một chờ đợi được thực hiện trên semaphore, nếu giá trị nguyên của nó bằng 0, khi đó tiến trình được ngắt và đặt vào hàng đợi của các tiến trình đang đợi. Ngắt một tiến trình là thao tác của hệ điều hành mà giữ cho tiến trình không được tiếp tục cho đến khi tiến trình được khôi phục lại. Sau đây là giả code chỉ ra semaphore được cài đặt như thế nào:

```
procedure wait (s : Semaphore)
begin // This procedure must be executed atomically
  if s.value > 0 then
```

```

        s.value := s.value - 1; // Enter section and decrement counter
    else
        suspend_on (s.queue); // Wait for other processes to finish
    end;

```

Khi một tiến trình rời vùng tới hạn, nó cần cho phép các tiến trình khác đang đợi semaphore gắn kết đó được tiếp tục. Thao tác *signal* kiểm tra hàng đợi semaphore xem có bất cứ tiến trình nào bị ngắt không. Nếu một hàng đợi là khác rỗng, một trong các tiến trình bị ngắt đang chờ sẽ được cho phép chạy. Nếu không có tiến trình nào đang chờ, thì giá trị số nguyên của semaphore sẽ được tăng lên. Đây là giả code chỉ ra thủ tục signal của semaphore được cài đặt như thế nào:

```

procedure signal (s : Semaphore)
begin // This procedure must be executed atomically
    if length (s.queue) = 0 then
        s.value := s.value + 1; // Increase count allowing other processes to enter
    else
        allow_one_process (s.queue); // Wake up one suspended process
    end
end

```

Nếu chỉ một tiến trình được phép, nó sẽ đánh thức tiến trình đầu tiên trên hàng đợi, sau đó mỗi tiến trình đang đợi sẽ phải đợi chỉ một tiến trình trước nó trên hàng đợi. Điều này đạt được nhờ *đợi nhận việc* như đã nói mục trước.

9.1.5 Giám sát – Monitors

Semaphore cung cấp loại trừ đa bên, nhưng chỉ nếu được sử dụng đúng đắn. Nếu một tiến trình gọi wait trước khi vào vùng tới hạn, nhưng thất bại khi gọi signal sau đó, lỗi này có thể đẩy chương trình vào tắc nghẽn. Kiểu lỗi này của chương trình rất khó tìm. Monitors, được phát triển vào đầu những năm 1970, đặt trách nhiệm đồng bộ lên các thao tác truy cập dữ liệu. Monitors là tương tự như một trừu tượng dữ liệu, với việc đồng bộ được đặt trong các thao tác của kiểu dữ liệu. Nó làm cho dễ viết code client đúng.

Một monitor bao gồm một hoặc nhiều thủ tục, một dãy khởi tạo ban đầu và dữ liệu cục bộ. Dữ liệu cục bộ được truy cập chỉ bởi các thủ tục của monitors, mà là chịu trách nhiệm cho mọi sự đồng bộ gắn kết với truy cập song song đến dữ liệu đó. Trong thuật ngữ truyền thống, một tiến trình vào monitor bằng việc triệu gọi một trong các thủ tục của nó. Code đồng bộ được thiết kế tổng quan sao cho chỉ một tiến trình có thể được ở trong monitor tại một thời điểm; bất cứ tiến trình nào khác mà gọi thủ tục monitor đều được ngắt và chờ monitor khi nó sẵn sàng.

Trong thuật ngữ hiện đại, một monitor có thể được gọi bởi một *đối tượng đồng bộ*. Hai cấu trúc semaphore và monitors là các cấu trúc đồng bộ tương đương. Tuy nhiên monitors có được chứng minh là khái niệm cấu trúc chương trình có ích hơn.

9.2 Song song trong Java

Song song Java là một phần tích hợp của ngôn ngữ Java và hệ thống thời gian chạy. Chúng ta tạo một đối tượng luồng Java bằng định nghĩa một lớp với phương thức *run()*, hoặc bằng kế thừa lớp *Thread* hoặc cài đặt giao diện *Runnable* trực tiếp. Khi một đối tượng luồng được kích hoạt, phương thức *run* chạy như một luồng riêng biệt bên trong máy ảo java (JVM). Truyền thông và đồng bộ Java được cung cấp bằng các cơ chế riêng biệt.

Các luồng Java có thể trao đổi bằng việc gọi các phương thức trên các đối tượng chia sẻ. Chẳng hạn, luồng *producer* có thể trao đổi với luồng *consumer* bằng việc sử dụng đối tượng hàng đợi *queue*. Khi một luồng *producer* muốn gửi đầu ra của nó cho *consumer*, *producer* gọi phương thức *enqueue* để đặt đầu ra của nó vào hàng đợi. Khi *consumer* sẵn sàng lấy đồ vật tiếp theo, luồng *consumer* có thể gọi phương thức *dequeue*, lấy đồ vật khác ra khỏi hàng đợi *queue*.

Trao đổi thông qua một đối tượng chia sẻ không đồng bộ các luồng tham gia – mỗi luồng có thể tìm cách thực hiện phương thức một cách độc lập. Để tránh các vấn đề gắn kết với các tài nguyên chia sẻ, Java cung cấp một số *nguyên thủy đồng bộ* khác nhau. Ngôn ngữ Java cung cấp một nguyên thủy semaphore (duy trì một hàng đợi các tiến trình đang chờ) và hỗ trợ trực tiếp monitors ở dạng các đối tượng đồng bộ, mà là các đối tượng mà cho phép chỉ một luồng triệu gọi một phương thức ở một thời điểm.

Dạng thứ hai của song song Java, mà chúng ta bàn ngắn gọn trong 9.2.4, nảy sinh khi các chương trình Java chạy trong một máy ảo VM trao đổi với các chương trình chạy trong máy ảo khác. Cơ chế Triệu gọi phương thức từ xa Java (RMI) cho phép một đối tượng chạy trong một JVM triệu gọi các phương thức của một đối tượng chạy trong một máy ảo JVM khác. Java RMI được sử dụng cho lập trình phân tán Java, với các nền tảng lập trình phân tán như JINI và JXTA dựa trên RMI ở các mức độ khác nhau.

Thiết kế luồng Java là một đặc trưng ngôn ngữ thành công, Java làm cho chương trình song song chuyển mang được qua nhiều nền tảng tính toán, như song song là một phần được chuẩn hóa của ngôn ngữ và hệ thống thời gian chạy. Các luồng Java được sử dụng trong thực thi nhiều chương trình Java, bộ dọn rác Java (Java garbage collector) chạy như một luồng tách biệt trong JVM. Mặc dù Java được sử dụng rộng rãi, vẫn còn một số khó khăn trong lập trình với các luồng Java. Vẫn còn một số vấn đề tổng quan gắn kết với sự khác biệt của lập trình tuần tự và song song. Các lớp được thiết kế để sử dụng trong lập trình tuần tự nói chung không sử dụng đồng bộ và không thể làm việc tốt trong các chương trình đa luồng. Ngược lại, code sử dụng đồng bộ có thể là không hiệu quả trong các chương trình tuần tự - đồng bộ buộc dùng khóa mà là không cần thiết trong chương trình đơn luồng.

9.2.1 Các luồng, trao đổi và đồng bộ

Chúng ta có thể tạo ra một luồng bằng kế thừa lớp Thread, mà cài đặt giao diện *Runnable*. Khi một lớp kế thừa Thread, nó kế thừa các phương thức *start* một luồng và tung ra ngoại lệ cho một luồng:

start: một phương thức sinh ra một luồng mới bằng cách buộc máy ảo thực thi phương thức *run* song song với luồng mà tạo ra đối tượng đó
interrupt: một phương thức mà ngắt và tung ra ngoại lệ cho luồng.

Sau đây là một ví dụ lớp con của Thread:

```
class PrintMany extends Thread {
    private String msg;
    public PrintMany (String m){msg=m; }
    public void run(){
        try { for (;;) { System.out.print(msg + " ");
                    sleep(10);
                }
        } catch (InterruptedException e){
            return;
        }
    }
}
```

Giả sử một chương trình tạo một đối tượng `pm = new PrintMany("Hi")`. Khi `pm.start` được gọi, một luồng được tạo mà in ra xâu “Hi” và cứ tiếp tục như vậy. Điều này được tiếp tục cho đến khi luồng bị ngắt bởi một lời gọi cho `pm.interrupt`. Ban đầu, các luồng Java có các phương thức bổ sung mà tác động đến thực thi của chúng, bao gồm hai phương thức sau:

Suspend: một phương thức mà dừng thực thi của luồng đó cho đến khi nó được khôi phục lại về sau.

stop: một phương thức mà bắt buộc luồng phải dừng.

Tuy nhiên, các phương thức này bây giờ bị phản đối vì các lập trình viên nhận thấy rằng các phương thức này thường tạo ra nhiều vấn đề hơn là nó hỗ trợ. Vấn đề với Thread.suspend là có thể góp phần tạo tắc nghẽn. Nếu một luồng giữ một khóa trong vùng tới hạn bị ngắt, thì không luồng nào có thể vào vùng tới hạn cho đến khi luồng đích đó được khôi phục lại. Vấn đề với Thread.stop là cũng bị phản đối. Khi một luồng bị dừng bằng Thread.stop thì nó buộc mọi khóa nó đang giữ được giải phóng. Nếu bất kỳ đối tượng nào trong số đang được bảo vệ nhờ các khóa này ở trong trạng thái không tương thích, thì các luồng khác có thể truy cập đến các đối tượng đã bị phá hỏng. Điều này làm cho chương trình có hành vi không mong đợi và không đúng.

Các luồng chạy bên trong cùng một máy ảo là được xếp lịch tùy theo độ ưu tiên của chúng. Các luồng với độ ưu tiên cao hơn nói chung được thực hiện ưu tiên hơn so với các luồng có độ ưu tiên thấp, nhưng không có đảm bảo nào là luồng có độ ưu tiên cao nhất sẽ luôn luôn được chạy. Một luồng Java kế thừa độ ưu tiên của luồng mẹ tạo ra nó và có thể thay đổi với phương thức setPriority.

Trao đổi giữa các luồng

Java không tạo bất cứ dự phòng chuyên biệt nào cho truyền thông: các luồng Java trao đổi thông qua việc gán các biến chia sẻ hoặc gọi các phương thức của các đối tượng chia sẻ. Java cung cấp các cơ chế đồng bộ chuyên biệt mà làm cho truy cập đến các đối tượng chia sẻ an toàn.

Các nguyên thủy đồng bộ

Đồng bộ Java dựa trên ba cơ chế cơ bản sau:

- Locks: Mỗi đối tượng có một khóa, được sử dụng để loại trừ đa bên.
- Wait sets: Mỗi đối tượng có một tập chờ, cung cấp một dạng của semaphore.
- Thread termination: Một tiến trình có thể chờ cho đến khi tiến trình khác dừng.

Các khóa được kiểm tra và đặt bởi các khối và các phương thức đồng bộ, các tập chờ được sử dụng bởi các phương thức methods wait, notify, và notifyAll mà được định nghĩa trên mọi đối tượng và dừng luồng thread termination được sử dụng bởi phương thức join trên các đối tượng luồng. Các khóa được sử dụng bởi các lệnh synchronized, mà có dạng

```
synchronized( object ){
    statements
}
```

Khi thực hiện, lệnh này tính toán chiếu đến đối tượng được chỉ ra và khóa các khóa gắn kết của nó trước khi thực thi thân của lệnh đó. Nếu khóa đã được khóa bởi một luồng khác, luồng này chờ cho đến khi khóa sẵn sàng. Sau khi thực hiện thân của lệnh đó, luồng giải phóng khóa. Bên trong một phương thức, nói chung sử dụng *this* như một đối tượng khóa. Tuy nhiên, có thể đồng bộ trên khóa của bất cứ đối tượng nào. Các khóa Java là vào lại được *reentrant*, nghĩa là một luồng có thể yêu cầu một khóa nhiều lần. Điều này xảy ra khi một luồng thực hiện một câu lệnh đồng bộ bên trong một cái khác hoặc với các phương thức đồng bộ mà gọi lẫn nhau. Chẳng hạn, phương thức đồng bộ *f* trong lớp sau gọi một phương thức đồng bộ *g*:

```
public class LockTwice {
    public synchronized void f(){ g(); }
    public synchronized void g(){ System.out.println("Method g() called");}
```

```
}
```

Khi `f` được gọi, luồng gọi sẽ yêu cầu khóa cho đối tượng `LockTwice`. Khi `f` gọi `g`, luồng này thử chiếm cùng một khóa một lần nữa. Vì các khóa Java là vào lại được, luồng gọi có thể chiếm khóa của đối tượng `LockTwice` một lần nữa và cả hai phương thức dừng và trả điều khiển lại. Trong hệ thống mà không hỗ trợ các khóa *reentrant*, lần gọi thứ hai đến một phương thức đồng bộ có thể gây ra tắc nghẽn. Các cài đặt semaphore trong mục trước là không vào lại được.

Các phương thức `wait`, `notify`, và `notifyAll` của lớp *Object* sử dụng tập chờ của một đối tượng cung cấp kiểm soát song song. Một luồng có thể ngắt chính nó bằng sử dụng `wait`. Một luồng được ngắt không thực hiện bất cứ lệnh nào cho đến khi một luồng khác đánh thức nó bằng lệnh `notify`. Tổ hợp `wait/notify` có thể được sử dụng, chẳng hạn, khi các luồng có quan hệ producer-consumer. Vì `wait` cảnh báo lập lịch của máy ảo rằng một tiến trình cần được ngắt, `wait` và `notify` là hiệu quả hơn hàng chờ nhận việc, mà có thể bao hàm cả vòng lặp mà kiểm tra lặp nhiều lần để xem một luồng có thể tiếp tục được không.

Phương thức `notifyAll` là tương tự như `notify`, ngoại trừ mỗi luồng trong tập `wait` của đối tượng đó được loại bỏ và được cho quyền xếp lịch luồng. Đa số sách Java đề xuất gọi phương thức `notify` hoặc `notifyAll` của một đối tượng chỉ sau khi luồng gọi có khóa của đối tượng bị khóa. Dạng thứ ba của đồng bộ, tạm dừng cho đến khi một luồng khác kết thúc, được cung cấp bởi phương thức `join`. Đây là ví dụ đoạn code ngắn thể hiện sử dụng đó:

```
class Compute_thread extends Thread {
    private int result;
    public void run(){ result = f(...); }
    public int getResult(){ return result;}
}

...
Compute_thread t = new Compute_thread;
t.start() // start compute thread

...
t.join(); x = t.getResult(); // wait and get result
...
```

Trong đoạn code này, một đối tượng `Compute thread` thực hiện một kiểu tính toán để xác định kết quả nào đó. Chương trình này tạo một đối tượng `Compute thread` và bắt đầu cho một luồng chạy sao nó sẽ tính một kết quả có ích. Về sau, khi kết quả này cần, chương trình gọi `t.join()` để chờ `Compute_thread` kết thúc trước khi yêu cầu kết quả.

9.2.2 Các phương thức đồng bộ

Các luồng Java thường trao đổi trên các đối tượng chia sẻ, với đồng bộ được sử dụng cung cấp loại trừ đa bên. Chẳng hạn, nếu một quản trị giao dịch cho phép chỉ một luồng được cam kết tại một thời điểm, lớp định nghĩa quản trị giao dịch trông như sau:

```
class TransactionManager{ ...
    public synchronized void commitTransaction(. . .){... }
}

...
```

Khi một phương thức được khai báo là `synchronized`, thì tác động giống như đặt thân của phương thức bên trong lệnh `synchronized`, sử dụng khóa trên đối tượng mà chứa phương thức. Do đó, ngay cả khi một số luồng truy cập đến một đối tượng `Transaction-Manager m`, thì chỉ một luồng có thể thực hiện `m.commitTransaction(...)` tại một thời điểm. Nếu một luồng triệu gọi `m.commitTransaction(...)`, trong khi một luồng khác đang trong tiến trình làm điều đó, luồng thứ hai sẽ bị khóa và chờ

cho đến khi khóa trên đối tượng m được giải phóng. Một lớp có thể có đồng thời một số phương thức synchronized và một số phương thức không synchronized. Nếu một phương thức không được khai báo là synchronized, thì nhiều hơn một luồng có thể gọi phương thức unsynchronized đó tại cùng một thời điểm, sau đó mỗi luồng sẽ có một bản ghi kích hoạt cho lời gọi phương thức trên ngăn xếp thời gian chạy của mình. Tuy nhiên, hai bản ghi kích hoạt sẽ trở đến các trường của cùng một đối tượng trên *heap* thời gian chạy, trên tác động cho phép các luồng tương tác thông qua các biến chia sẻ. Nói chung, một lập trình viên có thể muốn để có càng nhiều phương thức unsynchronized càng tốt, cho phép song song tối đa. Tuy nhiên, sự cẩn thận cần có để đồng bộ vừa đủ các phương thức để tránh đặt một đối tượng vào trạng thái không đúng đắn.

Đây là một ví dụ đơn giản của một lớp với hai phương thức đồng bộ và một phương thức không đồng bộ (giống lớp `LinkedList` có trong nhiều tài liệu về các luồng Java):

```
class LinkedList { // Lisp-style cons cell containing
    protected double value; // value and link to next cell
    protected LinkedList next;
    public LinkedList (double v, LinkedList t){
        value = v; next = t;
    }
    public synchronized double getValue(){
        return value;
    }
    public synchronized void setValue(double v){
        value = v; // assignment not atomic
    }
    public LinkedList next(){ // no synch needed
        return next;
    }
}
```

Lớp này cài đặt một linked cell chuẩn, như có thể được sử dụng để cài đặt stacks, queues, hoặc linked lists đơn với một trường dữ liệu và một trường *next*, có thể được sử dụng để móc nối với một linked cell tiếp theo. Lưu ý rằng dữ liệu có kiểu `double`. Điều này là quan trọng (để thể hiện sự cần thiết phải đồng bộ) vì phép gán `double` không là nguyên vẹn. Vì `double` được thể hiện bằng 8 bytes và các bộ xử lý 32 bit thường chuyển 4 bytes một lúc, cần hai chỉ lệnh riêng biệt để đặt và đọc một `double`. Nếu `setValue` là không đồng bộ, và hai luồng gọi `setValue` đồng thời, thì nó có thể đặt đối tượng `LinkedList` vào trạng thái không đúng đắn. Đặc biệt, xét hai lời gọi `c.setValue(x)` and `c.setValue(y)`, ở đó x và y là hai số `double`. Một đan xen có thể của các bước trong hai phép gán `c.setValue()` là

- Đặt nửa đầu của `c.setValue` cho nửa trước của x
- Đặt nửa đầu của `c.setValue` cho nửa trước của y
- Đặt nửa thứ hai của `c.setValue` cho nửa thứ hai của y
- Đặt nửa thứ hai của `c.setValue` cho nửa thứ hai của x

Sau khi thực hiện theo thứ tự này, `c.setValue()` không chứa x hoặc y , mà nó chứa một số ký lạ bằng việc kết hợp một số bit trong biểu diễn của x với một số bit trong biểu diễn của y . Vì vậy tác động mong muốn của `c.setValue()` là đặt giá trị của z vào một ô, chỉ lệnh đan xen không đúng với ngữ nghĩa mong muốn của phương thức đó. Cần thiết phải đồng bộ các lời gọi cho `setValue`. Lý do đồng bộ `getValue` là tương tự. Nếu một luồng gọi `getValue` trong khi một luồng khác gọi `setValue`, thì số `double` sẽ trả về bằng lời gọi `getValue` có thể chứa nửa số bit trong ô trước khi `setValue` có tác động và một nửa số bit sau đó. Để ngăn cản `getValue` và `setValue` không chạy đồng thời, chúng ta đồng bộ cả hai. Tổng quát hơn, mặc dù có thể hai luồng đọc một cấu trúc dữ liệu đồng thời, lỗi có thể xảy ra khi đọc xảy ra trong lúc viết. Cách đơn giản để ngăn lỗi này là khóa cả đọc trong khi viết. Một cách tiếp cận tốt hơn, mà có thể đáng giá cho một số ứng dụng là sử dụng tách bạch khóa

đọc và khóa viết, với khóa đọc cho phép bất cứ số nào cùng đọc được tiếp tục, nhưng cấm bất cứ viết nào. Phương thức *next* là không đồng bộ, khi hai luồng cùng gọi *next* đồng thời, chúng cả hai đều nhận được kết quả đúng. Không có phương thức nào thay đổi móc nối *next* của một ô, và phép gán con trỏ là nguyên vẹn trong Java.

Các phương thức đồng bộ và kế thừa

Thiết kế đồng bộ không phải là một phần của dấu hiệu phương thức, và do đó một lớp con có thể viết đè một phương thức đồng bộ bằng một phương thức không đồng bộ. Lập trình viên có thể linh hoạt sử dụng điều này. Tuy nhiên cũng có thể hủy hoàn toàn chính sách khóa của một lớp thông qua kế thừa và kiểu con. Do đó lập trình viên cần phải rất cẩn thận khi mở rộng các lớp trong các chương trình đa luồng.

9.3 Lập trình phân tán và triệu gọi phương thức từ xa (RMI)

RMI cho phép một luồng chạy trên một máy ảo JVM gọi một phương thức của một đối tượng khác trên máy ảo khác qua mạng. Cơ chế truyền thông này là một trong các khối xây dựng cơ bản của lập trình Java phân tán. Vì RMI là thao tác không đối xứng (một luồng thực hiện lời gọi phương thức, luồng kia thì không), thuận tiện tham chiếu đến hai luồng bằng hai tên khác nhau. Chúng ta sẽ tham chiếu đến luồng gọi là *client* và luồng cung cấp đối tượng ở xa là *server*.

Một ứng dụng *server* tiêu biểu tạo một số đối tượng ở xa, tạo các tham chiếu truy cập đến chúng, và chờ các *client* triệu gọi các phương thức trên các đối tượng ở xa này. Một khi chúng ta có một kết nối được thiết lập nào đó giữa *client* và *server*, chúng ta có thể sử dụng kết nối này để truyền tham chiếu đến các đối tượng. Nhưng làm sao chúng ta tạo kết nối đầu tiên giữa hai hệ thống? Một giải pháp được sử dụng trong lập trình Java phân tán là đăng ký RMI, mà là một thư mục truy cập từ xa của các đối tượng sẵn sàng.

Java cung cấp ba tiện ích tổng quan sau cho lập trình phân tán:

- *Xác định vị trí đối tượng ở xa - Locating remote objects*: Một máy chủ có thể đăng ký các đối tượng ở xa của nó bằng cách sử dụng RMI registry hoặc server có thể truyền các tham chiếu đối tượng quan kết nối khác đã được thiết lập bởi các chương trình.
- *Trao đổi với các đối tượng ở xa - Communicating with remote objects*: Trao đổi giữa các đối tượng ở xa được kiểm soát bởi RMI, mà trông giống như triệu gọi phương thức Java chuẩn đối với lập trình viên.
- *Truyền các đối tượng qua mạng - Transmitting objects across the network*: Một đối tượng có thể được gửi như một đối số đến một lời gọi từ xa hoặc như giá trị trả về của một lời gọi. Khi một đối tượng Java được truyền, một máy ảo truyền bytecode để thực thi đối tượng và dữ liệu gắn kết đến một máy ảo khác. Máy ảo nhận sẽ tải bytecode và tạo đối tượng.

Tạo các đối tượng ở xa

Có một số bước tham gia vào việc tạo một đối tượng và có thể được truy cập từ xa. Bước đầu tiên là định nghĩa một lớp mà cài đặt một giao diện từ xa, mà là giao diện mở rộng giao diện `java.rmi.Remote`. Mỗi phương thức của giao diện *remote* cần được khai báo ngoại lệ `java.rmi.RemoteException` trong mệnh đề *throws*. Điều này cho phép một *client* ở xa tung ra một ngoại lệ *remote* cho bất cứ đối tượng nào trong lớp này. Khi một đối tượng của một lớp mà cài đặt một giao diện *remote* được truyền từ một máy ảo *server* đến một máy ảo *client*, RMI truyền một *stub* từ xa cho đối tượng đó. Đối tượng *stub* này có cùng các phương thức như đối tượng gốc nằm trên *server*, nhưng các lời gọi cho phương thức trên đối tượng *stub* được truyền trên mạng và được thực

thi trên đối tượng *server*. Trong chương trình *client*, mọi điều này trông giống như lập trình Java thông thường: Một lời gọi đến một đối tượng mà xảy ra ở xa trả về một *stub* của một đối tượng khác, và các lời gọi nối tiếp đến các phương thức đó được truyền qua mạng và thực hiện trên máy chủ *server*. Biên dịch *server code* yêu cầu hai bước. Bước thứ nhất là sử dụng trình biên dịch Java thông thường để dịch các files nguồn, mà chứa cài đặt của các giao diện ở xa và các cài đặt và các lớp *server*. Bước thứ hai sử dụng trình biên dịch thứ hai tạo các *stub* cho các đối tượng ở xa.

Tải động Lớp

Một đặc tính quan trọng của Java RMI là cách mà bytecode được truyền qua mạng. Điều này có thể vì Java cung cấp một môi trường thực thi đồng nhất, máy ảo JVM, mà chạy bytecode chuẩn không phụ thuộc vào kiểu phần cứng và hệ điều hành được sử dụng. Khi RMI được sử dụng để truyền một đối tượng từ một máy ảo này đến một máy ảo khác, bytecode cho lớp của đối tượng được truyền và tải, nếu lớp đó không được cài sẵn trên máy nhận. Điều này cho phép các kiểu mới được đưa đến một máy ảo ở xa, mở rộng hành vi của một chương trình một cách động.

Đăng ký RMI

RMI registry, chạy trên máy chủ, cho phép các client ở xa nhận một tham chiếu đến một đối tượng ở xa. Đăng ký này thường được sử dụng để định vị chỉ đối tượng đầu tiên mà *client* cần, vì các phương thức của đối tượng đầu tiên đó có thể trả về các đối tượng ở xa khác.

Hệ thống RMI cho phép một lập trình viên gắn kết một tên định dạng URL của dạng `//host/objectname` với một đối tượng ở xa. Một khi một đối tượng ở xa đã đăng ký trên *server*, các người gọi có thể tìm kiếm đối tượng theo tên, nhận tham chiếu một đối tượng ở xa, và sau đó triệu gọi các phương thức trên đối tượng đó. Chẳng hạn, nếu *helo* là một đối tượng mà có thể được gọi từ xa để nói “hello world”, code sau trỏ buộc tên *HelloWorld* với một tham chiếu đến một đối tượng *helo*.

```
Naming.rebind( "//machine.stanford.edu/HelloWorld", helo);
```

Điều này cho phép chương trình *client* nhận được một *stub* cho *helo* bằng truy cập mạng đến trang web máy chủ. Có một số chi tiết tham gia vào thiết lập tất cả điều này một cách đúng đắn, mà không cần liên quan đến chúng ta. Vấn đề chính là một luồng Java có thể nhận được một đối tượng từ một máy chủ qua mạng bằng sử dụng một tên tượng trưng cho máy chủ và đối tượng, và điều này là đủ để cho phép luồng của *client* nhận nhiều hơn các đối tượng ở xa từ máy chủ đó.

TÓM TẮT CHƯƠNG

Các ngôn ngữ song song định nghĩa các hành động song song mà có thể được thực thi đồng thời trên một số bộ xử lý hoặc trong chế độ đan xen trên một bộ xử lý duy nhất. Các chương trình song song nói chung có hành vi không tất định, như thứ tự thực hiện có thể phụ thuộc vào tốc độ truyền thông hoặc thứ tự ở đó các tiến trình được xếp lịch. Các luồng độc lập có thể trao đổi để tính toán một kết quả có ích và phối hợp để tránh các vấn đề liên quan đến chia sẻ tài nguyên. Khi hai luồng tìm cách cập nhật một cấu trúc dữ liệu chia sẻ, chẳng hạn, một điều phối nào đó có thể là cần để giữ các hành động của một luồng khỏi việc can thiệp với luồng khác. Không có phối hợp, các phép gán của một luồng có thể xóa bỏ các phép gán của luồng khác và làm cho cấu trúc dữ liệu đó ở trạng thái, mà không có ý nghĩa cho chương trình đó.

Khóa, semaphores và monitors là các cơ chế hệ điều hành truyền thống để phối hợp các hành động của các tiến trình độc lập. Khóa là một dạng biến chia sẻ mà có thể được thiết lập để chỉ ra rằng, một tài nguyên được khóa ở mở khóa. Khi một tiến trình khóa một tài nguyên, các tiến trình khác

không được truy cập đến tài nguyên đó. Một vấn đề với khóa là kiểm tra và đặt khóa cần là nguyên vẹn, không phân tách được. Vì lý do này các khóa với kiểm tra và đặt nguyên vẹn thường được cung cấp bởi phần cứng.

Semaphore cấu trúc một tập các tiến trình chờ vào một danh sách mà có thể được cho quyền truy cập đến một tài nguyên chia sẻ, khi nó trở nên sẵn sàng. Các tiến trình sử dụng *semaphore* cần gọi chờ *wait* trước khi truy cập đến tài nguyên được bảo vệ và ra thông báo *signal* khi chúng được làm xong. Cả hai *wait* và *signal* đều là các hành động nguyên vẹn mà có thể được cài đặt bằng hệ điều hành sử dụng khóa (hoặc hệ thống thời gian chạy của ngôn ngữ lập trình). Đồng bộ nhóm *monitors* với các thao tác trên dữ liệu, xóa bỏ sự cần thiết cho tiến trình gọi phải gọi *wait* và *signal*.

Các chương trình song song trao đổi sử dụng các tài nguyên chia sẻ hoặc thông điệp. Về nguyên tắc, cơ chế truyền thông điệp có thể được đồng bộ hoặc không đồng bộ, lưu đệm hoặc không lưu đệm, và giữ hoặc không giữ thứ tự truyền.

Các luồng Java và đồng bộ

Java làm cho các chương trình song song chuyển mang được qua nhiều nền tảng tính toán. Chúng ta tạo một đối tượng luồng Java bởi định nghĩa một lớp với một phương thức *run()*, hoặc mở rộng lớp *Thread* hoặc cài đặt giao diện *Runnable* trực tiếp. Khi đối tượng *thread* được kích hoạt, phương thức *run* được chạy như một luồng riêng biệt bên trong máy ảo JVM. Các luồng Java nói chung trao đổi bằng gọi các phương thức của các đối tượng chia sẻ. Ngôn ngữ Java cung cấp một số nguyên thủy đồng bộ và hỗ trợ trực tiếp *monitors* thông qua các đối tượng với các phương thức *synchronized*.

Đồng bộ Java đạt được thông qua ba cơ chế cơ bản sau:

- Các khóa - Locks: Mỗi đối tượng có một khóa, được sử dụng cho loại trừ đa bên.
- Các tập chờ - Wait sets: Mỗi đối tượng có một tập chờ, cung cấp một dạng semaphore.
- Dừng luồng - Thread termination: Một tiến trình có thể dừng chờ cho đến khi một luồng khác kết thúc.

Các khóa được kiểm tra và thiết lập bởi các khối và các phương thức đồng bộ *synchronized*, các tập chờ được sử dụng bởi phương thức *wait*, *notify*, và *notifyAll*, mà được định nghĩa trên mọi đối tượng.

Lập trình Java phân tán và triệu gọi phương thức từ xa

RMI cho phép một luồng chạy trên một JVM gọi một phương thức của một đối tượng trên một máy ảo khác qua mạng. Một ứng dụng *server* điển hình tạo các đối tượng ở xa, làm cho tham chiếu đến chúng có thể truy cập được, và chờ clients triệu gọi các phương thức trên các đối tượng ở xa đó. Một máy chủ làm cho các đối tượng có thể truy cập được bằng sử dụng đăng ký RMI, mà là một thư mục các đối tượng sẵn sàng được truy cập từ xa. Khi RMI được sử dụng để truyền một đối tượng từ một máy ảo đến máy khác, bytecode cho lớp của đối tượng được truyền và tải, nếu một lớp chưa được cài trên máy nhận. Nó cho phép các kiểu mới được đưa đến một máy ảo ở xa, mở rộng hành vi của một chương trình một cách động.

BÀI 10: LẬP TRÌNH LOGIC

10.1 Lịch sử lập trình Logic

Xu thế lập trình logic có nguồn gốc trong chứng minh định lý tự động, từ đó có khái niệm của suy luận. Cái mới là trong quá trình suy luận là một số giá trị được tính toán. Việc tạo ra xu thế lập trình này là kết quả của một lịch sử lâu dài mà với hầu hết gắn với logic và chỉ muộn sau này là trong khoa học máy tính. Lập trình logic được dựa trên cú pháp của logic cấp 1, mà được đề xuất ban đầu trong nửa sau của thế kỷ thứ 19 bởi Gottlob Frege và sau đó được điều chỉnh dưới dạng hiện nay bởi Giuseppe Peano và Bertrand Russell.

Trong những năm 1930, Kurt Godel và Jacques Herbrand nghiên cứu khái niệm tính toán được dựa trên các suy diễn. Các công trình này có thể được xem xét như cội nguồn của xu thế “tính toán như suy luận”. Ngoài ra, Herbrand thảo luận trong luận án tiến sĩ của ông một tập các qui tắc để biến đổi các phương trình đại số trên thuật ngữ mà được xem xét như bản phác thảo của một thuật toán đồng nhất. Khoảng 30 năm sau, vào năm 1965, Alan Robinson xuất bản bài báo cơ sở của ông mà là nền tảng của lĩnh vực suy diễn tự động. Trong bài báo này, ông đã đưa ra nguyên lý hóa giải và khái niệm đồng nhất và một thuật toán đồng nhất. Sử dụng phương pháp hóa giải, người ta có thể chứng minh các định lý của logic cấp 1, nhưng bước tiếp theo là cần thiết để nhìn thấy người ta có thể tính toán như thế nào trong khuôn khổ đó.

Điều này đã đạt được trong năm 1974 nhờ Robert Kowalski (Predicate logic as a programming language, North-Holland, 1974.) Proc. IFIP'74, ở đó các chương trình Logic với một dạng hạn chế hóa giải được đưa ra. Sự khác biệt giữa dạng này hóa giải và cái đề xuất bởi Robinson là cú pháp ở đây hạn chế hơn, nhưng chứng minh bây giờ có tác động phụ ở dạng một phép thế thỏa mãn. Phép thế này có thể được xem như kết quả của một phép tính và kéo theo các biểu thức logic nào đó có thể được hiểu như các chương trình. Song song với đó, Alain Colmerauer và các cộng sự của anh ta làm việc trên ngôn ngữ lập trình cho xử lý ngôn ngữ tự nhiên dựa trên chứng minh định lý tự động. Điều này cuối cùng dẫn đến tạo ra Prolog 1973. Kowalski và Colmerauer với đội ngũ của ông ta tương tác với nhau từ 1971 đến 1973. Điều này ảnh hưởng đến cách nhìn của họ là giúp họ phê phán các ý tưởng.

Prolog có thể được xem như hiện thực hóa ý tưởng các chương trình logic. Nó bắt đầu như ngôn ngữ lập trình cho các ứng dụng trong xử lý ngôn ngữ tự nhiên, nhưng không lâu sau đó, nó được tìm thấy cũng có thể được sử dụng như một ngôn ngữ lập trình đa dụng. Một số các phép thử để hiện thực một tính toán như xu thế suy luận được đề xuất cũng khoảng thời gian này, đáng kể là bởi Cordell Green và Carl Hewitt, nhưng một đề xuất lập trình logic, có lẽ vì là đơn giản nhất và uyên bác nhất, đã trở nên thành công nhất.

Ban đầu, Prolog được cài đặt bởi Philippe Roussel, một đồng nghiệp của Colmerauer, ở dạng một trình thông dịch viết trên Algol-W. Một bước quan trọng tiến lên phía trước đạt được bởi David H. Warren, người đã đề xuất trong năm 1983 một máy trừu tượng, bây giờ gọi là Warren Abstract Machine (WAM), mà bao gồm một kiến trúc máy với một tập chỉ lệnh mà phục vụ như đích của các trình biên dịch Prolog độc lập máy. WAM trở thành cơ sở chuẩn cho cài đặt Prolog và các ngôn ngữ lập trình logic khác.

Xu thế lập trình logic ảnh hưởng đến một số phát triển trong khoa học máy tính. Như vào đầu những năm 1970, nó dẫn đến việc tạo ra cơ sở dữ liệu suy diễn mà mở rộng cơ sở dữ liệu quan hệ bằng việc cung cấp các khả năng suy luận. Một sự thúc đẩy tiếp theo cho chủ đề này đến một cách

không mong đợi từ Dự án thế hệ 5 Nhật bản cho các hệ thống tính toán thông minh (1982-1991), ở đó lập trình logic được chọn làm cơ sở. Gần đây, xu thế này dẫn đến lập trình logic ràng buộc mà hiện thực hóa một cách tiếp cận tổng quan đến tính toán, ở đó quá trình lập trình được hạn chế đến việc sinh ra các ràng buộc (các yêu cầu) và một giải pháp của chúng, và cho lập trình logic qui nạp, một cách tiếp cận dựa trên logic cho học máy.

10.2 Tổng quan về xu thế lập trình logic

Xu thế lập trình logic thực chất khác hẳn các xu thế lập trình khác. Nó có thể được tổng kết dựa trên ba đặc tính sau.

- Tính toán thực hiện trên miền các terms được định nghĩa là bảng chữ vạn năng.
- Các giá trị được gán cho các biến thông qua các phép thế tự động sinh, được gọi là bộ đồng nhất tổng quan nhất. Các giá trị này có thể chứa các biến, được gọi là các biến logic.
- Bộ điều khiển được cung cấp bởi cơ chế duy nhất: quay lui tự động.

Trong việc khai phá xu thế lập trình này, chúng ta sẽ nhấn mạnh ba điểm. Ngay cả một tổng quan ngắn gọn như vậy chỉ ra cả sức mạnh và điểm yếu của xu thế lập trình logic. Điểm mạnh của nó nằm ở sự rất đơn giản và ngắn gọn súc tích; điểm yếu của nó là phải làm việc với một số hạn chế về một cơ chế điều khiển và sử dụng một kiểu dữ liệu duy nhất.

Khung này buộc phải điều chỉnh và làm giàu thêm để làm cho nó thích nghi với các nhu cầu lập trình thông thường, chẳng hạn bằng việc cung cấp các cấu trúc điều khiển khác nhau và bằng việc đưa ra kiểu dữ liệu số nguyên với các phép toán số học thông thường. Nó có thể được làm và trên thực tế, các ngôn ngữ lập trình logic Prolog và ràng buộc là các ví dụ của việc phát triển của khung này.

10.2.1 Lập trình khai báo

Hai đặc tính bổ sung của lập trình logic là quan trọng cần phải nêu bật. Thứ nhất, trong dạng thuần túy, nó hỗ trợ lập trình khai báo. Một chương trình khai báo thừa nhận hai cách hiểu. Cái thứ nhất, gọi là giải nghĩa thủ tục, giải thích tính toán được tiến hành như thế nào và cái thứ hai gọi là giải nghĩa khai báo là liên quan đến câu hỏi, cái gì được tính toán.

Một cách không hình thức, giải nghĩa thủ tục là liên quan đến phương pháp, còn giải nghĩa khai báo là liên quan đến ý nghĩa. Trong giải nghĩa thủ tục, một chương trình khai báo được xem như mô tả một thuật toán mà có thể thực hiện. Trong giải nghĩa khai báo, một chương trình khai báo được xem như một công thức, và người ta có thể suy luận về tính đúng đắn của nó mà không cần tham chiếu đến cơ chế tính toán bên dưới. Nó làm cho các chương trình khai báo dễ dàng được hiểu và phát triển.

Như chúng ta dễ thấy, trong một số tình huống đặc tả một bài toán trong định dạng của lập trình logic đã gần tạo nên một lời giải thuật toán cho bài toán. Như vậy lập trình logic hỗ trợ lập trình khai báo và cho phép chúng ta viết các đặc tả thực thi. Nó cũng được bổ sung, tuy nhiên trên thực tế các chương trình Prolog nhận được theo cách này thường là không hiệu quả, như vậy cách tiếp cận lập trình này cần kết hợp với nhiều kỹ thuật tối ưu khác, và một sự tìm hiểu phù hợp cơ chế tính toán bên dưới sẽ là không thể thiếu được. Để làm rõ luận điểm này, chúng ta sẽ trình bày một số chương trình Prolog mà là khai báo và loại bỏ khỏi nó một số nguồn gốc không hiệu quả.

Giải nghĩa kép này của các chương trình khai báo cũng là cơ sở cho việc sử dụng kép của ngôn ngữ lập trình như một bộ hình thức cho lập trình và cho giải nghĩa tri thức và giải thích tầm quan trọng của lập trình logic trong lĩnh vực trí tuệ nhân tạo.

10.2.2 Lập trình tương tác

Một đặc tính quan trọng khác của lập trình logic là nó hỗ trợ lập trình tương tác. Nói cách khác, người sử dụng cần viết một chương trình đơn và tương tác với nó thông qua các câu truy vấn quan tâm và các câu trả lời cho chúng sẽ được tạo ra. Các hệ thống Prolog hỗ trợ rất nhiều sự tương tác như vậy và cung cấp phương tiện đơn giản để tính toán một hay nhiều lời giải cho câu truy vấn được đưa ra, để đưa ra câu truy vấn tiếp theo, và theo dõi quá trình thực hiện bằng việc thiết lập các điểm kiểm tra, nếu mong muốn, tất cả trong cùng một vòng lặp tương tác. Điều này dẫn đến tính linh hoạt của kiểu lập trình.

Điều này là hoàn toàn tương tự với cách lập trình hàm được sử dụng, ở nơi tương tác đạt được nhờ các biểu thức mà cần được tính toán sử dụng một tập cho trước các định nghĩa hàm số.

Tiếp theo, chúng ta sẽ giới thiệu Prolog, một ngôn ngữ lập trình được biết tốt nhất dựa trên xu thế lập trình logic. Prolog là dựa trên một tập con logic cấp 1. Chúng ta giải thích làm sao Prolog sử dụng cú pháp này theo một cách khác và mở rộng bằng một số đặc tính thú vị, đáng lưu ý bằng hỗ trợ ký hiệu trung vị và bằng cung cấp các siêu biến được gọi là vô danh. Trên thực tế, các mở rộng làm cho nó có khả năng hiện thực trong Prolog một ngôn ngữ lập trình bậc cao và siêu lập trình theo một cách rất đơn giản.

10.3 Các phương trình giải được bằng đồng nhất như những hành động nguyên tử

Chúng ta bắt đầu giải thích tính toán được thực hiện “ở mức nguyên tử” như thế nào trong lập trình logic, các hành động nguyên tử là các phương trình giữa các terms (các biểu thức tùy ý). Chúng được thực hiện thông qua quá trình đồng nhất mà cố gắng giải các phương trình này. Trong quá trình giải, các giá trị được gán cho các biến. Các giá trị này có thể là các terms bất kỳ. Trên thực tế, các biến này tất cả là cùng một kiểu mà gồm một tập hợp tất cả các terms.

10.3.1 Terms (các biểu thức)

Để giải thích kỹ càng, giả sử chúng ta bắt đầu đưa ra một bảng chữ mà gồm các lớp ký hiệu rời nhau như sau:

- Các biến - *variables*, ký hiệu bởi $x, y, z, \dots$ có thể có các chỉ số phụ dưới
- Các ký hiệu hàm - *function symbols*
- Các dấu ngoặc "(" và ")"
- Dấu phẩy ","

Chúng ta cũng đưa ra yêu cầu là mỗi ký hiệu hàm có một ngôi cố định, nó là số các đối số gắn kết với hàm. Hàm ngôi 0 được gọi là các hằng số, và thông thường được ký hiệu bởi $a, b, c, d, \dots$. Dưới đây chúng ta sử dụng các ký hiệu hàm bậc dương bằng $f, g, h, \dots$. Cuối cùng, *terms* được định nghĩa đệ qui như sau:

- Một biến là một term,
- Nếu f là một ký hiệu hàm ngôi n và $t_1, \dots, t_n$ là các terms, thì $f(t_1, \dots, t_n)$ là một term.

Trong trường hợp đặc biệt, mỗi hằng số là một term. Các term mà là các biến tự do thường được gọi là *term nền* (*ground terms*). Dưới đây chúng ta ký hiệu các terms bằng $s, t, u, w, \dots$.

Chẳng hạn a là hằng số, x và y là các biến, f là ký hiệu hàm hai ngôi và g là một ký hiệu hàm một ngôi, thì $f(f(x, g(b)), y)$ là một *term*.

Terms là khái niệm cơ bản trong logic toán học, nhưng thoát nhìn, chúng có vẻ không phổ biến trong khoa học máy tính. Tuy nhiên, chúng có thể được coi là tổng quan của khái niệm một cấu trúc thuộc trong ngôn ngữ hình thức. Trên thực tế, cấu trúc được xem như các terms được xây dựng từ một bảng chữ mà chỉ có các ký hiệu hàm số của nó là các phép ghép nối trong mỗi ngôi của nó. Một ví dụ quen thuộc khác của terms là các biểu thức số học. Chúng là các terms được xây dựng từ một bảng chữ, ở đó như các ký hiệu hàm, chúng ta lấy các biểu thức số học thông thường của phép cộng, trừ, nhân và chia số nguyên và như các hằng số là 0, -1, 1, -2, 2, ...

Trong lập trình logic, không có bảng chữ chuyên biệt nào được giả thiết. Trên thực tế, thuận tiện để giả thiết rằng, trong mỗi ngôi một số lượng vô hạn các ký hiệu hàm tồn tại và rằng mọi terms được viết trong một bảng chữ vạn năng này. Các ký hiệu hàm này có thể là các ký hiệu các phép toán số học, nhưng không có ý nghĩa nào được gán cho các ký hiệu hàm này. Điều này là tương phản với hầu hết các ngôn ngữ lập trình mệnh lệnh, ở đó chẳng hạn sử dụng “+” trong một biểu thức suy ra là chúng ta tham chiếu đến phép toán cộng. Hệ quả khác của lựa chọn này là không có kiểu nào được gán cho terms. Trên thực tế, không có kiểm tra nào được giả thiết và do đó không có sự khác nhau giữa biểu thức số học, biểu thức Bool và các terms biểu diễn danh sách. Mọi terms này được xem xét là cùng một kiểu.

10.3.2 Các phép thế

Không giống như lập trình mệnh lệnh, trong lập trình logic các biến có thể không được khởi tạo. Tuy nhiên các giá trị có thể của các biến là terms. Như vậy để giải thích quá trình tính toán, chúng ta cần kiểm tra lại khái niệm trạng thái.

Tại thời điểm bất kỳ trong tính toán, sẽ chỉ có hữu hạn các biến mà được *khởi tạo* – đó là các biến mà được gán các giá trị nào đó trong tính toán đang xem xét. Vì các giá trị này là các terms, nên hiển nhiên chúng ta dẫn đến việc xem xét các phép thế *substitutions*. Chúng là các ánh xạ hữu hạn từ các biến vào các terms sao cho không có biến nào được ánh xạ vào chính nó. Vì vậy phép thế cung cấp thông tin về các biến mà được khởi tạo.

Các phép thế sau đó tạo thành một khái niệm giống như trạng thái được sử dụng trong lập trình mệnh lệnh. Chúng ta ký hiệu các phép thế bởi

$$\{x_1/t_1, \dots, x_n/t_n\}$$

Ký hiệu này suy ra rằng $x_1, \dots, x_n$ là các biến khác nhau, $t_1, \dots, t_n$ là các terms và không term t_i nào bằng với x_i . Khi đó chúng ta nói rằng phép thế $\{x_1/t_1, \dots, x_n/t_n\}$ trói buộc biến x_i với term t_i .

Sử dụng một phép thế, chúng ta có thể tính toán một term theo cùng một cách như sử dụng một trạng thái chúng ta có thể tính toán giá trị của một biểu thức trong ngôn ngữ lập trình mệnh lệnh. Quá trình tính toán này được gọi là áp dụng một phép thế cho một term. Nó là kết quả của một sự thay thế đồng thời mỗi biến xuất hiện trong một miền của một phép thế bằng term tương ứng. Như vậy chẳng hạn áp dụng phép thế $\{x/f(z), y/g(z)\}$ cho term $h(x, y)$ nhận được term $h(f(z), g(z))$. Ở đây biến x được thay thế bởi term $f(z)$ và biến y được thay thế bởi term $g(z)$. Cũng theo cách này chúng ta định nghĩa áp dụng phép thế cho một atom, query hoặc một clause.

Như vậy một tính toán của một term sử dụng một phép thế cũng cho một term. Điều này đối ngược với lập trình mệnh lệnh, ở đó một tính toán một biểu thức sử dụng một trạng thái cho một giá trị thuộc về kiểu của biểu thức.

10.3.3 Bộ đồng nhất tổng quan nhất

Như đã nhấn mạnh ở trên, trong lập trình logic các hành động nguyên tử là các phương trình giữa các terms và quá trình đồng nhất được sử dụng để xác định ý nghĩa của chúng. Trước khi bàn về việc này một cách chi tiết, chúng ta xem xét một số ví dụ hiển nhiên về việc giải các phương trình có thể được sử dụng như một phép gán như thế nào.

Chúng ta giả thiết mọi biến được nhắc đến đều là chưa được khởi tạo. Bằng việc viết $x = a$, chúng ta gán hằng số a cho biến x . Vì trong ngôn ngữ lập trình dấu “=” là đối xứng, tác động cũng như vậy đạt được bằng cách viết $a = x$. Thú vị hơn, bằng việc viết $x = f(y)$ (hoặc, tương đương, $f(y) = x$) chúng ta gán term $f(y)$ cho biến x . Vì $f(y)$ là một term với một biến, chúng ta đã gán cho biến x một biểu thức với một biến trong đó. Nhắc lại một biến mà xuất hiện trong giá trị được gán cho một biến khác được gọi là biến logic. Do đó y là biến logic ở đây. Việc sử dụng các biến logic là một đặc trưng khác biệt của lập trình logic chúng ta dành một mục riêng cho nó. Cuối cùng bằng việc viết $f(y) = f(g(a))$, chúng ta gán term $g(a)$ cho biến y , như điều đó là cách mà làm cho hai terms này bằng nhau.

Các ví dụ này chỉ ra rằng phép bằng nhau “=” trong lập trình logic và phép gán trong C, cũng được viết sử dụng “=”, là hai khái niệm khác hẳn nhau.

Thực quan, đồng nhất – *unification* là một quá trình giải một phương trình giữa hai terms (chẳng hạn như làm cho hai terms bằng nhau) theo cách ràng buộc ít nhất. Phép thế kết quả (nếu nó tồn tại) được gọi là đồng nhất tổng quan nhất – *most general unifier (mgu)*. Chẳng hạn, phương trình $x = f(y)$ có thể được giải (tức là các terms x và $f(y)$ đồng nhất) theo một số cách, ví dụ như thông qua mỗi một trong các phép thế sau:

$$\{x/f(y)\}, \{x/f(a), y/a\}, \{x/f(a), y/a, z/g(b)\},$$

Rõ ràng, chỉ phép đầu tiên là “ít ràng buộc nhất”. Trên thực tế, trong số ba phép thế này, chỉ phép thế đầu tiên là cái đồng nhất tổng quan nhất của phương trình $x = f(y)$. Khái niệm phép thế ít ràng buộc nhất có thể được làm cho chính xác bằng định nghĩa một thứ tự các phép thế. Trong thứ tự này, phép thế $\{x/f(y)\}$ là tổng quát hơn $\{x/f(a), y/a\}, \dots$

Lưu ý rằng chúng ta làm cho hai terms x và $f(y)$ bằng nhau bằng cách chỉ định đến một trong chúng. Trường hợp đặc biệt như vậy của đồng nhất được gọi là *sánh – matching*, mà là cách gán giá trị của lập trình hàm. Đồng nhất là tổng quan hơn sánh như sau đây, một ví dụ kém hiển nhiên hơn chỉ ra. Xét phương trình

$$f(x, a) = f(b, y).$$

Ở đây, đồng nhất tổng quan nhất là

$$\{x/b, y/a\}.$$

Trái ngược với ví dụ trước, bây giờ không thể làm cho hai terms bằng nhau mà chỉ định đến một trong chúng.

Bài toán quyết định một phương trình giữa hai terms có lời giải không được gọi là *bài toán đồng nhất*.

Robinson (1965) chỉ ra rằng bài toán đồng nhất là giải được. Chính xác hơn, ông ta đưa ra một thuật toán đồng nhất với tính chất sau. Nếu một phương trình giữa các terms có một lời giải, thuật toán đó tạo ra một *mgv*, ngược lại nó báo lỗi. Một *mgv* của một phương trình là duy nhất sai khác bằng việc đổi tên các biến.

10.3.4 Thuật toán đồng nhất

Trong phần tiếp theo, chúng ta sẽ xem xét chi tiết quá trình đồng nhất sử dụng một thuật toán đồng nhất rất đẹp được đưa ra bởi Martelli và Montanari. Thuật toán này nhận đầu vào là một tập hữu hạn các phương trình terms

$$\{s_1 = t_1, \dots, s_n = t_n\}$$

Và thử tìm một *mgv* của chúng.

MARTELLI-MONTANARI ALGORITHM

Không tắt định chọn từ tập các phương trình một phương trình có dạng dưới đây và thực hiện hành động gắn kết

- | | |
|--|---|
| 1. $f(s_1, \dots, s_n) = f(t_1, \dots, t_n)$ | thay bởi các phương trình $s_1 = t_1, \dots, s_n = t_n$, |
| 2. $f(s_1, \dots, s_n) = g(t_1, \dots, t_m)$ trong đó $f \neq g$ | dừng, thất bại, |
| 3. $x = x$ | xóa phương trình này, |
| 4. $t = x$ trong đó t không là biến | thay bằng phương trình $x = t$, |
| 5. $x = t$ trong đó x không xuất hiện trong t
và x xuất hiện ở một chỗ khác | áp dụng phép thế $\{x/t\}$ với mọi phương trình khác |
| 6. $x = t$ trong đó x xuất hiện trong t và x khác t | dừng, thất bại. |

Thuật toán kết thúc khi không có hành động nào có thể được thực hiện hoặc thất bại. Trong trường hợp thành công, bằng việc thay trong tập cuối cùng các phương trình mọi dấu “=” bằng dấu “/” ta nhận được *mgv* mong muốn. Lưu ý rằng hành động (1) bao gồm cả trường hợp $c = c$ cho mỗi hằng số c mà dẫn đến xóa phương trình như vậy. ngoài ra, hành động (2) bao gồm trường hợp hai hằng số khác nhau.

Để thể hiện một thao tác của thuật toán này, chúng ta xét lại phương trình $f(x, a) = f(b, y)$. Sử dụng hành động (1) viết lại bằng tập hai phương trình $\{x = b, a = y\}$. Bằng hành động 4 chúng ta có tập hợp $\{x = b, y = a\}$. Tại thời điểm thuật toán dừng chúng ta có *mgv* $\{x/b, y/a\}$.

Như vậy bằng việc giải nghĩa ký hiệu bằng nhau như một yêu cầu tìm một đồng nhất tổng quan nhất của cặp term đang xem xét, mỗi phương trình chuyển thành một hành động nguyên tử mà hoặc tạo nên một pháp thể (một đồng nhất tổng quan nhất) hoặc lỗi. Khả năng có lỗi ở mức hành động nguyên tử là một đặc trưng khác biệt của lập trình logic. Bằng việc viết dãy các phương trình, chúng ta có thể tạo các chương trình logic đơn giản mà hoặc thành công và tạo đầu ra là một phép thể hoặc thất bại. Quan trọng là hiểu tính toán sau đó được tiến triển như thế nào. Chúng ta mô tả nó thông qua các ví dụ phức tạp hơn.

Trước hết xét dãy phương trình

$$f(x, a) = f(g(z), y), h(u) = h(d).$$

Phương trình thứ nhất tạo ra phép thể trực tiếp thứ nhất $\{x/g(z), y/a\}$ và phương trình thứ hai cho phép thể $\{u/d\}$. Bằng việc kết hợp hai phép thể này chúng ta nhận được một phép thể

$$\{x/g(z), y/a, u/d\}$$

được tạo bởi chương trình logic này.

Như một ví dụ ít hiển nhiên hơn, xét dãy phương trình

$$f(x, a) = f(g(z), y), h(x, z) = h(u, d).$$

Ở đây phép thể trực tiếp $\{x/g(z), y/a\}$ trói buộc biến x mà cũng xuất hiện trong phương trình thứ hai. Phương trình thứ hai này cần được tính trước hết ở trạng thái hiện thời, được biểu diễn bằng phép thể $\{x/g(z), y/a\}$, trước khi được thực hiện. Tính toán này tạo ra phương trình $h(g(z), z) = h(u, d)$. Phương trình tạo ra đồng nhất tổng quan nhất $\{u/g(d), z/d\}$ và phép thể cuối cùng ở đây là

$$\{x/g(d), y/a, u/g(d), z/d\}.$$

Điều xảy ra ở đây là phép thể $\{u/g(d), z/d\}$ đã được áp dụng cho phép thể trung gian $\{x/g(z), y/a\}$. Hiệu quả của áp dụng một phép thể, chẳng hạn δ , cho một phép thể khác, chẳng hạn γ , (hoặc kết hợp của các phép thể) là nhận được bởi

- áp dụng δ cho mỗi term mà xuất hiện trong phạm vi của γ
- bổ sung cho phép thể kết quả trói buộc các biến mà đã xuất hiện trong miền của δ nhưng không ở trong miền của γ

Trong ví dụ trên, bước đầu tiên tạo ra phép thể $\{x/g(d), y/a\}$, và bước thứ hai bổ sung trói buộc $u/g(d)$ và z/d vào phép thể cuối cùng. Quá trình kết hợp phép thể tương ứng với một cập nhật trạng thái trong lập trình mệnh lệnh, và điều này là chúng ta tham chiếu đến nó theo tuần tự như thế nào.

Như ví dụ cuối chúng ta xem xét dãy các phương trình

$$f(x, a) = f(g(z), y), h(x, z) = h(d, u).$$

Nó thất bại. Thực tế, sau khi thực hiện phương trình thứ nhất biến x được trói với $g(z)$, vì vậy phương trình thứ hai tạo ra $h(g(z), z) = h(d, u)$ và không có phép thể nào làm cho hai terms bằng nhau $h(g(z), z)$ và $h(d, u)$ (lưu ý d là hằng).

Là hữu ích khi so sánh giải các phương trình bằng đồng nhất với lệnh gán. Thứ nhất, lưu ý rằng, trái ngược với phép gán, đồng nhất có thể gán một term tùy ý cho một biến. Cũng như vậy, nó có thể thất bại, một điều gì đó nó không thể làm. Mặt khác, sử dụng phép gán người ta có thể thay đổi giá trị của biến, điều mà đồng nhất có thể thực hiện theo cách rất hạn chế: bằng việc làm cho term được sử dụng như một giá trị. Do đó, các hành động nguyên tử này là không so sánh được.

10.4 Các mệnh đề như các phần của khai báo thủ tục

Lập trình logic là một hình thức dựa trên các qui tắc và Prolog là ngôn ngữ dựa trên luật. Trong bối cảnh đó, các luật được gọi là các mệnh đề. Để hiểu hơn quan hệ giữa lập trình logic và lập trình mệnh lệnh, trước hết chúng ta sẽ đưa ra một dạng hạn chế của mệnh đề.

10.4.1 Các mệnh đề đơn giản

Sử dụng đồng nhất, chúng ta có thể thực hiện các chương trình cực kỳ đơn giản mà bao gồm một dãy các phương trình. Bây giờ chúng ta sẽ làm giàu khung chương trình này bằng cách bổ sung các thủ tục. Trong lập trình logic chúng được mô hình thông qua các ký hiệu, đôi khi được gọi là các mệnh đề. Dưới đây chúng ta biểu thị các ký hiệu quan hệ $p, q, r, \dots$. Như trong trường hợp các ký hiệu hàm số, chúng ta giả thiết rằng mỗi ký hiệu quan hệ có một ngôi cố định gắn kết với nó. Khi một ngôi là 0, thì ký hiệu quan hệ thường được gọi là một ký hiệu khẳng định – propositional.

Nếu p là một ký hiệu quan hệ n -ngôi và $t_1, \dots, t_n$ là các terms, thì ta gọi $p(t_1, \dots, t_n)$ là một nguyên tử - atom. Khi $n = 0$ các ký hiệu khẳng định trùng với các nguyên tử. Thử ví, khi các nguyên tử như vậy là hữu ích.

Về trực quan một ký hiệu quan hệ tương ứng với một tên thủ tục và một atom tương ứng với một lời gọi hàm. Một ký hiệu dấu bằng "=" là một ký hiệu quan hệ hai ngôi được viết dưới dạng trung tố, như vậy mỗi phương trình cũng là một atom. Tuy nhiên, ý nghĩa của sự bằng nhau là đã được xác định, như vậy nó có thể xem như là một thủ tục được xây bên trong, tức là một thủ tục với ý nghĩa được xác định trước.

Chúng ta cũng cần định nghĩa khai báo thủ tục và làm rõ cơ chế tham số được sử dụng. Cho một ký hiệu quan hệ n -ngôi p và các atoms $A_1, \dots, A_k$ chúng ta gọi một biểu thức dạng

$$p(x_1, \dots, x_n) :- A_1, \dots, A_k.$$

là một mệnh đề đơn giản. $p(x_1, \dots, x_n)$ được gọi là tiêu đề head của mệnh đề và $A_1, \dots, A_k$ là thân của nó. Dấu chấm "." ở cuối mệnh đề là quan trọng: nó báo cho chương trình dịch biết rằng đó là kết thúc của mệnh đề.

Giải nghĩa thủ tục của một mệnh đề đơn giản $p(x_1, \dots, x_n) :- A_1, \dots, A_k$ là:

"để thiết lập $p(x_1, \dots, x_n)$ cần thiết lập $A_1, \dots, A_k$ ",

trong khi giải nghĩa khai báo là:

" $p(x_1, \dots, x_n)$ là đúng nếu $A_1, \dots, A_k$ là đúng".

Giải nghĩa khai báo giải thích tại sao trong lý thuyết lập trình logic ký hiệu suy diễn chiều ngược " $\leftarrow$ " được sử dụng thay cho " $:-$ ".

Cuối cùng, một chương trình logic đơn giản là tập hữu hạn các mệnh đề. Một chương trình như vậy được kích hoạt bằng việc cung cấp một truy vấn - query ban đầu, mà là một dãy các nguyên tử atoms. Trong lập trình mệnh lệnh, biệt từ "một truy vấn" khi đó là một chương trình và một chương trình logic đơn giản là một tập các khai báo thủ tục. Về trực quan, cho một chương trình đơn giản, một tập các mệnh đề đơn giản với cùng ký hiệu quan hệ ở đầu tương ứng với một khai báo thủ tục trong ngôn ngữ mệnh lệnh. Sự lộn xộn cú pháp là ở chỗ trong lập trình logic dấu phẩy ";" được sử dụng để tách giữa các atoms tạo nên một truy vấn, trong khi trong ngôn ngữ mệnh lệnh dấu ";" được sử dụng cho mục đích này.

10.4.2 Quá trình tính toán

Tính không tắt định được đưa vào khung này vì cho phép các mệnh đề lặp với cùng một ký hiệu quan hệ ở tiêu đề. Trong lý thuyết lập trình logic, dạng không tắt định này (*được gọi là không tắt không biết*) được giữ bởi việc xem xét tất cả các tính toán mà có thể được sinh ra thông qua các mệnh đề lặp và bằng việc giữ lại một số mà dẫn đến thành công. “Không biết” tham chiếu đến một sự thật là về tổng quan chúng ta không biết tính toán nào sẽ dẫn đến thành công.

Trong Prolog, quá trình tính toán này được làm tắt định bằng thứ tự các mệnh đề mà là thứ tự văn bản và bằng khi phá quay lui tự động để khắc phục các lỗi. Tiếp theo, khi thiết kế chương trình Prolog, thường là hữu ích có tính không tắt định “không biết” trong đầu. Thực tế, trong giải thích các chương trình Prolog, các câu như “chương trình này không tắt định đoán một phần tử sao cho” là thường gặp. Bây giờ chúng ta giải thích chính xác hơn tính toán được thực hiện như thế nào trong Prolog. Để làm điều đó, chúng ta cần làm rõ cơ chế thủ tục được sử dụng và vai trò trong việc sử dụng các mệnh đề lặp.

Cơ chế thủ tục gắn kết với các mệnh đề đơn giản đưa ra ở trên là *gọi-theo-tên* theo đó các tham số hình thức là được thay thế đồng thời bởi các giá trị thật. Vì cơ chế thủ tục như vậy có thể đơn giản được giải thích bằng các phép thế: cho trước một mệnh đề đơn giản

$$p(x_1, \dots, x_n) :- A_1, \dots, A_k.$$

một lời gọi thủ tục a procedure call $p(t_1, \dots, t_n)$ dẫn đến thực hiện lệnh

$$(A_1, \dots, A_k)\{x_1/t_1, \dots, x_n/t_n\}$$

mà nhận được bằng việc áp dụng phép thế $\{x_1/t_1, \dots, x_n/t_n\}$ cho lệnh $A_1, \dots, A_k$. (chúng ta giả thiết ở đây là các biến của mệnh đề được đổi tên phù hợp để tránh xung đột biến). Một cách tương đương, chúng ta có thể nói rằng, lời gọi thủ tục $p(t_1, \dots, t_n)$ dẫn đến thực hiện các lệnh $A_1, \dots, A_k$ trong trạng thái (thể hiện bằng một phép thế) được cập nhật bởi phép thế $\{x_1/t_1, \dots, x_n/t_n\}$.

Các mệnh đề được thử theo thứ tự chúng xuất hiện trong văn bản chương trình. Chiến thuật tìm kiếm theo chiều sâu được suy ra từ việc là lời gọi thủ tục dẫn trực tiếp đến việc thực thi thân của mệnh đề đơn giản được chọn. Nếu tại một giai đoạn nào đó phát sinh lỗi, tính toán sẽ quay lui lại đến điểm lựa chọn cuối cùng (điểm trong tính toán mà ở đó một trong số các mệnh đề áp dụng được lựa chọn) và mệnh đề đơn giản tiếp theo được lựa chọn. Nếu mệnh đề đang được chọn là cuối cùng, tính toán sẽ quay lui lại đến điểm lựa chọn trước đó. Nếu không có điểm lựa chọn nào còn, lỗi sẽ phát sinh. Quay lui suy ra rằng trạng thái được lưu lại, sao cho mọi cập nhật trạng thái được thực hiện, vì việc tạo ra lựa chọn cuối cùng chưa được làm xong.

Bây giờ chúng ta sẽ minh họa định nghĩa này của quá trình tính toán Prolog bằng việc xem xét một chương trình Prolog được biết nhiều nhất mà mục đích của nó là ghép hai danh sách. Trong Prolog, một danh sách rỗng được ký hiệu là $[]$ và danh sách với phần tử đầu là h và đuôi là t ký hiệu bởi $[h | t]$. Term $[a | [b | s]]$ được ký hiệu dạng dễ đọc hơn là $[a, b | s]$, danh sách $[a, [b, []]]$ được viết tắt là $[a, b]$ và tương tự với các danh sách dài hơn. Ký hiệu này có thể được sử dụng cho cả các danh sách và các terms bất kỳ mà bắt đầu với thao tác tạo danh sách $[. | .]$.

Khi đó chương trình logic sau được định nghĩa bằng phép qui nạp trên đối số đầu ghép hai danh sách như thế nào. Ở đây và mọi chỗ khác chúng ta tuân theo qui định cú pháp của Prolog và ký hiệu các biến là các xâu bắt đầu bằng chữ in. Các tên kết thúc chữ “s” được sử dụng cho các biến theo nghĩa sẽ được biến thành danh sách.

% append(Xs, Ys, Zs) :- Zs là kết quả của phép ghép hai danh sách Xs và Ys.
 append(Xs, Ys, Zs) :- Xs = [], Zs = Ys.
 append(Xs, Ys, Zs) :- Xs = [H | Ts], Zs = [H | Us], append(Ts, Ys, Us).

Trong Prolog, các câu trả lời được sinh ra như các phép thế được viết dạng phương trình (như trong thuật toán Martelli-Montanari được trình bày ở trên). Tiếp theo, chúng ta thể hiện một truy vấn Q dạng $?-Q$. Ở đây $"?-"$ là lời nhắc của hệ thống và dấu chấm kết thúc $"."$ báo hiệu kết thúc truy vấn.

Người ta có thể kiểm tra truy vấn

$?- \text{append}([\text{jan}, \text{feb}, \text{mar}], [\text{april}, \text{may}], \text{Zs}).$

tạo ra $\text{Zs} = [\text{jan}, \text{feb}, \text{mar}, \text{april}, \text{may}]$ như câu trả lời và rằng truy vấn

$?- \text{append}([\text{jan}, \text{feb}, \text{mar}], [\text{april}, \text{may}], [\text{jan}, \text{feb}, \text{mar}, \text{april}, \text{may}]).$

Thành công và tạo ra phép thế rỗng như câu trả lời. Ngược lại truy vấn

$?- \text{append}([\text{jan}, \text{feb}, \text{mar}], [\text{april}, \text{may}], [\text{jan}, \text{feb}, \text{mar}, \text{april}]).$

thất bại. Trên thực tế, tính toán dẫn đến các lời gọi thủ tục liên tiếp sau

$\text{append}([\text{feb}, \text{mar}], [\text{april}, \text{may}], [\text{feb}, \text{mar}, \text{april}]),$
 $\text{append}([\text{mar}], [\text{april}, \text{may}], [\text{mar}, \text{april}])$ and
 $\text{append}([], [\text{april}, \text{may}], [\text{april}]),$

và cái cuối cùng thất bại, vì các terms $[\text{april}, \text{may}]$ and $[\text{april}]$ không thể đồng nhất được.

10.4.3 Các mệnh đề

Bước sau cùng trong định nghĩa các chương trình logic bao gồm cho phép các atoms bất kỳ như đầu của các mệnh đề. Một cách hình thức, cho trước các atoms $H, A_1, \dots, A_k$, chúng ta gọi một biểu thức dạng

$H:-A_1, \dots, A_k.$

là một *mệnh đề*. Nếu $k=0$, đây là cái mà có thân mệnh đề là rỗng, mệnh đề như vậy được gọi là một sự kiện - *fact* và ký hiệu $":-"$ là được bỏ qua. Nếu $k > 0$, đây là cái, mà nếu thân mệnh đề khác rỗng, mệnh đề như vậy được gọi là một luật - *rule*.

Một *chương trình logic* là một tập hữu hạn các mệnh đề và một *chương trình Prolog thuần túy* là một dãy hữu hạn các mệnh đề.

Cho trước một chương trình Prolog thuần túy, chúng ta gọi tập các mệnh đề của nó với quan hệ p ở đầu là một *định nghĩa của p* . Các định nghĩa tương ứng với khai báo thủ tục trong lập trình mệnh lệnh và các định nghĩa hàm trong lập trình hàm. Các biến mà xuất hiện trong thân của mệnh đề chứ không phải ở đầu của nó được gọi là cục bộ. Chúng tương ứng chặt chẽ đến các biến mà là cục bộ trong thân thủ tục trong ngôn ngữ mệnh lệnh với sự khác biệt là trong lập trình logic khai báo của chúng là không tường minh. Lập trình logic giống như Pascal, không có các lệnh khối.

Để giải thích quá trình tính toán được thực hiện như thế nào đối với các chương trình Prolog thuần túy, chúng ta đơn giản xem một mệnh đề dạng

$p(s_1, \dots, s_n):-A_1, \dots, A_k.$

như một dạng viết tắt của mệnh đề đơn giản

$$p(x_1, \dots, x_n) :- (x_1, \dots, x_n) = (s_1, \dots, s_n), A_1, \dots, A_k.$$

trong đó $x_1, \dots, x_n$ là các biến mới. Chúng ta sử dụng tiện ích cú pháp của Prolog tuân theo đó cho một dãy các terms $s_1, \dots, s_n$ thì $(s_1, \dots, s_n)$ cũng là term.

Vì vậy cho trước một lời gọi thủ tục $p(t_1, \dots, t_n)$, nếu mệnh đề trên $p(s_1, \dots, s_n) :- A_1, \dots, A_k$ được chọn, một phép thử được thực hiện để thống nhất $(t_1, \dots, t_n)$ với $(s_1, \dots, s_n)$. (Như trước kia, ta giả thiết rằng, không có xung đột biến nào; ngược lại các biến cần được đổi biến phù hợp). Nếu đồng nhất thành công và tạo ra một phép thế θ , trạng thái (được biểu diễn bằng phép thế) sẽ được cập nhật bằng việc áp dụng θ và tính toán được tiếp tục với lệnh $A_1, \dots, A_k$ trong trạng thái mới này. Ngược lại, một lỗi nảy sinh và mệnh đề tiếp theo được chọn.

Do đó, bằng việc sử dụng các mệnh đề thay cho các mệnh đề đơn giản, đồng nhất được nâng lên một cơ chế tham số một cách hiệu quả. Như một tác động phụ, điều này làm cho việc sử dụng tường minh đồng nhất, được mô hình thông qua "=", là không cần thiết.

Như một ví dụ, chúng ta xét lại chương trình ghép hai danh sách ở trên. Sử dụng mệnh đề, nó có thể được viết lại dưới dạng súc tích hơn, như chương trình APPEND sau:

```
% append(Xs, Ys, Zs) :- Zs is the result of concatenating the lists Xs and Ys.
append([], Ys, Ys).
append([X | Xs], Ys, [X | Zs]) :- append(Xs, Ys, Zs).
```

Ở đây, phân tích trường hợp ẩn trong chương trình trên được chuyển hiệu quả sang đầu của các mệnh đề. Việc sử dụng term trong đầu mệnh đề hoàn toàn tương tự với sử dụng các mẫu trong các định nghĩa trong lập trình hàm.

Để tổng kết, các thành phần đặc trưng trong khai báo thủ tục trong lập trình logic là khác hẳn với lập trình mệnh lệnh: ở đây sử dụng nhiều luật lặp và sử dụng các mẫu để lựa chọn giữa các luật đó.

10.5 Tiếp cận Prolog cho lập trình

Sức mạnh và tính độc đáo của phong cách lập trình Prolog nằm ở sự kết hợp quay lui tự động với việc sử dụng các quan hệ và các biến logic.

10.5.1 Sử dụng lặp một chương trình duy nhất

Như một minh họa đầu tiên của tính mới của cách tiếp cận Prolog cho lập trình, chúng ta minh họa khả năng sử dụng cùng một chương trình cho nhiều mục đích khác nhau. Ví dụ đơn giản nhất có thể là chương trình MEMBER sau. Chúng ta sử dụng trong đó một đặc trưng có ích khác của Prolog, được gọi là biến vô danh, được viết như ký tự “gạch dưới” là “_”. Mỗi xuất hiện của “_” trong truy vấn hoặc ở một mệnh đề được giải nghĩa như một biến khác nào đó. Các biến vô danh là tương tự mẫu wildcard trong ML hoặc Haskell.

```
% member(X, Xs):- X is a member of the list Xs.
member(X, [X | _]).
member(X, [_ | Xs]) :- member(X, Xs).
```

MEMBER có thể được sử dụng cho cả kiểm thử và tính toán:

```
?- member(wed, [mon, wed, fri]).
yes
```

```
?- member(X, [mon, wed, fri]).
Xs = mon ;
Xs = wed ;
Xs = fri ;
No
```

Ở đây ";" là yêu cầu của người sử dụng tạo ra câu trả lời tiếp theo. Nếu yêu cầu này thất bại, câu trả lời là "no" được in ra. Tiếp theo, cho trước một biến X và hai danh sách s và t, một truy vấn

```
member(X, s), member(X, t).
```

sinh ra mọi phần tử mà có mặt trong cả hai s và t. Về mặt thao tác, lời gọi thứ nhất sinh ra mọi phần tử của s và lời gọi thứ hai kiểm tra đối với mỗi phần tử trong số đó có phải là thành viên trong t hay không. Cũng như vậy, chương trình APPEND có thể được sử dụng cho một số mục đích, cụ thể nối hai danh sách và tách một danh sách theo mọi cách có thể. Chẳng hạn, ta có

```
?- append(Xs, Ys, [mon, wed, fri]).
Xs = []
Ys = [mon, wed, fri];
Xs = [mon]
Ys = [wed, fri];
Xs = [mon, wed]
Ys = [fri];
Xs = [mon, wed, fri]
Ys = [];
no
```

Điều này có thể đạt được với bất cứ phiên bản lập trình hàm nào của APPEND. Sự khác biệt đến từ sự kiện là trong lập trình logic các thủ tục được định nghĩa thông qua các quan hệ, trong khi đó trong lập trình hàm các hàm số được sử dụng. Trên thực tế, không có sự khác biệt giữa đối số đầu vào và đầu ra trong các thủ tục của chương trình logic.

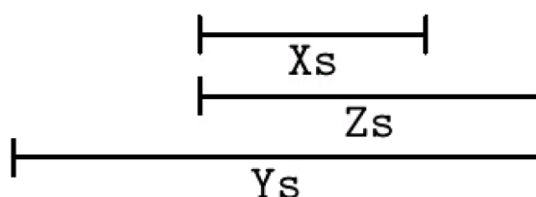
Để thấy hai cách sử dụng append trong một chương trình duy nhất, xét một chương trình mà kiểm tra một danh sách có là một danh sách con liên của một danh sách khác không. Một chương trình SUBLIST mà hình thức định nghĩa sau đây của danh sách con:

Danh sách Xs là một danh sách con của danh sách Ys nếu Xs là khúc đầu của một khúc đuôi của Ys.

% sublist(Xs, Ys) :- Xs là danh sách con của danh sách Ys.

sublist(Xs, Ys) :- append(_, Zs, Ys), append(Xs, _, Zs).

Ở đây, cả hai biến vô danh và Zs là các biến cục bộ. Trong luật này Zs là một khúc đuôi của Ys và Xs là một khúc đầu của Zs. Quan hệ này được thể hiện trên Hình vẽ sau



Hình vẽ 10.1: Xs là danh sách con của danh sách Ys

Về mặt thao tác, cho trước hai danh sách, as và bs , truy vấn $sublist(as, bs)$. Sẽ dẫn đến việc sinh ra các cách tách danh sách bs thông qua lời gọi $append(_, Zs, bs)$. Sau đó với mỗi đuôi được sinh Zs của bs nó kiểm tra xem có danh sách nào đó mà được gọi là biến vô danh mà lời gọi $append(as, _, Zs)$ là thành công không. Điều này xảy ra khi as là một khúc đầu của Zs . Như vậy sử dụng thông thường của chương trình này bao gồm quay lui.

10.5.2 Các biến logic

Bây giờ chúng ta trở lại các biến logic. Chúng là một đặc trưng quan trọng của lập trình logic, nhưng rất dễ không chú ý tới việc sử dụng chúng. Chẳng hạn, chúng đã xuất hiện trong các tính toán từ phiên bản đầu tiên của chương trình ghép hai danh sách, và sau này theo cách chúng ta định nghĩa quá trình tính toán, ở trong tính toán của chương trình `APPEND`. Trên thực tế, cho trước truy vấn

```
append([jan, feb, mar], [april, may], Zs).
```

qui tắc

```
append(Xs, Ys, Zs) :- Xs = [H | Ts], Zs = [H | Us], append(Ts, Ys, Us).
```

dẫn đến trói buộc biến Zs với term $[jan | Us]$. Giá trị của biến Us được tính về sau bằng lời gọi

```
append([feb, mar], [april, may], Us).
```

Lời gọi này trước tiên trói buộc Us với term $[feb | U1s]$, trong đó $U1s$ là một biến mới, và sau đó trói Zs với $[jan, feb | U1s]$. Việc xây dựng tiến triển này của đầu ra sử dụng các biến logic là điển hình cho Prolog. Sức mạnh thực tế của các biến logic là hiển nhiên sau khi xem xét các chương trình Prolog đặc sắc sau.

Một chương trình dãy

Tiếp theo, xét bài toán sau: sắp xếp ba số 1, ba số 2, ..., ba số 9 trên một dãy sao cho với mọi i thuộc $[1, 9]$, ở đây có đúng i số giữa các xuất hiện liên tiếp của i . Một ví dụ một dãy như vậy là

1, 9, 1, 2, 1, 8, 2, 4, 6, 2, 7, 9, 4, 5, 8, 6, 3, 4, 7, 5, 3, 9, 6, 8, 3, 5, 7.

Chương trình mong muốn là một hình thức gần đúng nguyên văn của bài toán trong Prolog.

```
% sequence(Xs) :- Xs is a list of 27 variables.
sequence([_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_]).
% question(Ss) :- Ss is a solution to the problem.
question(Ss) :-
    sequence(Ss),
    sublist([9,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([8,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([7,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([6,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([5,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([4,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([3,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([2,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss),
    sublist([1,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_], Ss).
```

Bây giờ chúng ta thấy các biến vô danh làm cho chương trình dễ đọc lên rất nhiều. Về thao tác, khi truy vấn

?- question (Ss).

dẫn tới lời gọi thủ tục *sequence* (Ss) mà khởi tại biến Ss cho danh sách 27 biến vô danh. Sau đó mỗi một trong 9 lời gọi của thủ tục *sublist* buộc tạo ra một mẫu *sublist* chuyên biệt trên Ss. Mỗi mẫu bao gồm các biến vô danh, mỗi một trong chúng thể hiện một biến logic. Mặc dầu là chương trình rất đơn giản và trong suốt, kết quả tính toán được kéo theo, vì sử dụng tích cực quay lui. Truy vấn sinh ra 6 lời giải cho bài toán.

Danh sách hiệu

Một trong những nhược điểm của phép ghép các danh sách được thực hiện bởi chương trình APPEND là đối với các danh sách s và t việc thực hiện truy vấn $\text{append}(s, t, Z)$ thực hiện một số bước mà tỷ lệ thuận với độ dài danh sách đầu s . Điều này rõ ràng là không hiệu quả. Trong ngôn ngữ mệnh lệnh, nếu người ta thể hiện một danh sách như một danh sách móc nối, để ghép hia danh sách thì chỉ cần điều chỉnh một con trỏ.

Danh sách hiệu là khái niệm tổng quan của khái niệm danh sách mà cho phép chúng ta thực hiện việc ghép trong thời gian hằng số. Thực tế rằng, nhiều chương trình dựa tường minh trên việc ghép danh sách giải thích quan trọng của khái niệm này.

Sau đây, chúng ta sẽ sử dụng phép toán “-” được viết dạng trung tố. Việc sử dụng nó không có gì liên quan đến số học, mặc dù trực quan người ta có thể đọc nó như phép “trừ”. Một cách hình thức một danh sách hiệu là một cấu trúc có dạng

$$[a_1, \dots, a_m | x] - x,$$

ở đó x là một biến và ở đó chúng ta sử dụng ký hiệu biến trong danh sách đã nói phần 10.4.2. Nó thể hiện một danh sách $[a_1, \dots, a_m]$ ở dạng tuân theo định nghĩa hiệu của phép ghép. Bây giờ, xét hai danh sách hiệu

$$[a_1, \dots, a_m | x] - x \text{ và } [b_1, \dots, b_n | y] - y.$$

Khi đó phép ghép của chúng là danh sách hiệu

$$[a_1, \dots, a_m, b_1, \dots, b_n | y] - y.$$

Quá trình ghép này đạt được bằng chương trình APPEND_DL sau:

```
% append(Xs, Ys, Zs) :- the difference list Zs is the result of concatenating
% the difference lists Xs and Ys.
append_dl(X-Y, Y-Z, X-Z).
```

Chẳng hạn, ta có:

```
?- append_dl([a,b|X]-X, [c,d|Y]-Y, U).
U = [a,b,c,d|Y]-Y,
X = [c,d|Y]
```

Điều trên chỉ ra rằng, U trở thành được gán cho danh sách hiệu thể hiện danh sách $[a,b,c,d]$. Chúng ta sẽ mô tả sử dụng danh sách hiệu trong 10.6.2.

10.6 Số học trong Prolog

Chương trình Prolog được thể hiện trên đây là khai báo, vì chúng cho phép đọc như một công thức. Giải pháp với số học trong Prolog thỏa hiệp một dạng mở rộng của trụ cột khai báo. Tuy nhiên, khó đi đến một giải pháp tốt hơn cái mà được đề xuất bởi những nhà thiết kế ban đầu của ngôn ngữ. Nhược điểm của giải pháp số học Prolog được khắc phục trong ngôn ngữ lập trình logic ràng buộc.

10.6.1 Các phép toán số học

Prolog cung cấp các số nguyên và các số thập phân như các cấu trúc dữ liệu được xây dựng sẵn, với các phép toán thông thường trên chúng. Các phép toán này bao gồm các phép toán số học thông thường như $+$, $-$, $*$ và $//$ (chia số nguyên).

Bây giờ, tuân theo các qui định ký hiệu thông thường trong lập trình logic và Prolog, các ký hiệu quan hệ và hàm số được viết ở dạng tiền tố, các phép toán đứng trước các đối số. Ngược lại theo cách sử dụng chúng trong số học, các phép toán số học nhị phân được viết dạng trung tố, phép toán ở giữa các đối số. Ngoài ra số đối của một số tự nhiên được viết dạng tiền tố không ngoặc bao quanh đối số.

Sự không nhất quán này trong cú pháp được giải quyết bằng việc xem xét các phép toán số học là các ký hiệu hàm được xây dựng sẵn viết dạng trung tố hoặc dạng tiền tố không ngoặc với thông tin về tính kết hợp của chúng và sức mạnh trói buộc mà cho phép chúng ta không nhập nhằng trong các biểu thức số học.

Thực tế, Prolog cung cấp công cụ để khai báo một ký hiệu hàm số bất kỳ như ký hiệu nhị phân trung tố hoặc như ký hiệu một ngôi tiền tố không ngoặc, với độ ưu tiên cố định mà xác định sức mạnh trói buộc của nó và một trí nhớ nào đó mà suy ra dạng kết hợp hoặc không.

Các ký hiệu hàm khai báo theo cách này được gọi là các phép toán. Như vậy, các phép toán số học có thể được xem như các phép toán được định nghĩa trước trong ngôn ngữ đã bao hàm

Ngoài phép toán số học chúng ta cũng tùy ý sử dụng nhiều vô hạn các hằng, số nguyên và nhiều vô hạn các số thập phân. Các phép toán số học và tập mọi số xác định duy nhất một tập các terms. Chúng ta gọi terms định nghĩa trong ngôn ngữ này là các biểu thức số học và đưa ra viết tắt *gae* cho các biểu thức số học nền tảng (tức là các biến tự do).

10.6.2 Các quan hệ so sánh số học

Với mỗi *gae*, chúng ta có thể gắn kết với một giá trị của nó, được tính toán theo cách mong muốn. Prolog cho phép chúng ta so sánh các giá trị của các *gae* thông qua sáu quan hệ so sánh số học sau:

$<$, $=<$, $=:=$ ("bằng nhau"), $=\backslash=$, ("khác nhau"), $>=$, và $>$.

Quan hệ "bằng nhau" $=:=$ không được nhầm với quan hệ "đồng nhất với" relation $=$ đã được xem xét trong mục 10.3.

Các quan hệ so sánh số học làm việc trên các *gae* và tạo ra kết quả mong muốn. Chẳng hạn, $>$ so sánh các giá trị của hai *gae* và thành công, nếu giá trị của đối số thứ nhất là lớn hơn giá trị của đối số thứ hai và thất bại nếu ngược lại.

Do đó, ví dụ

```
?- 6*2 == 3*4.
yes
?- 7 > 3+4.
no
```

Tuy nhiên, khi một trong các đối số của quan hệ so sánh số học không là *gae*, tính toán dừng với lỗi. Chẳng hạn chúng ta có

```
?- [] < 5.
```

lỗi trong biểu thức số học:[] không là số.

Như một ví dụ đơn giản của việc sử dụng quan hệ so sánh số học, xét chương trình sau đây sẽ kiểm tra một danh sách số có được sắp xếp không.

```
% ordered(Xs) :- Xs là một =< -danh sách sắp xếp các số
ordered([]).
ordered([_]).
ordered([X, Y | Xs]) :- X <= Y, ordered([Y | Xs]).
```

Nhắc lại rằng $[X, Y | Xs]$ là viết tắt của $[X | [Y | Xs]]$. Bây giờ ta có

```
?- ordered([1,1,2,3]).
Yes
```

Nhưng, cũng như vậy

```
?- ordered([1,X,1]).
```

Sẽ đưa ra lỗi. Ở đây lỗi thời gian chạy vì tại một giai đoạn nào đó quan hệ so sánh được áp dụng cho một đối số không là số.

Một ví dụ khác, xem xét phiên bản Prolog của thủ tục quicksort của C.A.R. Hoare. Tuân theo thủ tục sắp xếp này, một danh sách được chia thành hai danh sách con sử dụng một phần X của nó, một danh sách con gồm các phần tử nhỏ hơn và kết quả là các danh sách con sắp xếp được chèn X ở giữa. Điều này có thể được thể hiện trong Prolog thông qua chương trình QUICKSORT sau, ở đó X được chọn là phần tử đầu tiên trong danh sách đã cho:

```
% qs(Xs, Ys):- Ys is an ordered permutation of the list Xs.
qs([], []).
qs([X | Xs], Ys):-
    part(X, Xs, Littles, Bigs),
    qs(Littles, Ls),
    qs(Bigs, Bs),
    append(Ls, [X | Bs], Ys).
% part(X, Xs, Ls, Bs):- Ls is a list of elements of Xs which are < X,
% Bs is a list of elements of Xs which are >= X.
part(_, [], [], []).
part(X, [Y | Xs], [Y | Ls], Bs) :- X > Y, part(X, Xs, Ls, Bs).
part(X, [Y | Xs], Ls, [Y | Bs]) :- X <= Y, part(X, Xs, Ls, Bs).
```

Bây giờ chúng ta có, chẳng hạn

```
?- qs([7,9,8,1,5], Ys).
Ys = [1, 5, 7, 8, 9]
```

Và cũng có

```
?- qs([7,9,8,1,5], [1,5,7,9,8]).
no
```

Chương trình QUICKSORT sử dụng quan hệ ghép hai danh sách. Do đó, tính hiệu quả có thể được nâng cao nếu chúng ta sử dụng danh sách hiệu.

Đây là chương trình QUICKSORT_DL định nghĩa quan hệ qs được thay thế bởi:

```
% qs(Xs, Ys) :- Ys is an ordered permutation of the list Xs.
qs(Xs, Ys) :- qs_dl(Xs, Ys - []).
%qs_dl(Xs, Y) :- Y is a difference list representing the
% ordered permutation of the list Xs.
qs_dl([], Xs - Xs).
qs_dl([X | Xs], Ys - Zs) :-
part(X, Xs, Littles, Bigs),
qs_dl(Littles, Ys - [X | Y1s]),
qs_dl(Bigs, Y1s - Zs).
```

10.6.3 Tính toán giá trị biểu thức số học

Chúng ta đã thể hiện các chương trình sử dụng các biểu thức số học nền, nhưng chưa đưa ra bất kỳ cách tính toán nó. Chẳng hạn, không có công cụ nào được đưa ra để tính $3+4$. Tất cả những gì chúng ta có thể làm ở bước này là kiểm tra xem kết quả có phải là 7 không sử dụng quan hệ so sánh $==$ bằng truy vấn $7 == 3 + 4$. Tuy nhiên, sử dụng quan hệ so sánh không thể là phép gán cho $3+4$ bằng 7 cho một biến, chẳng hạn X . Lưu ý truy vấn $X == 3 + 4$. kết thúc với lỗi, trong khi $X = 3 + 4$ đồng nhất X với term $3 + 4$.

Để khắc phục điều này, bộ *tính toán số học nhị phân* được sử dụng trong Prolog, là một phép toán trung tố được định nghĩa như sau.

Xét lời gọi “is”. Khi đó t đã phải là một biểu thức nền – *gae*.

Lời gọi của s là t cho kết quả đồng nhất giá trị của *gae* t với s .

Nếu t không là *gae* thì lỗi thời gian chạy sẽ phát sinh.

Chẳng hạn, chúng ta có

```
?- 7 is 3+4.
yes
?- 8 is 3+4.
no
?- X is 3+4.
X=7
?- X is Y+1.
! Error in arithmetic expression: not a number
```

Như một ví dụ sử dụng bộ tính toán số học, ta xem xét hàm tính giai thừa. Nó có thể được tính sử dụng chương trình sau FACTORIAL:

```
% factorial(N, F) :- F is N!.
factorial(0, 1).
factorial(N, F) :- N > 0, N1 is N-1, factorial(N1, F1), F is N*F1.
```

Lưu ý, việc sử dụng biến cục bộ N1 trong atom N1 is N-1 để tính giá trị giảm của N và sử dụng một biến cục bộ F1 để tính giá trị N1 giai thừa. Atom N1 is N-1 tương ứng với lệnh gán $N = N-1$ của lập trình mệnh lệnh. Sự khác biệt ở đây là biến mới cần được sử dụng để tính giá trị của N-1. Việc sử dụng biến cục bộ như vậy là tiêu biểu khi tính toán các số nguyên trong Prolog.

Một ví dụ khác, xét chương trình tính toán độ dài của một danh sách

```
% length(Xs, N):- N is the length of the list Xs.
length([], 0).
length([_|Ts], N) :- length(Ts, M), N is M+1.
```

Chúng ta có

```
?- length([a,b,c], N).
N=3
```

Một điểm là các biểu thức nên được sinh ra không được tự động tính toán trong Prolog.

10.7 Điều khiển, cú pháp mâu thuẫn và các siêu biến

Trong khuôn khổ đã xem xét, không có cấu trúc điều khiển nào được trình bày. Bây giờ chúng ta xem chúng có thể được mô phỏng thông qua các đặc trưng đã giải thích như thế nào. Xét cấu trúc thông thường **if B then S else T fi**. Nó có thể được mô hình thông qua hai mệnh đề sau:

```
p(x):-B,S.
p(x) :- not B, T.
```

ở đó p là tên một thủ tục mới và một biến B, S và T đều được tập hợp lại trong x. Để thấy tính kém hiệu quả len vào kiểu lập trình này, ta xét hai trường hợp.

Trước hết giả thiết rằng mệnh đề thứ nhất được chọn và rằng B là *true*. Khi đó tính toán tiếp tục với S. Nhưng trong trường hợp tổng quát B là một truy vấn bất kỳ và vì tính không tắt định ẩn thể hiện B có thể thành công theo nhiều cách. Nếu tính toán S thất bại, các cách lựa chọn này của việc tính B sẽ tự động thử mặc dù chúng ta đã biết là B đã là *true*. Thứ hai, giả sử rằng mệnh đề thứ nhất và B là *false*. Khi đó quay lui sẽ thực hiện và mệnh đề thứ hai được thử. Tính toán sẽ tiếp tục với đánh giá *not B*. Điều này hoàn toàn không cần thiết, vì chúng ta biết rằng, ở giai đoạn này *not B* là *true*. Lưu ý rằng, việc bỏ qua *not B* trong mệnh đề thứ hai có thể nảy sinh vấn đề trong trường hợp thành công B được nối tiếp sau bởi thất bại của S. Khi đó quay lui T có thể được thực hiện.

10.7.1 Cut

Để xử lý các vấn đề như trên, Prolog cung cấp một ký hiệu quan hệ ngôi 0 được xây sẵn mức thấp được gọi là *cut* và ký hiệu là “!”. Để giải thích ý nghĩa của nó chúng ta viết lại mệnh đề thứ nhất ở trên sử dụng *cut*:

```
p(x):-B,! ,S.
p(x):-T.
```

Trong kết quả phân tích, hai khả năng có thể có. Thứ nhất, nếu B là *true*, thì cut sẽ được tính đến. Việc thực thi nó sẽ

- bỏ qua mọi các tính khác của B .
- bỏ qua mệnh đề thứ hai $p(x)$: T như một lựa chọn quay lui được cho lựa chọn hiện thời của mệnh đề thứ nhất.

Cả hai mục này có tác động là trong tính toán hiện tại một số mệnh đề là không sẵn sàng nữa. Thứ hai nếu B là *false*, thì quay lui sẽ thực hiện và mệnh đề thứ hai được thử. Tính toán này sẽ tiếp tục bởi tính toán trực tiếp T .

Như vậy, bằng việc sử dụng cut và viết lại như trên chúng ta đạt được tác động theo ý định và mô hình cho cấu trúc **if B then S else T fi** theo cách mong muốn.

Việc giải thích trên của tác động cut là điểm bắt đầu tốt để cung cấp định nghĩa tổng quan hơn. Xét định nghĩa một quan hệ p :

```
p(s1):- A1.
...
p(si):- B,! ,C..
...
p(sk):- A k.
```

Ở đây mệnh đề thứ i chứa cut atom. Bây giờ giả thiết trong khi thực hiện một truy vấn, một lời gọi $p(t)$ được xét đến và đến lượt mệnh đề thứ i được sử dụng và sự xuất hiện của cut được thực hiện. Khi đó sự xuất hiện của $!$ thành công ngay lập tức, nhưng thêm vào đó

1. mọi cách lựa chọn của B là được bỏ qua và
2. mọi tính toán $p(t)$ sử dụng mệnh đề từ thứ i đến thứ k cho p là được bỏ qua như các lựa chọn quay lui được cho lựa chọn mệnh đề thứ i .

Cut được đưa vào để cải tiến điều khiển ẩn có thông qua kết hợp quay lui và thứ tự văn bản của các mệnh đề. Vì sử dụng mẫu trong các phần đầu mệnh đề, nguồn không hiệu quả tiềm ẩn có thể đôi khi ẩn sâu hơn trong văn bản chương trình. Xét lại ví dụ QUICKSORT mục trước và truy vấn

```
?- qs([7,9,8,1,5], Ys).
```

Để nhìn thấy kết quả tính toán là không hiệu quả, lưu ý rằng mệnh đề thứ hai trong định nghĩa quan hệ *part* thất bại khi 7 so sánh với 9 và tiếp theo cuối cùng, mệnh đề thứ ba được thử. Tại thời điểm này 7 lại được so sánh với 9. Cùng sự dư thừa xuất hiện khi 1 so sánh với 5. Để tránh điều không hiệu quả này, định nghĩa *part* được viết lại sử dụng cut như sau:

```
part( , [], [], []).
part(X, [Y | Xs], [Y | Ls], Bs) :- X>Y,!, part(X, Xs, Ls, Bs).
part(X, [Y | Xs], Ls, [Y | Bs]) :- part(X, Xs, Ls, Bs).
```

Tất nhiên, cải tiến này có thể được áp dụng cho chương trình QUICKSORT_DL.

10.7.2 Cú pháp mâu thuẫn và các siêu biến

Trước khi tiếp tục, chúng ta cùng xem lại cơ sở của cú pháp Prolog đã được nhắc trên đây.

1. Các biến được ký hiệu bởi các xâu bắt đầu bởi chữ hoa hoặc dấu gạch dưới “_”. Trường hợp đặc biệt Prolog cho phép các biến vô danh, được viết như dấu gạch dưới “_”.
2. Các ký hiệu quan hệ (tên các thủ tục), các ký hiệu hàm và các hằng số không phải số được ký hiệu bởi các xâu bắt đầu chữ thường.
3. Các ký hiệu hàm số một ngôi và hai ngôi có thể được khai báo như các phép toán trung tố hoặc tiền tố không dấu ngoặc.

Bây giờ, ngược với logic cấp một, trong Prolog cùng một tên có thể được sử dụng cho cả ký hiệu hàm và ký hiệu quan hệ. Hơn nữa, một tên có thể được sử dụng cho các ký hiệu hàm số hoặc quan hệ có các ngôi khác nhau. Tiềm ẩn này được gọi là cú pháp mâu thuẫn. Một ký hiệu hàm số hoặc quan hệ f ngôi n được tham chiếu là f/n . Như vậy, trong một chương trình *Prolog*, chúng ta có thể sử dụng cả hai, một ký hiệu quan hệ $p/2$ và các ký hiệu hàm số $p/1$ và $p/2$ xây dựng các term hợp lệ hoặc các atoms như

$p(p(a, b), c, p(X))$.

Trong cú pháp mâu thuẫn, sự khác biệt giữa các ký hiệu hàm số và quan hệ và giữa các terms và các atoms biến mất, nhưng trong ngữ cảnh truy vấn và mệnh đề, nó là rõ ràng ký hiệu nào tham chiếu đến loại cú pháp nào.

Cú pháp mâu thuẫn cùng với tiềm ẩn Prolog khai báo các ký hiệu hàm hai ngôi (và cũng như các ký hiệu quan hệ hai ngôi) như các phép toán trung tố cho phép chúng ta truyền các truy vấn, các mệnh đề và các chương trình như các đối số. Trên thực tế, “:-/2” được khai báo bên trong như một phép toán trung tố và cũng như vậy dấu phẩy “,/2” giữa các atoms, như vậy mỗi mệnh đề thực tế là một term. Điều này làm thuận tiện *siêu lập trình*, tức là viết chương trình mà sử dụng chương trình khác như dữ liệu.

Tiếp theo, chúng ta sẽ giải thích siêu lập trình sẽ được hiện thực trong Prolog như thế nào. Để làm điều đó, chúng ta cần đưa ra thêm một đặc trưng cú pháp nữa. Prolog cho phép sử dụng các biến ở vị trí các atoms, cả trong truy vấn và trong thân mệnh đề. Một biến như vậy được gọi là *siêu biến*. Tính toán với sự có mặt của siêu biến được định nghĩa như trước. Chẳng hạn cho một chương trình Prolog

$p(a).$
a.

Thực hiện truy vấn Prolog $p(X)$, X. trước hết dẫn đến truy vấn a. và sau đó tiếp tục. Ở đây, a vừa là hằng số và là ký hiệu quan hệ ngôi 0. Prolog yêu cầu các siêu biến được khởi tạo đúng đắn trước khi chúng được thực hiện. Nói cách khác, chúng cần được tính toán như một term không phải số ở thời điểm chúng được bắt gặp trong một tính toán. Ngược lại, một lỗi thời gian chạy được phát sinh. Chẳng hạn, đối với chương trình trên và truy vấn $p(X)$, X, Y., tính toán Prolog kết thúc ở lỗi khi truy vấn Y. được tính đến.

10.7.3 Các tiện ích điều khiển

Bây giờ chúng ta xem cú pháp mâu thuẫn kết hợp với siêu biến hỗ trợ siêu lập trình như thế nào. Trong phần này chúng ta hạn chế đến các (siêu) chương trình mà chỉ ra việc đưa ra các tiện ích điều khiển mới như thế nào. Chúng ta bàn ở đây ba ví dụ, mỗi một đưa ra một tiện ích điều khiển thực tế đang sẵn sàng như đã được xây dựng sẵn trong Prolog.

Tuyển

Để bắt đầu, chúng ta có thể định nghĩa phép tuyển logic bằng chương trình đơn giản sau:

```
or(X,Y) :- X.
or(X,Y) :- Y.
```

Một truy vấn tiêu biểu là $or(Q, R)$, ở đó Q và R là các truy vấn qui chuẩn. Phép tuyển là phép toán trung tố $"/2"$ đã được xây dựng sẵn trong Prolog và được định nghĩa thông qua hai qui tắc trên với "or" được thay thế bằng ";". Như vậy thay cho $or(Q, R)$ ta viết $Q;R$.

If-then-else

Hai ví dụ khác bao hàm phép cut. Một cấu trúc **if B then S else T fi** đã xét có thể được đưa vào thông qua chương trình quen thuộc

```
if_then_else(B, S, T):- B,!,S.
if_then_else(B, S, T):- T.

if_then_else(B, S, T):- B,!, S;if_then_else(B, S, T) :- T.
```

Trong Prolog, `if_then_else` là cấu trúc xây sẵn được định nghĩa bên trong bởi hai luật trên. `if_then_else (B, S, T)` được viết như $B \rightarrow S;T$, ở đó $\rightarrow /2$ là được xây sẵn khai báo như phép toán trung tố. Như một ví dụ sử dụng nó, chúng ta viết lại định nghĩa quan hệ part trong chương trình QUICKSORT, bây giờ sử dụng $B \rightarrow S;T$. Để buộc phân tích đúng, chúng ta cần đóng ngoặc cho lệnh $B \rightarrow S;T$.

```
part( , [], [], []).
part(X, [Y | Xs], Ls, Bs) :-
  (X>Y->
  Ls = [Y | L1s], part(X, Xs, L1s, Bs)
  ;
  Bs=[Y | B1s], part(X, Xs, Ls, B1s)
  ).
```

Lưu ý, ở đây chúng ta phải miễn trừ sử dụng mẫu trong các vị trí ra và đưa vào sử dụng tường minh đồng nhất trong thân thủ tục. Bằng việc đưa ra lệnh khác $B \rightarrow S;T$ để xử lý phân tích trường hợp trong đối số thứ hai, chúng ta nhận được một định nghĩa quan hệ part rất giống chương trình hàm:

```
part(X, X1s, Ls, Bs) :-
  (X1s=[]->
  Ls = [], Bs = []
  ;
  X1s=[Y | Xs],
  (X>Y->
  Ls = [Y | L1s], part(X, Xs, L1s, Bs)
  ;
  Bs=[Y | B1s], part(X, Xs, Ls, B1s)
  )
  ).
```

Trên thực tế, trong chương trình này mọi sử dụng đồng nhất dẫn đến sánh và việc sử dụng nó không bao gồm quay lui. Ví dụ này giải thích sử dụng mẫu thường che dấu một phân tích trường hợp ẩn như thế nào. Bằng việc làm cho phân tích trường hợp này tường minh sử dụng cấu trúc **if-then-else**, chúng ta kết thúc với chương trình dài hơn. Cuối cùng, giải pháp độc đáo này với cut trông gần hơn với tinh thần của ngôn ngữ.

Phủ định

Cuối cùng, chúng ta xem xét phép toán phủ định, mà được giả thiết đảo ngược thất bại với thành công. Cái đó là dự định truy vấn *not Q*. Thành công nếu truy vấn *Q* thất bại. Thao tác này có thể dễ dàng được cài đặt thông qua siêu biến và *cut* như sau:

```
not(X):- X, !, fail.
not(_).
```

fail/0 là có sẵn trong Prolog với định nghĩa rỗng. Như vậy, lời gọi thủ tục không có tham số *fail* luôn thất bại. Chương trình hai dòng trên khai thác một số đặc trưng đã trao đổi của Prolog. Trong dòng đầu, *X* được sử dụng như một siêu biến. Bây giờ xét lời gọi *not(Q)*, trong *Q* là một truy vấn. Nếu *Q* thành công, khi đó *cut* được thực hiện. Điều này có tác động là mọi *cacsch* lựa chọn của tính toán *Q* là được bỏ qua và cũng bỏ qua mệnh đề thứ hai. Tiếp theo, thất bại có sẵn *fail* được thực hiện và một thất bại phát sinh. Vì mệnh đề sau đó được bỏ qua, nên truy vấn *not(Q)* thất bại. Nếu, mặt khác, truy vấn *Q* thất bại, thì quay lui sẽ thực thi và mệnh đề thứ hai *not(_)*, được chọn. Nó thành công ngay và vì vậy truy vấn ban đầu *not(Q)* thành công. Vì vậy định nghĩa này của *not* đạt được tác động mong muốn. *not/1* được định nghĩa bên trong bởi hai dòng trên gai cô với khai báo phủ hợp là một ký hiệu một ngôi tiền tố không ngoặc.

Lời gọi

Cuối cùng, chúng ta lưu ý rằng, Prolog cũng cung cấp một cách không trực tiếp sử dụng siêu biến thông qua quan hệ xây sẵn *call/1*. *call/1* được định nghĩa bên trong bởi luật:

```
call(X) :- X.
```

call/1 được sử dụng thường xuyên để che sử dụng tường minh của siêu biến, nhưng kết quả là như nhau.

10.7.4 Phủ định như thất bại

Sự khác biệt giữa tính toán thành công và thất bại là một trong những đặc trưng duy nhất của lập trình logic và Prolog. Trên thực tế, không bộ phận nào của tính toán thất bại tồn tại trong các xu thế lập trình khác.

Trong hầu hết cách tự nhiên sử dụng tính toán thất bại là khai thác phép toán phủ định *not* mà cho phép chúng ta biến thất bại thành thành công, bằng cách coi như truy vấn *not Q*. thành công khi và chỉ khi truy vấn *Q*. thất bại. Bằng cách này, chúng ta sử dụng *not* để thể hiện phủ định của một biểu thức Bool. Trên thực tế, chúng ta đã tham chiếu không hình thức đến sử dụng này của phủ định ở đầu mục 10.7.

Điều này đề xuất một giải nghĩa khai báo của phép toán *not* như một phủ định kinh điển. Giải nghĩa này là đúng chỉ nếu truy vấn phủ định luôn dừng và là truy vấn nền. Lưu ý, trong trường hợp cụ thể, cho trước một thủ tục được định nghĩa bởi qui tắc duy nhất rule p:-p. truy vấn *not p*. không kết thúc. Cũng như vậy, đối với truy vấn *not(X = 1)*., chúng ta nhận được kết quả sau:

```
?- not(X = 1).
no
```

Như vậy, để sinh ra mọi phần tử của một danh sách mà khác với 1, truy vấn đúng là

```
member(X, Ls), not(X = 1).
```

và

```
not not (X = 1), member(X, Ls).
```

Người ta thường tham chiếu đến cách này là phủ định được sử dụng trong Prolog như “phủ định là thất bại”.

Khi sử dụng đúng, nó là đặc trưng mạnh như được chứng thực bởi chương trình đá quý sau. Chúng ta xét bài toán xác định ai thắng trong một trò chơi hữu hạn hai người. Giả sử các di chuyển trong trò chơi này được thể hiện bằng quan hệ *move*. Trò chơi này được giả thiết là hữu hạn, vì vậy chúng ta mặc định là cho trước một vị trí *pos* truy vấn

```
move(pos, Y).
```

sẽ sinh ra hữu hạn các câu trả lời, mà là các di chuyển có thể từ *pos*. Một người chơi sẽ thua nếu anh ta đang ở một vị trí *pos*, từ đó không có di chuyển nào tồn tại, tức là nếu truy vấn *move(pos, Y)* thất bại.

Một vị trí là chiến thắng khi tồn tại một chuyển động mà dẫn đến một thất bại, tức là vị trí không chiến thắng. Sử dụng phép toán phủ định, điều này có thể được viết như:

```
% win(X) :- X is a winning position in the two-person finite game
% represented by the relation move.
win(X) :- move(X, Y), not win(Y).
```

Chương trình chính xác đáng kể này có một giải nghĩa đơn giản. Ngược lại, giải nghĩa thủ tục là khá phức tạp: truy vấn

```
win(pos).
```

xác định *pos* có là vị trí chiến thắng không bằng việc thực hiện một tìm kiếm *minimax* trên cây trò chơi 0-1 được thể hiện bằng quan hệ *move*. Trong thủ tục đệ qui này, trường hợp cơ sở xuất hiện khi lời gọi *move* thất bại. Khi đó lời gọi tương ứng của chiến thắng cũng thất bại.

10.7.5 Lập trình bậc cao và siêu lập trình trong Prolog.

Dựa vào cú pháp mâu thuẫn và siêu biến, lập trình bậc cao và một dạng siêu lập trình có thể dễ dàng hiện thực trong Prolog. Để giải thích điều này, chúng ta cần thêm hai quan hệ xây sẵn. Mỗi một trong chúng thuộc vào một phân loại khác nhau.

Các tiện ích giám sát Term

Prolog đề xuất một số quan hệ xây sẵn mà cho phép chúng ta giám sát, so sánh và phân tách các terms. Một trong số chúng là

```
=../2 (gọi là univ)
```

mà cho phép chúng ta chuyển đổi giữa một term và các thể hiện của nó như một danh sách. Thay vì mô tả chính xác ý nghĩa của chúng, chúng ta chỉ minh họa việc sử dụng thông qua truy vấn sau:

```
?- Atom =.. [square, [1,2,3,4], Ys].
Atom = square([1,2,3,4], Ys).
```

Bên trái là Atom được đồng nhất với một term (hoặc, tương đương, một atom), ở đây là square([1,2,3,4], Ys), thể hiện bởi một danh sách ở vế phải, ở đây là [square,[1,2,3,4], Ys]. Trong danh sách này, đầu danh sách là ký hiệu hàm và đuôi là các đối số của nó. Sử dụng *univ*, người ta xây dựng các *terms* và truyền chúng như các đối số. Thử ví hơn, người ta có thể xây dựng *atoms* và thực hiện chúng nhờ các tiện ích siêu biến. Điều này có thể hiện thực lập trình bậc cao trong Prolog theo nghĩa các quan hệ có thể được truyền như các đối số. Để mô tả điểm này, chúng ta xem xét chương trình MAP sau:

```
% map(P, Xs, Ys) :- the list Ys is the result of applying P
% elementwise to the list Xs.
map(P, [], []).
map(P, [X | Xs], [Y | Ys]) :- apply(P, [X, Y]), map(P, Xs, Ys).
% apply(P, [X1, ..., Xn]) :- execute the atom P(X1, ..., Xn).
apply(P, Xs) :- Atom =.. [P|Xs], Atom.
```

Theo qui tắc cuối cùng, *univ* được sử dụng để xây dựng một *atom*. Lưu ý việc sử dụng một siêu biến Atom. MAP là phần của Prolog trong chương trình hàm bậc cao quen thuộc và nó có hành vi theo cách mong muốn. Chẳng hạn, cho trước chương trình

```
%square(X, Y):- Y is the square of X.
square(X, Y) :- Y is X*X.
```

chúng ta nhận được

```
?- map(square, [1,2,3,4], Ys).
Ys = [1, 4, 9, 16]
```

10.8 Đánh giá Prolog

Prolog, vì sự độc đáo của nó trong chứng minh định lý tự động, là một ngôn ngữ lập trình bất thường. Nó dẫn đến một phong cách lập trình khác hẳn và một cách nhìn lập trình khác. Một số chương trình Prolog đẹp được thể hiện ở đây đã nói lên điều đó. Chúng ta cũng lưu ý rằng cùng một chương trình Prolog có thể được sử dụng cho các mục đích khác nhau như tính toán, kiểm thử hoặc hoàn thiện một giải pháp hoặc tính toán mọi giải pháp. Các chương trình có thể không dễ dàng được viết trong các xu thế lập trình khác. Các biến logic là một đặc trưng duy nhất, hữu ích của lập trình duy logic. Ngoài ra, các chương trình Prolog thuần túy có một giải nghĩa kép như các công thức logic. Theo nghĩa này lập trình logic hỗ trợ lập trình khai báo.

Prolog là một ngôn ngữ tự nhiên cho lập trình các ứng dụng trí tuệ nhân tạo, như chứng minh định lý tự động, các chương trình hệ chuyên gia, và học máy, ở đó suy luận cần thiết để kết hợp với tính toán, chương trình trò chơi và các hệ thống hỗ trợ quyết định.

Bây giờ ta điểm qua một vài nhược điểm của Prolog. Nó không hỗ trợ các kiểu theo nghĩa không kiểm tra các truy vấn có sử dụng chương trình theo cách mong muốn không. Vì không kiểm tra,

các lỗi kiểu dễ làm cho khó phát hiện. Không có kiểm tra kiểu làm cho một số cải tiến tính hiệu quả của cài đặt khó thực hiện và nhiều lỗi thời gian chạy không ngăn chặn được. Ngoài ra có thể kể đến việc tính toán số học khó thấy, dễ lỗi, các cơ chế điều khiển của Prolog là riêng biệt khó nắm bắt, ngữ nghĩa phức tạp của các tiện ích xây sẵn, không có modules, không có đối tượng.

Nhưng không có ngôn ngữ lập trình nào khác thành công bao hàm logic cấp một theo cách heeiju quả. Prolog là một ngôn ngữ dựa trên luật và nhờ vào một số rất lớn các quan hệ xây sẵn nó là một ngôn ngữ lập trình đa dụng.

Câu hỏi ôn tập Nguyên lý các ngôn ngữ lập trình

(Tài liệu xem bài giảng tiếng việt hoặc giáo trình Mitchell tiếng Anh)

Bài 1.

1. Người ta nói Pascal là ngôn ngữ lập trình có cấu trúc. Tính cấu trúc của Pascal được thể hiện qua 3 yếu tố: cấu trúc trong dữ liệu, cấu trúc trong các toán tử và cấu trúc trong công cụ thủ tục như thế nào? Tại sao khó xây dựng chương trình ứng dụng lớn trong Pascal.
2. *Tại sao nói C tỏ ra thích hợp với lập trình hệ thống, có tính tương thích cao, tốc độ cao, nhưng hệ thống kiểu không chặt chẽ và khó kiểm soát chương trình lớn.
3. *Tại sao nói C++ không phải hướng đối tượng hoàn toàn, mà là đa hướng và C++ là ngôn ngữ thích hợp cho việc xây dựng những chương trình lớn nhưng phức tạp và khó kiểm soát.
4. *Tại sao nói Java là ngôn ngữ hướng đối tượng hoàn toàn, đơn giản, độc lập với cấu trúc máy, dễ chuyển mang, an toàn. Lý do chương trình Java chậm, các cách khắc phục để tăng tốc độ.
5. *Họ các ngôn ngữ dot net có ưu nhược điểm gì: có nhiều công cụ hỗ trợ, có dễ dàng tích hợp với nhau được không, vai trò của CRL như thế nào, có phụ thuộc nền tảng không.
6. Nêu mục đích và các đặc tính của họ các ngôn ngữ kịch bản. So sánh ASP, Javascript và php?
7. *Bạn cho ví dụ truyền tham số hàm trong Javascript.
8. *Bạn cho ví dụ trả về hàm trong lời gọi hàm trong Javascript.

Bài 2.

1. So sánh những tình huống giống nhau trong việc xác định vị trí con trỏ trong C và đánh chỉ số mảng.
2. *Một đặc tính là C có khả năng khắc phục lỗi kiểu. Cho ví dụ minh họa. Đây là ưu điểm hay nhược điểm của C.
3. Người lập trình C phải tự xin cấp bộ nhớ và giải phóng bộ nhớ như thế nào. Theo bạn đây là ưu điểm hay nhược điểm?
4. *Bạn cho biết đặc tính của ngôn ngữ lập trình ML. Tại sao nói nó là ngôn ngữ lập trình hàm.
5. *Cho biết khai báo kiểu danh sách trong ML như thế nào. Cho ví dụ hàm trên danh sách.
6. *Cho biết khai báo hàm trong ML như thế nào? Sử dụng mẫu ra sao, cho ví dụ.
7. Khai báo kiểu dữ liệu do người sử dụng định nghĩa trong ML như thế nào? Cho ví dụ.
8. Thế nào là ô tham chiếu, L value và R value trong ML. Cho ví dụ.

Bài 3.

1. *Thế nào là an toàn kiểu. Bạn có thể nói gì về an toàn kiểu của C, C++, ML và Java.
2. *Kiểm tra kiểu thời gian dịch và kiểm tra kiểu thời gian chạy khác nhau như thế nào, ưu nhược điểm của chúng. Cho ví dụ minh họa.
3. *ML có thuật toán suy luận kiểu dựa trên cơ sở nào? Cho ví dụ suy luận kiểu của hàm trong ML
4. Nêu cách khai báo hàm đệ qui với nhiều thành phần trong. Cho ví dụ
5. *Nêu cách sử dụng đa hình tham số kiểu trong ML. Cho ví dụ cài đặt swap trong ML.
6. *Thế nào là đa hình ghi đè. Cho ví dụ trong ML.

7. Nêu ví dụ khai báo kiểu cây nhị phân đa hình trong ML (các lá của nó có kiểu tham số). Xét ví dụ trong ML xem một giá trị cho trước có trên cây không.
8. *Sắp xếp đa hình là gì? Cho ví dụ cài đặt sắp xếp đa hình trong ML

Bài 4.

1. Thế nào là ngôn ngữ lập trình có cấu trúc khối. Nêu khái niệm biến cục bộ và tổng thể. Cho các ví dụ minh họa.
2. *Hãy trình bày mô hình máy tính đơn giản. Nêu cách thức hoạt động của nó, khi thực thi một chương trình.
3. *Nêu cấu tạo của bản ghi kích hoạt. Lấy ví dụ bản ghi kích hoạt của các hàm.
4. *Nêu khái niệm phạm vi và thời gian sống của các biến. Cho ví dụ chỉ rõ sự khác biệt của chúng.
5. *Hai cơ chế truyền tham số cho lời gọi hàm là truyền tham trị và truyền tham chiếu khác nhau như thế nào. Cho ví dụ minh họa. Lợi thế của mỗi cơ chế.
6. *Thế nào là phạm vi tĩnh và phạm vi động. Cho ví dụ nêu sự khác biệt giữa hai loại phạm vi.
7. Giải thích: việc xác định hàm và các biến tổng thể trong C và ML có phạm vi tĩnh. Cho ví dụ minh họa thông qua liên kết điều khiển.
8. Giải thích: việc xác định các lỗi trong hầu hết các ngôn ngữ có phạm vi động. Cho ví dụ minh họa thông qua liên kết truy cập.
9. *Thế nào là đệ qui đuôi. Đối với đệ qui đuôi việc tái sử dụng ngăn xếp được thực hiện như thế nào cho ví dụ minh họa
10. *Cho ví dụ chuyển từ đệ qui thường sang đệ qui đuôi. Tại sao việc thực thi lời gọi đệ qui đuôi giống như việc thực hiện một vòng lặp.
11. *Thế nào là truyền tham số hàm. Cho ví dụ trong ML
12. *Thế nào là trả về hàm trong lời gọi hàm. Cho ví dụ trong ML. Theo bạn nghĩ nó đem lại những lợi ích gì?
13. *Cho ví dụ hàm map trên danh sách trong ML (hàm tác động đến từng phần tử của danh sách).

Bài 5:

1. Nêu các đặc tính của lập trình cấu trúc. Cho các ví dụ minh họa
2. Thế nào là trừu tượng thủ tục, cho ví dụ
3. Thế nào là trừu tượng dữ liệu, cho ví dụ
4. Cho ví dụ kiểu dữ liệu trừu tượng trong ML
5. *Nêu cách sử dụng C++ function templates. Cho ví dụ dùng template như tham số kiểu trong C++.
6. *Nêu sự khác biệt giữa cơ chế tham số kiểu trong C++ và trong ML. Cho ví dụ minh họa.
7. *Nêu đặc trưng và cho ví dụ dùng thư viện template chuẩn C++ (STL). Cho ví dụ minh họa.

Bài 6

1. *Nêu bốn khái niệm cơ bản trong ngôn ngữ hướng đối tượng
2. *Thế nào là tìm kiếm động. Cho ví dụ
3. *Thế nào là trừu tượng, cho các ví dụ minh họa.
4. *Thế nào là kiểu con, cho ví dụ minh họa
5. *Thế nào là kế thừa, cho ví dụ minh họa
6. Cho ví dụ kế thừa không phải kiểu con.

7. *Mẫu thiết kế (design patterns) dùng để làm gì. Cho ví dụ về singleton pattern hoặc visitor pattern.

Bài 7

1. *Nêu các mục đích và ràng buộc thiết kế C++. Sự tương thích với C và thành công của C++
2. Nêu các bổ sung cho C trong C++ mà không liên quan đến đối tượng
3. Nêu các đặc tính hướng đối tượng của C++.
4. *Nêu các quyết định tốt và các chỗ còn vấn đề khi thiết kế C++
5. Nêu định nghĩa lớp trong C++, cho ví dụ minh họa.
6. *Nêu định nghĩa lớp suy diễn trong C++, cho ví dụ minh họa. Nêu cách giải quyết xung đột đa kế thừa trong lớp suy diễn
7. *Nêu định nghĩa hàm ảo trong C++, cho ví dụ minh họa.

Bài 8

1. *Nêu các mục tiêu chính khi thiết kế ngôn ngữ lập trình Java. Tại sao nói Java là ngôn ngữ hướng đối tượng thuần túy.
2. *Các quyết định tốt làm cho Java được sử dụng rộng rãi hiện nay là gì?
3. *Lớp trong Java có gì khác biệt so với C++. Phương thức và trường tĩnh trong lớp Java có ý nghĩa gì. Cho ví dụ minh họa.
4. *Tính đa hình – tải chồng của các phương thức trong một lớp của Java được thực hiện như thế nào, cho ví dụ.
5. *Kế thừa trong Java có gì khác biệt so với C++. Phương thức lớp con ghi đè phương thức lớp cha như thế nào. Cho ví dụ.
6. *Giao diện trong Java dùng để làm gì. Một lớp có thể cài đặt nhiều giao diện không, cho ví dụ minh họa.
7. *Lớp trừu tượng dùng để làm gì, hãy phân biệt nó với giao diện. Cho ví dụ minh họa
8. Tính đa hình kiểu con trong Java được cài đặt qua lớp Object như thế nào. Lấy ví dụ chẳng hạn cài đặt Stack kiểu tham số qua Object.
9. *Phiên bản mới của Java có hỗ trợ cài đặt tham số kiểu trực tiếp không cần qua Object không, điều đó được thực hiện như thế nào

GỢI Ý TRẢ LỜI CÁC CÂU HỎI

Bài 1.

1. *Tại sao nói C tỏ ra thích hợp với lập trình hệ thống, có tính tương thích cao, tốc độ cao, nhưng hệ thống kiểu không chặt chẽ và khó kiểm soát chương trình lớn.

Trả lời : C là ngôn ngữ lập trình có cấu trúc. Nhưng nói chặt chẽ về mặt kỹ thuật thì C không phải là ngôn ngữ lập trình có cấu trúc chính cống vì trong C không cho phép các khối giống nhau (chẳng hạn bạn không thể khai báo hàm này trong hàm khác). C là ngôn ngữ cấp trung vì nó cho phép thao tác trên các bit, byte, và địa chỉ. C kết hợp các yếu tố mềm dẻo của ngôn ngữ bậc cao và khả năng điều khiển mạnh của ASSEMBLER. Do vậy, C tỏ ra thích hợp với lập trình hệ thống.

Một đặc điểm nổi bật của C là C có tính tương thích cao. Chương trình viết bằng C cho một loại máy hoặc hệ điều hành này có thể chuyển dễ dàng sang loại máy hoặc hệ điều hành khác. Có thể nạp nhanh và tương thích cho các loại chip vi xử lý. Hiện nay hầu hết các loại máy tính đều có trình biên dịch C.

Một chương trình được viết bằng ngôn ngữ lập trình C sẽ tối ưu, chạy với tốc độ cao và tiết kiệm bộ nhớ. ngôn ngữ lập trình C có tính mềm dẻo rất cao từ cách khai báo biến có thể trùng lặp trình cũng một chương trình do C có con trỏ nên có thể can thiệp sâu vào các hệ điều hành, các phân vùng dữ liệu quan trọng. do C có tính mềm dẻo như trên nên C cũng có nhược điểm của nó. Khi chạy những chương trình lớn cần các thuật toán phức tạp.

Tuy vậy, ngôn ngữ lập trình C chỉ thích hợp với những chương trình hệ thống hoặc những chương trình đòi hỏi tốc độ. Còn với những bài toán lớn và phức tạp thì cũng như PASCAL, C rất khó kiểm soát chương trình.

2. *Tại sao nói C++ không phải hướng đối tượng hoàn toàn, mà là đa hướng và C++ là ngôn ngữ thích hợp cho việc xây dựng những chương trình lớn nhưng phức tạp và khó kiểm soát.

Trả lời : C++ không phải là ngôn ngữ hướng đối tượng hoàn toàn mà là ngôn ngữ "đa hướng". Vì C++ hỗ trợ cả lập trình hướng hành động và lập trình hướng đối tượng.

C++ kết hợp những ý tưởng hay nhất của lập trình có cấu trúc với việc phân chia một bài toán thành các nhóm nhỏ có quan hệ với nhau, mỗi nhóm con đó là một đối tượng chứa các lệnh và dữ liệu riêng của nó.

C++ đưa vào các khái niệm hàm ảo, quá tải hàm, quá tải toán tử cho phép tạo ra các kiểu dữ liệu trừu tượng, hỗ trợ thừa kế bội.

Mục tiêu của C++ là tiếp cận những ý tưởng của phương pháp luận hướng đối tượng và trừu tượng dữ liệu.

Các đặc tính của C++ cho phép người lập trình xây dựng những thư viện phần mềm có chất lượng cao phục vụ những đề án lớn.

C++ là ngôn ngữ thích hợp cho việc xây dựng những chương trình lớn như các hệ soạn thảo, chương trình dịch, các hệ quản trị cơ sở dữ liệu, các hệ truyền thông.

3. *Tại sao nói Java là ngôn ngữ hướng đối tượng hoàn toàn, đơn giản, độc lập với cấu trúc máy, dễ chuyển mạng, an toàn. Lý do chương trình Java chậm, các cách khắc phục để tăng tốc độ.

Trả lời: Hướng đối tượng trong Java tương tự như C++ nhưng Java là một ngôn ngữ lập trình hướng đối tượng hoàn toàn. Tất cả mọi thứ đề cập đến trong Java đều liên quan đến các đối tượng được định nghĩa trước, thậm chí hàm chính của một chương trình viết bằng Java (đó là hàm main) cũng phải đặt bên trong một lớp. Hướng đối tượng trong Java không có tính đa kế thừa (multi inheritance) như trong C++ mà thay vào đó Java đưa ra khái niệm interface để hỗ trợ tính đa kế thừa.

Tính đơn giản :

Java đơn giản vì, mặc dù dựa trên cơ sở C++ nhưng Sun đã cẩn thận lược bỏ các tính năng khó nhất của C++ để làm cho ngôn ngữ này dễ sử dụng hơn. Là một ngôn ngữ lập trình hoàn toàn mới, nó buộc phải có dáng vẻ và sự cảm nhận tương tự như các ngôn ngữ phổ biến hiện hành đồng thời đòi hỏi khoảng thời gian huấn luyện lại tối thiểu và thân thiện hơn với người dùng.

Đơn giản, ngôn ngữ này cũng rất nhỏ - nên nhớ rằng từ đầu nó đã được xây dựng để dùng cho điện tử dân dụng như đầu chạy bằng video và hộp điều khiển từ xa, những thiết bị có không gian lưu trữ rất hạn chế.

Độc lập với cấu trúc máy : Đây là thuộc tính đặc sắc nhất của Java. Java được tạo ra với tiêu chí "Viết (code) một lần, thực thi khắp nơi" ("Write Once, Run Anywhere" (WORA)).

Tính năng biên dịch một lần và chạy nhiều nền có thể đạt được nhờ just-in-time compiler (JIT), chuyển mã bytecode của Java sang mã máy khi chương trình đang chạy. Chương trình phần mềm viết bằng Java có thể chạy trên mọi nền tảng (platform) khác nhau thông qua một môi trường thực thi với điều kiện có máy ảo Java thích hợp hỗ trợ nền tảng đó. Môi trường thực thi của Sun Microsystems hiện hỗ trợ Sun Solaris, Linux, Mac OS, FreeBSD, Windows và nhiều hệ thống cài đặt trên các thiết bị di động

Dễ chuyển mạng : Java được lập ra để hoạt động trên mạng và có các thủ tục để có thể quản lý các giao thức mạng như TCP/IP, FTP và HTTP. Nói cách khác, Java được xây dựng để thực hiện hoàn toàn thích hợp trên Internet. Chương trình Java thậm chí có thể xâm nhập vào các đối tượng khác thông qua Internet bằng cách sử dụng URL (địa chỉ Web) để định vị chúng.

An Toàn : Khả năng hướng mạng của Java tự động đưa ra yêu cầu về an toàn. Đặc tính an toàn của ngôn ngữ lập trình này bắt nguồn từ việc nó có những phần hạn chế được cài sẵn nhằm đề phòng các chương trình Java thực hiện những chức năng như ghi vào ổ cứng của người dùng hay cho phép virus từ mạng hoặc từ môi trường phân tán thâm nhập vào.

Lý do chương trình Java chậm : Do phải hoạt động thông qua máy tính ảo JVM nên tốc độ thực hiện ứng dụng của Java khá chậm. Điều này lý giải tại sao Java vẫn chưa thực sự được sử dụng như một ngôn ngữ lập trình chính hiện nay.

4. *Họ các ngôn ngữ dot net có ưu nhược điểm gì: có nhiều công cụ hỗ trợ, có dễ dàng tích hợp với nhau được không, vai trò của CRL như thế nào, có phụ thuộc nền tảng không.

Trả lời :

Ưu điểm ngôn ngữ lập trình .NET:

Mã nguồn rõ ràng, trong .NET 2.0, mã nguồn tách biệt với giao diện HTML.
 Chạy nhanh hơn PHP?!
 Visual Studio có thể sinh mã, tiết kiệm thời gian viết code.
 Có thể dùng C#, VB hoặc Managed C++.
 Đi cặp MSSQL, Oracle.
 Nhược điểm ngôn ngữ lập trình .NET:
 Không thể chạy trên Linux (mặc dù đã có thể nhưng thiết nghĩ ứng dụng lớn sẽ có sơ xuất?!)
 Nếu không có Visual Studio thì khó viết code!
 Ứng dụng tìm kiếm, tuy nhiên vẫn không có cơ sở chắc chắn!

5. ***Bạn cho ví dụ truyền tham số hàm trong Javascript.**

```
<html>
<script language="javascript">
function setFocus(tf)
{
var element = eval(document.frm1.+tf);
element.focus();
}
</script>

<body onload="setFocus(txt_name)">
<form name="frm1">
<input type="textfield" name="txt_name">
</form>
</body>
</html>
```

6. ***Bạn cho ví dụ trả về hàm trong lời gọi hàm trong Javascript.**

```
function hamCha(){
var bCha='Biến của cha';
//Khai báo inner-fuction
function hamCon(){
alert(bCha); //-->'Biến của cha'
}
//trả về hàm con (chứ không gọi hàm con)
return hamCon;
}
//ham sẽ tham chiếu đến hamCon (inner-function)
var ham = hamCha();
ham();
```

Bài 2.

Câu 2 *Một đặc tính là C có khả năng khắc phục lỗi kiểu. Cho ví dụ minh họa. Đây là ưu điểm hay nhược điểm của C.

C được tạo ra từ các ngôn ngữ không có kiểu. Sự tương thích ngược với code C đã tồn tại đã ngăn cản các hạn chế kiểu mạnh. Hầu hết lập trình viên C đều coi việc kiểm tra kiểu yếu của nhiều chương trình dịch C là một nhược điểm. Trên thực tế, một trong những ưu điểm của C++ đối với C là C++ cung cấp việc kiểm tra kiểu tốt hơn.

Mặc dù có những đặc điểm nêu trên, nhưng C hiển nhiên đáp ứng yêu cầu là ngôn ngữ cài đặt hệ thống đủ hiệu quả để thay thế ngôn ngữ Assembly, cũng khá khái quát và uyển chuyển để thể hiện các thuật toán và tương tác trong nhiều môi trường khác nhau.

Tùy từng trường hợp mà có thể xét đây là ưu hay nhược điểm của C

Câu 4. *Bạn cho biết đặc tính của ngôn ngữ lập trình ML. Tại sao nói nó là ngôn ngữ lập trình hàm.

ML có thể gọi gần như là ngôn ngữ lập trình hàm với các đặc trưng mệnh lệnh hoặc có thể là ngôn ngữ mệnh lệnh hướng hàm số. ML có các đặc trưng hàm linh hoạt, tương tự như LISP, cho phép tạo hàm trên cùng hàng như một phần của biểu thức, hàm số được truyền như đối số cho hàm và trả về như kết quả hàm số. ML cũng được mở rộng song song, làm cho nó phù hợp với việc phát triển các hệ thống song song và có cả mở rộng hướng đối tượng.

Một trong những đặc tính quan trọng của ML là hệ thống kiểu của nó được mở rộng từ hệ thống kiểu rất thành công của Pascal. Không giống như C mà có nhiều kẽ hở, hệ thống kiểu ML là được chứng minh theo nghĩa toán học chính xác.

Đặc biệt, nếu bộ kiểm tra kiểu của ML xác định biểu thức có một kiểu nào đó, thì mọi tính toán của biểu thức sẽ đảm bảo rằng sẽ tạo ra giá trị hợp lệ của kiểu đó. Chẳng hạn, biểu thức có kiểu 'con trỏ đến xâu', thì giá trị của biểu thức đó được đảm bảo là con trỏ đến bộ nhớ được cấp phát cho xâu. Nó không thể là con trỏ 'đu đưa' đến vị trí được cấp để lưu một giá trị nào khác ngoài xâu.

Câu 5. *Cho biết khai báo kiểu danh sách trong ML như thế nào. Cho ví dụ hàm trên danh sách.

Một danh sách là một tập hợp có thứ tự các giá trị cùng kiểu.

Một cách để tạo một danh sách là một chuỗi những giá trị trong những dấu móc vuông.

Danh sách các số nguyên

[1, 2, 3];

val it = [1,2,3] : int list

Danh sách chuỗi

- ["just", "a", "test"];

```
val it = ["just", "a", "test"] : string list
```

6*Cho biết khai báo hàm trong ML như thế nào? Sử dụng mẫu ra sao, cho ví dụ.

hàm có thể được khai báo với từ khóa `fun` thay cho `val`. Dạng tổng quát của đầu vào người sử dụng và đầu ra của chương trình dịch là

```
-- fun <identifier> <arguments> = <expression>;
val <identifier> = fn <arg_type> -> <result_type>
```

Nó khai báo hàm có tên là `identifier`. Kiểu đối được xác định bởi dạng `<arguments>` và kiểu kết quả được xác định bởi dạng của `<expression>`.

Sau đây là ví dụ :

```
-- fun f(x) = x + 5;
val f = fn : int -> int
```

Khai báo này gán giá trị hàm với định danh `f`. Giá trị của `f` là một hàm số từ số nguyên vào số nguyên. Cũng hàm đó có thể khai báo bởi khai báo `val` như sau :

```
-- val f = fn x => x+5;
val f = fn : int -> int
```

Bài 3.

9. *Thế nào là an toàn kiểu. Bạn có thể nói gì về an toàn kiểu của C, C++, ML và Java.

An toàn kiểu

Một ngôn ngữ lập trình là an toàn kiểu nếu không có chương trình nào được cho phép vi phạm sự khác biệt kiểu của nó. Đôi khi điều này là không rõ ràng hiểu khác biệt kiểu là gì trong một ngôn ngữ lập trình cụ thể nào đó. Tuy nhiên, có một số sự khác biệt kiểu mà có ý nghĩa và quan trọng trong tất cả các ngôn ngữ. Chẳng hạn, hàm có kiểu khác với số nguyên. Như vậy, bất cứ ngôn ngữ nào mà cho phép số nguyên được sử dụng như hàm là không an toàn kiểu. Một hành động khác mà chúng ta luôn xem xét là lỗi kiểu là truy cập bộ nhớ mà không cấp cho chương trình.

10. *Kiểm tra kiểu thời gian dịch và kiểm tra kiểu thời gian chạy khác nhau như thế nào, ưu nhược điểm của chúng. Cho ví dụ minh họa.

Kiểm tra thời gian chạy.

Trong các ngôn ngữ lập trình có kiểm tra kiểu trong thời gian chạy, chương trình dịch sinh ra mã sao cho khi mã được thực hiện, sẽ kiểm tra để tin tưởng rằng các toán hạng có kiểu đúng đắn.

Ưu điểm của kiểm tra kiểu thời gian chạy là nó bắt được lỗi kiểu.

Nhược điểm là thời gian chạy dành cho các kiểm tra này.

Kiểm tra thời gian dịch. Nhiều ngôn ngữ lập trình hiện đại được thiết kế sao cho có khả năng kiểm tra các biểu thức có tiềm năng lỗi kiểu. Trong các ngôn ngữ này, nói chung là từ chối các chương trình không qua được kiểm tra kiểu thời gian dịch.

Ưu điểm của kiểm tra kiểu thời gian dịch là nó bắt lỗi sớm hơn so với kiểm tra kiểu thời gian chạy: người phát triển chương trình được cảnh báo về lỗi trước khi chương trình được đưa cho người khác hoặc chuyển giao như sản phẩm. Vì kiểm tra kiểu trong thời gian chạy có thể loại bỏ sự cần thiết để kiểm tra một số lỗi trong thời gian chạy, kiểm tra thời gian dịch làm cho nó tạo mã hiệu quả hơn. Chẳng hạn, mã dịch ML là nhanh hơn Lisp từ hai đến bốn lần. Lý do chính tăng tốc độ này là kiểm tra kiểu tĩnh của chương trình ML giảm đáng kể sự cần thiết phải kiểm tra thời gian chạy.

Tính thận trọng của Kiểm tra kiểu thời gian dịch. Tính chất của kiểm tra kiểu thời gian dịch là chương trình dịch cẩn thận trọng. Điều đó có nghĩa là kiểm tra kiểu thời gian dịch sẽ tìm mọi câu lệnh và biểu thức mà sinh ra lỗi thời gian chạy, nhưng nó cũng đánh dấu các lệnh và biểu thức có lỗi ngay cả nếu chúng không gây ra lỗi thời gian chạy. Cụ thể hơn, đa số bộ kiểm tra đề sound và conservative. Bộ kiểm tra được gọi là sound, nếu không có chương trình nào có lỗi được xem là đúng đắn. Bộ kiểm tra được gọi là conservative, nếu có một số chương trình không có lỗi vẫn được coi là có lỗi.

Có lý do cho việc đa số bộ kiểm tra là conservative: Đối với mọi ngôn ngữ lập trình Turing - đầy đủ, tập các chương trình có thể gây ra lỗi thời gian chạy là không giải được. Điều đó suy ra từ tính không giải được của bài toán dừng. Để xem tại sao, xét đoạn chương trình sau:

```
if (complicated-expression-that-could-run-forever)
  then (expression-with-type-error)
  else (expression-with-type-error)
```

Không giải được bài toán, biểu thức này có sinh ra lỗi thời gian chạy hay không, vì chỉ có một cách cho biểu thức có lỗi kiểu được tính toán là ‘biểu thức phức tạp mà có thể chạy mãi mãi’ phải dừng. Do đó, việc giải quyết ‘biểu thức có gây ra lỗi thời gian chạy không’ bao gồm cả việc giải quyết ‘biểu thức phức tạp có thể chạy mãi mãi’ có dừng hay không.

Vì tập các chương trình có lỗi thời gian chạy là không giải được, nên không có bộ kiểm tra kiểu thời gian dịch có thể tìm được lỗi kiểu một cách chính xác. Vì mục đích của kiểm tra kiểu là ngăn chặn lỗi, nên các bộ kiểm tra kiểu của ngôn ngữ có kiểu an toàn là conservative. Sẽ có ích nếu kiểm tra kiểu tìm ra lỗi kiểu, và như hệ quả của việc không giải được ‘bài toán dừng’ một số chương trình mà có thể được thực hiện không có lỗi thời gian chạy sẽ không vượt qua được kiểm tra kiểu thời gian dịch

11. *ML có thuật toán suy luận kiểu dựa trên cơ sở nào? Cho ví dụ suy luận kiểu của hàm trong ML
- Suy luận kiểu là quá trình xác định kiểu của biểu thức dựa trên các kiểu đã biết của các ký hiệu xuất hiện trong các biểu thức đó. Sự khác nhau giữa suy luận kiểu và kiểm tra kiểu thời gian dịch là ở mức độ. Thuật toán kiểm tra kiểu duyệt qua chương trình kiểm tra các kiểu khai báo bởi lập trình viên có phù hợp với các yêu cầu của ngôn ngữ không. Trong suy luận kiểu, ý tưởng đưa ra là một số thông tin không cần đặc tả, và một số dạng suy diễn logic sẽ được dùng để xác định kiểu của các biến dựa theo cách chúng được sử dụng. Suy

diễn kiểu được phát triển bởi Robin Milner cho ngôn ngữ lập trình ML. Các ý tưởng tương tự được phát triển độc lập bởi Curry và Hindley trong khi nghiên cứu tính toán lambda.

Mặc dù các suy diễn kiểu thực tế đã được phát triển cho ML, nhưng suy diễn kiểu có thể được áp dụng cho rất nhiều ngôn ngữ lập trình. Chẳng hạn, về mặt nguyên tắc suy diễn kiểu có thể áp dụng cho C hoặc cho các ngôn ngữ khác. Chúng ta tìm hiểu suy diễn kiểu tương đối chi tiết, vì nó thể hiện các vấn đề trọng tâm của kiểm tra kiểu và vì suy diễn kiểu cũng thể hiện một số vấn đề quan trọng của thuật toán tìm các kiểu lỗi của chương trình.

Thêm vào đó, để cung cấp dạng kiểm tra kiểu thời gian dịch linh hoạt, suy diễn kiểu ML cũng hỗ trợ đa hình. Như chúng ta đã thấy, khi xem xét kỹ thuật toán suy diễn kiểu, thuật toán này sẽ nhận biết mọi kiểu của các biến và xác định chúng cần phải bằng một kiểu chuyên biệt nào đó như `int`, `bool` hoặc `string`. Trong trường hợp khác, kiểu của một hàm nào đó có thể chứa các biến kiểu mà không ràng buộc theo cách mà hàm được định nghĩa. Trong các trường hợp này, hàm đó có thể được áp dụng cho các đối số mà kiểu của chúng sánh với dạng được cho bởi biểu thức kiểu chứa các biến kiểu.

Mặc dù suy diễn kiểu và đa hình là các khái niệm độc lập, chúng ta sẽ bàn về đa hình trong bối cảnh suy diễn kiểu, vì đa hình được nảy sinh một cách tự nhiên theo cách các biến kiểu được sử dụng trong suy diễn kiểu.

Ví dụ :

```
- fun f1(x) = x+2;
val f1 = fn : int → int
```

Hàm `f1` cộng 2 cho đối số của nó. Trong ML, 2 là hằng nguyên; số thực 2 cần phải viết dạng `2.0`. Phép toán `+` được tải đè; nó có thể là phép cộng nguyên hoặc phép cộng thực. Trong hàm này, tuy nhiên, phép cộng cần phải là cộng nguyên, vì 2 là số nguyên. Do đó, đối số của hàm `x` cần phải là số nguyên. Tập hợp tất cả các quan sát này, chúng ta nhận được `f1` cần phải có kiểu `int -> int`

12. *Nêu cách sử dụng đa hình tham số kiểu trong ML. Cho ví dụ cài đặt swap trong ML.

Đặc trưng chính của đa hình tham số là tập các kiểu gắn kết với hàm hoặc một giá trị khác được cho trước bởi biểu thức kiểu mà chứa biến kiểu. Chẳng hạn, hàm ML mà sắp xếp các danh sách có thể có kiểu ML

```
sort:('a * 'a → bool) * 'a list → 'a list
```

Diễn tả bằng lời, sắp xếp có thể áp dụng cho một cặp bất kỳ bao gồm một hàm và một danh sách, hàm có kiểu `'a * 'a -> bool`, trong đó kiểu `'a` cần phải là kiểu của các phần tử của danh sách. Đối số hàm là thao tác nhỏ hơn được sử dụng để xác định thứ tự trên danh sách được sắp xếp.

Trong đa hình tham số, hàm có thể có vô hạn kiểu, sẽ có vô hạn cách thay các biến kiểu bằng các biến thực tế. Chẳng hạn, hàm sắp xếp có thể được dùng để sắp xếp danh sách số

nguyên, danh sách các danh sách số nguyên hoặc danh sách các danh sách các danh sách số nguyên, ...

Đa hình tham số có thể là ẩn hoặc tường minh. Trong đa hình tham số tường minh, văn bản chương trình chứa các biến kiểu mà xác định cách hàm hoặc các giá trị khác có thể được coi như đa hình. Thêm vào đó, đa hình tường minh thường bao gồm việc khởi tạo tường minh hoặc áp dụng kiểu để chỉ ra biến kiểu được thay thế như thế nào vào các kiểu nào đó trong khi dùng giá trị đa hình. Các templates C++ là các ví dụ phổ cập về đa hình tường minh. Đa hình ML được gọi là đa hình ẩn, vì chương trình mà khai báo và sử dụng hàm đa hình không cần chứa kiểu – thuật toán kiểm tra kiểu tính toán khi hàm là đa hình và tính toán khởi tạo biến kiểu nếu cần

Ví dụ:

```
- fun swap(x,y) = let val tmp = x in x := !y; y := !tmp end;
val swap = fn : 'a ref * 'a ref -> unit
```

13. *Thế nào là đa hình ghi đè. Cho ví dụ trong ML.

Nhiều cách khai báo kiểu và nhiều cách đồng nhất kiểu đã có trong các ngôn ngữ lập trình trước đây. Nguyên nhân tại sao khai báo kiểu và đồng nhất kiểu lại liên quan đến nhau là, khi tên kiểu được khai báo, điều quan trọng là quyết định xem đây có phải là kiểu mới không, nó có khác mọi kiểu khác không hoặc tên mới nhưng về ý nghĩa nó đồng nhất với một kiểu nào đó mà đã được sử dụng ở đâu đó trong chương trình.

Một số ngôn ngữ lập trình đã dùng một số dạng đồng nhất kiểu tương đối phức tạp dẫn đến một số dạng khai báo kiểu không rõ ràng.

14. *Sắp xếp đa hình là gì? Cho ví dụ cài đặt sắp xếp đa hình trong ML

. Chúng ta có thể thấy nó làm việc như thế nào bằng việc xem xét hàm sắp xếp ML sort mà tương tự với hàm sắp xếp C++ sort khai báo trước đây:

```
fun insert(less, x, nil) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                           else y::insert(less,x,ys);
fun sort(less, nil) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

Để sort là đa hình, thao tác nhỏ hơn cần phải được truyền như đối số của hàm cho sort. Các kiểu của insert và sort được suy ra bằng thuật toán suy luận kiểu như sau.

```
val insert = fn : ('a * 'a -> bool) * 'a * 'a list -> 'a list
val sort = fn : ('a * 'a -> bool) * 'a list -> 'a list
```

Trong các kiểu trên, biến kiểu 'a có thể được khởi tạo là kiểu bất kỳ, nếu cần thiết. Kết quả, các hàm này được coi như chúng là các “templates”. Bằng việc sử dụng kết hợp template C++, các hàm ML, cú pháp kiểu, các hàm được định nghĩa trước đây có thể được viết như sau:

```

template <type 'a>
fun insert(less : 'a * 'a -> bool, x : 'a, nil : 'a list) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                           else y::insert(less,x,ys);
template <type 'a>
fun sort(less : 'a * 'a -> bool, nil : 'a list) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));

```

Các khai báo trên là các phương án khai báo kiểu tường minh của các hàm ML đa hình ẩn. Nói cách khác, thuật toán suy luận kiểu ML có thể được hiểu chương trình tiền xử lý mà chuyển biểu thức ML không có thông tin kiểu về biểu thức trong ngôn ngữ trung gian nào đó có kiểu tường minh với các templates. Từ góc độ này, sự khác biệt giữa đa hình tường minh và ẩn là chương trình tiền xử lý của ngôn ngữ lập trình (như chương trình dịch ML) dùng cú pháp ẩn đơn giản hơn và tự động chèn các thông tin kiểu tường minh, chuyển từ dạng ẩn sang tường minh, trước khi chương trình được dịch và thực hiện.

Kết thúc ví dụ này, giả sử ta khai báo hàm nhỏ hơn cho số nguyên:

```

- fun less(x,y) = x < y;
val less = fn : int * int -> bool

```

Trong áp dụng sau của hàm sắp xếp đa hình, sort template được tự động khởi tạo thành kiểu int, như vậy sort có thể được dùng để sắp xếp danh sách nguyên.

```

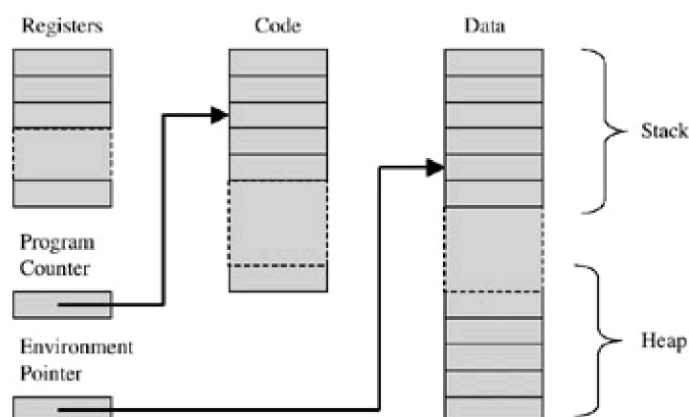
- sort (less, [1,4,5,3,2]);
val it = [1,2,3,4,5] : int list

```

Bài 4.

14. *Hãy trình bày mô hình máy tính đơn giản. Nêu cách thức hoạt động của nó, khi thực thi một chương trình.

Chúng ta sử dụng mô hình máy tính đơn giản trên Hình sau để xem xét việc quản trị bộ nhớ trong các ngôn ngữ cấu trúc khối.



Mô hình máy trên Hình 4.1 tách bộ nhớ code khỏi bộ nhớ dữ liệu. Bộ đếm chương trình lưu trữ địa chỉ của chỉ lệnh hiện tại của chương trình và được tăng một cách bình thường sau mỗi chỉ lệnh. Khi chương trình vào một block mới, một bản ghi kích hoạt activation record chứa bộ nhớ cho các biến cục bộ khai báo trong block được bổ sung thêm vào ngăn xếp thời gian chạy (run-time stack – được vẽ trên đỉnh bộ nhớ dữ liệu), và con trỏ môi trường được thiết lập đến bản ghi mới này. Khi chương trình thoát khỏi khối, bản ghi kích hoạt đó được xóa khỏi ngăn xếp và con trỏ môi trường sẽ được đặt lại vị trí cũ. Chương trình có thể lưu lại dữ liệu, mà có thể tồn tại lâu hơn sau khi thực hiện khối hiện thời, trên đống heap. Trên thực tế, bản ghi kích hoạt được cấp mới nhất là cái đầu tiên sẽ bị thu hồi, thường được gọi là stack discipline. Mặc dù hầu hết các ngôn ngữ cấu trúc khối được cài đặt bởi ngăn xếp, các hàm bậc cao có thể làm cho stack discipline bị lỗi

15. *Nêu cấu tạo của bản ghi kích hoạt. Lấy ví dụ bản ghi kích hoạt của các hàm.

Khi chương trình đang chạy đến khối in-line, không gian cần được cấp cho các biến khai báo trong khối. Chúng ta làm điều đó bằng cách cấp bộ nhớ được gọi là bản ghi kích hoạt (activation record) trên ngăn xếp thời gian chạy. Bản ghi kích hoạt đôi khi được gọi là khung ngăn xếp (stack frame).

Để thấy rõ nó làm việc như thế nào, xét ví dụ đoạn code sau. Nếu code này là một phần của chương trình lớn hơn, ngăn xếp này có thể chứa không gian cho các biến khác, trước khi khối này được thực hiện. Khi bắt đầu vào khối ngoài, bản ghi kích hoạt chứa không gian cho x và y được đẩy vào ngăn xếp. Khi các lệnh đặt giá trị cho x và y sẽ được thực hiện, làm cho các giá trị của x và y được lưu trên thanh ghi kích hoạt. Khi bắt đầu vào khối trong, bản ghi kích hoạt khác chứa không gian cho z được bổ sung vào ngăn xếp. Sau khi giá trị của z gán, bản ghi kích hoạt chứa giá trị này sẽ được lấy ra khỏi ngăn xếp. Cuối cùng, khi thoát khỏi khối ngoài, bản ghi kích hoạt chứa không gian cho x và y được lấy ra khỏi ngăn xếp.

```
{ int x=0;
  int y=x+1;
  { int z=(x+y)*(x-y);
    };
};
```

16. *Nêu khái niệm phạm vi và thời gian sống của các biến. Cho ví dụ chỉ rõ sự khác biệt của chúng.

Quan trọng là cần phân biệt phạm vi khai báo và thời gian sống của vị trí cấp.

- Phạm vi: vùng văn bản mà ở đó khai báo được nhìn thấy.
- Thời gian sống là quãng thời gian trong khi chạy chương trình, ở đó vị trí bộ nhớ được cấp như kết quả của một khai báo nào đó.

Chúng ta có thể so sánh thời gian sống và phạm vi qua ví dụ sau, các gạch dọc được sử dụng để chỉ ra điểm vào và điểm ra của từng khối.

```

{ int x = ... ;
  { int y = ... ;
    { int x = ... ;
      ....
    };
  };
};

```

Trong ví dụ này, khai báo bên trong của x giấu khai báo bên ngoài của nó. Khối bên trong được gọi là *lỗi hồng trong phạm vi* của khai báo bên ngoài của x, vì x bên ngoài không thể truy cập được ở trong khối bên trong. Ví dụ này chỉ ra rằng thời gian sống không trùng với phạm vi, vì thời gian sống của c bên ngoài bao gồm cả thời gian khi khối bên trong được thực hiện, nhưng phạm vi của x bên ngoài không bao gồm phạm vi của cái bên trong.

Câu 5. Hai cơ chế truyền tham số cho lời gọi hàm là truyền tham trị và truyền tham chiếu khác nhau như thế nào. Cho ví dụ minh họa. Lợi thế của mỗi cơ chế.

Tên tham số được sử dụng trong khai báo hàm được gọi là tham số hình thức. Khi hàm được gọi, biểu thức gọi tham số thực tế được sử dụng để tính giá trị tham số cho lời gọi đó. Sự khác biệt giữa tham số hình thức và thực tế được mô tả trong đoạn code sau.

```

proc p (int x, int y) {
  if (x > y) then ... else ... ;
  ...
  x := y*2 + 3;
  ...
}
p (z, 4*z+1);

```

Các định danh x và y là các tham số hình thức của thủ tục p. Các tham số thực tế trong lời gọi đến p là z và 4*z+1.

Cách mà các tham số thực tế được tính toán và truyền cho hàm phụ thuộc vào ngôn ngữ lập trình và cơ chế truyền tham số mà nó sử dụng. Sự khác biệt chính giữa các cơ chế truyền tham số là

- Thời điểm mà tham số thực tế được tính toán
- Vị trí được sử dụng để lưu giá trị tham số

Hầu hết các ngôn ngữ lập trình tính toán các tham số thực tế trước khi thực hiện thân hàm, nhưng cũng có một số ngoại lệ. (Một nguyên nhân là tối ưu ngôn ngữ hoặc chương trình có thể muốn trì hoãn việc tính toán tham số hình thức, vì việc tính toán này có thể lâu và có thể không được sử dụng trong một số lời gọi). Trong số các cơ chế mà tính toán các tham số thực tế trước khi thực hiện thân hàm, thông thường là

- Truyền tham chiếu: truyền L-giá trị (địa chỉ) của tham số thực tế
- Truyền tham trị: truyền R-giá trị (nội dung của địa chỉ) của tham số thực tế

Nhắc lại là chúng ta đã bàn luận về L-giá trị và R-giá trị trong mục 3.4.5 phần liên quan đến ô tham chiếu ML (vị trí gán được) và phép gán. Chúng ta sẽ bàn truyền tham chiếu và truyền tham trị làm việc như thế nào chi tiết hơn dưới đây. Một cơ chế khác là truyền kết quả giá trị (pass-by-value-result) được xét đến trong phần bài tập.

Sự khác nhau giữa truyền tham trị và truyền tham chiếu là rất quan trọng đối với người lập trình bởi một số lý do sau:

- Tác động phụ: việc gán bên trong thân hàm có thể thể có các tác động phụ khác nhau khi truyền tham trị hay truyền tham chiếu
- Bí danh: bí danh xảy ra khi hai tên tham chiếu đến cùng một đối tượng hoặc cùng một vị trí. Bí danh có thể xảy ra khi hai tham số được truyền tham chiếu hoặc một tham số được truyền tham chiếu có cùng vị trí như biến tổng thể của thủ tục.
- Tính hiệu quả: Truyền tham trị có thể không hiệu quả đối với các cấu trúc lớn, nếu giá trị của cấu trúc lớn cần sao lưu. Truyền tham chiếu có thể kém hiệu quả hơn truyền tham trị đối với các cấu trúc nhỏ mà có thể đẩy trực tiếp vào ngăn xếp, vì khi tham số truyền tham chiếu, chúng ta cần lần theo con trỏ để lấy giá trị của chúng.

Ở đây có hai cách giải thích ngữ nghĩa của lời gọi theo tham chiếu và theo tham trị. Một là vẽ ra bức tranh bộ nhớ máy tính và ngăn xếp chương trình thời gian chạy, chỉ ra ngăn xếp có chứa bản sao của tham số thực tế hay tham chiếu đến nó. Cách giải thích khác thể hiện bằng cách dịch mã sang ngôn ngữ mà phân biệt giữa L-giá trị và R-giá trị. Chúng ta sẽ dùng cách thứ hai ở đây, vì vậy phần còn lại của chương sẽ cho bạn cơ hội làm việc với một số hình ảnh về ngăn xếp thời gian chạy.

Ví dụ

Sau đây là ví dụ, viết trên ngôn ngữ tựa Algol, mà kết hợp truyền tham chiếu và truyền tham trị.

```
fun f(pass-by-ref x : int, pass-by-value y : int)
begin
  x := 2;
  y := 1;
  if x = 1 then return 1 else return 2;
end;
var z : int;
z := 0;
print f(z,z);
```

Dịch đoạn mã trên sang ML ta có:

```

fun f(x : int ref, y : int) =
  let val yy = ref y in
    x := 2;
    yy := 1;
    if (lx = 1) then 1 else 2
  end;
val z = ref 0;
f(z,lz);

```

Câu 6. Thế nào là phạm vi tĩnh và phạm vi động. Cho ví dụ nêu sự khác biệt giữa hai loại phạm vi.

- Phạm vi tĩnh: định danh tổng thể tham chiếu đến tên đó mà được khai báo trong phạm vi đóng gần nhất của văn bản chương trình.
- Phạm vi động: định danh tổng thể tham chiếu đến định danh đó được gắn kết với bản ghi kích hoạt mới nhất

Ví dụ

Sự khác biệt giữa phạm vi tĩnh và động được thể hiện qua đoạn mã sau, mà chứa hai khai báo của x.

```

int x=1;
function g(z) = x+z;
function f(y) = {
  int x = y+1;
  return g(y*x)
};
f(3);

```

Lời gọi f(3) dẫn đến lời gọi g(12) bên trong hàm f. Điều này buộc biểu thức x+z trong thân của g cần được tính toán. Sau khi gọi đến g, ngăn xếp thời gian chạy sẽ chứa các bản ghi kích hoạt cho các khai báo bên ngoài của x, khởi tạo của f, và khởi tạo của g, như chỉ ra trong thể hiện sau.

outer block	<table border="1"><tr><td>x</td><td>1</td></tr></table>	x	1		
x	1				
f(3)	<table border="1"><tr><td>y</td><td>3</td></tr><tr><td>x</td><td>4</td></tr></table>	y	3	x	4
y	3				
x	4				
g(12)	<table border="1"><tr><td>z</td><td>12</td></tr></table>	z	12		
z	12				

Tại điểm này, hai số nguyên tên x được lưu trên ngăn xếp, một của khối ngoài và một của khai báo x ở bên trong f. Trong phạm vi động, định danh x trong biểu thức x+z sẽ được biên dịch như một biến từ trong bản ghi kích hoạt tạo mới nhất, ở đó x = 4. Trong phạm vi tĩnh, định danh x+4 sẽ tham chiếu đến khai báo x từ khối chương trình gần

nhất, tìm trở lên từ chỗ $x+z$ xuất hiện trong văn bản chương trình. Trong phạm vi tĩnh, khai báo liên quan đó của x là khai báo của khối ngoài, với $x = 1$.

Câu 9. Thế nào là đệ qui đuôi. Đối với đệ qui đuôi việc tái sử dụng ngăn xếp được thực hiện như thế nào cho ví dụ minh họa

Khái niệm ngôn ngữ lập trình chính mà chúng ta cần, là khái niệm lời gọi đuôi. Giả sử hàm f gọi hàm g . Hàm f và g có thể là các hàm khác nhau hoặc f và g có thể là cùng một hàm. Lời gọi hàm f trong thân hàm g được gọi là gọi đuôi (tail call) nếu g trả về kết quả của lời gọi f mà không cần bất cứ tính toán nào nữa. Chẳng hạn, trong hàm

```
fun g(x) = if x=0 then f(x) else f(x)*2
```

Lời gọi thứ nhất của f trong g là lời gọi đuôi, vì nó trả về giá trị của g chính bằng giá trị của lời gọi f . Lời gọi thứ hai đến f trong thân của g không phải là lời gọi đuôi, vì g thực hiện tính toán gồm giá trị trả về của f trước khi g trả về kết quả.

Hàm f được gọi là đệ qui đuôi, nếu mọi lời gọi đệ qui trong thân hàm f đều là lời gọi đuôi đến f .

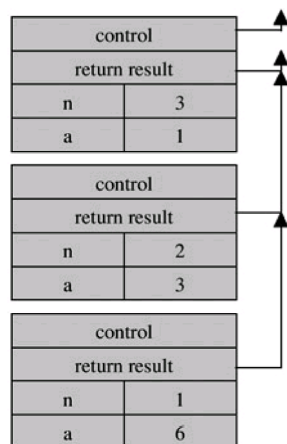
Ví dụ

Sau đây là hàm đệ qui đuôi tính toán giai thừa:

```
fun tlfact(n,a) = if n <= 1 then a else tlfact(n-1,n*a);
```

Cụ thể hơn, đối với mọi số nguyên n , $tlfact(n,a)$ trả về $n!$. Chúng ta có thể thấy $tlfact$ là hàm đệ qui đuôi, vì lời gọi duy nhất trong thân hàm $tlfact$ đó là lời gọi đuôi.

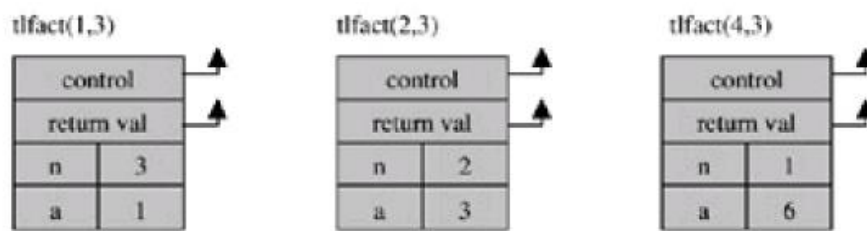
Ưu điểm của đệ qui đuôi là chúng ta có thể sử dụng cùng bản ghi kích hoạt cho mọi lời gọi đệ qui. Xét lời gọi $tlfact(3,1)$. Hình 4.7 chỉ ra các phần liên quan đến tính toán nói bên trên:



Sau lời gọi thứ ba kết thúc, nó truyền kết quả trả lại lời gọi thứ hai, mà sau đó truyền kết quả trả lại lời gọi thứ nhất. Chúng ta đơn giản hóa quá trình bằng cách cho lời gọi thứ ba trả kết quả về bản ghi kích hoạt mà tạo nên lời gọi từ đầu, $tlfact(3,1)$. Khi thực hiện phương án này, lời gọi thứ ba kết thúc, ta có thể gỡ bản ghi kích hoạt cho lời gọi

thứ hai, vì ta không cần nó nữa. Trên thực tế vì bản ghi kích hoạt cho lời gọi thứ nhất không cần nữa khi lời gọi thứ hai bắt đầu, ta có thể gỡ bỏ bản ghi kích hoạt thứ nhất ra khỏi ngăn xếp trước khi cấp bản ghi thứ hai. Sẽ tốt hơn, thay vì thu hồi bản ghi thứ nhất và sau đó cấp bản ghi thứ hai đồng nhất với bản ghi thứ nhất, ta sử dụng một bản ghi kích hoạt cho cả ba lời gọi.

Hình trên chỉ ra cùng một bản ghi kích hoạt có thể được sử dụng ba lần cho ba lời gọi liên tiếp đến `tfact` đệ quy đuôi như thế nào. Hình vẽ này chỉ ra nội dung bản ghi kích hoạt cho từng lời gọi. Khi lời gọi thứ nhất, `tfact(3,1)` bắt đầu, bản ghi kích hoạt với tham số (1,3) được tạo ra. Khi lời gọi thứ hai `tfact(2,3)` bắt đầu, giá trị tham số thay đổi sang (2,3). Khi đệ quy đuôi tái sử dụng bản ghi kích hoạt duy nhất cho mọi lời gọi hàm, sử dụng phép gán thay đổi giá trị các tham số của hàm cho mỗi lời gọi.



11. *Thế nào là truyền tham số hàm. Cho ví dụ trong ML

Trả lời:

Tên tham số được sử dụng trong khai báo hàm được gọi là tham số hình thức. Khi hàm được gọi, biểu thức gọi tham số thực tế được sử dụng để tính giá trị tham số cho lời gọi đó. Sự khác biệt giữa tham số hình thức và thực tế được mô tả trong đoạn code sau.

```

proc p (int x, int y) {
  if (x > y) then ... else ... ;
  ...
  x := y*2 + 3;
  ...
}
p (z, 4*z+1);

```

Các định danh `x` và `y` là các tham số hình thức của thủ tục `p`. Các tham số thực tế trong lời gọi đến `p` là `z` và `4*z+1`.

Cách mà các tham số thực tế được tính toán và truyền cho hàm phụ thuộc vào ngôn ngữ lập trình và cơ chế truyền tham số mà nó sử dụng. Sự khác biệt chính giữa các cơ chế truyền tham số là

- Thời điểm mà tham số thực tế được tính toán
- Vị trí được sử dụng để lưu giá trị tham số

12. *Thế nào là trả về hàm trong lời gọi hàm. Cho ví dụ trong ML. Theo bạn nghĩ nó đem lại những lợi ích gì?

Trả lời:

Một vấn đề liên quan nhưng phức tạp hơn là đôi khi được gọi là upward funnarg problem, mặc dù có thể gọi chính xác hơn là vấn đề kết quả - hàm phía trên, vì nó xảy ra khi trả về giá trị hàm từ phạm vi lồng nhau, nói chung như giá trị trả về của hàm số.

Ví dụ đơn giản của hàm mà trả về hàm là mã ML sau cho việc hợp hàm:

```
fun compose(f,g) = (fn x => g(f x));
```

Cho hai đối số hàm số f và g , phép hợp hàm trả về hàm hợp của f và g . Thân của phép hợp là mã yêu cầu tham số hàm x và sau đó tính toán $g(f(x))$. Mã này có ích chỉ khi nó gắn kết với cơ chế nào đó để tìm giá trị của f và g . Do đó, bao đóng được sử dụng để thể hiện phép hợp hàm (f,g) . Con trỏ mã của bao đóng này trỏ tới mã dịch “để nhận tham số x của hàm và sau đó tính toán $g(f(x))$ ” và con trỏ bản ghi kích hoạt của bao đóng này trỏ đến bản ghi kích hoạt của phép hợp lời gọi (f,g) vì vậy bản ghi kích hoạt này cho mã để tìm các hàm thực tế f và g cần thiết để tính hợp hàm của chúng.

13. *Cho ví dụ hàm map trên danh sách trong ML (hàm tác động đến từng phần tử của danh sách).

Trả lời:

Đây là ví dụ hàm ML mà yêu cầu một đối số hàm:

```
fun map (f, nil) = nil
  | map(f, x::xs) = f(x) :: map(f, xs)
```

Hàm map nhận hàm f và danh sách m như các đối số, áp dụng f lần lượt cho mỗi phần tử của m . Kết quả của $\text{map}(f, m)$ là danh sách các kết quả $f(x)$ cho các phần tử x trong danh sách m . Hàm này được sử dụng trong nhiều tình huống lập trình ở đó danh sách được sử dụng. Chẳng hạn, ta có danh sách các thời gian hết hạn của các sự kiện và ta muốn tăng mỗi thời gian hết hạn, chúng ta có thể áp dụng hàm tăng cho map.

Chúng ta sẽ thấy tại sao bao đóng là cần thiết bằng cách xét tương tác giữa phạm vi tĩnh và các đối số của hàm và giá trị trả về. C và C++ không hỗ trợ bao đóng vì nó kèm theo trả giá cài đặt. Tuy nhiên, việc cài đặt các đối tượng trong C++ và các ngôn ngữ khác liên quan đến cài đặt các giá trị hàm sẽ được bàn trong chương này. Lý do là bao đóng và đối tượng cả hai đều kết hợp dữ liệu với mã của hàm.

Bài 5.

5* Nêu cách sử dụng C++ function templates. Cho ví dụ dùng template như tham số kiểu trong C++.

- a. Cách sử dụng C++ function templates.

Chính là khai báo một tên kiểu T thông qua từ khóa “template”, một kiểu tổng quát để thay thế và khi sử dụng cùng một hàm hàm có thể truyền tham số dùng cho một kiểu cụ thể nào đó như “int, float, double,..”

Templates C++ được khởi tạo khi link chương trình. Cụ thể hơn, giả sử template hàm swap được lưu ở một file và được dịch, và một chương trình gọi swap được lưu trong file khác và được dịch riêng rẽ. Các tệp được gọi là đối tượng tái định vị được tạo ra khi chương trình gọi được dịch sẽ bao gồm cả thông tin chỉ ra rằng code được dịch gọi hàm swap của một kiểu nào đó. Chương trình linker được thiết kế để kết nối hai phần chương trình bằng việc gắn kết lời gọi hàm swap trong chương trình gọi đến định nghĩa của swap trong phần dịch riêng. Nó thực hiện như vậy bằng việc khởi tạo code được dịch cho swap ở dạng mà tạo ra code phù hợp với lời gọi đến swap. Nếu chương trình gọi swap với một số kiểu khác nhau, thì một số bản sao khởi tạo khác nhau sẽ được tạo ra. Mỗi bản sao khác nhau cần cho mỗi kiểu của lời gọi vì swap khai báo biến cục bộ tmp kiểu T. Không gian cho tmp cần được cấp trên bản ghi kích hoạt của swap. Do đó code dịch đối với swap cần được chỉnh sửa cho phù hợp với kích thước của biến kiểu T. Chẳng hạn, nếu T là cấu trúc hoặc đối tượng, thì kích thước là tương đối lớn. Mặt khác, nếu t là int, thì kích thước sẽ nhỏ. Trong mọi trường hợp, code dịch cần biết kích thước của dữ liệu để việc tham chiếu đến bản ghi kích hoạt được thực hiện đúng.

b. Các ví dụ.

Ví dụ: sử dụng template cho hàm hoán vị

```
template <typename T>
void sort( int count, T * A[count] ) {
    for (int i=0; i<count-1; i++)

        for (int j=i+1; j<count-1; j++)
            if (A[j] < A[i]) swap(A[i],A[j]);
}
```

Ví dụ : sử dụng template cho hàm so sánh tìm giá trị lớn nhất

```
Template <typename T>
T GetMax ( T a, T b)
{
    Return (a >b ? a:b);
}
```

6*.Nêu sự khác biệt giữa cơ chế tham số kiểu trong C++ và trong ML. Cho ví dụ minh họa.

Xét ví dụ với C++

```
template <typename T>
void swap(T& x, T& y){
    T tmp = x; x = y; y = tmp;
}
```

Hàm swap là hàm tổng quát mà làm việc tốt đối với nhiều kiểu của các đối số. Templates cho phép ta coi swap là hàm với đối số kiểu. Khi thực hiện

```
int i,j; ... swap(i,j); // Use swap with T replaced with int
float a,b; ... swap(a,b); // Use swap with T replaced with float
String s,t; ... swap(s,t); // Use swap with T replaced with String
```

Như vậy với cơ chế tham số kiểu trong C++ ta có thể định nghĩa nó là bất cứ loại đối tượng nào. Khi sử dụng ta chỉ cần khai báo kiểu tương ứng khi đó đối tượng Template hàm đã định nghĩa tự động gán tham số kiểu tương ứng.

Đối với ML

Trong đa hình ML, thuật toán suy luận kiểu đưa ra kết luận kiểu của hàm và kiểu áp dụng hàm. Khi hàm là đa hình, các hành động của thuật toán suy luận kiểu có thể được hiểu như việc chèn tự động khai báo template (“template declarations”) và khởi tạo template (“template installation”) vào chương trình

Như vậy ta có thể hình dung rằng với ML, chương trình suy luận tự động gán và khai báo.

Ví dụ hàm sắp xếp :

```
fun insert(less, x, nil) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                           else y::insert(less,x,ys);
fun sort(less, nil) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

Để sort là đa hình, thao tác nhỏ hơn cần phải được truyền như đối số của hàm cho sort. Các kiểu của insert và sort được suy ra bằng thuật toán suy luận kiểu như sau.

```
val insert = fn : ('a * 'a -> bool) * 'a * 'a list -> 'a list
val sort = fn : ('a * 'a -> bool) * 'a list -> 'a list
```

Trong các kiểu trên, biến kiểu ‘a có thể được khởi tạo là kiểu bất kỳ, nếu cần thiết. (giống việc định nghĩa template trong c++) Kết quả, các hàm này được coi như chúng là các “templates”. Bằng việc sử dụng kết hợp template C++, các hàm ML, cú pháp kiểu, các hàm được định nghĩa trước đây có thể được viết như sau:

```
template <type 'a>
fun insert(less : 'a * 'a -> bool, x : 'a, nil : 'a list) = [x]
| insert(less, x, y::ys) = if less(x,y) then x::y::ys
                           else y::insert(less,x,ys);
template <type 'a>
fun sort(less : 'a * 'a -> bool, nil : 'a list) = nil
| sort(less, x::xs) = insert(less, x, sort(less,xs));
```

Các khai báo trên là các phương án khai báo kiểu tường minh của các hàm ML đa hình ẩn. Nói cách khác, thuật toán suy luận kiểu ML có thể được hiểu chương trình tiền xử lý mà chuyển biểu thức ML không có thông tin kiểu về biểu thức trong ngôn ngữ trung gian nào đó có kiểu tường minh với các templates.

Như vậy sự khác nhau chính là việc ngôn ngữ ML có cơ chế tự động suy luận kiểu, tự động thực hiện định nghĩa các hàm như ví dụ mô tả sự định nghĩa tường minh việc suy luận kiểu bên trên thay cho vì phải định nghĩa tường minh trên C++

7*. Nêu đặc trưng và cho ví dụ dùng thư viện template chuẩn C++ (STL). Cho ví dụ minh họa. (C++ Standard Template Library được viết tắt là STL)

Thư viện chuẩn C++ dùng lại thư viện chuẩn C với một số điều chỉnh nhỏ để giúp nó hoạt động tốt hơn với ngôn ngữ C++. Một bộ phận lớn khác của thư viện C++ dựa trên Thư viện tiêu bản chuẩn (hay còn gọi là STL - viết tắt từ chữ *Standard Template Library*). Thư viện này có nhiều công cụ hữu dụng như là các thùng chứa (thí dụ như vector, danh sách liên kết và biến lặp (tổng quát hóa từ khái niệm con trỏ) để cung cấp những thùng chứa này sự truy cập giống như là truy cập mảng. Xa hơn nữa, bảng (địa) ánh xạ (mảng kết hợp) và (địa) tập, tất cả được cung cấp để có thể xuất ra các giao diện tương thích. Do đó, có thể dùng tiêu bản để viết các thuật toán tổng quát mà chúng làm việc được với bất kì thùng chứa nào hay với bất kì dãy nào được định nghĩa bởi biến lặp. Giống như C, các tính năng của thư viện này thì được truy cập bởi việc sử dụng lệnh dẫn hướng `#include` để bao gồm một tập tin tiêu đề chuẩn. C++ cung ứng 69 tiêu đề chuẩn, trong đó có 19 tiêu đề không còn hiệu lực nữa.

Một đặc trưng ẩn tượng của STL là tính hiệu quả thời gian chạy của mã được tạo ra. STL không chỉ làm cho dễ dàng viết một chương trình bằng việc cung cấp các cấu trúc hữu dụng, mà STL còn làm cho có thể viết code dựa trên thư viện mà chạy nhanh hoặc nhanh hơn so với code mà bạn có thể viết, nếu bạn bỏ ra nhiều công sức hơn và không sử dụng STL. Một lý do để STL hiệu quả là templates C++ được triển khai trong thời gian dịch/link, với các code riêng biệt được sinh (và tối ưu) cho mỗi khởi tạo. Lý do khác là việc sử dụng tải chồng, mà được hóa giải trong thời gian dịch, cũng cho phép compiler tối ưu code mà được chọn chính xác cho kiểu dữ liệu được sử dụng trong chương trình. Mặt khác, vì việc lập code là kết quả của việc triển khai template, mã dịch của chương trình sử dụng STL có thể rất lớn.

Có sáu loại thực thể trong STL:

- Containers, mỗi cái là một họ các đối tượng kiểu.
- Iterators, mà cho truy cập đến các đối tượng trong container. Về trực quan, iterator là khái quát của con trỏ hoặc cho địa chỉ đến một vị trí nào đó trong container.
- Thuật toán.
- Adapters, mà cho chuyển đổi giữa một dạng của thực thể và dạng khác. Ví dụ là iterator ngược, mà duyệt ngược hướng của một iterator.
- Đối tượng hàm (Function objects) mà là dạng closure (mã hàm và môi trường gắn kết). Chúng được sử dụng nhiều dưới dạng cho phép các đối số hàm cho template ở dạng in-line. Cụ thể hơn, đối tượng hàm được truyền như hai đối số riêng biệt, đối số template mang code và đối số run-time mang trạng thái

của hàm. Hệ thống kiểu được sử dụng để tin tưởng rằng code và trạng thái phù hợp với nhau.

- Allocators, mà đóng gói vùng bộ nhớ. Các allocators khác nhau có thể cung cấp bộ nhớ có khả năng dọn rác, bộ nhớ tham chiếu đếm, bộ nhớ bền,...

Ví dụ:

Sử dụng template vector, để làm việc với vector, chúng ta sử dụng một class đã được định nghĩa sẵn có tên cũng là "vector". Class này nằm trong namespace std nên chúng ta cần có câu lệnh sử dụng namespace này hoặc gọi trực tiếp std::vector mỗi khi cần thiết. Ngoài ra, các thao tác xử lý vector được định nghĩa trong thư viện <vector> của C++ nên chúng ta phải khai báo trước khi sử dụng

```
#include <vector>
```

```
using namespace std;
```

//Bây giờ chúng ta sẽ khai báo một vector, và chỉ rõ là vector đó dùng để lưu trữ các đối tượng loại gì:

```
vector<integer> name_of_vector;
```

//Để thêm một phần tử vào vị trí sau cùng của vector:

```
name_of_vector.push_back(name_of_element);
```

//Để bỏ đi phần tử cuối cùng của vector:

```
name_of_vector.pop_back();
```

//Để bỏ đi tất cả các phần tử của vector:

```
name_of_vector.clear();
```

//Để lấy ra phần tử đầu tiên của vector:

```
name_of_vector.front();
```

//Để lấy ra phần tử cuối cùng của vector:

```
name_of_vector.back();
```

//Để lấy ra phần tử vị trí thứ n của vector (đếm từ 0):

```
name_of_vector.at(n);
```

//Để biết số lượng phần tử của vector:

```
name_of_vector.size();
```

//Để biết vector có phần tử hay không:

```
name_of_vector.empty();
```

//Để duyệt vector bằng iterator:

//Khai báo con trỏ để duyệt

```
vector<integer>::iterator iter_name;
```

```
for (iter_name = name_of_vector.begin(); iter_name != name_of_vector.end(); iter_name++)
```

```
{
cout << *iter_name << endl;
```

Bài 6.

Câu 1. Bốn khái niệm cơ bản của lập trình hướng đối tượng:

- Tìm kiếm động (Dynamic lookup) có nghĩa là khi thông điệp được gửi tới đối tượng, mã hàm (hoặc phương thức) sẽ thực thi được xác định theo cách mà đối tượng được cài đặt, chứ không phải tính chất tĩnh nào đó của con trỏ hoặc biến được sử dụng để định danh đối tượng. Nói cách khác, đối tượng chọn cách phản hồi đến thông điệp và các đối tượng khác nhau có thể phản hồi khác nhau đến cùng một thông điệp.
- Trừu tượng (Abstraction) có nghĩa là chi tiết cài đặt được ẩn giấu bên trong đơn vị chương trình với giao diện riêng biệt. Đối với các đối tượng, giao diện thông thường bao gồm tập *public functions (hoặc public methods)* mà thao tác dữ liệu ẩn giấu đó.
- Kiểu con (Subtyping) có nghĩa là nếu một đối tượng a nào đó có mọi chức năng của đối tượng khác b, thì ta có thể sử dụng a trong mọi tình huống chờ đợi b.
- Kế thừa (Inheritance) là khả năng tái sử dụng định nghĩa một kiểu đối tượng để định nghĩa một kiểu khác của đối tượng.

Câu 7. Mẫu thiết kế (design patterns) dùng để làm gì. Cho ví dụ về singleton pattern hoặc visitor pattern

Design patterns là một giải pháp đã được kiểm nghiệm thành công khi đối diện một vấn đề lập trình phát sinh cụ thể. Khi dùng Design Patterns là nó giúp cho công việc lập trình dễ dàng tái sử dụng, dễ mở rộng và bảo trì.

Singleton Design Patterns

Mẫu thiết kế đơn là mẫu thiết kế tạo, nghĩa là đây là mẫu được dùng để tạo ra đối tượng theo một cách nào đó. Sau đây là tổng quan ngắn gọn về mẫu thiết kế đơn, mà sử dụng kiểu đối tượng tiêu đề được sử dụng chung trong các cuốn sách và các thể hiện khác của các mẫu thiết kế.

Mục tiêu

Mẫu thiết kế đơn có ích trong các tình huống, ở đó cần khởi tạo các đối tượng của một lớp. Mẫu này cho phép kiểm soát trực tiếp lớp về việc có bao nhiêu đối tượng cần được tạo. Nó tốt hơn là buộc lập trình viên có trách nhiệm chỉ để tạo một instance, cũng như việc hạn chế này được xây dựng trong chương trình.

Cài đặt.

```

namespace Singleton
{
    public class Database
    {
        private static Database singleObject;
        private string name;
        private Database(string name)
        {
            this.name = name;
        }
        public static Database createInstance(string name)
        {
            if (singleObject == null)
            {
                singleObject = new Database(name);
            }
            return singleObject;
        }
        public string getName()
        {
            return name;
        }
    }
}
class Program
{
    static void Main(string[] args)
    {
        Database database;
        database = Database.createInstance("products");
        Console.WriteLine("This is the {0} database",database.getName());
        database = Database.createInstance("employees");
        Console.WriteLine("This is the {0} database", database.getName());
    }
}

```

Bài 7.

8. *Nêu các mục đích và ràng buộc thiết kế C++. Sự tương thích với C và thành công của C++

Trả lời:

Mục đích chính của C++ là cung cấp các đặc tính hướng đối tượng trong ngôn ngữ dựa trên C mà không làm mất tính hiệu quả của C. Trong quá trình bổ sung đối tượng vào C, một số cải tiến khác đã được thực hiện. Mục tiêu thiết kế chính của C++ có thể tóm tắt như sau:

- Trừu tượng dữ liệu và các đặc tính hướng đối tượng
- Kiểm tra kiểu tĩnh tốt hơn
- Tính tương thích ngược lại với C. Nói cách khác hầu hết mã C cần được dịch như C++ hợp lệ, không đòi hỏi sự thay đổi đáng kể về mã.
- Tính hiệu quả của mã dịch, tuân thủ theo nguyên tắc ‘nếu bạn không sử dụng đặc tính nào đó, bạn không phải trả giá cho nó’

Nguyên lý này được khẳng định trong mục đích cuối là rất quan trọng và có thể đòi hỏi phải suy nghĩ để đánh giá. Nguyên lý này giả thiết chương trình C cần dịch hiệu quả dưới chương trình dịch C++ giống như dưới chương trình dịch C. Như vậy có thể vi phạm nguyên lý này khi cài đặt các số nguyên của C như các đối tượng và sử dụng phương thức tìm kiếm động để tìm hàm số nguyên trong thời gian chạy, như nó có thể giảm đáng kể việc thực thi tính toán số nguyên C. Nguyên lý này không có nghĩa là các lệnh C++ mà cũng xuất hiện trong C cần phải được cài đặt chính xác theo cùng một cách trong cả hai ngôn ngữ, nhưng bất cứ thay đổi nào được chấp nhận trong C++, không được làm chậm việc thực thi mã dịch trừ khi một số đặc tính chậm hơn của C++ cũng được sử dụng trong chương trình.

Tương thích với C

Quyết định giữ tính tương thích với C có tác động bao trùm lên thiết kế C++. Những người quen với C biết rằng C có mô hình máy chuyên biệt, bộc lộ nhiều cấu trúc của kiến trúc máy tính bên dưới. Cụ thể các thao tác C mà trả về địa chỉ của biến hoặc đặt mẫu bit bất kỳ vào vị trí tùy ý là cho chương trình C có thể dựa trên biểu diễn chính xác của dữ liệu. Do đó C++ cần tôn trọng biểu diễn dữ liệu giống như C.

Hầu hết các ngôn ngữ hướng đối tượng khác, bao gồm cả những ngôn ngữ thiết kế trước và sau C++, sử dụng thu dọn rác để giảm nhẹ cho lập trình viên khỏi nhiệm vụ định danh các đối tượng không truy cập và giải phóng các bộ nhớ gắn kết. Tuy nhiên, không có lý do cố hữu nào, tại sao các đối tượng cần được thu dọn rác. Kết nối mạnh nhất là với điểm nhấn gia tăng về tính trừu tượng và tính đúng đắn kiểu. Nó cần phải phù hợp với các mục tiêu khác của C++ để xuất dạng gom rác nào đó mà khả thi ở dạng không ảnh hưởng thời gian chạy của chương trình mà không sử dụng các đối tượng gom rác. Tuy nhiên, vì có các đặc tính của C mà làm cho gom rác rất khó, khó đặt các đối tượng C++ vào nơi gom rác. Không chỉ là bộ đếm gom rác cho triết lý C để lập trình viên kiểm soát bộ nhớ, mà còn là sự tương tự giữa con trỏ và số nguyên làm cho không thể xây dựng hiệu quả bộ gom rác mà làm việc trên chương trình C++ sử dụng số học con trỏ hoặc chuyển số nguyên thành con trỏ.

Một quyết định quan trọng khác là xử lý các đối tượng C++ như khái quát các cấu trúc struct của C. Nó buộc phải cho phép các đối tượng được khai báo và thao tác cùng một cách như structs. Nói riêng, các đối tượng được đặt trong các bản ghi kích hoạt của hàm hoặc các khối cục bộ, cũng như trên heap, và cần được thao tác trực tiếp (tức là không qua con trỏ). Đây là một điểm mà C++ đi trệch hướng so với Simula. Cụ thể, C++ cho phép dạng gán các đối tượng mà sao một đối tượng sang không gian trước đó được chiếm bởi đối tượng khác, trong khi đó hầu hết các ngôn ngữ đối tượng khác chỉ cho phép gán con trỏ cho đối tượng. Một số khía cạnh khác của các đối tượng C++ trên ngắn xếp được khám phá trong các bài tập về nhà.

Thành công của C++

C++ là ngôn ngữ được thiết kế rất cẩn thận mà được kế tiếp một cách đáng khâm phục, tuy rằng buộc thiết kế rất khó. Đo theo số người sử dụng, C++ không nghi ngờ gì nữa là ngôn ngữ thành công nhất của thập niên từ khi nó phát triển đến giữa những năm 1980 cho đến khi Java ra đời giữa những năm 1990.

Có lẽ tóm tắt công bằng cho thành công của C++ là nó được sử dụng rộng rãi, với hầu hết người sử dụng được chọn để lập trình trong một tập các ngôn ngữ mà họ hiểu và cảm thấy thuận tiện cho nhiệm vụ lập trình của họ. Nói cách khác, C++ là công cụ lập trình hữu ích mà cho phép người thiết kế tạo ra các chương trình hướng đối tượng tốt, nhưng nó không buộc phong cách lập trình tốt theo cách mà các thiết kế ngôn ngữ khác thường làm. Điều này hướng tới khẳng định rằng không có con số thống kê nào chống lại C++. Trên thực tế, nhiều thành công của nó có thể quy về cách mà C++ được thiết kế để cho các lập trình viên các lựa chọn và hạn chế lập trình viên theo một phong cách lập trình riêng nào.

4. *Nêu các quyết định tốt và các chỗ còn vấn đề khi thiết kế C++

Trả lời:

C++ là kết quả của nỗ lực lớn gồm cả phê phán và đề xuất từ nhiều lập trình viên có kinh nghiệm. Trong nhiều khía cạnh, ngôn ngữ được thiết kế tốt nhất có thể, với mục tiêu bổ sung đối tượng và kiểm tra kiểu thời gian dịch tốt hơn cho C, không làm mất đi tính hiệu quả hoặc tương thích ngược. Một số phần thiết kế thành công riêng là

- Encapsulation: chú ý cẩn thận đến tính nhìn thấy và ẩn giấu, bao gồm các mức độ nhìn thấy public, protected, private và các hàm và lớp friend.
- Sự tách biệt giữa subtyping và inheritance: các lớp có thể có các lớp cơ sở public hoặc private, cho lập trình viên kiểm soát tường minh nào đó về phân cấp kiểu con kết quả.
- Templates
- Kiểm tra kiểu tĩnh tốt hơn C

Cũng có một số các quyết định thành công nhỏ hơn trong C++. Một ví dụ là cách mà phép toán hóa giải phạm vi (được viết như ::) được sử dụng trong kết nối với kế thừa đơn và lập để hóa giải sự nhập nhằng mà là vấn đề trong các ngôn ngữ khác.

Một số chỗ có vấn đề

Ở đây có một số khía cạnh của C++ mà lập trình viên đôi khi thấy khó khăn. Một số chỗ chính có vấn đề là

- Ép kiểu và chuyển kiểu, mà có thể phức tạp và khó lường trước trong một số tình huống
- Các đối tượng gắn kết với ngăn xếp và các khía cạnh khác của việc quản lý bộ nhớ đối tượng
- Tải chồng, cơ chế chọn mã phức tạp trong C++ mà có thể tương tác khó đoán với tìm kiếm động (tìm kiếm hàm ảo)
- Kế thừa bội, mà là phức tạp hơn trong C++ so với trong ngôn ngữ khác vì cách các đối tượng và các bảng hàm ảo được cấu hình và truy cập.

Một số lập trình viên có thể nói rằng không có gom rác, hoặc không có giao diện chuẩn để viết chương trình song song là vấn đề trong C++.

Bài 8.

10. *Nêu các mục tiêu chính khi thiết kế ngôn ngữ lập trình Java. Tại sao nói Java là ngôn ngữ hướng đối tượng thuần túy.

Trả lời:

Ngôn ngữ lập trình Java và môi trường thực thi được thiết kế với các mục tiêu sau :

- Portability – Tính chuyển mang: nó cần dễ dàng truyền chương trình qua mạng và nhận được chúng chạy đúng đắn trên môi trường nhận, không phụ thuộc vào phần cứng, hệ điều hành và trình duyệt Web được sử dụng.
- Reliability – Tính tin cậy: vì chương trình sẽ được chạy từ xa bởi người sử dụng, người đó không viết code, các thông báo lỗi và sự cố chương trình cần đảm bảo loại bỏ tối đa càng tốt.
- Safety – Tính an toàn: môi trường tính toán nhận được chương trình cần được bảo vệ khỏi các lỗi của người lập trình và các chương trình có hại.
- Dynamic link – Liên kết động: chương trình được phân tán thành các phần, với các phần riêng biệt được tải vào môi trường thời gian chạy của Java tùy theo yêu cầu.
- Multithreaded Execution: để các chương trình song song chạy trên các phần cứng và hệ điều hành khác nhau, ngôn ngữ lập trình cần gồm phần hỗ trợ tường minh và giao diện chuẩn cho lập trình song song.
- Simplicity and Familiatity – Tính đơn giản và quen thuộc: ngôn ngữ có cảm giác đối với người thiết kế website của bạn cũng bình thường như với người lập trình C hoặc với người khá quen C/C++.
- Efficiency – Tính hiệu quả: điều này quan trọng nhưng là thứ yếu so với các yêu cầu khác.

Java là một ngôn ngữ lập trình hướng đối tượng thuần túy vì mọi thứ trong Java đều là đối tượng. Các ngôn ngữ lập trình hướng đối tượng có các modul có thể thay đổi và được xác định trước mà người lập trình có thể gọi ra để thực hiện những nhiệm vụ cụ

thể. Trong Java các modul này gọi là các lớp (class) và chúng được lưu trữ trong thư viện lớp tạo nên cơ sở của bộ công cụ phát triển Java (Java Development Kit). Trong Java tất cả các hàm và biến đều phải là thành phần của một lớp.

11. *Các quyết định tốt làm cho Java được sử dụng rộng rãi hiện nay là gì?

Trả lời:

Chúng ta có thể thấy tính quan trọng tương đối của tính hiệu quả trong quá trình thiết kế Java. Nó không có nghĩa là tính hiệu quả không được ưu tiên mà chỉ là tương đối so với các mục tiêu khác, tính hiệu quả không phải là mục tiêu quan trọng.

Hỗ trợ song song. Java có mô hình song song dựa trên các luồng, mà là các tiến trình song song độc lập. Đây là phần quan trọng của ngôn ngữ, cả hai vì thiết kế có ý nghĩa lớn và vì sự quan trọng có các yếu tố song song chuẩn như một thành phần của ngôn ngữ Java. Rõ ràng, nếu chương trình Java dựa trên hệ điều hành dành cho các cơ chế song song, thì chương trình sẽ không thể được chuyển mang trên các nền tảng hệ điều hành khác nhau.

Tính đơn giản. Mặc dù Java được phát triển theo từng năm, các đặc trưng như tính phân xạ và lớp trong không cho cảm giác đơn giản, ngôn ngữ vẫn đủ nhỏ và đơn giản trong thiết kế so với hầu hết các ngôn ngữ lập trình đa dụng chất lượng cao khác. Một cách đơn giản để nhìn thấy sự đơn giản tương đối của Java là liệt kê các đặc trưng của C++ mà không xuất hiện trong Java. Nó bao gồm các đặc trưng sau:

- Structures and unions: các cấu trúc mà được gom lại bởi các đối tượng mà một số sử dụng tập hợp có thể được thay thế bởi các lớp được chia sẻ bởi lớp cha chung.
- Các hàm được thay thế bởi các phương thức tĩnh
- Kế thừa bội là phức tạp và hầu hết các trường hợp có thể loại bỏ nếu khái niệm giao diện đơn giản hơn được sử dụng
- Lệnh Goto là không cần thiết
- Tải đè toán tử là phức tạp và cho rằng là không cần thiết; các hàm Java có thể tải đè.
- Chuyển kiểu tự động là phức tạp và cho rằng là không cần thiết
- Con trỏ là mặc định cho đối tượng và không cần thiết cho các kiểu khác. Kéo theo kiểu con trỏ riêng biệt là không cần thiết.

Một số đặc tính này xuất hiện trong C++ như là các tính chất cơ bản, vì mục tiêu thiết kế của C++ là tương thích ngược với C. Các đặc tính khác được bỏ qua bởi Java sau khi bàn luận, vì đã được quyết định là sự phức tạp nếu đảm bảo nó sẽ đáng kể hơn nhiều so với tính năng mà nó đem lại. Việc bỏ qua quan trọng nhất là kế thừa bội, chuyển kiểu tự động, tải đè toán tử và phép toán con trỏ trong một số dạng có trong C và C++.

Câu 3. Lớp trong Java có gì khác biệt so với C++. Phương thức và trường tĩnh trong lớp Java có ý nghĩa gì. Cho ví dụ minh họa.

1. Sự khác nhau giữa lớp trong Java và trong C++

- Khai báo lớp:

+ Trong C++:

Class:

class name« : **public** parentclasses» { members };

Protocol:

Không có

Namespace:

namespace name {members }

+ Trong Java:

Class:

class name« **extends** parentclass»« **implements** interfaces» { members } Protocol:**interface** name« **extends**parentinterfaces» { **members** }

Namespace:

package name;

members

- Hàm tạo và hàm hủy:

+ Trong C++:

Construction:

class(«parameters») «: initializers» { instructions }

Destruction:

~class() { instructions }

Finalize:

Không có

+ Trong Java:

Construction:

class(«parameters») { instructions }

Destruction:

Không có

Finalize:

void finalize() {instructions }

- Khởi tạo đối tượng:

+ Trong C++:

Construction:

class variable«(parameters)»; hoặc

class *variable = **new** class«(parameters)»;

Destruction:

delete pointer;

+ Trong Java:

Construction:

class variable = **new** class(parameters);

Destruction:

delete variable;

- Truy xuất:
 - + Trong C++:
 - Phương thức:
 - `x.method(parameters)` or
 - `ptr->method(parameters)`
 - Thuộc tính:
 - `x.field` or
 - `ptr->field`
 - Lớp con:
 - `cls::member`
 - Không gian tên:
 - `ns::member`
 - + Trong Java:
 - Phương thức:
 - `x.method(parameters)`
 - Thuộc tính:
 - `x.field`
 - Lớp con:
 - `cls.member`
 - Không gian tên:
 - `ns.member`
- Phương thức trong java:
 - Phương thức thực hiện một tác vụ thật sự của đối tượng
- Trường tĩnh:
 - Biến static chỉ là một biến không phụ thuộc vào một đối tượng nào cả, tức là mình có thể truy vấn trực tiếp mà không cần phải khởi tạo đối tượng

```
public class StaticSample {
    public static int count = 0;
    public int a = 0;
    public StaticSample() {
        count++;
        a++;
    }

    public static void main(String[] args) {
        StaticSample aClass = new StaticSample();
        StaticSample bClass = new StaticSample();
        //Đếm số đối tượng đã được tạo (biến static ko bị gán lại)
        System.out.println(StaticSample.count);
        //Các Biến non static bị gán lại ứng với mỗi đối tượng
        System.out.println(aClass.getA());
        System.out.println(bClass.getA());
    }
}
```

```

        public int getA() {
            return a;
        }
    }

```

Kết quả:

2

1

1

Câu 5. Kế thừa trong Java có gì khác biệt so với C++. Phương thức lớp con ghi đè phương thức lớp cha như thế nào. Cho ví dụ.

Kế thừa

C++ hỗ trợ đa kế thừa còn Java thì không

Java không cho phép đa kế thừa trực tiếp, nhưng cho phép cài đặt nhiều giao tiếp (Interface) để có thể thừa hưởng thêm các thuộc tính và phương thức của các giao tiếp đó

Ví dụ:

```

class SupperClass{
    int a;
    SupperClass()
    {
    }
    SupperClass( int b )
    {
        a = b;
    }
    public void message(){
        System.out.println("In the sub class");
    }
    class SubClass Extends SupperClass{
        int a;
        SubClass( int a ){
            this.a = a;
        }
        public static void main(){
            SupperClass spObj = new Subclass( 22 );
            spObj.message();
        }
    }
}

```

Câu 6. Giao diện trong Java dùng để làm gì. Một lớp có thể cài đặt nhiều giao diện không, cho ví dụ minh họa.

Ngôn ngữ Java bao gồm khái niệm về một giao diện (interface), nó chỉ đơn giản là một tập hợp có tên của các hành vi có sẵn công khai và/hoặc các phân tử dữ liệu không thay đổi mà trình triển khai thực hiện giao diện đó phải cung cấp mã lệnh. Nó không chỉ rõ các chi tiết hành vi. Về bản chất (và với trình biên dịch Java), một giao diện định nghĩa một kiểu dữ liệu mới và nó là một trong những đặc tính mạnh của ngôn ngữ này.

Các lớp khác triển khai thực hiện giao diện, có nghĩa là chúng có thể sử dụng bất kỳ các hằng số trong giao diện đó bằng tên và chúng phải chỉ rõ hành vi cho các định nghĩa phương thức trong giao diện.

Bất kỳ lớp nào trong hệ thống thứ bậc cũng có thể thực hiện một giao diện cụ thể nào đó. Điều đó có nghĩa là các lớp không liên quan nhau có thể thực hiện cùng một giao diện

- Định nghĩa:

```
public interface interfaceName {
    final constantType
    constantName = constantValue;
    ...
    returnType
    methodName( arguments );
    ...
}
```

- Bạn có thể triển khai thực hiện nhiều hơn một giao diện bằng cách liệt kê các tên giao diện, phân cách bằng dấu phẩy.

```
public class className extends superclassName implements
    interfaceName, ... {
    class body
}
```

Ví dụ:

```
interface CanFight {
    void fight();
}

interface CanSwim {
    void swim();
}

interface CanFly {
    void fly();
}

class ActionCharacter {
```

```
    public void fight() {
    }
}
class Hero extends ActionCharacter implements CanFight, CanSwim, CanFly
{
    public void swim() {
    }
    public void fly() {
    }
}
public class Adventure {
    public static void t(CanFight x) {
        x.fight();
    }
    public static void u(CanSwim x) {
        x.swim();
    }
    public static void v(CanFly x) {
        x.fly();
    }
    public static void w(ActionCharacter x) {
        x.fight();
    }
    public static void main(String[] args) {
        Hero h = new Hero();
        t(h); // Treat it as a CanFight
        u(h); // Treat it as a CanSwim
        v(h); // Treat it as a CanFly
        w(h); // Treat it as an ActionCharacter
    }
}
```

7*. Lớp trừu tượng dùng để làm gì, hãy phân biệt nó với giao diện. Cho ví dụ minh họa
Lớp trừu tượng là lớp khai báo ra để cho các lớp con kế thừa các phương thức trong lớp cha

Một lớp trừu tượng thì không có thể hiện nghĩa là ta không thể khởi tạo nó bằng toán tử new, và một phương thức trong nó là abstract thì chỉ được đưa ra định nghĩa (khai báo) mà không được thực thi và nó sẽ được overridden lại trong các lớp con kế thừa. Và trong lớp mà tồn tại phương thức abstract thì lớp đó cũng được định nghĩa abstract.

Đối với giao diện thì khác hoàn toàn với lớp trừu tượng, nó định nghĩa một cách cứng nhắc các phương thức và thuộc tính trong chúng nghĩa là không cho phép ta thực thi bất kỳ một đoạn mã nào. Và tất cả các thành viên trong nó đều được định nghĩa công khai (public). Một cách tổng quan về giao diện: Giao diện là ràng buộc, giao ước đảm bảo cho các lớp hay các cấu trúc sẽ thực hiện một điều gì đó. Khi một lớp thực thi một giao diện, thì lớp này báo cho các thành phần client biết rằng lớp này có hỗ trợ các phương thức, thuộc tính, sự kiện và các chỉ mục khai báo trong giao diện.

Một số đặc điểm để phân biệt khi sử dụng abstract hay interface

- Một class chỉ có thể kế thừa từ một abstract class, nhưng có thể kế thừa nhiều interface.
- Trong Interface chỉ có thể khai báo các fields, methods, mà không được hiện thực nó. Còn đối với abstract thì dùng các biến, hiện thực các methods.
- Các fields, methods trong interface đều là public và bắt buộc các class kế thừa phải cài đặt nó (abstract). Trong abstract class thì có các fields, methods có thể là private, internal, public, protected và có thể là abstract hoặc non-abstract.
- Interface dùng để gom các hành động cần được hiện thực, các khả năng của một đối tượng, còn abstract class cho các lớp thừa kế cùng 1 loại, tính chất hay trạng thái.
- Abstract class có tốc độ thực thi nhanh hơn interface
- Thêm 1 tính năng mới vào interface sẽ phá vỡ toàn bộ các lớp hiện thực, còn abstract thì không.

Ví dụ về interface, các thành viên của interface phải được thực thi trong các lớp mà kế thừa từ nó.

```
public interface IPlayer
{
    string Name
    {
        get;
        set;
    }
    Player Actor
    {
        get;
        set;
    }
}

Bitmap Image
{
    get;
    set;
}
}
```

Ví dụ dưới đây khai báo một lớp abstract có các thành viên trong nó là abstract và non-abstract, và được thực thi trong lớp con là Faculty

```
abstract class Employee
{
    protected string m_str_department;
    protected double m_db_salary;
    protected int m_i_dateHired;

    public string Department
    {
        get { return m_str_department; }
        set { m_str_department = value; }
    }

    public double Salary
    {
        get { return m_db_salary; }
        set { m_db_salary = value; }
    }

    public int DateHired
    {
        get { return m_i_dateHired; }
        set { m_i_dateHired = value; }
    }

    public override string ToString()
    {
        return "Employee: " + m_str_name
            + "\nEmail: " + m_str_email;
    }

    public abstract double CalculateBonus();
    public abstract int CalculateVacation();
}

class Faculty : Employee
{
    string m_str_rank;
    double m_db_hours;

    public override double CalculateBonus()
    {
        return 1000 + 0.05 * m_db_salary;
    }

    public override int CalculateVacation()
    {
        if (m_i_dateHired > 3)
        {
            if (m_str_rank == "Senior Lecture")
                return 6;
            return 5;
        }
        if (m_str_rank == "Senior Lecture")
            return 5;
    }
}
```

```
        return 4;
    }
}
```

Câu 9: Phiên bản mới của Java có hỗ trợ cài đặt tham số kiểu trực tiếp không cần qua Object không, điều đó được thực hiện như thế nào

- Phiên bản mới của Java có hỗ trợ cài đặt tham số kiểu trực tiếp không cần qua Object đối với phiên bản mới nhất. Java SE 7 (còn gọi là Dolphin), được bắt đầu từ tháng 8 năm 2006 và công bố ngày 28 tháng 7 năm 2011.

Trong Java có 2 loại dữ liệu, dữ liệu cơ bản và dữ liệu tham chiếu. Kiểu dữ liệu cơ bản là các kiểu dữ liệu như int, float, double... còn kiểu dữ liệu tham chiếu như mảng, lớp đối tượng(class), giao diện(interface). Và trong Java có quy định như sau. Kiểu cơ bản thì truyền theo kiểu tham trị, kiểu tham chiếu thì truyền theo kiểu tham chiếu.

- Chương trình minh họa:

```
public static void example(int m[], int n) {
    for(int i = 0; i < 10; ++i) {
        m[i] = 1;
    }
    n = n + 1;
}
```

đây là kiểu truyền và cài đặt tham số không cần đối tượng (Object)